Solidum FEM

Manual de Referencia

Especificaciones físicas, formulaciones numéricas y contratos
de los componentes del programa

Generado automáticamente desde `docs/specs/`
y `docs/user/*/specs/`

Autor: Jaime Retama Velasco

Facultad de Estudios Superiores Aragón
Universidad Nacional Autónoma de México

9 de noviembre de 2026

Índice general

Sobre este Manual xiii

1. Elementos 1D — Armaduras 1
- 1.1. Truss2D . . . 1
  - 1.1.1. Especificación física . . . 1
  - 1.1.2. Formulación numérica (FEM) . . . 2
  - 1.1.3. Contrato de implementación . . . 2
  - 1.1.4. Implementación . . . 3
  - 1.1.5. Diálogo . . . 4
- 1.2. Truss2DCorot . . . 5
  - 1.2.1. Especificación física . . . 5
  - 1.2.2. Formulación numérica (FEM) . . . 6
  - 1.2.3. Contrato de implementación . . . 7
  - 1.2.4. Implementación . . . 8
  - 1.2.5. Diálogo . . . 8
- 1.3. Truss3D . . . 10
  - 1.3.1. Especificación física . . . 10
  - 1.3.2. Formulación numérica (FEM) . . . 11
  - 1.3.3. Contrato de implementación . . . 11
  - 1.3.4. Implementación . . . 13
  - 1.3.5. Diálogo . . . 13
- 1.4. Truss3DCorot . . . 14
  - 1.4.1. Especificación física . . . 14
  - 1.4.2. Formulación numérica (FEM) . . . 14
  - 1.4.3. Contrato de implementación . . . 15
  - 1.4.4. Implementación . . . 16
  - 1.4.5. Diálogo . . . 17

2. Elementos 1D — Cables 19
- 2.1. Cable2DCorot . . . 19
  - 2.1.1. Especificación física . . . 19
  - 2.1.2. Formulación numérica (FEM) . . . 20
  - 2.1.3. Contrato de implementación . . . 21
  - 2.1.4. Implementación . . . 23
  - 2.1.5. Diálogo . . . 24

2.2. Cable3DCorot . . . 26
2.2.1. Especificación física . . . 26
2.2.2. Formulación numérica (FEM) . . . 27
2.2.3. Contrato de implementación . . . 28
2.2.4. Implementación . . . 30
2.2.5. Diálogo . . . 30

3. Elementos 1D — Marcos / Vigas **32**
3.1. Frame2DEuler . . . 32
3.1.1. Especificación física . . . 32
3.1.2. Formulación numérica (FEM) . . . 33
3.1.3. Contrato de implementación . . . 35
3.1.4. Implementación . . . 38
3.1.5. Diálogo . . . 39
3.2. Frame2DEulerCorot . . . 41
3.2.1. Especificación física . . . 41
3.2.2. Formulación numérica (FEM) . . . 42
3.2.3. Contrato de implementación . . . 44
3.2.4. Implementación . . . 46
3.2.5. Diálogo . . . 47
3.3. Frame2DTimoshenko . . . 49
3.3.1. Especificación física . . . 49
3.3.2. Formulación numérica (FEM) . . . 50
3.3.3. Contrato de implementación . . . 52
3.3.4. Implementación . . . 53
3.3.5. Diálogo . . . 54
3.4. Frame3D . . . 56
3.4.1. Especificación física . . . 56
3.4.2. Formulación numérica (FEM) . . . 57
3.4.3. Contrato de implementación . . . 60
3.4.4. Implementación . . . 62
3.4.5. Diálogo . . . 64

4. Secciones de marco **66**
4.1. FiberSection . . . 66
4.1.1. Especificación física . . . 66
4.1.2. Contrato . . . 69
4.1.3. Implementación . . . 71
4.1.4. Resultados de validación . . . 74
4.1.5. Diálogo . . . 75

5. Elementos 2D — Sólidos **78**
5.1. Tri3 . . . 78
5.1.1. Especificación física . . . 78
5.1.2. Formulación numérica (FEM) . . . 79
5.1.3. Limitación crítica — *shear locking* . . . 80

5.1.4. Contrato de implementación . . . 80
5.1.5. Implementación . . . 82
5.1.6. Diálogo . . . 82
5.2. Quad4 . . . 84
5.2.1. Especificación física . . . 84
5.2.2. Formulación numérica (FEM) . . . 85
5.2.3. Contrato de implementación . . . 87
5.2.4. Implementación . . . 88
5.2.5. Diálogo . . . 89
5.3. Tri6 . . . 91
5.3.1. Especificación física . . . 91
5.3.2. Contrato de implementación . . . 91
5.3.3. Implementación . . . 92
5.3.4. Diálogo . . . 92
5.4. Quad8 . . . 94
5.4.1. Especificación física . . . 94
5.4.2. Formulación numérica . . . 94
5.4.3. Contrato de implementación . . . 95
5.4.4. Implementación . . . 97
5.4.5. Diálogo . . . 97
5.5. Quad9 . . . 99
5.5.1. Especificación física . . . 99
5.5.2. Contrato de implementación . . . 99
5.5.3. Implementación . . . 100
5.5.4. Diálogo . . . 100

6. Elementos 3D — Sólidos 102
6.1. Tet4 . . . 102
6.1.1. Especificación física . . . 102
6.1.2. Formulación numérica (FEM) . . . 103
6.1.3. Limitación crítica — *shear locking* . . . 105
6.1.4. Contrato de implementación . . . 105
6.1.5. Implementación . . . 107
6.1.6. Diálogo . . . 107
6.2. Hex8 . . . 109
6.2.1. Especificación física . . . 109
6.2.2. Formulación numérica (FEM) . . . 110
6.2.3. Limitaciones declaradas . . . 112
6.2.4. Contrato de implementación . . . 112
6.2.5. Implementación . . . 114
6.2.6. Diálogo . . . 115
6.3. Tet10 . . . 117
6.3.1. Especificación física . . . 117
6.3.2. Formulación numérica (FEM) . . . 117
6.3.3. Limitaciones declaradas . . . 119

6.3.4. Contrato de implementación . . . 119
6.3.5. Implementación . . . 121
6.3.6. Diálogo . . . 122
6.4. Hex20 . . . 124
6.4.1. Especificación física . . . 124
6.4.2. Formulación numérica (FEM) . . . 124
6.4.3. Limitaciones declaradas . . . 128
6.4.4. Contrato de implementación . . . 129
6.4.5. Implementación . . . 131
6.4.6. Diálogo . . . 132
6.5. Hex27 . . . 134
6.5.1. Especificación física . . . 134
6.5.2. Formulación numérica (FEM) . . . 134
6.5.3. Limitaciones declaradas . . . 136
6.5.4. Contrato de implementación . . . 137
6.5.5. Implementación . . . 139
6.5.6. Diálogo . . . 140

7. Elementos Térmicos — 2D **141**
7.1. Quad4Thermal . . . 141
7.1.1. Especificación física . . . 141
7.1.2. Formulación numérica . . . 144
7.1.3. Contrato de implementación . . . 145
7.1.4. Implementación . . . 147
7.1.5. Diálogo . . . 148

8. Elementos Térmicos — 3D **151**
8.1. Hex8Thermal . . . 151
8.1.1. Qué comparte con `Quad4Thermal` . . . 151
8.1.2. Qué cambia respecto a `Quad4Thermal` . . . 151
8.1.3. Contrato de implementación . . . 153
8.1.4. Implementación . . . 156
8.1.5. Diálogo . . . 157

9. Modelos Constitutivos — 1D **158**
9.1. CableMaterial1D . . . 158
9.1.1. Especificación física . . . 158
9.1.2. Contrato de implementación . . . 159
9.1.3. Implementación . . . 160
9.1.4. Diálogo . . . 161
9.2. Elastic1D . . . 162
9.2.1. Especificación física . . . 162
9.2.2. Formulación numérica . . . 163
9.2.3. Contrato de implementación . . . 163
9.2.4. Implementación . . . 165
9.2.5. Diálogo . . . 165

9.3. Elastoplastic1D . . . 166
9.3.1. Especificación física . . . 166
9.3.2. Formulación numérica . . . 167
9.3.3. Contrato de implementación . . . 168
9.3.4. Implementación . . . 170
9.3.5. Diálogo . . . 170
9.4. IsotropicDamage1D . . . 171
9.4.1. Especificación física . . . 171
9.4.2. Formulación numérica . . . 171
9.4.3. Contrato de implementación . . . 172
9.4.4. Implementación . . . 174
9.4.5. Diálogo . . . 175

10. Modelos Constitutivos — 2D **176**
10.1. DruckerPrager2D . . . 176
10.1.1. Especificación física . . . 176
10.1.2. Formulación numérica . . . 178
10.1.3. Contrato de implementación . . . 182
10.1.4. Implementación . . . 185
10.1.5. Diálogo . . . 186
10.2. Elastic2D . . . 188
10.2.1. Especificación física . . . 188
10.2.2. Formulación numérica . . . 189
10.2.3. Contrato de implementación . . . 190
10.2.4. Implementación . . . 191
10.2.5. Diálogo . . . 192
10.3. IsotropicDamage2D . . . 193
10.3.1. Especificación física . . . 193
10.3.2. Formulación numérica . . . 195
10.3.3. Contrato de implementación . . . 197
10.3.4. Implementación . . . 201
10.3.5. Diálogo . . . 202
10.4. Orthotropic2D . . . 204
10.4.1. Especificación física . . . 204
10.4.2. Formulación numérica . . . 206
10.4.3. Contrato de implementación . . . 206
10.4.4. Diálogo . . . 209
10.4.5. Implementación . . . 209
10.5. VonMises2D . . . 211
10.5.1. Especificación física . . . 211
10.5.2. Formulación numérica . . . 213
10.5.3. Contrato de implementación . . . 215
10.5.4. Implementación . . . 218
10.5.5. Diálogo . . . 219

11. Modelos Constitutivos — 3D **222**
11.1. DruckerPrager3D . . . 222
11.1.1. Especificación física . . . 222
11.1.2. Formulación numérica . . . 226
11.1.3. Contrato de implementación . . . 228
11.1.4. Implementación . . . 232
11.1.5. Diálogo . . . 233
11.2. Elastic3D . . . 236
11.2.1. Especificación física . . . 236
11.2.2. Formulación numérica . . . 237
11.2.3. Contrato de implementación . . . 238
11.2.4. Implementación . . . 239
11.2.5. Diálogo . . . 240
11.3. IsotropicDamage3D . . . 241
11.3.1. Especificación física . . . 241
11.3.2. Formulación numérica . . . 243
11.3.3. Contrato de implementación . . . 244
11.3.4. Implementación . . . 249
11.3.5. Diálogo . . . 250
11.4. VonMises3D . . . 253
11.4.1. Especificación física . . . 253
11.4.2. Formulación numérica . . . 255
11.4.3. Contrato de implementación . . . 258
11.4.4. Implementación . . . 261
11.4.5. Diálogo . . . 262

12. Modelos Constitutivos — Térmicos **264**
12.1. ThermalConduction . . . 264
12.1.1. Especificación física . . . 264
12.1.2. Formulación numérica . . . 266
12.1.3. Contrato de implementación . . . 267
12.1.4. Implementación . . . 269
12.1.5. Diálogo . . . 270

13. Esquemas de Solución — Estáticos **272**
13.1. ArcLengthSolver . . . 272
13.1.1. Especificación física . . . 272
13.1.2. Formulación numérica . . . 273
13.1.3. Contrato de implementación . . . 276
13.1.4. Implementación . . . 279
13.1.5. Diálogo . . . 280
13.2. IndirectDisplacementSolver . . . 282
13.2.1. Especificación física . . . 282
13.2.2. Contrato de implementación . . . 284
13.2.3. Implementación . . . 286
13.2.4. Diálogo . . . 286

13.3. LinearSolver . . . 288
13.3.1. Especificación física . . . 288
13.3.2. Formulación numérica . . . 288
13.3.3. Contrato de implementación . . . 291
13.3.4. Implementación . . . 292
13.3.5. Diálogo . . . 293
13.4. NonlinearSolver . . . 295
13.4.1. Especificación física . . . 295
13.4.2. Formulación numérica . . . 295
13.4.3. Contrato de implementación . . . 298
13.4.4. Implementación . . . 300
13.4.5. Diálogo . . . 301

14. Esquemas de Solución — Modal y dinámicos **303**
14.1. CentralDifferenceSolver . . . 303
14.1.1. Especificación física . . . 303
14.1.2. Formulación numérica . . . 304
14.1.3. Contrato de implementación . . . 306
14.1.4. Implementación . . . 308
14.1.5. Diálogo . . . 309
14.2. HHTSolver . . . 310
14.2.1. Qué cambia respecto a `NewmarkSolver` . . . 310
14.2.2. Contrato de implementación . . . 311
14.2.3. Implementación . . . 313
14.2.4. Diálogo . . . 314
14.3. HarmonicSolver . . . 315
14.3.1. Especificación física . . . 315
14.3.2. Formulación numérica . . . 316
14.3.3. Contrato de implementación . . . 317
14.3.4. Implementación . . . 319
14.3.5. Diálogo . . . 319
14.4. ModalSolver . . . 320
14.4.1. Especificación física . . . 320
14.4.2. Formulación numérica . . . 321
14.4.3. Contrato de implementación . . . 322
14.4.4. Implementación . . . 325
14.4.5. Diálogo . . . 327
14.5. NewmarkSolver . . . 328
14.5.1. Especificación física . . . 328
14.5.2. Formulación numérica . . . 329
14.5.3. Contrato de implementación . . . 330
14.5.4. Implementación . . . 332
14.5.5. Diálogo . . . 333
14.6. NewtonHHTSolver . . . 335
14.6.1. Qué cambia respecto a `NewtonNewmarkSolver` . . . 335

14.6.2. Contrato de implementación . . . 336
14.6.3. Implementación . . . 338
14.6.4. Diálogo . . . 338
14.7. NewtonNewmarkSolver . . . 340
14.7.1. Qué cambia respecto a `NewmarkSolver` . . . 340
14.7.2. Contrato de implementación . . . 342
14.7.3. Implementación . . . 343
14.7.4. Diálogo . . . 344
14.8. ResponseSpectrumSolver . . . 346
14.8.1. Especificación física . . . 346
14.8.2. Formulación numérica . . . 347
14.8.3. Contrato de implementación . . . 348
14.8.4. Implementación . . . 350
14.8.5. Diálogo . . . 351
14.9. ThetaMethodSolver . . . 352
14.9.1. Especificación física . . . 352
14.9.2. Formulación numérica . . . 354
14.9.3. Contrato de implementación . . . 355
14.9.4. Implementación . . . 359
14.9.5. Diálogo . . . 361

15. Elementos — catálogo **363**
15.1. Truss2D — barra axial 2D . . . 363
15.1.1. Cinemática . . . 363
15.1.2. Formulación . . . 363
15.1.3. Convenciones . . . 364
15.1.4. Parámetros . . . 364
15.1.5. Cargas distribuidas . . . 364
15.1.6. Régimen de validez . . . 364
15.1.7. Validación . . . 364
15.2. Truss2DCorot — armadura 2D corotacional . . . 364
15.2.1. Cinemática . . . 364
15.2.2. Formulación . . . 365
15.2.3. Cargas distribuidas . . . 365
15.2.4. Régimen de validez . . . 365
15.2.5. Validación . . . 365
15.3. Truss3D — barra axial 3D . . . 365
15.3.1. Cinemática . . . 365
15.3.2. Formulación . . . 366
15.3.3. Convenciones . . . 366
15.3.4. Parámetros . . . 366
15.3.5. Cargas distribuidas . . . 366
15.3.6. Régimen de validez . . . 366
15.3.7. Validación . . . 366
15.4. Truss3DCorot — barra axial 3D corotacional . . . 367

15.4.1. Cinemática . . . 367
15.4.2. Formulación . . . 367
15.4.3. Cargas distribuidas . . . 367
15.4.4. Régimen de validez . . . 367
15.4.5. Validación . . . 367
15.5. Cable2DCorot — cable 2D corotacional . . . 368
15.5.1. Cinemática . . . 368
15.5.2. Pretensión . . . 368
15.5.3. Formulación . . . 368
15.5.4. Cargas distribuidas . . . 368
15.5.5. Régimen de validez . . . 368
15.5.6. Independencia del diseño . . . 369
15.5.7. Validación . . . 369
15.6. Cable3DCorot — cable 3D corotacional . . . 369
15.6.1. Cinemática . . . 369
15.6.2. Pretensión . . . 369
15.6.3. Formulación . . . 369
15.6.4. Cargas distribuidas . . . 370
15.6.5. Régimen de validez . . . 370
15.6.6. Independencia del diseño . . . 370
15.6.7. Validación . . . 370
15.7. Frame2DEuler — marco/viga 2D Euler-Bernoulli . . . 370
15.7.1. Cinemática . . . 370
15.7.2. Formulación . . . 370
15.7.3. Constitutiva: dos caminos (ADR 0021) . . . 371
15.7.4. Cargas distribuidas . . . 371
15.7.5. Masa . . . 371
15.7.6. Régimen de validez . . . 372
15.7.7. Independencia del diseño . . . 372
15.7.8. Validación . . . 372
15.8. Frame2DTimoshenko — marco/viga 2D Timoshenko . . . 372
15.8.1. Cinemática . . . 372
15.8.2. Formulación . . . 373
15.8.3. Parámetros . . . 373
15.8.4. Cargas distribuidas . . . 373
15.8.5. Masa . . . 373
15.8.6. Régimen de validez . . . 373
15.8.7. Independencia del diseño . . . 374
15.8.8. Validación . . . 374
15.9. Frame2DEulerCorot — viga 2D Euler-Bernoulli corotacional . . . 374
15.9.1. Cinemática corotacional . . . 374
15.9.2. Formulación . . . 374
15.9.3. Constitutiva: dos caminos (ADR 0021) . . . 375
15.9.4. Cargas distribuidas . . . 375
15.9.5. Masa . . . 375

15.9.6. Régimen de validez . . . 375
15.9.7. Dominios que abre . . . 375
15.9.8. Independencia del diseño . . . 375
15.9.9. Validación . . . 376
15.10. Frame3D — marco/viga 3D Euler-Bernoulli . . . 376
15.10.1. Cinemática . . . 376
15.10.2. Ejes locales y orientación de la sección . . . 376
15.10.3. Formulación . . . 376
15.10.4. Constitutiva: dos caminos (ADR 0021) . . . 377
15.10.5. Parámetros . . . 377
15.10.6. Cargas distribuidas . . . 377
15.10.7. Régimen de validez . . . 377
15.10.8. Masa . . . 377
15.10.9. Masa lumped: limitación en orientación oblicua . . . 378
15.10.10. Independencia del diseño . . . 378
15.10.11. Validación . . . 378
15.11. FiberSection — sección por fibras de los marcos Euler-Bernoulli . . . 379
15.12. Quad4 — cuadrilátero bilineal 2D . . . 381
15.13. Tri3 — triángulo lineal 2D (CST) . . . 382
15.14. Quad8 — cuadrilátero serendípito 2D de orden 2 . . . 382
15.15. Quad9 — cuadrilátero Lagrangiano 2D de orden 2 . . . 383
15.16. Tri6 — triángulo 2D cuadrático completo P_2 . . . 383
15.17. Hex8 — hexaedro trilineal 3D . . . 384
15.18. Hex20 — hexaedro serendípito 3D de orden 2 (20 nodos) . . . 385
15.19. Hex27 — hexaedro Lagrangiano 3D triquadrático (27 nodos) . . . 387
15.20. Tet10 — tetraedro cuadrático 3D (10 nodos) . . . 388
15.21. Tet4 — tetraedro lineal 3D (CST 3D) . . . 389
15.22. Quad4Thermal — cuadrilátero bilineal de conducción 2D . . . 391
15.23. Hex8Thermal — hexaedro trilineal de conducción 3D . . . 392
15.24. Cómo añadir un elemento nuevo . . . 393

16. Modelos constitutivos — catálogo **394**
16.1. Elastic1D — elástico lineal 1D . . . 394
16.2. Elastic2D — elástico lineal 2D . . . 395
16.3. Orthotropic2D — elástico lineal **ortótropo** 2D (plane stress) . . . 395
16.4. Elastic3D — elástico lineal isótropo 3D . . . 396
16.5. IsotropicDamage1D — daño isótropo 1D con softening exponencial . . . 397
16.6. IsotropicDamage3D — daño isótropo 3D con softening exponencial . . . 398
16.7. IsotropicDamage2D — daño isótropo 2D con softening exponencial . . . 399
16.8. Elastoplastic1D — plasticidad J2 1D con endurecimiento isótropo lineal . . . 401
16.9. CableMaterial1D — cable axial 1D con elasticidad unilateral . . . 401
16.10. DruckerPrager3D — plasticidad friccional cohesivo-friccional 3D . . . 402
16.11. DruckerPrager2D — plasticidad friccional cohesivo-friccional 2D plane strain . . . 404
16.12. VonMises3D — plasticidad J2 3D con endurecimiento isótropo lineal . . . 405
16.13. VonMises2D — plasticidad J2 2D con endurecimiento isótropo lineal . . . 406

16.14. ThermalConduction — conducción de calor de Fourier con conductividad tensorial 408
16.15. Cómo añadir un material nuevo . . . 409

17. Solvers — catálogo 410
17.1. LinearSolver — solución lineal directa en un paso . . . 411
17.2. NonlinearSolver — Newton-Raphson incremental con paso adaptativo . . . 411
17.3. ArcLengthSolver — método de longitud de arco cilíndrico (Crisfield) . . . 413
17.4. IndirectDisplacementSolver — control indirecto de desplazamiento (de Borst 1987) 414
17.5. ModalSolver — análisis modal por autovalores generalizados (ADR 0009) . . . 414
17.6. NewmarkSolver — análisis dinámico transitorio lineal (ADR 0009 fase 3) . . . 416
17.7. NewtonNewmarkSolver — análisis dinámico transitorio no lineal (ADR 0009 fase 4) 417
17.8. HHTSolver — análisis dinámico transitorio lineal con disipación numérica (Hilber-Hughes-Taylor α) . . . 418
17.9. NewtonHHTSolver — análisis dinámico transitorio no lineal HHT-α . . . 419
17.10. CentralDifferenceSolver — integración explícita por diferencias centradas (ADR 0009 fase 5) . . . 420
17.11. HarmonicSolver — respuesta forzada armónica en el dominio de la frecuencia (ADR 0009 fase 6) . . . 421
17.12. ResponseSpectrumSolver — análisis sísmico por combinación modal (ADR 0009 fase 7) . . . 422
17.13. ThetaMethodSolver — conducción de calor transitoria por integración θ (Etapa 8) 423
17.14. Cómo añadir un solver nuevo . . . 425

18. Anexos técnicos 427
18.1. Dos capas que se llaman ambas «solver» . . . 427
18.2. Backends disponibles y criterio de selección . . . 427
18.3. Red de seguridad del análisis estático (ADR 0019) . . . 428
18.4. Fallback automático SPD → LU . . . 429
18.5. Factorización reusable y Newton modificado . . . 429
18.6. Override desde YAML — herramienta de diagnóstico . . . 430
18.7. Backends opcionales: por qué importan . . . 430
18.7.1. Pardiso — recomendado (`pip install solidum-fem[fast]`) . . . 431
18.7.2. Cholesky / CHOLMOD (`scikit-sparse`) . . . 431
18.8. Solver iterativo — a petición (ADR 0018) . . . 431
18.8.1. Instalar AMG (`pyamg`) . . . 432

A. Módulos de usuario — discontinuities 434
A.1. Elementos — catálogo . . . 434
A.1.1. CST_Embedded2D — triángulo CST con discontinuidad interior embebida (KOS) . . . 435
A.2. Modelos constitutivos — catálogo . . . 436
A.2.1. CohesiveDamageIsotropic — daño cohesivo isótropo Modo-I con softening lineal/exponencial . . . 437
A.3. CST_Embedded2D . . . 439
A.3.1. Especificación física . . . 439
A.3.2. Formulación numérica (FEM) . . . 441

A.3.3. Caveats numéricos . . . 445
A.3.4. Contrato de implementación . . . 446
A.3.5. Implementación . . . 449
A.3.6. Diálogo . . . 450
A.4. CohesiveDamageIsotropic . . . 452
A.4.1. Especificación física . . . 452
A.4.2. Formulación numérica . . . 455
A.4.3. Contrato de implementación . . . 458
A.4.4. Implementación . . . 461
A.4.5. Diálogo . . . 462

B. Módulos de usuario — dissipation_control **464**
B.1. Solvers — catálogo . . . 464
B.1.1. DissipationArcLengthSolver — variante de arc-length por disipación (Gutiérrez 2004) . . . 465
B.2. DissipationArcLengthSolver . . . 468
B.2.1. Especificación física . . . 468
B.2.2. Formulación numérica — solo lo que cambia respecto al padre . . . 470
B.2.3. Contrato de implementación . . . 478
B.2.4. Implementación . . . 481
B.2.5. Diálogo . . . 485

Sobre este Manual

Este manual contiene la **referencia formal** de los componentes de Solidum FEM: cada elemento finito y cada modelo constitutivo se documenta con su especificación física (ecuaciones), su formulación numérica (matrices **B**, rigidez tangente, integración) y su contrato YAML.

El contenido se genera automáticamente desde los archivos `docs/specs/*.md` del repositorio, que constituyen la *fuente única de verdad* de cada componente. El cuerpo del manual describe el **programa principal**; las specs de los **módulos de usuario** (formulaciones no estándar, en `docs/user/*/specs/*.md`) y sus entradas de los catálogos se recogen en el apéndice *Módulos de usuario*, un capítulo por módulo. No editar este PDF manualmente; cualquier corrección debe hacerse sobre la spec correspondiente y regenerar el manual mediante:

```
python manuals/build_reference_manual.py
```

Para una guía orientada al uso del programa (sintaxis YAML, ejemplos, post-procesamiento), consulte el **Manual de Usuario** en `manuals/User_manual.pdf`.

Capítulo 1

Elementos 1D — Armaduras

1.1 Truss2D

*Orden de trabajo. El usuario escribe la **especificación física**, la **formulación numérica** y el **contrato**. La IA rellena **implementación** y responde en **diálogo** durante el trabajo.*

1.1.1 Especificación física

0. Descripción general

Elemento sólido 1D de primer orden, inmerso en el plano. Dos nodos articulados; transmite únicamente fuerza axial. Aproximación C^0 del desplazamiento.

1. Cinemática de desplazamientos

Interpolación lineal del desplazamiento axial a lo largo del eje $s \in [0, L]$:

$$u(s) = a_0 + a_1 s$$

2. Cinemática de deformaciones

Única componente (axial, en el sistema local de la barra):

$$\varepsilon_{ss}(s) = \frac{du}{ds}$$

3. Ecuación constitutiva

$$\sigma_{ss} = E\,\varepsilon_{ss}$$

4. Equilibrio — forma fuerte

$$-\frac{d}{ds}\left(EA\,\frac{du}{ds}\right) = b(s), \quad s \in (0, L),$$

con condiciones de contorno Dirichlet o Neumann ($\sigma A = \bar{F}$) en los extremos.

5. Equilibrio — forma débil

$$\int_0^L \frac{d\,\delta u}{ds}\,EA\,\frac{du}{ds}\,ds \;=\; \int_0^L \delta u\,b\,ds \;+\; \left[\delta u\,\bar{F}\right]_{\partial\Omega}, \quad \forall\,\delta u \in V_0.$$

1.1.2 Formulación numérica (FEM)

6. Discretización

Dos nodos en $s = 0$ y $s = L$. Cosenos directores del eje en globales:

$$c = \frac{x_2 - x_1}{L}, \qquad s_\theta = \frac{y_2 - y_1}{L}.$$

7. Funciones de forma

$$N_1(s) = 1 - \tfrac{s}{L}, \qquad N_2(s) = \tfrac{s}{L}.$$

8. Matriz deformación-desplazamiento

Local (1 DOF axial por nodo): $\mathbf{B}_{\text{loc}} = \frac{1}{L}[-1,\ 1]$. Global ($\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_x^{(2)}, u_y^{(2)}]^\top$):

$$\mathbf{B} = \tfrac{1}{L}\,[\,-c,\ -s_\theta,\ c,\ s_\theta\,], \qquad \varepsilon = \mathbf{B}\,\mathbf{u}_e.$$

9. Rigidez elemental

Integración analítica exacta (EA constante):

$$\mathbf{K}_e = \int_0^L \mathbf{B}^\top EA\,\mathbf{B}\,ds = \frac{EA}{L}\,\mathbf{d}^\top\mathbf{d}, \qquad \mathbf{d} = [-c,\ -s_\theta,\ c,\ s_\theta].$$

10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sigma A\,\mathbf{d}.$$

11. Cuadratura

No aplica (forma cerrada).

1.1.3 Contrato de implementación

```yaml
name: Truss2D
kind: element
status: validated        # draft → implemented → validated

interface:
  dof_names: [ux, uy]
  n_nodes: 2
```

```yaml
  strain_dim: 1
  n_integration_points: 1

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - bathe: "cap. 4-5 (elemento de barra)"
  general_codes:
    - abaqus: "T2D2"
    - opensees: "element truss"

parameters:
  - { name: A, type: float, required: true, desc: Área de la sección transversal }

material_contract:
  signature: "compute_stateε(, state) -> σ(, E_tangent, state')"
  strain_kind: axial escalar

conventions:
  sign: "σ > 0 ⇔ε  > 0 (elongación)"
  voigt: "N/A (escalar)"
  node_orientation: "libre; K_e y F_int invariantes bajo permutación"

validity:
  - "ε|| ≲ 1e-2"
  - pequeños desplazamientos y rotaciones
  - cargas exclusivamente axiales

out_of_scope:
  - flexión, cortante, momentos en extremos
  - grandes desplazamientos
  - pandeo

acceptance:
  - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
    setup: "línea recta inclinada; solución exacta impuesta en un extremo y sus fuerzas
        internas como cargas nodales en el otro; carga de línea como fuerza de cuerpo
       b(x); u* axial; nodos libres restringidos a moverse a lo largo del eje; n = 4,
       8, 16, 32"
    expect: "con campos polinómicos (carga de grado ≤ 4), error nodal de redondeo con 1
        y 3 elementos (exactitud nodal, Tong 1969); con u* suave, orden ≥ 2 - 0.1 en L
       2 y ≥ 1 - 0.1 en energía entre los dos últimos niveles, error decreciente"
    tol_abs: 0.1
  - name: barra axial bajo carga puntual
    setup: "empotrada en s=0, carga F axial en s=L"
    expect: "u(L) = F·L / (E·A)"
    tol_rel: 1.0e-10

references:
  - "Bathe K.-J., Finite Element Procedures, §4.2.1"
```

1.1.4 Implementación

- **Archivo**: solidum/elements/truss.py
- **Clase**: `Truss2D` (registrada vía `@ElementRegistry.register`)

- **Tests**:
- `tests/test_line_elements.py` (campo interior, energía igual a la de la rigidez, carga como campo) y `tests/validation/test_mms_frames.py` (soluciones manufacturadas).
- tests/test_truss.py · `TestTruss2D` — verifica `L0`, cosenos directores, entradas de $\mathbf{K}_e$ (incluyendo simetría), y evaluación de $\mathbf{F}_{\text{int}}$ sobre desplazamientos conocidos. Los valores numéricos esperados coinciden con $\mathbf{K}_e = (EA/L)\,\mathbf{d}\mathbf{d}^\top$ y $\mathbf{F}_{\text{int}} = N\mathbf{d}$ para geometría de prueba $L = 5$, $c = 0.6$, $s = 0.8$.
- tests/test_integration.py · `TestSolversIntegration` — resuelve end-to-end el caso de aceptación (barra empotrada en un extremo, carga axial en el otro) con `NonlinearSolver` y `ArcLengthSolver`; en régimen elástico precedente a la fluencia reproduce $u(L) = FL/(EA)$.
- **Notas de traducción**:
- `L0`, `c`, `s` se calculan una vez en `__init__` sobre la configuración inicial y no se actualizan — coherente con el régimen infinitesimal declarado.
- El contrato con el material es el estándar del proyecto: `material.compute_state(`ε`, state)` → `(`σ`, E`$_t$`, state')`.
- Para grandes desplazamientos/rotaciones usar `Truss2DCorot`, que hereda de esta clase.

1.1.5 Diálogo

- **2026-04-21** · Cierre de ciclo retroactivo. La clase `Truss2D` preexistía al flujo spec-first; la spec se creó documentando la formulación ya implementada. Los tests unitarios y de integración satisfacen el único criterio de `acceptance` declarado (analíticamente, vía los valores de $\mathbf{K}_e$ y end-to-end vía el solver). Promovido `status: validated`.
- **2026-09-22** · Auditoría global: `internal_forces(U, equivalent_load=None)`: cuando el `Assembler` ensambló peso propio o fuerza de cuerpo, `solidum.run` pasa el vector nodal equivalente del elemento (escalado por λ_final) y las fuerzas internas de extremo son `F_int - f_eq` en ejes locales. Antes se devolvía `K·u`, que con carga distribuida son fuerzas nodales, no fuerzas internas: un voladizo bajo peso propio daba V_i = qL/2 y M_i = -5qL^2/12 en vez de qL y $-$qL^2/2, contradiciendo a las reacciones. Para barras y cables se proyectan las fuerzas de extremo netas sobre el eje (`axial_end_forces`): con carga axial distribuida N_i ≠ N_j.
- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022, fase 4). `compute_body_load` acepta un campo `b(x)`, integrado con Gauss de 4 puntos y repartido con las funciones lineales en cada componente global (la heredan `Truss2DCorot`). Declara `REFERENCE_CELL = "line"` y expone su campo interior (`line_field`: `u` axial y ε) con `section_rigidities` (`EA`). Nueva aceptación «soluciones manufacturadas»: exactitud nodal y órdenes 2 y 1.

1.2 Truss2DCorot

*Orden de trabajo. Usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

1.2.1 Especificación física

0. Descripción general

Barra articulada 2D equivalente a `Truss2D` pero en régimen de **grandes desplazamientos y rotaciones** con **pequeña deformación axial**. La cinemática se actualiza en configuración corriente (corotacional / Updated Lagrangian): el eje de la barra gira con el movimiento y la deformación se mide sobre la longitud actual. Rigidez tangente = rigidez material + rigidez geométrica debida a la fuerza axial.

1. Cinemática

Configuraciones:

- Referencia: nodos $\mathbf{X}_1, \mathbf{X}_2$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Cosenos directores en configuración corriente:

$$c_\theta = \frac{x_2 - x_1}{l}, \qquad s_\theta = \frac{y_2 - y_1}{l}.$$

2. Medida de deformación

Ingenieril corotacional (válida para pequeña deformación aunque la rotación sea grande):

$$\varepsilon = \frac{l - L_0}{L_0}.$$

3. Ecuación constitutiva

Material 1D: $\sigma = E\,\varepsilon$ (elástico lineal); el contrato admite cualquier material con `STRAIN_DIM = 1`.

4. Equilibrio — forma débil incremental

Principio de trabajos virtuales completo:

$$\underbrace{\int_V \delta\varepsilon\,\sigma\,dV}_{\delta W_{\text{int}}} = \underbrace{\int_V \delta\mathbf{u}^\top \mathbf{b}\,dV + \int_{\partial V_t} \delta\mathbf{u}^\top \bar{\mathbf{t}}\,dA}_{\delta W_{\text{ext}}}.$$

El elemento calcula **solo el lado interno**:

$$\delta W_{\text{int}} = \delta\mathbf{u}_e^\top \mathbf{F}_{\text{int}},$$

con la rigidez tangente consistente separada en parte material y geométrica (ver §9). Las cargas externas (nodales y, si las hubiera, de cuerpo o tracción) las ensambla el solver a partir del input del usuario; el vector nodal de una fuerza de cuerpo lo hereda de `Truss2D` (`compute_body_load`, constante o campo `b(x)`), repartido en la geometría de referencia.

1.2.2 Formulación numérica (FEM)

6. Discretización

Dos nodos, 2 DOFs por nodo (u_x, u_y). Vector elemental $\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_x^{(2)}, u_y^{(2)}]^\top$.

7. Vector dirección actual

$$\mathbf{d} = [-c_\theta,\ -s_\theta,\ c_\theta,\ s_\theta]^\top, \qquad \mathbf{n} = [-s_\theta,\ c_\theta,\ s_\theta,\ -c_\theta]^\top,$$

con c_θ, s_θ evaluados en la configuración corriente.

8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0}\,\mathbf{d}^\top, \qquad \varepsilon = \mathbf{B}\,\mathbf{u}_e.$$

9. Rigidez tangente

$$\mathbf{K}_T = \mathbf{K}_M + \mathbf{K}_G,$$
$$\mathbf{K}_M = \frac{EA}{L_0}\,\mathbf{d}\,\mathbf{d}^\top,$$
$$\mathbf{K}_G = \frac{N}{l}\,\mathbf{n}\,\mathbf{n}^\top, \qquad N = \sigma A.$$

10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N\,\mathbf{d}.$$

11. Cuadratura

No aplica (forma cerrada; un único punto conceptual en el eje).

1.2.3 Contrato de implementación

```yaml
name: Truss2DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_borst_crisfield: "§3.1, armadura corrotacional (1.ª ed.: Crisfield 1991, §3.3)
        "
  general_codes:
    - opensees: "element corotTruss"
    - abaqus: "T2D2 con NLGEOM"

parameters:
  - { name: A, type: float, required: true, desc: Área de la sección transversal }

material_contract:
  signature: "compute_stateε(, state) -> σ(, E_tangent, state')"
  strain_kind: axial escalar (deformación ingenieril corotacional)

conventions:
  sign: "σ > 0 ⇔ε  > 0 (elongación)"
  voigt: "N/A (escalar)"
  node_orientation: "libre; K_T y F_int invariantes bajo permutación"
  configuration: "Updated Lagrangian -θ c_, θs_, l se recalculan en cada evaluación"

validity:
  - "ε|| ≲ 1e-2 (pequeña deformación axial)"
  - rotaciones y desplazamientos de cualquier magnitud
  - cargas exclusivamente axiales

out_of_scope:
  - flexión, cortante, momentos
  - pandeo por bifurcación (captura de snap-through requiere arc-length)
  - plasticidad con grandes deformaciones
  - "fuerzas de cuerpo sobre la geometría corriente (se reparten en la de referencia,
      heredado de Truss2D)"

acceptance:
  - name: equivalencia con Truss2D en régimen lineal
    setup: "barra axial empotrada, carga F pequeña ε( ~ 1e-6)"
    expect: "u(L) = F·L/(E·A) con tol_rel 1e-8"
    tol_rel: 1.0e-8

  - name: invariancia bajo rotación de cuerpo rígido
    setup: "aplicar rotación rígida de 45° a ambos nodos (sin deformar)"
    expect: "F_int = 0 y σ = 0"
    tol_abs: 1.0e-10
```

```
  - name: rigidez geométrica bajo tensión
    setup: "barra en tensión con N > 0; verificar autovalor transverso"
    expect: "autovalor de K_T en dirección n = N/l"
    tol_rel: 1.0e-10

references:
  - "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
     .1, §3.3"
  - "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.5"
```

1.2.4 Implementación

- **Archivo**: solidum/elements/truss.py
- **Clase**: `Truss2DCorot` — hereda de `Truss2D` (reutiliza `__init__`, `ClassVars` y metadatos de registro) y sobrescribe `compute_element_state` y `compute_internal_forces` para evaluar cosenos directores y longitud en configuración corriente.
- **Tests**: tests/test_truss.py · `TestTruss2DCorot` — tres tests, uno por cada `acceptance`:
- `test_acceptance_linear_limit_matches_truss2d` (criterio 1).
- `test_acceptance_rigid_body_rotation` (criterio 2).
- `test_acceptance_geometric_stiffness_under_traction` (criterio 3).
- **Notas de traducción**:
- La longitud corriente l y los cosenos c_θ, s_θ se recalculan en cada evaluación (no se cachean entre llamadas del solver, como exige Updated Lagrangian).
- El vector $\mathbf{d}$ de §7 se construye con signos $[-c_\theta, -s_\theta, c_\theta, s_\theta]$; el transverso $\mathbf{n}$ como $[-s_\theta, c_\theta, s_\theta, -c_\theta]$. Ambos con norma $\sqrt{2}$ — el factor se absorbe en los coeficientes EA/L_0 y N/l.
- La rigidez tangente se devuelve ya sumada $\mathbf{K}_M + \mathbf{K}_G$; el solver no distingue las dos contribuciones.
- El test 3 confronta $\mathbf{K}_T$ con la descomposición esperada y además verifica $\mathbf{K}_G\mathbf{n} = (2N/l)\mathbf{n}$ (autovalor sobre el vector sin normalizar, que es N/l sobre el versor — así queda la ambigüedad resuelta explícitamente).

1.2.5 Diálogo

- **2026-04-21** · Implementación inicial tras validar el diseño spec-first. Se optó por herencia `Truss2DCorot(Truss2D)` para reutilizar el constructor (L0, cosenos iniciales) y los `ClassVars` del registro; los métodos que dependen de configuración corriente se sobrescriben. La bandera `large_strains` que tenía `Truss2D` previamente se elimina en el mismo commit — el régimen corotacional vive ahora como elemento independiente.

- **2026-04-21** · Normalización del autovalor transverso (criterio 3): la fórmula del libro da N/l sobre el **versor** $\hat{\mathbf{n}} = \mathbf{n}/\sqrt{2}$; sobre el vector $\mathbf{n}$ directo (norma $\sqrt{2}$) el autovalor es $2N/l$. El test verifica la segunda forma para evitar raíces cuadradas y la nota de traducción documenta la equivalencia.

- **2026-09-22** · Auditoría global: `internal_forces(U, equivalent_load=None)`: cuando el `Assembler` ensambló peso propio o fuerza de cuerpo, `solidum.run` pasa el vector nodal equivalente del elemento (escalado por λ_final) y las fuerzas internas de extremo son `F_int - f_eq` en ejes locales. Antes se devolvía `K·u`, que con carga distribuida son fuerzas nodales, no fuerzas internas: un voladizo bajo peso propio daba V_i = qL/2 y M_i = -5qL^2/12 en vez de qL y $-$qL^2/2, contradiciendo a las reacciones. Para barras y cables se proyectan las fuerzas de extremo netas sobre el eje (`axial_end_forces`): con carga axial distribuida N_i ≠ N_j.

- **2026-10-27** · Hereda de `Truss2D` la fuerza de cuerpo como campo `b(x)` (ADR 0022, fase 4), repartida en la geometría de referencia; se corrigen dos notas desactualizadas que decían que el elemento no calculaba el vector nodal de una fuerza de cuerpo.

1.3 Truss3D

*Orden de trabajo. El usuario escribe la **especificación física**, la **formulación numérica** y el **contrato**. La IA rellena **implementación** y responde en **diálogo** durante el trabajo.*

1.3.1 Especificación física

0. Descripción general

Elemento sólido 1D de primer orden, inmerso en el espacio tridimensional. Dos nodos articulados; transmite únicamente fuerza axial. Aproximación C^0 del desplazamiento. Régimen estrictamente de **linealidad geométrica**: pequeños desplazamientos, pequeñas rotaciones, pequeñas deformaciones.

1. Cinemática de desplazamientos

Interpolación lineal del desplazamiento axial a lo largo del eje $s \in [0, L]$:

$$u(s) = a_0 + a_1 s.$$

2. Cinemática de deformaciones

Única componente (axial, en el sistema local de la barra):

$$\varepsilon_{ss}(s) = \frac{du}{ds}.$$

3. Ecuación constitutiva

$$\sigma_{ss} = E\,\varepsilon_{ss}.$$

El elemento admite cualquier material 1D con `STRAIN_DIM = 1` (elástico lineal, plástico 1D, daño 1D).

4. Equilibrio — forma fuerte

$$-\frac{d}{ds}\left(EA\,\frac{du}{ds}\right) = b(s), \quad s \in (0, L),$$

con condiciones de contorno Dirichlet o Neumann ($\sigma A = \bar{F}$) en los extremos.

5. Equilibrio — forma débil

$$\int_0^L \frac{d\,\delta u}{ds}\,EA\,\frac{du}{ds}\,ds \;=\; \int_0^L \delta u\, b\, ds \;+\; \left[\delta u\,\bar{F}\right]_{\partial\Omega}, \quad \forall\,\delta u \in V_0.$$

El elemento calcula **solo el lado interno**; las cargas externas las ensambla el solver a partir del input del usuario.

1.3.2 Formulación numérica (FEM)

6. Discretización

Dos nodos en $s = 0$ y $s = L$. Cosenos directores del eje en coordenadas globales:

$$c_x = \frac{x_2 - x_1}{L}, \quad c_y = \frac{y_2 - y_1}{L}, \quad c_z = \frac{z_2 - z_1}{L},$$

con $L = \|\mathbf{X}_2 - \mathbf{X}_1\|$ (configuración inicial, fija).

7. Funciones de forma

$$N_1(s) = 1 - \tfrac{s}{L}, \qquad N_2(s) = \tfrac{s}{L}.$$

8. Matriz deformación-desplazamiento

Local (1 DOF axial por nodo): $\mathbf{B}_{\text{loc}} = \frac{1}{L}[-1,\ 1]$. Global con $\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}]^\top$:

$$\mathbf{B} = \tfrac{1}{L}\,[-c_x,\ -c_y,\ -c_z,\ c_x,\ c_y,\ c_z], \qquad \varepsilon = \mathbf{B}\,\mathbf{u}_e.$$

9. Rigidez elemental

Integración analítica exacta (EA constante, $\mathbf{B}$ constante):

$$\mathbf{K}_e = \int_0^L \mathbf{B}^\top EA\,\mathbf{B}\,ds = \frac{EA}{L}\,\mathbf{d}\,\mathbf{d}^\top, \qquad \mathbf{d} = [-c_x,\ -c_y,\ -c_z,\ c_x,\ c_y,\ c_z]^\top.$$

$\mathbf{K}_e$ es una matriz 6×6 de rango 1.

10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sigma A\,\mathbf{d} = N\,\mathbf{d}.$$

11. Cuadratura

No aplica (forma cerrada, $\mathbf{B}$ y σ uniformes por elemento). El campo `n_integration_points: 1` se refiere al número de estados materiales conceptuales (uno por elemento), no a puntos de Gauss.

1.3.3 Contrato de implementación

```yaml
name: Truss3D
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 2
  strain_dim: 1
```

```yaml
  n_integration_points: 1

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - bathe: "cap. 4-5 (elemento de barra)"
  general_codes:
    - abaqus: "T3D2"
    - opensees: "element truss (ndm = 3)"

parameters:
  - { name: A, type: float, required: true, desc: Área de la sección transversal }

material_contract:
  signature: "compute_stateε(, state) -> σ(, E_tangent, state')"
  strain_kind: axial escalar

conventions:
  sign: "σ > 0 ⇔ε  > 0 (elongación)"
  voigt: "N/A (escalar)"
  node_orientation: "libre; K_e y F_int invariantes bajo permutación"
  configuration: "inicial fija - L, c_x, c_y, c_z no se actualizan con los
      desplazamientos"

validity:
  - "ε|| ≲ 1e-2"
  - pequeños desplazamientos y rotaciones
  - cargas exclusivamente axiales

out_of_scope:
  - flexión, cortante, momentos
  - grandes desplazamientos o rotaciones (para ese régimen usar `Truss3DCorot`)
  - pandeo
  - "fuerzas de cuerpo distribuidas sobre el eje: el usuario reparte masa a los nodos"

acceptance:
  - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
    setup: "línea recta inclinada; solución exacta impuesta en un extremo y sus fuerzas
        internas como cargas nodales en el otro; carga de línea como fuerza de cuerpo
       b(x); línea orientada al azar en el espacio; u* axial; nodos libres
       restringidos al eje; n = 4, 8, 16, 32"
    expect: "con campos polinómicos (carga de grado ≤ 4), error nodal de redondeo con 1
        y 3 elementos (exactitud nodal, Tong 1969); con u* suave, orden ≥ 2 - 0.1 en L
       2  y ≥ 1 - 0.1 en energía, error decreciente"
    tol_abs: 0.1
  - name: barra axial bajo carga puntual (3D)
    setup: "empotrada en s=0, carga F axial en s=L, orientación arbitraria en el
       espacio"
    expect: "u(L) = F·L / (E·A) proyectado sobre el eje"
    tol_rel: 1.0e-10

references:
  - "Bathe K.-J., Finite Element Procedures, §4.2.1"
  - "Cook R.D., Concepts and Applications of FEA, §2.3"
```

1.3.4 Implementación

- **Archivo**: solidum/elements/truss.py
- **Clase**: `Truss3D` — hereda directamente de `Element` (sin relación de herencia con `Truss2D` ni con `Truss2DCorot`).
- **Tests**: `tests/test_line_elements.py` (campo interior, energía igual a la de la rigidez, carga como campo) y `tests/validation/test_mms_frames.py` (soluciones manufacturadas); tests/test_truss.py · `TestTruss3D` — verifica `L0`, cosenos directores (c_x, c_y, c_z), entradas de $\mathbf{K}_e$ (incluyendo simetría, dimensiones 6×6) y evaluación de $\mathbf{F}_{\text{int}}$ sobre desplazamientos conocidos con geometría de prueba $L = 13$, $(c_x, c_y, c_z) = (3/13, 4/13, 12/13)$.
- **Notas de traducción**:
- `L0`, `cx`, `cy`, `cz` se calculan una vez en `__init__` sobre la configuración inicial y no se actualizan — coherente con el régimen geométricamente lineal declarado.
- El constructor tolera nodos con 2 ó 3 coordenadas (completa con $z = 0$ si falta), para facilitar transición de problemas planos embebidos.
- El contrato con el material es el estándar del proyecto: `material.compute_state(`ε`, state)` $\rightarrow$ `(`σ`, E`$_t$`, state')`.
- Para grandes desplazamientos/rotaciones en 3D usar `Truss3DCorot`, que hereda de esta clase.

1.3.5 Diálogo

- **2026-04-21** · Apertura de spec retroactiva. La clase `Truss3D` preexistía al flujo spec-first; la spec se crea documentando la formulación ya implementada y estableciendo explícitamente su alcance: régimen de linealidad geométrica, sin bandera ni variante corotacional. Los tests unitarios existentes cubren el criterio de `acceptance` (vía los valores de $\mathbf{K}_e = (EA/L)\mathbf{d}\mathbf{d}^\top$ que implican algebraicamente $u(L) = FL/(EA)$). Promovido `status: validated`.
- **2026-09-22** · Auditoría global: `internal_forces(U, equivalent_load=None)`: cuando el `Assembler` ensambló peso propio o fuerza de cuerpo, `solidum.run` pasa el vector nodal equivalente del elemento (escalado por λ_final) y las fuerzas internas de extremo son `F_int - f_eq` en ejes locales. Antes se devolvía `K·u`, que con carga distribuida son fuerzas nodales, no fuerzas internas: un voladizo bajo peso propio daba V_i = qL/2 y M_i = -5qL^2/12 en vez de qL y $-$qL^2/2, contradiciendo a las reacciones. Para barras y cables se proyectan las fuerzas de extremo netas sobre el eje (`axial_end_forces`): con carga axial distribuida N_i $\neq$ N_j.
- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022, fase 4), como en `Truss2D` (la hereda `Truss3DCorot`). Nueva aceptación «soluciones manufacturadas» con la línea orientada al azar.

1.4 Truss3DCorot

*Orden de trabajo. Usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

1.4.1 Especificación física

0. Descripción general

Barra articulada 3D equivalente a `Truss3D` pero en régimen de **grandes desplazamientos y rotaciones** con **pequeña deformación axial**. Updated Lagrangian / corotacional: el eje de la barra se redefine en cada iteración sobre la configuración corriente y la deformación se mide sobre la longitud actual. La rigidez tangente suma material + geométrica.

1. Cinemática

Configuraciones:

- Referencia: $\mathbf{X}_1, \mathbf{X}_2 \in \mathbb{R}^3$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Vector unitario del eje corriente:

$$\hat{\mathbf{e}} = \frac{\mathbf{x}_2 - \mathbf{x}_1}{l} = (c_x, c_y, c_z), \qquad c_x^2 + c_y^2 + c_z^2 = 1.$$

2. Medida de deformación

Ingenieril corotacional:

$$\varepsilon = \frac{l - L_0}{L_0}.$$

3. Ecuación constitutiva

Material 1D con `STRAIN_DIM = 1`: $\sigma = \sigma(\varepsilon)$ (el elemento no hardcodea la ley; la provee el material).

4. Equilibrio — forma débil incremental

PTV estándar. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^\top \, \mathbf{F}_{\text{int}}, \qquad \mathbf{K}_T = \frac{\partial \mathbf{F}_{\text{int}}}{\partial \mathbf{u}_e} = \mathbf{K}_M + \mathbf{K}_G.$$

1.4.2 Formulación numérica (FEM)

6. Discretización

Dos nodos, 3 DOFs por nodo (u_x, u_y, u_z). Vector elemental de 6 componentes:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}]^\top.$$

7. Vector dirección actual

$$\mathbf{d} = [-c_x,\ -c_y,\ -c_z,\ c_x,\ c_y,\ c_z]^\top,$$

con los cosenos evaluados en configuración **corriente**. $\|\mathbf{d}\|^2 = 2$.

8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0}\,\mathbf{d}^\top, \qquad \varepsilon = \mathbf{B}\,\mathbf{u}_e.$$

9. Rigidez tangente

Material:

$$\mathbf{K}_M = \frac{E_t\,A}{L_0}\,\mathbf{d}\,\mathbf{d}^\top \quad (6\times 6,\ \text{rango } 1).$$

Geométrica: en 3D la dirección perpendicular al eje es un **plano** (no un vector único). Se usa el proyector perpendicular $\mathbf{P}_3 = \mathbf{I}_3 - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ (matriz 3×3, rango 2). La rigidez geométrica en el elemento de 6 DOFs:

$$\mathbf{K}_G = \frac{N}{l}\begin{bmatrix} \mathbf{P}_3 & -\mathbf{P}_3 \\ -\mathbf{P}_3 & \mathbf{P}_3 \end{bmatrix} \quad (6\times 6,\ \text{rango } 2), \qquad N = \sigma A.$$

10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N\,\mathbf{d}.$$

11. Cuadratura

No aplica (forma cerrada).

1.4.3 Contrato de implementación

```yaml
name: Truss3DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_borst_crisfield: "§3.1, armadura corrotacional (1.ª ed.: Crisfield 1991, §3.3)
        "
  general_codes:
    - opensees: "element corotTruss (ndm = 3)"
```

```
    - abaqus: "T3D2 con NLGEOM"

parameters:
  - { name: A, type: float, required: true, desc: Área de la sección transversal }

material_contract:
  signature: "compute_stateε(, state) -> σ(, E_tangent, state')"
  strain_kind: axial escalar (deformación ingenieril corotacional)

conventions:
  sign: "σ > 0 ⇔ε  > 0 (elongación)"
  voigt: "N/A (escalar)"
  node_orientation: "libre"
  configuration: "Updated Lagrangian - l, c_x, c_y, c_z se recalculan en cada evaluació
      n"

validity:
  - "ε|| ≤ 1e-2"
  - rotaciones y desplazamientos de cualquier magnitud en el espacio
  - cargas exclusivamente axiales

out_of_scope:
  - flexión, cortante, momentos
  - pandeo por bifurcación
  - plasticidad con grandes deformaciones
  - "fuerzas de cuerpo sobre la geometría corriente (se reparten en la de referencia,
      heredado de Truss3D)"

acceptance:
  - name: equivalencia con Truss3D en régimen lineal
    setup: "barra 3D con orientación arbitraria, carga axial pequeña ε( ~ 1e-6)"
    expect: "K_T y F_int coinciden con Truss3D lineal a tolerancia rtol=1e-4"
    tol_rel: 1.0e-4

  - name: invariancia bajo rotación rígida 3D
    setup: "aplicar una rotación rígida arbitraria (p. ej. 30° sobre eje z, luego 45°
        sobre eje x)"
    expect: "F_int = 0 y σ = 0"
    tol_abs: 1.0e-10

  - name: rigidez geométrica transversa (plano perpendicular)
    setup: "barra en tensión con N > 0; aplicar K_G sobre dos vectores perpendiculares
        al eje linealmente independientes"
    expect: "K_G·v_perp = (N/l)·v_perp_struct, donde v_perp_struct es la extensión del
        vector transverso al espacio de 6 DOFs con signos opuestos en los dos nodos"
    tol_rel: 1.0e-10

references:
  - "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
      .1, §3.3"
  - "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.5"
```

1.4.4 Implementación

- **Archivo**: solidum/elements/truss.py

- **Clase**: `Truss3DCorot` — hereda de `Truss3D` (reutiliza `__init__`, tolerancia a nodos 2D/3D, metadatos de registro) y sobrescribe `compute_element_state` y `compute_internal_forces` para evaluar longitud y cosenos directores en configuración corriente.
- **Tests**: tests/test_truss.py · `TestTruss3DCorot` — tres tests, uno por cada `acceptance`:
- `test_acceptance_linear_limit_matches_truss3d` (criterio 1).
- `test_acceptance_rigid_body_rotation` (criterio 2 — dos rotaciones rígidas distintas, verifica σ=0 y F_int=0 en ambas).
- `test_acceptance_geometric_stiffness_transverse_plane` (criterio 3 — aplica $\mathbf{K}_G$ a **dos** direcciones linealmente independientes del plano perpendicular al eje).
- **Notas de traducción**:
- La clave respecto a 2D es el proyector $\mathbf{P}_3 = \mathbf{I} - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ (3×3, rango 2): el plano perpendicular al eje es bidimensional. $\mathbf{K}_G$ resulta entonces de rango 2, no rango 1.
- El proyector se construye con `solidum.math.geometry.perpendicular_projector(ê)`, helper compartido con `Cable3DCorot` (operación geométrica pura, no específica de armaduras ni de cables).
- La longitud corriente l y los cosenos c_x, c_y, c_z se recalculan en cada evaluación (Updated Lagrangian); no se cachean entre llamadas del solver.
- Autovalores de $\mathbf{K}_G$ sobre modos transversos (rotación de la barra alrededor de su centro): $2N/l$ sobre el vector sin normalizar — igual que en 2D, el factor 2 viene de que el vector de 6 componentes tiene norma $\sqrt{2}$. Sobre el versor el autovalor es N/l.

1.4.5 Diálogo

- **2026-04-21** · Implementación tras validar el diseño spec-first. Se optó por herencia `Truss3DCorot(Truss3D)` por paralelismo con `Truss2DCorot(Truss2D)` y reutilización del constructor. La diferencia estructural con el caso 2D es el proyector **P**: en 2D tenía rango 1 y generaba un $\mathbf{K}_G$ de rango 1 (vector **n** único); en 3D tiene rango 2 y $\mathbf{K}_G$ rango 2 (plano perpendicular al eje). Los tests del criterio 3 verifican explícitamente **dos** direcciones transversas linealmente independientes, capturando este cambio dimensional.
- **2026-04-21** · Test de invariancia bajo rotación rígida 3D: se verifican dos rotaciones distintas (plano x-y y plano x-z) para descartar coincidencias accidentales en ejes canónicos.
- **2026-09-22** · Auditoría global: `internal_forces(U, equivalent_load=None)`: cuando el `Assembler` ensambló peso propio o fuerza de cuerpo, `solidum.run` pasa el vector nodal equivalente del elemento (escalado por λ_final) y las fuerzas internas de extremo son `F_int - f_eq` en ejes locales. Antes se devolvía `K·u`, que con carga distribuida son fuerzas nodales, no fuerzas internas: un voladizo bajo peso propio daba V_i = qL/2 y M_i = -5qL^2/12 en vez de qL y $-$qL^2/2, contradiciendo a las reacciones. Para barras y cables se proyectan las fuerzas de extremo netas sobre el eje (`axial_end_forces`): con carga axial distribuida N_i $\neq$ N_j.

- **2026-10-27** · Hereda de `Truss3D` la fuerza de cuerpo como campo `b(x)` (ADR 0022, fase 4), repartida en la geometría de referencia; se corrige la nota desactualizada de `out_of_scope`.

Capítulo 2

Elementos 1D — Cables

2.1 Cable2DCorot

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

2.1.1 Especificación física

0. Descripción general

Elemento 1D inmerso en el plano 2D que modela un **cable**: dos nodos articulados que solo transmiten **fuerza axial de tensión**. En compresión o estado sin tensar, el elemento no transmite nada — se destensa. Cinemática **corotacional** (Updated Lagrangian): el eje rota con el movimiento, la longitud se mide sobre la configuración corriente.

La propiedad de unilateralidad vive en el **material**; el elemento es la maquinaria cinemática que aplica al material la deformación correcta. Para obtener el comportamiento propio de cable se empareja este elemento con un material unilateral como `CableMaterial1D`. Con un material elástico clásico, el elemento se comporta como una barra articulada corotacional (útil para pruebas, no para modelado realista de cables).

1. Cinemática

Configuraciones:

- Referencia: $\mathbf{X}_1, \mathbf{X}_2 \in \mathbb{R}^2$, longitud geométrica $L = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Sin tensión: longitud L_0 del cable descargado (parámetro opcional `L0`; por defecto $L_0 = L$).
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Cosenos directores del eje **corriente**:

$$c_\theta = \frac{x_2 - x_1}{l}, \qquad s_\theta = \frac{y_2 - y_1}{l}.$$

2. Medida de deformación

Ingenieril corotacional, respecto de la longitud sin tensión:

$$\varepsilon = \frac{l - L_0}{L_0}.$$

Pretensión (2026-10-01). En la configuración de referencia ($l = L$) la deformación vale $\varepsilon_0 = (L - L_0)/L_0$. Con $L_0 < L$ el cable arranca pretensado: con material elástico en tracción, $N_0 = E\,A\,\varepsilon_0$, es decir, $L_0 = L/(1 + N_0/(EA))$ para una pretensión N_0 dada. Con $L_0 > L$ arranca flojo. Con $L_0 = L$ (valor por defecto) se recupera exactamente el comportamiento anterior.

3. Ecuación constitutiva

Delegada al material con `STRAIN_DIM = 1`. El elemento invoca `material.compute_state(`ε`, state)` y recibe $(\sigma, E_t,$ nuevo estado) sin inspeccionar su naturaleza. La unilateralidad (si la hay) es responsabilidad del material — el elemento nunca filtra σ ni ε por signo.

4. Equilibrio — forma débil incremental

Principio de trabajos virtuales, lado interno:

$$\delta W_{\text{int}} = \int_V \delta\varepsilon\,\sigma\,dV = \delta\mathbf{u}_e^\top\,\mathbf{F}_{\text{int}}, \qquad \mathbf{K}_T = \frac{\partial \mathbf{F}_{\text{int}}}{\partial \mathbf{u}_e} = \mathbf{K}_M + \mathbf{K}_G.$$

Las cargas externas las ensambla el solver. El vector nodal equivalente de una fuerza de cuerpo $\mathbf{b}$ es $\mathbf{b}\,A\,L_0/2$ por nodo (`compute_body_load`): $A\,L_0$ es el volumen de material del cable descargado.

2.1.2 Formulación numérica (FEM)

6. Discretización

Dos nodos, 2 DOFs por nodo (u_x, u_y). Vector elemental:

$$\mathbf{u}_e = [u_x^{(1)},\ u_y^{(1)},\ u_x^{(2)},\ u_y^{(2)}]^\top.$$

7. Vectores de dirección (configuración corriente)

$$\mathbf{d} = [-c_\theta,\ -s_\theta,\ c_\theta,\ s_\theta]^\top, \qquad \mathbf{n} = [-s_\theta,\ c_\theta,\ s_\theta,\ -c_\theta]^\top.$$

$\mathbf{d}$ apunta a lo largo del eje corriente (norma $\sqrt{2}$); $\mathbf{n}$ al perpendicular (norma $\sqrt{2}$). Son ortogonales: $\mathbf{d}^\top\mathbf{n} = 0$.

8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0}\,\mathbf{d}^\top, \qquad \varepsilon = \mathbf{B}\,\mathbf{u}_e.$$

9. Rigidez tangente

$$\mathbf{K}_T = \mathbf{K}_M + \mathbf{K}_G,$$
$$\mathbf{K}_M = \frac{E_t\, A}{L_0}\,\mathbf{d}\,\mathbf{d}^\top, \qquad \mathbf{K}_G = \frac{N}{l}\,\mathbf{n}\,\mathbf{n}^\top, \qquad N = \sigma A.$$

Caso destensado (material unilateral con $\varepsilon \le 0$): el material devuelve $\sigma = 0$, $E_t = 0$, luego $N = 0$ y **ambas contribuciones se anulan** — $\mathbf{K}_T = \mathbf{0}$. El elemento aporta cero al sistema global, como físicamente corresponde a un cable flojo.

Caso pretensado ($L_0 < L$): en la configuración de referencia $N = N_0 > 0$, así que $\mathbf{K}_G = (N_0/L)\,\mathbf{n}\,\mathbf{n}^\top \neq \mathbf{0}$. El cable aporta rigidez transversal desde el primer paso, y un modelo formado sólo por cables pretensados deja de ser singular.

10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N\,\mathbf{d}.$$

Cuando el cable está destensado ($N = 0$), $\mathbf{F}_{\text{int}} = \mathbf{0}$.

11. Cuadratura

No aplica (forma cerrada).

2.1.3 Contrato de implementación

```yaml
name: Cable2DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_borst_crisfield: "§3.1, cinemática de la armadura corrotacional; la
        unilateralidad la aporta CableMaterial1D"
  general_codes:
    - ansys: "LINK180 con SECCONTROL de sólo tracción"
    - abaqus: "T2D2 + *NO COMPRESSION, NLGEOM"
    - opensees: "corotTruss + uniaxialMaterial Elastic con Eneg = 0"

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal del
      cable" }
  - { name: L0, type: float, required: false, desc: "Longitud sin tensión (> 0). Por
      defecto, la longitud geométrica entre nodos; menor ⇒ cable pretensado, mayor ⇒
      cable flojo" }
```

```
material_contract:
  signature: "compute_stateε(, state) -> σ(, E_tangent, state')"
  strain_kind: axial escalar (deformación ingenieril corotacional)
  expected_behaviour: "material unilateral (p. ej. CableMaterial1D) para obtener
      respuesta física de cable; cualquier material con STRAIN_DIM=1 es técnicamente
      aceptado pero producirá comportamiento de barra"

conventions:
  sign: "σ > 0 ⇔ε  > 0 (elongación); la responsabilidad de anular σ en compresión es
      del material"
  voigt: "N/A (escalar)"
  node_orientation: "libre; K_T y F_int invariantes bajo permutación"
  configuration: "Updated Lagrangian - l, θc_, θs_ se recalculan en cada evaluación"

validity:
  - "ε|| ≲ 1e-2 en régimen tensado"
  - rotaciones y desplazamientos de cualquier magnitud
  - cargas axiales (las transversales se transmiten por el destensado-retensado del
      cable)

out_of_scope:
  - flexión, cortante, momentos
  - pandeo (irrelevante: el cable no transmite compresión)
  - "catenaria: el cable es recto entre nodos; el peso propio se reparte a los nodos y
      la flecha se obtiene discretizando en varios elementos"

numerical_caveats:
  - "Un cable completamente destensado aporta K_T = 0 al sistema global. Si otros
      elementos no garantizan rigidez suficiente en los DOFs compartidos, la matriz
      tangente global puede ser singular o mal condicionada."
  - "En transiciones tensado ↔ destensado ε( cruza 0), Newton-Raphson puede oscilar por
       el salto discontinuo en E_t. Usar arc-length o pasos de carga finos si el
      problema atraviesa destensiones."

acceptance:
  - name: cable tensado coincide con barra corotacional
    setup: "cable horizontal con material lineal (Elastic1D, E=1000), u_x nodo 2 = 1e-3
        ⇒ε  = 1e-3 > 0"
    expect: "σ = E·ε, E_t = E, K_T = K_M + K_G con valores de barra corotacional"
    tol_rel: 1.0e-10

  - name: cable destensado produce aportación nula
    setup: "cable horizontal con CableMaterial1D, u_x nodo 2 = -1e-3 ⇒ε  < 0"
    expect: "σ = 0, F_int = 0, K_T = matriz cero"
    tol_abs: 1.0e-12

  - name: invariancia bajo rotación rígida
    setup: "cable inicialmente horizontal, rotación rígida de 45° de ambos nodos"
    expect: "σ = 0 y F_int = 0 (independiente del material)"
    tol_abs: 1.0e-10

  - name: cruce por cero
    setup: "u_e tal que ε = 0 exactamente, material unilateral"
    expect: "σ = 0, E_t = 0, F_int = 0, K_T = 0"
    tol_abs: 1.0e-12
```

```
  - name: pretensión inicial exacta
    setup: "CableMaterial1D (E=2e11), A=3e-4, L=4, L0 = L/(1 + T0/(E·A)) con T0 = 25 kN
        ; u_e = 0"
    expect: "N = T0, K_T = (E·A/L0)·d ⊤d + (T0/L)·n ⊤n, F_int = T0·d"
    tol_rel: 1.0e-12

  - name: cuerda pretensada con carga central
    setup: "dos cables colineales de L/2 con extremos fijos, pretensión T0, carga P = 3
         kN transversal en el nodo central; NonlinearSolver"
    expect: "flecha y axil iguales a la solución analítica no lineal del equilibrio 2·N
        (l)·w/l = P, con l = √((L/2)² + w²) y N = E·A·(l - L0)/L0; con P = 10 N ≪ T0, w
         → P·L/(4·T0)"
    tol_rel: 1.0e-9

  - name: cable flojo si L0 > L
    setup: "CableMaterial1D, L0 = 1.01·L, u_e = 0"
    expect: "σ = 0, F_int = 0, K_T = 0"
    tol_abs: 0.0

  - name: L0 por defecto
    setup: "mismo cable sin L0 y con L0 = longitud geométrica, u_e arbitrario"
    expect: "K_T y F_int idénticos bit a bit"
    tol_abs: 0.0

  - name: L0 inválida
    setup: "L0 ∈ {0, -1, ∞, NaN}"
    expect: "ValueError que nombra L0"

  - name: masa y carga de cuerpo con la longitud sin tensión
    setup: "CableMaterial1D con density = 7850, L0 de la pretensión T0"
    expect: "masa total ρ·A·L0 (consistente y concentrada) y carga de cuerpo ρ·g·A·L0/2
         por nodo"
    tol_rel: 1.0e-13

references:
  - "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
      .1, §3.3"
  - "Irvine H.M., Cable Structures, MIT Press, §1.2"
```

2.1.4 Implementación

- **Archivo**: solidum/elements/cable.py — archivo propio, no referencia a los módulos de armadura ni a ningún otro elemento.
- **Clase**: `Cable2DCorot` — hereda directamente de `Element` (no de `Truss2DCorot` ni de ningún elemento de armadura). La maquinaria cinemática corotacional se reimplementa íntegra dentro de la clase.
- **Tests**: tests/test_cable_elements.py · `TestCable2DCorot` — los cuatro criterios de `acceptance` más un test de registro:
- `test_acceptance_cable_tensado_como_barra_corot` (criterio 1).

- `test_acceptance_cable_destensado_aportacion_nula` (criterio 2).
- `test_acceptance_rotacion_rigida` (criterio 3).
- `test_acceptance_cruce_por_cero` (criterio 4).
- `test_registro_en_registry` — autodiscover.
- `TestPretensionLongitudSinTension` (criterios 5-10, compartidos con `Cable3DCorot`): `test_pretension_inicial_exacta_2d`, `test_cuerda_pretensada_carga_central_2d`, `test_cuerda_pretensada_limite_carga_pequena`, `test_cable_flojo_si_L0_mayor_que_la_geometrica`, `test_L0_por_defecto_es_la_longitud_geometrica`, `test_L0_invalido_se_rechaza`, `test_masa_y_carga_de_cuerpo_con_longitud_sin_tension` y `test_L0_desde_yaml`.
- **Notas de traducción**:
- La clase duplica deliberadamente la lógica de `Truss2DCorot`. No hay herencia. Es el precio de la independencia: si mañana se toca la armadura corotacional, este cable no se mueve.
- `_current_geometry(u_e)` devuelve `(l, c_θ, s_θ)` — método privado para aislar la única operación que depende de configuración corriente.
- En estado destensado, $\mathbf{K}_M = \mathbf{0}$ (porque $E_t = 0$ viene del material) y $\mathbf{K}_G = \mathbf{0}$ (porque $N = 0$); el elemento aporta literalmente ceros al ensamblaje, como debe ser físicamente un cable flojo. Los tests lo verifican con tolerancia absoluta.
- El elemento no valida que el material sea unilateral: es responsabilidad del usuario emparejar con `CableMaterial1D`. Emparejar con `Elastic1D` produce respuesta de barra corotacional y se usa internamente en el test del criterio 1 para comparar contra valores analíticos.

2.1.5 Diálogo

- **2026-04-21** · Elemento creado como componente totalmente independiente de `Truss2DCorot` por decisión explícita del usuario: la maquinaria cinemática se duplica dentro de la clase de cable para desacoplar su evolución futura del elemento de armadura. El coste es ~40 líneas duplicadas; la ganancia es que ninguna refactorización de las armaduras puede contaminar silenciosamente el comportamiento de los cables.
- **2026-04-21** · Validación de material unilateral: se optó por **no** imponerla en `__init__`. Motivo: el contrato elemento↔material del proyecto es genérico sobre `STRAIN_DIM=1`; introducir una verificación específica crearía un acoplamiento entre la clase del elemento y la identidad del material. Se documenta en `material_contract.expected_behaviour` del YAML para orientar al usuario.
- **2026-09-22** · Auditoría global: `internal_forces(U, equivalent_load=None)`: cuando el `Assembler` ensambló peso propio o fuerza de cuerpo, `solidum.run` pasa el vector nodal equivalente del elemento (escalado por λ_final) y las fuerzas internas de extremo son `F_int - f_eq` en ejes locales. Antes se devolvía `K·u`, que con carga distribuida son fuerzas nodales, no fuerzas internas: un voladizo bajo peso propio daba V_i = qL/2 y M_i = $-5qL^2/12$ en vez de

qL y $-\mathrm{qL}^2/2$, contradiciendo a las reacciones. Para barras y cables se proyectan las fuerzas de extremo netas sobre el eje (`axial_end_forces`): con carga axial distribuida N_i ≠ N_j.

- **2026-10-01** · **Pretensión por longitud sin tensión** (pedida desde SolidumBAR, formulación aprobada por el usuario). Parámetro opcional `L0`; por defecto, la longitud geométrica, con lo que nada cambia. Con $L_0 < L$ el cable arranca con $N_0 = EA(L - L_0)/L_0$ y rigidez geométrica no nula. Se descartaron la deformación inicial en el material (`CableMaterial1D` no tiene historia) y una fuerza N_0 sumada a la del material (con un material unilateral que mide ε desde L, para $L_0 < l < L$ el material devuelve cero y el axil quedaría congelado en N_0 en vez de decrecer hasta anularse en $l = L_0$). La masa y la carga de cuerpo usan $\rho A L_0$: el material del cable descargado. Se corrigen de paso tres afirmaciones desfasadas de esta spec: el elemento sí tiene matriz de masa (ADR 0009) y sí calcula la carga de cuerpo; la pretensión ya no está fuera de alcance. La cuerda pretensada en 3D destapó un falso positivo de la red de seguridad del ADR 0019 con nodos alineados, corregido en `model_checks.py` (ver su test de regresión en `tests/test_model_safety_net.py`).
- **2026-10-27** · Una fuerza de cuerpo como campo `b(x)` (ADR 0022, fase 4) se rechaza con un mensaje claro: el cable reparte la carga con su longitud sin tensión L_0, y el campo no está implementado; la constante sigue igual.

2.2 Cable3DCorot

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

2.2.1 Especificación física

0. Descripción general

Elemento 1D inmerso en el espacio 3D que modela un **cable**: dos nodos articulados que solo transmiten **fuerza axial de tensión**. En compresión o estado sin tensar, el elemento no transmite nada. Cinemática **corotacional** (Updated Lagrangian): el eje rota libremente en el espacio, la longitud se mide sobre la configuración corriente.

La unilateralidad vive en el **material**; el elemento es la maquinaria cinemática 3D. Para obtener respuesta física de cable, emparejar con `CableMaterial1D`. Con un material elástico clásico, el elemento se comporta como una barra corotacional 3D.

1. Cinemática

- Referencia: $\mathbf{X}_1, \mathbf{X}_2 \in \mathbb{R}^3$, longitud geométrica $L = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Sin tensión: longitud L_0 del cable descargado (parámetro opcional `L0`; por defecto $L_0 = L$).
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Vector unitario del eje corriente:

$$\hat{\mathbf{e}} = \frac{\mathbf{x}_2 - \mathbf{x}_1}{l} = (c_x, c_y, c_z), \qquad c_x^2 + c_y^2 + c_z^2 = 1.$$

2. Medida de deformación

Ingenieril corotacional, respecto de la longitud sin tensión:

$$\varepsilon = \frac{l - L_0}{L_0}.$$

Pretensión (2026-10-01): igual que en `Cable2DCorot`. Con $L_0 < L$ el cable arranca con $\varepsilon_0 = (L - L_0)/L_0$ y, con material elástico en tracción, $N_0 = E\,A\,\varepsilon_0$; con $L_0 > L$ arranca flojo; con $L_0 = L$ (por defecto) nada cambia.

3. Ecuación constitutiva

Delegada al material con `STRAIN_DIM = 1`. El elemento llama `material.compute_state(`ε`, state)` y recibe $(\sigma, E_t,$ nuevo estado$)$ sin inspeccionar su naturaleza. La unilateralidad (si la hay) es responsabilidad del material.

4. Equilibrio — forma débil incremental

PTV estándar; el elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^\top \, \mathbf{F}_{\text{int}}, \qquad \mathbf{K}_T = \frac{\partial \mathbf{F}_{\text{int}}}{\partial \mathbf{u}_e} = \mathbf{K}_M + \mathbf{K}_G.$$

El vector nodal equivalente de una fuerza de cuerpo $\mathbf{b}$ es $\mathbf{b}\, A\, L_0/2$ por nodo (`compute_body_load`): $A\, L_0$ es el volumen de material del cable descargado.

2.2.2 Formulación numérica (FEM)

6. Discretización

Dos nodos, 3 DOFs por nodo (u_x, u_y, u_z). Vector elemental de 6 componentes:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}]^\top.$$

7. Vector dirección actual

$$\mathbf{d} = [-c_x,\ -c_y,\ -c_z,\ c_x,\ c_y,\ c_z]^\top, \qquad \|\mathbf{d}\|^2 = 2,$$

con los cosenos evaluados en configuración **corriente**.

8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0}\, \mathbf{d}^\top, \qquad \varepsilon = \mathbf{B}\, \mathbf{u}_e.$$

9. Rigidez tangente

Material:

$$\mathbf{K}_M = \frac{E_t\, A}{L_0}\, \mathbf{d}\, \mathbf{d}^\top \quad (6 \times 6,\ \text{rango } 1).$$

Geométrica: en 3D la dirección perpendicular al eje es un plano bidimensional. Se usa el proyector perpendicular $\mathbf{P}_3 = \mathbf{I}_3 - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ $(3 \times 3$, rango 2):

$$\mathbf{K}_G = \frac{N}{l} \begin{bmatrix} \mathbf{P}_3 & -\mathbf{P}_3 \\ -\mathbf{P}_3 & \mathbf{P}_3 \end{bmatrix} \quad (6 \times 6,\ \text{rango } 2), \qquad N = \sigma A.$$

Caso destensado (material unilateral con $\varepsilon \le 0$): el material devuelve $\sigma = 0$, $E_t = 0$, luego $N = 0$ y **ambas contribuciones se anulan** — $\mathbf{K}_T = \mathbf{0}$. El elemento aporta cero al sistema global.

Caso pretensado ($L_0 < L$): en la configuración de referencia $N = N_0 > 0$ y $\mathbf{K}_G$ aporta la rigidez N_0/L en las dos direcciones perpendiculares al eje desde el primer paso.

10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N\, \mathbf{d}.$$

11. Cuadratura

No aplica (forma cerrada).

2.2.3 Contrato de implementación

```yaml
name: Cable3DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_borst_crisfield: "§3.1, cinemática de la armadura corrotacional; la
        unilateralidad la aporta CableMaterial1D"
  general_codes:
    - ansys: "LINK180 con SECCONTROL de sólo tracción"
    - abaqus: "T3D2 + *NO COMPRESSION, NLGEOM"
    - opensees: "corotTruss (ndm = 3) + uniaxialMaterial Elastic con Eneg = 0"

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal del
      cable" }
  - { name: L0, type: float, required: false, desc: "Longitud sin tensión (> 0). Por
      defecto, la longitud geométrica entre nodos; menor ⇒ cable pretensado, mayor ⇒
      cable flojo" }

material_contract:
  signature: "compute_stateε(, state) -> σ(, E_tangent, state')"
  strain_kind: axial escalar (deformación ingenieril corotacional)
  expected_behaviour: "material unilateral (p. ej. CableMaterial1D) para obtener
      respuesta física de cable; cualquier material con STRAIN_DIM=1 es técnicamente
      aceptado pero producirá comportamiento de barra"

conventions:
  sign: "σ > 0 ⇔ε  > 0 (elongación); la responsabilidad de anular σ en compresión es
      del material"
  voigt: "N/A (escalar)"
  node_orientation: "libre; K_T y F_int invariantes bajo permutación"
  configuration: "Updated Lagrangian - l, c_x, c_y, c_z se recalculan en cada evaluació
      n"

validity:
  - "ε|| ≲ 1e-2 en régimen tensado"
  - rotaciones y desplazamientos de cualquier magnitud en el espacio
  - cargas axiales (las transversales se transmiten por destensado-retensado del cable)

out_of_scope:
  - flexión, cortante, momentos, torsión
  - pandeo (irrelevante: el cable no transmite compresión)
```

```
  - "catenaria: el cable es recto entre nodos; el peso propio se reparte a los nodos y
     la flecha se obtiene discretizando en varios elementos"

numerical_caveats:
  - "Cable destensado ⇒ K_T = 0 en los DOFs del elemento. Si otros elementos no
     garantizan rigidez en esos DOFs, la matriz tangente global puede ser singular."
  - "En transiciones tensado ↔ destensado ε( cruza 0), Newton-Raphson puede oscilar.
     Usar arc-length o pasos finos si el problema atraviesa destensiones."

acceptance:
  - name: cable tensado coincide con barra corotacional 3D
    setup: "cable con orientación arbitraria, material lineal (Elastic1D),
       desplazamiento axial pequeño ε( ~ 1e-6)"
    expect: "K_T y F_int coinciden con Truss3DCorot a tolerancia rtol = 1e-4"
    tol_rel: 1.0e-4

  - name: cable destensado produce aportación nula
    setup: "cable sobre eje x con CableMaterial1D, u_x nodo 2 = -1e-3 ⇒ε  < 0"
    expect: "σ = 0, F_int = 0, K_T = matriz cero"
    tol_abs: 1.0e-12

  - name: invariancia bajo rotación rígida 3D
    setup: "cable inicial sobre eje x, rotación rígida de 45° en plano x-z"
    expect: "σ = 0 y F_int = 0 (independiente del material)"
    tol_abs: 1.0e-10

  - name: cruce por cero
    setup: "u_e = 0 con material unilateral"
    expect: "σ = 0, E_t = 0, F_int = 0, K_T = 0"
    tol_abs: 1.0e-12

  - name: pretensión inicial exacta en un cable oblicuo
    setup: "CableMaterial1D (E=2e11), A=3e-4, cable oblicuo en el espacio, L0 de la
       pretensión T0 = 25 kN; u_e = 0"
    expect: "N = T0, rigidez transversal ᵀv·K_T·v = T0/L para un desplazamiento
       unitario perpendicular al eje, F_int = T0·d"
    tol_rel: 1.0e-10

  - name: cuerda pretensada con carga central en 3D
    setup: "dos cables colineales sobre el eje x con extremos fijos, pretensión T0,
       carga P = 3 kN según -z en el nodo central; NonlinearSolver"
    expect: "misma solución analítica que el criterio 6 de Cable2DCorot; desplazamiento
       según y nulo"
    tol_rel: 1.0e-9

  - name: L0 inválida
    setup: "L0 ∈ {0, -1, ∞, NaN}"
    expect: "ValueError que nombra L0"

references:
  - "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
     .1, §3.3"
  - "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.5"
  - "Irvine H.M., Cable Structures, MIT Press, §1.2"
```

2.2.4 Implementación

- **Archivo**: solidum/elements/cable.py — comparte archivo con `Cable2DCorot` por convención temática del proyecto; no comparte código.
- **Clase**: `Cable3DCorot` — hereda directamente de `Element` (no de `Cable2DCorot`, no de `Truss3DCorot`, no de ningún otro elemento). Maquinaria cinemática 3D autónoma.
- **Tests**: tests/test_cable_elements.py · `TestCable3DCorot` — los 4 criterios de `acceptance` más un test de registro:
- `test_acceptance_cable_tensado_como_barra_corot_3d` (criterio 1).
- `test_acceptance_cable_destensado_aportacion_nula` (criterio 2).
- `test_acceptance_rotacion_rigida_3d` (criterio 3).
- `test_acceptance_cruce_por_cero` (criterio 4).
- `test_registro_en_registry` — autodiscover.
- `TestPretensionLongitudSinTension` (criterios compartidos con `Cable2DCorot`): `test_pretension_inicial_exacta_3d_oblicuo`, `test_cuerda_pretensada_carga_central_3d` y `test_L0_invalido_se_rechaza`.
- **Notas de traducción**:
- La clase duplica la maquinaria corotacional 3D de `Truss3DCorot`. Duplicación deliberada por la misma razón que en 2D: desacoplar la evolución futura de armaduras y cables.
- El proyector $\mathbf{P}_3 = \mathbf{I}_3 - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ se construye con `solidum.math.geometry.perpendicular_projector(ê)`, helper compartido con `Truss3DCorot` (operación geométrica pura, ortogonal al hecho de que el material sea unilateral o no).
- `_current_geometry` tolera nodos con 2 ó 3 coordenadas (completa con $z = 0$ si falta), heredando la flexibilidad de `Truss3D` para problemas embebidos.
- En estado destensado, $\mathbf{K}_M = \mathbf{0}$ y $\mathbf{K}_G = \mathbf{0}$ simultáneamente; el elemento aporta literalmente ceros al ensamblaje.
- El test del criterio 1 compara directamente contra `Truss3DCorot` con material `Elastic1D`: si ambos producen $\mathbf{K}$ y $\mathbf{F}_{\text{int}}$ iguales a tolerancia, la cinemática del cable y de la barra son consistentes entre sí en el régimen tensado.

2.2.5 Diálogo

- **2026-04-21** · Elemento creado como pieza totalmente autónoma por decisión del usuario: no hereda de `Cable2DCorot` ni de `Truss3DCorot`. Compartir archivo con `Cable2DCorot` (`solidum/elements/cable.py`) es convención temática del proyecto (elementos del mismo dominio conviven en un archivo, como las 4 armaduras y los 2 frames 2D en `structural.py`), no implica dependencia de código.

- **2026-04-21** · La única diferencia estructural con `Cable2DCorot` es el proyector 3D: en 2D la perpendicular es un vector único (**n**); en 3D es un plano (dimensión 2). Esto se refleja en el rango de $\mathbf{K}_G$: 1 en 2D, 2 en 3D.

- **2026-09-22** · Auditoría global: `internal_forces(U, equivalent_load=None)`: cuando el `Assembler` ensambló peso propio o fuerza de cuerpo, `solidum.run` pasa el vector nodal equivalente del elemento (escalado por λ_final) y las fuerzas internas de extremo son `F_int - f_eq` en ejes locales. Antes se devolvía `K·u`, que con carga distribuida son fuerzas nodales, no fuerzas internas: un voladizo bajo peso propio daba V_i = qL/2 y M_i = $-5qL^2/12$ en vez de qL y $-qL^2/2$, contradiciendo a las reacciones. Para barras y cables se proyectan las fuerzas de extremo netas sobre el eje (`axial_end_forces`): con carga axial distribuida N_i ≠ N_j.

- **2026-10-01** · **Pretensión por longitud sin tensión** `L0`, con la misma formulación y justificación que en `Cable2DCorot` (ver su diálogo). La validación del validador es compartida (`_unstressed_length` en `cable.py`). La cuerda pretensada recta destapó un falso positivo de la red de seguridad del ADR 0019: con todos los nodos sobre una recta, el giro alrededor de ella no mueve ningún DOF y se denunciaba como mecanismo. Corregido en `model_checks.py`.

- **2026-10-27** · Como `Cable2DCorot`: una fuerza de cuerpo como campo `b(x)` se rechaza con un mensaje claro (ADR 0022, fase 4).

Capítulo 3

Elementos 1D — Marcos / Vigas

3.1 Frame2DEuler

Orden de trabajo. El usuario escribe ***especificación física****,* ***formulación numérica*** *y* ***contrato****; la IA rellena* ***implementación*** *y responde en* ***diálogo****.*

3.1.1 Especificación física

0. Descripción general

Elemento 1D inmerso en el plano 2D que modela una **viga esbelta** según la teoría de **Euler-Bernoulli**. Dos nodos rígidamente conectados (transmiten momento), 3 grados de libertad por nodo (u_x, u_y, r_z). Captura tres acciones internas: **fuerza axial**, **cortante transverso** y **momento flector**. Régimen de **linealidad geométrica** (pequeños desplazamientos y rotaciones).

1. Hipótesis cinemática de Euler-Bernoulli

- Las secciones transversales se mantienen **planas** tras la deformación.
- Las secciones permanecen **perpendiculares al eje neutro deformado** (cizalladura transversal nula: $\gamma_{xy} = 0$).

Consecuencia: la rotación de la sección es igual a la pendiente de la elástica:

$$\theta(s) = \frac{dv}{ds}.$$

Válido solo para vigas **esbeltas** ($L/h \gtrsim 10$). En vigas peraltadas la cizalladura no es despreciable → `Frame2DTimoshenko`.

2. Campos de desplazamiento

- Axial: $u(s)$ a lo largo del eje local $s \in [0, L]$.
- Transverso: $v(s)$ perpendicular al eje local.
- Rotación de la sección: $\theta(s) = dv/ds$ (derivada de la flecha).

3. Medidas de deformación

- Axial: $\varepsilon_{axial}(s) = du/ds$ (constante para interpolación lineal en u).
- Curvatura: $\kappa(s) = d^2v/ds^2$.

La deformación axial en un punto genérico de la sección es $\varepsilon(s, y) = \varepsilon_{axial}(s) - y\,\kappa(s)$; al integrar sobre la sección elástica y simétrica respecto del eje se obtienen axial (N) y momento flector (M) desacoplados.

4. Ecuaciones constitutivas

Dos caminos excluyentes, elegidos al construir el elemento (ADR 0021):

- **Con `material` + `A`, `I`** (forma cerrada): $N = EA\,\varepsilon_{axial}$, $M = EI\,\kappa$. Sólo admite materiales **elásticos lineales** (`IS_LINEAR_ELASTIC = True`); un material inelástico se rechaza al construir con un mensaje que remite a la sección por fibras.
- **Con `section`** (+ `n_ip`): N y M son los de la ley de sección, que integra el esfuerzo σ_{xx} de cada fibra con su propio material 1D (FiberSection §2-§4). Plasticidad y daño distribuidos en la sección.

Hasta el 2026-09-29 el camino con material admitía materiales inelásticos y escalaba toda la rigidez local con la E_t de la deformación axial: con fluencia axial el flector se anulaba y con ablandamiento cambiaba de signo. Esa ley desapareció (ver *Diálogo*).

5. Equilibrio — forma débil

Principio de trabajos virtuales. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \int_0^L (N\,\delta\varepsilon_{axial} + M\,\delta\kappa)\,ds = \delta\mathbf{u}_e^\top\,\mathbf{F}_{\text{int}}.$$

3.1.2 Formulación numérica (FEM)

6. Discretización

Dos nodos ($s = 0$ y $s = L$), 3 DOFs por nodo en coordenadas globales:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, r_z^{(1)}, u_x^{(2)}, u_y^{(2)}, r_z^{(2)}]^\top.$$

7. Sistema local y transformación

Cosenos directores del eje (configuración inicial, fija): $c = (x_2 - x_1)/L$, $s_\theta = (y_2 - y_1)/L$. Matriz de transformación ortogonal 6×6:

$$\mathbf{T} = \begin{bmatrix} c & s_\theta & 0 & 0 & 0 & 0 \\ -s_\theta & c & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & c & s_\theta & 0 \\ 0 & 0 & 0 & -s_\theta & c & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

Las rotaciones r_z son invariantes ante rotación del sistema (escalares en el plano). $\mathbf{u}_{\text{local}} = \mathbf{T}\,\mathbf{u}_e$.

8. Funciones de forma

- Axial: lineal, $N_1^u(s) = 1 - s/L$, $N_2^u(s) = s/L$.
- Transverso (Hermite cúbicos, cuatro funciones que interpolan v y θ):

$$N_1^v = 1 - 3\xi^2 + 2\xi^3, \quad N_1^\theta = L(\xi - 2\xi^2 + \xi^3),$$
$$N_2^v = 3\xi^2 - 2\xi^3, \quad N_2^\theta = L(-\xi^2 + \xi^3),$$

con $\xi = s/L$.

9. Rigidez local

En el sistema local, la matriz 6×6 tiene forma cerrada por integración analítica:

$$\mathbf{K}_{\text{local}} = \begin{bmatrix} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & \frac{12EI}{L^3} & \frac{6EI}{L^2} & 0 & -\frac{12EI}{L^3} & \frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{4EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{2EI}{L} \\ -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -\frac{12EI}{L^3} & -\frac{6EI}{L^2} & 0 & \frac{12EI}{L^3} & -\frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{2EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{4EI}{L} \end{bmatrix}.$$

Es la forma cerrada del camino con material elástico lineal. En el camino con sección la matriz local se integra numéricamente, $\mathbf{K}_{\text{local}} = \sum_g w_g \frac{L}{2} \mathbf{B}_g^\top \mathbf{k}_{s,g} \mathbf{B}_g$ con $\mathbf{e}_g = [\varepsilon_0,\ \kappa_z(x_g)]^\top = \mathbf{B}_g\,\mathbf{u}_{\text{local}}$, las mismas funciones de forma del §8 y la tangente de la sección $\mathbf{k}_s$ (FiberSection §5); con fibras elásticas y `n_ip` ≥ 2 reproduce esta matriz exactamente.

10. Fuerzas internas

En el sistema local, camino con material:

$$\mathbf{F}_{\text{int,local}} = \mathbf{K}_{\text{local}}\,\mathbf{u}_{\text{local}}.$$

La componente axial se escribe con el esfuerzo que devuelve el material, $F_1 = -\sigma A$, $F_4 = +\sigma A$; con material elástico lineal coincide con la fila axial del producto. Camino con sección: $\mathbf{F}_{\text{int,local}} = \sum_g w_g \frac{L}{2} \mathbf{B}_g^\top \mathbf{s}_g$, con $\mathbf{s}_g = [N,\ M_z]$ de la sección en cada punto de Gauss.

11. Ensamblaje global

$$\mathbf{K}_{\text{global}} = \mathbf{T}^\top \mathbf{K}_{\text{local}}\,\mathbf{T}, \qquad \mathbf{F}_{\text{int}} = \mathbf{T}^\top \mathbf{F}_{\text{int,local}}.$$

12. Cuadratura

Camino con material: no aplica (forma cerrada; integración analítica de los polinomios de Hermite). El campo `n_integration_points: 1` se refiere al único estado material conceptual del elemento (la deformación axial media).

Camino con sección: Gauss-Legendre de `n_ip` puntos a lo largo del elemento (3 por defecto, mínimo 2). El elemento fija `N_INTEGRATION_POINTS = n_ip` como atributo de instancia; el atributo de clase sigue en 1.

13. Matriz de masa

Sin inercia rotatoria de la sección. La hipótesis de Euler-Bernoulli ($\theta = dv/ds$, §1) la desprecia: la energía cinética es la de la traslación del eje, $T = \frac{1}{2}\int_0^L \rho A\,(\dot{u}^2 + \dot{v}^2)\,ds$. La masa depende sólo de la densidad lineal ρA ($\rho \cdot A$ con material; $\sum_i \rho_i A_i$ con sección, FiberSection §6), no de I.

Consistente (`lumping="consistent"`, por defecto), $\int_0^L \rho A\,\mathbf{N}^\top\mathbf{N}\,ds$ con las funciones de forma del §8:

$$\mathbf{M}_{\text{axial}} = \frac{\rho AL}{6}\begin{bmatrix}2 & 1\\ 1 & 2\end{bmatrix} \text{ en } (u_1, u_2), \qquad \mathbf{M}_{\text{flex}} = \frac{\rho AL}{420}\begin{bmatrix}156 & 22L & 54 & -13L\\ 22L & 4L^2 & 13L & -3L^2\\ 54 & 13L & 156 & -22L\\ -13L & -3L^2 & -22L & 4L^2\end{bmatrix} \text{ en } (v_1, \theta_1, v_2, \theta_2).$$

Reproduce exactamente la energía cinética de los movimientos rígidos, que la interpolación contiene: $\rho AL\,|\mathbf{v}|^2/2$ en una traslación y $\frac{1}{2}\rho A\,\frac{L^3}{3}\,\omega^2$ en un giro alrededor de un extremo. Con ella las frecuencias discretas son cotas superiores de las de Euler-Bernoulli (Rayleigh-Ritz conforme).

Concentrada (`lumping="lumped"`), HRZ (Hinton, Rock y Zienkiewicz 1976): la diagonal consistente escalada para que los términos traslacionales de cada dirección sumen ρAL. En la flexión la diagonal es $\frac{\rho AL}{420}(156, 4L^2, 156, 4L^2)$ y el factor 420/312:

$$m_u = m_v = \frac{\rho AL}{2}, \qquad m_\theta = \frac{\rho AL^3}{78} \quad \text{por nodo.}$$

El bloque traslacional por nodo es $m\,\mathbf{I}_2$, invariante ante la rotación: la matriz es diagonal también en globales, requisito de `CentralDifferenceSolver`.

En globales, $\mathbf{M}_{\text{global}} = \mathbf{T}^\top\,\mathbf{M}_{\text{local}}\,\mathbf{T}$.

Hasta el 2026-09-29 la consistente sumaba $\frac{\rho IL}{6}\left[\begin{smallmatrix}2 & 1\\ 1 & 2\end{smallmatrix}\right]$ en (θ_1, θ_2) y la concentrada usaba $\rho IL/2$ por giro: inercia rotatoria de la cinemática de Timoshenko, no de Euler-Bernoulli (ver *Diálogo*).

3.1.3 Contrato de implementación

```yaml
name: Frame2DEuler
kind: element
status: validated

interface:
  dof_names: [ux, uy, rz]
  n_nodes: 2
  strain_dim: 1
```

```
  n_integration_points: 1   # camino con material; con section, N_INTEGRATION_POINTS =
      n_ip (atributo de instancia)

admission:                  # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - zienkiewicz_taylor: "vol. 2 (6.ª ed.), cap. 10: barras de Euler-Bernoulli (
        formulación base, material elástico)"
    - bathe: "cap. 4-5 (elementos de viga)"
  general_codes:
    - opensees: "elasticBeamColumn (2D)"
    - abaqus: "B23"

parameters:
  - { name: A, type: float, required: false, desc: "Área de la sección transversal.
      Obligatoria con material; no se declara con section" }
  - { name: I, type: float, required: false, desc: "Momento de inercia respecto al eje
      perpendicular al plano (z). Obligatorio con material; no se declara con section"
      }
  - { name: section, type: "id de sections", required: false, desc: "Sección de marco (
      FiberSection). Excluyente con material, A e I (ADR 0021)" }
  - { name: n_ip, type: int, required: false, default: 3, desc: "Puntos de Gauss-
      Legendre a lo largo del elemento, ≥ 2. Sólo con section" }

material_contract:
  signature: "compute_stateε(, state) -> σ(, E_tangent, state')"
  strain_kind: "axial escalar (deformación media axial ε = (u_2 - u_1)/L en sistema
      local)"
  nonlinearity_model: "camino con material: sólo materiales elásticos lineales (
      IS_LINEAR_ELASTIC = True; un material inelástico se rechaza al construir). La no
      linealidad material entra por section (FiberSection): cada fibra evalúa su
      material 1D con ε = ε₀−  y·κ_z"

conventions:
  sign: "σ > 0 ⇔ε  > 0 (elongación) en el eje axial"
  rotation_sign: "r_z positivo antihorario (convención matemática estándar)"
  node_orientation: "el eje local va del nodo 1 al nodo 2; permutar nodos invierte el
      signo de las fuerzas internas"
  configuration: "inicial fija - L, c, s, T no se actualizan con los desplazamientos"

validity:
  - "vigas esbeltas: L/h ≳ 10 (h = canto de la sección)"
  - pequeños desplazamientos y pequeñas rotaciones
  - "ε|_axial| ≲ 1e-2"

out_of_scope:
  - vigas peraltadas con deformación por cortante significativa (usar Frame2DTimoshenko
      )
  - grandes rotaciones o desplazamientos (usar Frame2DEulerCorot)
  - material inelástico con material + A, I (se rechaza al construir; la plasticidad y
      el daño entran por section, FiberSection)
  - pandeo (requiere rigidez geométrica, no implementada en esta viga lineal)
  - "cargas puntuales en el vano (una fuerza de cuerpo variable b(x) sí, desde el
      2026-10-27, ADR 0022)"

acceptance:
  - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
```

```
    setup: "línea recta inclinada; solución exacta impuesta en un extremo y sus fuerzas
        internas como cargas nodales en el otro; carga de línea como fuerza de cuerpo
       b(x); u* axial y v* transversal; n = 4, 8, 16, 32"
    expect: "con campos polinómicos (carga de grado ≤ 4), error nodal de redondeo con 1
        y 3 elementos (exactitud nodal, Tong 1969); con campos suaves, orden ≥ teórico
        - 0.1 entre los dos últimos niveles por componente: u 2, v 4, θ 3 en L²; ε 1,
       κ 2 en energía; error decreciente"
    tol_abs: 0.1
  - name: respuesta axial pura
    setup: "viga horizontal, empotrada en extremo izquierdo, carga axial F en extremo
       derecho"
    expect: "u_x (extremo libre) = F·L/(E·A); momentos y cortantes nulos"
    tol_rel: 1.0e-10

  - name: voladizo con carga transversal (flecha analítica)
    setup: "viga horizontal, empotrada en extremo izquierdo, carga vertical P en
       extremo derecho"
    expect: "v (extremo libre) = P·L³/(3·E·I); rotación θ = P·L²/(2·E·I)"
    tol_rel: 1.0e-10

  - name: simetría de K
    setup: "cualquier configuración"
    expect: "K_global = ᵀK_global (elemento conservativo en régimen elástico)"
    tol_abs: 1.0e-12

  - name: guarda de material elástico lineal            # añadido 2026-09-29 (ADR 0021)
    setup: "material Elastoplastic1D o IsotropicDamage1D con A e I"
    expect: "ValueError al construir, con un mensaje que nombra IS_LINEAR_ELASTIC y
       remite a FiberSection con 'section'"

  - name: camino con sección                            # añadido 2026-09-29 (ADR 0021)
    setup: "section = FiberSection; casos de acceptance de docs/specs/FiberSection.md"
    expect: "reproducción elástica con n_ip = 2, -momentocurvatura, interacción -NM,
       colapso del voladizo y de la viga biempotrada, signos"

  - name: matrices de masa consistente y concentrada   # añadido 2026-09-29
    setup: "elemento oblicuo, caminos con material y con sección ρ(A = Σρ_i A_i);
       M_local = T·M·ᵀT"
    expect: "consistente = axial ρAL/6·[[2,1],[1,2]] + hermítica ρAL/420·H, sin inercia
        rotatoria (independiente de I); concentrada HRZ = diagρ(AL/2, ρAL/2, ρAL³/78)
       por nodo, diagonal también en globales"
    tol_rel: 1.0e-12

  - name: energía cinética de movimientos rígidos        # añadido 2026-09-29
    setup: "traslación v y giro ω alrededor de un extremo"
    expect: "½ᵀuMu = ρAL|v|²/2 (consistente y concentrada) y ½·ρA·L³/3·ω² (consistente)
       "
    tol_rel: 1.0e-12

references:
  - "Bathe K.-J., Finite Element Procedures, §4.2 (formulación isoparamétrica) y cap. 5
      (vigas)"
  - "Cook R.D., Concepts and Applications of FEA, §2.7 (elementos de viga)"
  - "Hinton E., Rock T., Zienkiewicz O.C. (1976). A note on mass lumping and related
      processes in the finite element method. Earthquake Eng. Struct. Dyn. 4, 245-249 (
      masa concentrada HRZ)"
```

3.1.4 Implementación

- **Archivo**: solidum/elements/frame/euler.py — submódulo del paquete `solidum/elements/frame/` que aloja también `Frame2DTimoshenko` y `Frame2DEulerCorot`.
- **Clase**: `Frame2DEuler(FrameSectionMixin, Element)`. La construcción de la matriz de transformación $\mathbf{T}$ se delega a `build_geometry_2d` en solidum/elements/frame/_shared.py, compartida con `Frame2DTimoshenko` (la versión corotacional reconstruye $\mathbf{T}$ desde `alpha0` por su lógica propia). La constitutiva de los dos caminos (validación de argumentos, guarda de material elástico lineal, estado por punto de Gauss, integración de la sección) vive en `FrameSectionMixin`, solidum/elements/frame/_section.py, compartida con `Frame2DEulerCorot` y `Frame3D` (ADR 0021). `REFERENCE_KWARGS = {"material": MaterialRegistry, "section": SectionRegistry}`.
- **Tests**: `tests/test_line_elements.py` (campo interior, energía igual a la de la rigidez, carga como campo) y `tests/validation/test_mms_frames.py` (soluciones manufacturadas); tests/test_frame.py · `TestFrame2DEulerAcceptance` — los tres criterios físicos y el registro:
- `test_acceptance_respuesta_axial_pura` (criterio 1) — resuelve el voladizo con solver, verifica $u_x(L) = FL/(EA)$.
- `test_acceptance_voladizo_carga_transversal` (criterio 2) — carga transversal P, verifica flecha $v = PL^3/(3EI)$ y rotación $\theta = PL^2/(2EI)$.
- `test_acceptance_simetria_K` (criterio 3) — viga oblicua (c=0.6, s=0.8) para evitar ejes canónicos, verifica $\mathbf{K} = \mathbf{K}^\top$.
- `test_registro_en_registry` — autodiscover.
- Criterio 4 (guarda): tests/test_fiber_section.py · `TestGuardas` (`test_material_inelastico_remite_a_la_seccion`, `test_exactamente_un_camino`, `test_n_ip_invalido`, `test_n_ip_solo_con_seccion`).
- Criterio 5 (camino con sección): tests/validation/test_fiber_section_validation.py (casos 1-5 y 7 de la spec FiberSection) y `TestMarcosConSeccion`, `TestTangente`, `TestMasa`, `TestPesoPropio`, `TestYaml` de `tests/test_fiber_section.py`.
- Criterios 6 y 7 (masa): tests/test_frame_mass.py · `TestMarco2DEulerBernoulli` (consistente, HRZ, independencia de I), `TestCaminoConSeccion`, `test_traslacion_rigida_2d`, `test_giro_rigido_2d_consistente` y `test_frecuencias_biapoyada_cota_superior_de_euler_bernoulli`; la hermítica se contrasta además con la integral $\int \rho A\, \mathbf{N}^\top \mathbf{N}$ (`test_hermitica_es_la_integral_de_las_funciones_de_hermite`). Diagonalidad y masa total de la concentrada en tests/test_mass_lumping.py · `TestLumpedFrame2D`.
- **Notas de traducción**:
- La matriz $\mathbf{T}$ se calcula una sola vez en `__init__` sobre la configuración inicial y se guarda como atributo — coherente con el régimen geométricamente lineal.
- Camino con material: la componente axial de $\mathbf{F}_{\text{int,local}}$ se escribe con el σA que devuelve el material; con material elástico lineal (el único admitido) coincide con la fila axial de $\mathbf{K}_{\text{local}}\mathbf{u}_{\text{local}}$.

- Camino con sección: $\mathbf{B}_g$ (2×6) se precalcula al construir (`_build_section_B`) con $N_1'' \dots N_4''$ de `hermite_second_derivatives`; los pesos ya llevan la longitud ($w_g L$ con la regla sobre $(0, 1)$). `internal_forces` y `compute_internal_forces` evalúan sobre una instantánea del trial y la restauran; el diccionario legado añade `section_deformations` y `section_forces (n_ip, 2)`. Masa con ρA de la sección; peso propio con ρA y fuerza de cuerpo con el área bruta $\sum A_i$.

- Masa (§13): `_frame2d_consistent_mass_local` y `_frame2d_lumped_mass_local` de `_shared.py`, que reciben la densidad lineal ρA (`_linear_density`). Los bloques de la flexión (`hermite_consistent_mass` y la masa rotacional HRZ `hrz_rotational_mass`) los comparten los tres marcos Euler-Bernoulli; el argumento opcional `rotary_inertia` = ρI sólo lo pasa `Frame2DTimoshenko`.

3.1.5 Diálogo

- **2026-04-21** · Elemento movido a archivo propio `solidum/elements/frame.py` y desacoplado del helper compartido `_frame_geometry`. El método estático `_build_geometry` reimplementa la construcción de **T** dentro de la clase. `Frame2DTimoshenko` sigue en `structural.py` usando su propio acceso al helper compartido; si mañana también se independiza, duplicará o internalizará su propia versión.

- **2026-04-21** · Tests de aceptación cubren la flecha analítica del voladizo a 8 decimales ($v = PL^3/(3EI)$) usando el solver completo del proyecto — validan no solo la cinemática del elemento sino también su integración con ensamblador y solver.

- **2026-05-13** · `frame.py` se parte en paquete `solidum/elements/frame/{euler,timoshenko,euler_corot,_shared}.py`. La duplicación de `_build_geometry` entre Euler y Timoshenko se elimina: la construcción de **T** vuelve a vivir en un único lugar (`build_geometry_2d` en `_shared.py`), pero esta vez sin función helper externa flotante — pertenece al paquete `frame/`. Frame2DEulerCorot mantiene su propia reconstrucción de **T** desde `alpha0` porque su cinemática corotacional no se beneficia del helper común.

- **2026-09-22** · Auditoría global: `internal_forces(U, equivalent_load=None)`: cuando el `Assembler` ensambló peso propio o fuerza de cuerpo, `solidum.run` pasa el vector nodal equivalente del elemento (escalado por λ_final) y las fuerzas internas de extremo son `F_int - f_eq` en ejes locales. Antes se devolvía `K·u`, que con carga distribuida son fuerzas nodales, no fuerzas internas: un voladizo bajo peso propio daba V_i = qL/2 y M_i = -5qL^2/12 en vez de qL y $-$qL^2/2, contradiciendo a las reacciones.

- **2026-09-29** · Sección por fibras, etapa 1 (ADR 0021, FiberSection). La auditoría de pureza verificó que el modelo no lineal anterior —evaluar el material con la deformación axial y escalar **toda** la matriz local con su E_t— daba flectores incorrectos en silencio: nulos con fluencia axial, de signo cambiado con ablandamiento. Desaparece. El elemento tiene ahora dos caminos excluyentes: `material` + `A`, `I`, la forma cerrada de siempre, sin cambio de resultados, que sólo admite materiales con `IS_LINEAR_ELASTIC = True` (un material inelástico se rechaza al construir y el mensaje remite a la sección por fibras); y `section` + `n_ip` (3 por defecto, mínimo 2), elemento en desplazamientos que integra la sección en Gauss-Legendre. Contrato: parámetros `section` y `n_ip`; `A` e `I` pasan a ser obligatorios sólo con material; criterios de

aceptación 4 (guarda) y 5 (camino con sección). Un modelo que antes corría con material inelástico ahora falla al construir: antes daba resultados incorrectos.

- **2026-09-29** · Masa sin inercia rotatoria (hallazgo clas-4 de la auditoría; **cambio físico aprobado por el usuario**). La consistente sumaba $\frac{\rho I L}{6}\left[\begin{smallmatrix}2&1\\1&2\end{smallmatrix}\right]$ en (θ_1, θ_2): la energía cinética $\frac{1}{2}\rho I\dot{\theta}^2$ de un giro de sección interpolado linealmente e independiente de la flecha, que es cinemática de Timoshenko; la hipótesis de Euler-Bernoulli desprecia la inercia rotatoria. Se retira, por los dos caminos: la consistente es la clásica (axial lineal + hermítica) y la concentrada pasa de $\rho I L/2$ por giro (su justificación por OpenSees/SAP2000 no se sostenía) a la HRZ, $\rho A L/2$ traslacional y $m_\theta = \rho A L^3/78$ (§13). Consecuencias: la masa ya no depende de I; las frecuencias de marcos esbeltos cambian poco, porque la inercia rotatoria escala con $(r/L)^2$, y con la consistente vuelven a ser cotas superiores de las de Euler-Bernoulli. Contrato: §13 y criterios 6 y 7. `Frame2DTimoshenko` conserva su masa con inercia rotatoria.

- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022, fase 4). `compute_body_load` acepta un campo `b(x)`: la carga `A·b(x)` se proyecta a ejes locales en 4 puntos de Gauss y se reparte con las funciones de la rigidez (axial lineal, flexión de Hermite); con `b` constante, la forma cerrada de siempre. Declara `REFERENCE_CELL = "line"` y expone su campo interior (`line_field`: `u`, `v`, θ`z`, ε, κ`z`) con `section_rigidities` (`EA`, `EI`), cuya energía coincide con la de la rigidez. Nueva aceptación «soluciones manufacturadas»: exactitud nodal y órdenes 2 (u), 4 (v), 3 (θ), 1 (ε) y 2 (κ).

3.2 Frame2DEulerCorot

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.2.1 Especificación física

0. Descripción general

Viga 2D esbelta según Euler-Bernoulli en formulación **corotacional** (Updated Lagrangian). Dos nodos rígidamente conectados, 3 DOFs por nodo (u_x, u_y, r_z). Captura grandes desplazamientos y grandes rotaciones del elemento como un todo con **pequeñas deformaciones axiales** ($|\varepsilon_{axial}| \lesssim 1\,\%$) y rotaciones nodales deformacionales moderadas.

Abre dominios que la viga lineal no captura: **pandeo por flexión**, **post-pandeo**, **snap-through** de arcos planos, brazos flexibles esbeltos.

1. Cinemática corotacional

Se separa el movimiento del elemento en:

1. **Rotación rígida** α_e: el ángulo que el eje de la barra ha girado desde la configuración inicial.
2. **Rotación deformacional de cada sección** $\bar{\theta}_i$: la rotación nodal menos la rotación rígida (es lo que genera momento flector).

Sean:

$$\alpha_0 = \arctan 2(Y_2 - Y_1,\ X_2 - X_1) \quad \text{(ángulo del eje en config. inicial, fijo)},$$

$$\alpha = \arctan 2(y_2 - y_1,\ x_2 - x_1) \quad \text{(ángulo corriente)},$$

$$\alpha_e = \alpha - \alpha_0 \quad \text{(rotación rígida del elemento)}.$$

Rotaciones deformacionales:

$$\bar{\theta}_1 = \theta_1 - \alpha_e, \qquad \bar{\theta}_2 = \theta_2 - \alpha_e,$$

donde $\theta_i = r_z^{(i)}$ son las rotaciones totales de los nodos.

2. Medidas de deformación

- Axial: $\varepsilon_{axial} = (l - L_0)/L_0$ con l longitud corriente.
- Curvatura efectiva: $\kappa = (\bar{\theta}_2 - \bar{\theta}_1)/L_0$ (constante si se usa interpolación cúbica).

3. Ecuaciones constitutivas

La ley del elemento vive en el sistema básico $[\Delta l,\ \bar\theta_1,\ \bar\theta_2]$ con fuerzas básicas $[N,\ M_1,\ M_2]$. Dos caminos excluyentes, elegidos al construir (ADR 0021):

- **Con `material` + `A`, `I`** (forma cerrada): $N = EA\,\varepsilon_{axial}$ y $M = EI\,\kappa$. Sólo materiales **elásticos lineales** (`IS_LINEAR_ELASTIC = True`); un material inelástico se rechaza al construir con un mensaje que remite a la sección por fibras.
- **Con `section`** (+ `n_ip`): la ley de sección (FiberSection) se integra en los puntos de Gauss del elemento en el sistema básico corrotado (§6).

Hasta el 2026-09-29 el camino con material admitía materiales inelásticos y escalaba también la rigidez de flexión con la E_t axial, como `Frame2DEuler`; esa ley desapareció (ver *Diálogo*).

4. Equilibrio — forma débil

PTV en configuración corriente. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \delta\mathbf{u}_e^{\top}\,\mathbf{F}_{\text{int}}, \qquad \mathbf{K}_T = \mathbf{K}_{\text{material}} + \mathbf{K}_\sigma.$$

3.2.2 Formulación numérica (FEM)

5. DOFs locales deformacionales

Se reduce la descripción del estado deformacional del elemento a **3 magnitudes escalares**:

$$\mathbf{q}_l = [\,u_l,\ \bar\theta_1,\ \bar\theta_2\,]^{\top}, \qquad u_l = l - L_0.$$

6. Rigidez local

En el sistema de coordenadas corrotado (eje de la barra corriente), camino con material elástico lineal, la rigidez local 3×3:

$$\mathbf{K}_{ll} = \begin{bmatrix} \frac{EA}{L_0} & 0 & 0 \\ 0 & \frac{4EI}{L_0} & \frac{2EI}{L_0} \\ 0 & \frac{2EI}{L_0} & \frac{4EI}{L_0} \end{bmatrix}.$$

Fuerzas locales $\mathbf{F}_l = \mathbf{K}_{ll}\,\mathbf{q}_l = [N,\ M_1,\ M_2]^{\top}$, con $N = \sigma A$ del material (con material elástico lineal coincide con la fila axial del producto).

Camino con sección: en cada punto de Gauss $x_g = \xi_g L_0$,

$$\mathbf{e}_g = \begin{bmatrix} \varepsilon_0 \\ \kappa_z \end{bmatrix} = \mathbf{B}_g\,\mathbf{q}_l, \qquad \mathbf{B}_g = \frac{1}{L_0}\begin{bmatrix} 1 & 0 & 0 \\ 0 & 6\xi_g - 4 & 6\xi_g - 2 \end{bmatrix},$$

y $\mathbf{F}_l = \sum_g w_g L_0\,\mathbf{B}_g^{\top}\,\mathbf{s}_g$, $\mathbf{K}_{ll} = \sum_g w_g L_0\,\mathbf{B}_g^{\top}\,\mathbf{k}_{s,g}\,\mathbf{B}_g$ (pesos de Gauss-Legendre sobre $(0,1)$). Con fibras elásticas y `n_ip` ≥ 2 reproduce la matriz anterior exactamente. La cinemática corrotacional de §7-§8, incluida $\mathbf{K}_\sigma$ con N, M_1 y M_2, es la misma en los dos caminos.

7. Matriz de transformación B (3×6)

Relación entre variaciones de $\mathbf{q}_l$ y de los DOFs globales $\delta\mathbf{u}_e$:

$$\mathbf{B} = \begin{bmatrix} -c & -s & 0 & c & s & 0 \\ -s/l & c/l & 1 & s/l & -c/l & 0 \\ -s/l & c/l & 0 & s/l & -c/l & 1 \end{bmatrix}, \qquad c = \cos\alpha,\ s = \sin\alpha.$$

c, s y l se evalúan en **configuración corriente** — esta es la esencia corotacional.
Fuerzas internas globales:

$$\mathbf{F}_{\text{int}} = \mathbf{B}^\top \mathbf{F}_l.$$

8. Rigidez tangente

Se descompone en contribución material más geométrica:

$$\mathbf{K}_T = \mathbf{K}_{\text{material}} + \mathbf{K}_\sigma,$$

$$\mathbf{K}_{\text{material}} = \mathbf{B}^\top \mathbf{K}_{ll}\, \mathbf{B},$$

$$\mathbf{K}_\sigma = \frac{N}{l}\, \mathbf{z}\,\mathbf{z}^\top + \frac{M_1 + M_2}{l^2} \left(\mathbf{r}\,\mathbf{z}^\top + \mathbf{z}\,\mathbf{r}^\top\right),$$

con vectores geométricos de 6 componentes:

$$\mathbf{r} = [-c,\ -s,\ 0,\ c,\ s,\ 0]^\top \quad \text{(dirección axial)},$$

$$\mathbf{z} = [-s,\ c,\ 0,\ s,\ -c,\ 0]^\top \quad \text{(perpendicular)}.$$

La parte $(N/l)\mathbf{z}\mathbf{z}^\top$ es análoga al $\mathbf{K}_G$ de `Truss2DCorot`: captura la rigidización por tensión (y el ablandamiento precrítico por compresión, base del pandeo). La parte con momentos acopla rotaciones con traslaciones — crítica para problemas de flexión con desplazamientos significativos.

9. Cuadratura

Camino con material: no aplica (forma cerrada con Hermite cúbicos). Camino con sección: Gauss-Legendre de `n_ip` puntos (3 por defecto, mínimo 2), con `N_INTEGRATION_POINTS = n_ip` como atributo de instancia.

10. Matriz de masa

La de Frame2DEuler §13, evaluada en la configuración de referencia (L_0, α_0; Lagrangeano total): **sin inercia rotatoria** de la sección (Euler-Bernoulli), consistente $\frac{\rho A L_0}{6}\left[\begin{smallmatrix}2&1\\1&2\end{smallmatrix}\right]$ axial más la hermítica $\frac{\rho A L_0}{420}\mathbf{H}$ en $(v_1, \theta_1, v_2, \theta_2)$, y concentrada HRZ $\rho A L_0/2$ traslacional y $\rho A L_0^3/78$ rotacional por nodo. Depende sólo de ρA ($\sum_i \rho_i A_i$ con sección). Hasta el 2026-09-29 llevaba además la inercia rotatoria ρI (ver *Diálogo*).

3.2.3 Contrato de implementación

```yaml
name: Frame2DEulerCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1   # camino con material; con section, N_INTEGRATION_POINTS =
      n_ip (atributo de instancia)

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_borst_crisfield: "§9.3, elementos corrotacionales (formulación base, material
        elástico; 1.ª ed.: Crisfield 1991, §7.3)"
  general_codes:
    - opensees: "elasticBeamColumn + geomTransf Corotational"

parameters:
  - { name: A, type: float, required: false, desc: "Área de la sección transversal.
      Obligatoria con material; no se declara con section" }
  - { name: I, type: float, required: false, desc: "Momento de inercia respecto al eje
      perpendicular al plano. Obligatorio con material; no se declara con section" }
  - { name: section, type: "id de sections", required: false, desc: "Sección de marco (
      FiberSection), evaluada en el sistema básico corrotado. Excluyente con material,
      A e I (ADR 0021)" }
  - { name: n_ip, type: int, required: false, default: 3, desc: "Puntos de Gauss-
      Legendre a lo largo del elemento, ≥ 2. Sólo con section" }

material_contract:
  signature: "compute_stateε(, state) -> σ(, E_tangent, state')"
  strain_kind: "axial escalar corotacional ε( = (l −₀ L)/₀L)"
  nonlinearity_model: "camino con material: sólo materiales elásticos lineales (
      IS_LINEAR_ELASTIC = True; un material inelástico se rechaza al construir). La no
      linealidad material entra por section (FiberSection) en el sistema básico; el σK_
       geométrico depende sólo de N, ₁M, ₂M corrientes en los dos caminos"

conventions:
  sign: "σ > 0 ⇔ε  > 0 (elongación); r_z positivo antihorario"
  node_orientation: "eje local del nodo 1 al nodo 2"
  configuration: "Updated Lagrangian - l, c, s, α se recalculan en cada evaluación"

validity:
  - "ε|_axial| ≲ 1e-2 (pequeña deformación axial)"
  - rotaciones rígidas del elemento de cualquier magnitud, pero acumuladas en α|_e| <
      π entre commits (rotaciones continuas > π requerirían tracking de vueltas - fuera
       de alcance)
  - rotaciones deformacionales de las secciones moderadas θ(|_i| ≲ 30°); para valores
      mayores la interpolación Hermite cúbica pierde precisión
  - vigas esbeltas (L/h ≳ 10)

out_of_scope:
  - rotaciones continuas del eje > π sin pasar por commit (pendulums; aliasing en atan2
      )
```

```
  - deformaciones axiales grandes (requiere medida logarítmica)
  - material inelástico con material + A, I (se rechaza al construir; la plasticidad y
      el daño entran por section, FiberSection)
  - cortante transverso significativo (usar versión Timoshenko, pendiente)
  - "fuerzas de cuerpo sobre la geometría corriente (se reparten en la de referencia,
      constantes o como campo b(x))"

numerical_caveats:
  - "El ángulo α_e se unwrappea a π(-, π] por llamada. Si el problema tiene saltos de
      rotación > π entre iteraciones consecutivas del solver, la formulación produce
      resultados incorrectos. Usar pasos de carga finos."
  - "En problemas con pandeo, la rigidez tangente se hace singular al cruzar el punto
      crítico. Usar `ArcLengthSolver` para seguir la rama post-crítica."

acceptance:
  - name: límite lineal coincide con Frame2DEuler
    setup: "voladizo horizontal, cargas pequeñas ε( ~ 1e-8, v/L ~ 1e-6)"
    expect: "K_T y F_int coinciden con Frame2DEuler lineal a tolerancia rtol=1e-4"
    tol_rel: 1.0e-4

  - name: invariancia bajo rotación rígida
    setup: "viga inicialmente horizontal, rotada rígidamente 45° (ambos nodos)"
    expect: "F_int = 0, σ = 0 (ningún esfuerzo interno bajo rotación rígida pura)"
    tol_abs: 1.0e-10

  - name: coherencia tangente/fuerza interna (chequeo por diferencias finitas)
    setup: "configuración arbitraria no trivial; K_T analítica vs derivada numérica de
        F_int"
    expect: "‖K_T_analítica - K_T_numé‖rica / ‖K_T_analí‖tica ≤ 1e-5"
    tol_rel: 1.0e-5

  - name: simetría de K
    setup: "configuración arbitraria"
    expect: "K_T = ᵀK_T"
    tol_abs: 1.0e-6

  - name: Bathe-Bolourchi - cuarto de círculo                  # añadido 2026-05-19
    setup: "voladizo recto, 10 elementos, momento puro M = π(/2)·EI/L en la punta, 8
        pasos"
    expect: "tip se mueve a (R·sinπ(/2)-L, R·(1-cosπ(/2))) con R=EI/M; θ_tip = π/2"
    tol_abs: 0.005   # sobre L (cuerda ~0.2% en 10 elementos rectos)

  - name: Bathe-Bolourchi - medio círculo                      # añadido 2026-05-19
    setup: "voladizo recto, 20 elementos, momento puro M = π·EI/L en la punta, 20 pasos
        "
    expect: "tip a (-L, 2Lπ/) y θ_tip = π-  la viga se enrosca en semicírculo cerrado"
    tol_abs: 0.01    # sobre L

  - name: guarda de material elástico lineal                   # añadido 2026-09-29 (ADR
      0021)
    setup: "material Elastoplastic1D o IsotropicDamage1D con A e I"
    expect: "ValueError al construir, con un mensaje que nombra IS_LINEAR_ELASTIC y
        remite a FiberSection con 'section'"

  - name: camino con sección, giros grandes                    # añadido 2026-09-29 (ADR
      0021)
```

```
    setup: "section = FiberSection elástica frente a material + A, I integrados; giros
        de sólido rígido grandes y arco de Bathe-Bolourchi (caso 8 de FiberSection)"
    expect: "K_T y F_int iguales; misma solución del arco"
    tol_rel: 1.0e-10

  - name: matrices de masa                                    # añadido 2026-09-29
    setup: "elemento oblicuo, caminos con material y con sección"
    expect: "las de Frame2DEuler §13 en la configuración de referencia: consistente
        axial + hermítica sin inercia rotatoria, concentrada HRZ diagρ(₀AL/2, ρ₀AL/2,
        ρ₀AL³/78); ½ᵀuMu exacta en traslación y, con la consistente, en giro rígido
        alrededor de un extremo"
    tol_rel: 1.0e-12

references:
  - "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
      .1, §7.3 (corotational beams)"
  - "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.11"
  - "Wriggers P., Nonlinear Finite Element Methods, cap. 4"
```

3.2.4 Implementación

- **Archivo**: solidum/elements/frame/euler_corot.py — convive con `Frame2DEuler` y `Frame2DTimoshenko` en el paquete `solidum/elements/frame/`, sin heredar de ninguno. Comparte con ellos los helpers libres de _shared.py (carga de cuerpo, masa consistente, traducción a `ElementForces`), pero no `build_geometry_2d`: reconstruye `T` desde `alpha0` por su lógica corotacional propia.
- **Clase**: `Frame2DEulerCorot(FrameSectionMixin, Element)`. Métodos internos `_current_geometry` (longitud, cosenos, ángulo corriente y rigid-body α_e), `_unwrap` (reducción a $(-\pi, \pi]$), `_basic_deformations` y `_basic_response` (fuerzas básicas y $\mathbf{K}_{ll}$ por el camino con material o por la sección). La constitutiva común de los marcos vive en `FrameSectionMixin` (solidum/elements/frame/_section.py, ADR 0021). `REFERENCE_KWARGS = {"material": MaterialRegistry, "section": SectionRegistry}`.
- **Tests**: tests/test_frame.py · `TestFrame2DEulerCorotAcceptance` — 4 criterios + registro:
- `test_acceptance_limite_lineal_coincide_con_frame2deuler`
- `test_acceptance_invariancia_rotacion_rigida`
- `test_acceptance_coherencia_tangente_fuerza_interna` — **finite-difference check** de $\mathbf{K}_T$ contra derivada numérica de $\mathbf{F}_{\text{int}}$, red de seguridad clave en esta formulación.
- `test_acceptance_simetria_K`
- `test_registro_en_registry`
- Bathe-Bolourchi (criterios 5 y 6): `TestFrame2DEulerCorotBatheBolourchi` en el mismo archivo.
- Guarda (criterio 7): tests/test_fiber_section.py · `TestGuardas`.

- Camino con sección (criterio 8): tests/validation/test_fiber_section_validation.py · `test_caso1_reproduccion_elastica_marcos_2d`, `test_caso8_corotacional_elemento_giro_grande`, `test_caso8_corotacional_arco_de_Bathe_Bolourchi`; tangente frente a diferencias finitas con plasticidad y giro de sólido rígido grande en `tests/test_fiber_section.py` · `TestTangente.test_frame2d_euler_corot_con_giro_grande`.

- Masa (criterio 9): tests/test_frame_mass.py · `TestMarco2DEulerBernoulli` (parametrizado con los dos marcos, incluye `test_corrotacional_igual_al_lineal`), `TestCaminoConSeccion`, `test_traslacion_rigida_2d` y `test_giro_rigido_2d_consistente`.

- **Notas de traducción**:

- La formulación se basa en 3 DOFs deformacionales $[u, \bar{\theta}_1, \bar{\theta}_2]$ y una matriz $\mathbf{B}$ 3×6 que los relaciona con los DOFs globales. Evaluada en configuración corriente.

- **Signo de la contribución de momentos en $\mathbf{K}_\sigma$**: tras derivar $\partial(\mathbf{z}/l)/\partial\mathbf{u}_e$ analíticamente se obtiene $-(1/l^2)(\mathbf{r}\mathbf{z}^\top + \mathbf{z}\mathbf{r}^\top)$. El signo es **negativo** — un error común es ponerlo positivo. El test de diferencias finitas lo cazó en la primera implementación.

- `compute_internal_forces` proyecta $\mathbf{F}_{\text{int}}$ al sistema local corriente para reportar axial/cortante separados, utilizando la submatriz 2×2 de rotación del eje corriente.

- `_unwrap` lleva los ángulos al intervalo $(-\pi, \pi]$; la formulación es válida mientras la rotación acumulada por paso sea menor que π.

- Camino con sección: $\mathbf{B}_g$ (2×3) del sistema básico se precalcula al construir (`_build_basic_B`); `_basic_response` devuelve $[N, M_1, M_2]$ y $\mathbf{K}_{ll}$ integrados, y el resto de `compute_element_state` (matriz $\mathbf{B}$ 3×6, $\mathbf{K}_\sigma$) no distingue el camino. `internal_forces` y `compute_internal_forces` evalúan sobre una instantánea del trial y la restauran; el diccionario legado añade `section_deformations` y `section_forces (n_ip, 2)`. Masa y peso propio en la configuración de referencia con ρA de la sección.

- Masa (§10): las mismas funciones de `_shared.py` que `Frame2DEuler` (`_frame2d_consistent_mass_local`, `_frame2d_lumped_mass_local`) con la densidad lineal de `_linear_density` y la $\mathbf{T}$ de referencia reconstruida desde `alpha0`.

3.2.5 Diálogo

- **2026-04-21** · Implementación siguiendo Crisfield §7.3. Convive en `frame.py` con las vigas lineales por familia temática (todas son vigas 2D), sin herencia ni helpers compartidos.

- **2026-05-13** · `frame.py` se parte en paquete `solidum/elements/frame/`. EulerCorot vive ahora en `euler_corot.py`, sin herencia con las otras vigas 2D. Pasan a compartirse los helpers libres de carga de cuerpo, masa y traducción de fuerzas a través de `_shared.py`; la lógica corotacional propia (reconstrucción de `T` desde `alpha0`, cinemática actual) se mantiene íntegra dentro de la clase.

- **2026-04-21** · La primera versión tenía el signo equivocado en la parte de momentos de $\mathbf{K}_\sigma$ (+ en lugar de −). El test de coherencia tangente/fuerza interna por diferencias finitas lo

detectó inmediatamente: $\|\mathbf{K}_{T,\text{analítica}} - \mathbf{K}_{T,\text{FD}}\|_{\text{rel}}$ del orden de 1e-1 en lugar de 1e-5. Tras derivar $\partial(\mathbf{z}/l)/\partial\mathbf{u}_e$ cuidadosamente (ver notas) el signo correcto es negativo. Este episodio justifica retrospectivamente incluir un test FD como criterio de aceptación en toda formulación no-lineal geométrica futura.

- **2026-04-21** · El test del límite lineal usa $P = 10$ N para obtener $v/L \sim 10^{-4}$: suficientemente pequeño para estar en régimen lineal pero suficientemente grande para que el residuo del solver converja por debajo de `tol=1e-8` (cargas menores producen desplazamientos del orden del ruido numérico y el Newton oscila en la tolerancia relativa).

- **2026-09-22** · Auditoría global: `internal_forces(U, equivalent_load=None)`: cuando el `Assembler` ensambló peso propio o fuerza de cuerpo, `solidum.run` pasa el vector nodal equivalente del elemento (escalado por λ_final) y las fuerzas internas de extremo son `F_int - f_eq` en ejes locales. Antes se devolvía `K·u`, que con carga distribuida son fuerzas nodales, no fuerzas internas: un voladizo bajo peso propio daba V_i = qL/2 y M_i = −5qL²/12 en vez de qL y −qL²/2, contradiciendo a las reacciones.

- **2026-09-29** · Sección por fibras, etapa 1 (ADR 0021, FiberSection). El modelo no lineal anterior —E_t de la deformación axial escalando toda $\mathbf{K}_{ll}$— desaparece: con fluencia axial anulaba el flector y con ablandamiento le cambiaba el signo. Dos caminos excluyentes: `material` + `A`, `I` (forma cerrada, sin cambio de resultados, sólo materiales con `IS_LINEAR_ELASTIC = True`) y `section` + `n_ip` (la sección se integra en el sistema básico corrotado; la cinemática corrotacional y $\mathbf{K}_\sigma$ no cambian). Contrato: parámetros `section` y `n_ip`; `A` e `I` obligatorios sólo con material; criterios 7 (guarda) y 8 (camino con sección, verificado a 1e-10 con giros de hasta 150° y en el arco de Bathe-Bolourchi hasta $\theta = \pi/2$).

- **2026-09-29** · Masa sin inercia rotatoria (hallazgo clas-4 de la auditoría; **cambio físico aprobado por el usuario**), igual que en Frame2DEuler: se retira el bloque $\frac{\rho I L_0}{6}\left[\begin{smallmatrix}2&1\\1&2\end{smallmatrix}\right]$ de los giros (cinemática de Timoshenko; Euler-Bernoulli desprecia la inercia rotatoria) y la concentrada pasa de $\rho I L_0/2$ a la HRZ $\rho A L_0^3/78$ por giro. Por los dos caminos; la masa ya no depende de I. Contrato: §10 y criterio 9.

- **2026-10-27** · `compute_body_load` acepta un campo `b(x)` (ADR 0022, fase 4), repartido en la geometría de referencia con las funciones de Hermite, como el `Frame2DEuler`; declara `REFERENCE_CELL = "line"` y su campo interior es el linealizado en la referencia (no entra en la verificación por soluciones manufacturadas, que es lineal). Se corrige una nota desactualizada de `out_of_scope`: la fuerza de cuerpo ya la repartía `compute_body_load`.

3.3 Frame2DTimoshenko

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.3.1 Especificación física

0. Descripción general

Elemento 1D inmerso en el plano 2D que modela una **viga** según la teoría de **Timoshenko**. Dos nodos rígidamente conectados, 3 DOFs por nodo (u_x, u_y, r_z). Transmite **fuerza axial**, **cortante transverso** y **momento flector**. Incluye explícitamente la **deformación por cortante transversal**. Régimen de **linealidad geométrica** (pequeños desplazamientos y rotaciones).

1. Hipótesis cinemática de Timoshenko

- Las secciones transversales se mantienen **planas** tras la deformación (igual que Euler-Bernoulli).
- Las secciones **no** están obligadas a ser perpendiculares al eje neutro: se permite una **rotación adicional** respecto a la tangente.

Consecuencia: la rotación de la sección $\theta(s)$ y la pendiente de la elástica dv/ds son **variables independientes**:

$$\gamma(s) = \frac{dv}{ds} - \theta(s) \neq 0,$$

donde γ es la distorsión angular (deformación por cortante transverso). Euler-Bernoulli corresponde al caso $\gamma = 0$.

Apropiado para vigas **gruesas, cortas o peraltadas** ($L/h \lesssim 10$) donde la deformación por cortante no es despreciable.

2. Campos de desplazamiento

- Axial: $u(s)$ a lo largo del eje local $s \in [0, L]$.
- Transverso: $v(s)$ perpendicular al eje local.
- Rotación de la sección: $\theta(s)$, **independiente** de v.

3. Medidas de deformación

- Axial: $\varepsilon_{axial}(s) = du/ds$.
- Curvatura (flexión): $\kappa(s) = d\theta/ds$.
- Cortante: $\gamma(s) = dv/ds - \theta(s)$.

4. Ecuaciones constitutivas

- Axial: $N = EA\,\varepsilon_{axial}$.
- Flexión: $M = EI\,\kappa$.
- Cortante: $V = GA_s\,\gamma$, con $G = E/[2(1+\nu)]$ (módulo de corte) y A_s el **área efectiva de cortante** (menor que A por la distribución no uniforme del esfuerzo cortante σ_{xy} en la sección).

Sólo material elástico lineal (`IS_LINEAR_ELASTIC = True`; ADR 0021). Un material inelástico se rechaza al construir. Hasta el 2026-09-29 se admitía y su E_t axial escalaba toda la matriz local (flector y cortante nulos con fluencia axial, de signo cambiado con ablandamiento). La sección por fibras de este elemento, que necesita su propia cinemática con cortante, es la **etapa 2** del ADR 0021 y está pendiente; mientras tanto, la plasticidad distribuida en la sección está disponible en los marcos Euler-Bernoulli (`Frame2DEuler`, `Frame2DEulerCorot`, `Frame3D`) con `section`.

5. Factor de Phi (shear locking)

La formulación estándar con interpolaciones lineales de v y θ sufre **shear locking** para vigas esbeltas (la rigidez por cortante domina numéricamente aunque γ físicamente sea pequeño). La matriz local incorpora un factor correctivo:

$$\Phi = \frac{12\,E\,I}{G\,A_s\,L^2}.$$

Para $\Phi \to 0$ (viga esbelta) la matriz se reduce a la de Euler-Bernoulli.

6. Equilibrio — forma débil

Principio de trabajos virtuales (PTV). El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \int_0^L (N\,\delta\varepsilon_{axial} + M\,\delta\kappa + V\,\delta\gamma)\,ds = \delta\mathbf{u}_e^{\top}\,\mathbf{F}_{\text{int}}.$$

3.3.2 Formulación numérica (FEM)

7. Discretización

Dos nodos ($s = 0$ y $s = L$), 3 DOFs por nodo en coordenadas globales:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, r_z^{(1)}, u_x^{(2)}, u_y^{(2)}, r_z^{(2)}]^{\top}.$$

8. Sistema local y transformación

Cosenos directores del eje (configuración inicial, fija): $c = (x_2 - x_1)/L$, $s_\theta = (y_2 - y_1)/L$. Matriz de transformación ortogonal 6×6:

$$\mathbf{T} = \begin{bmatrix} c & s_\theta & 0 & 0 & 0 & 0 \\ -s_\theta & c & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & c & s_\theta & 0 \\ 0 & 0 & 0 & -s_\theta & c & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{u}_{\text{local}} = \mathbf{T}\,\mathbf{u}_e.$$

9. Rigidez local con corrección por shear locking

Coeficientes:

$$a = \frac{12\,EI}{L^3(1+\Phi)}, \quad b = \frac{6\,EI}{L^2(1+\Phi)}, \quad c_m = \frac{(4+\Phi)\,EI}{L(1+\Phi)}, \quad d = \frac{(2-\Phi)\,EI}{L(1+\Phi)}.$$

$$\mathbf{K}_{\text{local}} = \begin{bmatrix} EA/L & 0 & 0 & -EA/L & 0 & 0 \\ 0 & a & b & 0 & -a & b \\ 0 & b & c_m & 0 & -b & d \\ -EA/L & 0 & 0 & EA/L & 0 & 0 \\ 0 & -a & -b & 0 & a & -b \\ 0 & b & d & 0 & -b & c_m \end{bmatrix}.$$

Para $\Phi \to 0$: $a \to 12EI/L^3$, $b \to 6EI/L^2$, $c_m \to 4EI/L$, $d \to 2EI/L$ — se recupera Euler-Bernoulli.

10. Fuerzas internas

$$\mathbf{F}_{\text{int,local}} = \mathbf{K}_{\text{local}}\,\mathbf{u}_{\text{local}}.$$

La componente axial se escribe con el esfuerzo axial del material, $F_1 = -\sigma A$, $F_4 = +\sigma A$; con material elástico lineal (el único admitido) coincide con la fila axial del producto.

11. Ensamblaje global

$$\mathbf{K}_{\text{global}} = \mathbf{T}^\top\,\mathbf{K}_{\text{local}}\,\mathbf{T}, \qquad \mathbf{F}_{\text{int}} = \mathbf{T}^\top\,\mathbf{F}_{\text{int,local}}.$$

12. Cuadratura

No aplica (forma cerrada).

13. Matriz de masa

Con inercia rotatoria de la sección: el giro θ es independiente de la pendiente (§1) y su energía cinética $\frac{1}{2}\rho I\dot{\theta}^2$ no se desprecia, a diferencia de los marcos Euler-Bernoulli (Frame2DEuler §13).

- **Consistente**: axial $\frac{\rho AL}{6}\left[\begin{smallmatrix}2&1\\1&2\end{smallmatrix}\right]$, flexión hermítica $\frac{\rho AL}{420}\mathbf{H}$ en $(v_1, \theta_1, v_2, \theta_2)$ (la de Euler-Bernoulli; la interpolación dependiente de Φ añadiría términos de orden Φ que se desprecian) e inercia rotatoria $\frac{\rho IL}{6}\left[\begin{smallmatrix}2&1\\1&2\end{smallmatrix}\right]$ en (θ_1, θ_2).
- **Concentrada**: $\rho AL/2$ traslacional y $\rho IL/2$ rotacional por nodo.

Se conserva hasta la etapa 2 de la sección por fibras (ADR 0021).

3.3.3 Contrato de implementación

```yaml
name: Frame2DTimoshenko
kind: element
status: validated

interface:
  dof_names: [ux, uy, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - zienkiewicz_taylor: "vol. 2 (6.ª ed.), cap. 10: barras de Timoshenko (formulación
        base, material elástico)"
  general_codes:
    - opensees: "ElasticTimoshenkoBeam (2D)"

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal" }
  - { name: I, type: float, required: true, desc: "Momento de inercia respecto al eje
      perpendicular al plano" }
  - { name: As, type: float, required: true, desc: "Área efectiva de cortante" }
  - { name: nu, type: float, required: false, default: 0.3, desc: "Coeficiente de
      Poisson (si el material no lo expone)" }

material_contract:
  signature: "compute_stateε(, state) -> σ(, E_tangent, state')"
  strain_kind: "axial escalar (deformación media ε = (u_2 - u_1)/L en sistema local)"
  nonlinearity_model: "sólo materiales elásticos lineales (IS_LINEAR_ELASTIC = True; un
      material inelástico se rechaza al construir). Sección por fibras con cortante:
      etapa 2 del ADR 0021, pendiente"
  poisson_source: "si material.nu existe, se usa; en su defecto el parámetro `nu` del
      elemento con default 0.3 y advertencia por consola"

conventions:
  sign: "σ > 0 ⇔ε  > 0 (elongación) en el eje axial"
  rotation_sign: "r_z positivo antihorario"
  node_orientation: "eje local del nodo 1 al nodo 2"
  configuration: "inicial fija - L, c, s, T, Φ (en forma funcional) se calculan sin
      actualizar"

validity:
  - "vigas peraltadas o cortas (L/h ≤ 10) donde el cortante no es despreciable"
  - pequeños desplazamientos y pequeñas rotaciones
  - "ε|_axial| ≤ 1e-2"

out_of_scope:
  - grandes rotaciones o desplazamientos (no hay variante corotacional en el catálogo
      actual)
  - material inelástico y plasticidad distribuida en la sección (sección por fibras de
      Timoshenko: etapa 2 del ADR 0021; hoy se rechaza al construir)
  - pandeo
  - "cargas puntuales en el vano (una fuerza de cuerpo variable b(x) sí, desde el
      2026-10-27, ADR 0022)"
```

```yaml
acceptance:
  - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
    setup: "línea recta inclinada; solución exacta impuesta en un extremo y sus fuerzas
        internas como cargas nodales en el otro; carga de línea como fuerza de cuerpo
       b(x); u* axial y, en flexión, una función ψ de la que salen θ* = ψ' y v* = ψ-
       (EI/(G·A_s))·ψ'' (sin momento distribuido); viga gruesa, EI/(G·A_s) = 0.47 con
       longitud 2; n = 4, 8, 16, 32"
    expect: "con campos polinómicos (carga de grado ≤ 4), error nodal de redondeo con 1
        y 3 elementos (exactitud nodal, Tong 1969), con la carga repartida con las
       funciones de Friedman y Kosmatka; con campos suaves, orden ≥ teórico - 0.1 por
       componente: u 2, θ 3, v 2 en L²; ε 1, κ 2, γ 1 en energía; con A_s enorme Φ( ≪
       1), v orden 4 como Navier-Bernoulli"
    tol_abs: 0.1
  - name: convergencia a Euler-Bernoulli para viga esbelta
    setup: "voladizo con carga transversal P, viga esbelta (L/h ≥ 100) ⇒Φ→   0"
    expect: "v(L) ≈ PL³/(3EI) (solución Euler-Bernoulli) con tolerancia relativa ≤ 1e-3
       "
    tol_rel: 1.0e-3

  - name: respuesta axial pura desacoplada
    setup: "voladizo con carga axial F"
    expect: "u_x(L) = F·L/(E·A); momentos y cortantes nulos"
    tol_rel: 1.0e-10

  - name: simetría de K
    setup: "viga oblicua"
    expect: "K_global = ᵀK_global"
    tol_abs: 1.0e-12

  - name: guarda de material elástico lineal           # añadido 2026-09-29 (ADR 0021)
    setup: "material Elastoplastic1D"
    expect: "ValueError al construir, con un mensaje que dice que el elemento todavía
       no admite sección por fibras y nombra los marcos que sí la admiten"

references:
  - "Reddy J.N., An Introduction to the Finite Element Method, cap. 5 (vigas de
     Timoshenko)"
  - "Bathe K.-J., Finite Element Procedures, §5.4 (formulación con factor Φ)"
```

3.3.4 Implementación

- **Archivo**: solidum/elements/frame/timoshenko.py — submódulo del paquete `solidum/elements/frame/` que aloja también `Frame2DEuler` y `Frame2DEulerCorot`.
- **Clase**: `Frame2DTimoshenko(FrameSectionMixin, Element)`, sin herencia con las otras vigas 2D. Del mixin (solidum/elements/frame/_section.py, ADR 0021) sólo usa la guarda de material elástico lineal: no declara `section` en `REFERENCE_KWARGS` (sigue siendo `{"material": MaterialRegistry}`), así que su mensaje de rechazo dice que todavía no admite sección por fibras y nombra los marcos que sí. Comparte `build_geometry_2d` con `Frame2DEuler` en solidum/elements/frame/_shared.py; además, los helpers libres de carga de cuerpo, masa consistente y traducción a `ElementForces` también viven en `_shared.py`.

- **Tests**: `tests/test_line_elements.py` (campo interior, energía igual a la de la rigidez, carga como campo) y `tests/validation/test_mms_frames.py` (soluciones manufacturadas); tests/test_frame.py · `TestFrame2DTimoshenkoAcceptance`:
- `test_acceptance_convergencia_euler_en_viga_esbelta` (criterio 1) — viga con L/h muy grande, verifica que la flecha tiende a $PL^3/(3EI)$ con tolerancia relativa 10^{-3}.
- `test_acceptance_respuesta_axial_pura` (criterio 2) — carga axial pura, verifica $u_x = FL/(EA)$.
- `test_acceptance_simetria_K` (criterio 3) — viga oblicua, verifica $\mathbf{K} = \mathbf{K}^\top$.
- `test_registro_en_registry` — autodiscover.
- Criterio 4 (guarda): tests/test_fiber_section.py · `TestGuardas.test_timoshenko_solo_elastico`.
- **Notas de traducción**:
- Para no colisionar con la variable cinemática c (coseno director) dentro de `compute_element_state`, el coeficiente c_m de la matriz local se llama `c_coef` en el código y `d_coef` su análogo (antes eran `c` y `d` inline, ambiguos).
- El Poisson ν se toma de `material.nu` si el material lo expone; en su defecto, del parámetro `nu` del elemento con default 0.3 y una advertencia por consola la primera vez que se construye con ese default — atajo a errores silenciosos de usuarios que olvidan especificarlo.
- La matriz $\mathbf{T}$ se calcula una sola vez en `__init__` y se guarda como atributo (régimen geométricamente lineal).
- El factor $\Phi = 24(1+\nu)\,I/(A_s L^2)$ no depende del módulo del material (auditoría 2026-09-22) y, con material elástico lineal, tampoco la matriz local cambia con el estado.
- Masa (§13): las funciones de `_shared.py` compartidas con los marcos Euler-Bernoulli, con el argumento `rotary_inertia` $= \rho I$ que sólo pasa este elemento. Verificada en tests/test_frame_mass.py · `TestTimoshenkoSinCambio` y en la energía cinética del giro rígido (`test_giro_rigido_2d_consistente`, que suma $\frac{1}{2}\rho I L \omega^2$).

3.3.5 Diálogo

- **2026-04-21** · Elemento movido a archivo propio `solidum/elements/frame.py` y desacoplado del helper `_frame_geometry`. Con este movimiento, el helper compartido se elimina completamente del repositorio: `Frame2DEuler` y `Frame2DTimoshenko` replican la construcción de $\mathbf{T}$ como método estático `_build_geometry`, cada una en su clase. La duplicación (~15 líneas) es el precio aceptado de la independencia mutua.
- **2026-04-21** · Test del criterio 1: se eligió L/h grande en lugar de reproducir un caso específico de libro porque la convergencia Timoshenko → Euler es el comportamiento **esperado por construcción** (factor $\Phi \to 0$). Verificarlo directamente con el solver completo valida simultáneamente la cinemática de Timoshenko, el tratamiento del shear locking y la integración con ensamblador + solver.

- **2026-05-13** · `frame.py` se parte en paquete `solidum/elements/frame/`. La duplicación de `_build_geometry` ya no se justifica: ambas vigas comparten `build_geometry_2d` en `_shared.py`, helper interno al paquete (no flotante). Las clases siguen sin herencia entre sí.

- **2026-09-22** · Auditoría global: `internal_forces(U, equivalent_load=None)`: cuando el `Assembler` ensambló peso propio o fuerza de cuerpo, `solidum.run` pasa el vector nodal equivalente del elemento (escalado por λ_final) y las fuerzas internas de extremo son `F_int − f_eq` en ejes locales. Antes se devolvía `K·u`, que con carga distribuida son fuerzas nodales, no fuerzas internas: un voladizo bajo peso propio daba V_i = qL/2 y M_i = $-5qL^2/12$ en vez de qL y $-qL^2/2$, contradiciendo a las reacciones.

- **2026-09-22** · Auditoría global: Φ `= 24(1+`ν`)·I/(A_s·L²)`, independiente de E_t: con `E_t = 0` (plasticidad perfecta plastificada) la forma `12·E_t·I/(G·A_s·L²)` dividía 0/0.

- **2026-09-29** · Sólo material elástico lineal hasta la etapa 2 del ADR 0021. La auditoría de pureza verificó que el modelo no lineal anterior —la E_t de la deformación axial escalando toda la matriz local, flexión y cortante incluidos— daba fuerzas internas incorrectas en silencio (nulas con fluencia axial, de signo cambiado con ablandamiento). Desaparece: un material con `IS_LINEAR_ELASTIC = False` se rechaza al construir, con un mensaje que remite a los marcos Euler-Bernoulli con sección por fibras. La sección por fibras de este elemento necesita su propia cinemática con cortante (qué hacer con γ en las fibras, integración del cortante, tratamiento del bloqueo) y es una decisión física pendiente del usuario (etapa 2). Contrato: criterio de aceptación 4 (guarda); `nonlinearity_model` y `out_of_scope` actualizados.

- **2026-09-29** · Aclaración, sin cambio de resultados: los marcos Euler-Bernoulli pierden la inercia rotatoria de su masa (cambio aprobado por el usuario, Frame2DEuler *Diálogo*); este elemento la **conserva** (§13), porque en Timoshenko el giro es independiente, hasta la etapa 2 de las fibras. La matriz es idéntica bit a bit a la anterior; las funciones compartidas de `_shared.py` la reciben ahora como argumento explícito.

- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022, fase 4), **con la decisión física del usuario**: la carga variable se reparte con las funciones de interdependencia del propio elemento (Friedman y Kosmatka 1993), que dan exactamente su rigidez con la corrección Φ (comprobado con SymPy) y conservan la exactitud nodal; con las de Hermite quedaría un error nodal de 12 %, 2.8 % y 0.7 % con 2, 4 y 8 elementos. Con `b` constante coinciden (`qL/2`, $\pm$`qL²/12`) y no cambia ningún resultado. Campo interior (`line_field`: `u`, `v`, θ`z`, ε, κ`z`, γ`y` constante) con `section_rigidities` (`EA`, `EI`, `G·A_s`). Órdenes medidos: u 2, θ 3, κ 2, γ 1 y v 2: al refinar $\Phi \propto$ `1/h²` crece y la `v` del elemento aproxima `v''` con θ`'`, que difieren en γ`'`; con $\Phi \ll$ `1`, v orden 4 como Navier-Bernoulli. Sin momento distribuido (decisión del usuario), la solución manufacturada sale de una función ψ.

3.4 Frame3D

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.4.1 Especificación física

0. Descripción general

Elemento 1D inmerso en el espacio tridimensional que modela una **viga esbelta 3D**. Dos nodos rígidamente conectados, 6 DOFs por nodo $(u_x, u_y, u_z, r_x, r_y, r_z)$. Captura seis acciones internas:

- **Axial** (fuerza normal N a lo largo del eje local x).
- **Cortante en dos planos** (V_y en el plano xy, V_z en el plano xz).
- **Flexión en dos planos** (M_z alrededor del eje local z, M_y alrededor del eje local y).
- **Torsión** (M_x alrededor del eje longitudinal).

Régimen de **linealidad geométrica** (pequeños desplazamientos y rotaciones). Hipótesis de Euler-Bernoulli: secciones planas perpendiculares al eje deformado (sin cizalladura transversal).

1. Hipótesis

- Secciones planas y perpendiculares al eje neutro deformado (igual que Frame2DEuler).
- Flexiones en los dos planos **desacopladas** entre sí (ejes principales de inercia).
- Torsión de Saint-Venant pura (sin alabeo). Válida para secciones macizas o cerradas; abiertas de pared delgada requerirían formulación con alabeo (fuera de alcance).
- Camino con material: material isótropo 1D elástico lineal; $G = E/[2(1+\nu)]$. Camino con sección: la torsión es elástica y desacoplada, con el GJ que declara la sección (FiberSection §1).

2. Sistema de ejes locales

Tres ejes ortonormales en cada elemento:

- $\hat{\mathbf{x}}$ **local**: dirección del eje de la barra ($\hat{\mathbf{x}} = (\mathbf{x}_2 - \mathbf{x}_1)/L$).
- $\hat{\mathbf{y}}$ **local** y $\hat{\mathbf{z}}$ **local**: ejes principales de inercia de la sección, ortogonales al eje.

La orientación de $\hat{\mathbf{y}}, \hat{\mathbf{z}}$ no está determinada solo por la dirección del eje — hace falta un **vector de referencia** proporcionado por el usuario (o un default). Es la diferencia estructural respecto al caso 2D, donde el plano era único.

3. Medidas de deformación / fuerzas internas

- Axial: $\varepsilon_{axial} = du/ds$, fuerza axial $N = EA\,\varepsilon_{axial}$ (o $N = A\,\sigma$ con σ del material).
- Flexión en plano xy: curvatura $\kappa_z = d^2v/ds^2$, momento $M_z = EI_z\,\kappa_z$.
- Flexión en plano xz: curvatura $\kappa_y = -d^2w/ds^2$ (con $w' = -\theta_y$ por la regla de la mano derecha), momento $M_y = EI_y\,\kappa_y$.
- Torsión: $d\theta_x/ds$, momento torsor $M_x = GJ\,d\theta_x/ds$.

Constitutiva: dos caminos excluyentes (ADR 0021):

- **Con `material` + `A`, `Iy`, `Iz`, `J`** (+ `nu`): las leyes anteriores en forma cerrada. Sólo materiales **elásticos lineales** (`IS_LINEAR_ELASTIC = True`); un material inelástico se rechaza al construir con un mensaje que remite a la sección por fibras.
- **Con `section`** (+ `n_ip`): N, M_y, M_z los da la ley de sección, que integra el esfuerzo σ_{xx} de cada fibra con su material 1D (FiberSection §2-§4); la torsión es $T = GJ\,\theta_x'$ con el `GJ` de la sección, obligatorio.

Hasta el 2026-09-29 el camino con material admitía materiales inelásticos y escalaba toda la matriz local con la E_t axial, torsión incluida ($G = E_t/[2(1+\nu)]$); esa ley desapareció (ver *Diálogo*).

4. Equilibrio — forma débil

PTV. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \int_0^L \left(N\,\delta\varepsilon + M_z\,\delta\kappa_z + M_y\,\delta\kappa_y + M_x\,\delta\theta_x'\right)\,ds.$$

3.4.2 Formulación numérica (FEM)

5. Discretización

Dos nodos, 6 DOFs por nodo:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, r_x^{(1)}, r_y^{(1)}, r_z^{(1)},\ u_x^{(2)}, u_y^{(2)}, u_z^{(2)}, r_x^{(2)}, r_y^{(2)}, r_z^{(2)}]^\top.$$

Vector de 12 componentes → matrices 12×12.

6. Construcción de ejes locales

Dados $\mathbf{X}_1, \mathbf{X}_2$ y un vector de referencia $\mathbf{v}_{\text{ref}}$ proporcionado:

1. $\hat{\mathbf{x}} = (\mathbf{X}_2 - \mathbf{X}_1)/L$.
2. Proyectar $\mathbf{v}_{\text{ref}}$ al plano perpendicular a $\hat{\mathbf{x}}$:

$$\tilde{\mathbf{y}} = \mathbf{v}_{\text{ref}} - (\mathbf{v}_{\text{ref}} \cdot \hat{\mathbf{x}})\,\hat{\mathbf{x}}.$$

1. $\hat{\mathbf{y}} = \tilde{\mathbf{y}}/\|\tilde{\mathbf{y}}\|$ (falla si $\|\tilde{\mathbf{y}}\| \approx 0$: $\mathbf{v}_{\text{ref}}$ paralelo al eje).
2. $\hat{\mathbf{z}} = \hat{\mathbf{x}} \times \hat{\mathbf{y}}$.

Default de $\mathbf{v}_{\text{ref}}$ (cuando el usuario no lo proporciona): $[0, 0, 1]$ (eje z global como "vertical"). Si el eje del elemento es cercanamente paralelo a z global (viga vertical), se usa $[1, 0, 0]$ como fallback y se emite una advertencia.

7. Matriz de transformación global → local

$$\boldsymbol{\lambda} = \begin{bmatrix} \hat{\mathbf{x}}^\top \\ \hat{\mathbf{y}}^\top \\ \hat{\mathbf{z}}^\top \end{bmatrix} \quad (3 \times 3,\ \text{ortogonal}).$$

$$\mathbf{T} = \text{bloques diag}(\boldsymbol{\lambda}, \boldsymbol{\lambda}, \boldsymbol{\lambda}, \boldsymbol{\lambda}) \quad (12 \times 12).$$

Aplicada a los 4 bloques de 3 DOFs: desplazamientos nodo 1, rotaciones nodo 1, desplazamientos nodo 2, rotaciones nodo 2. Las rotaciones infinitesimales en 3D transforman como vectores.

$$\mathbf{u}_{\text{local}} = \mathbf{T}\,\mathbf{u}_e.$$

8. Matriz de rigidez local

Desacoplada en cuatro contribuciones: axial, torsión, flexión xy (plano con u_y y r_z, inercia I_z), flexión xz (plano con u_z y r_y, inercia I_y).

Definiciones:

$$a_z = \tfrac{12EI_z}{L^3},\ b_z = \tfrac{6EI_z}{L^2},\ c_z = \tfrac{4EI_z}{L},\ d_z = \tfrac{2EI_z}{L},$$
$$a_y = \tfrac{12EI_y}{L^3},\ b_y = \tfrac{6EI_y}{L^2},\ c_y = \tfrac{4EI_y}{L},\ d_y = \tfrac{2EI_y}{L}.$$

La rigidez $\mathbf{K}_{\text{local}}$ es simétrica, con los bloques desacoplados (0s fuera de su plano correspondiente). Los signos del bloque de flexión xz se invierten respecto al xy porque u_z y r_y forman una pareja dextrógira opuesta (consecuencia del producto vectorial $\hat{\mathbf{x}} \times \hat{\mathbf{y}} = \hat{\mathbf{z}}$).

Forma concreta en términos de los 12 DOFs locales (la matriz completa vive en la implementación).

9. Fuerzas internas

$$\mathbf{F}_{\text{int,local}} = \mathbf{K}_{\text{local}}\,\mathbf{u}_{\text{local}},$$

con la componente axial escrita con el σ que devuelve el material, $F_1 = -\sigma A$, $F_7 = +\sigma A$; con material elástico lineal (el único admitido en este camino) coincide con la fila axial del producto.

Camino con sección: con $\mathbf{e}_g = [\varepsilon_0,\ \kappa_y,\ \kappa_z]^\top(x_g) = \mathbf{B}_g\,\mathbf{u}_{\text{local}}$, donde la fila de κ_z es $[N_1'', N_2'', N_3'', N_4'']$ sobre $(v_1, \theta_{z1}, v_2, \theta_{z2})$ y la de κ_y es $[-N_1'', N_2'', -N_3'', N_4'']$ sobre $(w_1, \theta_{y1}, w_2, \theta_{y2})$ (FiberSection §5),

$$\mathbf{F}_{\text{int,local}} = \sum_g w_g\,\tfrac{L}{2}\,\mathbf{B}_g^\top\,\mathbf{s}_g, \qquad \mathbf{K}_{\text{local}} = \sum_g w_g\,\tfrac{L}{2}\,\mathbf{B}_g^\top\,\mathbf{k}_{s,g}\,\mathbf{B}_g,$$

más la torsión elástica: GJ/L en el bloque $(\theta_{x1}, \theta_{x2})$ de $\mathbf{K}_{\text{local}}$ y $\mp T$ con $T = GJ\,(\theta_{x2} - \theta_{x1})/L$ en $\mathbf{F}_{\text{int,local}}$. Con fibras elásticas y `n_ip` ≥ 2 reproduce la matriz de §8 exactamente.

10. Ensamblaje global

$$\mathbf{K}_{\text{global}} = \mathbf{T}^{\top}\,\mathbf{K}_{\text{local}}\,\mathbf{T}, \qquad \mathbf{F}_{\text{int}} = \mathbf{T}^{\top}\,\mathbf{F}_{\text{int,local}}.$$

11. Cuadratura

Camino con material: no aplica (forma cerrada; integración analítica de los polinomios de Hermite). Camino con sección: Gauss-Legendre de `n_ip` puntos a lo largo del elemento (3 por defecto, mínimo 2), con `N_INTEGRATION_POINTS = n_ip` como atributo de instancia.

12. Matriz de masa

Cuatro bloques desacoplados en ejes locales, como la rigidez. La flexión **no lleva inercia rotatoria** de la sección (hipótesis de Euler-Bernoulli); la torsión sí lleva la inercia del giro de la sección alrededor del eje, ρJ_p con $J_p = I_y + I_z$ (momento polar geométrico, no la constante de Saint-Venant J, que rige la rigidez). Con sección, $\rho A = \sum_i \rho_i A_i$ y $\rho J_p = \rho I_y + \rho I_z$ (FiberSection §6).

Consistente (`lumping="consistent"`, por defecto):

- Axial (u_1, u_2): $\frac{\rho A L}{6}\left[\begin{smallmatrix}2 & 1\\ 1 & 2\end{smallmatrix}\right]$.
- Torsión $(\theta_{x1}, \theta_{x2})$: $\frac{\rho J_p L}{6}\left[\begin{smallmatrix}2 & 1\\ 1 & 2\end{smallmatrix}\right]$.
- Flexión en el plano xy, $(v_1, \theta_{z1}, v_2, \theta_{z2})$ con $v' = \theta_z$: la hermítica de Frame2DEuler §13,

$$\mathbf{M}_{xy} = \frac{\rho A L}{420}\begin{bmatrix} 156 & 22L & 54 & -13L \\ 22L & 4L^2 & 13L & -3L^2 \\ 54 & 13L & 156 & -22L \\ -13L & -3L^2 & -22L & 4L^2 \end{bmatrix}.$$

- Flexión en el plano xz, $(w_1, \theta_{y1}, w_2, \theta_{y2})$: por $w' = -\theta_y$, los términos de acoplamiento traslación-giro cambian de signo, $\mathbf{M}_{xz} = \mathbf{S}\,\mathbf{M}_{xy}\,\mathbf{S}$ con $\mathbf{S} = \operatorname{diag}(1, -1, 1, -1)$:

$$\mathbf{M}_{xz} = \frac{\rho A L}{420}\begin{bmatrix} 156 & -22L & 54 & 13L \\ -22L & 4L^2 & -13L & -3L^2 \\ 54 & -13L & 156 & 22L \\ 13L & -3L^2 & 22L & 4L^2 \end{bmatrix}.$$

Reproduce exactamente la energía cinética de un giro rígido de velocidad angular $\boldsymbol{\omega}$ alrededor de un extremo: $\frac{1}{2}\rho A\,\frac{L^3}{3}\,|\boldsymbol{\omega}_{\perp}|^2 + \frac{1}{2}\rho J_p L\,\omega_{\parallel}^2$, con $\omega_{\parallel}$ la componente sobre el eje de la barra.

Concentrada (`lumping="lumped"`): por nodo, $\rho AL/2$ en u_x, u_y, u_z, $\rho J_p L/2$ en θ_x y la HRZ de la flexión, $\rho AL^3/78$, en θ_y y θ_z. Diagonal en locales; en globales sólo lo es si el eje del elemento coincide con un eje global: con eje oblicuo el bloque rotacional $\operatorname{diag}(\rho J_p L/2,\ \rho AL^3/78,\ \rho AL^3/78)$ no es escalar y la matriz queda diagonal por bloques por nodo (`CentralDifferenceSolver` la rechaza).

En globales, $\mathbf{M}_{\text{global}} = \mathbf{T}^{\top}\,\mathbf{M}_{\text{local}}\,\mathbf{T}$. Hasta el 2026-09-29 la flexión sumaba la inercia rotatoria $\frac{\rho I L}{6}\left[\begin{smallmatrix}2 & 1\\ 1 & 2\end{smallmatrix}\right]$ en θ_y (I_y) y θ_z (I_z), y la concentrada usaba $\rho I_y L/2$ y $\rho I_z L/2$ (ver *Diálogo*); la torsión no cambia.

3.4.3 Contrato de implementación

```
name: Frame3D
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz, rx, ry, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1   # camino con material; con section, N_INTEGRATION_POINTS =
      n_ip (atributo de instancia)

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - bathe: "cap. 5 (elementos de viga)"
    - zienkiewicz_taylor: "vol. 2 (6.ª ed.), cap. 10: barras (formulación base,
        material elástico)"
  general_codes:
    - opensees: "elasticBeamColumn (3D)"
    - abaqus: "B33"

parameters:
  - { name: A,  type: float, required: false, desc: "Área de la sección transversal.
      Obligatoria con material; no se declara con section" }
  - { name: Iy, type: float, required: false, desc: "Momento de inercia alrededor del
      eje local y. Obligatorio con material; no se declara con section" }
  - { name: Iz, type: float, required: false, desc: "Momento de inercia alrededor del
      eje local z. Obligatorio con material; no se declara con section" }
  - { name: J,  type: float, required: false, desc: "Constante torsional de Saint-
      Venant. Obligatoria con material; con section la torsión usa el GJ de la sección"
       }
  - { name: nu, type: float, required: false, default: 0.3, desc: "Coeficiente de
      Poisson (si el material no lo expone). Sólo con material" }
  - { name: ref_vector, type: list, required: false, default: "[0, 0, 1]",
      desc: "Vector 3D de referencia que fija la orientación de los ejes locales y, z
          de la sección. Default: eje z global." }
  - { name: section, type: "id de sections", required: false, desc: "Sección de marco (
      FiberSection) con GJ declarado. Excluyente con material, A, Iy, Iz, J y nu (ADR
      0021)" }
  - { name: n_ip, type: int, required: false, default: 3, desc: "Puntos de Gauss-
      Legendre a lo largo del elemento, ≥ 2. Sólo con section" }

material_contract:
  signature: "compute_stateε(, state) -> σ(, E_tangent, state')"
  strain_kind: "axial escalar ε( = (u_2 - u_1)/L en sistema local)"
  nonlinearity_model: "camino con material: sólo materiales elásticos lineales (
      IS_LINEAR_ELASTIC = True; un material inelástico se rechaza al construir). La no
      linealidad material entra por section (FiberSection): cada fibra evalúa su
      material 1D con ε = ε₀−  y·κ_z + z·κ_y; torsión elástica desacoplada con el GJ de
       la sección"
  poisson_source: "material.nu si existe; en su defecto parámetro `nu` del elemento (só
      lo camino con material)"

conventions:
```

```
  sign: "σ > 0 ⇔ε  > 0 (elongación); momentos siguen regla de la mano derecha respecto
      a los ejes locales"
  local_axes: "x_local = eje de la barra; y_local, z_local = ejes principales de
      inercia definidos por ref_vector"
  node_orientation: "eje local del nodo 1 al nodo 2"
  configuration: "inicial fija - L, T, ejes locales no se actualizan"

validity:
  - "vigas esbeltas (L/h ≥ 10 en ambos planos de flexión)"
  - "pequeños desplazamientos y pequeñas rotaciones en el espacio"
  - "ε|_axial| ≤ 1e-2"
  - "sección con ejes principales de inercia bien definidos por ref_vector"

out_of_scope:
  - grandes rotaciones espaciales (requeriría Frame3DCorot; pendiente)
  - torsión con alabeo restringido (secciones abiertas de pared delgada)
  - material inelástico con material + A, Iy, Iz, J (se rechaza al construir; la
      plasticidad y el daño entran por section, FiberSection)
  - degradación de la torsión y su interacción con N, My, Mz en el camino con sección (
      torsión elástica desacoplada, FiberSection §7)
  - pandeo por bifurcación
  - deformación por cortante transversal significativa (Timoshenko 3D, fuera de alcance
      )
  - "torsor distribuido (la fuerza de cuerpo, centrada en el eje, no produce torsión)"

numerical_caveats:
  - "Si ref_vector es paralelo al eje de la barra, la proyección al plano perpendicular
      se anula y el constructor aborta con mensaje claro - el usuario debe elegir otro
      vector."
  - "Para barras verticales (eje casi paralelo a z global) con ref_vector default, el
      constructor usa [1, 0, 0] como fallback y emite advertencia por consola."

acceptance:
  - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
    setup: "línea recta inclinada; solución exacta impuesta en un extremo y sus fuerzas
        internas como cargas nodales en el otro; carga de línea como fuerza de cuerpo
       b(x); línea orientada al azar; u*, v*, w* y θx* lineal (no hay torsor
       distribuido); n = 4, 8, 16, 32"
    expect: "con campos polinómicos (carga de grado ≤ 4), error nodal de redondeo con 1
        y 3 elementos (exactitud nodal, Tong 1969); con campos suaves, orden ≥ teórico
       - 0.1 por componente: u 2, v y w 4, θy y θz 3 en L²; ε 1, κy y κz 2 en energía
       ; torsión exacta"
    tol_abs: 0.1
  - name: respuesta axial pura
    setup: "voladizo alineado con eje x, carga F axial en extremo libre"
    expect: "u_x = F·L/(E·A); demás componentes nulas"
    tol_rel: 1.0e-10

  - name: flexión en plano xy (coherencia con Frame2D)
    setup: "voladizo alineado con eje x, carga P en dirección y, ref_vector = [0,0,1]"
    expect: "v_y = P·L³/(3·E·Iz), θ_z = P·L²/(2·E·Iz)"
    tol_rel: 1.0e-10

  - name: flexión en plano xz
    setup: "voladizo alineado con eje x, carga P en dirección z, ref_vector = [0,1,0]"
    expect: "v_z = P·L³/(3·E·Iy), θ_y = -P·L²/(2·E·Iy) (signo por mano derecha)"
```

```
    tol_rel: 1.0e-10

  - name: torsión pura
    setup: "voladizo con momento T aplicado alrededor del eje de la barra en el extremo
        libre"
    expect: "θ_x = T·L/(G·J); demás componentes nulas"
    tol_rel: 1.0e-10

  - name: simetría de K
    setup: "viga en orientación arbitraria con ref_vector genérico"
    expect: "K_global = ᵀK_global"
    tol_abs: 1.0e-8

  - name: guarda de material elástico lineal            # añadido 2026-09-29 (ADR 0021)
    setup: "material Elastoplastic1D o IsotropicDamage1D con A, Iy, Iz, J"
    expect: "ValueError al construir, con un mensaje que nombra IS_LINEAR_ELASTIC y
        remite a FiberSection con 'section'; una sección sin GJ también se rechaza"

  - name: camino con sección                            # añadido 2026-09-29 (ADR 0021)
    setup: "section = FiberSection; casos de acceptance de docs/specs/FiberSection.md"
    expect: "reproducción elástica con n_ip = 2 (elemento oblicuo, sección rectangular
        y mixta), -κM en los dos ejes, interacción -NM, sección circular a 45°, signos"

  - name: matrices de masa consistente y concentrada   # añadido 2026-09-29
    setup: "elemento oblicuo con ref_vector genérico, caminos con material y con secció
        n; M_local = T·M·ᵀT"
    expect: "consistente = axial + torsión ρJ_pL/6·[[2,1],[1,2]] (J_p = I_y + I_z) +
        hermítica en xy y, en xz, la hermítica con los acoplamientos traslación-giro de
         signo opuesto; sin inercia rotatoria de flexión. Concentrada = diagρ(AL/2 ×3,
        ρJ_pL/2, ρAL³/78, ρAL³/78) por nodo"
    tol_rel: 1.0e-12

  - name: energía cinética de movimientos rígidos        # añadido 2026-09-29
    setup: "traslación v; giro ω oblicuo alrededor de un extremo"
    expect: "½ᵀuMu = ρAL|v|²/2 (consistente y concentrada); ½·ρA·L³/3·ω⊥||² + ½·ρJ_p·L·
        ω‖² (consistente)"
    tol_rel: 1.0e-12

references:
  - "Przemieniecki J.S., Theory of Matrix Structural Analysis, Tabla 11.3"
  - "Cook R.D., Concepts and Applications of FEA, §2.8"
  - "Bathe K.-J., Finite Element Procedures, cap. 5"
  - "Hinton E., Rock T., Zienkiewicz O.C. (1976). A note on mass lumping and related
      processes in the finite element method. Earthquake Eng. Struct. Dyn. 4, 245-249 (
      masa concentrada HRZ)"
```

3.4.4 Implementación

- **Archivo**: solidum/elements/frame3d.py — archivo propio, sin dependencia de ningún otro elemento.
- **Clase**: `Frame3D(FrameSectionMixin, Element)`. Métodos privados `_build_local_frame` (longitud, matriz $\boldsymbol{\lambda}$ 3×3 y $\mathbf{T}$ 12×12), `_build_local_stiffness` (matriz $\mathbf{K}_{\text{local}}$ 12×12 con bloques axial, torsión y flexión en ambos planos, camino con material), `_build_section_B` y

`_section_element_state` (camino con sección), `_build_local_mass`, `_build_local_mass_lumped` y `_mass_properties` (masa, §12). Comparte con los marcos planos la constitutiva genérica de `FrameSectionMixin` (solidum/elements/frame/_section.py, ADR 0021) y los bloques de masa de la flexión de solidum/elements/frame/_shared.py (`hermite_consistent_mass`, `hrz_rotational_mass`). `REFERENCE_KWARGS = {"material": MaterialRegistry, "section": SectionRegistry}`.

- **Tests**: `tests/test_line_elements.py` (campo interior, energía igual a la de la rigidez, carga como campo) y `tests/validation/test_mms_frames.py` (soluciones manufacturadas); tests/test_frame3d.py · `TestFrame3DAcceptance` — cubre los 5 criterios + validación de `ref_vector` paralelo al eje + registro:
- `test_acceptance_respuesta_axial_pura`
- `test_acceptance_flexion_xy_coincide_con_frame2d`
- `test_acceptance_flexion_xz`
- `test_acceptance_torsion_pura`
- `test_acceptance_simetria_K`
- `test_rechazo_ref_vector_paralelo`
- `test_registro_en_registry`
- Criterio 6 (guarda): tests/test_fiber_section.py · `TestGuardas` (`test_material_inelastico_remite_a_la_seccion`, `test_exactamente_un_camino`, `test_frame3d_exige_GJ`).
- Criterio 7 (camino con sección): tests/validation/test_fiber_section_validation.py (casos 1, 2, 3, 6 y 7 de la spec FiberSection) y, en `tests/test_fiber_section.py`, `TestTangente.test_frame3d_con_torsion`, `TestMarcosConSeccion.test_torsion_elastica_desacoplada`, `TestMasa`, `TestPesoPropio` y `TestYaml.test_frame3d_torsion_con_GJ_del_yaml`.
- Criterios 8 y 9 (masa): tests/test_frame_mass.py · `TestFrame3D` (consistente en los dos planos y torsión, signo del plano xz, HRZ, I_y e I_z sólo en la torsión), `TestCaminoConSeccion.test_frame3d_con_rho_A_y_rho_Jp_de_las_fibras`, `test_traslacion_rigida_3d`, `test_giro_rigido_3d_consistente` y `test_giro_rigido_por_plano_local_3d`. Diagonalidad por bloques en orientación oblicua: tests/test_mass_lumping.py · `TestLumpedFrame3D`.
- **Notas de traducción**:
- La matriz $\boldsymbol{\lambda}$ 3×3 se construye como $[\hat{\mathbf{x}}^\top; \hat{\mathbf{y}}^\top; \hat{\mathbf{z}}^\top]$ — las filas son los ejes locales expresados en coordenadas globales. $\mathbf{T}$ 12×12 es un `blkdiag` de cuatro copias de $\boldsymbol{\lambda}$ (traslaciones y rotaciones transforman como vectores en 3D infinitesimal).
- El default de `ref_vector` es `[0, 0, 1]`; el fallback `[1, 0, 0]` se activa cuando $|\hat{\mathbf{x}} \cdot \hat{\mathbf{z}}_{\text{global}}| > 0.99$ (barra cercanamente vertical). Ambas rutas emiten advertencia por consola la primera vez que se usan con default.

- La matriz $\mathbf{K}_{\text{local}}$ se construye rellenando entradas no nulas en sus 4 bloques desacoplados (axial, torsión, flexión xy, flexión xz). Los signos de la flexión xz están invertidos respecto a la flexión xy porque la pareja (u_z, r_y) tiene orientación dextrógira opuesta a (u_y, r_z): este punto se verifica explícitamente en `test_acceptance_flexion_xz`.

- `compute_internal_forces` devuelve cortantes, torsión y momentos en ambos extremos separados — útil para post-proceso estructural (diagramas de fuerzas internas). Con sección añade `section_deformations` y `section_forces (n_ip, 3)` $= [\varepsilon_0, \kappa_y, \kappa_z]$ y $[N, M_y, M_z]$ en los puntos de Gauss; como `internal_forces`, evalúa sobre una instantánea del trial y la restaura.

- Camino con sección: sin `nu` (`self.nu = None`); la torsión la da `section.GJ`, y una sección sin `GJ` se rechaza al construir. Masa con ρA de la sección y $\rho J_p = \rho I_y + \rho I_z$ para la torsión, en las mismas funciones de masa consistente y concentrada; peso propio con ρA y fuerza de cuerpo con el área bruta $\sum A_i$.

- Masa (§12): `_mass_properties` devuelve $(\rho A, \rho J_p)$; el plano xz se obtiene de la hermítica del plano xy con $\mathbf{S} = \text{diag}(1, -1, 1, -1)$ (`_XZ_SIGNS`), el mismo cambio de signo que la rigidez.

3.4.5 Diálogo

- **2026-04-21** · Elemento creado como componente totalmente autónomo: no hereda de `Frame2DEuler` ni de ninguna otra viga. La maquinaria cinemática 3D se implementa íntegra dentro de `frame3d.py`, coherente con la filosofía aplicada a cables.

- **2026-04-21** · Decisión sobre `ref_vector` vs "tercer nodo de orientación". Se optó por `ref_vector` (vector 3D proyectado al plano perpendicular al eje) por ser más compacto en YAML y no requerir nodos fantasma en la malla. El default `[0, 0, 1]` cubre la mayoría de casos de ingeniería estructural (estructuras terrestres con gravedad en $-z$); el fallback para barras verticales usa `[1, 0, 0]` con advertencia explícita.

- **2026-04-21** · Verificación cruzada con `Frame2DEuler`: el criterio 2 aplica una viga alineada con el eje x global bajo carga en y; los desplazamientos resultantes deben coincidir con la solución 2D clásica $v = PL^3/(3EI_z)$, $\theta = PL^2/(2EI_z)$. El test lo verifica a 8 decimales. Esto blinda la coherencia dimensional: el 3D se reduce al 2D sin error acumulado cuando el problema es efectivamente plano.

- **2026-09-22** · Auditoría global: `internal_forces(U, equivalent_load=None)`: cuando el `Assembler` ensambló peso propio o fuerza de cuerpo, `solidum.run` pasa el vector nodal equivalente del elemento (escalado por λ_final) y las fuerzas internas de extremo son `F_int - f_eq` en ejes locales. Antes se devolvía `K·u`, que con carga distribuida son fuerzas nodales, no fuerzas internas: un voladizo bajo peso propio daba V_i = qL/2 y M_i = -5qL^2/12 en vez de qL y $-$qL^2/2, contradiciendo a las reacciones.

- **2026-09-29** · Sección por fibras, etapa 1 (ADR 0021, FiberSection). El modelo no lineal anterior —la E_t de la deformación axial escalando toda la matriz local, con $G = E_t/[2(1+\nu)]$— desaparece: con fluencia axial anulaba flectores y torsor, con ablandamiento les cambiaba el signo. Dos caminos excluyentes: `material` + `A`, `Iy`, `Iz`, `J` (+ `nu`), la forma cerrada, sin

cambio de resultados, sólo con materiales `IS_LINEAR_ELASTIC = True`; y `section` + `n_ip`, con $\mathbf{e} = [\varepsilon_0, \kappa_y, \kappa_z]$ integrado en Gauss-Legendre y torsión elástica desacoplada con el `GJ` de la sección (obligatorio). Contrato: parámetros `section` y `n_ip`; `A`, `Iy`, `Iz`, `J` obligatorios sólo con material; criterios 6 (guarda) y 7 (camino con sección). Corrección de notación en §3, sin efecto en código ni resultados: $\kappa_y = -d^2w/ds^2$ (antes $+d^2w/ds^2$), la definición coherente con $M_y = EI_y\,\kappa_y$, con $M_y > 0 \Rightarrow$ tracción en $z > 0$ (Reglas.md §5) y con la matriz de §8, que ya usaba $w' = -\theta_y$.

- **2026-09-29** · Masa de flexión sin inercia rotatoria (hallazgo clas-4 de la auditoría; **cambio físico aprobado por el usuario**). La consistente sumaba $\frac{\rho I L}{6}\left[\begin{smallmatrix}2&1\\1&2\end{smallmatrix}\right]$ en θ_y (con I_y) y en θ_z (con I_z): cinemática de Timoshenko, mientras que Euler-Bernoulli desprecia la inercia rotatoria por hipótesis. Se retira, por los dos caminos: flexión con la hermítica clásica en los dos planos (en xz, con $w' = -\theta_y$, los acoplamientos traslación-giro de signo opuesto) y concentrada HRZ $\rho A L^3/78$ en θ_y y θ_z en lugar de $\rho I_y L/2$ y $\rho I_z L/2$. La torsión no cambia ($\rho J_p L/6$ y $\rho J_p L/2$, $J_p = I_y + I_z$). Consecuencia: I_y e I_z ya sólo entran en la masa a través de J_p. Contrato: §12 y criterios 8 y 9. Los bloques de flexión se comparten ahora con los marcos planos (`_shared.py`).

- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022, fase 4). `compute_body_load` acepta un campo `b(x)`, proyectado a ejes locales y repartido con las funciones de la rigidez (axial lineal, Hermite en los dos planos con `w' = -θy`); centrada en el eje, la carga no produce torsión. Campo interior (`line_field`: `u`, `v`, `w`, `θx`, `θy`, `θz`, `ε`, `θx'`, `κy`, `κz`) con `section_rigidities` (`EA`, `GJ`, `EIy`, `EIz`). Nueva aceptación «soluciones manufacturadas»: exactitud nodal, órdenes 2, 4, 3, 1 y 2 y torsión exacta (sin torsor distribuido, decisión del usuario, `θx` lineal). Se corrige además una nota desactualizada de `out_of_scope`: la fuerza de cuerpo ya la repartía `compute_body_load`.

Capítulo 4

Secciones de marco

4.1 FiberSection

Orden de trabajo. La especificación física y la formulación numérica las decidió el usuario el 2026-09-29, una cuestión por vez (ver Diálogo); la arquitectura que las aloja está en el ADR 0021.

4.1.1 Especificación física

0. Descripción general

Ley de sección para marcos Euler-Bernoulli en la que las fuerzas internas de la sección (N, M_z en 2D; N, M_y, M_z en 3D) se obtienen **integrando el esfuerzo axial σ_{xx} de cada fibra** sobre la sección transversal. Cada fibra tiene su propio material 1D del catálogo y su propia historia, de modo que la plastificación o el daño avanzan fibra a fibra desde las fibras extremas hacia el eje neutro: **plasticidad distribuida en la sección**. Una misma sección puede mezclar materiales (concreto y acero). La consumen `Frame2DEuler`, `Frame2DEulerCorot` y `Frame3D` (etapa 1); `Frame2DTimoshenko` queda para la etapa 2.

Sustituye la ley anterior de los marcos con material inelástico, que escalaba toda la rigidez local con la E_t de la deformación axial: con fluencia axial el flector se anulaba y con ablandamiento cambiaba de signo (auditoría 2026-09-29).

1. Hipótesis

- **Secciones planas y perpendiculares al eje deformado** (Euler-Bernoulli): $\gamma_{xy} = \gamma_{xz} = 0$ en la cinemática.
- **Fibras uniaxiales**: cada fibra sólo tiene ε_{xx} y σ_{xx}; no hay esfuerzos cortantes en las fibras.
- **Torsión elástica y desacoplada** (3D): $T = GJ\,\theta'_x$, con GJ constante declarado en la sección. No se degrada cuando las fibras fluyen ni interactúa con N, M_y, M_z.
- **Pequeñas deformaciones** en la sección. En `Frame2DEulerCorot` los giros de sólido rígido pueden ser grandes; la sección trabaja en el sistema local corrotado.
- **Sin regularización del ablandamiento** (ver §8).

2. Cinemática de la sección

Coordenadas (y, z) de la fibra i en los ejes locales del elemento, medidas desde el **eje del elemento** (la línea de nodos), no desde el centroide: una sección excéntrica acopla N y M de forma natural. Con la convención de Reglas §5 ($M_z > 0 \Rightarrow$ tracción en $y < 0$; $M_y > 0 \Rightarrow$ tracción en $z > 0$):

- 2D: $\varepsilon_i = \varepsilon_0 - y_i\,\kappa_z$, con $\mathbf{e} = [\varepsilon_0,\ \kappa_z]^\top$.
- 3D: $\varepsilon_i = \varepsilon_0 - y_i\,\kappa_z + z_i\,\kappa_y$, con $\mathbf{e} = [\varepsilon_0,\ \kappa_y,\ \kappa_z]^\top$.

3. Constitutiva por fibra

$(\sigma_i,\ E_{t,i},\ \text{estado}_i') = \text{material}_i.\text{compute_state}(\varepsilon_i,\ \text{estado}_i)$, con cualquier material de `STRAIN_DIM = 1`. Las fibras de material unilateral (`IS_UNILATERAL`, p. ej. `CableMaterial1D`) se admiten mezcladas con fibras bilaterales; una sección hecha sólo de ellas se rechaza al construir, porque al invertirse el signo de la deformación no tendría rigidez y la matriz global quedaría singular (decisión del usuario, 2026-09-30).

Barras dentro de un parche: material desplazado (decisión del usuario, 2026-09-30). Las fibras de un parche cubren toda su área, también el sitio que ocupan las barras colocadas dentro de él. Sin más, en la posición (y_b, z_b) de cada barra de área A_b el área cuenta dos veces:

$$N_b = \big(\sigma_s(\varepsilon_b) + \sigma_c(\varepsilon_b)\big)\,A_b \quad \text{en lugar de} \quad \sigma_s(\varepsilon_b)\,A_b,$$

y lo mismo la rigidez, la masa y el peso propio. Es la práctica de OpenSees y el comportamiento por defecto: el error suele ser pequeño (cuantía de acero del 1 al 3 %, $E_c/E_s \approx 0.15$), pero es sistemático. Una capa de barras o una fibra suelta puede declarar el material que **desplaza** (`displaces`). La sección añade entonces, en la misma posición, una **fibra de compensación** ($y_b,\ z_b,\ -A_b,$ material desplazado), que se evalúa con la ley del material desplazado y con **su propia historia**, como cualquier otra fibra:

$$\sigma_s(\varepsilon_b)\,A_b + \sigma_c(\varepsilon_b)\,(-A_b) \quad \text{en la posición de la barra.}$$

Con ella, el concreto de la sección tiene exactamente el área neta $A_c - \sum A_b$. La barra sustituye al concreto que ocupa en lugar de sumarse a él, y el descuento es exacto con cualquier ley, también en régimen inelástico: resta el concreto que habría en ese punto con su deformación y su historia. Una barra con material efectivo $E_s - E_c$ sólo lo lograría en régimen elástico. El descuento es puntual: resta el área A_b en el punto de la barra, no en la celda del parche que la contiene, con la misma regla del punto medio que el resto de la sección. Condiciones:

- `displaces` es un material de `STRAIN_DIM = 1`, distinto del de la propia barra (con el mismo material la fibra de compensación anularía la barra), y debe ser el material, el mismo objeto, de alguna fibra declarada de la sección (en el YAML, el mismo id): si no, restaría un área que no existe.
- Las áreas negativas sólo existen en las fibras de compensación. Las fibras declaradas deben tener área > 0.
- La guarda de unilaterales cuenta sólo las fibras declaradas: una fibra de compensación resta rigidez, no la aporta.

- El área desplazada de cada material no puede superar el área declarada de ese material (la sección quedaría con área neta negativa, p. ej. $EA < 0$); se rechaza al construir.
- **Responsabilidad del modelo**: que cada barra con `displaces` esté de verdad dentro de un parche de ese material. No se comprueba la pertenencia geométrica; una barra fuera de todo parche restaría material donde no lo hay.

4. Fuerzas internas y tangente de la sección

2D:

$$\mathbf{s} = \begin{bmatrix} N \\ M_z \end{bmatrix} = \sum_i \sigma_i A_i \begin{bmatrix} 1 \\ -y_i \end{bmatrix}, \qquad \mathbf{k}_s = \sum_i E_{t,i} A_i \begin{bmatrix} 1 & -y_i \\ -y_i & y_i^2 \end{bmatrix}.$$

3D:

$$\mathbf{s} = \begin{bmatrix} N \\ M_y \\ M_z \end{bmatrix} = \sum_i \sigma_i A_i \begin{bmatrix} 1 \\ z_i \\ -y_i \end{bmatrix}, \qquad \mathbf{k}_s = \sum_i E_{t,i} A_i \begin{bmatrix} 1 & z_i & -y_i \\ z_i & z_i^2 & -y_i z_i \\ -y_i & -y_i z_i & y_i^2 \end{bmatrix}.$$

Son las integrales $N = \int \sigma \, dA$, $M_y = \int z \, \sigma \, dA$, $M_z = -\int y \, \sigma \, dA$ de Reglas §5 aproximadas por la regla del punto medio sobre las fibras. $\mathbf{k}_s = \partial \mathbf{s} / \partial \mathbf{e}$ es exacta respecto de esa aproximación y simétrica para cualquier valor de $E_{t,i}$. En régimen elástico con sección simétrica respecto del eje, $\mathbf{k}_s = \mathrm{diag}(EA,\ EI)$ (2D) o $\mathrm{diag}(EA,\ EI_y,\ EI_z)$ (3D).

Las sumas recorren todas las fibras, también las de compensación de §3, con $A_i = -A_b < 0$. En régimen elástico, con concreto y acero:

$$\mathbf{k}_s = E_c \sum_{\text{celdas}} A_k \, \mathbf{w}_k \mathbf{w}_k^\top + (E_s - E_c) \sum_{\text{barras}} A_b \, \mathbf{w}_b \mathbf{w}_b^\top, \qquad EA = E_c\Big(A_c - \sum A_b\Big) + E_s \sum A_b.$$

5. Elemento en desplazamientos

Interpolación de los marcos actuales: axial lineal y transversal cúbica de Hermite. En el sistema local, con $x \in [0, L]$:

- $\varepsilon_0 = (u_2 - u_1)/L$, constante en el elemento.
- $\kappa_z(x) = N_1'' \, v_1 + N_2'' \, \theta_{z1} + N_3'' \, v_2 + N_4'' \, \theta_{z2}$, con $N_1'' = (12x/L - 6)/L^2$, $N_2'' = (6x/L - 4)/L$, $N_3'' = (6 - 12x/L)/L^2$, $N_4'' = (6x/L - 2)/L$.
- 3D, plano xz ($w' = -\theta_y$ por la regla de la mano derecha): $\kappa_y(x) = -w'' = -N_1'' \, w_1 + N_2'' \, \theta_{y1} - N_3'' \, w_2 + N_4'' \, \theta_{y2}$.
- 3D, torsión: $\theta_x' = (\theta_{x2} - \theta_{x1})/L$, $T = GJ \, \theta_x'$.

Con $\mathbf{e}(x_g) = \mathbf{B}_g \, \mathbf{u}_{\text{local}}$ en los puntos de Gauss-Legendre x_g de pesos w_g:

$$\mathbf{F}_{\text{int,local}} = \sum_{g=1}^{n_{ip}} w_g \, \tfrac{L}{2} \, \mathbf{B}_g^\top \, \mathbf{s}_g \ (+ \text{ torsión}), \qquad \mathbf{K}_{\text{local}} = \sum_{g=1}^{n_{ip}} w_g \, \tfrac{L}{2} \, \mathbf{B}_g^\top \, \mathbf{k}_{s,g} \, \mathbf{B}_g \ (+ \, GJ/L \text{ en los DOF de torsión}).$$

- **Número de puntos**: $n_{ip} = 3$ por defecto, mínimo 2. Con 2 puntos la rigidez elástica se integra exactamente (el integrando es cuadrático) y se reproduce el marco elástico en forma cerrada; con 1 punto el modo de flexión antisimétrica no tiene energía.

- **Corrotacional** (`Frame2DEulerCorot`): la sección trabaja en el sistema básico $\mathbf{v} = [\Delta l,\ \bar{\theta}_1,\ \bar{\theta}_2]$ del elemento corrotado. $\varepsilon_0 = \Delta l / L_0$, $\kappa_z(x) = [(6x/L_0 - 4)\,\bar{\theta}_1 + (6x/L_0 - 2)\,\bar{\theta}_2]/L_0$; las fuerzas básicas $\mathbf{q} = [N,\ M_1,\ M_2]$ y su tangente se integran igual y se transforman con la cinemática corrotacional existente (incluida su rigidez geométrica con N, M_1, M_2).

6. Masa y peso propio

$\rho A = \sum_i \rho_i A_i$, $\rho I_z = \sum_i \rho_i y_i^2 A_i$, $\rho I_y = \sum_i \rho_i z_i^2 A_i$, con las mismas fórmulas de masa consistente y concentrada de los marcos elásticos (Frame2DEuler §13, Frame3D §12). Los marcos Euler-Bernoulli no llevan inercia rotatoria de flexión, así que su masa sólo usa ρA; ρI_y y ρI_z entran únicamente en la inercia torsional de `Frame3D`, $\rho J_p = \rho I_y + \rho I_z$. El peso propio usa ρA. Se desprecia el acoplamiento traslación-rotación de una distribución de masa excéntrica ($\sum \rho_i y_i A_i \neq 0$). Las sumas incluyen las fibras de compensación de §3: con `displaces`, $\rho A = \rho_c\,(A_c - \sum A_b) + \rho_s \sum A_b$, y el área bruta $\sum A_i$ que usa la fuerza de cuerpo es la de la sección, A_c. Sin `displaces` las barras se suman al concreto también en la masa y en el área bruta.

7. Límites de la formulación

- **Curvatura lineal y ε_0 constante** dentro del elemento: una rótula plástica se reparte sobre todo el elemento y hacen falta varios elementos por miembro cerca de las rótulas. Cuando el eje neutro se desplaza, N en los puntos de Gauss sólo cumple el equilibrio axial en promedio.

- **Torsión no degradada** y sin interacción con la flexión: del lado no conservador cuando la torsión es relevante.

- **Sin deformación por cortante** (etapa 2: `Frame2DTimoshenko`).

8. Ablandamiento (sin regularización)

Con fibras en ablandamiento, el daño se localiza en uno o dos puntos de Gauss de un elemento y la energía disipada es proporcional a $l_c = w_g L/2$: la respuesta posterior al pico depende de la malla y de n_{ip}. Como el daño continuo de los sólidos, que tampoco está regularizado. La regularización genérica (longitud característica que cada elemento entrega a todo material con ablandamiento) es deuda registrada en STATUS (deuda #29).

4.1.2 Contrato

```yaml
name: FiberSection
kind: section
status: validated

interface:
  deformations_2d: [eps0, kappa_z]
  forces_2d: [N, Mz]
  deformations_3d: [eps0, kappa_y, kappa_z]
```

```
  forces_3d: [N, My, Mz]
  torsion: "elástica desacoplada, T = GJ·θx' (sólo 3D)"
  consumers: [Frame2DEuler, Frame2DEulerCorot, Frame3D]

admission:
  textbooks:
    - bathe: "vigas isoparamétricas con integración numérica de σ sobre la sección para
        no linealidad material"
    - belytschko_liu_moran: "vigas y láminas con cuadratura a través del espesor para
      plasticidad"
  general_codes:
    - opensees: "dispBeamColumn con section Fiber (patch, layer, fiber); -GJ para la
      torsión 3D"
    - abaqus: "vigas B2x/B3x con integración numérica de la sección"

parameters:
  - { name: patches, type: list, required: false, desc: "Parches rectangulares (y, z,
    ny, nz, material), circulares (centro, radios, n_rad, n_circ, material), capas de
     barras (extremos, n, área, material y, opcional, displaces: material desplazado,
     §3)" }
  - { name: fibers, type: list, required: false, desc: "Fibras sueltas (y, z, A,
    material y, opcional, displaces)" }
  - { name: GJ, type: float, required: false, desc: "Rigidez torsional elástica;
    obligatoria si la sección la usa un Frame3D" }

element_parameters:
  - { name: section, type: "id de sections", desc: "Excluyente con material +
    propiedades geométricas" }
  - { name: n_ip, type: int, default: 3, min: 2, desc: "Puntos de Gauss-Legendre a lo
    largo del elemento" }

acceptance:
  - name: reproducción elástica
    setup: "fibras elásticas; los tres marcos con n_ip = 2 frente a su versión en forma
       cerrada con EA y EI (EIy, EIz) integrados por las fibras"
    expect: "K y F_int iguales; EI de n capas = b h^3/12 · (1 - 1/n^2)"
    tol_rel: 1.0e-10

  - name: -momentocurvatura, sección rectangular elastoplástica perfecta
    setup: "Elastoplastic1D con H = 0, N = 0, curvatura uniforme impuesta; 2D y los dos
       ejes de Frame3D"
    expect: "M_y = σ_y b h^2/6, M_p = σ_y b h^2/4, κ_y = σ2_y/(E h), Mκ() = M_p [1 -κ (_y
      κ/)^2/3] para κ≥κ  _y"
    tol_rel: "del orden del error de discretización de las fibras, O(1/n^2)"

  - name: interacción -axialflector
    setup: "N impuesta, curvatura creciente"
    expect: "M → M_p [1 - (N/N_p)^2], N_p = σ_y b h"

  - name: colapso de un voladizo con carga en el extremo
    setup: "mallas de 4, 8, 16 y 32 elementos, control por desplazamiento"
    expect: "carga máxima decreciente con la malla y convergente a P_c = M_p/L"

  - name: colapso de una viga biempotrada con carga puntual central
    setup: "mallas crecientes, control por desplazamiento"
```

```
    expect: "carga máxima decreciente con la malla y convergente a P_c = 8 M_p/L (ró
        tulas en empotramientos y centro)"

  - name: sección circular en flexión biaxial a 45°
    setup: "Frame3D, sección circular maciza de diámetro d, curvatura impuesta a 45° de
         los ejes"
    expect: "misma curva -κM que en flexión uniaxial; M_y = πσ _y d^3/32, M_p = σ_y d^3/6
        "

  - name: convenciones de signo
    setup: "curvaturas positivas en cada eje"
    expect: "Mz > 0 tracciona las fibras con y < 0; My > 0 las fibras con z > 0 (Reglas
         §5)"

  - name: corrotacional elástico con giros grandes
    setup: "Frame2DEulerCorot con fibras elásticas frente a la versión con A e I
        integrados"
    expect: "misma respuesta"
    tol_rel: 1.0e-10

references:
  - "Spacone E., Filippou F.C., Taucer F.F. (1996). Fibre -beamcolumn model for non-
      linear analysis of R/C frames: Part I. Formulation. Earthquake Engineering and
      Structural Dynamics -25:711725."
  - "Bathe K.-J. (2014). Finite Element Procedures, 2.ª ed., cap. 6 (elementos de viga
      con no linealidad material)."
  - "Neal B.G. (1977). The Plastic Methods of Structural Analysis, 3.ª ed. (-κM de la
      sección rectangular, interacción -NM, cargas de colapso)."
```

4.1.3 Implementación

- **Archivo**: solidum/sections/fiber.py — la sección y sus primitivas de discretización; el `__init__.py` del paquete solidum/sections/ las reexporta (`from solidum.sections import FiberSection, RectangularPatch, CircularPatch, BarLayer, Fiber`). El paquete `solidum/sections/` lo recorre el descubrimiento automático como `materials/` y `elements/`.
- **Clase**: `FiberSection`, registrada en `SectionRegistry` (familia YAML `sections`, `SPEC_KIND = "section"`, solidum/registry.py) y subclase de la base abstracta `Section` (solidum/core/section.py), que fija el contrato que consumen los marcos: `response_2d`, `response_3d`, `strains_2d`, `strains_3d`, `initial_states`, `materials`, `mass_properties`, `total_area` y el atributo `GJ`, más el método opcional `material_point_states` (por defecto, todos los estados) que usa el post-proceso. Primitivas: `RectangularPatch`, `CircularPatch`, `BarLayer`, `Fiber`.
- **Consumidores**: la constitutiva común de los marcos vive en solidum/elements/frame/_section.py (`FrameSectionMixin`: validación de los dos caminos, guarda de material elástico lineal, estado por punto de Gauss, integración $K = \Sigma\ w_g\ B^\top\ k_s\ B$, $F = \Sigma\ w_g\ B^\top\ s$ y evaluación sin huella en el trial). Cada marco aporta su cinemática: euler.py (`B_g` 2×6), euler_corot.py (`B_g` 2×3 del sistema básico) y frame3d.py (`B_g` 3×12 más la torsión).
- **Piezas genéricas que añade** (ADR 0021 §2 y §4): el gancho `from_yaml(data, resolve)` del lector YAML (solidum/utils/yaml_parser.py, `_parse_families` y `_resolver`), `Material.IS_LINEAR_ELASTIC` y `Element.materials()`.

- **Tests**:
- tests/validation/test_fiber_section_validation.py — los ocho casos de `acceptance`, escritos a ciegas desde esta spec (43 tests):

1. reproducción elástica — `test_caso1_capas_en_el_centro_y_EI_de_n_capas`, `test_caso1_rigidez_de_seccion_elastica`, `test_caso1_reproduccion_elastica_marcos_2d` y `test_caso1_reproduccion_elastica_frame3d` (sección rectangular y mixta concreto + acero; `n_ip = 2` y 3; elementos oblicuos);
2. momento–curvatura — `test_caso2_momento_curvatura_seccion` y `test_caso2_momento_curvatura_elemento` (2D y los dos ejes de `Frame3D`, con descarga elástica que prueba la historia de las fibras);
3. interacción axial–flector — `test_caso3_interaccion_axial_flector` (cinco combinaciones de eje y `N/N_p`, en tracción y en compresión);
4. colapso del voladizo — `test_caso4_colapso_voladizo_carga_en_el_extremo`;
5. colapso de la viga biempotrada — `test_caso5_colapso_viga_biempotrada_carga_central`;
6. sección circular a 45° — `test_caso6_disposicion_de_la_seccion_circular`, `test_caso6_momento_curvatura_circular_uniaxial_y_45` y `test_caso6_frame3d_flexion_biaxial_45`;
7. convenciones de signo — `test_caso7_signos_deformaciones_de_fibra` y `test_caso7_fibra_excentrica_fuerzas_internas_y_tangente`;
8. corrotacional con giros grandes — `test_caso8_corotacional_elemento_giro_grande` ($\alpha = 60°, 150°, -120°$) y `test_caso8_corotacional_arco_de_Bathe_Bolourchi` (voladizo enrollado hasta $\theta = \pi/2$).

- tests/test_fiber_section.py — pruebas unitarias de la implementación (155 tests): construcción y validaciones de la sección y de las primitivas, cinemática y signos, tangente frente a diferencias finitas en régimen inelástico (sección mixta con daño en ablandamiento y acero con endurecimiento; los tres marcos, el corrotacional con giro grande y `Frame3D` con torsión), guardas de los marcos, estado por punto de Gauss, masa y peso propio, simetría agregada (ADR 0003), YAML de extremo a extremo y exportación VTK. El material desplazado de §3 (`TestDisplaces`, `TestDisplacesYaml`, 25 tests): $\mathbf{k}_s$ elástica frente a la forma a mano con el concreto neto; $\mathbf{s}$ y $\mathbf{k}_s$ inelásticas, en 2D y 3D, frente a la sección sin doble conteo sumada a mano fibra a fibra a lo largo de carga, descarga, recarga e inversión, y frente a la sección que discretiza el concreto sin la zona de las barras; masa y área bruta; YAML y sus errores, incluido un id booleano (`displaces: true`); bit a bit sin `displaces`; guarda de unilaterales; `Internal_State` del VTK.

- **Notas de traducción**:
- **Forma matricial de §4.** Con `W` la matriz de filas $\mathbf{w}_i^\top = [1,\ -y_i]$ (2D) o $[1,\ z_i,\ -y_i]$ (3D): $\boldsymbol{\varepsilon} = \mathbf{W}\mathbf{e}$, $\mathbf{s} = \mathbf{W}^\top(\boldsymbol{\sigma} \circ \mathbf{A})$ y $\mathbf{k}_s = \mathbf{W}^\top \operatorname{diag}(\mathbf{E}_t \circ \mathbf{A})\,\mathbf{W}$. `W` se construye una vez al crear la sección. $\mathbf{k}_s$ se simetriza con $\frac{1}{2}(\mathbf{k}_s + \mathbf{k}_s^\top)$ para eliminar el redondeo del producto; la matriz local del elemento, igual.

- **Sin traducción de signos.** La cinemática de §2 es la de Reglas.md §5 tal cual; la API pública 2D (`internal_forces`) sigue exponiendo `V` con el signo de viga estructural a través de la misma traducción de siempre (`_frame2d_forces_from_local`), que no depende del camino.

- **Sección sin estado.** `initial_states()` devuelve una lista nueva de `None` (estado virgen de cada material) y `response_*` devuelve la lista de estados trial sin tocar la recibida. En el elemento, `state.vars[g]` guarda la lista de estados de las fibras del punto `g` y `state.stresses[g]` las fuerzas internas $\mathbf{s}_g$; `N_INTEGRATION_POINTS = n_ip` es atributo de instancia (la clase conserva `1`, el valor del camino con material).

- **Evaluación agrupada por material** (ADR 0021 §6): las fibras se agrupan por identidad del material y se evalúan con un bucle escalar sobre `compute_state`. Los marcos no van por el camino por lotes del ADR 0014; se vectorizará si la medición lo justifica.

- **Cuadratura.** `gauss_legendre_unit(n)` da los puntos $\xi_g \in (0,1)$ y pesos que suman 1, de modo que $\int_0^L f\,dx \approx L\sum_g w_g f(\xi_g L)$: es la regla de §5 con $w_g^{(0,1)} = w_g^{(-1,1)}/2$.

- **Corrotacional.** $\mathbf{B}_g$ (2×3) del sistema básico: $\varepsilon_0 = \Delta l/L_0$, $\kappa_z = [(6\xi-4)\,\bar\theta_1 + (6\xi-2)\,\bar\theta_2]/L_0$. Las fuerzas básicas $\mathbf{q} = [N, M_1, M_2]$ y su tangente entran en la cinemática corrotacional existente sin cambios, incluida la rigidez geométrica.

- **Frame3D.** La fila de κ_y de $\mathbf{B}_g$ es $[-N_1'',\ N_2'',\ -N_3'',\ N_4'']$ sobre $(w_1, \theta_{y1}, w_2, \theta_{y2})$; la torsión suma GJ/L en los DOF θ_x y $T = GJ\,(\theta_{x2} - \theta_{x1})/L$. Una sección sin `GJ` se rechaza al construir el elemento.

- **CircularPatch.** División radial en `n_rad` anillos de **igual espesor** $\Delta r = (r_{\text{ext}} - r_{\text{int}})/n_{\text{rad}}$ (como el `patch circ` de OpenSees) y angular en `n_circ` sectores iguales. Área exacta del sector $A = \frac{\Delta\theta}{2}(r_2^2 - r_1^2)$ y centroide a $r_c = \frac{2}{3}\frac{\sin\alpha}{\alpha}\frac{r_2^3 - r_1^3}{r_2^2 - r_1^2}$ del centro sobre la bisectriz ($\alpha = \Delta\theta/2$); ángulos desde $+y$ hacia $+z$, primer sector en 0. Con `n_circ` múltiplo de 8 la sección discreta es invariante ante un giro de 45°, que es lo que hace exacto el caso 6.

- **Masa y peso propio** (§6). `mass_properties()` devuelve `rhoA`, `rhoIy`, `rhoIz` respecto del eje del elemento y lanza `ValueError` con los materiales sin `density`; los marcos llaman a las mismas funciones de masa consistente y concentrada del camino con material, que reciben la densidad lineal ρA, y `Frame3D` además $\rho J_p = \rho I_y + \rho I_z$ para la torsión. La fuerza de cuerpo uniforme usa el área bruta $\sum A_i$ (`total_area()`); el peso propio, ρA.

- **Post-proceso.** `internal_forces` y `compute_internal_forces` evalúan el elemento sobre una instantánea del trial y la restauran: no son un iterado del paso. El diccionario legado añade `section_deformations` y `section_forces`, de forma `(n_ip, 2)` o `(n_ip, 3)`. En VTK, `Internal_State` promedia la `PRIMARY_STATE_VAR` sobre fibras y puntos de Gauss.

- **YAML.** Cada parche declara `shape` ∈ `rectangle`, `circle`, `layer` y sus parámetros son los del constructor de la primitiva (se validan por introspección: una clave desconocida es error). Los errores nombran el contexto, p. ej. `sections[0] (id=7), patches[0].material: referencia a material inexistente (9)`. `displaces` es, como `material`, un id de `materials` que `from_yaml` resuelve con `resolve` (contexto `patches[i].displaces` o `fibers[i].displaces`); sólo lo aceptan `layer` y las fibras sueltas. `resolve` rechaza un id

booleano: sin esa guarda, `displaces: true` (leído como bandera) resolvería en silencio al material de id 1, porque en Python `True == 1`.

- **Fibras de compensación** (§3). `BarLayer(material, start, end, n, area, displaces=None)` y `Fiber(y, z, area, material, displaces=None)` guardan el material desplazado; la sección lee el atributo `displaces` de cada primitiva (cualquier primitiva futura puede declararlo). Las fibras de compensación van **al final** de `y`, `z`, `area` y de la lista de estados, tras todas las fibras declaradas y en el orden de las barras que compensan. `is_compensation` las marca y `compensated_index` da la fibra declarada que compensa cada una. Entran en el grupo de evaluación de su material, que ya es el de alguna fibra declarada, así que `materials()` no cambia. Sin `displaces` los arreglos, los grupos y la aritmética son los de antes, bit a bit. `n_fibers`, `initial_states()` y `strains_*` cuentan todas las fibras. `material_point_states(states)` devuelve sólo los estados de las fibras declaradas: el `Internal_State` del VTK no promedia las fibras de compensación, que no son puntos materiales.

4.1.4 Resultados de validación

Datos comunes: $E = 200$ GPa, $\sigma_y = 250$ MPa, $b = 0.3$ m, $h = 0.5$ m, $L = 3$ m. Valores medidos el 2026-09-29.

Caso	Resultado	Referencia
1. Reproducción elástica	K y F_{int} de los tres marcos iguales a la forma cerrada a 1e-10 con `n_ip` = 2 y 3; EI de 10 capas igual a $Ebh^3/12\,(1-1/n^2)$ a redondeo	forma cerrada
1. M–κ rectangular, 200 capas	máx$\lvert\Delta M\rvert/M_p$ = 1.7e-5 en los tres ejes (cota $\frac{5}{3}/n^2$ = 4.2e-5); M con todas las fibras plastificadas = M_p a 2e-15	Neal (1977)
1. Interacción N–M	máx$\lvert\Delta M\rvert/M_p$ = 1.7e-5 (cota $\frac{8}{3}/n^2$ = 6.7e-5); en $\kappa = 100\,\kappa_y$ y $N/N_p = \pm 0.3$, M/M_p = 0.90995 frente a la forma cerrada 0.90997 y al límite plástico $1-(N/N_p)^2 = 0.91$ (con 0.6: 0.63995, 0.63997 y 0.64)	Neal (1977)
1. Voladizo, P_{max}/P_c	1.0710, 1.0351, 1.0176, 1.0088 con 4, 8, 16, 32 elementos (cota del modelo discreto 1.0767, 1.0369, 1.0181, 1.0090)	$P_c = M_p/L$
1. Biempotrada, P_{max}/P_c	1.3475, 1.1496, 1.0710, 1.0351 con 4, 8, 16, 32 elementos (cota 1.3983, 1.1661, 1.0767, 1.0369)	$P_c = 8M_p/L$
1. Circular, 20 × 64 sectores	máx$\lvert\Delta M\rvert/M_p$ = 7.2e-4 (cota 6.0e-3); a 45°, $M_y = M_z$ y $\lvert\mathbf{M}\rvert$ igual a la uniaxial a 1e-9 M_p, en la sección y en `Frame3D`	$M_y = \pi\sigma_y d^3/32$, $M_p = \sigma_y d^3/6$
1. Signos	$M_z > 0$ tracciona $y < 0$; $M_y > 0$ tracciona $z > 0$; fibra excéntrica con $\mathbf{s}$ y $\mathbf{k}_s$ exactos	Reglas.md §5
1. Corrotacional elástico	K y F_{int} iguales a 1e-10 con giros de 60°, 150° y −120°; arco de Bathe-Bolourchi hasta $\theta = \pi/2$ con $\lVert\Delta U\rVert/\lVert U\rVert \le$ 1e-10	versión con A e I integrados

Los casos 4 y 5 sobreestiman la carga de colapso en primer orden del tamaño del elemento (spec §7): el elemento en desplazamientos evalúa la sección en los puntos de Gauss, nunca en el extremo, y la rótula se reparte hacia el interior del elemento, con un brazo efectivo menor. La cota del modelo discreto la deriva el test por análisis de límite exacto del elemento raíz ($P_\infty/P_c = 1.398/(1.398 - 0.398/n)$ en el voladizo). En la biempotrada cada cuarto de vano es un voladizo de $n/4$ elementos, por eso sus cifras con n elementos repiten las del voladizo con $n/4$.

4.1.5 Diálogo

- **2026-09-29** · Decisiones del usuario, una por vez: (1) sección por fibras en lugar de restringir los marcos a material elástico; (2) elemento en desplazamientos, coherente con Reglas §0, no en fuerzas; (3) etapa 1 con los tres marcos Euler-Bernoulli, Timoshenko en una etapa 2 con su propia cinemática; (4) Gauss-Legendre, 3 puntos por defecto y mínimo 2; (5) torsión elástica desacoplada con GJ declarado en la sección; (6) sin regularización del ablandamiento en esta etapa, con la regularización genérica registrada como deuda; (7) los ocho casos de validación del contrato.

- **2026-09-29** · Implementada y validada: los ocho casos de `acceptance` pasan (43 tests en `tests/validation/test_fiber_section_validation.py`, escritos a ciegas desde esta spec, más 116 pruebas unitarias en `tests/test_fiber_section.py`); `status: validated`. Cifras en *Resultados de validación*. Dos observaciones para el usuario, ninguna es un defecto: (a) los casos 4 y 5 confirman §7 con números — la carga de colapso converge por arriba y en primer orden del tamaño del elemento, así que con pocos elementos por miembro la sobreestimación no es despreciable (biempotrada con 4 elementos en el vano: +35 %; con 32: +3.5 %); (b) la tolerancia del caso 2 (`tol_rel` "del orden de O($1/n^2$)") quedó fijada en el test como la cota $\frac{5}{3}M_p/n^2$ derivada para la regla del punto medio, y la del caso 3 como $\frac{8}{3}M_p/n^2$; los casos 4 y 5 se comprueban contra la cota de análisis de límite del modelo discreto y contra la predicción de la estática del elemento raíz con la M–κ analítica. Los cuatro marcos rechazan ahora un material inelástico en el camino con `material` + propiedades geométricas (`IS_LINEAR_ELASTIC`), con un mensaje que remite a esta sección.

- **2026-09-29** · §6: los marcos Euler-Bernoulli dejan de llevar inercia rotatoria de flexión en su masa (cambio físico aprobado por el usuario; ver *Diálogo* de Frame2DEuler y Frame3D). Con sección, la masa sólo usa $\rho A = \sum \rho_i A_i$, más $\rho J_p = \rho I_y + \rho I_z$ en la torsión de `Frame3D`. `mass_properties()` no cambia; la igualdad con el camino con material se verifica en `tests/test_fiber_section.py` · `TestMasa` y en `tests/test_frame_mass.py` · `TestCaminoConSeccion` (sección mixta concreto + acero, con ρA y ρJ_p calculados a mano).

- **2026-09-30** · §3: barras dentro de un parche. Decisión del usuario, opción (b) de tres: (a) dejar el doble conteo y documentarlo; (b) opción `displaces` que añade una fibra de compensación $-A_b$ con el material desplazado, evaluada con su propia ley e historia; (c) descontar siempre de forma automática, lo que exige saber qué parche contiene cada barra y rompe la compatibilidad con OpenSees. Por defecto nada cambia (compatible con OpenSees, verificado bit a bit) y el doble conteo queda documentado. Con `displaces` se comprueba: $\mathbf{k}_s$ elástica igual a la forma a mano con el concreto neto, $EA = E_c(A_c - \sum A_b) + E_s \sum A_b$; en régimen inelástico, concreto con daño y acero elastoplástico a lo largo de carga, descarga, recarga e inversión, $\mathbf{s}$ y $\mathbf{k}_s$ iguales a la suma a mano fibra a fibra sin doble conteo, con un error de redondeo de unas 10^{-3} veces la cota $\gamma_n \sum|\text{términos}|$; y la diferencia con la sección sin `displaces` es exactamente $\sum \sigma_c(\varepsilon_b) A_b \mathbf{w}_b$, del 0.4 al 1 % de $\mathbf{s}$ en ese estado dañado. Una validación que el usuario no pidió de forma explícita: `displaces` debe ser el material (el mismo objeto; en el YAML, el mismo id) de alguna fibra declarada de la sección, porque si no la compensación restaría un área que no existe. Pieza nueva en la base `Section`: el método opcional `material_point_states`, que el exportador VTK usa para no promediar las fibras de compensación en `Internal_State`.

- **2026-09-30** · Revisión adversaria de `displaces`, con scripts propios fuera del repositorio. (1) Historia no monótona (60 pasos con inversiones e iterados sin commit; concreto con daño de ablandamiento fuerte o elastoplástico): con las barras en centros de celda, el estado de cada fibra de compensación es idéntico, bit a bit, al de la celda de concreto en la misma posición en todos los pasos, y $\mathbf{s}$, $\mathbf{k}_s$ coinciden con la sección neta ($A_{\text{celda}} - A_b$) a $\leq 6 \times 10^{-16}$ veces la cota $\sum|\text{términos}|$. (2) $\mathbf{k}_s$ frente a diferencias finitas con todas las fibras de compensación en carga activa ($E_t < 0$): $\leq 3 \times 10^{-10}$ relativo. (3) $\mathbf{K}$ y $\mathbf{F}$ de `Frame2DEuler`, `Frame2DEulerCorot` y `Frame3D` frente a la sección neta a lo largo de una trayectoria no monótona con commits: $\leq 4 \times 10^{-16}$ relativo; frente a diferencias finitas, $\leq 1.5 \times 10^{-7}$. (4) y (5) voladizos con desplazamiento prescrito y `NonlinearSolver`: $\mathbf{U}$, fuerzas internas, VTK (`N`, `V`, `M`, `Internal_State`) y estados coinciden

con el modelo de sección neta; los iterados previos, las fuerzas internas y un paso rechazado no dejan huella. Único defecto encontrado, corregido: el lector YAML resolvía un id booleano como el id 1 (`displaces: true` ⇒ material 1, porque `True == 1`); `resolve` lo rechaza ahora (`TestDisplacesYaml.test_un_booleano_no_es_un_id`).

Capítulo 5

Elementos 2D — Sólidos

5.1 Tri3

Documentación retroactiva del elemento ya implementado.

5.1.1 Especificación física

0. Descripción general

Elemento sólido bidimensional plano de primer orden, triangular de tres nodos. Aproximación lineal C^0 del campo de desplazamientos en coordenadas naturales (ξ, η). Conocido en la literatura como elemento *constant strain triangle* (CST): la matriz $\mathbf{B}$ es constante sobre el elemento, por lo que la deformación $\boldsymbol{\varepsilon}$ y la tensión $\boldsymbol{\sigma}$ son uniformes dentro de cada Tri3. Régimen de pequeños desplazamientos y deformaciones.

1. Cinemática de desplazamientos

$$u_x(\xi,\eta) = \sum_{i=1}^{3} N_i(\xi,\eta)\, u_x^{(i)}, \qquad u_y(\xi,\eta) = \sum_{i=1}^{3} N_i(\xi,\eta)\, u_y^{(i)}.$$

2. Cinemática de deformaciones

Notación Voigt 2D $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$, idéntica a la de `Quad4`. Por la linealidad de N_i, las derivadas $\partial N_i/\partial x$ son constantes y $\boldsymbol{\varepsilon}$ es uniforme en el elemento.

3. Ecuación constitutiva

Delegada al material 2D (`STRAIN_DIM = 3`). Soporta `Elastic2D`, `IsotropicDamage2D`, `VonMises2D` (este último solo `plane_strain`).

4. Equilibrio — forma fuerte

$$-\nabla \cdot \boldsymbol{\sigma} = \mathbf{b} \quad \text{en } \Omega, \quad \boldsymbol{\sigma} \cdot \mathbf{n} = \bar{\mathbf{t}} \text{ en } \partial\Omega_N, \quad \mathbf{u} = \bar{\mathbf{u}} \text{ en } \partial\Omega_D.$$

5. Equilibrio — forma débil

Idéntica a la del `Quad4`; el principio variacional no depende de la forma del elemento.

5.1.2 Formulación numérica (FEM)

6. Discretización

Tres nodos en orden antihorario sobre el plano (x, y). Mapeo isoparamétrico desde el triángulo de referencia con vértices en $(0,0)$, $(1,0)$, $(0,1)$. Jacobiano $\mathbf{J} = \partial(x,y)/\partial(\xi,\eta)$ es constante sobre el elemento; el código aborta con error si $\det \mathbf{J} \leq \text{tol}$ (nodos colineales o invertidos).

7. Funciones de forma

Coordenadas baricéntricas:

$$N_1 = 1 - \xi - \eta, \qquad N_2 = \xi, \qquad N_3 = \eta.$$

Sus derivadas en coordenadas naturales son constantes:

$$\partial N_1/\partial\xi = -1, \quad \partial N_2/\partial\xi = 1, \quad \partial N_3/\partial\xi = 0;$$

$$\partial N_1/\partial\eta = -1, \quad \partial N_2/\partial\eta = 0, \quad \partial N_3/\partial\eta = 1.$$

8. Matriz deformación-desplazamiento

$\mathbf{B}$ tiene tamaño 3×6 y, una vez transformadas las derivadas a globales vía $\mathbf{J}^{-1}$, sus entradas son **constantes** sobre el elemento:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i/\partial x & 0 \\ 0 & \partial N_i/\partial y \\ \partial N_i/\partial y & \partial N_i/\partial x \end{bmatrix}.$$

9. Rigidez elemental

$$\mathbf{K}_e = \mathbf{B}^\top \mathbf{D}\, \mathbf{B}\, A_e\, t,$$

con $A_e = \frac{1}{2} \det \mathbf{J}$ el área del triángulo y t el espesor. La integración es exacta con un único punto central porque $\mathbf{B}$ es constante.

10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \mathbf{B}^\top \boldsymbol{\sigma}\, A_e\, t.$$

11. Cuadratura

Un punto central (en el baricentro) con peso $w = 1/2$ sobre el triángulo de referencia. Es exacto para formas constantes de $\mathbf{B}$ y $\boldsymbol{\sigma}$.

12. Cargas distribuidas consistentes

Fuerza de cuerpo b. Con N_i lineales y $\mathbf{b}$ uniforme la integral es exacta:

$$\mathbf{f}_e^{b} = \int_{\Omega_e} \mathbf{N}^{\top}\,\mathbf{b}\,t\,dA \implies \text{cada nodo recibe } \tfrac{1}{3}\,\mathbf{b}\,A_e\,t.$$

Se integra con la regla de 3 puntos `tri_3` (ADR 0022), exacta también para $\mathbf{b}$ lineal; $\mathbf{b}$ es constante o un campo $\mathbf{b}(\mathbf{x})$.

Tracción de superficie $\bar{\mathbf{t}}$ sobre un borde recto. Con $\bar{\mathbf{t}}$ constante el reparto es $\frac{L}{2}\,\bar{\mathbf{t}}\,t$ en cada uno de los dos nodos del borde. Bordes numerados según la conectividad:

Edge	Nodos
0	(0, 1)
1	(1, 2)
2	(2, 0)

$\bar{\mathbf{t}}$ se especifica en **coordenadas globales**: constante o campo $\bar{\mathbf{t}}(\mathbf{x}, \mathbf{n})$ de la posición y de la normal exterior unitaria del borde (ADR 0022); una presión p es $\bar{\mathbf{t}} = -p\,\mathbf{n}$.

13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve la misma estructura que en `Quad4` pero con un único punto (centroide). Para Tri3 ε y σ son constantes sobre el elemento, así que el dato del punto coincide con el promedio.

5.1.3 Limitación crítica — *shear locking*

El Tri3 es notoriamente rígido en flexión: tres DOFs de desplazamiento no bastan para representar el modo de flexión puro de una viga discretizada en triángulos, por lo que el elemento responde con cortante espurio (*shear locking*) y subestima sistemáticamente la deflexión. La severidad crece con la relación L/h del problema. Recomendación operativa: usar `Quad4` por defecto; reservar `Tri3` para zonas de transición de malla en regiones donde el campo de esfuerzos varía suavemente.

5.1.4 Contrato de implementación

```yaml
name: Tri3
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 3
  strain_dim: 3                 # Voigt 2D ε[_xx, ε_yy, γ_xy]
  n_integration_points: 1

admission:                  # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - zienkiewicz_taylor: "vol. 1 (6.ª ed.), cap. 2: triángulo lineal en elasticidad
        plana"
```

```
    - hughes: "cap. 3 (triángulo lineal)"
  general_codes:
    - abaqus: "CPS3 / CPE3"

parameters:
  - { name: thickness, type: float, required: false, default: 1.0, desc: Espesor para
      estado plano }

material_contract:
  signature: "compute_stateε(, state) -> σ(, C_tangent, state')"
  strain_kind: "Voigt 2D ε[_xx, ε_yy, γ_xy]"

conventions:
  sign: "esfuerzo positivo = tracción"
  voigt: "[xx, yy, xy] con γ_xy = 2·ε_xy"
  node_orientation: "antihorario (asegura det J > 0)"

validity:
  - "pequeños desplazamientos y deformaciones"
  - "campos de esfuerzo que varían suavemente sobre el elemento"

out_of_scope:
  - "flexión dominante (shear locking severo)"
  - "elementos de alto orden (Tri6)"
  - "grandes deformaciones"

acceptance:
  verification:
    - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
      setup: "u* suave no polinómico; cuadrado distorsionado (deformación plana,
          esfuerzo plano, ortótropo a 30°) y cuarto de corona con tracción sobre los
          arcos; mallas n×n, n = 4, 8, 16, 32"
      expect: "orden observado entre los dos últimos niveles ≥ 2 - 0.1 en L² y ≥ 1 -
          0.1 en energía; error decreciente en cada nivel"
      tol_abs: 0.1
    - name: patch_test_macneal_harder
      setup: "4 Tri3 distorsionados alrededor de un nodo interior descentrado
             con campo lineal u = 1e-3·(x + y/2), v = 1e-3·(y + x/2)
             impuesto en los 4 nodos del contorno del cuadrado unitario"
      expect: "nodo interior adopta el campo lineal y ε = (1e-3, 1e-3,
           1e-3) en cada elemento; σ uniforme"
      tol_rel: 1.0e-10
  specific:
    - name: parche de deformación constante sobre malla triangular regular
      setup: "malla triangular con campo de desplazamiento lineal y material Elastic2D"
      expect: "ε constante (CST reproduce campos lineales por construcción)"
      tol_rel: 1.0e-12
    - name: jacobiano degenerado abortado
      setup: "tres nodos colineales"
      expect: "ValueError en compute_kinematics_tri3"
    - name: cargas consistentes - body load uniforme
      setup: "Tri3 con b uniforme"
      expect: "cada nodo recibe (1/3)·b·A_e·t; Σ = b·A_e·t"
      tol_rel: 1.0e-12
    - name: cargas consistentes - tracción uniforme en un borde
      setup: "Tri3 con tracción constante en un borde"
```

```
      expect: "Σf = -t·L·t y reparto L/2 a cada nodo del borde, 0 en el opuesto"
      tol_rel: 1.0e-12

references:
  - "Cook, Malkus, Plesha, Witt, Concepts and Applications of FEA, §3.6 (CST)"
  - "Zienkiewicz O.C., The Finite Element Method, vol. 1, §5 (limitations of CST)"
```

5.1.5 Implementación

- **Archivo**: solidum/elements/solid_2d/tri3.py
- **Clase**: `Tri3` (registrada vía `@ElementRegistry.register`)
- **Función núcleo**: `compute_kinematics_tri3(coords)`, decorada con `@njit`. Reutiliza `compute_integrands` (compartida con Quad4) con peso $w = 0.5$. Ambas son API pública de cinemática (ADR 0020, P6): se importan desde `solidum.elements.solid_2d`, que las reexporta; su código vive en _shared.py, módulo interno que puede reorganizarse.
- **Tests**:
- tests/test_solid_2d.py — patch test sobre malla triangular regular y modos de cuerpo rígido.
- tests/test_patch_solid_2d.py — patch test de MacNeal-Harder con nodo interior libre (NAFEMS, V&V).
- tests/validation/test_mms_solid_2d.py (soluciones manufacturadas), tests/test_distributed_loads_2d.py (cargas variables).

5.1.6 Diálogo

- **2026-04-30** · Spec retroactiva. `Tri3` precedía al protocolo spec-first; la spec se creó documentando la formulación ya implementada. Promovido `status: validated`.
- **2026-04-30** · Añadido patch test de MacNeal-Harder (NAFEMS) en `acceptance.verification`. Cuatro triángulos alrededor de un nodo interior descentrado reproducen un campo lineal impuesto en el contorno con ε constante en cada elemento.
- **2026-05-04** · Expuesta `compute_gauss_state(U)` (1 punto central). El `VtkExporter` consume σ por elemento y promedia a nodos (`Sigma_*_nodal`).
- **2026-05-04** · Añadidas cargas distribuidas consistentes (`compute_body_load`, `compute_edge_traction`) con la misma semántica que `Quad4`. Para Tri3 ambos integrandos son exactos analíticamente: body load reparte 1/3 por nodo, tracción de borde reparte L/2 a cada uno de los dos nodos del borde.
- **2026-09-22** · Auditoría global: tolerancia del jacobiano ahora **relativa** (`det J` $\leq$ `JACOBIAN_RTOL` · $\Pi\|$`fila_i(J)`$\|$, cota de Hadamard, adimensional): la absoluta `1e-10` sobre `det J` rechazaba mallas finas en metros sin distorsión.

- **2026-09-22** · Ensamblaje por lotes (ADR 0014): la cinemática del elemento pasa a un núcleo compilado compartido (`_kin2d_from_dN`) y se expone como `BATCH_KINEMATICS`; `compute_element_state` no cambia de formulación. El `Assembler` agrupa los elementos en familias y los evalúa en un kernel único; equivalencia con el camino por elemento verificada a 1e-14 en `K` y `F_int` sobre todos los materiales compatibles (`tests/test_batch_assembly.py`).

- **2026-09-22** · Cierre del ADR 0014 (§9): la cinemática por lotes (`BATCH_KINEMATICS`) ya **no lanza** con jacobiano degenerado —devuelve `det J` $\leq$ `0` y el ensamblador lanza el `ValueError` con el id del elemento—, porque el kernel de familia tiene ahora variante paralela (`prange`, bit a bit igual a la serie) y una excepción dentro de la región paralela se pierde; el camino por elemento sigue lanzando como siempre. Declara las funciones de forma para el post-proceso por familia (`Family.gauss_state`, mismo resultado que `compute_gauss_state`, verificado en `tests/test_batch_postprocess.py`). Formulación sin cambios.

- **2026-09-29** · Cinemática pública para elementos de usuario (ADR 0020, P6): `_compute_kinematics_tri3` y `_compute_integrands` pasan a llamarse `compute_kinematics_tri3` y `compute_integrands`, y su ruta pública es `solidum.elements.solid_2d` (el módulo `_shared` sigue siendo interno). Sólo cambia el nombre; formulación sin cambios.

- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022). `compute_body_load` acepta un campo `b(x)` y `compute_edge_traction` un campo `t(x, n)`, con `n` la normal exterior unitaria del borde (una presión es `-p·n`). La fuerza de cuerpo se integra con la regla de la masa (`tri_3`; antes el reparto 1/3 exacto sólo para b constante); la de borde, sobre la geometría real del borde con Gauss de 3 puntos. Declara `REFERENCE_CELL` y `REFERENCE_NODES`. Nueva aceptación «soluciones manufacturadas»: orden 2 en L^2 y 1 en energía en el cuadrado distorsionado y en la corona.

5.2 Quad4

Documentación retroactiva del elemento ya implementado. Cubre la formulación, el contrato de implementación y la trazabilidad con tests existentes.

5.2.1 Especificación física

0. Descripción general

Elemento sólido bidimensional plano de primer orden, cuadrilátero de cuatro nodos. Aproximación bilineal C^0 del campo de desplazamientos en coordenadas naturales $(\xi, \eta) \in [-1, 1]^2$. Formulación isoparamétrica: la geometría y los desplazamientos comparten las mismas funciones de forma. Régimen de pequeños desplazamientos y deformaciones.

1. Cinemática de desplazamientos

$$u_x(\xi, \eta) = \sum_{i=1}^{4} N_i(\xi, \eta)\, u_x^{(i)}, \qquad u_y(\xi, \eta) = \sum_{i=1}^{4} N_i(\xi, \eta)\, u_y^{(i)}.$$

2. Cinemática de deformaciones

Tensor infinitesimal en notación Voigt 2D $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$:

$$\varepsilon_{xx} = \frac{\partial u_x}{\partial x}, \quad \varepsilon_{yy} = \frac{\partial u_y}{\partial y}, \quad \gamma_{xy} = \frac{\partial u_x}{\partial y} + \frac{\partial u_y}{\partial x}.$$

3. Ecuación constitutiva

Delegada al material 2D (`STRAIN_DIM = 3`). El elemento llama `material.compute_state(`ε`, state)` $\rightarrow$ `(`σ`, C_tangent, state')` por punto de Gauss y soporta cualquier ley constitutiva 2D (`Elastic2D`, `IsotropicDamage2D`, `VonMises2D`).

4. Equilibrio — forma fuerte

$$-\nabla \cdot \boldsymbol{\sigma} = \mathbf{b} \quad \text{en } \Omega, \qquad \boldsymbol{\sigma} \cdot \mathbf{n} = \bar{\mathbf{t}} \quad \text{en } \partial\Omega_N, \qquad \mathbf{u} = \bar{\mathbf{u}} \quad \text{en } \partial\Omega_D.$$

5. Equilibrio — forma débil

$$\int_\Omega \delta\boldsymbol{\varepsilon}^\top \boldsymbol{\sigma}\, dV = \int_\Omega \delta\mathbf{u}^\top \mathbf{b}\, dV + \int_{\partial\Omega_N} \delta\mathbf{u}^\top \bar{\mathbf{t}}\, dS, \qquad \forall\, \delta\mathbf{u} \in V_0.$$

5.2.2 Formulación numérica (FEM)

6. Discretización

Cuatro nodos en orden antihorario sobre el plano (x, y). Mapeo isoparamétrico desde el cuadrado de referencia $[-1, 1]^2$:

$$x(\xi,\eta) = \sum_{i=1}^{4} N_i(\xi,\eta)\, x^{(i)}, \qquad y(\xi,\eta) = \sum_{i=1}^{4} N_i(\xi,\eta)\, y^{(i)}.$$

Jacobiano $\mathbf{J} = \partial(x,y)/\partial(\xi,\eta)$ y su determinante $\det \mathbf{J}$ entran en el cambio de variable de la integración. El código aborta con error si $\det \mathbf{J} \le \text{tol}$ (elemento degenerado o nodos en orden invertido).

7. Funciones de forma

Producto tensorial bilineal:

$$N_i(\xi,\eta) = \tfrac{1}{4}(1+\xi_i\,\xi)(1+\eta_i\,\eta), \qquad (\xi_i,\eta_i) = (\pm 1, \pm 1).$$

Convención de numeración (antihorario): $(\xi_1,\eta_1) = (-1,-1)$, $(\xi_2,\eta_2) = (+1,-1)$, $(\xi_3,\eta_3) = (+1,+1)$, $(\xi_4,\eta_4) = (-1,+1)$.

8. Matriz deformación-desplazamiento

Las derivadas en globales se obtienen vía $\partial N_i/\partial x = \mathbf{J}^{-1}\,\partial N_i/\partial \xi$. La matriz $\mathbf{B}$ de tamaño 3×8 se ensambla por nodo:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i/\partial x & 0 \\ 0 & \partial N_i/\partial y \\ \partial N_i/\partial y & \partial N_i/\partial x \end{bmatrix}, \qquad \boldsymbol{\varepsilon} = \mathbf{B}\,\mathbf{u}_e.$$

9. Rigidez elemental

$$\mathbf{K}_e = \int_\Omega \mathbf{B}^\top \mathbf{D}\,\mathbf{B}\, t\, dA = \sum_{p=1}^{n_g} \mathbf{B}(\xi_p,\eta_p)^\top\, \mathbf{D}_p\, \mathbf{B}(\xi_p,\eta_p)\, \det \mathbf{J}_p\, w_p\, t,$$

donde t es el espesor (`thickness`) y $\mathbf{D}_p$ es el tensor constitutivo tangente devuelto por el material en cada punto.

10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sum_{p=1}^{n_g} \mathbf{B}(\xi_p,\eta_p)^\top\, \boldsymbol{\sigma}_p\, \det \mathbf{J}_p\, w_p\, t.$$

11. Cuadratura

Por defecto Gauss–Legendre 2×2 (cuatro puntos): orden de exactitud 3 — integra exactamente $\mathbf{K}_e$ para Jacobiano constante (geometría de paralelogramo) y captura el comportamiento dominante en geometrías irregulares moderadas. Soporta esquemas alternativos vía el parámetro `quadrature` (lista nombrada en `QuadratureRegistry`).

Aviso sobre integración reducida: el esquema 1×1 está disponible pero el código emite advertencia explícita; un único punto central no detecta los modos de hourglass (energía nula con desplazamientos de signo alternado), que pueden contaminar la respuesta sin estabilización adicional. No se implementa estabilización tipo Flanagan-Belytschko.

12. Cargas distribuidas consistentes

Fuerza de cuerpo $\mathbf{b}$ (e.g. peso propio $\mathbf{b} = (0, -\rho g)$, fuerza centrífuga). Vector nodal equivalente:

$$\mathbf{f}_e^{\mathbf{b}} = \int_{\Omega_e} \mathbf{N}^\top \mathbf{b}\, t\, dA = \sum_{p=1}^{n_g} \mathbf{N}(\xi_p, \eta_p)^\top \mathbf{b}\, \det \mathbf{J}_p\, w_p\, t,$$

donde $\mathbf{N}$ es la matriz 2×8 de funciones de forma. Se integra con la regla 2×2 de la masa consistente aunque la rigidez use 1×1 (ADR 0022). $\mathbf{b}$ es constante o un campo $\mathbf{b}(\mathbf{x})$ evaluado en cada punto de Gauss.

Tracción de superficie $\bar{\mathbf{t}}$ sobre un borde $\Gamma_e \subset \partial\Omega_N$:

$$\mathbf{f}_e^{\mathbf{t}} = \int_{\Gamma_e} \mathbf{N}^\top \bar{\mathbf{t}}\, t\, dS.$$

Cada borde es una recta entre dos nodos; la integral se evalúa con Gauss de 3 puntos sobre el borde. Para $\bar{\mathbf{t}}$ constante da el reparto exacto $\frac{L}{2}\,\bar{\mathbf{t}}\, t$ a cada uno de los dos nodos del borde, donde L es la longitud del segmento. Bordes numerados según la conectividad nodal:

Edge	Nodos
0	(0, 1)
1	(1, 2)
2	(2, 3)
3	(3, 0)

$\bar{\mathbf{t}}$ se especifica en **coordenadas globales**: constante o campo $\bar{\mathbf{t}}(\mathbf{x}, \mathbf{n})$ de la posición y de la normal exterior unitaria del borde (ADR 0022); una presión p es $\bar{\mathbf{t}} = -p\,\mathbf{n}$.

13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve un dict con $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ en cada uno de los n_g puntos de Gauss del elemento, junto con sus coordenadas naturales y globales. Habilita post-proceso fino (mapas no promediados, suavizado nodal, extrapolación tipo Barlow). `compute_internal_forces` queda como atajo de promedio.

5.2.3 Contrato de implementación

```yaml
name: Quad4
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 4
  strain_dim: 3                 # Voigt 2D ε[_xx, ε_yy, γ_xy]
  n_integration_points: 4       # default Gauss 2×2

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 3 (cuadrilátero bilineal isoparamétrico)"
    - bathe: "§5.3 (elementos continuos isoparamétricos)"
  general_codes:
    - abaqus: "CPS4 / CPE4 (integración completa)"

parameters:
  - { name: thickness, type: float, required: false, default: 1.0, desc: Espesor para
      estado plano }
  - { name: quadrature, type: tuple, required: false, default: "2x2", desc: Regla de
      cuadratura desde QuadratureRegistry }

material_contract:
  signature: "compute_stateε(, state) -> σ(, C_tangent, state')"
  strain_kind: "Voigt 2D ε[_xx, ε_yy, γ_xy]"

conventions:
  sign: "esfuerzo positivo = tracción"
  voigt: "[xx, yy, xy] con γ_xy = 2·ε_xy (engineering shear strain)"
  node_orientation: "antihorario (asegura det J > 0)"

validity:
  - "pequeños desplazamientos y deformaciones"
  - "geometría sin distorsión severa (det J > tol en todos los puntos de Gauss)"
  - "compatible con materiales Elastic2D, IsotropicDamage2D, VonMises2D (este último
      solo plane_strain)"

out_of_scope:
  - "grandes deformaciones, formulaciones corotacionales o totales lagrangianas"
  - "estabilización contra hourglass para integración reducida"
  - "elementos de alto orden (Q8, Q9)"

acceptance:
  verification:
    - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
      setup: "u* suave no polinómico; cuadrado distorsionado (deformación plana,
          esfuerzo plano, ortótropo a 30°) y cuarto de corona con tracción sobre los
          arcos; mallas n×n, n = 4, 8, 16, 32"
      expect: "orden observado entre los dos últimos niveles ≥ 2 - 0.1 en L² y ≥ 1 -
          0.1 en energía; error decreciente en cada nivel"
      tol_abs: 0.1
    - name: patch_test_macneal_harder
      setup: "5 Quad4 distorsionados (4 nodos exteriores + 4 interiores)
```

```
              con campo lineal u = 1e-3·(x + y/2), v = 1e-3·(y + x/2)
              impuesto en los 4 nodos del contorno"
      expect: "nodos interiores adoptan el campo lineal y ε = (1e-3, 1e-3,
               1e-3) en todos los puntos de Gauss; σ uniforme"
      tol_rel: 1.0e-10
  specific:
    - name: parche de tracción uniaxial sobre malla regular
      setup: "malla regular con campo de desplazamiento lineal y material Elastic2D"
      expect: "ε_xx constante en todos los puntos de Gauss"
      tol_rel: 1.0e-12
    - name: jacobiano degenerado abortado
      setup: "elemento con nodos colineales o en orden inverso"
      expect: "ValueError en _compute_kinematics"
    - name: cargas consistentes - body load uniforme
      setup: "Quad4 regular y distorsionado con b uniforme; sumar componentes nodales"
      expect: "Σf_x = b_x·A·t y Σf_y = b_y·A·t (invariante de geometría)"
      tol_rel: 1.0e-10
    - name: cargas consistentes - tracción uniforme en un borde
      setup: "Quad4 con tracción constante en un borde"
      expect: "Σf = t̄·L·t y reparto L/2 a cada nodo del borde, 0 en los demás"
      tol_rel: 1.0e-12
    - name: patch físico de tracción uniaxial
      setup: "Quad4 cuadrado L×H, borde izquierdo con ux=0 (n0,n3) + uy=0 (n0); borde
          derecho con tracción uniforme (p,0)"
      expect: "u reproduce ux=p·x/E, uyν=-·p·y/E (plane stress); σ_xx=p, σ_yyσ=_xy=0"
      tol_rel: 1.0e-10

references:
  - "Bathe K.-J., Finite Element Procedures, §5.3 (isoparametric elements)"
  - "Cook, Malkus, Plesha, Witt, Concepts and Applications of FEA, §6 (plane stress/
      strain)"
```

5.2.4 Implementación

- **Archivo**: solidum/elements/solid_2d/quad4.py
- **Clase**: `Quad4` (registrada vía `@ElementRegistry.register`)
- **Funciones núcleo**: `_compute_kinematics(xi, eta, coords)` y `compute_integrands(B, C, σ, detJ, w, t)`, ambas decoradas con `@njit` (Numba) para evaluar $\mathbf{B}$, $\det \mathbf{J}$ y los integrandos a velocidad cercana a Fortran. Viven en solidum/elements/solid_2d/_shared.py. `_compute_kinematics` es interna del Quad4; `compute_integrands` se comparte con Tri3 y es API pública de cinemática (ADR 0020, P6), con ruta pública `solidum.elements.solid_2d`.
- **Tests**:
- tests/test_solid_2d.py — ensamblaje de $\mathbf{B}$, simetría de $\mathbf{K}_e$, modos de cuerpo rígido y tracción uniaxial sobre malla regular.
- tests/test_patch_solid_2d.py — patch test de MacNeal-Harder sobre 5 Quad4 distorsionados (NAFEMS, V&V).
- tests/validation/test_mms_solid_2d.py (soluciones manufacturadas), tests/test_distributed_loads_2d.py (cargas variables).

5.2.5 Diálogo

- **2026-04-30** · Spec retroactiva. `Quad4` precedía al protocolo spec-first; la spec se creó documentando la formulación ya implementada y verificada. Promovido `status: validated`.
- **2026-04-30** · Añadido patch test de MacNeal-Harder (NAFEMS) en `acceptance.verification`. Cinco Quad4 distorsionados con cuatro nodos interiores libres reproducen exactamente un campo lineal impuesto en el contorno, con ε constante e igual al gradiente analítico en todos los puntos de Gauss.
- **2026-05-04** · Expuesta `compute_gauss_state(U)` con σ y ε por punto de Gauss y coordenadas naturales/globales. `compute_internal_forces` queda como promedio derivado. El `VtkExporter` añade campos nodales `Sigma_XX_nodal`, `Sigma_YY_nodal`, `Tau_XY_nodal` y `Von_Mises_nodal` por promedio simple sobre los elementos contiguos a cada nodo (suavizado de orden 0). *(Nota 2026-09-30: sólo promedian los sólidos mecánicos; un elemento térmico superpuesto en un dominio multicampo ya no divide el esfuerzo nodal. Ver el docstring del exportador y el manual de usuario, capítulo de post-proceso.)*
- **2026-05-04** · Añadidas cargas distribuidas consistentes (`compute_body_load`, `compute_edge_traction`). Para tracciones uniformes en bordes rectos el reparto es exacto (L/2 a cada nodo del borde) y se evita la cuadratura 1D; las fuerzas de cuerpo se integran con la cuadratura del elemento. Solo tracciones **constantes por borde** y en **coordenadas globales** en este paso; tracción variable y presión normal quedan para una iteración posterior.
- **2026-09-22** · Auditoría global: `quadrature` acepta clave del registro o tupla `(points, weights)` en todos los sólidos (`resolve_quadrature` valida la familia por medida de referencia y dimensión). La masa consistente se integra siempre con la regla completa 2×2 aunque K use integración reducida (con la regla del elemento y 1 punto, M quedaba de rango 2/8 y el problema modal era singular). tolerancia del jacobiano ahora **relativa** (`det J` $\leq$ `JACOBIAN_RTOL` · $\Pi\|$`fila_i(J)`$\|$, cota de Hadamard, adimensional): la absoluta `1e-10` sobre `det J` rechazaba mallas finas en metros sin distorsión.
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): la cinemática del elemento pasa a un núcleo compilado compartido (`_kin2d_from_dN`) y se expone como `BATCH_KINEMATICS`; `compute_element_state` no cambia de formulación. El `Assembler` agrupa los elementos en familias y los evalúa en un kernel único; equivalencia con el camino por elemento verificada a 1e-14 en `K` y `F_int` sobre todos los materiales compatibles (`tests/test_batch_assembly.py`).
- **2026-09-22** · Cierre del ADR 0014 (§9): la cinemática por lotes (`BATCH_KINEMATICS`) ya **no lanza** con jacobiano degenerado —devuelve `det J` $\leq$ `0` y el ensamblador lanza el `ValueError` con el id del elemento—, porque el kernel de familia tiene ahora variante paralela (`prange`, bit a bit igual a la serie) y una excepción dentro de la región paralela se pierde; el camino por elemento sigue lanzando como siempre. Declara las funciones de forma para el post-proceso por familia (`Family.gauss_state`, mismo resultado que `compute_gauss_state`, verificado en `tests/test_batch_postprocess.py`). Formulación sin cambios.
- **2026-09-29** · Cinemática pública para elementos de usuario (ADR 0020, P6): `_compute_integrands` pasa a llamarse `compute_integrands`, con ruta pública `solidum.elements.solid_2d`. Sólo cambia el nombre; formulación sin cambios.

- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022). `compute_body_load` acepta un campo `b(x)` y `compute_edge_traction` un campo `t(x, n)`, con `n` la normal exterior unitaria del borde (una presión es `-p·n`). La fuerza de cuerpo se integra con la regla de la masa (2×2, también con rigidez 1×1); la de borde, sobre la geometría real del borde con Gauss de 3 puntos. Declara `REFERENCE_CELL` y `REFERENCE_NODES`. Nueva aceptación «soluciones manufacturadas»: orden 2 en L^2 y 1 en energía en el cuadrado distorsionado y en la corona.

5.3 Tri6

Triángulo isoparamétrico de 6 nodos. Cura el shear locking severo del `Tri3` y reproduce campos cuadráticos exactamente. Útil cuando la malla es triangular por geometría.

5.3.1 Especificación física

Idéntica al `Tri3` salvo por la cinemática y la cuadratura.

7. Funciones de forma

Coordenadas baricéntricas $L_1 = 1 - \xi - \eta$, $L_2 = \xi$, $L_3 = \eta$:

$$N_i^{vert} = L_i(2L_i - 1), \quad i = 1, 2, 3$$

$$N_4 = 4L_1L_2, \quad N_5 = 4L_2L_3, \quad N_6 = 4L_3L_1.$$

Numeración: 0,1,2 vértices en (0,0), (1,0), (0,1); 3 medio del borde 0-1, 4 medio del 1-2, 5 medio del 2-0.

11. Cuadratura

Cuadratura triangular de 3 puntos (puntos medios de los lados; peso 1/6 cada uno) — exacta para polinomios hasta grado 2 sobre el triángulo de referencia.

12. Cargas distribuidas

- Body load con la regla `tri_6` de la masa: la `tri_3` del elemento no integra $N_i\, b$ con b cuadrático (ADR 0022).
- Edge traction como en `Quad8`, sobre la geometría real del borde; 1/6, 4/6, 1/6 sólo con borde recto y nodo medio centrado. Cargas constantes o campos.

5.3.2 Contrato de implementación

```yaml
name: Tri6
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 6
  strain_dim: 3
  n_integration_points: 3

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 3 (triángulos de orden superior)"
    - zienkiewicz_taylor: "vol. 1 (6.ª ed.), cap. 4: triángulos cuadráticos"
  general_codes:
    - abaqus: "CPS6 / CPE6"
```

```yaml
acceptance:
  verification:
    - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
      setup: "u* suave no polinómico; cuadrado distorsionado (deformación plana,
          esfuerzo plano, ortótropo a 30°) y cuarto de corona con tracción sobre los
          arcos; mallas n×n, n = 2, 4, 8, 16"
      expect: "orden observado entre los dos últimos niveles ≥ 3 - 0.1 en L² y ≥ 2 -
          0.1 en energía; error decreciente en cada nivel"
      tol_abs: 0.1
    - name: patch_cuadratico
      setup: "u = (x², 0) impuesto en los 6 nodos"
      expect: "ε_xx = 2x exacto en cada Gauss"
      tol_rel: 1.0e-10
    - name: cooks_membrane_4x4
      setup: "Trapezoide de Cook triangulado 4×4 (32 Tri6, diagonal →c1c3 con center
          node compartido)"
      expect: "u_y(48, 52) ≈ 23.91 (Bathe; Hughes)"
      tol_abs: 1.5
    - name: cooks_membrane_refinamiento
      setup: "Mallas 2×2 → 4×4 → 6×6 con la misma triangulación"
      expect: "Error |u_y - 23.91| decreciente monótonamente"
      tol_rel: 0.0
```

5.3.3 Implementación

- **Archivo**: solidum/elements/solid_2d/tri6.py · clase `Tri6` (subclase de la base interna `_HigherOrderSolid2D` en _shared.py, compartida con Quad8 y Quad9). Declara `_MASS_QUADRATURE = "tri_6"` (Dunavant 6 puntos, orden 4) porque la cuadratura del elemento (`tri_3`, orden 2) subintegra el producto cuadrático×cuadrático de la masa consistente.

- **Tests**: tests/test_higher_order_solid_2d.py (patch cuadrático), tests/test_cooks_membrane.py (benchmark Bathe/Hughes con malla triangulada 4×4 + refinamiento monótono, añadido Fase B 2026-05-19), tests/validation/test_mms_solid_2d.py (soluciones manufacturadas), tests/test_distributed_loads_2d.py (cargas variables).

5.3.4 Diálogo

- **2026-09-22** · Auditoría global: `quadrature` acepta clave del registro o tupla `(points, weights)` en todos los sólidos (`resolve_quadrature` valida la familia por medida de referencia y dimensión). tolerancia del jacobiano ahora **relativa** (`det J` $\leq$ `JACOBIAN_RTOL` · $\Pi\|$`fila_i(J)`$\|$, cota de Hadamard, adimensional): la absoluta `1e-10` sobre `det J` rechazaba mallas finas en metros sin distorsión.

- **2026-09-22** · Ensamblaje por lotes (ADR 0014): la cinemática del elemento pasa a un núcleo compilado compartido (`_kin2d_from_dN`) y se expone como `BATCH_KINEMATICS`; `compute_element_state` no cambia de formulación. El `Assembler` agrupa los elementos en familias y los evalúa en un kernel único; equivalencia con el camino por elemento verificada a 1e-14 en `K` y `F_int` sobre todos los materiales compatibles (`tests/test_batch_assembly.py`).

- **2026-09-22** · Cierre del ADR 0014 (§9): la cinemática por lotes (`BATCH_KINEMATICS`) ya **no lanza** con jacobiano degenerado —devuelve `det J` $\leq$ `0` y el ensamblador lanza el `ValueError` con el id del elemento—, porque el kernel de familia tiene ahora variante paralela (`prange`, bit a bit igual a la serie) y una excepción dentro de la región paralela se pierde; el camino por elemento sigue lanzando como siempre. Declara las funciones de forma para el post-proceso por familia (`Family.gauss_state`, mismo resultado que `compute_gauss_state`, verificado en `tests/test_batch_postprocess.py`). Formulación sin cambios.
- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022). `compute_body_load` acepta un campo `b(x)` y `compute_edge_traction` un campo `t(x, n)`, con `n` la normal exterior unitaria del borde (una presión es `-p·n`). La fuerza de cuerpo se integra con la regla de la masa (`tri_6`; antes `tri_3`); la de borde, sobre la geometría real del borde con Gauss de 3 puntos, lo que salda la deuda #13. Declara `REFERENCE_CELL` y `REFERENCE_NODES`. Nueva aceptación «soluciones manufacturadas»: orden 3 en L^2 y 2 en energía en el cuadrado distorsionado y en la corona.

5.4 Quad8

Elemento sólido 2D de orden cuadrático con interpolación serendípita.

Reproduce exactamente campos hasta Q_2^{seren} (todos los polinomios de grado total ≤ 2 más algunos de orden 3 sin el término $\xi^2\eta^2$). Mucho más preciso que ***Quad4*** *en flexión y geometrías curvas con el mismo grado de malla.*

5.4.1 Especificación física

0. Descripción general

Cuadrilátero plano de 8 nodos: 4 vértices + 4 nodos en los puntos medios de los bordes. Pequeñas deformaciones, isoparamétrico.

1. Cinemática de desplazamientos

$$u_x(\xi,\eta) = \sum_{i=1}^{8} N_i(\xi,\eta)\, u_x^{(i)}, \qquad u_y(\xi,\eta) = \sum_{i=1}^{8} N_i(\xi,\eta)\, u_y^{(i)}.$$

2. Cinemática de deformaciones

Voigt 2D $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$, idéntica al `Quad4`.

3. Ecuación constitutiva

Material 2D (`STRAIN_DIM = 3`).

4-5. Equilibrio

Idénticas a `Quad4`.

5.4.2 Formulación numérica

6. Discretización

Nodos en orden estándar: 0-3 vértices antihorarios desde $(-1,-1)$; 4 medio del borde 0-1, 5 medio del 1-2, 6 medio del 2-3, 7 medio del 3-0. Mapeo isoparamétrico.

7. Funciones de forma serendípitas

Vértices ($i = 0,1,2,3$ con $(\xi_i,\eta_i) \in \{\pm 1\}^2$):

$$N_i = \tfrac{1}{4}(1+\xi_i\xi)(1+\eta_i\eta)(\xi_i\xi+\eta_i\eta-1).$$

Medios de borde:

- Bordes con $\xi = 0$ (nodos 4, 6): $N_i = \tfrac{1}{2}(1-\xi^2)(1+\eta_i\eta)$ con $\eta_i = \mp 1$.
- Bordes con $\eta = 0$ (nodos 5, 7): $N_i = \tfrac{1}{2}(1+\xi_i\xi)(1-\eta^2)$ con $\xi_i = \pm 1$.

8. Matriz B

Ensamblaje estándar a partir de $\partial N_i/\partial x = \mathbf{J}^{-1}\partial N_i/\partial \xi$. Tamaño 3×16.

9-10. Rigidez y fuerzas internas

$$\mathbf{K}_e = \int \mathbf{B}^\top \mathbf{D}\,\mathbf{B}\, t\, dA, \qquad \mathbf{F}^e_{\text{int}} = \int \mathbf{B}^\top \boldsymbol{\sigma}\, t\, dA.$$

Cuadratura Gauss 3×3 por defecto.

11. Cuadratura

Gauss–Legendre 3×3 (9 puntos) — exacta para polinomios hasta grado 5, suficiente para integrar $\mathbf{B}^\top\mathbf{D}\mathbf{B}$ del Quad8 sobre Jacobiano constante. La opción `2x2` queda disponible (reducida) pero introduce modos espurios; emite advertencia.

12. Cargas distribuidas consistentes

- **Body load**: $\int N^\top \mathbf{b}\, t\, dA$ con la regla 3×3 de la masa, también cuando la rigidez usa 2×2; $\mathbf{b}$ constante o campo $\mathbf{b}(\mathbf{x})$ (ADR 0022).
- **Edge traction**: $\int_{\Gamma_e} N_k\, \bar{\mathbf{t}}(\mathbf{x}, \mathbf{n})\, t\, ds$ sobre la geometría real del borde: interpolación cuadrática de sus tres nodos, Gauss de 3 puntos y jacobiano $|d\mathbf{x}/ds|$ (ADR 0022). Con $\bar{\mathbf{t}}$ constante sobre un borde recto con el nodo medio centrado reparte $L/6$, $4L/6$, $L/6$ (regla 1-4-1); con el nodo medio descentrado o el borde curvo el reparto es otro. Hasta el 2026-10-27 se aplicaba siempre el 1-4-1 sobre la cuerda (deuda #13). $\bar{\mathbf{t}}$ es constante o campo de la posición y de la normal exterior unitaria; una presión es $-p\,\mathbf{n}$. Bordes:

Edge	Nodos extremos	Nodo medio
0	(0, 1)	4
1	(1, 2)	5
2	(2, 3)	6
3	(3, 0)	7

13. Salida por punto de Gauss

`compute_gauss_state(U)` con la misma estructura que `Quad4`: 9 puntos por defecto.

5.4.3 Contrato de implementación

```yaml
name: Quad8
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 8
  strain_dim: 3
```

```
  n_integration_points: 9       # default Gauss 3×3

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 3 (elementos serendípitos)"
    - zienkiewicz_taylor: "vol. 1 (6.ª ed.), cap. 4: familia serendípita"
  general_codes:
    - abaqus: "CPS8 / CPE8"

parameters:
  - { name: thickness, type: float, required: false, default: 1.0 }
  - { name: quadrature, type: tuple, required: false, default: "3x3" }

material_contract:
  signature: "compute_stateε(, state) -> σ(, C_tangent, state')"
  strain_kind: "Voigt 2D ε[_xx, ε_yy, γ_xy]"

conventions:
  sign: "tensión positiva = tracción"
  voigt: "[xx, yy, xy] con γ_xy = 2·ε_xy"
  node_orientation: "vértices antihorarios; medios en el orden de los bordes"

validity:
  - "pequeños desplazamientos y deformaciones"

out_of_scope:
  - "grandes deformaciones"
  - "estabilización contra modos espurios bajo integración reducida"

acceptance:
  verification:
    - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
      setup: "u* suave no polinómico; cuadrado distorsionado (deformación plana,
          esfuerzo plano, ortótropo a 30°) y cuarto de corona con tracción sobre los
          arcos; mallas n×n, n = 2, 4, 8, 16"
      expect: "orden observado entre los dos últimos niveles ≥ 3 - 0.1 en L² y ≥ 2 -
          0.1 en energía; error decreciente en cada nivel"
      tol_abs: 0.1
    - name: patch_lineal
      setup: "campo u = ₀a + ₁a·x + ₂a·y impuesto en el contorno"
      expect: "ε constante en todos los Gauss e igual al gradiente analítico"
      tol_rel: 1.0e-10
    - name: patch_cuadratico
      setup: "campo u_x = x²·1e-3 impuesto en los 8 nodos"
      expect: "ε_xx(x, y) = 2e-3·x exacto en todos los puntos de Gauss"
      tol_rel: 1.0e-10
  specific:
    - name: cargas consistentes - body load uniforme
      setup: "Σf = b·A·t (regular y distorsionado)"
      tol_rel: 1.0e-10
    - name: cargas consistentes - tracción uniforme en un borde
      setup: "Σf = ¯t·L·t y reparto 1/6, 4/6, 1/6 en (vértice, medio, vértice) con el
          nodo medio centrado; con el nodo medio en a_m, (L/6 - (L - 2a_m)/3, 2L/3, L/6
          + (L - 2a_m)/3)·¯t·t"
      tol_rel: 1.0e-12
    - name: Cook's membrane (Cook 1974)                             # 2026-05-19
```

```
      setup: "trapezoid (0,0)-(48,44)-(48,60)-(0,44), E=1, ν=1/3, plane stress,
          cortante total F=1 distribuido en borde derecho; malla 4×4 Q8"
      expect: "u_y en (48,52) ≈ 23.91 ± 0.5; refinamiento 2×→24×→46×6 reduce error
          monotónicamente"
      tol_abs: 0.5

references:
  - "Bathe K.-J., Finite Element Procedures, §5.3"
  - "Cook, Malkus, Plesha, Witt, Concepts and Applications of FEA, §6"
  - "Zienkiewicz O.C., The Finite Element Method, vol. 1, §8"
```

5.4.4 Implementación

- **Archivo**: solidum/elements/solid_2d/quad8.py · clase `Quad8` (subclase de la base interna `_HigherOrderSolid2D` en _shared.py, compartida con Quad9 y Tri6).
- **Tests**: tests/test_higher_order_solid_2d.py, tests/validation/test_mms_solid_2d.py (soluciones manufacturadas), tests/test_distributed_loads_2d.py (cargas variables).

5.4.5 Diálogo

- **2026-09-22** · Auditoría global: `quadrature` acepta clave del registro o tupla `(points, weights)` en todos los sólidos (`resolve_quadrature` valida la familia por medida de referencia y dimensión). `_MASS_QUADRATURE = "3x3"` explícita: la masa se integra con la regla completa aunque K use 2×2 (antes M de rango 8/16). tolerancia del jacobiano ahora **relativa** (`det J` $\leq$ `JACOBIAN_RTOL` · $\Pi\|$`fila_i(J)`$\|$, cota de Hadamard, adimensional): la absoluta `1e-10` sobre `det J` rechazaba mallas finas en metros sin distorsión.
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): la cinemática del elemento pasa a un núcleo compilado compartido (`_kin2d_from_dN`) y se expone como `BATCH_KINEMATICS`; `compute_element_state` no cambia de formulación. El `Assembler` agrupa los elementos en familias y los evalúa en un kernel único; equivalencia con el camino por elemento verificada a 1e-14 en `K` y `F_int` sobre todos los materiales compatibles (`tests/test_batch_assembly.py`).
- **2026-09-22** · Cierre del ADR 0014 (§9): la cinemática por lotes (`BATCH_KINEMATICS`) ya **no lanza** con jacobiano degenerado —devuelve `det J` $\leq$ `0` y el ensamblador lanza el `ValueError` con el id del elemento—, porque el kernel de familia tiene ahora variante paralela (`prange`, bit a bit igual a la serie) y una excepción dentro de la región paralela se pierde; el camino por elemento sigue lanzando como siempre. Declara las funciones de forma para el post-proceso por familia (`Family.gauss_state`, mismo resultado que `compute_gauss_state`, verificado en `tests/test_batch_postprocess.py`). Formulación sin cambios.
- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022). `compute_body_load` acepta un campo `b(x)` y `compute_edge_traction` un campo `t(x, n)`, con `n` la normal exterior unitaria del borde (una presión es `-p·n`). La fuerza de cuerpo se integra con la regla de la masa (3×3, también con rigidez 2×2); la de borde, sobre la geometría real del borde con Gauss de 3 puntos, lo que salda la deuda #13 (1-4-1 sobre la cuerda; con él el orden L^2 del MMS caía de 3 a 2). Declara `REFERENCE_CELL` y `REFERENCE_NODES`. Nueva aceptación

«soluciones manufacturadas»: orden 3 en L^2 y 2 en energía en el cuadrado distorsionado y en la corona.

5.5 Quad9

Variante Lagrangiana del Quad8: añade un noveno nodo central interior.
Espacio polinómico completo Q_2 (incluye el término $\xi^2\eta^2$ que el serendípito omite).

5.5.1 Especificación física

Idéntica a `Quad8` salvo por las funciones de forma.

7. Funciones de forma

Producto tensorial de los polinomios de Lagrange 1D de orden 2:

$$L_0(\xi) = \tfrac{1}{2}\xi(\xi-1), \quad L_1(\xi) = 1-\xi^2, \quad L_2(\xi) = \tfrac{1}{2}\xi(\xi+1).$$

$N_i(\xi,\eta) = L_a(\xi)\,L_b(\eta)$ con (a,b) asociado al nodo i según la numeración:

- 0..3: vértices (-1,-1), (+1,-1), (+1,+1), (-1,+1).
- 4: medio del borde 0-1 (η = -1).
- 5: medio del borde 1-2 (ξ = +1).
- 6: medio del borde 2-3 (η = +1).
- 7: medio del borde 3-0 (ξ = -1).
- 8: nodo central interior (ξ, η) = (0, 0) — *bubble*.

11. Cuadratura

Gauss 3×3 por defecto.

12. Cargas distribuidas

Body load con la regla 3×3 de la masa. Tracción en bordes como en `Quad8`, sobre la geometría real del borde (el nodo 8 no toca bordes). Cargas constantes o campos (ADR 0022).

5.5.2 Contrato de implementación

```yaml
name: Quad9
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 9
  strain_dim: 3
  n_integration_points: 9

admission:                # criterio de admisión al programa principal (Reglas.md §1)
```

```
  textbooks:
    - hughes: "cap. 3 (elementos lagrangianos)"
    - zienkiewicz_taylor: "vol. 1 (6.ª ed.), cap. 4: familia lagrangiana"
  general_codes:
    - code_aster: "QUAD9 en las modelizaciones D_PLAN / C_PLAN"

acceptance:
  verification:
    - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
      setup: "u* suave no polinómico; cuadrado distorsionado (deformación plana,
          esfuerzo plano, ortótropo a 30°) y cuarto de corona con tracción sobre los
          arcos; mallas n×n, n = 2, 4, 8, 16"
      expect: "orden observado entre los dos últimos niveles ≥ 3 - 0.1 en L² y ≥ 2 -
          0.1 en energía; error decreciente en cada nivel"
      tol_abs: 0.1
    - name: patch_cuadratico
      setup: "u = (x², 0) impuesto en los 9 nodos"
      expect: "ε_xx = 2x exacto en todos los Gauss"
      tol_rel: 1.0e-10
    - name: Cook's membrane (Cook 1974)                                  # 2026-05-19
      setup: "trapezoid (0,0)-(48,44)-(48,60)-(0,44), E=1, ν=1/3, plane stress,
          cortante total F=1 distribuido en borde derecho; malla 4×4 Q9"
      expect: "u_y en (48,52) ≈ 23.91 ± 0.5"
      tol_abs: 0.5
```

5.5.3 Implementación

- **Archivo**: solidum/elements/solid_2d/quad9.py · clase `Quad9` (subclase de la base interna `_HigherOrderSolid2D` en _shared.py, compartida con Quad8 y Tri6).
- **Tests**: tests/test_higher_order_solid_2d.py, tests/validation/test_mms_solid_2d.py (soluciones manufacturadas), tests/test_distributed_loads_2d.py (cargas variables).

5.5.4 Diálogo

- **2026-09-22** · Auditoría global: `quadrature` acepta clave del registro o tupla `(points, weights)` en todos los sólidos (`resolve_quadrature` valida la familia por medida de referencia y dimensión). `_MASS_QUADRATURE = "3x3"` explícita: la masa se integra con la regla completa aunque K use 2×2 (antes M de rango 8/18). tolerancia del jacobiano ahora **relativa** (`det J` $\leq$ `JACOBIAN_RTOL` · $\Pi\|$`fila_i(J)`$\|$, cota de Hadamard, adimensional): la absoluta `1e-10` sobre `det J` rechazaba mallas finas en metros sin distorsión.
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): la cinemática del elemento pasa a un núcleo compilado compartido (`_kin2d_from_dN`) y se expone como `BATCH_KINEMATICS`; `compute_element_state` no cambia de formulación. El `Assembler` agrupa los elementos en familias y los evalúa en un kernel único; equivalencia con el camino por elemento verificada a 1e-14 en `K` y `F_int` sobre todos los materiales compatibles (`tests/test_batch_assembly.py`).
- **2026-09-22** · Cierre del ADR 0014 (§9): la cinemática por lotes (`BATCH_KINEMATICS`) ya **no lanza** con jacobiano degenerado —devuelve `det J` $\leq$ `0` y el ensamblador lanza el `ValueError` con el id del elemento—, porque el kernel de familia tiene ahora variante paralela (`prange`, bit

a bit igual a la serie) y una excepción dentro de la región paralela se pierde; el camino por elemento sigue lanzando como siempre. Declara las funciones de forma para el post-proceso por familia (`Family.gauss_state`, mismo resultado que `compute_gauss_state`, verificado en `tests/test_batch_postprocess.py`). Formulación sin cambios.

- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022). `compute_body_load` acepta un campo `b(x)` y `compute_edge_traction` un campo `t(x, n)`, con `n` la normal exterior unitaria del borde (una presión es `-p·n`). La fuerza de cuerpo se integra con la regla de la masa (3×3); la de borde, sobre la geometría real del borde con Gauss de 3 puntos, lo que salda la deuda #13. Declara `REFERENCE_CELL` y `REFERENCE_NODES`. Nueva aceptación «soluciones manufacturadas»: orden 3 en L^2 y 2 en energía en el cuadrado distorsionado y en la corona.

Capítulo 6

Elementos 3D — Sólidos

6.1 Tet4

Elemento sólido 3D de primer orden, espejo natural de `Tri3`*. Pieza de la Etapa 7. Convención Voigt 6D y numeración de caras por ADR 0012.*

6.1.1 Especificación física

0. Descripción general

Elemento sólido tridimensional de primer orden, tetraedro de cuatro nodos. Aproximación **lineal** C^0 del campo de desplazamientos en coordenadas baricéntricas. Análogo 3D del `Tri3`: la matriz $\mathbf{B}$ es **constante sobre el elemento**, por lo que $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ son uniformes dentro de cada `Tet4` (*constant strain tetrahedron* en la literatura). Régimen de pequeños desplazamientos y deformaciones.

1. Cinemática de desplazamientos

$$u_x = \sum_{i=1}^{4} N_i \, u_x^{(i)}, \qquad u_y = \sum N_i \, u_y^{(i)}, \qquad u_z = \sum N_i \, u_z^{(i)}.$$

2. Cinemática de deformaciones

Notación Voigt 6D del proyecto (ADR 0012):

$$\boldsymbol{\varepsilon} = [\varepsilon_{xx},\ \varepsilon_{yy},\ \varepsilon_{zz},\ \gamma_{xy},\ \gamma_{yz},\ \gamma_{xz}]^\top$$

Por la linealidad de N_i, las derivadas $\partial N_i / \partial \mathbf{x}$ son constantes y $\boldsymbol{\varepsilon}$ es uniforme en el elemento. Idéntico patrón conceptual al `Tri3` en 2D.

3. Ecuación constitutiva

Delegada al material 3D (`STRAIN_DIM = 6`). En esta etapa solo `Elastic3D` está disponible.

4. Equilibrio — forma fuerte

Idéntica a la del `Hex8`. El principio variacional no depende de la forma del elemento.

5. Equilibrio — forma débil

Idéntica a la del `Hex8`.

6.1.2 Formulación numérica (FEM)

6. Discretización

Cuatro nodos en orden estándar de tetraedro VTK. En el tetraedro de referencia con vértices:

Nodo local	(ξ_i, η_i, ζ_i)
0	$(0, 0, 0)$
1	$(1, 0, 0)$
2	$(0, 1, 0)$
3	$(0, 0, 1)$

El orden de nodos asegura que el vector $(\mathbf{x}_1 - \mathbf{x}_0) \times (\mathbf{x}_2 - \mathbf{x}_0)$ apunte hacia $\mathbf{x}_3$ (volumen orientado positivo). Mapeo isoparamétrico desde el tetraedro de referencia; jacobiano $\mathbf{J}$ constante en todo el elemento; $\det \mathbf{J} = 6\,V_e$ con V_e el volumen del tetraedro. El código aborta con `ValueError` si $\det \mathbf{J} \leq \text{tol}$ (nodos coplanares o en orden invertido).

7. Funciones de forma

Coordenadas baricéntricas / tetraédricas:

$$N_1 = 1 - \xi - \eta - \zeta, \qquad N_2 = \xi, \qquad N_3 = \eta, \qquad N_4 = \zeta.$$

Sus derivadas en coordenadas naturales son constantes:

$$\partial N_i/\partial \xi = (-1, 1, 0, 0), \quad \partial N_i/\partial \eta = (-1, 0, 1, 0), \quad \partial N_i/\partial \zeta = (-1, 0, 0, 1).$$

8. Matriz deformación-desplazamiento

$\mathbf{B}$ tiene tamaño 6×12 y, una vez transformadas las derivadas a globales vía $\mathbf{J}^{-1}$, sus entradas son **constantes** sobre el elemento:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i/\partial x & 0 & 0 \\ 0 & \partial N_i/\partial y & 0 \\ 0 & 0 & \partial N_i/\partial z \\ \partial N_i/\partial y & \partial N_i/\partial x & 0 \\ 0 & \partial N_i/\partial z & \partial N_i/\partial y \\ \partial N_i/\partial z & 0 & \partial N_i/\partial x \end{bmatrix}, \qquad \boldsymbol{\varepsilon} = \mathbf{B}\,\mathbf{u}_e.$$

Mismo patrón filas/columnas que `Hex8` (ver §8 de su spec).

9. Rigidez elemental

$$\mathbf{K}_e = \mathbf{B}^\top \mathbf{D}\, \mathbf{B}\, V_e,$$

con $V_e = \frac{1}{6} \det \mathbf{J}$ el volumen del tetraedro. La integración es **exacta con un único punto central** porque $\mathbf{B}$ y $\boldsymbol{\sigma}$ son constantes.

10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \mathbf{B}^\top \boldsymbol{\sigma}\, V_e.$$

11. Cuadratura

Un punto central (baricentro del tetraedro) con peso $w = 1/6$ sobre el tetraedro de referencia. Es exacto para formas constantes de $\mathbf{B}$ y $\boldsymbol{\sigma}$.

12. Cargas distribuidas consistentes

Fuerza de cuerpo b. Con N_i lineales y $\mathbf{b}$ uniforme la integral es exacta:

$$\mathbf{f}_e^{b} = \int_{\Omega_e} \mathbf{N}^\top \mathbf{b}\, dV \implies \text{cada nodo recibe } \tfrac{1}{4}\, \mathbf{b}\, V_e.$$

Se integra con la regla de 4 puntos `tet_4` (ADR 0022), exacta también para $\mathbf{b}$ lineal; $\mathbf{b}$ es constante o un campo $\mathbf{b}(\mathbf{x})$.

Tracción de superficie $\bar{\mathbf{t}}$ sobre una cara triangular. Caras numeradas con normal saliente (ADR 0012):

Cara	Nodos	Nodo opuesto
0	(1, 2, 3)	0
1	(0, 3, 2)	1
2	(0, 1, 3)	2
3	(0, 2, 1)	3

Sobre una cara triangular con tracción constante el reparto es exacto:

$$\mathbf{f}_e^{t} = \tfrac{1}{3}\, \bar{\mathbf{t}}\, A_{\text{cara}} \text{ por nodo del triángulo}, \qquad 0 \text{ al nodo opuesto}.$$

Cuadratura de 3 puntos sobre la cara (`tri_3`), exacta para tracción lineal. $\bar{\mathbf{t}}$ se especifica en **coordenadas globales**: constante o campo $\bar{\mathbf{t}}(\mathbf{x}, \mathbf{n})$ de la posición y de la normal exterior unitaria de la cara, que sale de la orientación antihoraria de ‘`FACE_NODES`‘ vista desde fuera (ADR 0022); una presión p es $\bar{\mathbf{t}} = -p\, \mathbf{n}$. Se integra sobre la geometría real de la cara con sus funciones de forma.

13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve la misma estructura que `Hex8` pero con un único punto (centroide). Para `Tet4` $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ son constantes sobre el elemento, así que el dato del punto coincide con el promedio del elemento. Por ADR 0012, `internal_forces` devuelve `None`.

6.1.3 Limitación crítica — *shear locking*

El `Tet4` es notoriamente rígido en flexión y dominado por cortante, exactamente como su análogo `Tri3` en 2D: cuatro DOFs de desplazamiento por dirección no bastan para representar el modo de flexión puro de una viga discretizada en tetraedros, por lo que el elemento responde con cortante espurio y subestima sistemáticamente la deflexión. La severidad crece con la relación L/h del problema.

Recomendación operativa: usar `Hex8` por defecto en mallas hexaédricas o mixtas; reservar `Tet4` para zonas de transición de malla donde la generación automática de hexaedros falla y se requiere malla tetraédrica de baja regularidad. Para problemas dominados por flexión se necesitan tetraedros cuadráticos `Tet10` (sub-etapa posterior). Locking volumétrico con $\nu \to 0.5$ es **aún peor** que en `Hex8` por la misma razón: el espacio lineal es muy pobre.

6.1.4 Contrato de implementación

```
name: Tet4
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 4
  strain_dim: 6                  # Voigt 3D ε[_xx, ε_yy, ε_zz, γ_xy, γ_yz, γ_xz]
  n_integration_points: 1

admission:                 # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 3 (tetraedro lineal)"
    - zienkiewicz_taylor: "vol. 1 (6.ª ed.), cap. 4: tetraedros"
  general_codes:
    - abaqus: "C3D4"

parameters: []

material_contract:
  signature: "compute_stateε(, state) -> σ(, C_tangent, state')"
  strain_kind: "Voigt 3D ε[_xx, ε_yy, ε_zz, γ_xy, γ_yz, γ_xz] con γ_ij = 2·ε_ij"

conventions:
  sign: "σ > 0 ⇔ tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] con γ_ij = 2·ε_ij (ADR 0012)"
  node_orientation: "convención VTK_TETRA; nodos 0-2 base con normal hacia 3 (volumen
      orientado positivo); asegura det J > 0"
  face_numbering: "ADR 0012 - 4 caras con normal saliente; cara i opuesta al nodo i"

validity:
  - "pequeños desplazamientos y deformaciones"
  - "campos de tensión que varían suavemente sobre el elemento"
  - "compatible con materiales con STRAIN_DIM = 6 (en esta etapa: Elastic3D)"

out_of_scope:
  - "flexión dominante (shear locking severo)"
  - "casi-incompresibilidad ν( → 0.5 - locking volumétrico aún peor que Hex8)"
```

```
  - "elementos de alto orden (Tet10 - sub-etapa posterior)"
  - "grandes deformaciones"
  - "internal_forces (ADR 0012: sólidos exponen compute_gauss_state)"

acceptance:
  verification:
    - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
      setup: "u* suave no polinómico; cubo distorsionado (caras planas con nodos
          deslizados, elementos interiores curvos) y sector de cilindro hueco con
          tracción sobre las caras cilíndricas; Elastic3D; mallas n×n×n, n = 4, 8, 16"
      expect: "orden observado entre los dos últimos niveles ≥ 2 - 0.1 en L² y ≥ 1 -
          0.1 en energía; error decreciente en cada nivel"
      tol_abs: 0.1
    - name: patch_test_3d
      setup: "malla tetraédrica con al menos un nodo interior (e.g.
             división de un cubo en 5 tetraedros con un nodo central
             libre); campo lineal u_i = a_ij·x_j impuesto en el contorno"
      expect: "nodo interior adopta el campo lineal exacto y ε constante
              e igual a 0.5·(a_ij + a_ji) en cada elemento; σ uniforme"
      tol_rel: 1.0e-10
    - name: modos_de_cuerpo_rigido
      setup: "evaluar K_e sobre un Tet4 cualquiera y proyectar sobre los 6
             modos de cuerpo rígido"
      expect: "K_e · u_rigid ≈ 0 en cada uno de los 6 modos"
      tol_abs: 1.0e-9
    - name: simetria_K_e
      setup: "evaluar K_e con Elastic3D"
      expect: "K_e == K_e.T exacto"
      tol_abs: 1.0e-12
  specific:
    - name: parche_deformacion_constante
      setup: "malla tetraédrica regular con campo lineal y Elastic3D"
      expect: "ε constante (CST 3D reproduce campos lineales por construcción)"
      tol_rel: 1.0e-12
    - name: jacobiano_degenerado_abortado
      setup: "cuatro nodos coplanares"
      expect: "ValueError en _compute_kinematics_tet4"
    - name: body_load_uniforme
      setup: "Tet4 con b uniforme"
      expect: "cada nodo recibe (1/4)·b·V_e; Σ = b·V_e"
      tol_rel: 1.0e-12
    - name: traccion_uniforme_cara
      setup: "Tet4 con tracción constante en una cara"
      expect: "Σf = t̄·A_cara; reparto A_cara/3 t̄ a cada uno de los 3
              nodos de la cara, 0 al nodo opuesto"
      tol_rel: 1.0e-12
    - name: cubo_lame_3d_malla_tetraedrica
      setup: "cubo unitario discretizado con tetraedros (e.g. 5 o 6 Tet4
             por cubo); tracción uniaxial σ_xx = p en cara ξ+"
      expect: "u_x(L) ≈ p·L/E con error decreciente al refinar; orden
              de convergencia O(h) en norma energética"
      tol_rel: "documentar convergencia, sin tolerancia absoluta"

references:
  - "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and Applications
     of FEA. §6.8 (CST 3D)."
```

```
  - "Zienkiewicz O.C., The Finite Element Method, vol. 1, §6.7 (limitations of linear
     tets)."
  - "Bathe K.J. (2014). Finite Element Procedures. §6.6.2."
  - "ADR 0012 - Sólidos 3D: convención Voigt 6D y caras."
```

6.1.5 Implementación

- **Archivo**: solidum/elements/solid_3d/tet4.py.
- **Clase**: `Tet4` (registrada vía `@ElementRegistry.register`).
- **Función núcleo**: `_compute_kinematics_tet4(coords)` con `@njit` en solidum/elements/solid_3d/_shared.py. Reutiliza `_compute_integrands_3d` (también en `_shared`, compartido con Hex8) con peso `1/6` (volumen del tet de referencia).
- **Masa consistente analítica**: ‘`M_ij = ρ·V_e·(1 + δ_ij)/20 (matriz escalar 4×4, expandida 3D vía` _expand_scalar_mass_3d‘). Evita cuadratura insuficiente en el centroide.
- **Tests**:
- tests/validation/test_mms_solid_3d.py (soluciones manufacturadas), tests/test_distributed_loads_3d.py (cargas variables).
- tests/test_solid_3d.py — clase `TestTet4Element` (10 tests: dimensiones/DOFs, simetría exacta de K (no numérica, por fórmula cerrada), volumen, patch tracción uniaxial, jacobiano degenerado abortado, body load, face traction (cara opuesta a nodo recibe cero, reparto A/3 a los 3 nodos restantes), masa consistente y lumped HRZ (reparto equitativo ρV/4 por DOF)).
- tests/test_rigid_body_modes.py — `TestRBMTet4` (5 tests: traslación, 3 rotaciones, rank-deficiency = 6).
- tests/validation/test_cube_lame_3d.py — `test_uniaxial_traction_tet4_mesh_exact` (cubo dividido en 5 tetraedros con tracción uniaxial; solución exacta nodal).

6.1.6 Diálogo

- **2026-05-19** · Spec inicial. Elemento espejo natural de `Tri3`; convención Voigt 6D y numeración de caras fijadas por ADR 0012. Sin elementos de alto orden en esta etapa. Shear locking y locking volumétrico declarados como limitaciones; `Hex8` es la opción preferente cuando la malla lo permite.
- **2026-09-22** · Auditoría global: tolerancia del jacobiano ahora **relativa** (`det J` $\leq$ `JACOBIAN_RTOL` · $\Pi\|$`fila_i(J)`$\|$, cota de Hadamard, adimensional): la absoluta `1e-10` sobre `det J` rechazaba mallas finas en metros sin distorsión.
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): la cinemática del elemento pasa a un núcleo compilado compartido (`_kin3d_from_dN`) y se expone como `BATCH_KINEMATICS`; `compute_element_state` no cambia de formulación. El `Assembler` agrupa los elementos en familias y los evalúa en un kernel único; equivalencia con el camino por elemento verificada a 1e-14 en `K` y `F_int` sobre todos los materiales compatibles (`tests/test_batch_assembly.py`).

- **2026-09-22** · Cierre del ADR 0014 (§9): la cinemática por lotes (`BATCH_KINEMATICS`) ya **no lanza** con jacobiano degenerado —devuelve `det J` $\leq$ `0` y el ensamblador lanza el `ValueError` con el id del elemento—, porque el kernel de familia tiene ahora variante paralela (`prange`, bit a bit igual a la serie) y una excepción dentro de la región paralela se pierde; el camino por elemento sigue lanzando como siempre. Declara las funciones de forma para el post-proceso por familia (`Family.gauss_state`, mismo resultado que `compute_gauss_state`, verificado en `tests/test_batch_postprocess.py`). Formulación sin cambios.
- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022, fase 2). `compute_body_load` acepta un campo `b(x)` y `compute_face_traction` un campo `t(x, n)`, con `n` la normal exterior unitaria de la cara (una presión es `-p·n`); implementación común en `_SolidLoads3D`. La fuerza de cuerpo se integra con la regla de la masa (`tet_4`; antes el reparto 1/4 exacto sólo para b constante); la de cara, sobre la geometría real con las funciones de forma de la cara. Declara `REFERENCE_CELL` y `REFERENCE_NODES`. Nueva aceptación «soluciones manufacturadas»: orden 2 en L^2 y 1 en energía en el cubo distorsionado y en el cilindro hueco; el Tet4 converge al orden asintótico desde abajo y despacio (L^2: 1.50, 1.75, 1.91 con n = 4, 8, 16).

6.2 Hex8

Elemento sólido 3D de primer orden, espejo natural de `Quad4`*. Pieza central de la Etapa 7. Convención Voigt 6D y caras fijadas por ADR 0012.*

6.2.1 Especificación física

0. Descripción general

Elemento sólido tridimensional de primer orden, hexaedro de ocho nodos. Aproximación **trilineal** C^0 del campo de desplazamientos en coordenadas naturales $(\xi, \eta, \zeta) \in [-1, 1]^3$. Formulación isoparamétrica: la geometría y los desplazamientos comparten las mismas funciones de forma. Régimen de pequeños desplazamientos y deformaciones.

1. Cinemática de desplazamientos

$$u_x(\xi,\eta,\zeta) = \sum_{i=1}^{8} N_i(\xi,\eta,\zeta)\, u_x^{(i)}, \qquad u_y = \sum N_i\, u_y^{(i)}, \qquad u_z = \sum N_i\, u_z^{(i)}.$$

2. Cinemática de deformaciones

Tensor infinitesimal en notación Voigt 6D del proyecto (ADR 0012):

$$\boldsymbol{\varepsilon} = [\varepsilon_{xx},\ \varepsilon_{yy},\ \varepsilon_{zz},\ \gamma_{xy},\ \gamma_{yz},\ \gamma_{xz}]^\top$$

con

$$\varepsilon_{xx} = \frac{\partial u_x}{\partial x}, \quad \varepsilon_{yy} = \frac{\partial u_y}{\partial y}, \quad \varepsilon_{zz} = \frac{\partial u_z}{\partial z},$$

$$\gamma_{xy} = \frac{\partial u_x}{\partial y} + \frac{\partial u_y}{\partial x}, \quad \gamma_{yz} = \frac{\partial u_y}{\partial z} + \frac{\partial u_z}{\partial y}, \quad \gamma_{xz} = \frac{\partial u_x}{\partial z} + \frac{\partial u_z}{\partial x}.$$

3. Ecuación constitutiva

Delegada al material 3D (`STRAIN_DIM = 6`). El elemento llama `material.compute_state(`ε`, state)` $\rightarrow$ `(`σ`, C_tangent, state')` por punto de Gauss. En esta etapa solo `Elastic3D` está disponible; el contrato deja la puerta abierta a futuros `VonMises3D`, `DruckerPrager3D`, `IsotropicDamage3D`.

4. Equilibrio — forma fuerte

$$-\nabla \cdot \boldsymbol{\sigma} = \mathbf{b} \quad \text{en } \Omega, \qquad \boldsymbol{\sigma} \cdot \mathbf{n} = \bar{\mathbf{t}} \quad \text{en } \partial\Omega_N, \qquad \mathbf{u} = \bar{\mathbf{u}} \quad \text{en } \partial\Omega_D.$$

5. Equilibrio — forma débil

$$\int_\Omega \delta\boldsymbol{\varepsilon}^\top \boldsymbol{\sigma}\, dV = \int_\Omega \delta\mathbf{u}^\top \mathbf{b}\, dV + \int_{\partial\Omega_N} \delta\mathbf{u}^\top \bar{\mathbf{t}}\, dS, \qquad \forall\, \delta\mathbf{u} \in V_0.$$

6.2.2 Formulación numérica (FEM)

6. Discretización

Ocho nodos en orden estándar de hexaedro VTK (compatible con la mayoría de pre-procesadores). En el cubo de referencia $[-1,1]^3$:

Nodo local	(ξ_i, η_i, ζ_i)
0	$(-1,-1,-1)$
1	$(+1,-1,-1)$
2	$(+1,+1,-1)$
3	$(-1,+1,-1)$
4	$(-1,-1,+1)$
5	$(+1,-1,+1)$
6	$(+1,+1,+1)$
7	$(-1,+1,+1)$

Cara inferior $\zeta = -1$ recorrida `(0, 1, 2, 3)` antihorario visto desde dentro; cara superior $\zeta = +1$ recorrida `(4, 5, 6, 7)` antihorario visto desde fuera. Mapeo isoparamétrico desde el cubo de referencia:

$$x(\xi,\eta,\zeta) = \sum_{i=0}^{7} N_i\, x^{(i)}, \quad y = \sum N_i\, y^{(i)}, \quad z = \sum N_i\, z^{(i)}.$$

Jacobiano $\mathbf{J} = \partial(x,y,z)/\partial(\xi,\eta,\zeta)$ es una matriz 3×3 con determinante $\det \mathbf{J}$. El código aborta con `ValueError` si $\det \mathbf{J} \le \text{tol}$ (elemento degenerado o nodos en orden invertido).

7. Funciones de forma

Producto tensorial trilineal:

$$N_i(\xi,\eta,\zeta) = \tfrac{1}{8}(1+\xi_i\,\xi)(1+\eta_i\,\eta)(1+\zeta_i\,\zeta), \qquad (\xi_i,\eta_i,\zeta_i) \in \{-1,+1\}^3.$$

8. Matriz deformación-desplazamiento

Las derivadas en globales se obtienen vía $\partial N_i/\partial \mathbf{x} = \mathbf{J}^{-1}\,\partial N_i/\partial \boldsymbol{\xi}$. La matriz $\mathbf{B}$ tiene tamaño $\mathbf{6\times24}$ y se ensambla por nodo $i \in \{0,\dots,7\}$:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i/\partial x & 0 & 0 \\ 0 & \partial N_i/\partial y & 0 \\ 0 & 0 & \partial N_i/\partial z \\ \partial N_i/\partial y & \partial N_i/\partial x & 0 \\ 0 & \partial N_i/\partial z & \partial N_i/\partial y \\ \partial N_i/\partial z & 0 & \partial N_i/\partial x \end{bmatrix}, \qquad \boldsymbol{\varepsilon} = \mathbf{B}\,\mathbf{u}_e.$$

El orden de las filas reproduce exactamente el orden Voigt del proyecto (ADR 0012). El orden de las columnas por nodo es `[ux, uy, uz]`, agrupado por nodo i para `i` $\in$ `{0..7}`.

9. Rigidez elemental

$$\mathbf{K}_e = \int_\Omega \mathbf{B}^\top \mathbf{D}\, \mathbf{B}\, dV = \sum_{p=1}^{n_g} \mathbf{B}(\boldsymbol{\xi}_p)^\top \mathbf{D}_p\, \mathbf{B}(\boldsymbol{\xi}_p)\, \det \mathbf{J}_p\, w_p.$$

A diferencia del 2D, no hay parámetro de espesor — el "espesor" está incluido en el volumen integrado.

10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sum_{p=1}^{n_g} \mathbf{B}(\boldsymbol{\xi}_p)^\top \boldsymbol{\sigma}_p\, \det \mathbf{J}_p\, w_p.$$

11. Cuadratura

Por defecto **Gauss–Legendre 2×2×2** (ocho puntos): orden de exactitud 3 en cada dirección — integra exactamente $\mathbf{K}_e$ para Jacobiano constante (geometría de paralelepípedo) y captura el comportamiento dominante en geometrías irregulares moderadas. Soporta esquemas alternativos vía el parámetro `quadrature` desde `QuadratureRegistry` (e.g. `hex_3x3x3` para problemas con materiales no lineales severos).

Aviso sobre integración reducida: el esquema $1 \times 1 \times 1$ (un solo punto central) está disponible pero el código emite advertencia explícita; un único punto no detecta los modos de hourglass (energía nula con desplazamientos alternados) que en 3D son **doce** modos espurios (no cuatro como en 2D). No se implementa estabilización tipo Flanagan-Belytschko. Recomendación operativa: usar el default $2 \times 2 \times 2$ salvo necesidad explícita.

12. Cargas distribuidas consistentes

Fuerza de cuerpo $\mathbf{b}$ (e.g. peso propio $\mathbf{b} = (0, 0, -\rho g)$). Vector nodal equivalente:

$$\mathbf{f}_e^{\mathbf{b}} = \int_{\Omega_e} \mathbf{N}^\top \mathbf{b}\, dV = \sum_{p=1}^{n_g} \mathbf{N}(\boldsymbol{\xi}_p)^\top \mathbf{b}\, \det \mathbf{J}_p\, w_p,$$

donde $\mathbf{N}$ es la matriz 3×24 de funciones de forma. Se integra con la regla $2 \times 2 \times 2$ de la masa aunque la rigidez use $1 \times 1 \times 1$ (ADR 0022). $\mathbf{b}$ es constante o un campo $\mathbf{b}(\mathbf{x})$.

Tracción de superficie $\bar{\mathbf{t}}$ sobre una cara $\Gamma_e \subset \partial\Omega_N$. Caras numeradas con normal saliente (ADR 0012):

Cara	Nodos	Normal saliente
0	(0, 3, 2, 1)	$-\zeta$ (inferior)
1	(4, 5, 6, 7)	$+\zeta$ (superior)
2	(0, 1, 5, 4)	$-\eta$ (frontal)
3	(1, 2, 6, 5)	$+\xi$ (derecha)
4	(2, 3, 7, 6)	$+\eta$ (trasera)
5	(3, 0, 4, 7)	$-\xi$ (izquierda)

$$\mathbf{f}_e^t = \int_{\Gamma_e} \mathbf{N}^\top \, \bar{\mathbf{t}} \, dS.$$

Cuadratura 2D sobre la cara: Gauss 2×2 (cuatro puntos por cara). Exacto para tracción constante sobre cara plana. $\bar{\mathbf{t}}$ se especifica en **coordenadas globales**: constante o campo $\bar{\mathbf{t}}(\mathbf{x}, \mathbf{n})$ de la posición y de la normal exterior unitaria de la cara, que sale de la orientación antihoraria de ‘`FACE_NODES`‘ vista desde fuera (ADR 0022); una presión p es $\bar{\mathbf{t}} = -p\,\mathbf{n}$. Se integra sobre la geometría real de la cara con sus funciones de forma.

13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve un dict con $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ en cada uno de los n_g puntos de Gauss del elemento (8 por defecto), junto con coordenadas naturales y globales. Habilita post-proceso fino (mapas no promediados, suavizado nodal, recovery superconvergente futuro). Por ADR 0012, `internal_forces` devuelve `None` (sólidos no tienen fuerzas internas seccionales discretas — son su campo $\boldsymbol{\sigma}(x)$ continuo).

6.2.3 Limitaciones declaradas

Locking volumétrico con $\nu \to 0.5$

`Hex8` con integración completa $2 \times 2 \times 2$ y material casi-incompresible (`Elastic3D` con $\nu \to 0.5$, o un futuro `VonMises3D` en régimen plástico desarrollado) sufre **locking volumétrico**: la rigidez crece artificialmente porque el espacio de funciones trilineal no contiene desplazamientos puramente isocóricos suficientes. El elemento se queda atrapado en respuestas erróneamente rígidas.

Tratamiento en este proyecto: declarar limitación, **blindar con test** (`test_volumetric_locking_3d.py`, espejo del análogo 2D) que documente cuantitativamente el colapso, **no implementar mitigación** (B-bar, F-bar, mixed u-p). Política idéntica al `Quad4` (ver `STATUS.md` "Limitaciones declaradas").

Hourglass con integración reducida

`Hex8` integrado con un solo punto central tiene **12 modos de hourglass** (energía nula con desplazamientos alternados). Reducción disponible vía `quadrature` con warning; estabilización Flanagan-Belytschko no implementada.

Mass lumping en orientación arbitraria

`compute_mass_matrix(lumping="lumped")` usa HRZ canónico (`solidum/math/mass_lumping.py::lump_hrz`). Diagonal en ejes locales del elemento. En sólidos 3D la matriz lumped traslacional es estrictamente diagonal porque $\mathtt{m_t} \cdot \mathtt{I}_3$ es invariante bajo SO(3) — sin el caveat de `Frame3D` que sí aparece en su bloque rotacional (no hay rotaciones nodales en sólidos).

6.2.4 Contrato de implementación

```
name: Hex8
kind: element
status: validated
```

```yaml
interface:
  dof_names: [ux, uy, uz]
  n_nodes: 8
  strain_dim: 6                  # Voigt 3D ε[_xx, ε_yy, ε_zz, γ_xy, γ_yz, γ_xz]
  n_integration_points: 8        # default Gauss 2×2×2

admission:                 # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 3 (hexaedro trilineal)"
    - zienkiewicz_taylor: "vol. 1 (6.ª ed.), cap. 4: hexaedros"
  general_codes:
    - abaqus: "C3D8"

parameters:
  - { name: quadrature, type: str, required: false, default: "hex_2x2x2",
      desc: "Regla de cuadratura desde QuadratureRegistry (hex_2x2x2 default, hex_3x3x3
           alternativa)" }

material_contract:
  signature: "compute_stateε(, state) -> σ(, C_tangent, state')"
  strain_kind: "Voigt 3D ε[_xx, ε_yy, ε_zz, γ_xy, γ_yz, γ_xz] con γ_ij = 2·ε_ij"

conventions:
  sign: "σ > 0 ⇔ tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] con γ_ij = 2·ε_ij (ADR 0012)"
  node_orientation: "convención VTK_HEXAHEDRON; nodos 0-3 cara inferior antihorario
      visto desde dentro, 4-7 cara superior antihorario visto desde fuera; asegura det
      J > 0"
  face_numbering: "ADR 0012 - 6 caras con normal saliente: 0 -ζ(), 1 ζ(+), 2 -η(), 3
      ξ(+), 4 η(+), 5 -ξ()"

validity:
  - "pequeños desplazamientos y deformaciones"
  - "geometría sin distorsión severa (det J > tol en todos los puntos de Gauss)"
  - "compatible con materiales con STRAIN_DIM = 6 (en esta etapa: Elastic3D)"

out_of_scope:
  - "grandes deformaciones, formulaciones corotacionales o lagrangianas totales"
  - "estabilización contra hourglass para integración reducida"
  - "elementos de alto orden (Hex20, Hex27 - sub-etapa posterior)"
  - "tracción variable sobre cara o presión normal"
  - "internal_forces (ADR 0012: sólidos exponen compute_gauss_state)"

acceptance:
  verification:
    - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
      setup: "u* suave no polinómico; cubo distorsionado (caras planas con nodos
          deslizados, elementos interiores curvos) y sector de cilindro hueco con
          tracción sobre las caras cilíndricas; Elastic3D; mallas n×n×n, n = 4, 8, 16"
      expect: "orden observado entre los dos últimos niveles ≥ 2 - 0.1 en L² y ≥ 1 -
          0.1 en energía; error decreciente en cada nivel"
      tol_abs: 0.1
    - name: patch_test_3d
      setup: "8 Hex8 alrededor de un nodo interior (cubo 2×2×2 partido en
             ocho octantes) con campo lineal u_i = a_ij·x_j impuesto en
             los 26 nodos del contorno; nodo interior libre"
```

```
      expect: "nodo interior adopta el campo lineal exacto y ε constante e
               igual a 0.5·(a_ij + a_ji) en todos los puntos de Gauss; σ
               uniforme"
      tol_rel: 1.0e-10
    - name: modos_de_cuerpo_rigido
      setup: "evaluar K_e sobre un Hex8 cualquiera y proyectar sobre los 6
             modos de cuerpo rígido (3 traslaciones + 3 rotaciones)"
      expect: "K_e · u_rigid ≈ 0 en cada uno de los 6 modos"
      tol_abs: 1.0e-9
    - name: simetria_K_e
      setup: "evaluar K_e con Elastic3D"
      expect: "K_e == K_e.T exacto"
      tol_abs: 1.0e-12
  specific:
    - name: cubo_lame_3d
      setup: "cubo unitario con Elastic3D; tracción uniaxial σ_xx = p en
             cara ξ+; cara opuesta restringida en ux; restricciones
             mínimas para evitar movimiento rígido"
      expect: "u_x(L, y, z) = p·L/E; u_y = ν-·p·y/E; u_z = ν-·p·z/E
               exactos en todos los nodos"
      tol_rel: 1.0e-10
    - name: traccion_uniaxial_sobre_cara
      setup: "Hex8 con tracción constante (p, 0, 0) sobre cara 3 ξ(+)"
      expect: "Σf_x = p·A_cara, Σf_y = Σf_z = 0; reparto p·A_cara/4 a cada
               nodo de la cara para Hex8 regular"
      tol_rel: 1.0e-12
    - name: body_load_uniforme
      setup: "Hex8 regular y distorsionado con b uniforme"
      expect: "Σf = b·V_e (invariante de geometría); por simetría en Hex8
               regular cada nodo recibe b·V_e / 8"
      tol_rel: 1.0e-10
    - name: jacobiano_degenerado_abortado
      setup: "Hex8 con cuatro nodos coplanares de la cara superior sobre
             la inferior (det J ≈ 0)"
      expect: "ValueError en _compute_kinematics"
    - name: locking_volumetrico_documentado
      setup: "viga esbelta empotrada-libre con Hex8 plane strain forzado
             (cara z restringida); ν∈ {0.3, 0.49, 0.4999}"
      expect: "deflexión vs Frame3D decrece monotonamente con ν→ 0.5
               (locking documentado, no mitigado)"
      tol_rel: "documentar valores, sin tolerancia de aceptación"

references:
  - "Bathe K.J. (2014). Finite Element Procedures. §6.6 (sólidos 3D isoparamétricos)."
  - "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and Applications
     of FEA. §6.8."
  - "Zienkiewicz O.C., Taylor R.L. (2005). The Finite Element Method, vol. 1, §6.7."
  - "ADR 0012 - Sólidos 3D: convención Voigt 6D y caras."
```

6.2.5 Implementación

- **Archivo**: solidum/elements/solid_3d/hex8.py.
- **Clase**: `Hex8` (registrada vía `@ElementRegistry.register`).

- **Funciones núcleo**: `_compute_kinematics_hex8(xi, eta, zeta, coords)`, `_det_jacobian_hex8(...)` y `_shape_functions_hex8(...)` con `@njit` (Numba) en solidum/elements/solid_3d/_shared.py. Integrandos comunes 3D (`_compute_integrands_3d`) y expansor masa traslacional (`_expand_scalar_mass_3d`) también en `_shared`.
- **Tests**:
- tests/validation/test_mms_solid_3d.py (soluciones manufacturadas), tests/test_distributed_loads_3d.py (cargas variables).
- tests/test_solid_3d.py — clase `TestHex8Element` (10 tests: dimensiones/DOFs, simetría K, patch tracción uniaxial, jacobiano negativo abortado, body load, face traction (balance + nodos activos + índice fuera de rango), masa consistente y lumped HRZ, gauss state).
- tests/test_rigid_body_modes.py — `TestRBMHex8` (5 tests: traslación, 3 rotaciones independientes en (x, y, z), rank-deficiency = 6 modos rígidos).
- tests/validation/test_cube_lame_3d.py — `test_uniaxial_traction_hex8_exact`, `test_hydrostatic_compression_hex8` (solución exacta a precisión máquina).
- tests/validation/test_macneal_beam_3d.py — shear locking documentado en malla coarse + convergencia h monótona.
- tests/test_volumetric_locking_3d.py — locking volumétrico declarado y blindado por test.

6.2.6 Diálogo

- **2026-05-19** · Spec inicial. Elemento espejo natural de `Quad4`; convención Voigt 6D y numeración de caras fijadas por ADR 0012. Sin variantes corotacionales ni de alto orden en esta etapa. Locking volumétrico declarado como limitación con test que lo blinda; sin mitigación implementada (política idéntica al 2D).
- **2026-09-22** · Auditoría global: `quadrature` acepta clave del registro o tupla `(points, weights)` en todos los sólidos (`resolve_quadrature` valida la familia por medida de referencia y dimensión). La masa consistente se integra siempre con la regla completa 2×2×2 aunque K use integración reducida (con la regla del elemento y 1 punto, M quedaba de rango 3/24 y el problema modal era singular). tolerancia del jacobiano ahora **relativa** (`det J` $\leq$ `JACOBIAN_RTOL` · $\Pi\|$`fila_i(J)`$\|$, cota de Hadamard, adimensional): la absoluta `1e-10` sobre `det J` rechazaba mallas finas en metros sin distorsión.
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): la cinemática del elemento pasa a un núcleo compilado compartido (`_kin3d_from_dN`) y se expone como `BATCH_KINEMATICS`; `compute_element_state` no cambia de formulación. El `Assembler` agrupa los elementos en familias y los evalúa en un kernel único; equivalencia con el camino por elemento verificada a 1e-14 en `K` y `F_int` sobre todos los materiales compatibles (`tests/test_batch_assembly.py`).
- **2026-09-22** · Cierre del ADR 0014 (§9): la cinemática por lotes (`BATCH_KINEMATICS`) ya **no lanza** con jacobiano degenerado —devuelve `det J` $\leq$ `0` y el ensamblador lanza el `ValueError` con el id del elemento—, porque el kernel de familia tiene ahora variante paralela (`prange`, bit

a bit igual a la serie) y una excepción dentro de la región paralela se pierde; el camino por elemento sigue lanzando como siempre. Declara las funciones de forma para el post-proceso por familia (`Family.gauss_state`, mismo resultado que `compute_gauss_state`, verificado en `tests/test_batch_postprocess.py`). Formulación sin cambios.

- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022, fase 2). `compute_body_load` acepta un campo `b(x)` y `compute_face_traction` un campo `t(x, n)`, con `n` la normal exterior unitaria de la cara (una presión es `-p·n`); implementación común en `_SolidLoads3D`. La fuerza de cuerpo se integra con la regla de la masa (2×2×2, también con rigidez 1×1×1); la de cara, sobre la geometría real con las funciones de forma de la cara. Declara `REFERENCE_CELL` y `REFERENCE_NODES`. Nueva aceptación «soluciones manufacturadas»: orden 2 en L^2 y 1 en energía en el cubo distorsionado y en el cilindro hueco.

6.3 Tet10

Elemento sólido 3D de orden cuadrático sobre el simplex tetraédrico. Análogo 3D del '`Tri6 2D. Reproduce campos cuadráticos completos en (ξ, η, ζ). Mitiga drásticamente el shear locking y el locking volumétrico del Tet4 (CST 3D). Sub-fase 3 de A.ter; subclase de _HigherOrderSolid3D`'.

6.3.1 Especificación física

0. Descripción general

Tetraedro tridimensional de segundo orden con **10 nodos**: 4 vértices + 6 medios de arista. Aproximación cuadrática C^0 en coordenadas baricéntricas L_1, L_2, L_3, L_4 con $\sum_i L_i = 1$. Formulación isoparamétrica. Régimen de pequeños desplazamientos y deformaciones.

1-5. Cinemática, ecuación constitutiva, equilibrio

Idénticas al '`Hex20/Hex27`' — la diferencia con los hexaedros está exclusivamente en la geometría de referencia (simplex tetraédrico vs cubo).

6.3.2 Formulación numérica (FEM)

6. Discretización — convención VTK_QUADRATIC_TETRA

Los 10 nodos sobre el tetraedro de referencia con vértices $(0,0,0)$, $(1,0,0)$, $(0,1,0)$, $(0,0,1)$:

Nodo	**Tipo**	**Coords naturales** (ξ, η, ζ)
0	vértice (idéntico Tet4)	$(0, 0, 0)$
1	vértice (idéntico Tet4)	$(1, 0, 0)$
2	vértice (idéntico Tet4)	$(0, 1, 0)$
3	vértice (idéntico Tet4)	$(0, 0, 1)$
4	medio arista 0-1	$(0.5, 0, 0)$
5	medio arista 1-2	$(0.5, 0.5, 0)$
6	medio arista 2-0	$(0, 0.5, 0)$
7	medio arista 0-3	$(0, 0, 0.5)$
8	medio arista 1-3	$(0.5, 0, 0.5)$
9	medio arista 2-3	$(0, 0.5, 0.5)$

Mapeo isoparamétrico desde el tetraedro de referencia con $L_1 = 1 - \xi - \eta - \zeta$, $L_2 = \xi$, $L_3 = \eta$, $L_4 = \zeta$.

7. Funciones de forma

Vértices ($i \in \{1, 2, 3, 4\}$ — corresponde a los nodos locales $\{0, 1, 2, 3\}$):

$$N_i = L_i\,(2\,L_i - 1).$$

Medios de arista entre vértices i, j:

$$N_{ij} = 4\,L_i\,L_j.$$

Las seis aristas se etiquetan en el orden VTK_QUADRATIC_TETRA: $(1,2), (2,3), (3,1), (1,4), (2,4), (3,$ — nodos locales $(0,1), (1,2), (2,0), (0,3), (1,3), (2,3)$. Garantizan partición de la unidad $\sum N_i = 1$ y propiedad delta de Kronecker.

Espacio reproducido: el ‘`Tet10`‘ reproduce **completamente** el espacio cuadrático en barycentric coords — todos los polinomios de grado total ≤ 2 en (ξ, η, ζ). 10 monomios, idéntico al número de nodos.

8. Matriz B

Tamaño **6×30**, ensamblada con la misma plantilla 6×3 del ‘`Hex8/Hex20/Hex27`‘ por nodo. Derivadas en coords naturales obtenidas con regla del producto: para vértices $\partial N_i/\partial x_k = (4L_i - 1)\,\partial L_i/\partial x_k$; para medios $\partial N_{ij}/\partial x_k = 4(L_j\,\partial L_i/\partial x_k + L_i\,\partial L_j/\partial x_k)$.

9-10. Rigidez y fuerzas internas

Implementadas en la base ‘`_HigherOrderSolid3D`‘. Tamaño 30×30 para $\mathbf{K}_e$.

11. Cuadratura

Por defecto **Stroud 4 puntos** (‘`tet_4`‘): orden de exactitud 2. Para Tet10 con Jacobiano constante el integrando de $\mathbf{B}^\top\mathbf{D}\,\mathbf{B}$ es a lo sumo de grado 2 (B es lineal en barycentric), así que la regla es **exacta** para K.

Para problemas con Jacobiano variable (geometría distorsionada, mid-edges fuera del centro real) el integrando puede subir de grado; en esos casos el usuario puede seleccionar `tet_15` (Keast 15 puntos, orden 5) vía el parámetro `quadrature`.

Cuadratura de masa: el ‘`Tet10 declara _MASS_QUADRATURE = "tet_15" (Keast 15 puntos, orden 5) **independientemente** de la cuadratura de K. El integrando` $\rho\mathbf{N}^\top\mathbf{N}$ `es de grado 4 (cuadrático × cuadrático) y la regla de 4 puntos lo subintegra; tet_15 lo integra exactamente. Esto garantiza que la masa total sea correcta y que análisis modal/transitorio con` Tet10‘ no sufran errores de cuadratura.

12. Cargas distribuidas consistentes

Body load: ‘$\int$ `N^T b dV con la regla tet_15 de la masa (ADR 0022; hasta el 2026-10-27, la tet_4 del elemento); b constante o campo b(x). Reparto no uniforme entre vértices y medios; suma global b·V_e exacta para b`‘ uniforme y geometría sin distorsión severa.

Face traction: cada cara del ‘`Tet10 tiene **6 nodos** (3 vértices + 3 medios de arista) y se aproxima por funciones cuadráticas Tri6 sobre el triángulo de referencia` $(0,0)$-$(1,0)$-$(0,1)$. `Cuadratura tri_6 (Dunavant, orden 4), exacta para` N_kN_l `de la cara (hasta el 2026-10-27, tri_3, exacta sólo para tracción constante).` $\bar{\mathbf{t}}$ `se especifica en **coordenadas globales**: constante o campo` $\bar{\mathbf{t}}(\mathbf{x}, \mathbf{n})$ `de la posición y de la normal exterior unitaria de la cara, que sale de la orientación antihoraria de FACE_NODES`‘ vista desde fuera (ADR 0022); una presión p es $\bar{\mathbf{t}} = -p\,\mathbf{n}$. Se integra sobre la geometría real de la cara con sus funciones de forma.

Cara	Vértices Tet10	Medios Tet10 (en orden Tri6)	Normal saliente
0	(1, 2, 3)	(5, 9, 8)	opuesta al nodo 0
1	(0, 3, 2)	(7, 9, 6)	opuesta al nodo 1
2	(0, 1, 3)	(4, 8, 7)	opuesta al nodo 2
3	(0, 2, 1)	(6, 5, 4)	opuesta al nodo 3

El orden dentro de cada cara reproduce la convención de ‘`_N_tri6 del paquete 2D: vértices` (0, 1, 2) `antihorarios, luego medios` (0-1, 1-2, 2-0)‘.

13. Salida por punto de Gauss

`compute_gauss_state(U)` con 4 puntos por defecto. Sin `internal_forces` (ADR 0012).

6.3.3 Limitaciones declaradas

Locking volumétrico atenuado pero presente

`Tet10` con cuadratura ‘`tet_4 y material casi-incompresible` (Elastic3D con $\nu \to 0.5$, `o` VonMises3D `plástico desarrollado) sufre **locking volumétrico** mucho menor que el` Tet4 `(cuyo shear locking severo es la motivación principal del` Tet10`), pero todavía presente. Sin mitigación implementada. Blindado por test (mismo patrón que` Hex20/Hex27‘).

Distorsión severa

Para Tet10 con mid-edges desplazados fuera del centro real de la arista (mid-edges curvos para representar geometría curva), el Jacobiano deja de ser constante y la cuadratura ‘`tet_4 puede subintegrar K. Recomendación: usar tet_15 en mallas con curvatura severa. *Medido el 2026-10-27 (ADR 0022)*: con lados curvos, tet_4 no pasa la prueba de la parcela exactamente (queda un error de consistencia que baja como h^3.3:` $8{\cdot}10^{-8}$ `y` $7{\cdot}10^{-9}$ `con n = 2 y 4 en el cubo distorsionado); con tet_15 es exacta. El orden de convergencia no se resiente: el barrido de soluciones manufacturadas da orden 3 en` L^2 `y 2 en energía con tet_4‘.

6.3.4 Contrato de implementación

```yaml
name: Tet10
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 10
  strain_dim: 6
  n_integration_points: 4       # default Stroud tet_4

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 3 (tetraedro cuadrático)"
```

```
    - zienkiewicz_taylor: "vol. 1 (6.ª ed.), cap. 4: tetraedros"
  general_codes:
    - abaqus: "C3D10"

parameters:
  - { name: quadrature, type: str, required: false, default: "tet_4",
      desc: "Cuadratura tetraédrica desde QuadratureRegistry. Default
             tet_4 (Stroud 4 puntos, orden 2, suficiente para K con J
             constante). Alternativa tet_15 (Keast 15 puntos, orden 5)
             para geometrías distorsionadas." }

material_contract:
  signature: "compute_stateε(, state) -> σ(, C_tangent, state')"
  strain_kind: "Voigt 3D ε[_xx, ε_yy, ε_zz, γ_xy, γ_yz, γ_xz] con γ_ij = 2·ε_ij"

conventions:
  sign: "σ > 0 ⇔ tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] con γ_ij = 2·ε_ij (ADR 0012)"
  node_orientation: "VTK_QUADRATIC_TETRA; 0-3 vértices (Tet4 con volumen
                     positivo), 4-9 medios de arista en orden (0-1, 1-2,
                     2-0, 0-3, 1-3, 2-3)"
  face_numbering: "ADR 0012 - 4 caras triangulares con normal saliente,
                   paritarias con Tet4 en los vértices; cada cara añade
                   3 medios de arista en orden Tri6"

validity:
  - "pequeños desplazamientos y deformaciones"
  - "geometría sin distorsión severa (det J > tol en todos los Gauss)"
  - "compatible con materiales con STRAIN_DIM = 6"

out_of_scope:
  - "grandes deformaciones, formulaciones corotacionales"
  - "internal_forces (ADR 0012)"
  - "mid-edges severamente curvos (usar tet_15 si es necesario)"

acceptance:
  verification:
    - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
      setup: "u* suave no polinómico; cubo distorsionado (caras planas con nodos
          deslizados, elementos interiores curvos) y sector de cilindro hueco con
          tracción sobre las caras cilíndricas; Elastic3D; mallas n×n×n, n = 2, 4, 8"
      expect: "orden observado entre los dos últimos niveles ≥ 3 - 0.1 en L² y ≥ 2 -
          0.1 en energía; error decreciente en cada nivel"
      tol_abs: 0.1
    - name: patch_test_lineal
      setup: "campo lineal u_i = a_ij·x_j en los 10 nodos del contorno"
      expect: "ε constante = sym(a) en todos los Gauss"
      tol_rel: 1.0e-12
    - name: patch_test_cuadratico
      setup: "campo u_x = c·x² en los 10 nodos"
      expect: "ε_xx(x) = 2c·x exacto en todos los Gauss"
      tol_rel: 1.0e-12
    - name: modos_de_cuerpo_rigido
      setup: "K_e proyectado sobre 6 modos rígidos 3D"
      expect: "exactamente 6 autovalores nulos"
      tol_abs: 1.0e-9
```

```
    - name: simetria_K_e
      setup: "K_e con Elastic3D"
      expect: "K_e == K_e.T exacto"
      tol_abs: 1.0e-12
  specific:
    - name: cubo_lame_3d_uniaxial
      setup: "malla 5 tetraedros del cubo unitario, refinados a Tet10,
              con tracción uniaxial; el campo analítico es lineal y
              Tet10 lo reproduce exacto"
      expect: "u_x = p·x/E etc. exacto a precisión máquina"
      tol_rel: 1.0e-10
    - name: macneal_3d_tet10_dominates_tet4
      setup: "cantilever de Tet10 con la misma malla que Tet4 sufre
              dramáticamente menos shear locking"
      expect: "Tet10 alcanza > 80% u_EB con ~12 tetraedros (Tet4 << 20%)"
    - name: body_load_uniforme
      setup: "Tet10 ref con b = (0, 0, -g)"
      expect: "Σf_z = -g·V_e = -g/6"
      tol_rel: 1.0e-12
    - name: face_traction_balance
      setup: "tracción uniforme sobre una cara del Tet10"
      expect: "Σf = A_face · t̄ exacto; nodos fuera de la cara reciben 0"
      tol_rel: 1.0e-12
    - name: mass_consistent_total_with_tet15
      setup: "Tet10 cubo ref con ρ=7850, masa consistente con tet_15"
      expect: "Σ M = 3·ρ·V_e = 3·ρ/6"
      tol_rel: 1.0e-9
    - name: mass_lumped_HRZ_positivo
      setup: "masa lumped HRZ del Tet10"
      expect: "diagonal estricta, todas las entradas positivas,
              conservación de masa total"
      tol_abs: 1.0e-12

references:
  - "Bathe K.J. (2014). Finite Element Procedures. §5.3.3 (tetraedro 3D)."
  - "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and
    Applications of FEA. §6.5."
  - "Keast P. (1986). Moderate-degree tetrahedral quadrature formulas.
    CMAME 55(3), 339-348 (regla tet_15 de masa)."
  - "Stroud A.H. (1971). Approximate calculation of multiple integrals
    (regla tet_4 de K)."
  - "ADR 0012 - Sólidos 3D: convención Voigt 6D y caras."
```

6.3.5 Implementación

- **Archivo**: solidum/elements/solid_3d/tet10.py.
- **Clase**: `Tet10` (registrada vía `@ElementRegistry.register`); subclase de `_HigherOrderSolid3D`.
- **Funciones núcleo**: `_N_tet10(xi, eta, zeta)`, `_dN_tet10(xi, eta, zeta)` en solidum/elements/solid_3d/_shared.py. Implementadas en numpy puro vía coordenadas baricéntricas L_i.
- **Cuadraturas registradas**: `tet_4` (Stroud, orden 2) y `tet_15` (Keast, orden 5) en soli-

dum/math/integration.py. El `tet_4` cubre la integración de K; el `tet_15` se usa exclusivamente para la masa consistente (declarada en `Tet10._MASS_QUADRATURE`).

- **Tests**:
- tests/validation/test_mms_solid_3d.py (soluciones manufacturadas), tests/test_distributed_loads_3d.py (cargas variables).
- `tests/test_solid_3d_higher_order.py` — `TestTet10Element` (patch lineal + cuadrático, simetría K, body load, face traction, masa consistente con tet_15 y lumped HRZ, gauss state).
- `tests/test_rigid_body_modes.py` — `TestRBMTet10` (3 trans + 3 rot + rank-deficiency = 6).
- `tests/validation/test_cube_lame_3d.py` — extensión con malla Tet10 del cubo unitario; uniaxial e hidrostática exactas a precisión máquina.

6.3.6 Diálogo

- **2026-05-27** · Spec inicial. Sub-fase 3 de A.ter — cierra el catálogo cuadrático 3D con la rama tetraédrica. Subclase de `_HigherOrderSolid3D` igual que `Hex20`/`Hex27`, pero con cara triangular (`Tri6` shape functions importadas del paquete 2D, cuadratura `tri_3`) y cuadratura volumétrica tetraédrica nueva (`tet_4` + `tet_15`).
- **2026-05-27** · Implementado y validado. Patch test lineal y cuadrático exactos a precisión máquina, 6 modos rígidos, body load suma exacta $\mathtt{b}\cdot\mathtt{V_e}$, face traction balance exacto sobre la cara triangular, masa consistente con `tet_15` exacta a precisión máquina (conservación $3\cdot\rho\cdot\mathtt{V}$), lumped HRZ con todas las entradas positivas. Spec a `status: validated`.
- **2026-09-22** · Auditoría global: `quadrature` acepta clave del registro o tupla `(points, weights)` en todos los sólidos (`resolve_quadrature` valida la familia por medida de referencia y dimensión). tolerancia del jacobiano ahora **relativa** ($\mathtt{det\ J} \le \mathtt{JACOBIAN_RTOL} \cdot \Pi\|\mathtt{fila_i(J)}\|$, cota de Hadamard, adimensional): la absoluta `1e-10` sobre `det J` rechazaba mallas finas en metros sin distorsión.
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): la cinemática del elemento pasa a un núcleo compilado compartido (`_kin3d_from_dN`) y se expone como `BATCH_KINEMATICS`; `compute_element_state` no cambia de formulación. El `Assembler` agrupa los elementos en familias y los evalúa en un kernel único; equivalencia con el camino por elemento verificada a 1e-14 en `K` y `F_int` sobre todos los materiales compatibles (`tests/test_batch_assembly.py`).
- **2026-09-22** · Cierre del ADR 0014 (§9): la cinemática por lotes (`BATCH_KINEMATICS`) ya **no lanza** con jacobiano degenerado —devuelve $\mathtt{det\ J} \le \mathtt{0}$ y el ensamblador lanza el `ValueError` con el id del elemento—, porque el kernel de familia tiene ahora variante paralela (`prange`, bit a bit igual a la serie) y una excepción dentro de la región paralela se pierde; el camino por elemento sigue lanzando como siempre. Declara las funciones de forma para el post-proceso por familia (`Family.gauss_state`, mismo resultado que `compute_gauss_state`, verificado en `tests/test_batch_postprocess.py`). Formulación sin cambios.

- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022, fase 2). `compute_body_load` acepta un campo `b(x)` y `compute_face_traction` un campo `t(x, n)`, con `n` la normal exterior unitaria de la cara (una presión es `-p·n`); implementación común en `_SolidLoads3D`. La fuerza de cuerpo se integra con la regla de la masa (`tet_15`; antes `tet_4`, y la cara con `tri_6`, antes `tri_3`); la de cara, sobre la geometría real con las funciones de forma de la cara. Declara `REFERENCE_CELL` y `REFERENCE_NODES`. Nueva aceptación «soluciones manufacturadas»: orden 3 en L^2 y 2 en energía en el cubo distorsionado y en el cilindro hueco. Las reglas antiguas también daban el orden óptimo (una regla exacta de grado $2p - 2$ basta): el cambio mejora la exactitud, no el orden. Con `tet_4` y lados curvos la parcela no es exacta (ver «Distorsión severa»).

6.4 Hex20

Elemento sólido 3D de orden cuadrático con interpolación serendípita.

Reproduce campos cuadráticos completos en 3D (todos los polinomios de grado total $\leq$ 2 más algunos de orden 3 sin los términos triquadráticos). Es al ***Hex8*** *lo que el* ***Quad8*** *al* ***Quad4****: mucho más preciso en flexión y geometrías curvas con el mismo número de elementos. Primer elemento de la sub-etapa A.ter.*

6.4.1 Especificación física

0. Descripción general

Hexaedro tridimensional de segundo orden con **20 nodos**: 8 vértices + 12 medios de arista, sin nodos de cara ni centroide. Aproximación serendípita C^0 en coordenadas naturales $(\xi, \eta, \zeta) \in [-1, 1]^3$. Formulación isoparamétrica: la geometría y los desplazamientos comparten las mismas funciones de forma. Régimen de pequeños desplazamientos y deformaciones.

1. Cinemática de desplazamientos

$$u_x(\xi, \eta, \zeta) = \sum_{i=0}^{19} N_i(\xi, \eta, \zeta)\, u_x^{(i)}, \qquad u_y = \sum N_i\, u_y^{(i)}, \qquad u_z = \sum N_i\, u_z^{(i)}.$$

2. Cinemática de deformaciones

Tensor infinitesimal en notación Voigt 6D del proyecto (ADR 0012):

$$\boldsymbol{\varepsilon} = [\varepsilon_{xx},\ \varepsilon_{yy},\ \varepsilon_{zz},\ \gamma_{xy},\ \gamma_{yz},\ \gamma_{xz}]^\top$$

con $\gamma_{ij} = 2\,\varepsilon_{ij}$ *engineering*. Idéntica al `Hex8`.

3. Ecuación constitutiva

Delegada al material 3D (`STRAIN_DIM = 6`). El elemento llama `material.compute_state(`ε`, state)` $\rightarrow$ `(`σ`, C_tangent, state')` por punto de Gauss. Compatible con todo el catálogo 3D actual: `Elastic3D`, `VonMises3D`, `DruckerPrager3D`, `IsotropicDamage3D`.

4-5. Equilibrio

Forma fuerte y débil idénticas al `Hex8` — la diferencia entre `Hex8` y `Hex20` está exclusivamente en el espacio de aproximación discreto.

6.4.2 Formulación numérica (FEM)

6. Discretización

Veinte nodos en orden VTK_QUADRATIC_HEXAHEDRON (estándar gmsh/VTK/Abaqus). En el cubo de referencia $[-1, 1]^3$:

Nodo local	(ξ_i, η_i, ζ_i)	Tipo
0	$(-1,-1,-1)$	vértice
1	$(+1,-1,-1)$	vértice
2	$(+1,+1,-1)$	vértice
3	$(-1,+1,-1)$	vértice
4	$(-1,-1,+1)$	vértice
5	$(+1,-1,+1)$	vértice
6	$(+1,+1,+1)$	vértice
7	$(-1,+1,+1)$	vértice
8	$(0,-1,-1)$	medio arista 0-1 (eje ξ, cara inferior)
9	$(+1,0,-1)$	medio arista 1-2 (eje η, cara inferior)
10	$(0,+1,-1)$	medio arista 2-3 (eje ξ, cara inferior)
11	$(-1,0,-1)$	medio arista 3-0 (eje η, cara inferior)
12	$(0,-1,+1)$	medio arista 4-5 (eje ξ, cara superior)
13	$(+1,0,+1)$	medio arista 5-6 (eje η, cara superior)
14	$(0,+1,+1)$	medio arista 6-7 (eje ξ, cara superior)
15	$(-1,0,+1)$	medio arista 7-4 (eje η, cara superior)
16	$(-1,-1,0)$	medio arista 0-4 (eje ζ, vertical)
17	$(+1,-1,0)$	medio arista 1-5 (eje ζ, vertical)
18	$(+1,+1,0)$	medio arista 2-6 (eje ζ, vertical)
19	$(-1,+1,0)$	medio arista 3-7 (eje ζ, vertical)

Mapeo isoparamétrico desde el cubo de referencia:

$$x(\xi,\eta,\zeta) = \sum_{i=0}^{19} N_i\, x^{(i)}, \quad y = \sum N_i\, y^{(i)}, \quad z = \sum N_i\, z^{(i)}.$$

Jacobiano $\mathbf{J} = \partial(x,y,z)/\partial(\xi,\eta,\zeta)$ es una matriz 3×3 con determinante $\det \mathbf{J}$. El código aborta con `ValueError` si $\det \mathbf{J} \le$ tol en cualquier punto de Gauss (elemento severamente distorsionado o nodos en orden invertido).

7. Funciones de forma serendípitas

Vértices ($i \in \{0,\dots,7\}$, $(\xi_i,\eta_i,\zeta_i) \in \{\pm1\}^3$):

$$N_i = \frac{1}{8}(1+\xi_i\xi)(1+\eta_i\eta)(1+\zeta_i\zeta)(\xi_i\xi+\eta_i\eta+\zeta_i\zeta-2).$$

El multiplicador $(\xi_i\xi+\eta_i\eta+\zeta_i\zeta-2)$ vale 1 en el propio vértice y 0 en los demás vértices y en todos los medios de arista — garantía δ_{ij} del esquema.

Medios de arista paralelos a ξ (nodos 8, 10, 12, 14; $\xi_i = 0$, $\eta_i, \zeta_i \in \{\pm1\}$):

$$N_i = \frac{1}{4}(1-\xi^2)(1+\eta_i\eta)(1+\zeta_i\zeta).$$

Medios de arista paralelos a η (nodos 9, 11, 13, 15; $\eta_i = 0$):

$$N_i = \frac{1}{4}(1+\xi_i\xi)(1-\eta^2)(1+\zeta_i\zeta).$$

Medios de arista paralelos a ζ (nodos 16, 17, 18, 19; $\zeta_i = 0$):

$$N_i = \frac{1}{4}(1+\xi_i\xi)(1+\eta_i\eta)(1-\zeta^2).$$

Espacio reproducido: $\{1, \xi, \eta, \zeta, \xi^2, \eta^2, \zeta^2, \xi\eta, \eta\zeta, \xi\zeta, \xi^2\eta, \xi\eta^2, \eta^2\zeta, \eta\zeta^2, \xi^2\zeta, \xi\zeta^2, \xi\eta\zeta, \xi^2\eta\zeta, \xi\eta^2\zeta, \xi\eta\zeta^2\}$ — 20 monomios, omitiendo los puramente cúbicos (ξ^3, etc.) y el término $\xi^2\eta^2\zeta^2$.

8. Matriz deformación-desplazamiento

Las derivadas en globales se obtienen vía $\partial N_i/\partial \mathbf{x} = \mathbf{J}^{-1}\,\partial N_i/\partial \boldsymbol{\xi}$. La matriz $\mathbf{B}$ tiene tamaño **6×60** y se ensambla por nodo $i \in \{0, \ldots, 19\}$ con la misma plantilla 6×3 del `Hex8`:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i/\partial x & 0 & 0 \\ 0 & \partial N_i/\partial y & 0 \\ 0 & 0 & \partial N_i/\partial z \\ \partial N_i/\partial y & \partial N_i/\partial x & 0 \\ 0 & \partial N_i/\partial z & \partial N_i/\partial y \\ \partial N_i/\partial z & 0 & \partial N_i/\partial x \end{bmatrix}, \qquad \boldsymbol{\varepsilon} = \mathbf{B}\,\mathbf{u}_e.$$

El orden de las filas reproduce exactamente el orden Voigt del proyecto (ADR 0012). El orden de las columnas por nodo es `[ux, uy, uz]`, agrupado por nodo i para `i ∈ {0..19}`.

Derivadas explícitas (cerradas vía regla del producto):

Vértices ($i \in \{0, \ldots, 7\}$, con $u = \xi_i\xi$, $v = \eta_i\eta$, $w = \zeta_i\zeta$):

$$\frac{\partial N_i}{\partial \xi} = \frac{\xi_i}{8}(1+v)(1+w)(2u+v+w-1),$$

$$\frac{\partial N_i}{\partial \eta} = \frac{\eta_i}{8}(1+u)(1+w)(u+2v+w-1),$$

$$\frac{\partial N_i}{\partial \zeta} = \frac{\zeta_i}{8}(1+u)(1+v)(u+v+2w-1).$$

Medio paralelo a ξ (nodos 8, 10, 12, 14):

$$\frac{\partial N_i}{\partial \xi} = -\frac{\xi}{2}(1+\eta_i\eta)(1+\zeta_i\zeta), \quad \frac{\partial N_i}{\partial \eta} = \frac{\eta_i}{4}(1-\xi^2)(1+\zeta_i\zeta), \quad \frac{\partial N_i}{\partial \zeta} = \frac{\zeta_i}{4}(1-\xi^2)(1+\eta_i\eta).$$

Medio paralelo a η (nodos 9, 11, 13, 15):

$$\frac{\partial N_i}{\partial \xi} = \frac{\xi_i}{4}(1-\eta^2)(1+\zeta_i\zeta), \quad \frac{\partial N_i}{\partial \eta} = -\frac{\eta}{2}(1+\xi_i\xi)(1+\zeta_i\zeta), \quad \frac{\partial N_i}{\partial \zeta} = \frac{\zeta_i}{4}(1+\xi_i\xi)(1-\eta^2).$$

Medio paralelo a ζ (nodos 16, 17, 18, 19):

$$\frac{\partial N_i}{\partial \xi} = \frac{\xi_i}{4}(1+\eta_i\eta)(1-\zeta^2), \quad \frac{\partial N_i}{\partial \eta} = \frac{\eta_i}{4}(1+\xi_i\xi)(1-\zeta^2), \quad \frac{\partial N_i}{\partial \zeta} = -\frac{\zeta}{2}(1+\xi_i\xi)(1+\eta_i\eta).$$

9. Rigidez elemental

$$\mathbf{K}_e = \int_\Omega \mathbf{B}^\top \mathbf{D}\,\mathbf{B}\, dV = \sum_{p=1}^{n_g} \mathbf{B}(\boldsymbol{\xi}_p)^\top \mathbf{D}_p\, \mathbf{B}(\boldsymbol{\xi}_p)\, \det \mathbf{J}_p\, w_p.$$

Sin parámetro de espesor — el volumen ya está incluido en $\det \mathbf{J}_p \cdot w_p$.

10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sum_{p=1}^{n_g} \mathbf{B}(\boldsymbol{\xi}_p)^\top\, \boldsymbol{\sigma}_p\, \det \mathbf{J}_p\, w_p.$$

11. Cuadratura

Por defecto **Gauss–Legendre 3×3×3** (27 puntos): orden de exactitud 5 en cada dirección. Suficiente para integrar exactamente $\mathbf{B}^\top\mathbf{D}\,\mathbf{B}$ del `Hex20` sobre Jacobiano constante (el integrando es a lo sumo de grado 3 en cada dirección) y para integrar exactamente la masa consistente $\rho\mathbf{N}^\top\mathbf{N}$ (grado 4 en cada dirección).

Aviso sobre integración reducida: el esquema $2\times2\times2$ (`hex_2x2x2`, 8 puntos) está disponible y reduce el coste computacional, pero introduce **6 modos de hourglass espurios** sobre un elemento aislado (verificación numérica directa del proyecto; Belytschko-Liu-Moran §8.4.4). Estos modos **no propagan** a través de mallas ensambladas con condiciones de Dirichlet razonables — la rigidez global ensamblada queda rank-sufficient en la mayoría de casos prácticos, pero un elemento aislado o una capa con BC pobres puede mostrar hourglass. El código permite seleccionar la cuadratura reducida vía el parámetro `quadrature` pero **no estabiliza** los modos espurios. Recomendación operativa: usar el default $3\times3\times3$ salvo necesidad experimental. La matriz de masa se integra con $3\times3\times3$ independientemente de la cuadratura elegida para $\mathbf{K}$, para evitar subintegrarla.

12. Cargas distribuidas consistentes

Fuerza de cuerpo b. Vector nodal equivalente:

$$\mathbf{f}_e^{\boldsymbol{b}} = \int_{\Omega_e} \mathbf{N}^\top\, \mathbf{b}\, dV = \sum_{p=1}^{n_g} \mathbf{N}(\boldsymbol{\xi}_p)^\top\, \mathbf{b}\, \det \mathbf{J}_p\, w_p,$$

donde $\mathbf{N}$ es la matriz 3×60 de funciones de forma. Se integra con la regla $3\times3\times3$ de la masa, también cuando la rigidez usa $2\times2\times2$; $\mathbf{b}$ es constante o un campo $\mathbf{b}(\mathbf{x})$ (ADR 0022). Para $\mathbf{b}$ uniforme y geometría regular el resultado es exacto. **Para `Hex20` el reparto no es uniforme entre nodos**: la fórmula serendípita asigna pesos distintos a vértices y medios (los medios reciben más carga total que los vértices, con suma global $\rho \cdot V_e$ correcta).

Tracción de superficie $\bar{\mathbf{t}}$ sobre una cara. Cada cara del `Hex20` tiene **8 nodos** (4 vértices + 4 medios de arista) y se aproxima por funciones serendípitas Quad8 sobre el parámetro 2D $(s,t) \in [-1,1]^2$. La numeración de caras conserva la convención del `Hex8` (ADR 0012) extendida con los medios:

Cara	Vértices	Medios de arista	Normal saliente
0	(0,3,2,1)	(11, 10, 9, 8)	$-\zeta$ (inferior)
1	(4,5,6,7)	(12, 13, 14, 15)	$+\zeta$ (superior)
2	(0,1,5,4)	(8, 17, 12, 16)	$-\eta$ (frontal)
3	(1,2,6,5)	(9, 18, 13, 17)	$+\xi$ (derecha)
4	(2,3,7,6)	(10, 19, 14, 18)	$+\eta$ (trasera)
5	(3,0,4,7)	(11, 16, 15, 19)	$-\xi$ (izquierda)

El orden dentro de cada cara reproduce la convención VTK_QUADRATIC_HEXAHEDRON: primero los 4 vértices, luego los 4 medios en el mismo orden cíclico (medio entre vértice k y vértice $k+1$).

$$\mathbf{f}_e^t = \int_{\Gamma_e} \mathbf{N}^\top \bar{\mathbf{t}}\, dS.$$

Cuadratura 2D sobre la cara: Gauss 3×3 (nueve puntos) — exacta para tracción constante sobre cara plana (integrando degree 2 en (s,t)). $\bar{\mathbf{t}}$ se especifica en **coordenadas globales**: constante o campo $\bar{\mathbf{t}}(\mathbf{x}, \mathbf{n})$ de la posición y de la normal exterior unitaria de la cara, que sale de la orientación antihoraria de 'FACE_NODES vista desde fuera (ADR 0022); una presión p es $\bar{\mathbf{t}} = -p\,\mathbf{n}$. Se integra sobre la geometría real de la cara con sus funciones de forma. **Para Hex20' el reparto entre vértices y medios no es trivial** (la integral cuadrática de Quad8 sobre cara plana asigna fracciones específicas: los vértices reciben $-1/12$ de la fuerza total cada uno y los medios $4/12$ cada uno, con suma global exacta — ver test de balance).

13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve un dict con $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ en cada uno de los n_g puntos de Gauss del elemento (27 por defecto), junto con coordenadas naturales y globales. Habilita post-proceso fino (mapas no promediados, suavizado nodal, recovery superconvergente futuro). Por ADR 0012, `internal_forces` devuelve `None` (sólidos no tienen fuerzas internas seccionales discretas).

6.4.3 Limitaciones declaradas

Locking volumétrico con $\nu \to 0.5$

`Hex20` con integración completa $3 \times 3 \times 3$ y material casi-incompresible (`Elastic3D` con $\nu \to 0.5$, o `VonMises3D` en régimen plástico desarrollado) sufre **locking volumétrico** menos severo que el `Hex8` pero todavía presente: el espacio serendípito triquadrático tampoco contiene desplazamientos puramente isocóricos suficientes en mallas distorsionadas (Cook-Malkus-Plesha §13.6).

Tratamiento en este proyecto: declarar limitación, **blindar con test cuantitativo** (`test_volumetric_locking_hex20.py`, espejo del análogo `Hex8`) que documente la mitigación parcial respecto al lineal, **no implementar B-bar/F-bar/mixed u-p**. Política idéntica a `Quad4`/`Quad8` y `Hex8` (ver `STATUS.md` "Limitaciones declaradas").

Hourglass con integración reducida

`Hex20` integrado con `hex_2x2x2` tiene **6 modos de hourglass espurios** por elemento aislado (vs 12 del `Hex8` reducido). En mallas ensambladas con Dirichlet razonable los modos suelen no propagar — la rigidez global queda rank-sufficient — pero un elemento aislado o una capa con BC pobres lo mostraría. Reducción disponible vía `quadrature`; estabilización no implementada. Modo recomendado: full integration $3 \times 3 \times 3$.

Mass lumping

`compute_mass_matrix(lumping="lumped")` usa HRZ canónico (`solidum/math/mass_lumping.py::lump_hrz`). Para `Hex20` (con nodos de medio de arista) HRZ es la única opción razonable: row-sum produciría masas **negativas** en los vértices (los pesos de Gauss son negativos para los vértices en la cuadratura serendípita Lobatto equivalente), inadmisible para análisis dinámico (Bathe FEP §9.2.4). HRZ preserva masa total escalando la diagonal consistente por un factor común.

6.4.4 Contrato de implementación

```yaml
name: Hex20
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 20
  strain_dim: 6                 # Voigt 3D ε[_xx, ε_yy, ε_zz, γ_xy, γ_yz, γ_xz]
  n_integration_points: 27      # default Gauss 3×3×3

admission:                  # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 3 (elementos serendípitos)"
    - zienkiewicz_taylor: "vol. 1 (6.ª ed.), cap. 4: familia serendípita 3D"
  general_codes:
    - abaqus: "C3D20 / C3D20R (integración reducida 2×2×2)"

parameters:
  - { name: quadrature, type: str, required: false, default: "hex_3x3x3",
      desc: "Regla de cuadratura desde QuadratureRegistry (hex_3x3x3 default,
             hex_2x2x2 reducida con 4 modos de hourglass espurios)" }

material_contract:
  signature: "compute_stateε(, state) -> σ(, C_tangent, state')"
  strain_kind: "Voigt 3D ε[_xx, ε_yy, ε_zz, γ_xy, γ_yz, γ_xz] con γ_ij = 2·ε_ij"

conventions:
  sign: "σ > 0 ⇔ tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] con γ_ij = 2·ε_ij (ADR 0012)"
  node_orientation: "convención VTK_QUADRATIC_HEXAHEDRON; vértices 0-7 idénticos
                     al Hex8; medios 8-11 bottom edges, 12-15 top edges, 16-19
                     vertical edges; asegura det J > 0 en todos los Gauss"
  face_numbering: "ADR 0012 - 6 caras con normal saliente, idéntica a Hex8 en
                   los vértices; cada cara añade 4 nodos medios en el mismo
                   orden cíclico (Quad8 serendipity)"
```

```yaml
validity:
  - "pequeños desplazamientos y deformaciones"
  - "geometría sin distorsión severa (det J > tol en todos los puntos de Gauss)"
  - "compatible con materiales con STRAIN_DIM = 6 (Elastic3D, VonMises3D,
     DruckerPrager3D, IsotropicDamage3D)"

out_of_scope:
  - "grandes deformaciones, formulaciones corotacionales o lagrangianas totales"
  - "estabilización contra hourglass para integración reducida hex_2x2x2"
  - "tracción variable sobre cara o presión normal con gradiente"
  - "internal_forces (ADR 0012: sólidos exponen compute_gauss_state)"
  - "Hex27 lagrangiano (sub-fase 2 de A.ter)"

acceptance:
  verification:
    - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
      setup: "u* suave no polinómico; cubo distorsionado (caras planas con nodos
          deslizados, elementos interiores curvos) y sector de cilindro hueco con
          tracción sobre las caras cilíndricas; Elastic3D; mallas n×n×n, n = 2, 4, 8"
      expect: "orden observado entre los dos últimos niveles ≥ 3 - 0.1 en L² y ≥ 2 -
          0.1 en energía; error decreciente en cada nivel"
      tol_abs: 0.1
    - name: patch_test_lineal
      setup: "Hex20 con campo lineal u_i = a_ij·x_j impuesto en los 20 nodos
             del contorno; sin nodos interiores libres (todos están en la
             frontera de un elemento aislado, pero al menos los medios no
             están fijados en su posición referencial)"
      expect: "ε constante e igual a sym(a) en todos los puntos de Gauss; σ
              uniforme"
      tol_rel: 1.0e-12
    - name: patch_test_cuadratico
      setup: "Hex20 con campo u_x = c·x², u_y = u_z = 0 impuesto en los 20 nodos
             del contorno (todos)"
      expect: "ε_xx(x) = 2c·x exacto en todos los puntos de Gauss"
      tol_rel: 1.0e-12
    - name: modos_de_cuerpo_rigido
      setup: "K_e del Hex20 proyectado sobre los 6 modos de cuerpo rígido"
      expect: "K_e · u_rigid ≈ 0 (autovalores ≈ 0: exactamente 6)"
      tol_abs: 1.0e-9
    - name: simetria_K_e
      setup: "evaluar K_e con Elastic3D"
      expect: "K_e == K_e.T exacto"
      tol_abs: 1.0e-12
  specific:
    - name: cubo_lame_3d
      setup: "cubo unitario con un Hex20; tracción uniaxial σ_xx = p en cara ξ+;
             cara opuesta restringida en ux; restricciones mínimas para evitar
             movimiento rígido"
      expect: "u_x(L, y, z) = p·L/E; u_y = -ν·p·y/E; u_z = -ν·p·z/E exactos en
              todos los nodos"
      tol_rel: 1.0e-10
    - name: body_load_uniforme
      setup: "Hex20 regular cubo unitario con b = (0, 0, -g)"
      expect: "Σf_z = -g·V_e = -g; reparto no uniforme entre vértices y medios"
      tol_rel: 1.0e-12
```

```
    - name: face_traction_balance
      setup: "Hex20 cubo unitario con tracción (p, 0, 0) sobre cara ξ+ (face 3)"
      expect: "Σf_x = p·A_cara = p·1; vértices reciben -p/12 cada uno y medios
               4p/12 cada uno (suma = 4·-(1/12) + 4·(4/12) = 12/12 = 1, OK)"
      tol_rel: 1.0e-12
    - name: jacobiano_degenerado_abortado
      setup: "Hex20 con un nodo medio desplazado fuera del rango cuasi-recto"
      expect: "ValueError en _compute_kinematics_hex20 cuando det J ≤ tol"
    - name: convergencia_cantilever_macneal
      setup: "cantilever esbelto L/h = 30 con 1×1 sección + N elementos a lo
              largo; flecha vs Euler-Bernoulli"
      expect: "Hex20 con 1×1×6 alcanza >0.9 · u_EB (Hex8 con 1×1×6 solo ≈ 0.5);
               convergencia h monotónica al refinar"
      tol_rel: "documentar - sin tolerancia estricta"
    - name: locking_volumetrico_atenuado
      setup: "cantilever esbelto con Hex20; νϵ  {0.3, 0.4999}"
      expect: "ratio u_tipν(=0.4999) / u_tipν(=0.3) significativamente mayor
               que el del Hex8 (Hex20 < 0.6 → Hex20 > 0.8 esperado al menos)"
      tol_rel: "documentar valores; cota relativa Hex20 vs Hex8"
    - name: mass_consistent_total
      setup: "Hex20 cubo unitario con ρ = 7850, masa consistente"
      expect: "Σ M (todas las entradas) = 3·ρ·V = 3·7850·1 (conservación 3D)"
      tol_rel: 1.0e-9
    - name: mass_lumped_total_HRZ
      setup: "Hex20 cubo unitario con ρ = 7850, masa lumped HRZ"
      expect: "diag(M) suma = 3·ρ·V; matriz estrictamente diagonal; todas las
               entradas estrictamente positivas (HRZ canónico, no row-sum)"
      tol_abs: 1.0e-12

references:
  - "Bathe K.J. (2014). Finite Element Procedures. §5.3.2 (serendipity 3D)."
  - "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and
    Applications of FEA. §6.5, §13.6 (locking volumétrico Hex20)."
  - "Zienkiewicz O.C., Taylor R.L. (2005). The Finite Element Method, vol. 1,
    §8.7 (familia serendípita 3D)."
  - "ADR 0012 - Sólidos 3D: convención Voigt 6D y caras."
```

6.4.5 Implementación

- **Archivo**: solidum/elements/solid_3d/hex20.py.
- **Clase**: `Hex20` (registrada vía `@ElementRegistry.register`); subclase de `_HigherOrderSolid3D` desde **sub-fase 2** de A.ter — la entrada del `Hex27` disparó la centralización (regla de los dos casos reales). El cuerpo de `hex20.py` quedó como pura declaración de atributos: funciones de forma, cuadraturas, FACE_NODES y funciones de cara Quad8 (`_N_quad8`/`_dN_quad8` del paquete 2D).
- **Funciones núcleo**: `_N_hex20(xi, eta, zeta)`, `_dN_hex20(xi, eta, zeta)` en solidum/elements/solid_3d/_shared.py. Numpy puro (no `@njit`) por la complejidad de la rama por nodo serendipity, paritario con `_N_quad8`/`_dN_quad8` del 2D. El kinematics genérico '`_kinematics_higher_order_3d` (también en _shared.py) es reutilizado por la base con cualquier par (shape_fn, grad_fn)' 3D.

- **Tests**:
- tests/validation/test_mms_solid_3d.py (soluciones manufacturadas), tests/test_distributed_loads_3d.py (cargas variables).
- `tests/test_solid_3d_higher_order.py` — clase `TestHex20Element` (dimensiones/DOFs, patch lineal, patch cuadrático, simetría K, jacobiano degenerado abortado, body load suma, face traction suma + reparto vértice/medio, masa consistente y lumped HRZ, gauss state).
- `tests/test_rigid_body_modes.py` — `TestRBMHex20` (3 trans + 3 rot + rank-deficiency = 6).
- `tests/validation/test_cube_lame_3d.py` (extender) — `test_uniaxial_traction_hex20_exact`.
- `tests/validation/test_macneal_beam_3d.py` (extender) — convergencia Hex20 1×1 sección vs Hex8 1×1.
- `tests/test_volumetric_locking_3d.py` (extender) — `TestHex20VolumetricLocking` documentando mitigación parcial.

6.4.6 Diálogo

- **2026-05-27** · Spec inicial. Sub-fase 1 de la sub-etapa A.ter (sólidos 3D cuadráticos). Hex20 standalone — sin base interna compartida en esta sub-fase; el segundo caso (Hex27) entra en sub-fase 2 y dispara la centralización en `_HigherOrderSolid3D` (regla de los dos casos reales aplicada).
- **2026-05-27** · Implementado y validado. Patch test lineal y cuadrático exactos a precisión máquina, 6 modos rígidos exactos, cubo Lamé 3D uniaxial e hidrostático exactos. Demostración cuantitativa de la mitigación del shear locking sobre MacNeal 3D: Hex20 6×1×1 alcanza 97 % de la flecha analítica EB, frente a $< 55\,\%$ de Hex8 12×1×1. Mitigación parcial del locking volumétrico documentada: ratio u(ν=0.4999)/u(ν=0.3) ≈ 0.80 (vs < 0.6 del Hex8). Conteo de modos espurios con cuadratura reducida `hex_2x2x2`: 12 = 6 rígidos + 6 hourglass por elemento aislado (corregido desde el "4" inicial tras verificación numérica directa; coincide con Belytschko-Liu-Moran §8.4.4). Spec actualizada a `status: validated`.
- **2026-05-27** · Refactor a la base `_HigherOrderSolid3D` con la entrada del `Hex27` (sub-fase 2). El cuerpo del archivo `hex20.py` queda como declaración de atributos (`_SHAPE_FN`, `_GRAD_FN`, `_DEFAULT_QUADRATURE`, `_MASS_QUADRATURE`, `_FACE_N_FN`, `_FACE_DN_FN`, `_FACE_QUADRATURE`, `FACE_NODES`). La maquinaria de cálculo (bucle de Gauss para K, gauss state, body load, face traction, masa consistente, HRZ) ahora vive en la base — paritaria con la centralización 2D `_HigherOrderSolid2D` (Quad8/Quad9/Tri6). Los 26 tests previos pasan sin modificaciones.
- **2026-09-22** · Auditoría global: `quadrature` acepta clave del registro o tupla `(points, weights)` en todos los sólidos (`resolve_quadrature` valida la familia por medida de referencia y dimensión). tolerancia del jacobiano ahora **relativa** (`det J` $\leq$ `JACOBIAN_RTOL` · $\Pi\|$`fila_i(J)`$\|$, cota de Hadamard, adimensional): la absoluta `1e-10` sobre `det J` rechazaba mallas finas en metros sin distorsión.

- **2026-09-22** · Ensamblaje por lotes (ADR 0014): la cinemática del elemento pasa a un núcleo compilado compartido (`_kin3d_from_dN`) y se expone como `BATCH_KINEMATICS`; `compute_element_state` no cambia de formulación. El `Assembler` agrupa los elementos en familias y los evalúa en un kernel único; equivalencia con el camino por elemento verificada a 1e-14 en `K` y `F_int` sobre todos los materiales compatibles (`tests/test_batch_assembly.py`).

- **2026-09-22** · Cierre del ADR 0014 (§9): la cinemática por lotes (`BATCH_KINEMATICS`) ya **no lanza** con jacobiano degenerado —devuelve `det J` $\leq$ `0` y el ensamblador lanza el `ValueError` con el id del elemento—, porque el kernel de familia tiene ahora variante paralela (`prange`, bit a bit igual a la serie) y una excepción dentro de la región paralela se pierde; el camino por elemento sigue lanzando como siempre. Declara las funciones de forma para el post-proceso por familia (`Family.gauss_state`, mismo resultado que `compute_gauss_state`, verificado en `tests/test_batch_postprocess.py`). Formulación sin cambios.

- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022, fase 2). `compute_body_load` acepta un campo `b(x)` y `compute_face_traction` un campo `t(x, n)`, con `n` la normal exterior unitaria de la cara (una presión es `-p·n`); implementación común en `_SolidLoads3D`. La fuerza de cuerpo se integra con la regla de la masa (3×3×3, también con rigidez 2×2×2); la de cara, sobre la geometría real con las funciones de forma de la cara. Declara `REFERENCE_CELL` y `REFERENCE_NODES`. Nueva aceptación «soluciones manufacturadas»: orden 3 en L^2 y 2 en energía en el cubo distorsionado y en el cilindro hueco, también con 2×2×2. Con 2×2×2 y lados curvos la prueba de la parcela no es exacta (error de consistencia que baja como h^3.3, sin efecto en el orden); con 3×3×3 sí.

6.5 Hex27

*Elemento sólido 3D de orden cuadrático con interpolación **Lagrangiana completa** — producto tensorial de Lagrange cuadrático en las tres direcciones. Reproduce **todos** los polinomios triquadráticos `'ξ^a η^b ζ^c con a, b, c ∈ {0, 1, 2}, incluyendo los puramente triquadráticos (ξ²η²ζ² y combinaciones) que faltan en el serendípito Hex20. Sub-fase 2 de A.ter; dispara la centralización en _HigherOrderSolid3D'` (regla de los dos casos reales).*

6.5.1 Especificación física

0. Descripción general

Hexaedro tridimensional de segundo orden con **27 nodos**: 8 vértices + 12 medios de arista + 6 centros de cara + 1 centro del cuerpo. Aproximación Lagrangiana C^0 completa en coordenadas naturales $(\xi, \eta, \zeta) \in [-1, 1]^3$. Formulación isoparamétrica. Régimen de pequeños desplazamientos y deformaciones.

1-5. Cinemática, ecuación constitutiva, equilibrio

Idénticas al '`Hex20`' — la diferencia entre serendipity y Lagrange está exclusivamente en el espacio de aproximación discreto, no en la formulación variacional.

6.5.2 Formulación numérica (FEM)

6. Discretización — convención VTK_TRIQUADRATIC_HEXAHEDRON

Los 27 nodos en el cubo de referencia $[-1, 1]^3$:

Rango	**Tipo**	**Posiciones** (ξ_i, η_i, ζ_i)
0-7	vértices	idénticos al `Hex8`/`Hex20`
8-19	medios de arista	idénticos al `Hex20`
20	centro de cara	$(-1, 0, 0)$ — cara 5 $(-\xi)$
21	centro de cara	$(+1, 0, 0)$ — cara 3 $(+\xi)$
22	centro de cara	$(0, -1, 0)$ — cara 2 $(-\eta)$
23	centro de cara	$(0, +1, 0)$ — cara 4 $(+\eta)$
24	centro de cara	$(0, 0, -1)$ — cara 0 $(-\zeta)$
25	centro de cara	$(0, 0, +1)$ — cara 1 $(+\zeta)$
26	centro del cuerpo	$(0, 0, 0)$

7. Funciones de forma — producto tensorial de Lagrange cuadrático

Para cada nodo se asigna una terna de índices $(i, j, k) \in \{-1, 0, +1\}^3$ correspondiente a su posición natural. Las funciones de forma son productos tensoriales:

$$N_n(\xi, \eta, \zeta) = L_{i_n}(\xi)\, L_{j_n}(\eta)\, L_{k_n}(\zeta),$$

con los tres polinomios de Lagrange cuadrático 1D:

$$L_{-1}(x) = \tfrac{1}{2}\,x(x-1), \qquad L_0(x) = 1 - x^2, \qquad L_{+1}(x) = \tfrac{1}{2}\,x(x+1).$$

Cumplen $L_{-1}(-1) = L_0(0) = L_{+1}(+1) = 1$ y cero en los demás. Garantiza propiedad delta de Kronecker $N_m(\xi_n, \eta_n, \zeta_n) = \delta_{mn}$ y partición de la unidad $\sum_n N_n = 1$.

Espacio reproducido: el ‘`Hex27 reproduce **completamente** el espacio triquadrático` $Q_2 = \mathrm{span}\{\xi^a\eta^b\zeta^c : a,b,c \in \{0,1,2\}\}$ `--- 27 monomios, idéntico al número de nodos. La diferencia con el Hex20 serendípito está en los términos puramente triquadráticos` $\xi^2\eta^2$, $\eta^2\zeta^2$, $\xi^2\zeta^2$, $\xi^2\eta^2\zeta$, $\xi^2\eta\zeta^2$, $\xi\eta^2\zeta^2$, $\xi^2\eta^2\zeta^2$ `que Hex20`‘ no captura.

8. Matriz B

Tamaño **6×81**, ensamblada con la misma plantilla 6×3 del ‘`Hex8/Hex20`‘ por nodo. Derivadas obtenidas vía regla del producto sobre los polinomios de Lagrange:

$$\frac{\partial N_n}{\partial \xi} = L'_{i_n}(\xi)\, L_{j_n}(\eta)\, L_{k_n}(\zeta),$$

con

$$L'_{-1}(x) = x - \tfrac{1}{2}, \qquad L'_0(x) = -2x, \qquad L'_{+1}(x) = x + \tfrac{1}{2}.$$

9-10. Rigidez y fuerzas internas

Idénticas a ‘`Hex8/Hex20`‘ en estructura. El tamaño es 81×81.

11. Cuadratura

Por defecto **Gauss–Legendre 3×3×3** (27 puntos): orden de exactitud 5 en cada dirección. Suficiente para integrar exactamente $\mathbf{B}^\top\mathbf{D}\,\mathbf{B}$ del ‘`Hex27`‘ sobre Jacobiano constante (el integrando es a lo sumo de grado 3 en cada dirección — derivada cuadrático×cuadrático) y la masa consistente $\rho\mathbf{N}^\top\mathbf{N}$ (grado 4 en cada dirección).

Aviso sobre integración reducida: el esquema $2\times2\times2$ (`hex_2x2x2`, 8 puntos) está disponible pero **introduce muchos modos espurios** sobre un elemento aislado: verificación numérica directa del proyecto cuenta **27 modos de hourglass** (= 81 DOFs − 8 Gauss × 6 strain − 6 rigid; jump espectral limpio entre eigs[32] ≈ 1e-10 y eigs[33] ≈ 7e3). Es muchísimo peor que en el `Hex20` reducido (6 hourglass) — el espacio extra del Lagrangiano completo es subintegrado severamente por 2×2×2. La cita clásica de "1 modo bubble" (Cook-Malkus-Plesha-Witt §11.6) corresponde al **único modo que persiste tras ensamblar** en mallas estructuradas; en un elemento aislado el conteo real es el de arriba. Sin estabilización implementada. Recomendación operativa estricta: usar el default $3\times3\times3$ con `Hex27`.

12. Cargas distribuidas consistentes

Fuerza de cuerpo: integración con la regla $3\times3\times3$ de la masa (constante o campo $\mathbf{b}(\mathbf{x})$, ADR 0022), reparto no uniforme entre vértices, medios, centros de cara y centro del cuerpo. Suma global $\mathbf{b}\cdot V_e$ exacta para $\mathbf{b}$ uniforme.

Tracción de superficie: cada cara del ‘`Hex27`‘ tiene **9 nodos** (4 vértices + 4 medios de arista + 1 centro de cara) que se aproximan por funciones Lagrangianas Quad9 sobre el parámetro 2D $(s,t) \in [-1,1]^2$.

Cara	Vértices	Medios de arista	Centro de cara	Normal saliente
0	(0,3,2,1)	(11, 10, 9, 8)	24	$-\zeta$
1	(4,5,6,7)	(12, 13, 14, 15)	25	$+\zeta$
2	(0,1,5,4)	(8, 17, 12, 16)	22	$-\eta$
3	(1,2,6,5)	(9, 18, 13, 17)	21	$+\xi$
4	(2,3,7,6)	(10, 19, 14, 18)	23	$+\eta$
5	(3,0,4,7)	(11, 16, 15, 19)	20	$-\xi$

El orden dentro de cada cara reproduce la convención de ‘`_N_quad9 del paquete 2D: primero los 4 vértices, después los 4 medios en orden cíclico, y finalmente el nodo central. Cuadratura 2D sobre la cara: Gauss` 3×3 `(exacta para tracción constante sobre cara plana).` $\bar{\mathbf{t}}$ `se especifica en **coordenadas globales**: constante o campo` $\bar{\mathbf{t}}(\mathbf{x}, \mathbf{n})$ `de la posición y de la normal exterior unitaria de la cara, que sale de la orientación antihoraria de FACE_NODES`‘ vista desde fuera (ADR 0022); una presión p es $\bar{\mathbf{t}} = -p\,\mathbf{n}$. Se integra sobre la geometría real de la cara con sus funciones de forma.

13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve dict con $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ en cada uno de los 27 puntos de Gauss del elemento, junto con coordenadas naturales y globales. Sin `internal_forces` (ADR 0012).

6.5.3 Limitaciones declaradas

Coste computacional vs ‘`Hex20`‘

`Hex27` tiene **81 DOFs por elemento** vs 60 del `Hex20` (35 % más). El espacio extra que captura (términos puramente triquadráticos) **rara vez gobierna la solución** en problemas físicos estándar — el `Hex20` serendípito es la elección por defecto en la mayoría de aplicaciones (Bathe FEP §5.3.2). El `Hex27` se prefiere cuando:

- La geometría tiene curvatura severa y se mapea con elementos isoparamétricos cuadráticos: el nodo central de cuerpo ayuda a representar la curvatura interior.
- La sensibilidad al locking es crítica y se prefiere intentar el Q_2 completo antes que recurrir a B-bar/F-bar.
- Se busca convergencia óptima $O(h^{p+1})$ con $p = 2$ en problemas donde el contenido espectral cubre el espacio completo.

Para problemas con flexión simple y geometría rectilínea, `Hex20` y `Hex27` dan resultados prácticamente idénticos a un coste menor para el primero.

Locking volumétrico con $\nu \to 0.5$

Comparable al ‘Hex20‘: el espacio extra triquadrático añade flexibilidad sin eliminar la incompresibilidad — formulación mixta u-p o B-bar/F-bar son necesarias para eliminarlo. Declarado como limitación, blindado por test cuantitativo.

Hourglass con integración reducida

Hex27 integrado con hex_2x2x2 tiene **27 modos de hourglass espurios por elemento aislado** (vs 6 del Hex20 reducido y 12 del Hex8 reducido). El espacio Lagrangiano completo es subintegrado severamente por 8 puntos. En mallas estructuradas la mayoría no propagan tras ensamblar, pero el conteo a nivel de elemento es alto. Sin estabilización.

6.5.4 Contrato de implementación

```yaml
name: Hex27
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 27
  strain_dim: 6
  n_integration_points: 27      # default Gauss 3×3×3

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 3 (elementos lagrangianos)"
    - zienkiewicz_taylor: "vol. 1 (6.ª ed.), cap. 4: familia lagrangiana 3D"
  general_codes:
    - abaqus: "C3D27"

parameters:
  - { name: quadrature, type: str, required: false, default: "hex_3x3x3",
      desc: "Regla de cuadratura desde QuadratureRegistry; hex_2x2x2
             reducida con 1 modo de hourglass espurio (bubble) por
             elemento aislado, sin estabilización" }

material_contract:
  signature: "compute_stateε(, state) -> σ(, C_tangent, state')"
  strain_kind: "Voigt 3D ε[_xx, ε_yy, ε_zz, γ_xy, γ_yz, γ_xz] con γ_ij = 2·ε_ij"

conventions:
  sign: "σ > 0 ⇔ tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] con γ_ij = 2·ε_ij (ADR 0012)"
  node_orientation: "VTK_TRIQUADRATIC_HEXAHEDRON; 0-7 vértices (Hex8), 8-19
                     medios de arista (Hex20), 20-25 centros de cara en orden
                     (-x, +x, -y, +y, -z, +z), 26 centro del cuerpo"
  face_numbering: "ADR 0012 - 6 caras con normal saliente, paritarias con
                   Hex8/Hex20 en los vértices y medios; cada cara añade el
                   nodo de centro correspondiente (índice 20-25)"

validity:
  - "pequeños desplazamientos y deformaciones"
```

```
  - "geometría sin distorsión severa (det J > tol en todos los puntos de Gauss)"
  - "compatible con materiales con STRAIN_DIM = 6 (Elastic3D, VonMises3D,
     DruckerPrager3D, IsotropicDamage3D)"

out_of_scope:
  - "grandes deformaciones, formulaciones corotacionales"
  - "estabilización contra hourglass para integración reducida"
  - "tracción variable sobre cara"
  - "internal_forces (ADR 0012)"
  - "Tet10 cuadrático (sub-fase 3)"

acceptance:
  verification:
    - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
      setup: "u* suave no polinómico; cubo distorsionado (caras planas con nodos
          deslizados, elementos interiores curvos) y sector de cilindro hueco con
          tracción sobre las caras cilíndricas; Elastic3D; mallas n×n×n, n = 2, 4, 8"
      expect: "orden observado entre los dos últimos niveles ≥ 3 - 0.1 en L² y ≥ 2 -
          0.1 en energía; error decreciente en cada nivel"
      tol_abs: 0.1
    - name: patch_test_lineal
      setup: "campo lineal u_i = a_ij·x_j en los 27 nodos del contorno"
      expect: "ε constante = sym(a) en todos los Gauss; σ uniforme"
      tol_rel: 1.0e-12
    - name: patch_test_triquadratico
      setup: "campo u_x = c·x²·y²·z² impuesto en los 27 nodos"
      expect: "ε_xx(x,y,z) = 2c·x·y²·z² exacto en todos los Gauss (el espacio
              Q_2 lagrangiano completo lo contiene; Hex20 serendípito NO)"
      tol_rel: 1.0e-12
    - name: modos_de_cuerpo_rigido
      setup: "evaluar K_e y proyectar sobre 6 modos rígidos 3D"
      expect: "K_e · u_rigid ≈ 0; exactamente 6 autovalores nulos"
      tol_abs: 1.0e-9
    - name: simetria_K_e
      setup: "K_e con Elastic3D"
      expect: "K_e == K_e.T exacto"
      tol_abs: 1.0e-12
  specific:
    - name: cubo_lame_3d
      setup: "1 Hex27 en cubo unitario con tracción uniaxial y BCs mínimas"
      expect: "u_x = p·x/E etc. exacto a precisión máquina (campo lineal)"
      tol_rel: 1.0e-10
    - name: body_load_uniforme
      setup: "Hex27 cubo unitario con b = (0, 0, -g)"
      expect: "Σf_z = -g·V_e = -g"
      tol_rel: 1.0e-12
    - name: face_traction_balance
      setup: "tracción (p, 0, 0) sobre cara ξ+ del cubo unitario"
      expect: "Σf_x = p·A_cara = p; reparto consistente Lagrange Q_2 (vértices,
              medios y centro de cara con sus pesos respectivos)"
      tol_rel: 1.0e-12
    - name: reduced_integration_introduces_hourglass
      setup: "Hex27 con cuadratura reducida hex_2x2x2"
      expect: "81 - 8·6 = 33 modos cero totales (6 rígidos + 27 hourglass).
              Verificación con jump espectral limpio entre eigs[32] ≈ 1e-10
              y eigs[33] ≈ O(10^3) (escala del módulo de cortante × volumen)"
```

```
    - name: mass_consistent_total
      setup: "Hex27 cubo unitario con ρ=7850, masa consistente"
      expect: "Σ M = 3·ρ·V = 23550"
      tol_rel: 1.0e-9
    - name: mass_lumped_total_HRZ
      setup: "Hex27 cubo unitario con ρ=7850, masa lumped HRZ"
      expect: "diag(M) suma = 3·ρ·V; matriz diagonal; todas las entradas
               positivas (HRZ canónico)"
      tol_abs: 1.0e-12

references:
  - "Bathe K.J. (2014). Finite Element Procedures. §5.3.2 (Lagrangiano 3D)."
  - "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and
    Applications of FEA. §6.6, §11.6 (Q2/E27 con 2×2×2 reducida)."
  - "Zienkiewicz O.C., Taylor R.L. (2005). The Finite Element Method, vol. 1,
    §8.6 (familia Lagrangiana 3D)."
  - "ADR 0012 - Sólidos 3D: convención Voigt 6D y caras."
```

6.5.5 Implementación

- **Archivo**: solidum/elements/solid_3d/hex27.py.
- **Clase**: `Hex27` (registrada vía `@ElementRegistry.register`); subclase de `_HigherOrderSolid3D`.
- **Funciones núcleo**: `_N_hex27(xi, eta, zeta)`, `_dN_hex27(xi, eta, zeta)` en solidum/elements/solid_3d/_shared.py. Implementadas en numpy puro vía productos tensoriales de los polinomios Lagrange 1D `_L_lagrange_quad` y `_dL_lagrange_quad`.
- **Centralización (sub-fase 2)**: `_HigherOrderSolid3D` abstrae el bucle de Gauss para `K`, `F_int`, `gauss_state`, `body_load`, `face_traction` y masa consistente. Comparte con `Hex20` la misma kinematics y la misma maquinaria; sólo cambian las funciones de forma 3D, la cuadratura, las funciones de forma 2D de cara y los `FACE_NODES`.
- **Tests**:
- tests/validation/test_mms_solid_3d.py (soluciones manufacturadas), tests/test_distributed_loads_3d.py (cargas variables).
- `tests/test_solid_3d_higher_order.py` — `TestHex27Element` (patch lineal, patch **triquadrático**, simetría K, jacobiano degenerado, body load, face traction balance, masa consistente y lumped HRZ, reduced integration 1 hourglass).
- `tests/test_rigid_body_modes.py` — `TestRBMHex27` (3 trans + 3 rot + rank-deficiency = 6).
- `tests/validation/test_cube_lame_3d.py` — `test_uniaxial_traction_hex27_exact`, `test_hydrostatic_compression_hex27`.
- `tests/validation/test_macneal_beam_3d.py` — `test_macneal_3d_hex27_*` (convergencia espejo del Hex20).
- `tests/test_volumetric_locking_3d.py` — `TestHex27VolumetricLocking` (mitigación comparable al Hex20).

6.5.6 Diálogo

- **2026-05-27** · Spec inicial. Sub-fase 2 de A.ter — el segundo caso real (Hex27) dispara la centralización en `_HigherOrderSolid3D` (regla de los dos casos reales aplicada en el momento canónico, igual que el `_HigherOrderSolid2D` cuando entró el `Quad9` después del `Quad8`).

- **2026-05-27** · Implementado y validado. Patch test lineal, bilineal mixto (`u_x = c·xy`) y triquadrático completo (`u_x = c·x²y²z²`) exactos a precisión máquina — esta última es la capacidad distintiva del Hex27 respecto al Hex20 serendípito. K simétrica, 6 modos rígidos, masas consistente y lumped HRZ con todas las entradas positivas, balance de cargas distribuidas correcto. Spec a `status: validated`.

- **2026-09-22** · Auditoría global: `quadrature` acepta clave del registro o tupla `(points, weights)` en todos los sólidos (`resolve_quadrature` valida la familia por medida de referencia y dimensión). tolerancia del jacobiano ahora **relativa** (`det J` $\leq$ `JACOBIAN_RTOL` · $\Pi\|$`fila_i(J)`$\|$, cota de Hadamard, adimensional): la absoluta `1e-10` sobre `det J` rechazaba mallas finas en metros sin distorsión.

- **2026-09-22** · Ensamblaje por lotes (ADR 0014): la cinemática del elemento pasa a un núcleo compilado compartido (`_kin3d_from_dN`) y se expone como `BATCH_KINEMATICS`; `compute_element_state` no cambia de formulación. El `Assembler` agrupa los elementos en familias y los evalúa en un kernel único; equivalencia con el camino por elemento verificada a 1e-14 en `K` y `F_int` sobre todos los materiales compatibles (`tests/test_batch_assembly.py`).

- **2026-09-22** · Cierre del ADR 0014 (§9): la cinemática por lotes (`BATCH_KINEMATICS`) ya **no lanza** con jacobiano degenerado —devuelve `det J` $\leq$ `0` y el ensamblador lanza el `ValueError` con el id del elemento—, porque el kernel de familia tiene ahora variante paralela (`prange`, bit a bit igual a la serie) y una excepción dentro de la región paralela se pierde; el camino por elemento sigue lanzando como siempre. Declara las funciones de forma para el post-proceso por familia (`Family.gauss_state`, mismo resultado que `compute_gauss_state`, verificado en `tests/test_batch_postprocess.py`). Formulación sin cambios.

- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022, fase 2). `compute_body_load` acepta un campo `b(x)` y `compute_face_traction` un campo `t(x, n)`, con `n` la normal exterior unitaria de la cara (una presión es `-p·n`); implementación común en `_SolidLoads3D`. La fuerza de cuerpo se integra con la regla de la masa (3×3×3); la de cara, sobre la geometría real con las funciones de forma de la cara. Declara `REFERENCE_CELL` y `REFERENCE_NODES`. Nueva aceptación «soluciones manufacturadas»: orden 3 en L^2 y 2 en energía en el cubo distorsionado y en el cilindro hueco.

Capítulo 7

Elementos Térmicos — 2D

7.1 Quad4Thermal

Spec pre-redactada por la IA (2026-08-25). ***El usuario revisa la física; el plumbing está resuelto.*** *Los puntos abiertos de §Diálogo son físicos o de alcance, nunca arquitecturales.*

Segundo componente de la Etapa 8 — C1 núcleo mínimo. Primer elemento de la familia térmica. Consume `ThermalConduction`*.*

7.1.1 Especificación física

0. Descripción general

Cuadrilátero isoparamétrico de 4 nodos que resuelve **conducción de calor en 2D**. Misma geometría, mismas funciones de forma bilineales y misma cuadratura que el `Quad4` mecánico; lo que cambia es el campo que interpola y la ecuación que discretiza.

Un grado de libertad por nodo: la temperatura T. `DOF_NAMES = ['T']`.

Es una **clase distinta** del `Quad4` mecánico, no un modo de operación suyo. `DOF_NAMES` es atributo de clase y el registro, la validación temprana y el cruce spec↔código dependen de poder leerlo sin construir el objeto; un elemento que decidiera sus DOFs según el material recibido rompería ese contrato declarativo. Es también la separación que adoptan los códigos de referencia (Abaqus `CPE4` vs `DC2D4`; ANSYS `PLANE182` vs `PLANE55`).

1. Interpolación del campo

$$T(\xi,\eta) = \sum_{i=0}^{3} N_i(\xi,\eta)\, T_i, \qquad N_i = \tfrac{1}{4}(1+\xi_i\xi)(1+\eta_i\eta)$$

Las N_i son **las mismas** del `Quad4` mecánico. Isoparamétrico: geometría y temperatura comparten interpolación.

2. Gradiente de temperatura

$$\nabla T = \mathbf{B}\,\mathbf{T}_e, \qquad \mathbf{B} = \begin{bmatrix} \partial N_0/\partial x & \cdots & \partial N_3/\partial x \\ \partial N_0/\partial y & \cdots & \partial N_3/\partial y \end{bmatrix} \in \mathbb{R}^{2\times 4}$$

Diferencia esencial con el mecánico: aquí $\mathbf{B}$ es literalmente el gradiente de las funciones de forma, sin reordenamiento. En el `Quad4` mecánico las mismas derivadas $\partial N_i/\partial x_j$ se reordenan en una matriz 3×8 que produce la deformación en notación Voigt. La materia prima es idéntica —el kernel `_compute_kinematics` ya la calcula—; el ensamblaje difiere.

Consecuencia: el gradiente ∇T es **bilineal por elemento** y su valor en cada punto de Gauss no es constante, a diferencia del CST.

3. Ecuación discretizada — forma débil

Partiendo del balance $\rho c\,\dot{T} = \nabla\cdot(\mathbf{k}\nabla T) + Q$, multiplicando por una función de peso w e integrando por partes:

$$\int_\Omega \rho c\, w\, \dot{T}\, d\Omega + \int_\Omega (\nabla w)^\top \mathbf{k} \nabla T\, d\Omega = \int_\Omega w\, Q\, d\Omega - \int_{\partial\Omega} w\, \bar{q}\, d\Gamma$$

que en forma matricial es el sistema semidiscreto

$$\mathbf{C}\,\dot{\mathbf{T}} + \mathbf{K}\,\mathbf{T} = \mathbf{F}$$

de primer orden en el tiempo — a diferencia del mecánico $\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{F}$, que es de segundo. Por eso Newmark no aplica y el transitorio requiere solver propio (spec separada).

En **régimen estacionario** el término $\mathbf{C}\dot{\mathbf{T}}$ se anula y queda $\mathbf{K}\mathbf{T} = \mathbf{F}$, que resuelve el `LinearSolver` existente sin modificación.

4. Matriz de conductividad

$$\mathbf{K}_e = \int_\Omega \mathbf{B}^\top\, \mathbf{k}\, \mathbf{B}\; t\; d\Omega = \sum_g \mathbf{B}_g^\top\, \mathbf{k}\, \mathbf{B}_g\; t\; |\mathbf{J}_g|\; w_g$$

4×4, **simétrica** (por serlo $\mathbf{k}$) y **semidefinida positiva**: tiene un modo nulo, el campo de temperatura uniforme $\mathbf{T} = c\,[1,1,1,1]^\top$, que no produce flujo. Es el análogo térmico exacto de los modos de sólido rígido en mecánica, y como allí, el sistema estacionario global sólo se vuelve resoluble al imponer al menos una condición de Dirichlet (en el transitorio la capacidad lo hace resoluble, §10).

El **espesor** t interviene igual que en el mecánico: el problema plano representa una rebanada de profundidad t, y el flujo total escala con ella.

5. Matriz de capacidad

$$\mathbf{C}_e = \int_\Omega \rho c\, \mathbf{N}^\top \mathbf{N}\; t\; d\Omega$$

4×4, simétrica y **definida positiva**. Estructuralmente idéntica a la matriz de masa consistente con ρc en lugar de ρ; de hecho $\mathbf{C}_e = \rho c\, \mathbf{M}_e^{escalar}/\rho$ sobre el mismo integrando.

Por decisión de alcance, el default es `lumped`, al revés que en dinámica estructural. La razón es física, no de conveniencia:

- La capacidad **consistente** produce **oscilaciones espurias** en los primeros pasos cuando existe un frente térmico abrupto (un salto de temperatura impuesto en la frontera). El campo calculado oscila alrededor de la solución y puede dar temperaturas **fuera del rango de los datos** —por debajo del mínimo o por encima del máximo impuestos—, lo que viola el **principio del máximo** de la ecuación de difusión: una propiedad matemática de la solución exacta, no un artefacto tolerable.
- La capacidad **lumped** amortigua esas oscilaciones y preserva la monotonía, a cambio de algo de precisión en régimen suave.

Ambas quedan disponibles con el mismo parámetro `lumping` del resto del catálogo. La coherencia arquitectural se preserva en el **mecanismo**; el default lo fija el criterio físico del dominio (Lewis-Nithiarasu-Seetharamu §6; Zienkiewicz-Taylor Vol.1 §7).

6. Condiciones de frontera y cargas

Dirichlet — temperatura impuesta. $T = \bar{T}$ en Γ_T. Se impone por eliminación directa, con la maquinaria existente del ADR 0004, sin cambios: el mecanismo es agnóstico al significado del DOF.

Neumann — flujo impuesto. $\bar{q}$ [W/m^2] normal a un borde: constante o campo $\bar{q}(\mathbf{x}, \mathbf{n})$ de la posición y de la normal exterior unitaria del borde (ADR 0022); un flujo de Fourier conocido es $\bar{q} = \mathbf{q} \cdot \mathbf{n}$. El vector consistente sobre el borde Γ_q, integrado con Gauss de 3 puntos sobre el borde:

$$\mathbf{f}_e^{\bar{q}} = -\int_{\Gamma_q} \bar{q}\, \mathbf{N}^\top\, t\, d\Gamma$$

Para $\bar{q}$ uniforme sobre un borde recto de longitud L, la integral da $\bar{q}\, L\, t/2$ a cada uno de los dos nodos del borde, y cero a los otros dos — el mismo reparto mitad-mitad que `compute_edge_traction` produce en el mecánico, con la misma numeración de bordes `EDGE_NODES = ((0,1),(1,2),(2,3),(3,0))`.

Convención de signo: $\bar{q} > 0$ significa flujo **saliente** del dominio (enfriamiento), coherente con que la normal $\mathbf{n}$ es saliente y con la deducción de la forma débil, donde el término de frontera aparece con signo negativo. Un flujo entrante se declara con $\bar{q} < 0$.

Un borde sin condición declarada queda con $\bar{q} = 0$ — **frontera adiabática**, aislada. Es el default natural de Neumann homogéneo, exactamente como un borde mecánico sin tracción queda libre.

Fuente volumétrica. Q [W/m^3], constante o campo $Q(\mathbf{x})$, se aplica como carga del elemento (decisión de alcance; ver §Diálogo de `ThermalConduction`). Se integra con la regla de la capacidad (2×2), también con conductividad 1×1 (ADR 0022):

$$\mathbf{f}_e^{Q} = \int_{\Omega} Q\, \mathbf{N}^\top\, t\, d\Omega$$

Para Q uniforme la suma de las componentes nodales es exactamente $Q\, A_e\, t$, invariante ante distorsión de la geometría — el mismo criterio de verificación que se usa en `compute_body_load` del mecánico.

7. Salida por punto de Gauss

`compute_gauss_state(T_global)` devuelve, en paralelo al contrato de los sólidos (ADR 0012):

- `points_natural`, `points_global` — posición de los puntos de Gauss.
- `grad_T` — gradiente $(n_g, 2)$.
- `flux` — vector de flujo $\mathbf{q} = -\mathbf{k}\nabla T$, $(n_g, 2)$.

No expone `internal_forces`: por el cierre por dominio del ADR 0012, ese método corresponde a elementos estructurales 1D. La familia térmica sigue el mismo criterio que los sólidos.

7.1.2 Formulación numérica

8. Cuadratura

Gauss 2×2 (4 puntos) por defecto, la misma del `Quad4` mecánico, mediante `QuadratureRegistry`. Es **exacta** para $\mathbf{K}_e$ y $\mathbf{C}_e$ con jacobiano constante (elemento paralelogramo).

La reducción 1×1 queda disponible por el mismo parámetro, pero **no se recomienda**: al igual que en el mecánico introduce modos espurios. En el caso térmico el rango de $\mathbf{K}_e$ con un punto es 2, frente a los 3 necesarios (4 DOFs menos el modo de temperatura uniforme), de modo que aparece **un modo hourglass** de temperatura no detectado por la matriz. Se documenta y se blinda con test de conteo de modos nulos.

9. Reutilización de kernels

`_shape_functions_quad4`, `_det_jacobian_quad4` y las derivadas de `_compute_kinematics` ya existen compiladas con Numba en `solidum/elements/solid_2d/_shared.py` y se reutilizan sin duplicar. Lo específico térmico es el ensamblaje $\mathbf{B}^\top \mathbf{k}\, \mathbf{B}$, que es más simple que el mecánico.

10. Caveats numéricos

- **Modo nulo de temperatura uniforme**: $\mathbf{K}$ global es singular sin Dirichlet. El **estacionario** debe tener al menos un nodo con temperatura impuesta; sin él, el `LinearSolver` lo rechaza nombrando el modo, como un modelo mecánico sin apoyos. El **transitorio** admite el cuerpo aislado (desde el 2026-10-20): $\mathbf{C} + \theta \Delta t\, \mathbf{K}$ es definida positiva por la capacidad (spec de `ThetaMethodSolver` §10).
- **Sin locking**: el problema escalar no tiene el análogo de la incompresibilidad ni del bloqueo por cortante. No aplica ninguna de las limitaciones que arrastra el `Quad4` mecánico.
- **Número de Péclet**: sin término convectivo en la ecuación (fuera de alcance C1), no hay riesgo de oscilaciones por advección dominante. Se anota porque será la primera consideración a revisar si en el futuro entra convección forzada del medio.
- **Distorsión de la malla**: como en el mecánico, un jacobiano casi degenerado degrada la solución; se valida $|\mathbf{J}| > 0$ en todos los puntos de Gauss.

7.1.3 Contrato de implementación

```
name: Quad4Thermal
kind: element
status: validated        # draft → implemented → validated

interface:
  field: temperature            # campo primario (Etapa 8) - no displacement
  dof_names: [T]
  n_nodes: 4
  flux_dim: 2                   # dimensión de ∇T y de q en 2D (no hay strain_dim)
  n_integration_points: 4       # default Gauss 2×2

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 2-3 (conducción de calor con elementos isoparamétricos)"
    - zienkiewicz_taylor: "vol. 1: problemas de campo (conducción de calor)"
  general_codes:
    - abaqus: "DC2D4"
    - ansys: "PLANE55"

parameters:
  - { name: thickness,  type: float, required: false, default: 1.0,
      desc: "Espesor de la rebanada plana [m]; el flujo total escala con él" }
  - { name: quadrature, type: str,   required: false, default: "2x2",
      desc: "Regla desde QuadratureRegistry. 1x1 disponible pero introduce
             un modo hourglass de temperatura - sin estabilización" }

material_contract:
  signature: "compute_flux∇(T) -> (q, k)"
  gradient_kind: "gradiente físico ∂[T∂/x, ∂T∂/y] - sin notación Voigt"
  accepted: [ThermalConduction]

conventions:
  sign_flux: "q > 0 ⇔ flujo SALIENTE del dominio (enfriamiento); normal exterior"
  adiabatic_default: "borde sin condición declarada ⇒ q = 0 (aislado)"
  node_orientation: "antihorario (asegura det J > 0) - idéntico al Quad4 mecánico"
  edge_numbering: "EDGE_NODES = ((0,1),(1,2),(2,3),(3,0)) - paritario con Quad4"
  capacity_default: "lumping='lumped' por default, al contrario que en dinámica
                     estructural; razón física en §5"

validity:
  - "conducción lineal; k independiente de T"
  - "material con field='temperature' (familia ThermalMaterial)"
  - "det J > 0 en todos los puntos de Gauss"
  - "el estacionario debe imponer al menos un Dirichlet de temperatura (el
     transitorio admite el cuerpo aislado)"

out_of_scope:
  - "convección (Robin) y radiación en la frontera - Etapa 8 posterior"
  - "acoplamiento termomecánico y deformación térmica"
  - "conducción con advección (término convectivo del medio)"
  - "cambio de fase / calor latente"
  - "estabilización del modo hourglass con cuadratura reducida"

acceptance:
```

```
verification:
  - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
    setup: "T* suave no polinómica; k anisótropa de tensor lleno; cuadrado
        distorsionado (lados rectos con nodos deslizados, elementos interiores curvos
        ) y cuarto de corona con flujo ¯q*(x, n) sobre los arcos; estacionario con n =
         4, 8, 16; transitorio Crank-Nicolson con Δt = h/2, capacidad concentrada y
        consistente; en el tiempo, T* lineal en x sobre malla 4×4 con θ = 1, 2/3, 1/2
         y 0"
    expect: "orden observado entre los dos últimos niveles ≥ 2 - 0.1 en L² y ≥ 1 -
        0.1 en energía ‖(‖e_E² = ∫∇e·k·∇e); en el tiempo ≥ orden del esquema - 0.1 (2
        con θ = 1/2, 1 con los demás); error decreciente en cada nivel; con T* lineal
         en x y en t, error de redondeo para todo θ"
    tol_abs: 0.1
  - name: patch_test_termico_lineal
    setup: "5 Quad4Thermal distorsionados (4 exteriores + 1 interior);
           imponer T = a + b·x + c·y en los nodos del contorno"
    expect: "los nodos interiores adoptan el campo lineal exacto y ∇T =
            (b, c) constante en TODOS los puntos de Gauss; q = -k·(b,c)"
    tol_rel: 1.0e-10
  - name: modo_nulo_temperatura_uniforme
    setup: "K_e de un elemento aislado; campo T = [1,1,1,1]"
    expect: "K_e·T = 0 exacto; rango(K_e) = 3; exactamente un autovalor nulo"
    tol_abs: 1.0e-12
  - name: simetria_y_semidefinicion
    setup: "K_e y C_e con k isótropo y ortótropo, elemento regular y distorsionado"
    expect: "K_e == K_e.T y C_e == C_e.T exactas; autovalores de K_e ≥ 0 con un
             único cero; autovalores de C_e > 0 estrictos"
    tol_abs: 1.0e-12
specific:
  - name: conduccion_1d_pared_plana
    setup: "franja de Quad4Thermal entre x=0 y x=L; T(0)=T_1, T(L)=T_2;
           bordes superior e inferior adiabáticos; estacionario"
    expect: "perfil lineal exacto T(x) = T_1 + (T_2-T_1)·x/L en todos los nodos;
            flujo uniforme q_x = k·(T_1-T_2)/L en todos los Gauss"
    tol_rel: 1.0e-12
  - name: cilindro_hueco_logaritmico
    setup: "corona circular r ∈ [a, b] mallada con Quad4Thermal;
           T(a)=T_a, T(b)=T_b; estacionario"
    expect: "perfil logarítmico analítico T(r) = T_a + (T_b-T_a)·ln(r/a)/ln(b/a);
            convergencia con refinamiento h. Es el análogo térmico exacto del
            benchmark de Lamé que el proyecto ya usa en mecánica"
    tol_rel: 5.0e-3
  - name: fuente_volumetrica_uniforme
    setup: "Quad4Thermal regular y distorsionado con Q uniforme; sumar
           componentes nodales de f^Q"
    expect: "Σf = Q·A_e·t exacto, invariante ante distorsión de la geometría"
    tol_rel: 1.0e-12
  - name: flujo_neumann_en_un_borde
    setup: "Quad4Thermal con ¯q constante sobre un borde"
    expect: "Σf = ¯q·L·t con reparto L/2 a cada nodo del borde y 0 en los demás"
    tol_rel: 1.0e-12
  - name: balance_energetico_estacionario
    setup: "dominio con fuente Q y Dirichlet en parte de la frontera; estacionario"
    expect: "el calor generado iguala al evacuado por las reacciones nodales:Σ
             Q·A·t + Σ reacciones = 0"
    tol_rel: 1.0e-10
```

```
    - name: capacidad_lumped_vs_consistente
      setup: "C_e en ambos modos para elemento regular"
      expect: "ambas suman ρ·c·A_e·t (capacidad total conservada); la lumped es
              diagonal y estrictamente positiva"
      tol_rel: 1.0e-12
    - name: anisotropia_desvia_el_flujo
      setup: "k = diag(k1, k2) con k1 ≠ k2; campo lineal a 45° de los ejes"
      expect: "q no es antiparalelo a ∇T; coincide con -k·∇T calculado a mano"
      tol_rel: 1.0e-12
    - name: jacobiano_degenerado_abortado
      setup: "elemento con nodos colineales o en orden inverso"
      expect: "ValueError con mensaje claro"
    - name: rechazo_material_mecanico
      setup: "construir Quad4Thermal con un material de la familia mecánica (Elastic2D)
          "
      expect: "TypeError/ValueError identificando la incompatibilidad de familia;
               nunca aceptación silenciosa"

references:
  - "Lewis R.W., Nithiarasu P., Seetharamu K.N. (2004). Fundamentals of the FEM for
     Heat and Fluid Flow. Wiley. §3 y §6 (capacidad lumped vs consistente,
     oscilaciones)."
  - "Zienkiewicz O.C., Taylor R.L. (2000). The Finite Element Method, Vol. 1. §7 (
     problemas de campo escalar)."
  - "Reddy J.N., Gartling D.K. (2010). The Finite Element Method in Heat Transfer and
     Fluid Dynamics. CRC Press. §4."
  - "Carslaw H.S., Jaeger J.C. (1959). Conduction of Heat in Solids. Oxford. §7.2 (
     cilindro hueco, solución logarítmica)."
  - "Bathe K.-J. (2014). Finite Element Procedures. §7 (formulación de campo escalar;
     analogía con elasticidad)."
```

7.1.4 Implementación

- Archivo: `solidum/elements/thermal/quad4_thermal.py`
- Clase: `Quad4Thermal`, registrada en `ElementRegistry`
- Base compartida: `solidum/elements/thermal/_shared.py` · `_ThermalSolid`
- Tests: `tests/test_quad4_thermal.py` — 31 verdes; `tests/test_thermal_loads_fields.py` (fuente y flujo variables); `tests/validation/test_mms_thermal.py` (soluciones manufacturadas); `tests/validation/test_thermal_hollow_cylinder.py` (cilindro hueco)

Notas de implementación:

- **Base `_ThermalSolid` introducida con el primer elemento**, no con el segundo. No contradice la regla de los dos casos reales: el segundo caso —`Hex8Thermal`, de esta misma etapa— está especificado y difiere **únicamente en geometría**. La ecuación discretizada $\mathbf{C}\dot{\mathbf{T}} + \mathbf{K}\mathbf{T} = \mathbf{F}$, la validación del material, la capacidad, la fuente y el post-proceso son idénticos en 2D y 3D; la dimensión sólo cambia el tamaño de $\mathbf{B}$. La subclase concreta aporta funciones de forma, gradiente, cuadratura por defecto y topología de frontera.

- `STRAIN_DIM = None` **y validación propia.** `Element._validate_material_compatibility` compara `STRAIN_DIM` del elemento contra el del material; un elemento térmico no tiene deformación. Se sobreescribe el método para comprobar **familia** (`isinstance(mat, ThermalMaterial)`, con mensaje que explica la diferencia de contrato) y **dimensión** (`FLUX_DIM`). Un material mecánico pasado por error se detecta al construir, no más tarde con un `AttributeError` sobre `compute_flux`.

- `_init_state` **devuelve** `None`**.** La conducción de Fourier no tiene variables internas, así que no se crea `ElementState`: no hay historia que promover trial → committed. Cuando entre un material térmico con memoria, se reimplementa con el caso real delante.

- **Kernel** `_compute_gradient_kinematics_quad4` **añadido a** `solid_2d/_shared.py`, no a la familia térmica. `_compute_kinematics` ya calculaba `dN_dx` pero sólo devolvía la `B` mecánica ensamblada en Voigt; el nuevo kernel expone el gradiente crudo, que **es** la `B` térmica. Vive junto a su gemelo mecánico porque comparte el jacobiano y su inversión: separarlos habría duplicado esa aritmética.

- `lump_hrz` **con** `total_mass = ` $\rho c \cdot$`V_e`**.** El campo es escalar (un DOF por nodo), así que `n_translational_dirs=1`. El esquema escala la diagonal para conservar la capacidad total, que es la propiedad que importa: la energía almacenable no debe depender de cómo se reparta entre nodos. Verificado en test para ambos modos.

- `compute_edge_flux` **devuelve valores negativos para** $\bar{q} > 0$**.** El signo sale de la forma débil, donde el término de frontera aparece como $-\int w\,\bar{q}\,d\Gamma$: un flujo saliente positivo **extrae** energía. Blindado con test explícito de signo en ambos sentidos.

- **Aliases** `compute_global_stiffness` **y** `compute_mass_matrix` delegan en conductividad y capacidad, de modo que el elemento encaja en el ensamblador y el pipeline existentes sin ramas especiales.

Validación end-to-end confirmada: el problema de pared plana resuelto con `Assembler` + `LinearSolver` reproduce el perfil lineal analítico con error máximo 1.1×10^{-13}, sin ninguna modificación del ensamblador ni de la imposición de Dirichlet (ADR 0004). Confirma que la infraestructura era genuinamente agnóstica al tipo de campo.

Benchmark del cilindro hueco: hecho el 2026-10-27. El caso `cilindro_hueco_logaritmico` esperaba un mallador de coronas, que ahora da `solidum.mesh.structured_mesh` (ADR 0022). Error nodal relativo a $|T_b - T_a|$: 4.4×10^{-4}, 1.2×10^{-4} y 3.0×10^{-5} con 4, 8 y 16 elementos por dirección (orden 2), frente a la tolerancia 5×10^{-3}. Sigue pendiente el `balance_energetico_estacionario`. Los tests entregados cubren el patch test, el modo nulo, las cargas, la capacidad en ambos modos y la pared plana end-to-end. El estado `validated` refleja que la formulación está verificada contra solución analítica; los dos casos restantes añaden cobertura sobre geometría curva y conservación global, no verifican la formulación básica.

7.1.5 Diálogo

Puntos abiertos para el usuario. Sólo física y alcance.
2026-08-25 — IA → usuario. Dos puntos:

1. ~~**Convención de signo del flujo de frontera.**~~ ✓ **Resuelto 2026-08-25** — $\bar{q} > 0$ **es flujo SALIENTE del dominio** (decisión del usuario).

Es la que resulta de la deducción de la forma débil con normal exterior **n**, donde el término de frontera aparece con signo negativo, y coincide con Abaqus y con la mayoría de la literatura de MEF. Un flujo entrante (calentamiento) se declara con $\bar{q} < 0$.

Consecuencia: es convención del proyecto y debe registrarse en `Reglas.md §5` junto a las de signos mecánicas, al cerrar la etapa. Coherente además con `q = -k·∇T` del material: ambos signos salen de la misma deducción, no son elecciones independientes.

1. ~~**Espesor `thickness`.**~~ ✓ **Resuelto 2026-08-25** — **se mantiene, con `default = 1.0`** (decisión del usuario).

Misma semántica que en el `Quad4` mecánico: el problema plano representa una rebanada de profundidad t y el flujo total escala con ella. Tres razones:

- **Consistencia** con los cinco sólidos planos del catálogo, que ya lo declaran.
- **Balances energéticos en unidades absolutas**: el test `balance_energetico_estacionario` da watts reales, no "watts por metro de profundidad".
- **Extensión futura**: si entra el acoplamiento termomecánico, el elemento térmico y el mecánico deben compartir profundidad; tenerlo desde el principio evita una asimetría posterior.

El coste es nulo: quien no lo declare obtiene exactamente el comportamiento "por unidad de profundidad" que asume la literatura térmica. Omitirlo habría sido particularizar al caso de hoy.

- **2026-09-22** · Auditoría global: `quadrature` acepta clave del registro o tupla `(points, weights)` en todos los sólidos (`resolve_quadrature` valida la familia por medida de referencia y dimensión). La capacidad se integra siempre con `CAPACITY_QUADRATURE` (regla completa) aunque K use integración reducida (con un punto C_e era de rango 1). tolerancia del jacobiano ahora **relativa** (`det J ≤ JACOBIAN_RTOL · Π‖fila_i(J)‖`, cota de Hadamard, adimensional): la absoluta `1e-10` sobre `det J` rechazaba mallas finas en metros sin distorsión.
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): el gradiente ∂`N/`∂`x` pasa a un núcleo compilado compartido con el elemento mecánico gemelo y se expone como `BATCH_KINEMATICS`; la familia térmica se evalúa en el mismo kernel de familia con `k·∇T` como "esfuerzo", de modo que $\int$`B`$^\top$`(k·∇T) dΩ = K_e·T_e`. Sin `ElementState` (no hay historia).
- **2026-09-22** · Cierre del ADR 0014 (§9): la cinemática por lotes ya no lanza con jacobiano degenerado (devuelve `det J ≤ 0`; el ensamblador lanza con el id), el kernel de familia tiene variante paralela bit a bit igual a la serie, y el post-proceso por familia devuelve `grad_T` y `flux = -k·∇T` iguales a `compute_gauss_state` (`tests/test_batch_postprocess.py`). Formulación sin cambios.
- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022, fase 3). `compute_body_source` acepta un campo `Q(x)` y `compute_edge_flux` un campo $\bar{\mathtt{q}}$`(x, n)`, con `n` la normal exterior unitaria del borde; el flujo se integra con Gauss de 3 puntos sobre el

borde y la fuente con la regla de la capacidad (2×2, también con conductividad 1×1). Con flujo y fuente constantes el resultado no cambia salvo, con 1×1, la fuente, que deja de concentrarse en el centro. Declara `REFERENCE_CELL` y `REFERENCE_NODES`. Nueva aceptación «soluciones manufacturadas» (estacionario, transitorio en el espacio y en el tiempo, con conductividad anisótropa) y hecho el benchmark del cilindro hueco.

Capítulo 8

Elementos Térmicos — 3D

8.1 Hex8Thermal

Spec pre-redactada por la IA (2026-08-25). ***Spec de extensión*** *(`Reglas.md` §4): documenta* ***sólo lo que cambia*** *respecto a `Quad4Thermal`, que es el hermano 2D y donde vive la formulación completa de la familia térmica. Sin ADR propio: reusa las decisiones de alcance de la Etapa 8.*

Tercer componente de la Etapa 8 — C1 núcleo mínimo. Consume `ThermalConduction`.

8.1.1 Qué comparte con `Quad4Thermal`

Todo lo esencial de la formulación, que **no se repite aquí**:

- Un DOF por nodo, la temperatura T. `DOF_NAMES = ['T']`.
- Sistema semidiscreto $\mathbf{C}\dot{\mathbf{T}} + \mathbf{K}\mathbf{T} = \mathbf{F}$, de primer orden en el tiempo. Estacionario $\mathbf{K}\mathbf{T} = \mathbf{F}$ con el `LinearSolver` existente.
- $\mathbf{K}_e = \int \mathbf{B}^\top \mathbf{k}\, \mathbf{B}\, d\Omega$ simétrica y **semidefinida positiva**, con el modo nulo de temperatura uniforme.
- $\mathbf{C}_e = \int \rho c\, \mathbf{N}^\top \mathbf{N}\, d\Omega$, con `lumped` **por defecto** y la justificación física del principio del máximo.
- Convención de signo: $\bar{q} > 0$ es flujo **saliente** del dominio; frontera sin declarar es **adiabática**.
- Fuente Q como carga del elemento.
- Sin locking de ningún tipo; sin `internal_forces` (cierre por dominio del ADR 0012).
- Clase separada del `Hex8` mecánico, por el mismo argumento del `ClassVar DOF_NAMES`.

8.1.2 Qué cambia respecto a `Quad4Thermal`

1. Dimensión del gradiente y de la matriz B

$$\nabla T = \mathbf{B}\,\mathbf{T}_e, \qquad \mathbf{B} = \begin{bmatrix} \partial N_0/\partial x & \cdots & \partial N_7/\partial x \\ \partial N_0/\partial y & \cdots & \partial N_7/\partial y \\ \partial N_0/\partial z & \cdots & \partial N_7/\partial z \end{bmatrix} \in \mathbb{R}^{3\times 8}$$

`flux_dim = 3`. La conductividad $\mathbf{k}$ es 3×3; el material `ThermalConduction` la expande a $k\mathbf{I}_3$ cuando recibe un escalar.

$\mathbf{K}_e$ y $\mathbf{C}_e$ son 8×8. El modo nulo sigue siendo único (temperatura uniforme), luego rango$(\mathbf{K}_e) = 7$.

2. Sin espesor

`Hex8Thermal` **no lleva** `thickness`**.** El volumen sale de la geometría, igual que en los sólidos 3D mecánicos. Es la diferencia visible más inmediata para el usuario del YAML respecto al hermano 2D.

3. Interpolación trilineal

$$N_i(\xi, \eta, \zeta) = \tfrac{1}{8}(1 + \xi_i\xi)(1 + \eta_i\eta)(1 + \zeta_i\zeta)$$

Las mismas del `Hex8` mecánico, orden de nodos VTK_HEXAHEDRON. El gradiente resulta **trilineal** por elemento.

4. Cuadratura

Gauss $2 \times 2 \times 2$ (8 puntos) por defecto — `hex_2x2x2` del `QuadratureRegistry`, idéntica al `Hex8` mecánico. Exacta para $\mathbf{K}_e$ y $\mathbf{C}_e$ con jacobiano constante.

Reducida `hex_1x1x1` disponible por el mismo parámetro pero **no recomendada**: con un solo punto el rango de $\mathbf{K}_e$ es 3 frente a los 7 necesarios, dejando **4 modos hourglass** de temperatura sin detectar (frente al único del `Quad4Thermal` reducido). Se blinda con test de conteo.

5. Flujo de frontera por cara, no por borde

El flujo Neumann se aplica sobre **caras** con normal saliente, no sobre bordes:

$$\mathbf{f}_e^{\bar{q}} = -\int_{\Gamma_q} \bar{q}\, \mathbf{N}^\top \, d\Gamma$$

Numeración **paritaria con el `Hex8` mecánico** — misma `FACE_NODES`, mismas 6 caras: 0 $(-\zeta)$, 1 $(+\zeta)$, 2 $(-\eta)$, 3 $(+\xi)$, 4 $(+\eta)$, 5 $(-\xi)$. Reutiliza la maquinaria de integración sobre caras que ya existe.

$\bar{q}$ es constante o un campo $\bar{q}(\mathbf{x}, \mathbf{n})$ de la posición y de la normal exterior unitaria de la cara, que sale de la orientación antihoraria de `FACE_NODES` vista desde fuera; se integra sobre la geometría bilineal real de la cara, alabeada o no, con Gauss 2×2 (ADR 0022). La fuente Q admite también un campo $Q(\mathbf{x})$ y se integra con la regla de la capacidad $(2 \times 2 \times 2)$.

Para $\bar{q}$ uniforme sobre una cara plana de área A, el reparto es $\bar{q}A/4$ a cada uno de los 4 nodos de la cara y cero a los otros cuatro.

6. Validación específica 3D

Los benchmarks del hermano 2D se extienden, más uno propio de la dimensión:

- **Pared plana 3D**: perfil lineal exacto; verifica que la tercera dimensión no introduce error.

- **Cilindro hueco 3D**: perfil logarítmico, extensión del 2D — análogo térmico del Lamé 3D que el proyecto ya valida en mecánica.
- **Cross-check contra `Quad4Thermal`**: la misma pared plana resuelta en 2D y con una capa de `Hex8Thermal` con las caras z adiabáticas debe dar campos idénticos nodo a nodo. Es el análogo del cross-check 3D↔2D `plane_strain` que en A.bis blindó los materiales a 10-14 decimales, y **cubre los errores en la tercera dimensión sin necesidad de mallador**.

La **esfera hueca** —único caso con flujo genuinamente tridimensional— queda **diferida** de esta etapa (decisión del usuario, 2026-08-25): exige mallar un octante de corona esférica, y la cobertura que aportaría ya la da el cross-check anterior. Ver §Diálogo.

8.1.3 Contrato de implementación

```
name: Hex8Thermal
kind: element
status: validated        # draft → implemented → validated

interface:
  field: temperature
  dof_names: [T]
  n_nodes: 8
  flux_dim: 3                    # ∇T y q en 3D
  n_integration_points: 8        # default Gauss 2×2×2

admission:                 # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 2-3 (conducción de calor con elementos isoparamétricos)"
    - zienkiewicz_taylor: "vol. 1: problemas de campo (conducción de calor)"
  general_codes:
    - abaqus: "DC3D8"
    - ansys: "SOLID70"

parameters:
  - { name: quadrature, type: str, required: false, default: "hex_2x2x2",
      desc: "Regla desde QuadratureRegistry. hex_1x1x1 disponible pero deja 4
             modos hourglass de temperatura - sin estabilización" }
  # Sin `thickness`: el volumen sale de la geometría (igual que los sólidos 3D).

material_contract:
  signature: "compute_flux∇(T) -> (q, k)"
  gradient_kind: "gradiente físico ∂[T∂/x, ∂T∂/y, ∂T∂/z] - sin notación Voigt"
  accepted: [ThermalConduction]

conventions:
  sign_flux: "q > 0 ⇔ flujo SALIENTE del dominio; normal exterior"
  adiabatic_default: "cara sin condición declarada ⇒ q = 0 (aislada)"
  node_orientation: "orden VTK_HEXAHEDRON, volumen positivo - idéntico al Hex8"
  face_numbering: "FACE_NODES paritaria con Hex8: -ζ0() ζ1(+) -η2() ξ3(+) η4(+) -ξ5()"
  capacity_default: "lumping='lumped' por default; razón física en Quad4Thermal §5"

validity:
  - "conducción lineal; k independiente de T"
  - "material con field='temperature' (familia ThermalMaterial)"
```

```
  - "det J > 0 en todos los puntos de Gauss"
  - "el estacionario debe imponer al menos un Dirichlet de temperatura (el
     transitorio admite el cuerpo aislado)"

out_of_scope:
  - "convección (Robin) y radiación en la frontera - Etapa 8 posterior"
  - "acoplamiento termomecánico"
  - "conducción con advección; cambio de fase"
  - "estabilización del hourglass con cuadratura reducida"

acceptance:
  verification:
    - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
      setup: "T* suave no polinómica; k anisótropa de tensor lleno; cubo distorsionado
          (caras planas con nodos deslizados, elementos interiores curvos) y sector de
          cilindro hueco con flujo ¯q*(x, n) sobre las caras cilíndricas; estacionario
          con n = 4, 8, 16; transitorio Crank-Nicolson con Δt = h/2, capacidad
          concentrada, cubo; en el tiempo, T* lineal en x sobre malla 2×2×2 con θ = 1 y
           1/2"
      expect: "orden observado entre los dos últimos niveles ≥ 2 - 0.1 en L² y ≥ 1 -
          0.1 en energía ‖(‖e_E² = ∫∇e·k·∇e); en el tiempo ≥ orden del esquema - 0.1 (2
          con θ = 1/2, 1 con los demás); error decreciente en cada nivel; con T* lineal
           en x y en t, error de redondeo para todo θ"
      tol_abs: 0.1
    - name: patch_test_termico_lineal_3d
      setup: "malla de Hex8Thermal distorsionados con nodo(s) interior(es);
             imponer T = a + b·x + c·y + d·z en los nodos del contorno"
      expect: "los nodos interiores adoptan el campo lineal exacto y ∇T =
              (b, c, d) constante en TODOS los puntos de Gauss"
      tol_rel: 1.0e-10
    - name: modo_nulo_temperatura_uniforme
      setup: "K_e de un elemento aislado; campo T = ones(8)"
      expect: "K_e·T = 0 exacto; rango(K_e) = 7; exactamente un autovalor nulo"
      tol_abs: 1.0e-12
    - name: hourglass_con_integracion_reducida
      setup: "K_e con quadrature='hex_1x1x1' sobre elemento aislado"
      expect: "rango(K_e) = 3; 5 autovalores nulos (1 físico + 4 hourglass).
               Documenta la limitación, no la corrige"
      tol_abs: 1.0e-12
    - name: simetria_y_semidefinicion
      setup: "K_e y C_e con k isótropo y ortótropo, elemento regular y distorsionado"
      expect: "K_e == K_e.T y C_e == C_e.T exactas; autovalores de K_e ≥ 0 con un
               único cero; autovalores de C_e > 0 estrictos"
      tol_abs: 1.0e-12
  specific:
    - name: conduccion_1d_pared_plana_3d
      setup: "bloque de Hex8Thermal entre x=0 y x=L; T(0)=T_1, T(L)=T_2;
             las otras cuatro caras adiabáticas; estacionario"
      expect: "perfil lineal exacto en todos los nodos; q uniforme y sin
              componentes transversales (q_y = q_z = 0)"
      tol_rel: 1.0e-12
    - name: cilindro_hueco_logaritmico_3d
      setup: "sector de corona circular extruido; T(a)=T_a, T(b)=T_b;
             caras planas adiabáticas; estacionario"
      expect: "perfil logarítmico T(r) = T_a + (-T_bT_a)·ln(r/a)/ln(b/a);
              convergencia con refinamiento h"
```

```
      tol_rel: 5.0e-3
    # Esfera hueca (perfil T(r) = T_a + (-T_bT_a)·(1/a - 1/r)/(1/a - 1/b)):
    # DIFERIDA de esta etapa por decisión del usuario (2026-08-25). Es el único
    # caso con flujo genuinamente tridimensional, pero exige mallar un octante
    # de corona esférica -trabajo geométrico que ya difirió la campaña de
    # validación 3D de A.bis por el mismo -motivo y la cobertura que aportaría
    # sobre la tercera dimensión ya la da `consistencia_con_quad4thermal`, que
    # no necesita mallador. Retomar si entra un mallador 3D nativo (gmsh API)
    # o si aparece un caso de uso con flujo radial esférico.
    - name: fuente_volumetrica_uniforme
      setup: "Hex8Thermal regular y distorsionado con Q uniforme"
      expect: "Σf = Q·V_e exacto, invariante ante distorsión"
      tol_rel: 1.0e-12
    - name: flujo_neumann_en_una_cara
      setup: "Hex8Thermal con ¯q constante sobre cada una de las 6 caras"
      expect: "Σf = ¯q·A_cara con reparto A/4 a cada nodo de la cara y 0 en el
               resto; verificado en las 6 caras"
      tol_rel: 1.0e-12
    - name: balance_energetico_estacionario
      setup: "dominio con fuente Q y Dirichlet en parte de la frontera"
      expect: "Σ Q·V + Σ reacciones = 0"
      tol_rel: 1.0e-10
    - name: capacidad_lumped_vs_consistente
      setup: "C_e en ambos modos para elemento regular"
      expect: "ambas suman ρ·c·V_e; la lumped es diagonal y estrictamente positiva"
      tol_rel: 1.0e-12
    - name: anisotropia_desvia_el_flujo_3d
      setup: "k = diag(k1, k2, k3) distintos; campo lineal oblicuo a los ejes"
      expect: "q coincide con -k·∇T calculado a mano; no antiparalelo a ∇T"
      tol_rel: 1.0e-12
    - name: consistencia_con_quad4thermal
      setup: "problema de pared plana resuelto con Quad4Thermal (2D) y con una
              capa de Hex8Thermal con las caras z adiabáticas"
      expect: "campos de temperatura idénticos nodo a nodo; es el cross-check
               dimensional análogo al que el proyecto usa entre materiales 3D y
               2D plane_strain"
      tol_rel: 1.0e-12
    - name: jacobiano_degenerado_abortado
      setup: "elemento con nodos coplanares o numeración invertida"
      expect: "ValueError con mensaje claro"
    - name: rechazo_material_mecanico
      setup: "construir Hex8Thermal con un material de la familia mecánica"
      expect: "TypeError/ValueError identificando la incompatibilidad de familia"

references:
  - "Lewis R.W., Nithiarasu P., Seetharamu K.N. (2004). Fundamentals of the FEM for
     Heat and Fluid Flow. Wiley. §3, §6."
  - "Zienkiewicz O.C., Taylor R.L. (2000). The Finite Element Method, Vol. 1. §7."
  - "Carslaw H.S., Jaeger J.C. (1959). Conduction of Heat in Solids. Oxford. §7.2 (
     cilindro hueco). §9.3 cubre la esfera hueca, diferida de esta etapa."
  - "Reddy J.N., Gartling D.K. (2010). The Finite Element Method in Heat Transfer and
     Fluid Dynamics. CRC Press. §4."
```

8.1.4 Implementación

- Archivo: `solidum/elements/thermal/hex8_thermal.py`
- Clase: `Hex8Thermal`, registrada en `ElementRegistry`
- Base compartida: `solidum/elements/thermal/_shared.py` · `_ThermalSolid`
- Tests: `tests/test_hex8_thermal.py` — 26 verdes; `tests/test_thermal_loads_fields.py` (fuente y flujo variables); `tests/validation/test_mms_thermal.py` (soluciones manufacturadas); `tests/validation/test_thermal_hollow_cylinder.py` (cilindro hueco)

Notas de implementación:

- **La spec de extensión se confirmó en el código**: la clase concreta ocupa poco más de cien líneas y sólo aporta funciones de forma, gradiente, `FACE_NODES` y el flujo por cara. Todo lo demás —validación del material, conductividad, capacidad en ambos modos, fuente volumétrica, post-proceso— se hereda intacto de `_ThermalSolid`. Es la evidencia de que centralizar con el primer elemento fue correcto: el segundo caso real no necesitó tocar la base.
- **Kernel `_compute_gradient_kinematics_hex8` en `solid_3d/_shared.py`**, junto a su gemelo mecánico, por la misma razón que en 2D: comparte el jacobiano 3×3 y su inversión por cofactores, que es la parte cara.
- **`thickness` heredado como 1.0 y no expuesto** en el constructor. En 3D el volumen es geométrico; permitir el parámetro habría abierto la puerta a escalar el volumen dos veces. Blindado con test: escalar la geometría ×2 multiplica la fuente por 8, no por otro factor.

Predicciones de la spec confirmadas por test:

Predicción	Medido
`rango(K_e) = 7`, un único modo nulo	✔ exacto
Reducida `hex_1x1x1` ⇒ rango 3, 4 hourglass	✔ 5 autovalores nulos (1 físico + 4 espurios)
Flujo por cara reparte $-\bar{q}\cdot A/4$	✔ en las 6 caras
Patch test lineal 3D exacto	✔ también en elemento distorsionado

Cross-check 2D↔3D — el test central de esta spec. La misma pared plana resuelta con `Quad4Thermal` y con una capa de `Hex8Thermal` con las caras z adiabáticas da campos **idénticos nodo a nodo con error** 9.9×10^{-14}, y ambos coinciden con el perfil lineal analítico. Es el análogo térmico del cross-check 3D↔2D `plane_strain` que en A.bis blindó los materiales, y es la razón por la que el benchmark de la esfera hueca pudo diferirse sin dejar hueco de cobertura: detecta errores en la tercera dimensión sin necesitar mallador.

Cilindro hueco 3D: hecho el 2026-10-27 con `solidum.mesh.structured_mesh` (ADR 0022): sector extruido con las caras planas adiabáticas, el mismo error nodal que el 2D (4.4×10^{-4} con 4 elementos por dirección, tolerancia 5×10^{-3}). Sigue pendiente el balance energético. El estado `validated` refleja que la formulación está verificada contra solución analítica y contra el elemento 2D.

8.1.5 Diálogo

Puntos abiertos para el usuario. Sólo física y alcance.
2026-08-25 — IA → usuario. Un punto:

1. ~~**Benchmark de la esfera hueca.**~~ ✓ **Resuelto 2026-08-25 — DIFERIDA de esta etapa** (decisión del usuario).

La esfera hueca es el único caso propuesto con flujo genuinamente tridimensional: su perfil $\propto 1/r$ no lo producen ni la pared plana ni el cilindro, donde la temperatura varía en una sola dirección. Pero exige **mallar un octante de corona esférica**, trabajo geométrico que ya difirió la campaña de validación 3D de A.bis por el mismo motivo.

Se difiere porque **la cobertura que aportaría está cubierta por otra vía**: `consistencia_con_quad4thermal` detecta errores en la tercera dimensión comparando el mismo problema resuelto en 2D y en 3D, y no necesita mallador. La esfera daría confirmación adicional, no una comprobación ausente.

Nota técnica: cuando se retome, el `Hex8Thermal` **no** sufrirá el problema que bloquea al `Tet10` sobre superficie curva (deuda técnica #8) — al ser trilineal no tiene nodos intermedios que puedan quedar sobre aristas rectas. El bloqueante es sólo de mallado.

Criterio de retoma: si entra un mallador 3D nativo (gmsh API) o aparece un caso de uso con flujo radial esférico.

- **2026-09-22** · Auditoría global: `quadrature` acepta clave del registro o tupla `(points, weights)` en todos los sólidos (`resolve_quadrature` valida la familia por medida de referencia y dimensión). La capacidad se integra siempre con `CAPACITY_QUADRATURE` (regla completa) aunque K use integración reducida (con un punto C_e era de rango 1). tolerancia del jacobiano ahora **relativa** (`det J` $\leq$ `JACOBIAN_RTOL` $\cdot$ $\Pi\|$`fila_i(J)`$\|$, cota de Hadamard, adimensional): la absoluta `1e-10` sobre `det J` rechazaba mallas finas en metros sin distorsión.

- **2026-09-22** · Ensamblaje por lotes (ADR 0014): el gradiente ∂`N/`∂`x` pasa a un núcleo compilado compartido con el elemento mecánico gemelo y se expone como `BATCH_KINEMATICS`; la familia térmica se evalúa en el mismo kernel de familia con `k`$\cdot\nabla$`T` como "esfuerzo", de modo que $\int$`B`$^\top$`(k`$\cdot\nabla$`T) dΩ = K_e`$\cdot$`T_e`. Sin `ElementState` (no hay historia).

- **2026-09-22** · Cierre del ADR 0014 (§9): la cinemática por lotes ya no lanza con jacobiano degenerado (devuelve `det J` $\leq$ `0`; el ensamblador lanza con el id), el kernel de familia tiene variante paralela bit a bit igual a la serie, y el post-proceso por familia devuelve `grad_T` y `flux =` `-k`$\cdot\nabla$`T` iguales a `compute_gauss_state` (`tests/test_batch_postprocess.py`). Formulación sin cambios.

- **2026-10-27** · Cargas variables y verificación por soluciones manufacturadas (ADR 0022, fase 3). `compute_body_source` acepta un campo `Q(x)` y `compute_face_flux` un campo $\bar{\mathbf{q}}$`(x, n)`, con `n` la normal exterior unitaria de la cara; el flujo se integra sobre la geometría real de la cara con sus funciones de forma bilineales (`face_load`, el mismo que la tracción del `Hex8`) y la fuente con la regla de la capacidad. Con flujo y fuente constantes el resultado no cambia. Declara `REFERENCE_CELL`, `REFERENCE_NODES` y las funciones de forma de sus caras. Nueva aceptación «soluciones manufacturadas» y hecho el benchmark del cilindro hueco 3D.

Capítulo 9

Modelos Constitutivos — 1D

9.1 CableMaterial1D

*Orden de trabajo. El usuario escribe **especificación física**, **formulación** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

9.1.1 Especificación física

0. Descripción general

Material 1D memoryless que modela la respuesta axial de un cable: **elástico lineal en tensión y nulo en compresión**. No tiene variables internas ni historia — la respuesta depende exclusivamente de la deformación instantánea. Captura la propiedad esencial de un cable: que solo transmite esfuerzo cuando está tensado.

1. Ecuación constitutiva

$$\sigma(\varepsilon) \;=\; E\,\langle\varepsilon\rangle^{+} \;=\; \begin{cases} E\,\varepsilon & \varepsilon > 0 \\ 0 & \varepsilon \le 0 \end{cases}$$

Rampa lineal de pendiente E en tensión ($\varepsilon > 0$), cero en compresión o estado sin deformación ($\varepsilon \le 0$). $\langle\cdot\rangle^{+}$ denota la parte positiva (operador de MacAuley).

2. Módulo tangente

$$E_t(\varepsilon) \;=\; \frac{d\sigma}{d\varepsilon} \;=\; \begin{cases} E & \varepsilon > 0 \\ 0 & \varepsilon \le 0 \end{cases}$$

Discontinuo en $\varepsilon = 0$. Por convención el valor en el punto de corte se asigna al régimen compresivo ($E_t = 0$), lo que evita generar rigidez espuria cuando el cable está exactamente sin tensión.

3. Variables internas

No hay. La respuesta es puramente función de ε actual; el estado del material es el vacío.

4. Interpretación física

- $\varepsilon > 0$: cable tensado — se comporta como barra elástica.
- $\varepsilon < 0$: cable destensado — no transmite nada, la carga pasa por otros caminos del modelo.
- $\varepsilon = 0$: frontera neutra — no hay tensión, no hay rigidez.

La no linealidad es **constitutiva** (en la respuesta del material), no geométrica. Un elemento con este material puede tener cualquier cinemática (lineal o corotacional); la unilateralidad solo vive aquí.

9.1.2 Contrato de implementación

```yaml
name: CableMaterial1D
kind: material
status: validated

interface:
  strain_dim: 1
  primary_state_var: null      # memoryless, no exporta variable principal

admission:                 # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - bathe: "cap. 4-5 (barra con σ = E·ε): formulación base; la anulación en compresió
        n no aparece como tal en los textos de la lista. Clasificado estándar por el
        usuario el 2026-09-30 (el criterio no decidía; está en los tres códigos)"
  general_codes:
    - ansys: "LINK180 con SECCONTROL de sólo tracción"
    - abaqus: "*ELASTIC + *NO COMPRESSION"
    - opensees: "uniaxialMaterial Elastic con Eneg = 0"

parameters:
  - { name: E, type: float, required: true, desc: "Módulo de Young (en tensión)" }

signature:
  compute_state: "ε(: float, state_vars=None) -> σ(: float, E_tangent: float,
      state_vars)"
  strain_kind: axial escalar

conventions:
  sign: "σ > 0 ⇔ε  > 0 (elongación); ε≤  0 ⇒σ  = 0"
  state_passthrough: "state_vars se devuelve intacto (el material no almacena historia)
      "

validity:
  - "E > 0"
  - "ε|| ≤ 1e-2 en el régimen tensado (elasticidad lineal)"
  - "IS_UNILATERAL = True ⇒ solo admitido en elementos con ACCEPTS_UNILATERAL = True (
      Cable2DCorot, Cable3DCorot). La base Element rechaza la combinación con Truss* en
       construcción."
```

```
out_of_scope:
  - plasticidad, fluencia, fatiga
  - viscoelasticidad / dependencia temporal
  - "pretensión inicial: no es del material; se modela con la longitud sin tensión L0
     de Cable2DCorot y Cable3DCorot (2026-10-01)"
  - regularización numérica en compresión (no hay rigidez residual en ε≤ 0)
  - anisotropía direccional; la respuesta es puramente axial escalar

numerical_caveats:
  - "En transiciones tensado ↔ destensado ε( cruza 0), E_tangent salta de E a 0 y
     Newton-Raphson puede oscilar. Mitigación: usar arc-length o pasos de carga finos
     si el problema tiene destensiones reales."
  - "Un cable completamente destensado tiene K_M = 0 y K_G = 0 ⇒ matriz tangente
     singular en los DOFs del elemento. El usuario debe garantizar que otros elementos
      del modelo proporcionen rigidez suficiente, o pretensar el cable antes de
     aplicar cargas que puedan destensarlo."

acceptance:
  - name: respuesta en tensión pura
    setup: "ε = 1e-4"
    expect: "σ = E · 1e-4, E_tangent = E"
    tol_rel: 1.0e-12

  - name: respuesta en compresión pura
    setup: "ε = -1e-4"
    expect: "σ = 0, E_tangent = 0"
    tol_abs: 1.0e-15

  - name: punto de corte
    setup: "ε = 0"
    expect: "σ = 0, E_tangent = 0 (régimen compresivo por convención)"
    tol_abs: 1.0e-15

  - name: state_vars se devuelve intacto
    setup: "state_vars = {'dummy': 42}, ε arbitrario"
    expect: "el tercer retorno es idéntico al state_vars de entrada"

references:
  - "Irvine H.M., Cable Structures, MIT Press, §1.2 (modelo elástico del cable)"
  - "MacAuley M.A., on the deflection of beams, 1919 (operador parte positiva)"
```

9.1.3 Implementación

- **Archivo**: solidum/materials/cable_1d.py — archivo propio, sin dependencia de otros módulos de material salvo la clase base `Material`.
- **Clase**: `CableMaterial1D` — hereda directamente de `Material` (no de `Elastic1D` ni de ningún otro material), registrada vía `@MaterialRegistry.register`.
- **Tests**: tests/test_cable_material.py · `TestCableMaterial1D` — seis tests:
- `test_acceptance_tension_pura` (criterio 1).

- `test_acceptance_compresion_pura` (criterio 2).
- `test_acceptance_punto_de_corte` (criterio 3).
- `test_acceptance_state_vars_passthrough` (criterio 4).
- `test_registro_en_registry` — verifica autodiscover.
- `test_validacion_E_positivo` — rechaza $E \leq 0$.
- **Notas de traducción**:
- La ramificación en `compute_state` es un único `if strain > 0.0`. El `>` estricto implementa la convención del punto de corte (el cero se asigna al régimen compresivo). El `else` cubre tanto $\varepsilon < 0$ como $\varepsilon = 0$ con las mismas salidas.
- `state_vars` se devuelve como llega (identidad, no copia). El test `test_acceptance_state_vars_passthrough` usa `assertIs` para confirmar identidad.
- Validación de $E > 0$ al construir, con mensaje claro. Atrapa typos del input YAML temprano.
- El material acepta `**kwargs` en `compute_state` por compatibilidad con elementos que pudieran pasar argumentos adicionales (convención del proyecto).

9.1.4 Diálogo

- **2026-04-21** · Material creado como pieza independiente para la futura implementación de elementos de cable (`Cable2DCorot`, `Cable3DCorot`). Deliberadamente **no hereda** de `Elastic1D` aunque su rama en tensión sea idéntica: mantener el material autocontenido simplifica su lectura y evita acoplamientos futuros si la rama elástica cambia en otro archivo.
- **2026-04-21** · Discontinuidad en $\varepsilon = 0$: documentada explícitamente en `out_of_scope: regularización numérica`. Si aparece un caso de uso con destensiones frecuentes donde Newton-Raphson oscile, se añadirá una variante `CableMaterial1DRegularized` con $E_c \ll E$ en compresión — **no** se modificará este material (contrato limpio).
- **2026-05-12** · Restricción de emparejamiento elevada a contrato: `IS_UNILATERAL = True` en el material; `ACCEPTS_UNILATERAL = True` en `Cable2DCorot`/`Cable3DCorot`. La base `Element._validate_material_compatibility` rechaza la combinación con `Truss*` (cuyo tangente colapsa la rigidez global sin K_G que lo compense). Antes el contrato lo permitía y solo se desaconsejaba en docstring; ahora falla limpio al construir.
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): declara `STATE_SCHEMA` (obligatorio en todo el catálogo). Sin kernel por lotes: los elementos 1D siguen el camino por elemento (fase 4 de la propuesta, diferida).

9.2 Elastic1D

*Spec **retroactiva**: el material existe desde el origen del proyecto y está validado por tests preexistentes. Esta spec lo documenta sin cambiar comportamiento — H-5.3 de la auditoría 2026-05-18.*

9.2.1 Especificación física

0. Descripción general

Modelo elástico lineal isótropo unidimensional. Captura la relación σ–ε axial de barras, cables traccionados (no unilaterales — para unilateralidad ver `CableMaterial1D`), y la respuesta elástica de cualquier elemento 1D del catálogo (truss, frame Euler en su componente axial). Independiente de la velocidad, sin variables históricas.

Es el modelo más sencillo del catálogo: **una sola constante elástica** (módulo de Young E) y una sola ecuación constitutiva.

1. Descomposición de la deformación

No aplica — no hay parte plástica. Toda la deformación es elástica:

$$\varepsilon = \varepsilon^e$$

2. Ley elástica (Hooke 1D)

$$\sigma = E\,\varepsilon$$

con $E > 0$ módulo de Young. Tangente constante $E_t = E$ en todo régimen.

3. Superficie de fluencia / criterio de daño

No aplica — el material es indefinidamente elástico.

4. Regla de flujo

No aplica.

5. Endurecimiento

No aplica.

6. Condiciones de Kuhn-Tucker

No aplica.

7. Variables internas

Ninguna — `state_vars = None` se acepta y se devuelve intacto. Sin `PRIMARY_STATE_VAR`.

8. Notación

Esfuerzo y deformación escalares en convención axial: $\sigma > 0 \Leftrightarrow$ tracción, $\varepsilon > 0 \Leftrightarrow$ alargamiento (Reglas §5).

9.2.2 Formulación numérica

9. Esquema temporal

No aplica — el material no tiene historia. Cada evaluación de `compute_state` es independiente.

10. Return mapping / actualización

No aplica. La evaluación se reduce a una multiplicación escalar:

$$\sigma_{n+1} = E\,\varepsilon_{n+1}, \qquad E_t = E$$

11. Tangente algorítmica consistente

$$C^{\text{alg}} = E$$

Coincide con la tangente continua y es simétrica trivialmente. `IS_SYMMETRIC = True` (por default del contrato base).

12. Caveats numéricos

Ninguno — el modelo es perfectamente bien condicionado para $E > 0$.

9.2.3 Contrato de implementación

```yaml
name: Elastic1D
kind: material
status: validated

interface:
  strain_dim: 1
  primary_state_var: null    # sin variables internas
  is_symmetric: true

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - bathe: "cap. 4-5 (barra con σ = E·ε)"
  general_codes:
    - abaqus: "*ELASTIC en T2D2 / T3D2"
    - opensees: "uniaxialMaterial Elastic"

parameters:
  - { name: E,       type: float, required: true,  desc: "Módulo de Young (>0)" }
  - { name: density, type: float, required: false, default: null,
      desc: "Densidad (kg/m^3). Opcional al construir; obligatoria solo si se ensambla
          peso propio o masa (ADR 0008)" }
```

```yaml
signature:
  compute_state: "ε(: float, state_vars=None, **kwargs) -> σ(: float, E_t: float,
      state_vars)"
  strain_kind: "axial escalar"

state_variables: []        # ninguna

conventions:
  sign: "σ > 0 ⇔ tracción, ε > 0 ⇔ alargamiento (Reglas §5)"
  units: "[E] = σ[] / ε[] = Pa (o consistente con el sistema)"

validity:
  - "E > 0"
  - "régimen elástico indefinido - sin umbral superior"
  - "pequeñas deformaciones ε(|| ≲ 1e-2 por hipótesis cinemática global)"

out_of_scope:
  - "endurecimiento / fluencia ⇒ usar Elastoplastic1D"
  - "softening por daño ⇒ usar IsotropicDamage1D"
  - "unilateralidad (sin compresión) ⇒ usar CableMaterial1D"
  - "anisotropía o dependencia de temperatura"
  - "viscoelasticidad"

acceptance:
  # Cubierto por tests existentes de elementos 1D (Truss, Frame, Cable, etc.)
  # que usan Elastic1D como material por default. La verificación se hace
  # implícita en cada test de elemento.
  verification:
    - name: ley_de_hooke_exacta
      setup: "evaluar compute_state con ε arbitrario; comparar σ = E·ε"
      expect: "σ = E·ε exacto; tangente = E"
      tol_rel: 1.0e-14
    - name: tangente_constante
      setup: "evaluar dos veces con strain distintos"
      expect: "E_t devuelto idéntico en ambas evaluaciones"
      tol_rel: 1.0e-14
    - name: ciclo_carga_descarga_sin_historia
      setup: "ε creciente luego decreciente; verificar reversibilidad σ↔ε"
      expect: "σε()=E·ε para todo ε (sin histéresis, sin state_vars retornados
          modificados)"
      tol_rel: 1.0e-14
    - name: rechazo_E_no_positivo
      setup: "construir Elastic1D(E=0) y Elastic1D(E=-1)"
      expect: "ValueError con mensaje claro"
    - name: rechazo_density_negativa
      setup: "construir Elastic1D(E=210e9, density=-1)"
      expect: "ValueError con mensaje claro"

references:
  - "Cualquier texto elemental de mecánica de materiales (Beer, Hibbeler, Timoshenko §
      1)."
```

9.2.4 Implementación

- **Archivo**: solidum/materials/elastic.py.
- **Clase**: `Elastic1D`, registrada vía `@MaterialRegistry.register`.
- **Estado**: `state_vars=None` permitido; se acepta y se devuelve sin tocar (compatibilidad con elementos no lineales que pasan diccionarios históricos).
- `admissibility_scale`: hereda el comportamiento por defecto del contrato `Material` (no usado — sin criterio de admisibilidad).
- **Tests**: ningún test unitario propio de `Elastic1D` (es trivial); su correctitud se ejercita implícitamente en todos los tests de elementos 1D que lo usan como material (decenas de tests en `tests/test_truss*.py`, `tests/test_frame*.py`, `tests/test_cable*.py`, `tests/test_dynamic*.py`).

9.2.5 Diálogo

- **2026-05-19** · Spec creada retroactivamente para cerrar el hueco H-5.3 de la auditoría 2026-05-18. El material es anterior a la convención de specs validadas; no se modifica código. La aceptación se considera cumplida por cobertura indirecta en tests de elementos.
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): declara `STATE_SCHEMA` (obligatorio en todo el catálogo). Sin kernel por lotes: los elementos 1D siguen el camino por elemento (fase 4 de la propuesta, diferida).

9.3 Elastoplastic1D

*Spec **retroactiva**: el material existe desde la fase inicial del proyecto y está validado por tests unitarios y de integración. Esta spec lo documenta sin cambiar comportamiento — H-5.3 de la auditoría 2026-05-18.*

9.3.1 Especificación física

0. Descripción general

Modelo elastoplástico unidimensional independiente de la velocidad ("rate-independent") con criterio de fluencia $|\sigma| \leq \sigma_y + H\,\alpha$ y **endurecimiento isótropo lineal**. Es el análogo 1D de `VonMises2D` — la versión axial sobre la que se valida y benchmarkean los pipelines plásticos. Se usa en barras (`Truss2D/3D`) y como componente axial conceptual de marcos plásticos (pendiente `FiberSection`).

1. Descomposición aditiva

$$\varepsilon = \varepsilon^e + \varepsilon^p$$

con ε^p histórica (memoria del material).

2. Ley elástica

$$\sigma = E\,\varepsilon^e = E\,(\varepsilon - \varepsilon^p)$$

3. Criterio de fluencia

$$f(\sigma, \alpha) = |\sigma| - (\sigma_y + H\,\alpha) \;\leq\; 0$$

donde $\sigma_y > 0$ es la fluencia inicial y $H \geq 0$ el módulo de endurecimiento isótropo lineal.

4. Regla de flujo (asociada)

$$\dot{\varepsilon}^p = \dot{\gamma}\ \mathrm{sign}(\sigma)$$

5. Endurecimiento isótropo lineal

$$\dot{\alpha} = \dot{\gamma}$$

α es la deformación plástica acumulada equivalente, escalar monótona no decreciente.

6. Condiciones de Kuhn-Tucker

$$\dot{\gamma} \geq 0, \qquad f \leq 0, \qquad \dot{\gamma}\,f = 0$$

7. Variables internas

Símbolo	Tipo	Significado
ε^p	escalar	deformación plástica acumulada con signo
α	escalar	deformación plástica acumulada equivalente (≥ 0)

`alpha` es la variable principal exportable (`PRIMARY_STATE_VAR = 'alpha'`).

8. Notación

Escalares en convención axial: $\sigma > 0 \Leftrightarrow$ tracción, $\varepsilon > 0 \Leftrightarrow$ alargamiento (Reglas §5). ε^p con signo (puede ser negativa tras carga compresiva).

9.3.2 Formulación numérica

9. Esquema temporal — Backward Euler implícito

Dado el estado convergido $\{\varepsilon^p_n, \alpha_n\}$ al paso n y la deformación total ε_{n+1}, resolver para $\{\sigma_{n+1}, \varepsilon^p_{n+1}, \alpha_{n+1}\}$.

10. Return mapping — forma cerrada

Predictor elástico:

$$\sigma^{\text{trial}} = E\,(\varepsilon_{n+1} - \varepsilon^p_n)$$

Función de fluencia trial:

$$f^{\text{trial}} = |\sigma^{\text{trial}}| - (\sigma_y + H\,\alpha_n)$$

- Si $f^{\text{trial}} \leq \text{tol} \Rightarrow$ **paso elástico**: $\sigma_{n+1} = \sigma^{\text{trial}}$, $E_t = E$, estado intacto.
- Si $f^{\text{trial}} > \text{tol} \Rightarrow$ **paso plástico**, retorno radial cerrado:

$$\Delta\gamma = \frac{f^{\text{trial}}}{E + H}, \qquad \sigma_{n+1} = \sigma^{\text{trial}} - \Delta\gamma\, E\, \text{sign}(\sigma^{\text{trial}})$$

$$\varepsilon^p_{n+1} = \varepsilon^p_n + \Delta\gamma\, \text{sign}(\sigma^{\text{trial}}), \qquad \alpha_{n+1} = \alpha_n + \Delta\gamma$$

Sin Newton local — la forma cerrada es exacta en 1D.

11. Tangente algorítmica consistente

En régimen elástico: $E_t = E$. En régimen plástico:

$$E_t = \frac{E\,H}{E + H}$$

con $H = 0$ recuperando $E_t = 0$ (plasticidad perfecta, descenso de carga indefinido bajo control de desplazamiento).

`IS_SYMMETRIC = True` (trivial en 1D).

12. Caveats numéricos

- **Plasticidad perfecta** ($H = 0$): tangente $E_t = 0$ tras la fluencia. Newton-Raphson global puede oscilar o requerir line search en problemas multibarra (modos de mecanismo). Mitigación: usar incremento pequeño cerca del umbral o activar `line_search=True` en el solver.
- **Switching exacto trial elástico ↔ plástico**: la tolerancia `is_admissible` (ADR 0006) usa `admissibility_scale = σ_y + H·α` y patrón atol + rtol·escala — invariancia bajo cambio de unidades.

9.3.3 Contrato de implementación

```yaml
name: Elastoplastic1D
kind: material
status: validated

interface:
  strain_dim: 1
  primary_state_var: alpha   # deformación plástica acumulada equivalente
  is_symmetric: true

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_souza_neto: "§6.2 (modelo uniaxial) y cap. 7 (integración implícita)"
  general_codes:
    - opensees: "uniaxialMaterial Hardening"
    - abaqus: "*PLASTIC (Mises) en T2D2 / T3D2"

parameters:
  - { name: E,       type: float, required: true,  desc: "Módulo de Young (>0)" }
  - { name: sigma_y, type: float, required: true,  desc: "Esfuerzo de fluencia inicial
      (>0)" }
  - { name: H,       type: float, required: false, default: 0.0,
      desc: "Módulo de endurecimiento isótropo lineal ≥(0). H=0 ⇒ plasticidad perfecta"
          }
  - { name: density, type: float, required: false, default: null,
      desc: "Densidad (kg/m³). Opcional al construir; obligatoria solo si se ensambla
          peso propio o masa (ADR 0008)" }

signature:
  compute_state: "ε(: float, state_vars=None) -> σ(: float, E_t: float, state_vars')"
  strain_kind: "axial escalar"
```

```
state_schema:
  eps_p: "float - deformación plástica acumulada con signo"
  alpha: "float ≥ 0 - deformación plástica acumulada equivalente"

conventions:
  sign: "σ > 0 ⇔ tracción, ε^p con signo (puede ser negativa)"
  units: "[Eσ]=[_y]=[H]=Pa; α adimensional"

validity:
  - "E > 0, σ_y > 0, H ≥ 0"
  - "carga proporcional o no proporcional (sin restricción de trayectoria)"
  - "pequeñas deformaciones ε(|| ≲ 1e-2)"

out_of_scope:
  - "endurecimiento cinemático (back-stress) ⇒ requiere variante 1D dedicada"
  - "endurecimiento isótropo no lineal (Voce, exponencial)"
  - "ablandamiento (H<0) ⇒ requiere regularización"
  - "viscoplasticidad / dependencia de la velocidad"
  - "grandes deformaciones (descomposición multiplicativa F = F^e · F^p)"

acceptance:
  # Cubierto por tests/test_materials_unit.py y tests/test_truss_plastic.py
  unit:
    - name: paso_elastico
      setup: "ε = 0.5·σ_y/E (mitad del umbral); state_vars=None"
      expect: "σ = E·ε exacto, α=0, ε^p=0, tangente=E"
      tol_rel: 1.0e-12
    - name: fluencia_uniaxial_traccion
      setup: "ε = 1.5·σ_y/E (1.5× el umbral elástico)"
      expect: "σ = σ_y (con H=0); α=0.5·σ_y/E; ε^p > 0; tangente=0"
      tol_rel: 1.0e-10
    - name: endurecimiento_lineal
      setup: "carga monotónica hasta ε = 2·σ_y/E con H = E/10"
      expect: "σ_final = σ_y + H·α; tangente = E·H/(E+H) en régimen plástico"
      tol_rel: 1.0e-10
    - name: ciclo_carga_descarga
      setup: "ε creciente hasta plástico, luego decreciente"
      expect: "descarga elástica con tangente E; α permanece constante durante descarga
          ; histéresis correcta"
      tol_rel: 1.0e-10
    - name: cambio_de_signo
      setup: "tracción plástica luego compresión hasta fluencia inversa"
      expect: "fluencia inversa en σ = σ-(_y + H·α) (endurecimiento isótropo es simé
          trico)"
      tol_rel: 1.0e-10
    - name: monotonicidad_alpha
      setup: "trayectoria arbitraria"
      expect: "α no decrece nunca"

  # Cubierto por tests de integración con barras plásticas
  integration:
    - name: truss_uniaxial_plastico
      setup: "Truss2D con Elastoplastic1D, carga axial creciente"
      expect: "respuesta fuerza-desplazamiento bilineal exacta (E hasta fluencia, E_t
          después)"
```

```
      tol_rel: 1.0e-6

references:
  - "Simó J.C., Hughes T.J.R. (1998). Computational Inelasticity. Springer. §1 (J2 1D
     return mapping)."
  - "de Souza Neto E.A., ćPeri D., Owen D.R.J. (2008). Computational Methods for
     Plasticity. Wiley. §6.2 (uniaxial)."
  - "Lubliner J. (1990). Plasticity Theory. Macmillan. §3.2."
```

9.3.4 Implementación

- **Archivo**: solidum/materials/plastic_1d.py.
- **Clase**: `Elastoplastic1D`, registrada vía `@MaterialRegistry.register`.
- `admissibility_scale`: $\sigma_y + H\,\alpha$ (ADR 0006). El switching elástico/plástico usa el patrón atol + rtol·escala.
- **State schema**: `{'eps_p': float, 'alpha': float}`. `state_vars=None` se interpreta como estado virgen ($\varepsilon^p = 0$, $\alpha = 0$).
- **Tests**:
- tests/test_materials_unit.py · clase `TestElastoplastic1D` (carga, descarga, endurecimiento, plasticidad perfecta).
- Integración: cualquier test de truss con material plástico.

9.3.5 Diálogo

- **2026-05-19** · Spec creada retroactivamente para cerrar el hueco H-5.3 de la auditoría 2026-05-18. Material anterior a la convención de specs; no se modifica código. La aceptación se considera cumplida por los tests unitarios existentes.
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): declara `STATE_SCHEMA` (obligatorio en todo el catálogo). Sin kernel por lotes: los elementos 1D siguen el camino por elemento (fase 4 de la propuesta, diferida).

9.4 IsotropicDamage1D

*Orden de trabajo. El usuario escribe **especificación física**, **formulación** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

*Spec **retroactiva con ampliación**: modelo implementado desde sesiones anteriores con tangente secante; esta spec lo documenta y añade la tangente algorítmica consistente, replicando la derivación de [[IsotropicDamage2D]] reducida a 1D escalar.*

9.4.1 Especificación física

Versión 1D de `IsotropicDamage2D`. Modelo de daño escalar isótropo en pequeñas deformaciones para barras, cables y elementos axiales. Captura degradación progresiva de la rigidez axial con ablandamiento exponencial.

Ecuaciones

- **Constitutiva**: $\sigma = (1 - d)\, E\, \varepsilon$, con $\sigma^{\text{eff}} = E\varepsilon$.
- **Deformación equivalente**: $\varepsilon_{\text{eq}} = |\varepsilon|$.
- **Variable histórica**: $\kappa_{n+1} = \max(\kappa_n, |\varepsilon_{n+1}|)$, con κ_0 inicial.
- **Daño**: $d(\kappa) = \begin{cases} 0 & \kappa \le \kappa_0 \\ 1 - (\kappa_0/\kappa)\, e^{-\alpha(\kappa - \kappa_0)} & \kappa > \kappa_0 \end{cases}$, saturado a `DAMAGE_MAX`.

Todas las propiedades físicas (irreversibilidad, asintótica $d \to 1$, papel de α) son las del modelo 2D restringidas a un escalar. No distingue tracción de compresión.

9.4.2 Formulación numérica

Tangente algorítmica consistente

$$\frac{\partial \sigma}{\partial \varepsilon} = (1 - d)\, E - E\varepsilon\, \frac{\partial d}{\partial \kappa}\, \frac{\partial \kappa}{\partial \varepsilon}$$

con:

- $\dfrac{\partial d}{\partial \kappa} = (1 - d)\left(\dfrac{1}{\kappa} + \alpha\right)$ (idéntico a 2D).
- $\dfrac{\partial \kappa}{\partial \varepsilon} = \operatorname{sign}(\varepsilon)$ en **carga activa** ($|\varepsilon| > \kappa_n$); 0 en descarga.

Tangente final:

$$E^{\text{alg}} = \begin{cases} E & \kappa \le \kappa_0 \text{ (sin daño)} \\ (1 - d)\, E & \text{descarga} \\ (1 - d)\, E\big[1 - (1/\kappa + \alpha)\, \kappa\big] = -(1 - d)\, E\, \alpha\, \kappa & \text{carga activa, } d < d_{\max} \\ (1 - d_{\max})\, E & d = d_{\max} \text{ (saturación)} \end{cases}$$

donde se ha usado $(1/\kappa + \alpha)\kappa = 1 + \alpha\kappa$ y $E\varepsilon\, \operatorname{sign}(\varepsilon) = E|\varepsilon| = E\kappa$ en carga activa.

Signo de la tangente consistente

En carga activa **siempre es negativa**: $E^{\text{alg}} = -(1-d)\,E\,\alpha\,\kappa < 0$ porque $1-d > 0$, $E > 0$, $\alpha > 0$, $\kappa > 0$. Refleja físicamente el régimen de **softening** (ablandamiento): la curva σ–ε desciende tras el umbral.

Implicaciones numéricas:

- La rigidez tangente del elemento ($\mathbf{K}_e = \int B^\top E^{\text{alg}} B\, dV$) puede tener autovalores negativos. La matriz **sigue siendo simétrica** (escalar × forma cuadrática) — `IS_SYMMETRIC = True` se mantiene.
- La matriz global puede dejar de ser **positiva definida**. El despachador algebraico (ADR 0003) detecta el fallo de Cholesky y degrada a LU automáticamente — no requiere cambio declarativo.
- Control de carga puede no converger en problemas con suficiente daño para producir snap-back; en ese régimen se requiere control de longitud de arco (`ArcLengthSolver`).

Por qué no tests de integración en esta spec

A diferencia de [[IsotropicDamage2D]], no se añade benchmark de integración Truss2D + IsotropicDamage1D + NonlinearSolver: con un solo elemento axial el comportamiento global está dominado por la inestabilidad de softening ($E_{\text{alg}} < 0$) y la única vía limpia de seguir la respuesta post-pico es arc-length. La validación de la tangente consistente se cubre con tests unitarios (incluyendo diferencia finita centrada como verificación de la derivada cerrada). Si en el futuro aparece un caso real de problema con daño 1D ramificado, se añadirá un benchmark dedicado con `ArcLengthSolver`.

9.4.3 Contrato de implementación

```yaml
name: IsotropicDamage1D
kind: material
status: validated

interface:
  strain_dim: 1
  primary_state_var: damage
  is_symmetric: true       # tangente escalar ⇒ contribución elemento simétrica (puede
      ser indefinida pero simétrica)

admission:                 # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_borst_crisfield: "§6.2 (daño isótropo basado en elasticidad, ablandamiento
        exponencial)"
    - de_souza_neto: "cap. 12 (mecánica del daño)"
  general_codes:
    - code_aster: "MAZARS, ENDO_ISOT_BETON (daño escalar isótropo; leyes de
        ablandamiento propias)"

parameters:
  - { name: E,        type: float, required: true,  desc: "Módulo de Young intacto" }
  - { name: kappa_0,  type: float, required: true,  desc: "Umbral de deformación
      equivalente ε||" }
```

```
  - { name: alpha,    type: float, required: true,  desc: "Velocidad de degradación
      exponencial (>0)" }
  - { name: density,  type: float, required: false, default: null, desc: "Densidad (ADR
      0008)" }

signature:
  compute_state: "ε(: float, state_vars=None) -> σ(: float, E_tan: float, state_vars')"
  strain_kind: axial escalar

state_schema:
  kappa:  "float ≥κ _0 - máxima ε|| histórica"
  damage: "float ∈ [0, DAMAGE_MAX]"

conventions:
  sign: "σ > 0 ⇔ tracción; el modelo no distingue tracción de compresión en la activaci
      ón del daño"
  damage_irreversibility: "κ monótono no decreciente; dκ() no decreciente"

validity:
  - "E > 0, κ_0 > 0, α > 0"
  - "ε|| ≲ 1e-2 (régimen de pequeñas deformaciones)"

out_of_scope:
  - "split tracción/compresión"
  - "fatiga / acumulación cíclica"
  - "regularización para mesh-dependency en problemas con varios elementos en serie"
  - "test de integración con NonlinearSolver - requiere arc-length para superar el snap
      -back, fuera de scope hasta caso real"

numerical_caveats:
  - "Tangente consistente NEGATIVA en carga activa (softening). La rigidez global puede
      no ser PD; el despachador algebraico (ADR 0003) degrada a LU automáticamente al
      detectar fallo de Cholesky."
  - "Control de carga con un elemento aislado tras alcanzar el pico se vuelve inestable
      . Usar arc-length para seguir la rama post-pico."
  - "Transición ↔cargadescarga discontinua en la tangente (rama loading vs unloading):
      aceptable para Newton convergente, problemático cerca del umbral en problemas
      patológicos."

acceptance:
  unit_existing_no_regression:
    - name: secant_response_below_threshold
      expect: "ε pequeño, κ≤κ  _0 ⇒ d=0, σ=E·ε, tangente = E"
      tol_rel: 1.0e-12

    - name: damage_evolution_exponential
      expect: "ley exponencial reproducida con la fórmula"
      tol_rel: 1.0e-10

  unit_consistent_tangent:
    - name: tangent_equals_E_below_threshold
      setup: "ε < κ_0"
      expect: "E_tan = E (sin daño)"
      tol_rel: 1.0e-12

    - name: tangent_equals_secant_on_unloading
```

```
      setup: "cargar a κ > κ_0; descargar a ε|| < κ"
      expect: "E_tan = (1-d)·E exacto"
      tol_rel: 1.0e-12

    - name: tangent_negative_on_loading
      setup: "carga activa con κ > κ_0, d ∈ (0, DAMAGE_MAX)"
      expect: "E_tan = -(1-d)·E·α·κ < 0"
      tol_rel: 1.0e-12

    - name: tangent_equals_secant_at_saturation
      setup: "ε enorme tal que d = DAMAGE_MAX"
      expect: "E_tan = (1-DAMAGE_MAX)·E (sin término consistente al saturar)"
      tol_rel: 1.0e-12

    - name: tangent_matches_finite_difference
      setup: "carga activa; ∂σ∂ε/ por FD centrada (h=1e-7) vs fórmula cerrada"
      expect: "error relativo < 1e-5"
      tol_rel: 1.0e-5

    - name: tangent_consistent_with_2D_when_uniaxial
      setup: "evaluar IsotropicDamage1D(E, κ_0, α) en ε=eps y comparar contra
          IsotropicDamage2D(E, ν=0, κ_0, α) plane stress en ε=[eps, 0, 0]; la primera
          componente de σ y la entrada [0,0] de C_alg deben coincidir"
      expect: "σ_1D = σ_xx_2D; E_tan_1D = C_alg_2D[0,0]"
      tol_rel: 1.0e-10

references:
  - "Lemaitre J., Chaboche J.-L. (1990). Mechanics of Solid Materials. Cambridge UP.
     Cap. 7."
  - "Simó J.C., Ju J.W. (1987). Strain- and stress-based continuum damage models - I.
     Int. J. Solids Struct. 23, 821-840."
  - "Spec hermana: [docs/specs/IsotropicDamage2D.md](IsotropicDamage2D.md) - modelo 2D,
     derivación detallada de la tangente consistente."
```

9.4.4 Implementación

- **Archivo**: solidum/materials/damage_1d.py.
- **Clase**: `IsotropicDamage1D`, registrada vía `@MaterialRegistry.register`. `IS_SYMMETRIC` queda `True` (default heredado de `Material`) — la contribución elemental es simétrica aunque pueda ser indefinida.
- **Algoritmo** `compute_state`:

1. Recuperar κ_n y calcular $\varepsilon_{\text{eq}} = |\varepsilon|$.
2. Si $\varepsilon_{\text{eq}} > \kappa_n$: carga activa, $\kappa_{n+1} = \varepsilon_{\text{eq}}$. Si no: descarga, $\kappa_{n+1} = \kappa_n$.
3. Calcular d por la ley exponencial, saturar a `DAMAGE_MAX`.
4. $\sigma = (1 - d)\, E\, \varepsilon$.
5. Tangente: ramas (sin daño / descarga / saturación / carga activa) idénticas en estructura al modelo 2D, con la fórmula escalar resultante. En carga activa con daño efectivo no saturado, `E_tan = -(1-d)·E·`$\alpha\cdot\kappa$.

- **Tests**:
- tests/test_materials_unit.py · `TestIsotropicDamage1D` (existente, no regresión) + nueva clase `TestIsotropicDamage1DConsistentTangent`.
- Sin tests de integración (justificado arriba).

9.4.5 Diálogo

- **2026-05-14** · Spec creada retroactivamente sobre material existente (tangente secante) y extendida con tangente algorítmica consistente. Réplica directa de [[IsotropicDamage2D]] reducida a escalar. Sin ADR — extensión local al material, no afecta a contratos.
- **2026-05-14** · `IS_SYMMETRIC` permanece `True`: la rigidez axial es escalar y la contribución elemental `B^T·E_tan·B` es simétrica aunque pueda ser indefinida cuando `E_tan < 0`. El sistema global puede dejar de ser PD; el fallback automático de Cholesky a LU del despachador algebraico (ADR 0003) lo gestiona sin cambios declarativos.
- **2026-05-14** · No se añade test de integración. Justificación: con tangente consistente <0 en carga activa, un benchmark Truss2D bajo control de carga se vuelve inestable tras el pico; el seguimiento de la rama post-pico requiere arc-length que cae fuera del scope inmediato. Tests unitarios (incluyendo FD numérica y consistencia con la versión 2D reducida) cubren la corrección de la fórmula.
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): declara `STATE_SCHEMA` (obligatorio en todo el catálogo). Sin kernel por lotes: los elementos 1D siguen el camino por elemento (fase 4 de la propuesta, diferida).

Capítulo 10

Modelos Constitutivos — 2D

10.1 DruckerPrager2D

Orden de trabajo. El usuario escribe ***especificación física****,* ***formulación numérica*** *y* ***contrato****; la IA rellena* ***implementación*** *y responde en* ***diálogo****.*

10.1.1 Especificación física

0. Descripción general

Modelo elastoplástico **friccional** independiente de la velocidad. Suelos, hormigón, granulares, materiales cuasi-frágiles cohesivo-friccionales. Suaviza la pirámide hexagonal de Mohr-Coulomb a un **cono circular** en el espacio de esfuerzos principales, eliminando las aristas que complican el return mapping.

Diferencia clave con J2:

- **Dependencia de la presión**. La resistencia al corte aumenta con la presión de confinamiento (componente friccional).
- **Plasticidad no asociada**. La dilatancia volumétrica real del flujo plástico (ψ) es típicamente menor que la fricción del criterio (ϕ). Forzar $\psi = \phi$ (asociada) sobreestima la dilatancia y el volumen post-fluencia.
- **Retorno al ápice**. El criterio define un cono con vértice; estados de prueba en zona puramente hidrostática extrema retornan al vértice (zona singular en la dirección de flujo).

Hipótesis cinemática soportada: `plane_strain` únicamente en esta spec.

1. Descomposición aditiva y elasticidad

Pequeñas deformaciones: $\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e + \boldsymbol{\varepsilon}^p$. Ley elástica isótropa: $\boldsymbol{\sigma} = \mathbf{C}_e(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p)$.

2. Criterio de fluencia (Drucker-Prager)

$$f(\boldsymbol{\sigma}, \alpha) = \sqrt{J_2} + \eta_f \, I_1 - k(\alpha) \;\leq\; 0$$

con:

- $J_2 = \frac{1}{2}\,\mathbf{s} : \mathbf{s}$, segundo invariante del desviador.
- $I_1 = \mathrm{tr}(\boldsymbol{\sigma}) = \sigma_{xx} + \sigma_{yy} + \sigma_{zz}$, primer invariante.
- η_f, coeficiente friccional del cono (adimensional, ≥ 0).
- $k(\alpha) = k_0 + H_k\,\alpha$, radio del cono (el parámetro de cohesión del criterio: k_0 sale de c_0 según la calibración, §3) con endurecimiento isótropo lineal en el multiplicador plástico acumulado α (§5).

La superficie es un cono con eje en la línea hidrostática ($\sigma_{xx} = \sigma_{yy} = \sigma_{zz}$), vértice en $I_1 = k/\eta_f$, abertura controlada por η_f.

3. Calibración con Mohr-Coulomb

Los parámetros (η_f, k_0) se derivan de los parámetros físicos (c_0, ϕ) — cohesión y ángulo de fricción interna — según la variante del cono elegida:

variant	η_f	k_0
`plane_strain_matched` (default)	$\tan(\phi)/\sqrt{9 + 12\tan^2(\phi)}$	$3\,c_0/\sqrt{9 + 12\tan^2(\phi)}$
`outer_cone`	$2\sin(\phi)/[\sqrt{3}\,(3 - \sin(\phi))]$	$6\,c_0\cos(\phi)/[\sqrt{3}\,(3 - \sin(\phi))]$
`inner_cone`	$2\sin(\phi)/[\sqrt{3}\,(3 + \sin(\phi))]$	$6\,c_0\cos(\phi)/[\sqrt{3}\,(3 + \sin(\phi))]$

`plane_strain_matched` es la variante que coincide exactamente con Mohr-Coulomb en condiciones de plane strain (Drucker, 1953). Las otras dos circunscriben (outer) o inscriben (inner) el hexágono de MC en el plano π. Como esta spec solo soporta plane strain, `plane_strain_matched` es el default natural.

4. Regla de flujo (no asociada por defecto)

Potencial plástico g con coeficiente de **dilatancia** η_g (función del ángulo de dilatancia ψ, calibrado con la misma fórmula que η_f pero usando ψ):

$$g(\boldsymbol{\sigma}) = \sqrt{J_2} + \eta_g \, I_1$$

$$\dot{\boldsymbol{\varepsilon}}^p = \dot{\gamma}\,\frac{\partial g}{\partial \boldsymbol{\sigma}} = \dot{\gamma}\,\left(\frac{\mathbf{s}}{2\sqrt{J_2}} + \eta_g\,\mathbf{I}\right)$$

Descomposición:

- **Parte desviadora**: $\dot{\mathbf{e}}^p = \dot{\gamma}\,\mathbf{s}/(2\sqrt{J_2})$ — análoga a J2.
- **Parte volumétrica**: $\mathrm{tr}(\dot{\boldsymbol{\varepsilon}}^p) = 3\dot{\gamma}\,\eta_g$ — **dilatación plástica positiva** si $\eta_g > 0$.

Si $\psi = \phi$ (i.e. $\eta_g = \eta_f$) el modelo se vuelve **asociado**; tangente algorítmica simétrica. Para $\psi < \phi$ (típico en suelos: $\psi \approx \phi/2$ o incluso $\psi = 0$, "non-dilatant"), el modelo es **no asociado** y la tangente algorítmica es **asimétrica**.

5. Endurecimiento isótropo lineal

$$\dot{\alpha} = \dot{\gamma}$$

con α el **multiplicador plástico acumulado** γ (convención del proyecto para Drucker-Prager). La superficie crece manteniendo eje y abertura, solo k aumenta linealmente.

Parámetro del usuario (desde el 2026-09-29): la parametrización de de Souza Neto, Perić y Owen (2008, §8.3). La cohesión se endurece con la deformación plástica equivalente, $c(\bar{\varepsilon}^p) = c_0 + H\,\bar{\varepsilon}^p$, con $\bar{\varepsilon}^p = \xi\,\gamma$ y $k = \xi\,c$, donde $\xi = k_0/c_0$ es el factor de la calibración (§3). El parámetro `H` que escribe el usuario es $H = H_c = dc/d\bar{\varepsilon}^p$. El return mapping trabaja con el módulo del radio del cono respecto del multiplicador acumulado, $H_k = dk/d\alpha$, que es

$$H_k = \xi^2\,H_c.$$

Endurecimiento por fricción/dilatancia (cambio de η_f o η_g con α) queda fuera de esta spec — escenario que no necesitamos hoy y que añade no-linealidad fuerte al return mapping.

6. Condiciones de Kuhn-Tucker

$$\dot{\gamma} \geq 0, \qquad f \leq 0, \qquad \dot{\gamma}\,f = 0$$

7. Variables internas

Símbolo	Tipo	Significado
$\boldsymbol{\varepsilon}^p$	tensor 4 componentes `[xx, yy, zz, xy_tensorial]`	deformación plástica
α	escalar ≥ 0	multiplicador plástico acumulado (= acumulado de $\Delta\gamma$; no es una deformación plástica equivalente, §5)

`alpha` es la variable principal exportable (`PRIMARY_STATE_VAR = 'alpha'`).

10.1.2 Formulación numérica

8. Esquema implícito — Backward Euler

Dado $\{\boldsymbol{\varepsilon}^p_n, \alpha_n\}$ y $\boldsymbol{\varepsilon}_{n+1}$, predictor elástico:

$$\boldsymbol{\varepsilon}^{\text{trial}} = \boldsymbol{\varepsilon}_{n+1} - \boldsymbol{\varepsilon}^p_n, \qquad \mathbf{s}^{\text{trial}} = 2G\,\mathbf{e}^{\text{trial}}_{\text{dev}}, \qquad p^{\text{trial}} = K\ \text{tr}(\boldsymbol{\varepsilon}^{\text{trial}})$$

con $I_1^{\text{trial}} = 3\,p^{\text{trial}}$, $\sqrt{J_2^{\text{trial}}} = \|\mathbf{s}^{\text{trial}}\|/\sqrt{2}$.

Función de fluencia trial:

$$f^{\text{trial}} = \sqrt{J_2^{\text{trial}}} + \eta_f\,I_1^{\text{trial}} - k(\alpha_n)$$

Si $f^{\text{trial}} \leq \text{tol}$ → paso elástico. Si no, intentar **return regular**.

9. Return regular (cone surface)

La dirección desviadora $\hat{\mathbf{n}} = \mathbf{s}^{\text{trial}}/(2\sqrt{J_2^{\text{trial}}}) = \mathbf{s}^{n+1}/(2\sqrt{J_2^{n+1}})$ es invariante bajo carga radial ($\hat{\mathbf{n}} = \partial\sqrt{J_2}/\partial\boldsymbol{\sigma}$, de norma tensorial $1/\sqrt{2}$; es la misma que usa la tangente de §12). Las actualizaciones son lineales en $\Delta\gamma$:

$$\sqrt{J_2^{n+1}} = \sqrt{J_2^{\text{trial}}} - G\,\Delta\gamma$$

$$p^{n+1} = p^{\text{trial}} - 3K\,\eta_g\,\Delta\gamma, \qquad I_1^{n+1} = 3\,p^{n+1}$$

La consistencia $f(\sigma_{n+1}, \alpha_{n+1}) = 0$ con $\alpha_{n+1} = \alpha_n + \Delta\gamma$ y $k_{n+1} = k_0 + H_k(\alpha_n + \Delta\gamma)$ da:

$$\boxed{\Delta\gamma = \frac{f^{\text{trial}}}{G + 9K\,\eta_f\,\eta_g + H_k}}$$

Forma cerrada — sin Newton local. Cada término del denominador:

- G: cambio en $\sqrt{J_2}$ por la parte desviadora del flujo.
- $9K\,\eta_f\,\eta_g$: cambio en $\eta_f I_1$ por la parte volumétrica dilatante del flujo (multiplicado por η_f del criterio y η_g del potencial).
- H_k: endurecimiento del radio del cono, $dk/d\alpha$ (§5).

Actualizaciones:

$$\mathbf{s}^{n+1} = \frac{\sqrt{J_2^{n+1}}}{\sqrt{J_2^{\text{trial}}}}\,\mathbf{s}^{\text{trial}}$$

$$\boldsymbol{\sigma}^{n+1} = \mathbf{s}^{n+1} + p^{n+1}\,\mathbf{I}$$

$$\boldsymbol{\varepsilon}^p_{n+1} = \boldsymbol{\varepsilon}^p_n + \Delta\gamma\left(\frac{\mathbf{s}^{n+1}}{2\sqrt{J_2^{n+1}}} + \eta_g\,\mathbf{I}\right)$$

Condición de validez del return regular: $\sqrt{J_2^{n+1}} \geq 0$. Si $\Delta\gamma_{\text{reg}} > \sqrt{J_2^{\text{trial}}}/G$, el desviador "se pasa de cero" y el algoritmo regular es inválido — toca **return al ápice**.

10. Return al ápice

Cuando el estado de prueba cae en zona puramente hidrostática extrema (p_trial muy traccionante respecto a la cohesión), la proyección al cono cae en su vértice, donde $\mathbf{s} = 0$ y la dirección desviadora $\hat{\mathbf{n}}$ no está definida.

En el ápice: $\boldsymbol{\sigma}^{n+1} = \big(k_{n+1}/(3\eta_f)\big)\,\mathbf{I}$ (puramente hidrostático; $I_1^{n+1} = k_{n+1}/\eta_f$ es el vértice del cono de §2), $\sqrt{J_2^{n+1}} = 0$.

Por conservación volumétrica:

$$p^{n+1} = p^{\text{trial}} - 3K\,\eta_g\,\Delta\gamma = \frac{k_{n+1}}{3\,\eta_f}$$

con $k_{n+1} = k_0 + H_k(\alpha_n + \Delta\gamma)$. Multiplicando por $3\eta_f$:

$$3\eta_f\,p^{\text{trial}} - 9K\,\eta_f\,\eta_g\,\Delta\gamma = k(\alpha_n) + H_k\,\Delta\gamma$$

y, con $I_1^{\text{trial}} = 3\,p^{\text{trial}}$, despejando:

$$\boxed{\Delta\gamma_{\text{apex}} = \frac{I_1^{\text{trial}}\,\eta_f - k(\alpha_n)}{9K\,\eta_f\,\eta_g + H_k}}$$

Es la fórmula regular sin la parte desviadora, porque la rama de apex es puramente volumétrica: el numerador es $f^{\text{trial}} - \sqrt{J_2^{\text{trial}}}$ y el denominador pierde el término G.

Actualizaciones (apex):

$$\mathbf{s}^{n+1} = 0, \qquad p^{n+1} = p^{\text{trial}} - 3K\,\eta_g\,\Delta\gamma_{\text{apex}} \;\equiv\; \frac{k_{n+1}}{3\,\eta_f}, \qquad p^{\text{trial}} = K\,\text{tr}(\boldsymbol{\varepsilon}_{n+1} - \boldsymbol{\varepsilon}_n^p)$$

(el código evalúa directamente $p^{n+1} = k_{n+1}/(3\eta_f)$.)

$$\boldsymbol{\sigma}^{n+1} = p^{n+1}\,\mathbf{I}$$

$$\boldsymbol{\varepsilon}_{n+1}^p = \boldsymbol{\varepsilon}_n^p + \Delta\gamma_{\text{apex}}\,\eta_g\,\mathbf{I} + \mathbf{e}_{\text{dev}}^{\text{trial}}$$

con $\mathbf{e}_{\text{dev}}^{\text{trial}}$ el desviador de la deformación de prueba de §8 (la parte desviadora plástica absorbe completamente el desviador de prueba, ya que $\mathbf{s}^{n+1} = 0$).

11. Detección del régimen de retorno

Algoritmo de despacho:

1. Calcular $\Delta\gamma_{\text{reg}}$ por la fórmula regular.
2. Si $\sqrt{J_2^{\text{trial}}} - G\,\Delta\gamma_{\text{reg}} \geq 0 \rightarrow$ return regular.
3. Si no $\rightarrow$ return al ápice con $\Delta\gamma_{\text{apex}}$.

El criterio es geométricamente: el predictor cae más allá del ápice si la proyección desviadora exigiría reducir $\sqrt{J_2}$ por debajo de cero.

12. Tangente algorítmica consistente

Return regular ($\sqrt{J_2^{n+1}} > 0$). Linealización del algoritmo de §9, forma compacta (de Souza Neto §8.3.4) en Voigt 3 plane strain; es la que implementa el código y la verifica `tests/test_consistent_tangent.py` por diferencias finitas:

$$\mathbf{C}^{\text{alg}} = K\,\mathbf{v}\otimes\mathbf{v} + 2G\,(1-\beta)\,\mathbf{I}_{\text{dev}} + 4G\,\beta\,\hat{\mathbf{n}}\otimes\hat{\mathbf{n}} - \frac{1}{A}\,\mathbf{b}_g\otimes\mathbf{b}_f$$

con:

- $\mathbf{v} = [1, 1, 0]$ (gradiente de I_1 respecto de la deformación en Voigt 3 *engineering*).
- $\mathbf{I}_{\text{dev}}$: bloque en plano 3×3 del proyector desviador, $\begin{bmatrix} 2/3 & -1/3 & 0 \\ -1/3 & 2/3 & 0 \\ 0 & 0 & 1/2 \end{bmatrix}$ (el 1/2 del cortante lleva γ_{xy} a ε_{xy} tensorial).
- $\hat{\mathbf{n}} = [s_{xx}, s_{yy}, s_{xy}]^{\text{trial}}/(2\sqrt{J_2^{\text{trial}}})$, con el cortante tensorial (sin factor 2); coincide con $\mathbf{s}^{n+1}/(2\sqrt{J_2^{n+1}})$ (§9).
- $\beta = G\,\Delta\gamma/\sqrt{J_2^{\text{trial}}}$.
- $\mathbf{b}_f = 2G\,\hat{\mathbf{n}} + 3K\,\eta_f\,\mathbf{v}$, $\mathbf{b}_g = 2G\,\hat{\mathbf{n}} + 3K\,\eta_g\,\mathbf{v}$.
- $A = G + 9K\,\eta_f\,\eta_g + H_k$ (denominador de $\Delta\gamma$, §9).

El término $+4G\beta\,\hat{\mathbf{n}}\otimes\hat{\mathbf{n}}$ recoge el cambio de la dirección de flujo con $\boldsymbol{\varepsilon}$ a través de $\mathbf{s}^{\text{trial}}$ (no sólo el de su magnitud); sin él la tangente cierra mal en las componentes de cortante alineadas con el flujo (lo detecta la diferencia finita). Con $\mathbf{N} = \sqrt{2}\,\hat{\mathbf{n}}$ (normal unitaria del desviador) es $2G\beta\,\mathbf{N}\otimes\mathbf{N}$, y el conjunto es la forma de de Souza Neto et al. (2008, §8.3) con $\eta = 3\eta_f$, $\bar{\eta} = 3\eta_g$ (su criterio usa p, éste $I_1 = 3p$) y $\xi^2 H_c = H_k$. **Asimétrica si** $\eta_f \neq \eta_g$ (no asociada): $\mathbf{b}_g\otimes\mathbf{b}_f$ sólo es simétrico con $\mathbf{b}_g = \mathbf{b}_f$.

Return al ápice ($\sqrt{J_2^{n+1}} = 0$):

$$\mathbf{C}^{\text{alg}} = K\left(1 - \frac{3K\eta_f\eta_g}{3K\eta_f\eta_g + H_k/3}\right)\mathbf{v}\otimes\mathbf{v}$$

— rigidez puramente volumétrica reducida (el material en el ápice se comporta como un fluido con compresibilidad finita; sin rigidez desviadora alguna), con $\mathbf{v} = [1, 1, 0]$ como en el return regular.

Forma equivalente más limpia: $\mathbf{C}^{\text{alg}}_{\text{apex}} = \dfrac{K\,H_k}{9K\eta_f\eta_g + H_k}\,\mathbf{v}\otimes\mathbf{v}$ (con $H_k = 0$, el ápice se vuelve completamente plástico volumétrico y la tangente colapsa).

Piso de la tangente en el ápice. Si $KH_k/(9K\eta_f\eta_g + H_k) < 10^{-6}K$, el código devuelve $10^{-6}K\,\mathbf{v}\otimes\mathbf{v}$ en lugar de la tangente exacta, para no dejar la matriz global singular mientras el punto siga en el ápice. Actúa en **todo retorno al ápice con** $H_k = 0$ (perfectamente plástico, el caso más común, donde la tangente exacta es nula) y, en general, siempre que $H_k/(9K\eta_f\eta_g + H_k) < 10^{-6}$; con $\psi = 0$ el retorno al ápice no tiene solución y el kernel lanza `ValueError` antes de llegar aquí (Diálogo, 2026-09-22). Es una regularización de la tangente, no la tangente consistente: no cambia $\boldsymbol{\sigma}$ ni el estado convergido, sólo la convergencia del Newton global.

13. Tolerancia de fluencia

ADR 0006: `admissibility_scale = k(α) = k_0 + H_k·α` (radio corriente del cono). Tiene unidades de esfuerzo, igual que f.

10.1.3 Contrato de implementación

```
name: DruckerPrager2D
kind: material
status: validated

interface:
  strain_dim: 3
  primary_state_var: alpha
  is_symmetric: false      # tangente asimétrica si no asociada ψ( ≠φ )

admission:                 # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_souza_neto: "§8.3 (Drucker-Prager: retorno al cono y al ápice, tangentes)"
  general_codes:
    - abaqus: "*DRUCKER PRAGER (lineal) en CPE*"
    - code_aster: "DRUCK_PRAGER / DRUCK_PRAG_N_A"
    - opensees: "nDMaterial DruckerPrager"

parameters:
  - { name: E,          type: float, required: true,  desc: "Módulo de Young" }
  - { name: nu,         type: float, required: true,  desc: "Coeficiente de Poisson" }
  - { name: cohesion,   type: float, required: true,  desc: "Cohesión c_0 inicial (
      esfuerzo)" }
  - { name: phi_deg,    type: float, required: true,  desc: "Ángulo de fricción interna
       en grados; típico suelos 20-40°, hormigón 30-37°" }
  - { name: psi_deg,    type: float, required: false, default: null, desc: "Ángulo de
      dilatancia en grados. Default = phi_deg (asociada). Típico no asociada: ψ < φ, e.
      g. 0 o φ/2." }
  - { name: H,          type: float, required: false, default: 0.0, desc: "Módulo de
      endurecimiento isótropo lineal de la cohesión H = dc/εᵖd ≥(0), parametrización de
       de Souza Neto et al. (2008, §8.3); internamente H_k = ξ²·H con ξ = k_0/c_0 (§5).
       H=0 ⇒ perfectamente plástico." }
  - { name: hypothesis, type: str,   required: false, default: "plane_strain", desc: "
      plane_strain (único soportado)" }
  - { name: variant,    type: str,   required: false, default: "plane_strain_matched",
      desc: "plane_strain_matched | outer_cone | inner_cone - calibración con Mohr-
      Coulomb" }
  - { name: density,    type: float, required: false, default: null, desc: "Densidad (
      ADR 0008)" }

signature:
  compute_state: "ε(: ndarray(3,), state_vars=None) -> σ(: ndarray(3,), C_alg: ndarray
      (3,3), state_vars')"
  strain_kind: "plano Voigt -ε [_xx, ε_yy, γ_xy] con γ_xy = 2·ε_xy"

state_schema:
  eps_p:   "ndarray(4,) -ε [^p_xx, ε^p_yy, ε^p_zz, ε^p_xy_tensorial]"
  alpha:   "float ≥ 0 - multiplicador plástico acumulado"
```

```
conventions:
  sign: "σ > 0 ⇔ tracción (Reglas §5)"
  voigt: "γ_xy = 2·ε_xy en deformación; ε^p_xy en tensorial (sin factor 2)"
  associated_flow: "ψ = φ⇒  asociada (tangente simétrica); ψ≠φ⇒    no asociada"

validity:
  - "E > 0, c_0 > 0, 0 ≤φ  < 90°, 0 ≤ψ≤φ   "
  - "ν ∈ (-1, 0.5)"
  - "H ≥ 0"
  - "hypothesis == 'plane_strain' (único soportado en esta spec)"
  - "variant ∈ {'plane_strain_matched', 'outer_cone', 'inner_cone'}"

out_of_scope:
  - "plane_stress - la proyección de Drucker-Prager con σ_zz=0 acoplada a regla de
     flujo dilatante es notoriamente delicada; difiere hasta caso real"
  - "endurecimiento por cambio de φ y/o ψ con α-  no lineal fuerte en el return mapping
     "
  - "endurecimiento cinemático (back-stress hidrostático y desviador)"
  - "cap (modelo cap-cone) para suelos compactables"
  - "ablandamiento (H<0) - requiere regularización"
  - "grandes deformaciones"

numerical_caveats:
  - "Tangente algorítmica asimétrica si ψ≠φ→    IS_SYMMETRIC=False, el despachador
     algebraico usa LU (ADR 0003)."
  - "Modos de carga muy traccionantes pueden caer en la rama de ápice. La detección
     automática del régimen (regular vs apex) está en el algoritmo; si en el futuro
     aparece oscilación entre ramas en pasos cerca de la transición, considerar
     smoothing con vertex regularization."
  - "Con H_k = 0 (perfectamente plástico), en la rama de ápice la tangente volumétrica
     exacta se anula → singularidad. El código la sustituye por el piso K·1e-6·⊗11 en
     todo retorno al ápice con H_k/(9ηηK_f_g + H_k) < 1e-6 (§12): regularización de la
      tangente, sin efecto sobre σ; típicamente sale del ápice en el siguiente paso.
     Con ψ = 0 el retorno al ápice no tiene solución y el kernel lanza ValueError."
  - "El return regular puede generar α creciente con incrementos pequeños cerca de la
     transición ↔regularapex. Newton global converge igual; si en problema concreto se
      detecta oscilación, usar arc-length."

acceptance:
  unit:
    - name: parameter_validation
      expect: "constructor rechaza variant inválida, φ≥  90°, ψ > φ, hypothesis
          distinto de plane_strain"

    - name: calibration_matched_consistency
      expect: "para variant='plane_strain_matched' con ψ = φ, asociada →η _f = η_g; con
           ψ = 0, η_g = 0 (sin dilatación)"

    - name: paso_elastico_below_yield
      expect: "ε pequeño; σ = C_e·ε, eps_p y alpha intactos"
      tol_rel: 1.0e-10

    - name: regular_return_under_shear
      setup: "ε cortante puro suficiente para fluir ε(_xy = γ/2, sin parte hidrostática
          )"
```

```
      expect: "return regular activo, α > 0, √₂J del estado final = kα() -η _f·₁I ≥ 0 (
          f = 0, cumple criterio)"
      tol_rel: 1.0e-8

    - name: apex_return_under_pure_tension
      setup: "ε puramente hidrostática traccionante ε(_xx = ε_yy > 0, sin cortante)"
      expect: "return al ápice; σ_xx = σ_yy = kη/(3_f) tras converger; √₂J = 0"
      tol_rel: 1.0e-8

    - name: associated_tangent_is_symmetric
      setup: "ψ = φ (asociada), estado plástico activo"
      expect: "C_alg simétrica ‖(C_alg - C_alg^‖T < tol)"
      tol_abs: 1.0e-8

    - name: nonassociated_tangent_is_asymmetric
      setup: "ψ = 0, φ = 30°, estado plástico activo no degenerado"
      expect: "C_alg - C_alg^T ≠ 0 con norma significativa"

    - name: tangent_matches_finite_difference
      setup: "estado plástico regular; comparar C_alg cerrada con diferencia finita
          centrada"
      expect: "error relativo < 1e-5"
      tol_rel: 1.0e-5

    - name: alpha_monotonic_under_increasing_load
      expect: "bajo carga creciente α no decrece"

    - name: unloading_recovers_C_e
      setup: "cargar a plástico, descargar elásticamente"
      expect: "tangente = C_e en el paso de descarga, α se conserva"
      tol_rel: 1.0e-10

    - name: unit_invariance_MPa_vs_Pa
      expect: "mismo path adimensional en (MPa,mm,N) y (Pa,m,N) ⇒α  idéntico"
      tol_rel: 1.0e-12

  integration:
    - name: quad4_pipeline_elastico
      setup: "Quad4 unitario plane strain, carga muy por debajo del yield"
      expect: "respuesta elástica pura, α=0"
      tol_rel: 1.0e-8

    - name: quad4_pipeline_plastico_converges
      setup: "Quad4 unitario plane strain, carga que produce plasticidad activa, 4
          pasos, max_iter=8"
      expect: "Newton converge en cada paso, α > 0 al final"

    - name: quad4_yield_onset_confined_tension          # DP-1 (2026-05-19)
      setup: "Quad4 unitario plane strain con desplazamientos prescritos en los 4 nodos
           para ε_xx=k·ε_y, ε_yy=0, γ_xy=0"
      expect: "ε_xx=0.5·ε_y elástico σ(_xx coincide con λμ(+2)·ε exacto, α=0); ε_xx
          =0.99·ε_y aún α=0; ε_xx=1.5·ε_y plástico con α>0 idéntico en los 4 Gauss
          points"
      tol_rel: 1.0e-8

    - name: quad4_flow_rule_invariant                   # DP-2 (2026-05-19)
```

```
      setup: "Quad4 unitario plane strain, carga monótona en rama regular (5 niveles
          1.2·ε_y → 2.6·ε_y), ψ≠φ  "
      expect: "invariante cinemática trε(_p) = 3·η_g·α en cada Gauss point y en cada
          nivel"
      tol_rel: 1.0e-8

    - name: quad4_apex_under_biaxial_tension            # DP-2bis (2026-05-19)
      setup: "Quad4 unitario plane strain, ε_xx = ε_yy ~ 1e-2 (predictor hidrostático
          extremo)"
      expect: "estado final cae en ápice: σ_xx ≈σ _yy = kα()/(3·η_f), σ_xy ≈ 0, en cada
           Gauss point"
      tol_rel: 1.0e-6

references:
  - "Drucker D.C., Prager W. (1952). Soil mechanics and plastic analysis or limit
      design. Quart. Appl. Math. 10, 157-165."
  - "Drucker D.C. (1953). Coulomb friction, plasticity, and limit loads. J. Appl. Mech.
       21, 71-74. (calibración plane_strain_matched)"
  - "Simó J.C., Hughes T.J.R. (1998). Computational Inelasticity. Springer. §6 (cone
      plasticity, apex return)."
  - "de Souza Neto E.A., ćPeri D., Owen D.R.J. (2008). Computational Methods for
      Plasticity. Wiley. §8 (Drucker-Prager, tangentes algorítmicas detalladas)."
  - "Chen W.F., Han D.J. (1988). Plasticity for Structural Engineers. Springer. §7 (
      cohesivo-friccionales)."
```

10.1.4 Implementación

- **Archivo**: solidum/materials/drucker_prager_2d.py.
- **Clase**: `DruckerPrager2D`, registrada vía `@MaterialRegistry.register`. `IS_SYMMETRIC = False` declarativo (la tangente puede ser asimétrica; el despachador agrega el flag sobre todos los materiales del dominio).
- **Kernel Numba** `_compute_drucker_prager_plane_strain`:

1. Predictor elástico desviador-volumétrico (paralelo a J2 plane strain, ε^p_{zz} libre).
2. Check f^{trial}.
3. Si elástico: devolver σ_trial restaurando contribución hidrostática y volumétrica.
4. Si no: intentar $\Delta\gamma_{\text{reg}}$ cerrado; comprobar $\sqrt{J_2^{n+1}} \geq 0$.
5. Si regular válido: actualizar σ, ε^p, α, tangente regular (de Souza Neto §8.3.4).
6. Si regular inválido: rama de ápice — $\Delta\gamma_{\text{apex}}$ cerrado, σ puramente hidrostático, tangente volumétrica reducida.

- **Tangente regular** (§12): `C_alg = K·v⊗v + 2G·(1−β)·I_dev + 4G·β·n̂⊗n̂ − (1/A)·b_g⊗b_f` con `b = 2G·n̂ + 3K·η·v`.
- **Piso de la tangente en el ápice** (§12): `K_apex_eff < K·1e-6` ⇒ `K·1e-6`; actúa en todo retorno al ápice con `H = 0`.

- **Cálculo de** (η_f, k_0, η_g): en `__init__` desde $(c_0, \phi, \psi, \text{variant})$ — sin coste runtime. Se valida $\psi \leq \phi$ (físicamente sensato).
- `admissibility_scale`: $k(\alpha) = k_0 + H_k \cdot \alpha$, $H_k = \xi^2 H$.
- **Tests**:
- tests/test_materials_unit.py · `TestDruckerPrager2D`.
- tests/test_solid_2d_drucker_prager.py.
- tests/test_consistent_tangent.py · `TestDruckerPrager2DConsistentTangent` — tangente regular y de ápice frente a diferencias finitas.

10.1.5 Diálogo

- **2026-05-14** · Spec creada. Decisiones clave del alcance MVP:
- Solo `plane_strain` en esta entrega; `plane_stress` con cono en σ_zz=0 es notoriamente delicado y de uso geotécnico raro — se difiere.
- Parámetros físicos `(c_0, φ, ψ)` en lugar de adimensionales `(k_0, η_f, η_g)` — más natural para el usuario de geotecnia. Calibración interna por `variant`.
- **No asociada por defecto** (ψ es parámetro independiente con default = φ que la fuerza a asociada solo si el usuario no especifica). Plasticidad asociada en suelos sobreestima dilatancia.
- Endurecimiento solo en cohesión (lineal); fricción/dilatancia constantes.
- `IS_SYMMETRIC = False` declarativo a nivel material (independiente del caso particular de uso; conservador y consistente con el patrón del proyecto: el despachador agrega).
- **2026-05-14** · Sin ADR. El modelo encaja en el patrón establecido por `VonMises2D` (kernel Numba, despacho hypothesis aunque solo soporte una, semántica trial/commit) y no rompe contratos transversales.
- **2026-09-22** · Auditoría global: retorno al ápice con $\psi = 0$: sin componente volumétrica del flujo no existe solución consistente con σ = C_e:($\varepsilon - \varepsilon^{\wedge}$p) (con H = 0 se clampaba `A_apex` a 1e-30 y salía $\Delta\gamma \approx$ 1e30; con H > 0 se devolvía el σ del ápice sin la $\varepsilon^{\wedge}$p volumétrica que lo justifica). Ahora el kernel lanza `ValueError` al entrar en la rama del ápice con η_g = 0 y el constructor avisa al construir con $\psi = 0$. Aclarado que el endurecimiento actúa sobre `k(α) = k₀ + H·α` (radio del cono), no sobre la cohesión `c` (dk/dc vale 0.83–1.2 según la calibración). Nuevo `out_of_plane_stress(σ, state)`: σ_zz de plane strain desde $\varepsilon^{\wedge}$p_zz (helper `materials/_plane_strain.py`), validado contra `DruckerPrager3D` con ε_zz = 0.
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): declara `STATE_SCHEMA` y un adaptador `@njit` con firma `MAT_SIG` que llama al mismo return mapping; la tolerancia de admisibilidad (ADR 0006) se evalúa dentro del kernel con las mismas operaciones que `admissibility_tol`. Formulación sin cambios; equivalencia con `compute_state` exacta en esfuerzo, tangente y estado (`tests/test_batch_kernels.py`).

- **2026-09-22** · Cierre del ADR 0014 (§9): `batch_out_of_plane_stress(sigma, S)`, versión por arreglos de `out_of_plane_stress` para el post-proceso por familia (exportador VTK); misma fórmula, ε`^p_zz` leída de la fila de estado cuando aplica. Verificado archivo VTK idéntico por ambos caminos.

- **2026-09-29** · Revisión documental (sin cambio de código ni de resultados): (1) α es el multiplicador plástico acumulado γ, no una deformación plástica equivalente, y H endurece el radio del cono k respecto de γ; conversión desde la parametrización $c(\bar{\varepsilon}^p)$ de de Souza Neto et al. (2008), $H_k = \xi^2 H_c$ con $\xi = k_0/c_0$ (§5). Sustituye a la cota «dk/dc vale 0.83–1.2» de la entrada anterior, que sólo vale para $\phi = 30°$. (2) El piso $K \cdot 10^{-6}$ de la tangente en el ápice actúa en todo retorno al ápice con $H_k = 0$, no sólo con $\psi = 0$, caso que desde el 2026-09-22 lanza `ValueError` (§12 y `numerical_caveats`).

- **2026-09-29** · Erratas de la spec frente al código, que es correcto (tangente regular y de ápice verificadas por diferencias finitas en `tests/test_consistent_tangent.py`; ápice contra `k/(3`η`_f)` en `tests/test_materials_unit.py`). Sólo cambia el texto; ni el código ni la formulación implementada cambian. (a) §10: el esfuerzo en el ápice es $\boldsymbol{\sigma}^{n+1} = (k_{n+1}/(3\eta_f))\,\mathbf{I}$, no $(k_{n+1}/\eta_f)\,\mathbf{I}$; es la presión $p = k/(3\eta_f)$ de la línea siguiente, del código (`p_new = k_new/(3`η`_f)`) y de la spec 3D. (b) §10: la primera expresión del $\Delta\gamma_{\text{apex}}$ encuadrado, con numerador $\eta_f(p^{\text{trial}} - k)$ y denominador $3K\eta_f\eta_g + H_k$, no era igual a la final; queda sólo la correcta, $(I_1^{\text{trial}}\eta_f - k)/(9K\eta_f\eta_g + H_k)$, con el paso intermedio del despeje. (c) §12: la forma compacta de la tangente regular usaba $\mathbf{b} = G\hat{\mathbf{n}} + 3K\eta\,\mathbf{1}$ con $\hat{\mathbf{n}} = \mathbf{s}/(2\sqrt{J_2})$ y omitía el término $+4G\beta\,\hat{\mathbf{n}}\otimes\hat{\mathbf{n}}$; el código usa $2G\hat{\mathbf{n}}$ y lleva ese término, como documenta la spec `DruckerPrager3D` §14. Se retira además la forma «por linealización» previa, con términos Θ, $\mathbf{a}_f$, $\mathbf{a}_g$ y «corrección» sin definir. Erratas menores encontradas al hacerlo, en el mismo sentido: la actualización del ápice escribía $p^{n+1} = K(\operatorname{tr}\boldsymbol{\varepsilon}_{n+1} - 3\eta_g\Delta\gamma)$ sin restar $\boldsymbol{\varepsilon}_n^p$ (ahora $p^{\text{trial}} - 3K\eta_g\Delta\gamma$ con p^{trial} de §8); el desviador que absorbe $\boldsymbol{\varepsilon}^p$ en el ápice es $\mathbf{e}_{\text{dev}}^{\text{trial}}$ de §8; §9 definía $\hat{\mathbf{n}} = \mathbf{s}/\sqrt{2J_2}$ (normal unitaria) y §12 $\mathbf{s}/(2\sqrt{J_2})$, ahora ambas la segunda, como el código; las tangentes de ápice usan $\mathbf{v} = [1, 1, 0]$ en Voigt 3, y el `expect` de `regular_return_under_shear` decía $\sqrt{J_2} = \sqrt{2/3}\,k - \eta_f I_1$, cuando el criterio de §2 da $\sqrt{J_2} = k - \eta_f I_1$ (el test comprueba $\alpha > 0$ y el cortante, no ese valor).

- **2026-09-29** · Cambio del significado del parámetro `H`, aprobado por el usuario (hallazgo clas-6 de la auditoría): pasa a ser $H_c = dc/d\bar{\varepsilon}^p$, la parametrización de de Souza Neto et al. (2008, §8.3), y el material calcula $H_k = \xi^2 H_c$ con $\xi = k_0/c_0$. Antes `H` era directamente H_k: un H_c tomado de un ensayo o de la literatura daba un endurecimiento distinto por el factor ξ^2, sin aviso. Con endurecimiento lineal la física es la misma; cambia sólo el número que se escribe. El return mapping, la tangente y los resultados con `H = 0` no cambian. Blindado en `tests/test_drucker_prager_hardening.py` ($k(\gamma) = \xi\, c(\xi\gamma)$ en las tres calibraciones y estado final sobre la superficie endurecida con H_k); los tests con `H > 0` pasan a usar H_k en sus expresiones analíticas.

10.2 Elastic2D

*Spec **retroactiva**: el material existe desde el origen del subsistema 2D y está validado por tests preexistentes (todos los pipelines sólido 2D elástico). Esta spec lo documenta sin cambiar comportamiento — H-5.3 de la auditoría 2026-05-18.*

10.2.1 Especificación física

0. Descripción general

Modelo elástico lineal isótropo bidimensional, parametrizado por (E, ν). Soporta dos hipótesis cinemáticas 2D mutuamente excluyentes seleccionadas en construcción:

- `plane_stress` (default) — $\sigma_{zz} = \sigma_{xz} = \sigma_{yz} = 0$. Láminas o membranas delgadas en su plano (chapa traccionada, placa fina).
- `plane_strain` — $\varepsilon_{zz} = \varepsilon_{xz} = \varepsilon_{yz} = 0$. Cuerpos prismáticos largos cargados transversalmente (presa, túnel, cilindro largo).

Independiente de la velocidad, sin variables históricas. Es el material elástico canónico para todos los sólidos 2D del catálogo.

1. Descomposición de la deformación

No aplica — toda la deformación es elástica:

$$\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e$$

2. Ley elástica

$$\boldsymbol{\sigma} = \mathbf{C}_e\, \boldsymbol{\varepsilon}$$

con $\boldsymbol{\varepsilon} = [\varepsilon_{xx},\, \varepsilon_{yy},\, \gamma_{xy}]^\top$ en notación Voigt del proyecto ($\gamma_{xy} = 2\varepsilon_{xy}$) y $\boldsymbol{\sigma} = [\sigma_{xx},\, \sigma_{yy},\, \sigma_{xy}]^\top$.
Matriz constitutiva plane stress (3×3):

$$\mathbf{C}_e^{ps} = \frac{E}{1-\nu^2}\begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1-\nu)/2 \end{bmatrix}$$

Matriz constitutiva plane strain (3×3):

$$\mathbf{C}_e^{pe} = \frac{E}{(1+\nu)(1-2\nu)}\begin{bmatrix} 1-\nu & \nu & 0 \\ \nu & 1-\nu & 0 \\ 0 & 0 & (1-2\nu)/2 \end{bmatrix}$$

Tangente constante $\mathbf{C}_e$ en todo régimen. Simétrica y positiva definida en el rango admisible de ν.

3. Superficie de fluencia / criterio de daño

No aplica — el material es indefinidamente elástico.

4-6. Flujo, endurecimiento, Kuhn-Tucker

No aplican.

7. Variables internas

Ninguna — `state_vars = None` se acepta y se devuelve intacto. Sin `PRIMARY_STATE_VAR`.

8. Notación Voigt

Convención del proyecto: $\boldsymbol{\varepsilon} = [\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$ con $\gamma_{xy} = 2\varepsilon_{xy}$ (deformación angular *engineering*). Los esfuerzos son los componentes tensoriales reales $[\sigma_{xx}, \sigma_{yy}, \sigma_{xy}]$.

10.2.2 Formulación numérica

9. Esquema temporal

No aplica — sin historia.

10. Return mapping / actualización

No aplica. Evaluación reducida a producto matriz-vector:

$$\boldsymbol{\sigma}_{n+1} = \mathbf{C}_e\, \boldsymbol{\varepsilon}_{n+1}, \qquad \mathbf{C}^{\text{alg}} = \mathbf{C}_e$$

La matriz $\mathbf{C}_e$ se precalcula y guarda en el constructor — coste de `compute_state` es un producto $3{\times}3 \cdot 3$ sin alocación adicional.

11. Tangente algorítmica consistente

$$\mathbf{C}^{\text{alg}} = \mathbf{C}_e$$

Simétrica (`IS_SYMMETRIC = True` por default). En plane stress la matriz es PD para $\nu \in (-1, 0.5)$; en plane strain también, con singularidad en $\nu \to 0.5$ (incompresible — requiere formulación mixta, no soportada).

12. Caveats numéricos

- **Plane strain** $\nu \to 0.5$: la matriz constitutiva diverge ($1 - 2\nu \to 0$). Recomendado $\nu \leq 0.49$ para metales (típico $\nu = 0.3$). El constructor rechaza $\nu \geq 0.5$ con `ValueError`.
- **Locking volumétrico** en plane strain con elementos de bajo orden (Quad4, Tri3) cuando $\nu \to 0.5$: declarado limitación arquitectural, sin mitigación implementada (B-bar, F-bar diferidas).
- Para $\nu < 0$ el material es admisible termodinámicamente (auxético) pero infrecuente en la práctica; el constructor lo permite hasta $\nu > -1$.

10.2.3 Contrato de implementación

```
name: Elastic2D
kind: material
status: validated

interface:
  strain_dim: 3
  primary_state_var: null
  is_symmetric: true

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 2 (elastostática: esfuerzo plano y deformación plana)"
    - zienkiewicz_taylor: "vol. 1: elasticidad plana"
  general_codes:
    - abaqus: "*ELASTIC en CPS* / CPE*"
    - opensees: "nDMaterial ElasticIsotropic"

parameters:
  - { name: E,          type: float, required: true,  desc: "Módulo de Young (>0)" }
  - { name: nu,         type: float, required: true,  desc: "Coeficiente de Poisson ∈
      (-1, 0.5)" }
  - { name: hypothesis, type: str,   required: false, default: "plane_stress",
      desc: "plane_stress | plane_strain" }
  - { name: density,    type: float, required: false, default: null,
      desc: "Densidad (kg/m³). Opcional al construir; obligatoria solo si se ensambla
          peso propio o masa (ADR 0008)" }

signature:
  compute_state: "ε(: ndarray(3,), state_vars=None) -> σ(: ndarray(3,), C_e: ndarray
      (3,3), state_vars)"
  strain_kind: "plano Voigt -ε [ε_xx, ε_yy, γ_xy] con γ_xy = 2·ε_xy"

state_variables: []

conventions:
  sign: "σ > 0 ⇔ tracción (Reglas §5)"
  voigt: "γ_xy = 2·ε_xy en deformación; σ_xy tensorial sin factor"
  hypothesis_dispatch: "selección por kwarg al construir; matriz C precalculada"

validity:
  - "E > 0"
  - "ν ∈ (-1, 0.5) estricto; ν = 0.5 rechazado (incompresible)"
  - "hypothesis ∈ {'plane_stress', 'plane_strain'}"
  - "pequeñas deformaciones ε(|| ≲ 1e-2)"

out_of_scope:
  - "anisotropía (ortótropo, monoclínico)"
  - "plasticidad ⇒ VonMises2D, DruckerPrager2D"
  - "daño ⇒ IsotropicDamage2D"
  - "incompresibilidad estricta ν(=0.5) ⇒ requiere formulación mixta u-p"
  - "grandes deformaciones"

acceptance:
  # Cubierto por tests existentes de elementos sólidos 2D que usan
```

```
  # Elastic2D como material por default.
  verification:
    - name: ley_de_hooke_3x3_exacta
      setup: "evaluar compute_state con ε arbitrario; comparar σ = C_e·ε en ambas hipó
          tesis"
      expect: "σ = C_e·ε exacto componente a componente; tangente = C_e"
      tol_rel: 1.0e-14
    - name: simetria_C_e
      setup: "construir C_e en ambas hipótesis"
      expect: "C_e == C_e.T exacto"
      tol_abs: 1.0e-14
    - name: positividad_definida
      setup: "autovalores de C_e en ambas hipótesis con νϵ  {0, 0.3, 0.49}"
      expect: "todos los autovalores estrictamente positivos"
      tol_abs: 0.0
    - name: plane_stress_traccion_uniaxial
      setup: "ε = ε(_0, ν-·ε_0, 0); plane_stress"
      expect: "σ_xx = E·ε_0, σ_yy = 0, σ_xy = 0 (uniaxial puro emergente)"
      tol_rel: 1.0e-12
    - name: plane_strain_confinamiento
      setup: "ε = ε(_0, 0, 0); plane_strain"
      expect: "σ_yy = νν/(1-) ·σ _xx (reacción de confinamiento clásica)"
      tol_rel: 1.0e-12
    - name: rechazo_inputs_invalidos
      setup: "construir Elastic2D con ≤E0, ν fuera de rango, hypothesis desconocida,
          density<0"
      expect: "ValueError con mensaje claro en cada caso"

references:
  - "Timoshenko S., Goodier J.N. (1970). Theory of Elasticity. McGraw-Hill. §3-4 (plane
       stress / plane strain)."
  - "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and Applications
      of Finite Element Analysis. Wiley. §3.6."
  - "Bathe K.J. (2014). Finite Element Procedures. Prentice Hall. §4.2."
```

10.2.4 Implementación

- **Archivo**: solidum/materials/elastic_2d.py.
- **Clase**: `Elastic2D`, registrada vía `@MaterialRegistry.register`.
- **Despacho por hipótesis**: la matriz `C` se precalcula en el constructor según `hypothesis`. `compute_state` solo hace `self.C @ strain` sin ramificaciones — coste mínimo.
- **Validación temprana** en constructor: `E > 0`, $\nu \in$ `(-1, 0.5)`, `hypothesis` $\in$ {`plane_stress`, `plane_strain`}, `density` $\geq$ `0` si se pasa. Mensajes en español.
- **Tests**: cobertura implícita amplísima vía todos los pipelines de sólidos 2D (Quad4, Quad8, Quad9, Tri3, Tri6) que usan Elastic2D como material por default (`tests/test_solid_2d_*.py`, decenas de tests).

10.2.5 Diálogo

- **2026-05-19** · Spec creada retroactivamente para cerrar el hueco H-5.3 de la auditoría 2026-05-18. Material anterior a la convención de specs validadas; no se modifica código. Aceptación cumplida por cobertura indirecta en pipelines existentes.
- **2026-09-22** · Auditoría global: nuevo `out_of_plane_stress(σ, state)`: σ_zz = ν(σ_xx + σ_yy) en plane strain, 0 en plane stress. Lo consume el exportador VTK para el Von Mises 3D (antes usaba la fórmula de plane stress también en plane strain).
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): `STATE_SCHEMA = {}` y kernel lineal compartido (`materials/_batch.linear_material_kernel`, σ = C·ε); `batch_matrix()` devuelve la `C` precalculada.
- **2026-09-22** · Cierre del ADR 0014 (§9): `batch_out_of_plane_stress(sigma, S)`, versión por arreglos de `out_of_plane_stress` para el post-proceso por familia (exportador VTK); misma fórmula, `ε^p_zz` leída de la fila de estado cuando aplica. Verificado archivo VTK idéntico por ambos caminos.

10.3 IsotropicDamage2D

*Orden de trabajo. El usuario escribe **especificación física**, **formulación** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

*Spec **retroactiva con ampliación**: el modelo está implementado desde sesiones anteriores con **tangente secante**. Esta spec lo documenta y añade la **tangente algorítmica consistente** para recuperar convergencia cuadrática del Newton global. Desde el 2026-09-29 la **deformación equivalente es configurable** entre tres medidas de la literatura (ver §3 y Diálogo).*

10.3.1 Especificación física

0. Descripción general

Modelo de **daño isótropo escalar** en pequeñas deformaciones para sólidos 2D. Captura degradación progresiva e isótropa de la rigidez en respuesta a la deformación máxima histórica, con ablandamiento exponencial. Pensado para hormigón, cerámicos y materiales cuasi frágiles donde la microfisuración distribuida puede modelarse mediante un escalar de daño en lugar de microestructura explícita. Con las medidas de Mazars o de von Mises modificado (§3) el inicio del daño distingue tracción de compresión.

1. Cinemática

Pequeñas deformaciones, descomposición elástica pura: la deformación plástica no existe en este modelo (sin plasticidad). El daño se manifiesta como degradación de la rigidez efectiva.

2. Esfuerzo nominal vs esfuerzo efectivo

$$\boldsymbol{\sigma} = (1 - d)\,\boldsymbol{\sigma}^{\text{eff}}, \qquad \boldsymbol{\sigma}^{\text{eff}} = \mathbf{C}_e\,\boldsymbol{\varepsilon}$$

con $d \in [0, d_{\max}]$ la variable de daño isótropa y $\mathbf{C}_e$ el tensor constitutivo elástico isótropo (2D, plane stress o plane strain según `hypothesis` heredada del `Elastic2D` interno).

El esfuerzo efectivo $\boldsymbol{\sigma}^{\text{eff}}$ es el que el material intacto soportaría con la deformación corriente. El esfuerzo nominal $\boldsymbol{\sigma}$ es lo que efectivamente transmite considerando la degradación.

3. Deformación equivalente

Cantidad escalar que cuantifica la "intensidad" de la deformación y gobierna el daño. Se elige con el parámetro `equivalent_strain` entre tres medidas estándar. Todas se evalúan sobre el **estado 3D asociado** al estado plano, en notación Voigt $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$ con $\gamma_{xy} = 2\varepsilon_{xy}$:

$$\boldsymbol{\varepsilon}^{\text{3D}} = [\varepsilon_{xx},\ \varepsilon_{yy},\ \varepsilon_{zz},\ \gamma_{xy},\ 0,\ 0], \qquad \varepsilon_{zz} = c_{zz}\,(\varepsilon_{xx} + \varepsilon_{yy}), \quad c_{zz} = \begin{cases} -\dfrac{\nu}{1-\nu} & \text{tensión plana} \\ 0 & \text{deformación plana} \end{cases}$$

En tensión plana ε_{zz} no depende del daño: con daño escalar isótropo, $\sigma_{zz} = (1-d)\,(\lambda\,\mathrm{tr}\,\boldsymbol{\varepsilon} + 2\mu\,\varepsilon_{zz}) = 0$ para cualquier d. La dirección z es principal ($\gamma_{xz} = \gamma_{yz} = 0$). Con cualquiera de las tres medidas el 2D es la **restricción exacta** de `IsotropicDamage3D` en las dos hipótesis.

Las tres medidas están **calibradas en tracción uniaxial de esfuerzo**: $\varepsilon_{\text{eq}} = \varepsilon$, de modo que el daño empieza en $\sigma = E\,\kappa_0$ y $\kappa_0 = f_t/E$. Se diferencian en compresión y en estados multiaxiales.

1. **Energética** (`energy`, por defecto; Simó y Ju 1987):

$$\varepsilon_{\text{eq}} = \sqrt{\frac{\boldsymbol{\varepsilon}^\top \mathbf{C}_e\, \boldsymbol{\varepsilon}}{E}}$$

con $\mathbf{C}_e$ la matriz elástica Voigt del propio material (la de tensión plana o la de deformación plana). Con la de tensión plana, $\boldsymbol{\varepsilon}^\top \mathbf{C}_{\text{ps}} \boldsymbol{\varepsilon} = \boldsymbol{\sigma} : \boldsymbol{\varepsilon}$ del 3D con $\sigma_{zz} = 0$; en deformación plana el término $\sigma_{zz}\varepsilon_{zz}$ es nulo. Por eso esta medida no necesita tratar ε_{zz} aparte. Simétrica en tracción y compresión (en compresión uniaxial de esfuerzo, $\varepsilon_{\text{eq}} = |\varepsilon|$).

1. **Mazars** (`mazars`; Mazars 1986):

$$\varepsilon_{\text{eq}} = \sqrt{\textstyle\sum_{i=1}^{3} \langle \varepsilon_i \rangle_+^2}$$

con ε_i las deformaciones principales del tensor 3D asociado: las dos del plano y ε_{zz}. Sólo las extensiones dañan: en compresión uniaxial de esfuerzo daña la extensión lateral, $\varepsilon_{\text{eq}} = \sqrt{2}\,\nu\,|\varepsilon|$.

1. **von Mises modificado** (`modified_von_mises`; de Vree, Brekelmans y van Gils 1995), con $k = f_c/f_t$:

$$\varepsilon_{\text{eq}} = \frac{k-1}{2k(1-2\nu)}\, I_1 + \frac{1}{2k}\sqrt{\left(\frac{k-1}{1-2\nu}\right)^2 I_1^2 + \frac{12k}{(1+\nu)^2}\, J_2}$$

con $I_1 = \varepsilon_{xx} + \varepsilon_{yy} + \varepsilon_{zz}$ y $J_2 = \frac{1}{6}\left[(\varepsilon_{xx} - \varepsilon_{yy})^2 + (\varepsilon_{yy} - \varepsilon_{zz})^2 + (\varepsilon_{zz} - \varepsilon_{xx})^2\right] + \frac{1}{4}\gamma_{xy}^2$. En compresión uniaxial de esfuerzo, $\varepsilon_{\text{eq}} = |\varepsilon|/k$.

Hasta el 2026-09-29 la medida era la norma del tensor, $\varepsilon_{\text{eq}} = \sqrt{\boldsymbol{\varepsilon} : \boldsymbol{\varepsilon}}$: convención propia, no estándar y mal calibrada (en tracción uniaxial de esfuerzo daba $\varepsilon_{\text{eq}} = \varepsilon\sqrt{1+2\nu^2}$, así que $\kappa_0 \neq f_t/E$). Se retiró (ver *Diálogo*).

4. Variable histórica y condición de carga (Kuhn-Tucker)

$$\kappa_{n+1} = \max(\kappa_n, \varepsilon_{\text{eq}}(\boldsymbol{\varepsilon}_{n+1})), \qquad \kappa_0 \text{ inicial}$$

Esto codifica:

- **Carga activa** ($\dot\kappa > 0$): κ crece con ε_{eq}; el daño puede aumentar.
- **Descarga / recarga** ($\dot\kappa = 0$): κ queda congelado; el daño no cambia.
- **Irreversibilidad**: κ es monótono no decreciente.

5. Ley de evolución del daño

$$d(\kappa) = \begin{cases} 0 & \kappa \le \kappa_0 \\ 1 - \dfrac{\kappa_0}{\kappa}\, e^{-\alpha(\kappa-\kappa_0)} & \kappa > \kappa_0 \end{cases}$$

con saturación numérica $d \le d_{\max} =$ `DAMAGE_MAX` (`solidum.constants.DAMAGE_MAX`, evita rigidez nula).

Propiedades:

- $d(\kappa_0) = 0$ (continuidad).
- $d'(\kappa) > 0$ para $\kappa > \kappa_0$ (irreversibilidad).
- $\lim_{\kappa\to\infty} d = 1$ (asintótico).
- α controla la velocidad de degradación (ablandamiento más abrupto si α grande).

10.3.2 Formulación numérica

6. Algoritmo en un paso

Dado $\{\kappa_n\}$ y $\boldsymbol{\varepsilon}_{n+1}$:

1. $\boldsymbol{\sigma}^{\text{eff}} = \mathbf{C}_e\, \boldsymbol{\varepsilon}_{n+1}$ y ε_{eq} con la medida elegida (§3), junto con su derivada $\partial\varepsilon_{\text{eq}}/\partial\boldsymbol{\varepsilon}$.
2. Si $\varepsilon_{\text{eq}} > \kappa_n$: **carga activa**, $\kappa_{n+1} = \varepsilon_{\text{eq}}$. Si no: **descarga**, $\kappa_{n+1} = \kappa_n$.
3. Aplicar ley de daño con κ_{n+1}; saturar a $d_{\max}$.
4. $\boldsymbol{\sigma}_{n+1} = (1-d)\, \mathbf{C}_e\, \boldsymbol{\varepsilon}_{n+1}$.
5. Tangente: **secante** en descarga / sin daño / saturación; **algorítmica consistente** en carga activa con daño no saturado (ver §7).

El algoritmo es **explícito**: no requiere Newton local. La actualización de κ y d es directa. Las deformaciones principales de Mazars salen en forma cerrada: las dos del plano del bloque 2×2 y la tercera es ε_{zz}.

7. Tangente algorítmica consistente (ampliación)

$$\mathbf{C}^{\text{alg}} = \frac{\partial \boldsymbol{\sigma}_{n+1}}{\partial \boldsymbol{\varepsilon}_{n+1}}$$

Derivando $\boldsymbol{\sigma} = (1-d)\, \mathbf{C}_e\, \boldsymbol{\varepsilon}$:

$$\mathbf{C}^{\text{alg}} = (1-d)\, \mathbf{C}_e - \boldsymbol{\sigma}^{\text{eff}} \otimes \frac{\partial d}{\partial \boldsymbol{\varepsilon}}, \qquad \frac{\partial d}{\partial \boldsymbol{\varepsilon}} = \frac{\partial d}{\partial \kappa}\, \frac{\partial \kappa}{\partial \boldsymbol{\varepsilon}}$$

donde:

- $\dfrac{\partial d}{\partial \kappa} = (1-d)\left(\dfrac{1}{\kappa}+\alpha\right)$ para la ley exponencial. (Demostración: derivando $d = 1-(\kappa_0/\kappa)e^{-\alpha(\kappa-\kappa_0)}$ y reagrupando vía $1-d=(\kappa_0/\kappa)e^{-\alpha(\kappa-\kappa_0)}$.)
- $\dfrac{\partial \kappa}{\partial \boldsymbol{\varepsilon}} = \dfrac{\partial \varepsilon_{\text{eq}}}{\partial \boldsymbol{\varepsilon}}$ en **carga activa** y **0** en **descarga**.

Derivadas de la deformación equivalente respecto de la deformación en Voigt *engineering*. En las medidas que usan el estado 3D asociado, la regla de la cadena por $\varepsilon_{zz} = c_{zz}(\varepsilon_{xx}+\varepsilon_{yy})$ añade $c_{zz}\,\partial\varepsilon_{\text{eq}}/\partial\varepsilon_{zz}$ a las componentes xx e yy:

- **Energética**: $\dfrac{\partial \varepsilon_{\text{eq}}}{\partial \boldsymbol{\varepsilon}} = \dfrac{\mathbf{C}_e\,\boldsymbol{\varepsilon}}{E\,\varepsilon_{\text{eq}}}$ (la cadena de ε_{zz} ya está en $\mathbf{C}_{\text{ps}}$).
- **Mazars**: $\dfrac{\partial \varepsilon_{\text{eq}}}{\partial \boldsymbol{\varepsilon}} = \dfrac{1}{\varepsilon_{\text{eq}}}\sum_i \langle\varepsilon_i\rangle_+ \dfrac{\partial \varepsilon_i}{\partial \boldsymbol{\varepsilon}}$, con $\partial\varepsilon_i/\partial\boldsymbol{\varepsilon}$ el proyector $\mathbf{n}_i\otimes\mathbf{n}_i$ en Voigt *engineering* (componente de γ_{ab}: $n_a n_b$). La suma es la parte positiva del tensor, $\boldsymbol{\varepsilon}_+ = \sum_i \langle\varepsilon_i\rangle_+ \mathbf{n}_i\otimes\mathbf{n}_i$: en 2D, $[(\varepsilon_+)_{xx}+c_{zz}\langle\varepsilon_{zz}\rangle_+,\ (\varepsilon_+)_{yy}+c_{zz}\langle\varepsilon_{zz}\rangle_+,\ (\varepsilon_+)_{xy}]/\varepsilon_{\text{eq}}$. Está bien definida con deformaciones principales repetidas, porque $\sum_i\langle\lambda_i\rangle_+^2$ es una función simétrica C^1 de los valores principales.
- **von Mises modificado**: con $S=\sqrt{b^2 I_1^2 + c\,J_2}$, $b=(k-1)/(1-2\nu)$, $c=12k/(1+\nu)^2$:

$$\frac{\partial \varepsilon_{\text{eq}}}{\partial \boldsymbol{\varepsilon}^{3\text{D}}} = \frac{b+b^2 I_1/S}{2k}\frac{\partial I_1}{\partial \boldsymbol{\varepsilon}^{3\text{D}}} + \frac{c}{4kS}\frac{\partial J_2}{\partial \boldsymbol{\varepsilon}^{3\text{D}}}, \qquad \frac{\partial I_1}{\partial \boldsymbol{\varepsilon}^{3\text{D}}} = [1,1,1,0,0,0], \quad \frac{\partial J_2}{\partial \boldsymbol{\varepsilon}^{3\text{D}}} = [e_{xx}, e_{yy}, e_{zz}, \tfrac{1}{2}\gamma_{xy}, \tfrac{1}{2}\gamma_{yz}, \tfrac{1}{2}\gamma_{xz}]$$

(e la parte desviadora) y la cadena de ε_{zz} para pasar al 2D.

Tangente final:

$$\mathbf{C}^{\text{alg}} = \begin{cases} \mathbf{C}_e & \kappa \le \kappa_0 \text{ (sin daño)} \\ (1-d)\,\mathbf{C}_e & \text{descarga} \\ (1-d)\,\mathbf{C}_e \;-\; (1-d)\left(\dfrac{1}{\kappa}+\alpha\right)(\mathbf{C}_e\boldsymbol{\varepsilon})\otimes\dfrac{\partial \varepsilon_{\text{eq}}}{\partial \boldsymbol{\varepsilon}} & \text{carga activa, } d<d_{\max} \\ (1-d_{\max})\,\mathbf{C}_e & d=d_{\max} \text{ (saturación)} \end{cases}$$

8. Simetría de la tangente

- **Energética**: $\partial\varepsilon_{\text{eq}}/\partial\boldsymbol{\varepsilon} \parallel \mathbf{C}_e\boldsymbol{\varepsilon}$, así que el término de carga activa es $\propto (\mathbf{C}_e\boldsymbol{\varepsilon})\otimes(\mathbf{C}_e\boldsymbol{\varepsilon})$: la tangente consistente es **simétrica** (en la implementación, bit a bit).
- **Mazars y von Mises modificado**: $\mathbf{C}_e\boldsymbol{\varepsilon}$ y $\partial\varepsilon_{\text{eq}}/\partial\boldsymbol{\varepsilon}$ no son proporcionales en general: tangente **no simétrica**.

Consecuencia algebraica (ADR 0003): `IS_SYMMETRIC` es `False` en la clase (cota conservadora) y la **instancia** lo pone a `True` con la medida energética; `domain_is_symmetric` lo lee de la instancia (mismo patrón que `DruckerPrager2D` asociado). Con Mazars o von Mises modificado el despachador elige LU (o Pardiso). Con la energética puede elegir Cholesky si está disponible; como la tangente de ablandamiento puede no ser definida positiva, el corrector degrada a LU en cuanto Cholesky la rechaza.

9. Decisión de régimen durante el Newton

Durante una iteración del Newton global, el material recibe un $\boldsymbol{\varepsilon}$ trial y debe decidir carga/descarga comparando con κ_n (el estado committed del paso anterior). La transición entre ramas (loading $\leftrightarrow$ unloading) es **discontinua en la tangente**: al cruzar $\varepsilon_{\text{eq}} = \kappa_n$ el segundo término aparece/desaparece bruscamente. En la práctica esto no genera problemas porque el Newton converge antes de oscilar entre ramas; en problemas patológicos puede requerir continuation o arc-length.

10.3.3 Contrato de implementación

```yaml
name: IsotropicDamage2D
kind: material
status: validated

interface:
  strain_dim: 3
  primary_state_var: damage
  is_symmetric: false      # cota de clase; la instancia es true con equivalent_strain
      ='energy' (ver §8)

admission:                 # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_borst_crisfield: "§6.2 (daño isótropo escalar con ablandamiento exponencial y
        medidas de deformación equivalente)"
    - de_souza_neto: "cap. 12 (mecánica del daño)"
  general_codes:
    - code_aster: "MAZARS, ENDO_ISOT_BETON (daño escalar isótropo; MAZARS con la
        deformación equivalente de Mazars)"

parameters:
  - { name: E,          type: float, required: true,  desc: "Módulo de Young intacto" }
  - { name: nu,         type: float, required: true,  desc: "Coeficiente de Poisson" }
  - { name: kappa_0,    type: float, required: true,  desc: "Umbral de deformación
      equivalente; daño nulo mientras κ≤κ_0. Con las tres medidas κ_0 = f_t/E" }
  - { name: alpha,      type: float, required: true,  desc: "Velocidad de degradación
      exponencial (>0)" }
  - { name: hypothesis, type: str,   required: false, default: "plane_stress", desc: "
      plane_stress | plane_strain (delegado a Elastic2D interno)" }
  - { name: equivalent_strain, type: str, required: false, default: "energy", desc: "
      energy (Simó-Ju 1987) | mazars (Mazars 1986) | modified_von_mises (de Vree et al.
       1995)" }
  - { name: k,          type: float, required: false, default: null, desc: "f_c/f_t ≥(
      1); obligatorio con modified_von_mises y no admitido con las otras medidas" }
  - { name: density,    type: float, required: false, default: null, desc: "Densidad (
      ADR 0008)" }

signature:
  compute_state: "ε(: ndarray(3,), state_vars=None) -> σ(: ndarray(3,), C_tan: ndarray
      (3,3), state_vars')"
  strain_kind: "plano Voigt -ε [ε_xx, ε_yy, γ_xy] con γ_xy = 2·ε_xy"

state_schema:
  kappa:  "float ≥κ_0 - máxima deformación equivalente histórica"
  damage: "float ∈ [0, DAMAGE_MAX] - variable de daño escalar isótropa"
```

```
conventions:
  sign: "σ > 0 ⇔ tracción; energy no distingue tracción de compresión; mazars sólo daña
       con extensiones principales; modified_von_mises daña en compresión uniaxial con
      ε_eq = ε||/k"
  voigt: "γ_xy = 2·ε_xy en deformación; ε_eq sobre el estado 3D ε[_xx, ε_yy, ε_zz, γ_xy
      , 0, 0] con ε_zz = c_zz·ε(_xx + ε_yy): c_zz = -ν-ν/(1) en tensión plana, 0 en
      deformación plana"
  equivalent_strain: "energy: √ε^T(·C_e·ε/E) con la C_e plana del material; mazars:
      √Σ⟨ε⟩+_i^2) sobre las tres principales; modified_von_mises: (-k1)/(2k-ν(12))·I1 +
      1/(2k)·√(((-k1)-ν/(12))^2·I1^2 + 12kν/(1+)^2·J2). Las tres dan ε_eq = ε en tracción
      uniaxial de esfuerzo"
  damage_irreversibility: "κ monótono no decreciente vía maxκ(_old, ε_eq); dκ()
      creciente"

validity:
  - "E > 0, κ_0 > 0, α > 0"
  - "ν ∈ (-1, 0.5) (delegación a Elastic2D)"
  - "hypothesis ∈ {'plane_stress', 'plane_strain'}"
  - "equivalent_strain ∈ {'energy', 'mazars', 'modified_von_mises'}"
  - "k ≥ 1 finito, sólo con equivalent_strain = 'modified_von_mises' (obligatorio ahí);
       con k < 1 no dañaría en tracción triaxial y sí en compresión hidrostática (
      decisión del usuario, 2026-09-30)"

out_of_scope:
  - "modelo de Mazars completo con dos daños (d_t, d_c) ponderados por α_t, α_c; aquí s
      ólo su medida de deformación equivalente con un único daño escalar"
  - "anisotropía del daño (tensor de daño D vs escalar)"
  - "regularización para evitar mesh-dependency en regime de ablandamiento (longitud
      característica, no-local, gradient)"
  - "acoplamiento con plasticidad"
  - "fatiga / acumulación cíclica del daño"
  - "restitución de rigidez al cerrar fisuras (efecto unilateral) - el daño escalar
      degrada igual en tracción y compresión una vez activado"

numerical_caveats:
  - "IS_SYMMETRIC por instancia: true con energy (tangente consistente simétrica),
      false con mazars y modified_von_mises (LU/Pardiso, ADR 0003). Con energy y
      Cholesky disponible, la tangente de ablandamiento puede no ser definida positiva:
       el corrector degrada a LU al primer rechazo."
  - "Transición ↔loadingunloading es discontinua en la tangente. Newton converge antes
      de oscilar entre ramas en casos típicos. Para problemas patológicos cerca del
      umbral, considerar arc-length."
  - "Mazars: ε_eq es C^1 pero no C^2 donde una deformación principal (incluida ε_zz en
      esfuerzo plano) cruza cero, es decir, donde entra o sale una extensión. ∂ε_eq∂ε/
      y la tangente consistente son continuas en ese cruce; lo que salta es su derivada
       (la curvatura de ⟨ε⟩+_i^2), y la tangente sigue siendo Lipschitz, así que el
      Newton conserva la convergencia cuadrática. Con deformaciones principales
      repetidas la derivada está bien definida."
  - "Saturación d=DAMAGE_MAX corta la corrección consistente ∂(d∂κ/ deja de ser
      efectivamente (1-d)κα(1/+)); el algoritmo devuelve tangente secante en ese caso
      para evitar términos espurios."
  - "Sin regularización, la solución es mesh-dependent en régimen de ablandamiento (
      localización en una banda de elementos). Para benchmark cuantitativo se requiere
      mismo refinamiento de malla; para comparaciones cualitativas (presencia de banda,
       dirección) basta con regularización de gradiente o longitud característica (
```

```
      fuera de scope)."

acceptance:
  # tests/test_materials_unit.py · TestIsotropicDamage2D,
      TestIsotropicDamage2DConsistentTangent;
  # tests/test_damage_equivalent_strain.py (tres medidas); tests/
      test_damage_2d_3d_consistency.py;
  # tests/test_solid_2d_damage.py (integración).
  unit_existing_no_regression:
    - name: secant_response_below_threshold
      expect: "κ ≤κ _0 ⇒ d=0, σ = C_e·ε, tangente = C_e"
      tol_rel: 1.0e-12

    - name: damage_evolution_exponential
      expect: "para κ > κ_0, d cumple ley exponencial con ε_eq de la medida (energy:
          ε_eq = ε_xx√-ν/(1²) para ε = ε[_xx, 0, 0] en tensión plana)"
      tol_rel: 1.0e-10

  unit_equivalent_strain:
    - name: eps_eq_matches_independent_reference
      setup: "25 estados aleatorios por medida e hipótesis; referencia sobre el tensor
          3D σε(: con Lamé, autovalores de numpy, J2 = ½·e:e)"
      expect: "κ tras un paso desde κ = 0 coincide con ε_eq de referencia"
      tol_rel: 1.0e-12

    - name: uniaxial_stress_calibration
      setup: "tracción y compresión uniaxial de esfuerzo en tensión plana ε( = ε[, -νε,
          0])"
      expect: "tracción: ε_eq = ε con las tres medidas κ(_0 = f_t/E); compresión:
          energy ε||, mazars √2·ν·ε||, modified_von_mises ε||/k"
      tol_rel: 1.0e-13

    - name: damage_onset_at_kappa_0
      expect: "d = 0 con ε_xx = 0,999·κ_0 y d > 0 con ε_xx = 1,001·κ_0 en tracción
          uniaxial de esfuerzo, con las tres medidas"

    - name: rotational_invariance
      expect: "ε_eq invariante frente a giros en el plano con las tres medidas"
      tol_rel: 1.0e-12

    - name: constructor_validation
      expect: "ValueError con el nombre del material si equivalent_strain es
          desconocida, si falta k con modified_von_mises, si se da k con otra medida o
          si k ≤ 0 / no finito"

  unit_consistent_tangent:
    - name: tangent_equals_secant_on_unloading
      setup: "carga hasta κκ>_0, descarga con ε de menor ε_eq"
      expect: "C_tan = (1-d)·C_e exacto (segundo término se anula con ∂κ∂ε/ = 0)"
      tol_rel: 1.0e-12

    - name: tangent_equals_secant_below_threshold
      setup: "ε pequeño, κ_old = κ_0"
      expect: "C_tan = C_e (sin daño activo, ni secante con d=0 ni corrección)"
      tol_rel: 1.0e-12
```

```
  - name: tangent_equals_secant_at_saturation
    setup: "carga hasta κ enorme tal que d alcanza DAMAGE_MAX"
    expect: "C_tan = (1-DAMAGE_MAX)·C_e (el término consistente se corta porque ∂d∂κ/
        efectivamente 0 al saturar)"
    tol_rel: 1.0e-12

  - name: tangent_consistent_term_on_loading
    setup: "carga activa con κ > κ_0 y d < DAMAGE_MAX"
    expect: "C_tan ≠ (1-d)·C_e - incluye el término rank-1 -(1-d)κα(1/+)·(C_e·ε) ⊗∂ε
        _eq∂ε/ (§7)"
    tol_rel: 1.0e-9

  - name: tangent_symmetry_in_loading
    setup: "carga activa con estado no degenerado ε(_xx ≠ε _yy, γ_xy ≠ 0)"
    expect: "energy: C_tan = ᵀC_tan bit a bit; mazars y modified_von_mises: ‖C_tan
        -ᵀ‖ C_tan_F‖/‖C_tan_F > 1e-3"

  - name: tangent_finite_difference_consistency
    setup: "carga activa desde κ = κ_0, con cada medida, en tensión y deformación
        plana; estados general, todo extensión y principales del plano repetidas;
        diferencia centrada con h = 1e-10 sobre ε|| ~ 1e-4"
    expect: "|C_FD - C_tan| ≤ 1e-6·max|C_FD| (truncamiento ~1e-12 y redondeo ~1e-10
        relativos)"
    tol_rel: 1.0e-6

unit_arch:
  - name: is_symmetric_attribute
    expect: "IsotropicDamage2D.IS_SYMMETRIC == False en la clase; instancia True con
        energy, False con mazars y modified_von_mises"

  - name: batch_kernel_matches_compute_state
    expect: "el kernel por lotes reproduce bit a bit σ, C_tan y la fila de estado de
        compute_state con las tres medidas (carga, descarga, recarga, compresión)"

cross_consistency:
  - name: restriction_of_isotropic_damage_3d
    setup: "historia de carga, descarga y recarga en cuatro direcciones; tensión
        plana ε(_zz = c_zz·ε(_xxε+_yy)) y deformación plana ε(_zz = 0); las tres
        medidas"
    expect: "κ, d y σ en el plano iguales a IsotropicDamage3D con el estado 3D
        asociado; σ_zz 3D = 0 en tensión plana"
    tol_rel: 1.0e-12

integration:
  - name: quad4_damage_newton_converges_in_few_iter
    setup: "Quad4 unitario, tensión plana, tracción uniaxial de esfuerzo por
        desplazamiento prescrito δ = 4·κ_0 en 5 pasos, sin bisección; las tres
        medidas"
    expect: "Newton converge en ≤6 iteraciones por paso; d y σ_xx = -(1d)·E·δ analí
        ticos ε(_eq = δ)"
    tol_rel: 1.0e-8

  - name: yaml_equivalent_strain
    setup: "Quad4 con mazars y con la medida por defecto, Hex8 (IsotropicDamage3D)
        con modified_von_mises y k, leídos desde YAML"
```

```
      expect: "parámetros leídos; reacción -(1d)·E·δ analítica; modified_von_mises sin
          k rechazado al leer"

references:
  - "Lemaitre J., Chaboche J.-L. (1990). Mechanics of Solid Materials. Cambridge UP.
      Cap. 7 (continuum damage mechanics)."
  - "Oliver J. (1989). A consistent characteristic length for smeared cracking models.
      IJNME 28, 461-474."
  - "Simó J.C., Ju J.W. (1987). Strain- and stress-based continuum damage models - I.
      Formulation. Int. J. Solids Struct. 23, 821-840 (deformación equivalente energé
      tica; tangente consistente para daño isótropo)."
  - "Mazars J. (1986). A description of micro- and macroscale damage of concrete
      structures. Eng. Fract. Mech. 25, 729-737 (deformación equivalente de Mazars)."
  - "de Vree J.H.P., Brekelmans W.A.M., van Gils M.A.J. (1995). Comparison of nonlocal
      approaches in continuum damage mechanics. Comput. Struct. 55, 581-588 (deformació
      n equivalente de von Mises modificado)."
  - "de Borst R., Crisfield M.A., Remmers J.J.C., Verhoosel C.V. (2012). Nonlinear
      Finite Element Analysis of Solids and Structures, 2.ª ed. Wiley. Cap. 6 (daño)."
  - "de Souza Neto E.A., ćPeri D., Owen D.R.J. (2008). Computational Methods for
      Plasticity. Wiley. §12 (damage models)."
```

10.3.4 Implementación

- **Archivo**: solidum/materials/damage_2d.py; medidas de deformación equivalente en solidum/materials/_equivalent_strain.py, compartidas con `IsotropicDamage3D`.
- **Clase**: `IsotropicDamage2D`, registrada vía `@MaterialRegistry.register`. `IS_SYMMETRIC = False` como ClassVar; el constructor pone `self.IS_SYMMETRIC = True` con `equivalent_strain='energy'`.
- **Estado**: dict `{'kappa': float, 'damage': float}`. `PRIMARY_STATE_VAR = 'damage'`.
- **Constructor**: valida `equivalent_strain` y `k` con `parse_equivalent_strain` (mensajes con el nombre del material y del parámetro) y guarda `equivalent_strain`, `k` (`None` salvo en von Mises modificado) y `c_zz`.
- **Algoritmo** `compute_state`: la física vive en el núcleo `@njit _compute_damage_2d`, que consumen `compute_state` y el adaptador por lotes (ADR 0014):

1. $\boldsymbol{\sigma}^{\text{eff}} = \mathbf{C}_e\boldsymbol{\varepsilon}$ y `equivalent_strain_2d` $\rightarrow (\varepsilon_{\text{eq}}, \mathbf{v}, s)$ con $\partial\varepsilon_{\text{eq}}/\partial\boldsymbol{\varepsilon} = s\,\mathbf{v}$. Con la energética $\mathbf{v} = \boldsymbol{\sigma}^{\text{eff}}$ y $s = 1/(E\varepsilon_{\text{eq}})$: el producto externo $(\boldsymbol{\sigma}^{\text{eff}}) \otimes \mathbf{v}$ es simétrico bit a bit.
2. Kuhn-Tucker con κ_n, ley exponencial, saturación a `DAMAGE_MAX`.
3. $\boldsymbol{\sigma} = (1-d)\mathbf{C}_e\boldsymbol{\varepsilon}$ y tangente según §7.

- **Mazars en 2D**: forma cerrada del bloque 2×2 del plano (casos todo extensión, todo contracción y mixto, este último con $(\mathbf{A} - e_2\mathbf{I})/(e_1 - e_2)$ evaluado sin cancelación) más ε_{zz}. **von Mises modificado en 2D**: sobre el estado 3D asociado y regla de la cadena.

- **Camino por lotes**: `batch_params = [E, κ_0, α, tol_abs, tol_rel, DAMAGE_MAX, c_zz, ν, código de la medida, k]` (códigos 0 energy, 1 mazars, 2 modified_von_mises; `k = 0` sin uso fuera de von Mises modificado). Sigue siendo cacheable en disco (verificado: el kernel de familia y los núcleos del material cargan del caché de Numba sin recompilar).
- `admissibility_scale`: `E·κ_0` (umbral inicial — la escala no evoluciona con el estado porque el criterio se mide contra κ_0, no contra κ).
- **Tests**:
- tests/test_materials_unit.py · `TestIsotropicDamage2D` y `TestIsotropicDamage2DConsistentTange`
- tests/test_damage_equivalent_strain.py: las tres medidas (valores, calibración, tangente por diferencias finitas, lotes, simetría, constructor, YAML).
- tests/test_damage_2d_3d_consistency.py: restricción exacta del 3D con las tres medidas.
- tests/test_solid_2d_damage.py: integración Quad4 + NonlinearSolver.

10.3.5 Diálogo

- **2026-05-14** · Spec creada retroactivamente sobre material existente (tangente secante) y extendida con tangente algorítmica consistente. Sin ADR: extiende un material existente añadiendo un término a la tangente; no rompe contratos arquitecturales transversales. Sí cambia `IS_SYMMETRIC` para este material, lo cual el despachador algebraico ya soporta vía agregación sobre el dominio (no requiere refactor).
- **2026-05-14** · Decisión: la transición loading↔unloading se decide con κ_n committed (no con κ trial dentro del Newton). Esto es lo estándar y evita oscilaciones del Newton entre ramas en pasos con ambigüedad. Si el paso converge, κ se compromete y la decisión es consistente con la trayectoria.
- **2026-05-14** · `IsotropicDamage1D` queda con tangente secante. Misma deuda gemela conceptual, pero fuera del alcance pactado (A2 era solo 2D). Si el usuario la prioriza luego, replicar es trivial: misma fórmula reducida a escalar.
- **2026-09-22** · Auditoría global: nuevo `out_of_plane_stress` delegado al elástico base ($\sigma = (1-\mathrm{d})\cdot \mathrm{C_e}\cdot\varepsilon$ escala todas las componentes, así que $\sigma_zz = \nu(\sigma_xx + \sigma_yy)$ sigue valiendo).
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): la física de `compute_state` se movió a un núcleo `@njit` (`_compute_damage_2d`) que consumen el camino por elemento y el adaptador por lotes; `evaluate_exponential_damage` se compila y recibe el cap como argumento. Sin cambio de formulación; `STATE_SCHEMA = {kappa, damage}` con estado inicial κ = κ_0.
- **2026-09-22** · Cierre del ADR 0014 (§9): `batch_out_of_plane_stress(sigma, S)`, versión por arreglos de `out_of_plane_stress` para el post-proceso por familia (exportador VTK); misma fórmula, `ε^p_zz` leída de la fila de estado cuando aplica. Verificado archivo VTK idéntico por ambos caminos.

- **2026-09-29** · Cambio de formulación aprobado por el usuario (hallazgo clas-3 de la auditoría de pureza): en tensión plana ε_{eq} incluye $\varepsilon_{zz} = -\nu/(1-\nu)\,(\varepsilon_{xx} + \varepsilon_{yy})$, que con daño escalar isótropo no depende de d. Antes se omitía y, con el mismo estado de esfuerzo, el 2D retrasaba el inicio del daño respecto de `IsotropicDamage3D` (con $E = 30$ GPa, $\nu = 0{,}2$, $\kappa_0 = 10^{-4}$: $d = 0$ en 2D frente a $d = 1{,}4\,\%$ en 3D). La tangente consistente incluye el término de ε_{zz} en $\partial\varepsilon_{\text{eq}}/\partial\boldsymbol{\varepsilon}$ (verificada por diferencias finitas). Blindado en `tests/test_damage_2d_3d_consistency.py`: el 2D reproduce al 3D en toda una historia de carga, descarga y recarga, en tensión y en deformación plana, y el test falla si se retira el término. Queda abierta la otra mitad del hallazgo: si la norma del tensor es la medida adecuada o conviene una de la literatura (Mazars, von Mises modificado); pendiente del usuario.

- **2026-09-29** · Segunda mitad del hallazgo, **cambio aprobado por el usuario**: se retira la norma del tensor $\sqrt{\boldsymbol{\varepsilon} : \boldsymbol{\varepsilon}}$, convención propia no estándar y mal calibrada (en tracción uniaxial de esfuerzo $\varepsilon_{\text{eq}} = \varepsilon\sqrt{1 + 2\nu^2}$, así que $\kappa_0 \neq f_t/E$). La deformación equivalente pasa a ser configurable con `equivalent_strain`: **energética** por defecto (Simó y Ju 1987, la referencia que ya citaba la spec; simétrica en tracción y compresión, como la anterior), y **Mazars** y **von Mises modificado** (este con `k = f_c/f_t` obligatorio) como opciones que distinguen tracción de compresión. Las tres dan $\varepsilon_{\text{eq}} = \varepsilon$ en tracción uniaxial ($\kappa_0 = f_t/E$). Ley de daño, variable histórica, `DAMAGE_MAX`, esfuerzo y ramas de la tangente sin cambios; `IsotropicDamage1D` tampoco cambia ($|\varepsilon|$ es la restricción 1D de las tres). Consecuencias: (i) con la energética la tangente consistente es simétrica y la instancia declara `IS_SYMMETRIC = True` (§8), lo que habilita Cholesky cuando está instalado, con la degradación a LU ya existente si la tangente de ablandamiento no es definida positiva; (ii) las medidas viven en `solidum/materials/_equivalent_strain.py`, compartido con el 3D (dos casos reales, Reglas §1); (iii) el camino por lotes recibe la medida como código numérico en `batch_params`. Tests existentes recalculados con la medida nueva a partir de las fórmulas; `tests/test_solid_2d_damage.py` pasa a desplazamiento prescrito, porque con fuerza por encima del pico ($\sigma \approx E\kappa_0$) sólo había equilibrio con el daño saturado y el paso adaptativo ocultaba las iteraciones.

10.4 Orthotropic2D

*Spec **pre-redactada por la IA** a partir de la sesión 2026-09-09, según `Reglas.md §4` y el criterio de [[feedback-filtrar-puntos-abiertos]]: el plumbing arquitectural lo decide la IA con razón documentada; los puntos abiertos que quedan son **físicos y de alcance**, y los valida el usuario en §Diálogo.*

Decisión arquitectural de fondo en ADR 0013 — dónde vive la orientación material.

10.4.1 Especificación física

0. Descripción general

Modelo elástico lineal **ortótropo** bidimensional bajo hipótesis **plane stress**, parametrizado por cuatro constantes independientes $(E_1, E_2, G_{12}, \nu_{12})$ y un ángulo de orientación θ que sitúa los ejes principales del material respecto a los ejes globales.

Un material ortótropo posee tres planos de simetría material mutuamente perpendiculares. Sus **ejes principales** son las direcciones privilegiadas respecto a las cuales el comportamiento se describe sin acoplamiento. En bambú:

Eje	Dirección física	Qué gobierna
1 (L)	Longitudinal, a lo largo del culmo	Haces vasculares — dirección rígida
2 (R o T)	Transversal en el plano de análisis	Sólo parénquima

Independiente de la velocidad, sin variables históricas. Es el primer material anisótropo del catálogo.

`plane_strain` **está fuera de alcance** — ver §Fuera de alcance y ADR 0013 §3.

1. Descomposición de la deformación

No aplica — toda la deformación es elástica: $\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e$.

2. Ley elástica

$$\boldsymbol{\sigma} = \mathbf{C}_e \, \boldsymbol{\varepsilon}$$

con la notación Voigt 2D del proyecto (`Reglas.md §5`): $\boldsymbol{\varepsilon} = [\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]^\top$ con $\gamma_{xy} = 2\varepsilon_{xy}$ *engineering*, y $\boldsymbol{\sigma} = [\sigma_{xx}, \sigma_{yy}, \sigma_{xy}]^\top$.

2.1 Flexibilidad en ejes del material La forma inversa es donde las constantes tienen significado físico directo y medible:

$$\begin{Bmatrix} \varepsilon_{11} \\ \varepsilon_{22} \\ \gamma_{12} \end{Bmatrix} = \begin{bmatrix} 1/E_1 & -\nu_{21}/E_2 & 0 \\ -\nu_{12}/E_1 & 1/E_2 & 0 \\ 0 & 0 & 1/G_{12} \end{bmatrix} \begin{Bmatrix} \sigma_{11} \\ \sigma_{22} \\ \sigma_{12} \end{Bmatrix}$$

Los ceros del bloque cortante expresan que **en ejes del material no hay acoplamiento** entre esfuerzos normales y distorsión angular.

2.2 Reciprocidad La simetría de la matriz de flexibilidad —consecuencia de la existencia de una energía de deformación— impone:

$$\boxed{\frac{\nu_{12}}{E_1} = \frac{\nu_{21}}{E_2}} \qquad \Longrightarrow \qquad \nu_{21} = \nu_{12}\,\frac{E_2}{E_1}$$

Por eso hay **cuatro** constantes independientes y no cinco: ν_{21} queda determinada.

Consecuencia contraintuitiva pero correcta: como $E_1 \gg E_2$ en bambú, $\nu_{21} \ll \nu_{12}$. Con $E_1/E_2 = 20$ y $\nu_{12} = 0.35$ resulta $\nu_{21} = 0.0175$. **Valores de Poisson diminutos son normales** en materiales muy ortótropos, no error de medición.

2.3 Rigidez en ejes del material Invirtiendo la flexibilidad:

$$\mathbf{C}_{\text{mat}} = \begin{bmatrix} \dfrac{E_1}{1-\nu_{12}\nu_{21}} & \dfrac{\nu_{12}E_2}{1-\nu_{12}\nu_{21}} & 0 \\ \dfrac{\nu_{12}E_2}{1-\nu_{12}\nu_{21}} & \dfrac{E_2}{1-\nu_{12}\nu_{21}} & 0 \\ 0 & 0 & G_{12} \end{bmatrix}$$

El factor $1-\nu_{12}\nu_{21}$ juega el papel estructural que $1-\nu^2$ tiene en el isótropo.

2.4 Rotación a ejes globales Con θ el ángulo de los ejes del material respecto a los globales (positivo antihorario, coherente con `Reglas.md §5`), $c = \cos\theta$, $s = \sin\theta$, la transformación de **deformaciones** en Voigt *engineering* es:

$$\mathbf{T} = \begin{bmatrix} c^2 & s^2 & cs \\ s^2 & c^2 & -cs \\ -2cs & 2cs & c^2 - s^2 \end{bmatrix} \qquad \Longrightarrow \qquad \boxed{\mathbf{C}_{\text{glob}} = \mathbf{T}^{\top}\,\mathbf{C}_{\text{mat}}\,\mathbf{T}}$$

Para $\theta \neq 0$ la matriz **se llena por completo**:

$$\mathbf{C}_{\text{glob}} = \begin{bmatrix} C_{11} & C_{12} & C_{16} \\ C_{12} & C_{22} & C_{26} \\ C_{16} & C_{26} & C_{66} \end{bmatrix}$$

Los términos C_{16}, C_{26} —nulos en ejes del material— son el **acoplamiento tracción–cortante**: un ε_{xx} puro genera σ_{xy}. Físicamente, una tira cortada a 45° de la fibra no sólo se alarga al traccionarla: se tuerce. Este acoplamiento no existe en ningún material isótropo.

$\mathbf{C}_{\text{glob}}$ **sigue siendo simétrica** (lo garantiza la energía de deformación), luego `IS_SYMMETRIC = True` y el despachador algebraico del ADR 0003 la trata igual que a las isótropas.

3-6. Fluencia, flujo, endurecimiento, Kuhn-Tucker

No aplican — el material es indefinidamente elástico.

7. Variables internas

Ninguna. `PRIMARY_STATE_VAR = None`, como el resto de materiales puramente elásticos del catálogo.

8. Notación Voigt

La del proyecto (`Reglas.md §5`), sin componentes auxiliares fuera del plano: bajo plane stress $\sigma_{33} = 0$ y ε_{33} no interviene en la respuesta en el plano.

Convención de subíndices de Poisson — fijada explícitamente. Existen dos convenciones opuestas en la literatura y confundirlas **invierte los papeles de E_1 y E_2** sin fallo ruidoso. Se adopta la de Jones/Tsai:

$$\nu_{12} = -\frac{\varepsilon_{22}}{\varepsilon_{11}} \quad \text{bajo carga uniaxial en la dirección 1}$$

Primer índice = dirección de carga. Segundo = dirección de la contracción medida.

10.4.2 Formulación numérica

9-10. Esquema temporal y actualización de estado

No aplican — material sin historia. $\mathbf{C}_{\text{glob}}$ se **precalcula una sola vez en el constructor** (igual que `Elastic2D` precalcula `self.C` según la hipótesis) y `compute_state` devuelve ($\mathbf{C}\boldsymbol{\varepsilon}$, $\mathbf{C}$, state_vars) sin recomputar nada.

11. Tangente

Constante e igual a $\mathbf{C}_{\text{glob}}$ en todo régimen. Simétrica y definida positiva dentro del rango admisible (§12).

12. Admisibilidad y caveats

No se puede reutilizar la validación de `Elastic2D`. Ese material valida $-1 < \nu < 0.5$, criterio correcto para isótropo e **incorrecto aquí**. La condición ortótropa es que $\mathbf{C}$ sea definida positiva:

$$E_1,\, E_2,\, G_{12} > 0 \qquad \text{y} \qquad 1 - \nu_{12}\nu_{21} > 0 \iff |\nu_{12}| < \sqrt{E_1/E_2}$$

Con $E_1/E_2 = 20$ el límite es $|\nu_{12}| < 4.47$: $\nu_{12} > 0.5$ **es físicamente legítimo** en un material muy ortótropo. Un validador heredado del isótropo rechazaría datos válidos de bambú.

Caveat de implementación — el factor 2 del cortante. Con γ *engineering*, las matrices de transformación de esfuerzo y de deformación **no coinciden**: difieren en factores 2 en el bloque cortante. La forma $\mathbf{T}^\top\mathbf{C}\mathbf{T}$ es válida precisamente para la $\mathbf{T}$ de deformaciones de §2.4. Es el error clásico al programar esto y debe quedar blindado por el test de invariancia (`§acceptance`).

G_{12} **es independiente**: no se deriva de E y ν como en isótropo. Debe medirse.

10.4.3 Contrato de implementación

```yaml
name: Orthotropic2D
kind: material
status: validated

interface:
  strain_dim: 3
```

```yaml
  primary_state_var: null
  is_symmetric: true

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - zienkiewicz_taylor: "vol. 1: elasticidad plana anisótropa (matriz D y cambio de
        ejes)"
  general_codes:
    - abaqus: "*ELASTIC, TYPE=LAMINA + *ORIENTATION"
    - ansys: "MP: EX, EY, PRXY, GXY"

parameters:
  - { name: E1,    type: float, required: true,  desc: "Módulo de Young en la dirección
      1 (fibra). > 0" }
  - { name: E2,    type: float, required: true,  desc: "Módulo de Young en la dirección
      2 (transversal). > 0" }
  - { name: G12,   type: float, required: true,  desc: "Módulo de cortante en el plano
      1-2. Independiente de E y nu. > 0" }
  - { name: nu12,  type: float, required: true,  desc: "Poisson mayor: carga en 1,
      contracción medida en 2. Admisible |nu12| < sqrt(E1/E2)" }
  - { name: theta, type: float, required: false, default: 0.0, desc: "Ángulo en GRADOS
      de los ejes del material respecto a los globales, positivo antihorario. ADR 0013:
      atajo consciente, migrará al elemento." }
  - { name: density, type: float, required: false, default: null, desc: "Densidad (ADR
      0008). Opcional; requerida por peso propio y matriz de masa." }

state_variables: []

conventions:
  voigt: "[eps_xx, eps_yy, gamma_xy] con gamma_xy = 2*eps_xy (engineering); [sig_xx,
      sig_yy, sig_xy]"
  units: "E1, E2, G12 en Pa; nu12 adimensional; theta en grados; density en kg/m^3"
  sign: "Tracción positiva. theta positivo antihorario (Reglas.md §5). nu_ij: primer í
      ndice = dirección de carga (Jones/Tsai)."

validity:
  - "Pequeñas deformaciones, elasticidad lineal indefinida"
  - "Hipótesis plane_stress únicamente"
  - "Ortotropía con ejes principales constantes en todo el material (theta uniforme)"

out_of_scope:
  - "plane_strain - requiere E3, nu13, nu23; el problema plano no cierra sin datos 3D (
      ADR 0013 §3)"
  - "Orthotropic3D - nueve constantes independientes, rotación 6x6; cuando lo pida un
      caso real"
  - "Criterios de falla ortótropos (Tsai-Wu, Hashin, Hoffman) - componente distinto y
      posterior"
  - "Material gradado (propiedades función de la posición)"
  - "Ortotropía cilíndrica (ejes que siguen la geometría del culmo) - requiere
      orientación por punto de Gauss, ADR 0013 §2"
  - "Orientación variable por elemento - migración a ADR 0013 opción B"

acceptance:
  verification:
    - name: degeneracion_al_isotropo
      setup: "Construir con E1 = E2 = E, nu12 = nu, G12 = E/(2(1+nu)), theta = 0.
```

```
            Comparar C contra Elastic2D(E, nu, 'plane_stress').C"
      expect: "Coincidencia exacta: el isótropo es el caso particular del ortótropo"
      tol_rel: 1.0e-14
    - name: respuesta_elastica_en_ejes
      setup: "theta = 0. Deformación uniaxial eps_xx impuesta, resto nulo"
      expect: "sigma = C_mat @ eps exacto; sin acoplamiento: sigma_xy = 0"
      tol_rel: 1.0e-12
    - name: tangente_constante
      setup: "Evaluar compute_state con varias deformaciones distintas"
      expect: "La tangente devuelta es idéntica en todas: material sin historia"
      tol_rel: 1.0e-14
  specific:
    - name: acoplamiento_traccion_cortante
      setup: "theta = 45 grados, deformación uniaxial pura eps_xx"
      expect: "sigma_xy != 0 (acoplamiento). Verificar contra C_glob = T^T C_mat T
               calculada independientemente en el test"
      tol_rel: 1.0e-12
    - name: sin_acoplamiento_en_ejes
      setup: "theta = 0 y theta = 90 grados"
      expect: "C[0,2] = C[1,2] = 0 en ambos casos (ejes principales alineados);
               en theta=90 los papeles de E1 y E2 quedan intercambiados"
      tol_rel: 1.0e-14
    - name: invariancia_rotacion_material_y_carga
      setup: "Rotar material y estado de deformación el mismo ángulo; comparar la
              energía de deformación 0.5*eps^T C eps y los invariantes de sigma"
      expect: "Invariantes idénticos: la respuesta física no depende del sistema
               de referencia. Blinda el factor 2 del cortante engineering (§12)"
      tol_rel: 1.0e-12
    - name: simetria_y_definicion_positiva
      setup: "Barrido de theta en [0, 180) grados con datos típicos de bambú"
      expect: "C simétrica a precisión máquina y autovalores > 0 para todo theta"
      tol_rel: 1.0e-14
    - name: reciprocidad
      setup: "Verificar nu21 derivado internamente"
      expect: "nu12/E1 == nu21/E2"
      tol_rel: 1.0e-14
    - name: admisibilidad_rechaza_invalidos
      setup: "E1<=0, E2<=0, G12<=0, |nu12| >= sqrt(E1/E2)"
      expect: "ValueError con mensaje que nombra el parámetro y el criterio violado"
      tol_rel: null
    - name: poisson_mayor_que_medio_es_admisible
      setup: "E1/E2 = 20, nu12 = 0.6 (excede el límite isótropo de 0.5)"
      expect: "Se construye sin error: |nu12| < sqrt(20) = 4.47. Blinda que NO se
               copió la validación de Elastic2D"
      tol_rel: null

references:
  - "Jones R.M. (1999). Mechanics of Composite Materials, 2a ed. Taylor & Francis. Cap.
     2 (constitutiva ortótropa 2D, reciprocidad, transformación de ejes)."
  - "Lekhnitskii S.G. (1963). Theory of Elasticity of an Anisotropic Elastic Body.
     Holden-Day. (Restricciones de admisibilidad por definición positiva.)"
  - "Ting T.C.T. (1996). Anisotropic Elasticity: Theory and Applications. Oxford
     University Press."
```

10.4.4 Diálogo

Puntos abiertos para el usuario

Ninguno bloqueante — las tres decisiones de alcance (sólo `plane_stress`, sólo 2D, sólo elasticidad) se acordaron en sesión el 2026-09-09 y están registradas en el ADR 0013 §3.

Queda una cuestión **física** que la campaña experimental del PAPIIT resolverá y que conviene anotar aquí cuando haya datos:

1. **Valores de referencia de la especie estudiada.** Los órdenes de magnitud usados en los tests son de literatura general, no de una especie concreta:

Constante	Orden típico en bambú
E_1	10–20 GPa
E_2	0.5–1 GPa
G_{12}	0.5–1 GPa
ν_{12}	0.3–0.4

La relación $E_1/E_2 \approx 20$ sitúa al bambú en el mismo régimen que un laminado unidireccional de carbono (15–20) o la madera (15–25). Cuando existan valores medidos, sustituirlos en el test de barrido y anotar la especie.

1. **¿El plano de análisis es L–R o L–T?** Ambos son ortótropos 2D válidos y la spec los cubre igual, pero E_2 toma valores distintos según cuál sea. Es dato de la campaña, no decisión de diseño.

10.4.5 Implementación

Rellena la IA tras programar.

- **Archivo**: `solidum/materials/orthotropic_2d.py`
- **Clase**: `Orthotropic2D` (auto-registrada vía `@MaterialRegistry.register`)
- **Tests (verificación)**: `tests/test_orthotropic_2d.py` — 26 tests + 57 subtests.
- **Validación externa**: `tests/validation/test_off_axis_orthotropic.py` — 20 tests + 221 subtests contra las formas cerradas de **Jones (1999) §2.8**: módulo aparente `E_x(`θ`)` (ec. 2.85, 1e-12 rel.), Poisson aparente ν`_xy(`θ`)` (ec. 2.86, 1e-10) y coeficiente de influencia mutua η`_xy,x(`θ`)`. Incluye consistencia FEM sobre un `Quad4` real (1e-10), no sólo sobre la constitutiva. Cifra citable: a 45° la rigidez cae **por debajo del 10 % de E_1**, muy por debajo de lo que sugiere `E1/E2 = 18.75`, porque el término cruzado `(1/G12 - 2ν12/E1)` domina la zona intermedia. **Mutation test documentado**: quitar el factor 2 del bloque cortante de `T` produce 39 fallos — el benchmark no pasa por construcción. **Ampliado 2026-09-10**: las tres constantes se miden ahora sobre modelos FEM de los **cinco** elementos sólidos 2D (Quad4, Tri3, Quad8, Quad9, Tri6), no sólo sobre `Quad4`; en particular η`_xy,x` medido sobre el modelo valida la recuperación del cortante engineering en todo el pipeline. El mutation test pasa de 39 a **102 fallos**.
- **Patch test ortótropo**: `tests/test_patch_orthotropic_2d.py` — 11 tests + 40 subtests. MacNeal-Harder sobre malla distorsionada con los cinco elementos y $\theta \in 0°, 30°, 45°, 90°$, verificando ε **y** σ en cada punto de Gauss vía `compute_gauss_state()`. Con material isótropo `C[0,2] = C[1,2] = 0` enmascara errores de acoplamiento; con la constitutiva llena esos caminos quedan ejercitados. *Alcance acotado*: compara σ contra la misma `C` que usa el elemento,

así que valida el **elemento**, no la constitutiva — verificado por mutación (caveat en `tests/validation/README.md` §11).

- **Integración**: `tests/test_orthotropic_integration.py` — 21 tests + 52 subtests. Cubre los caminos externos que ningún otro frente tocaba: regresión de la tabla publicada en el README del ejemplo (siete orientaciones vía `run_yaml`), matriz de masa con `density` (la masa no debe ver la anisotropía), validación por YAML del criterio ortótropo, degeneración isótropa end-to-end contra `Elastic2D`, y el contrato genérico del material —incluida la periodicidad $\theta \equiv \theta+180°$— que fija lo que la migración a orientación-en-el-elemento del ADR 0013 §2 debe preservar.

- Suite global 1176 → 1220 → **1259**. Frente ortótropo: **78 tests + 370 subtests** repartidos en cuatro archivos (verificación, validación externa, patch test, integración).

- **Notas de implementación**:

- `C_mat` y `C` (rotada) se **precalculan en el constructor**; `compute_state` no recomputa nada. Mismo patrón que `Elastic2D` con su hipótesis.

- **Tolerancias adimensionales en todos los tests** (ADR 0006). Primera versión usaba `assertAlmostEqual(..., places=6)` absoluto y fallaba en `theta` $\in$ `{90°, 180°}` con residuos de ~1e-6 sobre magnitudes de ~1e10 Pa — que son **1e-16 en relativo**, precisión máquina. El origen es que `sin(180°)` no da cero exacto en punto flotante. Comparar contra cero absoluto convertía el test en una medida de la magnitud de `E1`, no de la física. Verificado que el criterio es invariante ante cambio de unidades en 15 órdenes de magnitud (Pa, MPa, micro-unidades).

- **Bring-up end-to-end** con `Quad4` + `LinearSolver`: `E_aparente` = 15.10 GPa (θ=0°), 4.82 GPa (θ=45°), 0.805 GPa (θ=90°), con σ`_xy` $\neq$ `0` sólo fuera de ejes principales y `K` simétrica en los tres casos.

- **Notas de traducción**: ninguna — la formulación de Jones se implementa en la convención del proyecto sin cambio de signos.

- **2026-09-22** · Ensamblaje por lotes (ADR 0014): `STATE_SCHEMA = {}` y kernel lineal compartido (`materials/_batch.linear_material_kernel`, σ `= C`$\cdot\varepsilon$); `batch_matrix()` devuelve la `C` precalculada.

10.5 VonMises2D

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

*Spec **retroactiva con ampliación**: la hipótesis `plane_strain` está implementada y testeada desde sesiones anteriores; esta spec la documenta y añade `plane_stress` como hipótesis a implementar en esta sesión.*

10.5.1 Especificación física

0. Descripción general

Modelo elastoplástico independiente de la velocidad ("rate-independent") basado en el criterio **J2 / Von Mises** con regla de flujo **asociada** y **endurecimiento isótropo lineal**. Captura plasticidad de metales dúctiles en régimen de pequeñas deformaciones, sin efectos cinemáticos (Bauschinger), térmicos ni viscosos.

Soporta dos hipótesis cinemáticas 2D mutuamente excluyentes seleccionadas en construcción:

- `plane_strain` — $\varepsilon_{zz} = \varepsilon_{xz} = \varepsilon_{yz} = 0$. Cuerpos prismáticos largos cargados transversalmente (presa, túnel, cilindro largo).
- `plane_stress` — $\sigma_{zz} = \sigma_{xz} = \sigma_{yz} = 0$. Láminas o membranas delgadas en su plano (chapa traccionada, placa fina).

1. Descomposición aditiva

En pequeñas deformaciones, el tensor de deformación se descompone aditivamente:

$$\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e + \boldsymbol{\varepsilon}^p$$

La parte elástica $\boldsymbol{\varepsilon}^e$ gobierna el esfuerzo vía la ley elástica isótropa; la parte plástica $\boldsymbol{\varepsilon}^p$ es **histórica** y evoluciona según la regla de flujo.

2. Ley elástica

$$\boldsymbol{\sigma} = \mathbf{C}_e : \boldsymbol{\varepsilon}^e = \mathbf{C}_e : (\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p)$$

con $\mathbf{C}_e$ tensor de rigidez elástica isótropa parametrizado por (E, ν) o equivalentemente (K, G):

$$K = \frac{E}{3(1-2\nu)}, \qquad G = \frac{E}{2(1+\nu)}$$

3. Superficie de fluencia (J2)

$$f(\boldsymbol{\sigma}, \alpha) = \|\mathbf{s}\| - \sqrt{\tfrac{2}{3}}\,(\sigma_y + H\,\alpha) \;\le\; 0$$

donde $\mathbf{s} = \boldsymbol{\sigma} - \frac{1}{3}\,\mathrm{tr}(\boldsymbol{\sigma})\mathbf{I}$ es el tensor desviador, $\|\mathbf{s}\| = \sqrt{\mathbf{s} : \mathbf{s}}$, $\sigma_y > 0$ es la fluencia inicial y $H \ge 0$ el módulo de endurecimiento isótropo lineal. El factor $\sqrt{2/3}$ alinea σ_y con la fluencia uniaxial estándar.

4. Regla de flujo (asociada)

$$\dot{\boldsymbol{\varepsilon}}^p = \dot{\gamma}\,\frac{\partial f}{\partial \boldsymbol{\sigma}} = \dot{\gamma}\,\mathbf{N}, \qquad \mathbf{N} = \frac{\mathbf{s}}{\|\mathbf{s}\|}$$

El flujo es puramente desviador ($\mathrm{tr}(\mathbf{N}) = 0$) — **plasticidad incompresible**, propiedad central de J2.

5. Endurecimiento isótropo lineal

$$\dot{\alpha} = \sqrt{\tfrac{2}{3}}\,\dot{\gamma}$$

α es la **deformación plástica acumulada equivalente** (escalar, monótona no decreciente). La superficie de fluencia se expande uniformemente con α manteniendo su centro en el origen del espacio de tensiones desviadoras.

6. Condiciones de Kuhn-Tucker

$$\dot{\gamma} \geq 0, \qquad f \leq 0, \qquad \dot{\gamma}\, f = 0$$

Garantizan que $\dot{\gamma} > 0$ solo si el estado intenta salir de la superficie, y que el estado real siempre cumple $f \leq 0$.

7. Variables internas

Símbolo	Tipo	Significado
$\boldsymbol{\varepsilon}^p$	tensor (4 componentes Voigt extendido en plane strain, ver §8)	deformación plástica
α	escalar	deformación plástica acumulada equivalente

`alpha` es la variable principal exportable (`PRIMARY_STATE_VAR = 'alpha'`); `eps_p` es interna al algoritmo de return mapping.

8. Notación Voigt y manejo de ε^p_{zz}

Convención del proyecto para 2D: $\boldsymbol{\varepsilon} = [\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$ con $\gamma_{xy} = 2\varepsilon_{xy}$. Pero la **descomposición desviadora** requiere todas las componentes 3D, así que internamente el material maneja $\boldsymbol{\varepsilon}^p = [\varepsilon^p_{xx}, \varepsilon^p_{yy}, \varepsilon^p_{zz}, \varepsilon^p_{xy}]$ (4 componentes, ε^p_{xy} tensorial sin factor 2).

- **Plane strain**: $\varepsilon_{zz} = 0$ se impone *en deformación total*. La componente ε^p_{zz} es no nula y evoluciona libremente (la incompresibilidad plástica acopla $\varepsilon^p_{zz} = -(\varepsilon^p_{xx} + \varepsilon^p_{yy})$). Por simetría, σ_{zz} resulta no nulo pero no se expone en el API público 2D — vive solo en el cómputo interno.
- **Plane stress**: $\sigma_{zz} = 0$ se impone *en esfuerzo*. La componente ε_{zz} es no nula (extensión transversal de Poisson + Poisson plástico), y aparece como variable adicional en el problema local de retorno (ver §10).

10.5.2 Formulación numérica

9. Esquema temporal — Backward Euler implícito

Dado el estado convergido $\{\boldsymbol{\varepsilon}^p_n, \alpha_n\}$ al paso n y la deformación total prescrita $\boldsymbol{\varepsilon}_{n+1}$, resolver para $\{\boldsymbol{\sigma}_{n+1}, \boldsymbol{\varepsilon}^p_{n+1}, \alpha_{n+1}\}$ con discretización implícita:

$$\boldsymbol{\varepsilon}^p_{n+1} = \boldsymbol{\varepsilon}^p_n + \Delta\gamma\,\mathbf{N}_{n+1}, \qquad \alpha_{n+1} = \alpha_n + \sqrt{\tfrac{2}{3}}\,\Delta\gamma$$

más $f(\boldsymbol{\sigma}_{n+1}, \alpha_{n+1}) \le 0$ con la condición de complementariedad.

10. Return mapping plane strain — algoritmo radial cerrado (existente)

Descomposición volumétrica-desviadora del estado de prueba:

$$\boldsymbol{\varepsilon}^{\text{trial}}_{n+1} = \boldsymbol{\varepsilon}_{n+1} - \boldsymbol{\varepsilon}^p_n, \qquad \mathbf{s}^{\text{trial}} = 2G\,\mathbf{e}^{\text{trial}}, \qquad p^{\text{trial}} = K\ \text{tr}(\boldsymbol{\varepsilon}_{n+1})$$

donde $\mathbf{e}^{\text{trial}}$ es la parte desviadora de $\boldsymbol{\varepsilon}^{\text{trial}}_{n+1}$. Función de fluencia trial:

$$f^{\text{trial}} = \|\mathbf{s}^{\text{trial}}\| - \sqrt{\tfrac{2}{3}}\,(\sigma_y + H\,\alpha_n)$$

Si $f^{\text{trial}} \le \text{tol} \to$ paso elástico, $\boldsymbol{\sigma}_{n+1} = \mathbf{C}_e : \boldsymbol{\varepsilon}_{n+1}$, estado intacto.
Si no, el flujo es radial (la dirección no cambia entre trial y final por isotropía + asociado en J2):

$$\Delta\gamma = \frac{f^{\text{trial}}}{2G + \frac{2}{3}H} \qquad \text{(forma cerrada)}$$

$$\mathbf{N} = \frac{\mathbf{s}^{\text{trial}}}{\|\mathbf{s}^{\text{trial}}\|}, \quad \mathbf{s}_{n+1} = \mathbf{s}^{\text{trial}} - 2G\,\Delta\gamma\,\mathbf{N}, \quad \boldsymbol{\sigma}_{n+1} = \mathbf{s}_{n+1} + p^{\text{trial}}\,\mathbf{I}$$

Tangente consistente algorítmica (derivada del algoritmo, no de la ley continua):

$$\mathbf{C}^{\text{alg}} = K\,\mathbf{1}\otimes\mathbf{1} + 2G(1-\beta)\,\mathbf{I}_{\text{dev}} - 2G\,\bar{\gamma}\,\mathbf{N}\otimes\mathbf{N}$$

con $\beta = \dfrac{2G\,\Delta\gamma}{\|\mathbf{s}^{\text{trial}}\|}$, $\bar{\gamma} = \dfrac{1}{1 + H/(3G)} - \beta$.

11. Return mapping plane stress — algoritmo proyectado (a implementar)

Referencia: Simó & Hughes 1998, *Computational Inelasticity*, §3.4.1 ("Plane stress projected algorithm").

La restricción $\sigma_{zz} = 0$ se impone **eliminando** ε_{zz} del problema mediante un operador de proyección, no como ecuación extra. Resultado: el problema local sigue siendo escalar en $\Delta\gamma$, sin nested Newton.

Matriz elástica plane stress (3×3, base Voigt $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$):

$$\mathbf{C}^{ps}_e = \frac{E}{1-\nu^2}\begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1-\nu)/2 \end{bmatrix}$$

Predictor elástico en el subespacio plane stress:

$$\boldsymbol{\sigma}^{\text{trial}} = \mathbf{C}_e^{ps}\,(\boldsymbol{\varepsilon}_{n+1} - \mathbf{P}_\varepsilon\,\boldsymbol{\varepsilon}_n^p)$$

con $\mathbf{P}_\varepsilon$ extrayendo las componentes plane $[\varepsilon_{xx}^p, \varepsilon_{yy}^p, 2\varepsilon_{xy}^p]$ del tensor 4D.

Operador P — norma desviadora bajo $\sigma_{zz} = 0$ (Simó-Hughes Box 3.1):

$$\mathbf{P} = \frac{1}{3}\begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & 0 \\ 0 & 0 & 6 \end{bmatrix}$$

Esta matriz codifica el producto escalar desviador en el subespacio plane stress; reemplaza $\|\mathbf{s}\|^2$ por $\boldsymbol{\sigma}^\top \mathbf{P}\,\boldsymbol{\sigma}$ con todas las componentes 3D ya eliminadas analíticamente.

Función de fluencia plane stress proyectada:

$$\bar{f}(\boldsymbol{\sigma}, \alpha) = \tfrac{1}{2}\,\boldsymbol{\sigma}^\top \mathbf{P}\,\boldsymbol{\sigma} - \tfrac{1}{3}\big(\sigma_y + H\,\alpha\big)^2 \;\le\; 0$$

Esquema de retorno:

$$\boldsymbol{\sigma}_{n+1} = \big[\mathbf{C}_e^{ps}\big]^{-1}\big(\boldsymbol{\sigma}^{\text{trial}}\big) - \Delta\gamma\,\mathbf{P}\,\boldsymbol{\sigma}_{n+1}$$

reordenando:

$$\boldsymbol{\sigma}_{n+1} = \mathbf{A}(\Delta\gamma)^{-1}\,\boldsymbol{\sigma}^{\text{trial}}, \qquad \mathbf{A}(\Delta\gamma) = \mathbf{I} + \Delta\gamma\,\mathbf{C}_e^{ps}\,\mathbf{P}$$

La matriz $\mathbf{A}$ es 3×3, diagonalizable en los autovectores ortonormales de $\mathbf{C}_e^{ps}\mathbf{P}$. Los autovalores son cerrados:

$$\mu_1 = \frac{E}{3(1-\nu)}, \quad \mu_2 = 2G, \quad \mu_3 = 2G$$

con autovectores $\mathbf{v}_1 = \frac{1}{\sqrt{2}}[1,1,0]$ (hidrostático plano), $\mathbf{v}_2 = \frac{1}{\sqrt{2}}[1,-1,0]$ (desviador plano), $\mathbf{v}_3 = [0,0,1]$ (cortante). Diagonalizan simultáneamente $\mathbf{C}_e^{ps}$ y $\mathbf{P}$, con $\mathbf{v}_i^\top \mathbf{P}\,\mathbf{v}_i \in \{1/3, 1, 2\}$.

Actualización de la deformación plástica acumulada equivalente. La regla $\dot{\alpha} = \sqrt{2/3}\,\dot{\gamma}$ usada en plane strain **no** es válida aquí: en plane stress projected $\dot{\gamma}$ tiene unidades [1/esfuerzo] (porque $\dot{\boldsymbol{\varepsilon}}^p = \dot{\gamma}\,\mathbf{P}\boldsymbol{\sigma}$ con $\mathbf{P}\boldsymbol{\sigma}$ en unidades de esfuerzo). La fórmula físicamente correcta y dimensionalmente consistente, obtenida de $\dot{\alpha} = \sqrt{2/3}\,\|\dot{\boldsymbol{\varepsilon}}^p\|_{\text{Frob}}$:

$$\alpha_{n+1} = \alpha_n + \Delta\gamma\,\sqrt{\tfrac{2}{3}\,\boldsymbol{\sigma}_{n+1}^\top \mathbf{P}\,\boldsymbol{\sigma}_{n+1}}$$

Newton local sobre $\Delta\gamma$. En base autovectores $\sigma_i(\Delta\gamma) = a_i/(1 + \Delta\gamma\,\mu_i)$, así $\boldsymbol{\sigma}^\top \mathbf{P}\,\boldsymbol{\sigma}$ se expresa cerrado:

$$\boldsymbol{\sigma}^\top \mathbf{P}\,\boldsymbol{\sigma}(\Delta\gamma) = \frac{a_1^2}{3(1+\Delta\gamma\mu_1)^2} + \frac{a_2^2}{(1+\Delta\gamma\mu_2)^2} + \frac{2\,a_3^2}{(1+\Delta\gamma\mu_3)^2}$$

donde a_i son las proyecciones de $\boldsymbol{\sigma}^{\text{trial}}$ en los autovectores. La ecuación residual:

$$\bar{f}(\Delta\gamma) = \tfrac{1}{2}\,\boldsymbol{\sigma}^\top \mathbf{P}\,\boldsymbol{\sigma}(\Delta\gamma) - \tfrac{1}{3}\big(\sigma_y + H\,\alpha(\Delta\gamma)\big)^2 = 0$$

con $\alpha(\Delta\gamma) = \alpha_n + \Delta\gamma\sqrt{(2/3)\,\boldsymbol{\sigma}^\top \mathbf{P}\,\boldsymbol{\sigma}}$ es monótona decreciente; tangente cerrada con regla de la cadena. **Newton local converge** típicamente en 3-6 iteraciones desde $\Delta\gamma = 0$. Coste: O(1) por punto de Gauss, comparable a plane strain.

Tangente consistente plane stress. Sea $\mathbf{D} = \mathbf{A}^{-1}\mathbf{C}_e^{ps}$, $\mathbf{q} = \boldsymbol{\sigma}_{n+1}^\top \mathbf{P}\,\mathbf{D}\,\mathbf{P}\,\boldsymbol{\sigma}_{n+1}$, $w = \sqrt{(2/3)\boldsymbol{\sigma}_{n+1}^\top \mathbf{P}\,\boldsymbol{\sigma}_{n+1}}$, $m = (2/3)\,R\,H$, $n = 2\Delta\gamma/(3w)$. De la condición de consistencia ($\dot{\bar{f}} = 0$) con la regla correcta de α:

$$\mathbf{C}_{ps}^{\text{alg}} = \mathbf{D} - \frac{(\mathbf{D}\,\mathbf{P}\boldsymbol{\sigma}_{n+1})\,(\boldsymbol{\sigma}_{n+1}^\top \mathbf{P}\,\mathbf{D})}{q + m\,w/(1 - m\,n)}$$

Si $H = 0$ (plasticidad perfecta) el término m se anula y el denominador se reduce a q. Forma cerrada — sin diferenciación numérica.

Predictor elástico con $\boldsymbol{\varepsilon}^p \neq 0$. Tanto en plane strain como en plane stress, el predictor debe construirse como $\boldsymbol{\sigma}^{\text{trial}} = \mathbf{C}_e\,(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_n^p)$ (o equivalente desviador + presión), **no** como $\mathbf{C}_e\,\boldsymbol{\varepsilon}$, que ignoraría la deformación plástica acumulada. Importante en descargas elásticas desde estado plástico y al reevaluar el material con `compute_gauss_state(U_final)` tras un análisis converged.

12. Tolerancia de fluencia

Vía ADR 0006 (`Material.admissibility_tol`), patrón atol + rtol·escala:

- **Plane strain**: `admissibility_scale =` $\sqrt{(2/3)}$`(`σ`_y + H·`α`)` (norma desviadora característica en la frontera).
- **Plane stress**: `admissibility_scale =` $(\sigma$`_y + H·`$\alpha)^2$`/3` (escala de $\bar{f}$ proyectada; los dos términos son del mismo orden).

10.5.3 Contrato de implementación

```yaml
name: VonMises2D
kind: material
status: validated

interface:
  strain_dim: 3
  primary_state_var: alpha   # deformación plástica acumulada equivalente

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_souza_neto: "cap. 7 (deformación plana) y cap. 9 (esfuerzo plano)"
    - de_borst_crisfield: "§7.2-7.4 (J2: retorno y tangente consistente)"
  general_codes:
    - abaqus: "*PLASTIC (Mises) en CPE* / CPS*"
    - ansys: "TB,BISO"

parameters:
  - { name: E,          type: float, required: true,  desc: "Módulo de Young" }
  - { name: nu,         type: float, required: true,  desc: "Coeficiente de Poisson" }
  - { name: sigma_y,    type: float, required: true,  desc: "Tensión de fluencia
      inicial" }
```

```
  - { name: H,          type: float, required: false, default: 0.0, desc: "Módulo de
      endurecimiento isótropo lineal ≥(0). H=0 ⇒ plasticidad perfecta" }
  - { name: hypothesis, type: str,   required: false, default: "plane_strain", desc: "
      plane_strain | plane_stress" }
  - { name: density,    type: float, required: false, default: null, desc: "Densidad (
      kg/m^3). Opcional al construir; obligatoria solo si se ensambla peso propio o masa
       (ADR 0008)" }

signature:
  compute_state: "ε(: ndarray(3,), state_vars=None) -> σ(: ndarray(3,), C_alg: ndarray
      (3,3), state_vars')"
  strain_kind: "plano Voigt -ε [_xx, ε_yy, γ_xy] con γ_xy = 2·ε_xy"

state_schema:
  eps_p:   "ndarray(4,) -ε [^p_xx, ε^p_yy, ε^p_zz, ε^p_xy] (xy tensorial, sin factor 2)
      "
  alpha:   "float ≥ 0 - deformación plástica acumulada equivalente"

conventions:
  sign: "σ > 0 ⇔ tracción (Reglas §5)"
  voigt: "γ_xy = 2·ε_xy en deformación total; ε^p_xy almacenada en componente tensorial
       (sin factor 2)"
  hypothesis_dispatch: "selección por kwarg al construir; mutuamente excluyentes"

validity:
  - "E > 0, σ_y > 0, H ≥ 0"
  - "ν ∈ (-1, 0.5) en plane_strain; ν∈  (-1, 0.5) en plane_stress ν(=0.5 no admitido
      por singularidad)"
  - "ε|| ≲ 1e-2 (régimen de pequeñas deformaciones)"
  - "hypothesis ∈ {'plane_strain', 'plane_stress'}"

out_of_scope:
  - "endurecimiento cinemático (back-stress); usar variante futura VonMises2DKinematic"
  - "endurecimiento isótropo no lineal (Voce, exponencial)"
  - "endurecimiento por ablandamiento (H<0); requiere regularización (longitud caracter
      ística, no-local)"
  - "regla de flujo no asociada"
  - "acoplamiento con daño; usar modelo plástico-dañado dedicado"
  - "viscoplasticidad y dependencia de la velocidad de deformación"
  - "grandes deformaciones (descomposición multiplicativa F = F^e·F^p)"
  - "anisotropía (Hill, Barlat) - el modelo es estrictamente J2 isótropo"

numerical_caveats:
  - "Plane strain: incompresibilidad plástica genera locking volumétrico en elementos
      de bajo orden (Quad4, Tri3) cuando ν→0.5 o el campo plástico domina. Mitigaciones
       futuras: B-bar, F-bar, elementos mixtos u-p. No incluidas en esta spec."
  - "Plane stress: el Newton local sobre Δγ asume H ≥ 0; con H = 0 (perfectamente plá
      stico) la ecuación residual sigue siendo monótona pero su pendiente al final del
      paso plástico tiende a 0 - convergencia más lenta. Mitigación: bisección de
      respaldo si Newton local diverge."
  - "Para ν cercano a 0.5 en plane strain, K →∞  y el predictor elástico pierde precisi
      ón numérica. Recomendado ν≤  0.49 en metales (típico ν=0.3)."

acceptance:
  # Cubierto por tests/test_materials_unit.py · TestVonMises2D y
      TestVonMises2DPlaneStress
```

```
unit_plane_strain:
  - name: paso_elastico_bajo_fluencia
    expect: "ε=[1e-5,0,0]: σ=C_e·ε exacto, alpha=0, tangente=C_e"
    tol_rel: 1.0e-10
  - name: fluencia_traccion_uniaxial_confinada
    expect: "fluencia activa con ε=[0.01,-0.01,0]: alpha>0 tras un solo paso"
  - name: monotonicidad_alpha
    expect: "α no decrece bajo carga creciente con eps_yyε=-; α_final > 0"
  - name: no_further_yield_on_frontier
    expect: "repetir ε en frontera de fluencia no incrementa α (return mapping
        consistente)"
    tol_rel: 1.0e-10
  - name: hypothesis_validation
    expect: "constructor rechaza hypothesis distinto de 'plane_strain'/'plane_stress'
         con ValueError"

unit_plane_stress:
  - name: paso_elastico_bajo_fluencia
    expect: "ε pequeño: σ=C_eˆps·ε, tangente=C_eˆps, alpha=0, eps_p=0"
    tol_rel: 1.0e-10
  - name: degeneracion_a_1d_traccion_pura_elastico
    expect: "ε=(epsν,-·eps,0): σ_xx=E·eps, σ≈_yy0 (regimen elástico, no Eν/(1-²))"
    tol_rel: 1.0e-4
  - name: yield_uniaxial_at_sigma_y
    expect: "primer yield en σ_xxσ=_y bajo tracción uniaxial pura (no Eν/(1-²)·ε)"
    tol_rel: 1.0e-2
  - name: traccion_biaxial_isotropa
    expect: "fluencia biaxial isótropa exactamente en σ_xxσ=_yyσ=_y (||sσ||=·√(2/3)
        iguala √(2/3)σ_y); ε_yieldσ=_yν(1-)/E"
    tol_rel: 1.0e-2
  - name: incompresibilidad_plastica
    expect: "trε(ˆp) = εˆp_xx + εˆp_yy + εˆp_zz = 0 tras cualquier paso plástico"
    tol_abs: 1.0e-12
  - name: alpha_monotonic
    expect: "α no decrece bajo carga monotónica creciente"
  - name: descarga_recupera_C_e_ps
    expect: "tras estado plástico, paso siguiente con menor ε produce tangente=C_eˆps
         y α intacto"
    tol_rel: 1.0e-10
  - name: tangente_simetrica
    expect: "C_alg simétrica (J2 asociado ⇒ IS_SYMMETRIC=True)"
    tol_abs: 1.0e-8
  - name: unit_invariance_MPa_vs_Pa
    expect: "mismo path adimensional en (MPa,mm,N) y (Pa,m,N) ⇒α  y εˆp idénticos"
    tol_rel: 1.0e-12

# Cubierto por tests/test_solid_2d_plasticity.py
integration:
  - name: pipeline_plane_strain_elastico
    setup: "Quad4 unitario, confinamiento uy=0; tracción horizontal por debajo del
        yield"
    expect: "u_x = ε_xx_target exacto; α=0"
    tol_rel: 1.0e-8

  - name: pipeline_plane_strain_plastico_vs_material_directo
    setup: "mismo setup, carga 1.5×σ_xx_yield → régimen plástico; integrar 10 pasos"
```

```
      expect: "σ_xx FEM (gauss avg) coincide con material standalone en ε_xx_final bajo
          trayectoria monótona; α coincide"
      tol_rel: 1.0e-3

    - name: pipeline_plane_stress_elastico_uniaxial
      setup: "Quad4 unitario, lado izq apoyado en ux; nodo (0,0) y (1,0) en uy; nodo
          (1,1) libre en y. Tracción horizontal por debajo del yield"
      expect: "u_x = ε_xx_target; contracción transversal u_y(0,1) = ν·ε_xx (Poisson);
          α=0"
      tol_rel: 1.0e-6

    - name: pipeline_plane_stress_plastico_vs_1d
      setup: "mismo setup, carga σ1.2_y para entrar plástico; comparar contra
          Elastoplastic1D bajo mismo path ε_xx"
      expect: "σ_xx FEM = σ_1D, σ_yy FEM ≈ 0 (uniaxial puro emergente); α_FEM = α_1D"
      tol_rel: 1.0e-3

    - name: pipeline_converges_few_iter_plane_strain
      setup: "carga plástica significativa, 4 incrementos, max_iter=8"
      expect: "solver converge en todos los pasos sin agotar max_iter; alpha final > 0"

    - name: pipeline_converges_few_iter_plane_stress
      setup: "tracción uniaxial pura plane stress σ1.5_y, 4 incrementos, max_iter=8"
      expect: "solver converge en todos los pasos; alpha final > 0"

references:
  - "Simó J.C., Hughes T.J.R. (1998). Computational Inelasticity. Springer. §3.3 (plane
      strain return mapping), §3.4 (plane stress projected algorithm), Box 3.1 (
      operador P)."
  - "Crisfield M.A. (1991). Non-linear Finite Element Analysis of Solids and Structures
      , Vol. 1. Wiley. Cap. 6 (J2 plasticity)."
  - "Chen W.F., Han D.J. (1988). Plasticity for Structural Engineers. Springer. Cap. 5
      (cilindro a presión interna, sol. cerrada Hill)."
  - "de Souza Neto E.A., ćPeri D., Owen D.R.J. (2008). Computational Methods for
      Plasticity. Wiley. §9.4 (plane stress projection alternativa)."
```

10.5.4 Implementación

- **Archivo**: solidum/materials/von_mises_2d.py.
- **Clase**: `VonMises2D`, registrada vía `@MaterialRegistry.register`. Despacho por `hypothesis` en construcción (sin coste runtime).
- **Kernels Numba**:
- `_compute_j2_plane_strain`: descomposición volumétrica-desviadora 3D (ε^p_{zz} libre), return mapping radial cerrado, tangente algorítmica consistente cerrada. Predictor elástico construido como $\mathbf{s}_{\text{trial}} + p\mathbf{I}$ con $\mathbf{s}_{\text{trial}} = 2G(\mathbf{e}^{\text{dev}} - \boldsymbol{\varepsilon}^p_n)$ — respeta $\boldsymbol{\varepsilon}^p_n$ en descargas/reevaluaciones.
- `_compute_j2_plane_stress`: predictor $\boldsymbol{\sigma}^{\text{trial}} = \mathbf{C}^{ps}_e(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p_n)$, función de fluencia $\bar{f}$ proyectada, Newton local sobre $\Delta\gamma$ con tangente cerrada y máximo `_PLANE_STRESS_MAX_LOCAL_ITER = 25` iteraciones. Construcción de $\mathbf{A}^{-1}$ y σ_new vía base de autovectores conocida $(\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3)$; incompresibilidad plástica cierra $\varepsilon^p_{zz} = -(\varepsilon^p_{xx} + \varepsilon^p_{yy})$.

- **State schema**: `eps_p` ndarray(4,) `[xx, yy, zz, xy_tensorial]`, `alpha` escalar. Las cuatro componentes de `eps_p` son obligatorias en ambas hipótesis para preservar invariantes (incompresibilidad, normas tensoriales).
- `admissibility_scale`:
- plane_strain: $\sqrt{(2/3)\cdot(\sigma_y + H\cdot\alpha)}$
- plane_stress: $(\sigma_y + H\cdot\alpha)^2/3$
- **Tests**:
- tests/test_materials_unit.py · `TestVonMises2D` (5 tests plane strain pre-existentes, no regresión) + `TestVonMises2DPlaneStress` (9 tests plane stress).
- tests/test_solid_2d_plasticity.py · 6 tests de integración del pipeline Quad4 + VonMises2D + NonlinearSolver en ambas hipótesis.
- tests/test_density_self_weight.py · `test_vonmises2d_density` — densidad ADR 0008.
- **Notas de traducción**:
- El kernel plane stress trabaja con $\boldsymbol{\sigma}$ en Voigt mixto: las componentes $\boldsymbol{\sigma} = [\sigma_{xx}, \sigma_{yy}, \sigma_{xy}]$ son las "engineering" mientras que $\boldsymbol{\varepsilon}$ usa $\gamma_{xy} = 2\varepsilon_{xy}$. El operador $\mathbf{P}$ en esta convención tiene la forma con 6 en la entrada (3,3); $\mathbf{P}\boldsymbol{\sigma}$ devuelve la componente cortante como $2\sigma_{xy}$ (convención engineering), que se convierte a tensorial dividiendo por 2 antes de acumular en ε^p_{xy}.
- La actualización $\alpha_{n+1} = \alpha_n + \Delta\gamma\sqrt{(2/3)\boldsymbol{\sigma}^\top\mathbf{P}\boldsymbol{\sigma}}$ se evalúa **con $\boldsymbol{\sigma}$ final** del paso plástico (tras converger Newton local). Durante las iteraciones se usa la misma fórmula con el $\boldsymbol{\sigma}$ corriente.
- Validación de parámetros en constructor: `E>0`, σ`_y>0`, `H`$\geq$`0`, $\nu\in$`(-1,0.5)`, `hypothesis` $\in$ `{plane_strain, plane_stress}`. Mensajes en español describiendo la violación.

Bug fixes detectados al implementar

- **Predictor elástico plane strain ignoraba ε^p**. Versión previa devolvía $\boldsymbol{\sigma} = \mathbf{C}_e\boldsymbol{\varepsilon}$ cuando $f^{\text{trial}} \leq 0$ — correcto solo si $\boldsymbol{\varepsilon}^p = 0$. En descargas elásticas desde estado plástico, o al reevaluar el material vía `compute_gauss_state(U_final)` tras un análisis converged, devolvía σ inconsistente con el estado interno. Corregido a $\boldsymbol{\sigma} = \mathbf{s}_{\text{trial}} + p\mathbf{I}$. Detectado por el test de integración `test_plastic_regime_matches_standalone_material`: σ FEM reportado coincidía con el valor de equilibrio (F/A) en Newton (donde sí entra a return mapping), pero `compute_gauss_state` post-converged daba el valor incorrecto del predictor elástico, divergiendo del material standalone.

10.5.5 Diálogo

- **2026-05-14** · Spec creada retroactivamente sobre material existente (J2 plane strain) y extendida con J2 plane stress. Decisión de algoritmo plane stress: **proyección de Simó-Hughes §3.4.1** en lugar de nested Newton sobre ε_{zz}. Justificación: forma cerrada del problema local (Newton local escalar sobre $\Delta\gamma$ con tangente cerrada, 3-6 iteraciones, O(1) por Gauss);

Numba-compatible (kernel plano); tangente consistente cerrada vía operador $\mathbf{P}$. Nested Newton implicaría ~5-10× el coste por anidación de return mapping completo dentro de cada iteración externa sobre $\sigma_{zz} = 0$, justificable solo con endurecimiento no lineal complejo (Voce, plasticidad mixta) que no es el caso. Registrado aquí, no en ADR — afecta a un material, no a contratos arquitecturales transversales.

- **2026-05-14** · Corrección durante la implementación: la regla heurística $\alpha_{n+1} = \alpha_n + \sqrt{2/3}\,\Delta\gamma$ heredada de plane strain rompe la invariancia bajo cambio de unidades en plane stress, porque allí $\Delta\gamma$ tiene unidades [1/esfuerzo] (la regla de flujo $\dot{\boldsymbol{\varepsilon}}^p = \dot{\gamma}\,\mathbf{P}\boldsymbol{\sigma}$ tiene $\mathbf{P}\boldsymbol{\sigma}$ con dimensión de esfuerzo). La fórmula físicamente correcta es $\alpha_{n+1} = \alpha_n + \Delta\gamma\,\sqrt{(2/3)\boldsymbol{\sigma}^\top\mathbf{P}\boldsymbol{\sigma}}$ — adimensional. Detectado por el test unitario `test_unit_invariance_MPa_vs_Pa` que daba un factor 1e6 entre los dos sistemas. Implica recalcular la tangente consistente con $\partial\alpha/\partial\Delta\gamma \neq \sqrt{2/3}$ (ver §11).

- **2026-05-14** · Corrección plane strain: el predictor elástico devolvía $\boldsymbol{\sigma} = \mathbf{C}_e\boldsymbol{\varepsilon}$, ignorando $\boldsymbol{\varepsilon}^p_n$. Bug latente preexistente sin tests que lo cubrieran (los unitarios solo probaban primer paso con $\boldsymbol{\varepsilon}^p = 0$). Detectado al validar el pipeline FEM: `compute_gauss_state(U_final)` reportaba σ del predictor elástico (incorrecto) en lugar del σ post return-mapping. Corregido a $\boldsymbol{\sigma} = \mathbf{s}_{\text{trial}} + p\mathbf{I}$. La rama plane stress ya lo hacía bien desde el principio (predictor incluía la sustracción de $\boldsymbol{\varepsilon}^p$).

- **2026-05-14** · Validación numérica de plane strain queda implícita en los tests unitarios existentes; los tests de integración `tests/test_solid_2d_plasticity.py` abren por primera vez el pipeline completo Quad4 + VonMises2D + NonlinearSolver en ambas hipótesis — hasta hoy no había cobertura de integración sólido 2D + material plástico.

- **2026-05-14** · No se introduce ADR. La spec no rompe contratos: extiende un material existente con una rama nueva del despacho por `hypothesis`. Sí se actualiza `docs/catalogo_materiales.md` al validar.

- **2026-09-22** · Auditoría global: Newton local del return mapping plane stress: tolerancia recalculada cada iteración con la escala corriente (`tol_abs + tol_rel·R_curr`^{2}`/3`, ADR 0006; con H > 0 la tolerancia fija en α_old quedaba bajo el redondeo y el bucle agotaba siempre el presupuesto) y warm start en la cota inferior de la raíz para H = 0 ($\Delta\gamma_0$ `= (`$\sqrt{}$`(3·½σP`σ`_trial)/R - 1)/`μ`_max`; desde $\Delta\gamma = 0$ con predictores lejanos salía sin converger y sin avisar). El agotamiento se registra con WARNING (una vez por instancia). Nuevo `out_of_plane_stress` en plane strain desde ε^p_zz, validado contra `VonMises3D` con ε_zz = 0. Kernels con `cache=True`.

- **2026-09-22** · Ensamblaje por lotes (ADR 0014): declara `STATE_SCHEMA` y un adaptador `@njit` con firma `MAT_SIG` que llama al mismo return mapping; la tolerancia de admisibilidad (ADR 0006) se evalúa dentro del kernel con las mismas operaciones que `admissibility_tol`. Formulación sin cambios; equivalencia con `compute_state` exacta en esfuerzo, tangente y estado (`tests/test_batch_kernels.py`). El return mapping se reescribió sin asignaciones de memoria (`_j2_*_core`) con el mismo orden de operaciones: verificado bit a bit sobre 20 000 estados aleatorios en ramas elástica y plástica.

- **2026-09-22** · Cierre del ADR 0014 (§9): `batch_out_of_plane_stress(sigma, S)`, versión por arreglos de `out_of_plane_stress` para el post-proceso por familia (exportador VTK);

misma fórmula, ε`^p_zz` leída de la fila de estado cuando aplica. Verificado archivo VTK idéntico por ambos caminos.

Capítulo 11

Modelos Constitutivos — 3D

11.1 DruckerPrager3D

*Orden de trabajo. El usuario revisa **especificación física**, **formulación numérica** y **contrato**; la IA propone, ejecuta y rellena **implementación** + **diálogo**.*

***Pre-redactada por la IA** sobre la base de `DruckerPrager2D.md` (mismo modelo físico) y `VonMises3D.md` (mismo patrón Voigt 6D). Segunda entrega de la sub-etapa **A.bis** (materiales 3D no lineales).*

11.1.1 Especificación física

0. Descripción general

Modelo elastoplástico **friccional** independiente de la velocidad. Suelos, hormigón, granulares, materiales cuasi-frágiles cohesivo-friccionales en régimen 3D completo. Suaviza la pirámide hexagonal de Mohr-Coulomb a un **cono circular** en el espacio de esfuerzos principales, eliminando las aristas que complican el return mapping. Formulado íntegramente en Voigt 6D del proyecto (ADR 0012).

Diferencias clave con J2 3D (`VonMises3D`):

- **Dependencia de la presión**. La resistencia al corte aumenta con la presión de confinamiento (componente friccional).
- **Plasticidad no asociada**. La dilatancia volumétrica real del flujo plástico (ψ) es típicamente menor que la fricción del criterio (ϕ). Forzar $\psi = \phi$ (asociada) sobreestima la dilatancia.
- **Retorno al ápice**. El criterio define un cono con vértice; estados de prueba en zona puramente hidrostática extrema retornan al vértice (zona singular en la dirección de flujo).

Comparado con `DruckerPrager2D`:

- **Sin variantes de hipótesis** (igual que VM3D): en 3D todas las componentes son activas.
- **El catálogo de calibraciones cambia**: la variante 2D `plane_strain_matched` no tiene análogo en 3D (Mohr-Coulomb en 3D es una pirámide hexagonal sin coincidencia circular global). Las calibraciones 3D disponibles son `outer_cone` (circunscribe MC en el meridiano de compresión, **default**) e `inner_cone` (inscribe MC en el meridiano de extensión).

1. Descomposición aditiva y elasticidad

Pequeñas deformaciones: $\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e + \boldsymbol{\varepsilon}^p$. Ley elástica isótropa con la matriz $\mathbf{C}_e$ 6×6 de `Elastic3D`:

$$\boldsymbol{\sigma} = \mathbf{C}_e\,(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p)$$

equivalente desviador + presión:

$$\boldsymbol{\sigma} = \mathbf{s} + p\,\mathbf{I}, \qquad \mathbf{s} = 2G\,\mathbf{e}^e_{\text{dev}}, \qquad p = K\ \text{tr}(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p)$$

A diferencia de J2, $\text{tr}(\boldsymbol{\varepsilon}^p) \neq 0$ en general (dilatancia plástica si $\eta_g > 0$), por lo que p depende de la deformación plástica acumulada vía $\text{tr}(\boldsymbol{\varepsilon}^p)$.

2. Criterio de fluencia (Drucker-Prager)

$$f(\boldsymbol{\sigma}, \alpha) = \sqrt{J_2} + \eta_f\, I_1 - k(\alpha)\ \leq\ 0$$

con:

- $J_2 = \frac{1}{2}\,\mathbf{s} : \mathbf{s}$, segundo invariante del desviador.
- $I_1 = \text{tr}(\boldsymbol{\sigma}) = \sigma_{xx} + \sigma_{yy} + \sigma_{zz}$, primer invariante.
- η_f, coeficiente friccional del cono (adimensional, ≥ 0).
- $k(\alpha) = k_0 + H_k\,\alpha$, radio del cono (el parámetro de cohesión del criterio: k_0 sale de c_0 según la calibración, §3) con endurecimiento isótropo lineal en el multiplicador plástico acumulado α (§5).

La superficie es un **cono circular** con eje en la línea hidrostática ($\sigma_{xx} = \sigma_{yy} = \sigma_{zz}$), vértice en $I_1 = k/\eta_f$, abertura controlada por η_f.

Cómputo de J_2 en Voigt 6D del proyecto (componentes cortantes tensoriales sin factor 2):

$$J_2 = \frac{1}{2}\left(s_{xx}^2 + s_{yy}^2 + s_{zz}^2 + 2\,(s_{xy}^2 + s_{yz}^2 + s_{xz}^2)\right)$$

El factor 2 contabiliza las dos posiciones simétricas del tensor.

3. Calibración con Mohr-Coulomb (3D)

Los parámetros (η_f, k_0) se derivan de los parámetros físicos (c_0, ϕ) — cohesión y ángulo de fricción interna — según la variante del cono elegida. En 3D, **el cono no puede coincidir con la pirámide hexagonal de MC en todo el plano** π (las aristas de MC quedan fuera o dentro del círculo), por lo que las calibraciones eligen un meridiano de referencia:

variant	η_f	k_0	Comportamiento
`outer_cone` (default)	$2\sin(\phi)/[\sqrt{3}\,(3-\sin(\phi))]$	$6\,c_0\cos(\phi)/[\sqrt{3}\,(3-\sin(\phi))]$	Circunscribe MC en el meridiano de compresión triaxial ($\theta_L=-30°$). Más conservador en compresión, menos conservador en extensión.
`inner_cone`	$2\sin(\phi)/[\sqrt{3}\,(3+\sin(\phi))]$	$6\,c_0\cos(\phi)/[\sqrt{3}\,(3+\sin(\phi))]$	Inscribe MC en el meridiano de extensión triaxial ($\theta_L=+30°$). Cota inferior segura para diseño con MC.

El default `outer_cone` se elige porque la compresión triaxial es el modo dominante en problemas geotécnicos típicos (cimentaciones, muros de contención, taludes). Si la aplicación prioriza la extensión (e.g. excavaciones), `inner_cone` es la elección segura.

Caveat fundamental: el cono no captura efectos de ángulo de Lode (la pirámide hexagonal de MC sí). Para problemas donde el estado de esfuerzo rota fuera de los meridianos puros, considerar Mohr-Coulomb 3D directo (out-of-scope) o variantes con dependencia explícita del Lode angle (e.g. Matsuoka-Nakai, también out-of-scope).

No se incluye `plane_strain_matched` (variante 2D de DP2D): es un artefacto matemático que coincide con MC exactamente solo cuando $\varepsilon_{zz}=0$ está impuesto cinemáticamente; no tiene análogo 3D porque MC en 3D depende del Lode angle activo.

4. Regla de flujo (no asociada por defecto)

Potencial plástico g con coeficiente de **dilatancia** η_g (función del ángulo de dilatancia ψ, calibrado con la misma fórmula que η_f pero usando ψ):

$$g(\boldsymbol{\sigma}) = \sqrt{J_2} + \eta_g\, I_1$$

$$\dot{\boldsymbol{\varepsilon}}^p = \dot{\gamma}\,\frac{\partial g}{\partial \boldsymbol{\sigma}} = \dot{\gamma}\,\left(\frac{\mathbf{s}}{2\sqrt{J_2}} + \eta_g\,\mathbf{I}\right)$$

Descomposición:

- **Parte desviadora**: $\dot{\mathbf{e}}^p = \dot{\gamma}\,\mathbf{s}/(2\sqrt{J_2})$ — análoga a J2.
- **Parte volumétrica**: $\mathrm{tr}(\dot{\boldsymbol{\varepsilon}}^p) = 3\dot{\gamma}\,\eta_g$ — **dilatación plástica positiva** si $\eta_g > 0$.

Si $\psi=\phi$ (i.e. $\eta_g=\eta_f$) el modelo se vuelve **asociado**; tangente algorítmica simétrica. Para $\psi<\phi$ (típico en suelos: $\psi\approx\phi/2$ o incluso $\psi=0$, "non-dilatant"), el modelo es **no asociado** y la tangente algorítmica es **asimétrica**.

5. Endurecimiento isótropo lineal

$$\dot{\alpha} = \dot{\gamma}$$

con α el **multiplicador plástico acumulado** γ (convención del proyecto para Drucker-Prager). La superficie crece manteniendo eje y abertura, solo k aumenta linealmente. Endurecimiento por fricción/dilatancia (cambio de η_f o η_g con α) queda fuera de esta spec.

Parámetro del usuario (desde el 2026-09-29): la parametrización de de Souza Neto, Perić y Owen (2008, §8.3). La cohesión se endurece con la deformación plástica equivalente, $c(\bar{\varepsilon}^p) = c_0 + H\,\bar{\varepsilon}^p$, con $\bar{\varepsilon}^p = \xi\,\gamma$ y $k = \xi\,c$, donde $\xi = k_0/c_0$ es el factor de la calibración (§3). El parámetro `H` que escribe el usuario es $H = H_c = dc/d\bar{\varepsilon}^p$. El return mapping trabaja con el módulo del radio del cono respecto del multiplicador acumulado, $H_k = dk/d\alpha$, que es

$$H_k = \xi^2\,H_c.$$

6. Condiciones de Kuhn-Tucker

$$\dot{\gamma} \geq 0, \qquad f \leq 0, \qquad \dot{\gamma}\,f = 0$$

7. Variables internas

Símbolo	Tipo	Significado
$\boldsymbol{\varepsilon}^p$	tensor 6 componentes `[xx, yy, zz, xy, yz, xz]` (cortantes tensoriales)	deformación plástica
α	escalar ≥ 0	multiplicador plástico acumulado (acumulado de $\Delta\gamma$; no es una deformación plástica equivalente, §5)

`alpha` es la variable principal exportable (`PRIMARY_STATE_VAR = 'alpha'`). $\boldsymbol{\varepsilon}^p$ **no es incompresible** en este modelo (a diferencia de J2): $\mathrm{tr}(\boldsymbol{\varepsilon}^p) = 3\eta_g\,\alpha$ en cualquier estado, lo que es un invariante cinemático verificable por test.

8. Notación Voigt 6D

Convención del proyecto (ADR 0012):

$$\boldsymbol{\varepsilon} = [\varepsilon_{xx},\,\varepsilon_{yy},\,\varepsilon_{zz},\,\gamma_{xy},\,\gamma_{yz},\,\gamma_{xz}]^\top, \qquad \gamma_{ij} = 2\,\varepsilon_{ij}$$

Almacenamiento interno de $\boldsymbol{\varepsilon}^p$: 6 componentes con cortantes **tensoriales sin factor 2**, idéntica convención a `VonMises3D` (consistencia entre materiales 3D no lineales del proyecto).

9. Relación con `DruckerPrager2D` plane strain

`DruckerPrager2D` plane strain con variante `outer_cone` (o `inner_cone`) es la restricción de este modelo bajo $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ con $\varepsilon^p_{zz} \neq 0$ libre (dilatancia volumétrica). Esta relación se verifica como test cruzado en §13.

No se puede establecer la equivalencia con la variante `plane_strain_matched` de DP2D: esa calibración es 2D-only por construcción. El test cruzado usa `outer_cone` (o `inner_cone`) que es la única variante compartida.

11.1.2 Formulación numérica

10. Esquema implícito — Backward Euler

Dado $\{\boldsymbol{\varepsilon}_n^p, \alpha_n\}$ y $\boldsymbol{\varepsilon}_{n+1}$, predictor elástico. Conversión engineering→tensorial dividiendo los cortantes por 2 (única vez al construir el desviador):

$$\boldsymbol{\varepsilon}_{n+1}^{\text{tens}} = [\varepsilon_{xx},\, \varepsilon_{yy},\, \varepsilon_{zz},\, \tfrac{\gamma_{xy}}{2},\, \tfrac{\gamma_{yz}}{2},\, \tfrac{\gamma_{xz}}{2}]$$

Deformación elástica de prueba (incluye toda la $\boldsymbol{\varepsilon}_n^p$, incluida su parte volumétrica por dilatancia):

$$\boldsymbol{\varepsilon}_{n+1}^{e,\text{trial}} = \boldsymbol{\varepsilon}_{n+1}^{\text{tens}} - \boldsymbol{\varepsilon}_n^p$$

Descomposición volumétrica-desviadora:

$$\text{tr}(\boldsymbol{\varepsilon}_{n+1}^{e,\text{trial}}) = (\varepsilon_{xx} + \varepsilon_{yy} + \varepsilon_{zz}) - (\varepsilon_{xx,n}^p + \varepsilon_{yy,n}^p + \varepsilon_{zz,n}^p)$$

$$\mathbf{e}^{\text{trial}} = \text{dev}(\boldsymbol{\varepsilon}_{n+1}^{e,\text{trial}}), \qquad \mathbf{s}^{\text{trial}} = 2G\,\mathbf{e}^{\text{trial}}, \qquad p^{\text{trial}} = K\ \text{tr}(\boldsymbol{\varepsilon}_{n+1}^{e,\text{trial}})$$

con $I_1^{\text{trial}} = 3\,p^{\text{trial}}$ y $\sqrt{J_2^{\text{trial}}} = \|\mathbf{s}^{\text{trial}}\|/\sqrt{2}$ (norma Frobenius en Voigt 6D tensorial).
Función de fluencia trial:

$$f^{\text{trial}} = \sqrt{J_2^{\text{trial}}} + \eta_f\, I_1^{\text{trial}} - k(\alpha_n)$$

Si $f^{\text{trial}} \le \text{tol}$ → paso elástico. Si no → return mapping.

11. Return regular (cone surface)

La dirección desviadora $\hat{\mathbf{n}} = \mathbf{s}^{\text{trial}}/(2\sqrt{J_2^{\text{trial}}})$ es invariante bajo carga radial. Las actualizaciones son lineales en $\Delta\gamma$:

$$\sqrt{J_2^{n+1}} = \sqrt{J_2^{\text{trial}}} - G\,\Delta\gamma$$

$$p^{n+1} = p^{\text{trial}} - 3K\,\eta_g\,\Delta\gamma, \qquad I_1^{n+1} = 3\,p^{n+1}$$

La consistencia $f(\boldsymbol{\sigma}_{n+1}, \alpha_{n+1}) = 0$ con $\alpha_{n+1} = \alpha_n + \Delta\gamma$ y $k_{n+1} = k_0 + H_k(\alpha_n + \Delta\gamma)$ da:

$$\boxed{\Delta\gamma = \frac{f^{\text{trial}}}{G + 9K\,\eta_f\,\eta_g + H_k}}$$

Forma cerrada — sin Newton local. Idéntica al 2D plane strain en estructura algebraica.
Actualizaciones:

$$\mathbf{s}^{n+1} = \frac{\sqrt{J_2^{n+1}}}{\sqrt{J_2^{\text{trial}}}}\,\mathbf{s}^{\text{trial}}, \qquad \boldsymbol{\sigma}^{n+1} = \mathbf{s}^{n+1} + p^{n+1}\,\mathbf{I}$$

$$\boldsymbol{\varepsilon}^p_{n+1} = \boldsymbol{\varepsilon}^p_n + \Delta\gamma \left(\frac{\mathbf{s}^{n+1}}{2\sqrt{J_2^{n+1}}} + \eta_g \mathbf{I} \right)$$

donde $\mathbf{I} = [1, 1, 1, 0, 0, 0]$ en Voigt 6D (parte volumétrica) y la componente desviadora de $\Delta\boldsymbol{\varepsilon}^p$ se almacena en convención tensorial.

Condición de validez: $\sqrt{J_2^{n+1}} \geq 0$. Si $\Delta\gamma_{\text{reg}} > \sqrt{J_2^{\text{trial}}}/G$, el desviador "se pasa de cero" y el algoritmo regular es inválido → toca **return al ápice**.

12. Return al ápice

Cuando el estado de prueba cae en zona puramente hidrostática extrema (predictor muy traccionante respecto a la cohesión), la proyección al cono cae en su vértice, donde $\mathbf{s} = 0$ y la dirección desviadora $\hat{\mathbf{n}}$ no está definida.

En el ápice: $\boldsymbol{\sigma}^{n+1} = (k_{n+1}/(3\eta_f))\,\mathbf{I}$ (puramente hidrostático), $\sqrt{J_2^{n+1}} = 0$.

$$\boxed{\Delta\gamma_{\text{apex}} = \frac{I_1^{\text{trial}}\,\eta_f - k(\alpha_n)}{9K\,\eta_f\,\eta_g + H_k}}$$

(Equivalente a la fórmula regular sin el término G, porque la rama de apex es puramente volumétrica.)

Actualizaciones (apex):

$$\mathbf{s}^{n+1} = 0, \qquad p^{n+1} = \frac{k_{n+1}}{3\,\eta_f}, \qquad \boldsymbol{\sigma}^{n+1} = p^{n+1}\,\mathbf{I}$$

$$\boldsymbol{\varepsilon}^p_{n+1} = \boldsymbol{\varepsilon}^p_n + \mathbf{e}^{\text{trial}} + \Delta\gamma_{\text{apex}}\,\eta_g\,\mathbf{I}$$

(El desviador trial se absorbe completamente; la parte volumétrica del flujo añade $3\eta_g\Delta\gamma$ a la traza, distribuida $\eta_g\Delta\gamma$ por componente diagonal.)

13. Detección del régimen de retorno

Algoritmo de despacho (idéntico al 2D):

1. Calcular $\Delta\gamma_{\text{reg}}$ por la fórmula regular.
2. Si $\sqrt{J_2^{\text{trial}}} - G\,\Delta\gamma_{\text{reg}} \geq 0$ → return regular.
3. Si no → return al ápice con $\Delta\gamma_{\text{apex}}$.

14. Tangente algorítmica consistente

Return regular ($\sqrt{J_2^{n+1}} > 0$), forma compacta (de Souza Neto §8.3.4 extendida a 6D):

$$\mathbf{C}^{\text{alg}} = K\,\mathbf{v}\otimes\mathbf{v} + 2G\left(1 - \frac{G\Delta\gamma}{\sqrt{J_2^{\text{trial}}}}\right)\mathbf{I}_{\text{dev}} + 4G\,\beta\,\hat{\mathbf{n}}\otimes\hat{\mathbf{n}} - \frac{1}{A}\,\mathbf{b}_g\otimes\mathbf{b}_f$$

con:

- $\mathbf{v} = [1, 1, 1, 0, 0, 0]$ (gradiente de I_1 en Voigt 6D engineering input).
- $\mathbf{I}_{\text{dev}}$: proyector desviador 6×6 con 1/2 en los cortantes (mapea engineering γ_{ij} a tensorial ε_{ij}).
- $\hat{\mathbf{n}} = \mathbf{s}^{\text{trial}}/(2\sqrt{J_2^{\text{trial}}})$ en Voigt 6D tensorial.
- $\beta = G\,\Delta\gamma/\sqrt{J_2^{\text{trial}}}$.
- $\mathbf{b}_f = 2G\,\hat{\mathbf{n}} + 3K\,\eta_f\,\mathbf{v}$, $\mathbf{b}_g = 2G\,\hat{\mathbf{n}} + 3K\,\eta_g\,\mathbf{v}$.
- $A = G + 9K\,\eta_f\,\eta_g + H_k$.

Asimétrica si $\eta_f \neq \eta_g$ (no asociada). El término $+4G\beta\,\hat{\mathbf{n}} \otimes \hat{\mathbf{n}}$ recoge el cambio de dirección de flujo con $\boldsymbol{\varepsilon}$ a través de $\mathbf{s}^{\text{trial}}$ (no solo el cambio de magnitud); sin él la tangente cierra mal en componentes de cortante alineados con el flujo (verificable por diferencia finita, idéntico al hallazgo del DP2D).

Return al ápice ($\sqrt{J_2^{n+1}} = 0$):

$$\mathbf{C}^{\text{alg}}_{\text{apex}} = \frac{K\,H_k}{9K\eta_f\eta_g + H_k}\,\mathbf{v} \otimes \mathbf{v}$$

Rigidez puramente volumétrica reducida; sin rigidez desviadora alguna. Con $H_k = 0$ y $\eta_g > 0$ la rigidez tangente colapsa por completo.

Piso de la tangente en el ápice (idéntico al 2D). Si $KH_k/(9K\eta_f\eta_g + H_k) < 10^{-6}K$, el código devuelve $10^{-6}K\,\mathbf{v}\otimes\mathbf{v}$ en lugar de la tangente exacta, para no dejar la matriz global singular mientras el punto siga en el ápice. Actúa en **todo retorno al ápice con** $H_k = 0$ (perfectamente plástico, el caso más común, donde la tangente exacta es nula) y, en general, siempre que $H_k/(9K\eta_f\eta_g + H_k) < 10^{-6}$; con $\psi = 0$ el retorno al ápice no tiene solución y el kernel lanza `ValueError` antes de llegar aquí (Diálogo, 2026-09-22). Es una regularización de la tangente, no la tangente consistente: no cambia $\boldsymbol{\sigma}$ ni el estado convergido, sólo la convergencia del Newton global.

15. Tolerancia de fluencia

ADR 0006: `admissibility_scale = k(α) = k_0 + H_k·α` (radio corriente del cono). Tiene unidades de esfuerzo, igual que f.

11.1.3 Contrato de implementación

```yaml
name: DruckerPrager3D
kind: material
status: validated        # draft → implemented → validated

interface:
  strain_dim: 6
  primary_state_var: alpha
  is_symmetric: false      # tangente asimétrica si no asociada ψ( ≠φ );
                           # override por instancia a True cuando ψ = φ (mismo
                           # patrón que DP2D - el despachador algebraico ADR
                           # 0003 elige Cholesky/ᵀLDL en problemas asociados).
```

```yaml
admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_souza_neto: "§8.3 (Drucker-Prager: retorno al cono y al ápice, tangentes)"
  general_codes:
    - abaqus: "*DRUCKER PRAGER (lineal)"
    - code_aster: "DRUCK_PRAGER / DRUCK_PRAG_N_A"
    - opensees: "nDMaterial DruckerPrager"

parameters:
  - { name: E,          type: float, required: true,  desc: "Módulo de Young (>0)" }
  - { name: nu,         type: float, required: true,  desc: "Coeficiente de Poisson ∈
      (-1, 0.5)" }
  - { name: cohesion,   type: float, required: true,  desc: "Cohesión c_0 inicial (
      esfuerzo, >0)" }
  - { name: phi_deg,    type: float, required: true,  desc: "Ángulo de fricción interna
       en grados; típico suelos 20-40°, hormigón 30-37°" }
  - { name: psi_deg,    type: float, required: false, default: null,
      desc: "Ángulo de dilatancia en grados. Default = phi_deg (asociada). Típico no
          asociada: ψ < φ, e.g. 0 o φ/2." }
  - { name: H,          type: float, required: false, default: 0.0,
      desc: "Módulo de endurecimiento isótropo lineal de la cohesión H = dc/εᵖd ≥(0),
          parametrización de de Souza Neto et al. (2008, §8.3); internamente H_k = ξ²·H
           con ξ = k_0/c_0 (§5). H=0 ⇒ perfectamente plástico." }
  - { name: variant,    type: str,   required: false, default: "outer_cone",
      desc: "outer_cone (default, circunscribe MC en compresión) | inner_cone (inscribe
           MC en extensión)" }
  - { name: density,    type: float, required: false, default: null, desc: "Densidad (
      ADR 0008)" }

signature:
  compute_state: "ε(: ndarray(6,), state_vars=None) -> σ(: ndarray(6,), C_alg: ndarray
      (6,6), state_vars')"
  strain_kind: "3D Voigt -ε [_xx, ε_yy, ε_zz, γ_xy, γ_yz, γ_xz] con γ_ij = 2·ε_ij"

state_schema:
  eps_p: "ndarray(6,) -ε [^p_xx, ε^p_yy, ε^p_zz, ε^p_xy, ε^p_yz, ε^p_xz] (cortantes
      tensoriales, sin factor 2). trε(^p) = 3·η_g·α invariante cinemático."
  alpha: "float ≥ 0 - multiplicador plástico acumulado γ (no es una deformación plá
      stica equivalente, §5)"

conventions:
  sign:  "σ > 0 ⇔ tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] (ADR 0012); γ_ij = 2·ε_ij engineering input;ε
          ^p_ij almacenada tensorial; σ_ij tensorial sin factor"
  associated_flow: "ψ = φ⇒  asociada (tangente simétrica); ψ≠φ⇒    no asociada (
      tangente asimétrica)"

validity:
  - "E > 0, c_0 > 0, 0 ≤φ  < 90°, 0 ≤ψ≤φ   "
  - "ν ∈ (-1, 0.5) estricto"
  - "H ≥ 0"
  - "variant ∈ {'outer_cone', 'inner_cone'}"

out_of_scope:
```

```
  - "Mohr-Coulomb 3D con aristas (pirámide hexagonal): el cono circular DP no captura
     efectos de ángulo de Lode"
  - "Variantes con dependencia explícita del Lode angle (Matsuoka-Nakai, Lade-Duncan):
     formulaciones distintas"
  - "Calibración 'plane_strain_matched': sin análogo 3D (la coincidencia ↔DPMC en plane
      strain es 2D-only)"
  - "Endurecimiento por cambio de φ y/o ψ con α (no lineal fuerte en return mapping)"
  - "Endurecimiento cinemático (back-stress hidrostático y desviador)"
  - "Cap (modelo cap-cone) para suelos compactables"
  - "Ablandamiento (H<0) - requiere regularización"
  - "Grandes deformaciones"

numerical_caveats:
  - "Tangente algorítmica asimétrica si ψ≠φ→    IS_SYMMETRIC=False, el despachador
     algebraico usa LU (ADR 0003). Cuando ψ = φ se vuelve simétrica y el override por
     instancia activa Cholesky/ᵀLDL."
  - "Modos de carga muy traccionantes pueden caer en la rama de ápice. La detección
     automática del régimen (regular vs apex) está en el algoritmo."
  - "Con H_k = 0 (perfectamente plástico), en la rama de ápice la tangente volumétrica
     exacta se anula → singularidad. El código la sustituye por el piso K·1e-6·⊗vv en
     todo retorno al ápice con H_k/(9ηηK_f_g + H_k) < 1e-6 (§14): regularización de la
      tangente, sin efecto sobre σ; típicamente sale del ápice en el siguiente paso.
     Con ψ = 0 el retorno al ápice no tiene solución y el kernel lanza ValueError."
  - "Locking volumétrico en Hex8/Tet4 cuando ψ > 0 (flujo dilatante) combinado con ν
     moderado-alto. Mismo régimen documentado para Elastic3D/VonMises3D; mitigaciones
     B-bar/F-bar diferidas."
  - "El cono circular DP es independiente del ángulo de Lode. Para trayectorias de
     carga donde el modo rota fuera del meridiano de calibración (compresión triaxial
     → cortante → extensión triaxial), la diferencia DP vs MC puede ser hasta el 15-20%
      en la frontera. Es limitación inherente del modelo, no del algoritmo."

acceptance:
  unit:
    - name: parameter_validation
      setup: "constructor con variant inválida, φ≥  90°, ψ > φ, ≤E0, ≤c_00, H<0,
          ν∉(-1,0.5)"
      expect: "ValueError con mensaje claro en cada caso"

    - name: calibration_outer_inner_consistency
      setup: "comparar η_f de outer_cone vs inner_cone para φ=30°"
      expect: "outer_cone η_f > inner_cone η_f (el outer es geométricamente mayor);
          ambos > 0; k_0 análogo"

    - name: paso_elastico_below_yield
      setup: "ε pequeño 6D, todas las componentes activas"
      expect: "σ = C_e·ε; eps_p y alpha intactos"
      tol_rel: 1.0e-10

    - name: regular_return_under_pure_shear
      setup: "ε cortante puro suficiente para fluir γ(_xy creciente, otras componentes
          0)"
      expect: "return regular activo, α > 0; estado final cumple f ≤ tol"
      tol_rel: 1.0e-8

    - name: apex_return_under_hydrostatic_tension
```

```
    setup: "ε hidrostática traccionante ε(_xx = ε_yy = ε_zz > 0 suficientemente
        grande, sin cortante)"
    expect: "return al ápice; σ_xx = σ_yy = σ_zz = kα()η/(3_f); √₂J ≈ 0"
    tol_rel: 1.0e-8

  - name: tr_eps_p_invariant
    setup: "carga monotónica en rama regular (5 niveles), ψ≠φ   (no asociada con η_g
        > 0)"
    expect: "trε(^p) = 3·η_g·α en cada paso plástico"
    tol_rel: 1.0e-10

  - name: associated_tangent_is_symmetric
    setup: "ψ = φ (asociada), estado plástico activo en rama regular"
    expect: "C_alg simétrica ‖(C_alg - C_alg^‖ϖT_ ≤ tol)"
    tol_abs: 1.0e-8

  - name: nonassociated_tangent_is_asymmetric
    setup: "ψ = 0, φ = 30°, estado plástico activo no degenerado"
    expect: "‖C_alg - C_alg^‖ϖT_ con magnitud significativa relativa a ‖‖ϖC_alg_"

  - name: tangent_matches_finite_difference
    setup: "estado plástico regular; comparar C_alg cerrada con diferencia finita
        centrada"
    expect: "error relativo < 1e-4 en todas las componentes"
    tol_rel: 1.0e-4

  - name: alpha_monotonic
    setup: "carga creciente con plasticidad sostenida"
    expect: "α no decrece nunca"

  - name: unloading_recovers_C_e
    setup: "cargar a plástico, descargar elásticamente"
    expect: "tangente = C_e en el paso de descarga; α se conserva"
    tol_rel: 1.0e-10

  - name: unit_invariance_MPa_vs_Pa
    setup: "mismo path adimensional con (MPa,mm,N) y (Pa,m,N)"
    expect: "α y eps_p (adimensionales) idénticos; σ/E idéntico"
    tol_rel: 1.0e-12

cross_consistency:
  - name: equivalencia_plane_strain_outer_cone
    setup: "DP3D con variant='outer_cone' y ε = ε(_xx, ε_yy, 0, γ_xy, 0, 0);
           DP2D con variant='outer_cone' (plane_strain) y ε = ε(_xx, ε_yy, γ_xy);
           path multi-paso con plasticidad activa en rama regular"
    expect: "σ_xx, σ_yy, σ_xy y α coinciden entre 3D y 2D PS;
            componentes plásticas planas coinciden (xx, yy, zz, xy_tens);ε
            ^p_yz = ε^p_xz = 0 en VM3D (no se activan)"
    tol_rel: 1.0e-10

integration:
  - name: hex8_pipeline_elastico
    setup: "Hex8 unitario, carga muy por debajo del yield"
    expect: "respuesta elástica pura, α=0 en todos los Gauss points"
    tol_rel: 1.0e-8
```

```
    - name: hex8_pipeline_plastico_converges
      setup: "Hex8 unitario con confinamiento + cortante; carga plástica significativa;
          4 pasos, max_iter=8"
      expect: "Newton converge en cada paso; α > 0 al final; tangente asimétrica activa
          LU"

    - name: hex8_apex_under_hydrostatic
      setup: "Hex8 unitario con ε hidrostática traccionante (todos los nodos del cubo
          desplazados radialmente desde el centro)"
      expect: "estado final cae en ápice en los 8 puntos de Gauss: σ_xx ≈σ _yy ≈σ _zz,
          √₂J ≈ 0"
      tol_rel: 1.0e-6

    - name: tet4_basic_pipeline
      setup: "Tet4 con confinamiento + cortante, plasticidad activa"
      expect: "convergencia + α > 0 + trε(^p) = 3·η_g·α invariante en el único Gauss
          point"

references:
  - "Drucker D.C., Prager W. (1952). Soil mechanics and plastic analysis or limit
      design. Quart. Appl. Math. 10, 157-165."
  - "Simó J.C., Hughes T.J.R. (1998). Computational Inelasticity. Springer. §6 (cone
      plasticity, apex return)."
  - "de Souza Neto E.A., ćPeri D., Owen D.R.J. (2008). Computational Methods for
      Plasticity. Wiley. §8 (Drucker-Prager 3D, calibraciones outer/inner, tangentes
      algorítmicas detalladas)."
  - "Chen W.F., Han D.J. (1988). Plasticity for Structural Engineers. Springer. §7 (
      cohesivo-friccionales 3D)."
  - "ADR 0012 - Sólidos 3D: convención Voigt 6D del proyecto."
  - "ADR 0003 - Despachador algebraico: tangente asimétrica fuerza LU global."
  - "ADR 0006 - Tolerancias adimensionales: patrón atol + rtol·escala con escala física
      kα()."
```

11.1.4 Implementación

- **Archivo**: solidum/materials/drucker_prager_3d.py.
- **Clase**: `DruckerPrager3D`, registrada vía `@MaterialRegistry.register`. `IS_SYMMETRIC = False` a nivel de clase; override por instancia a `True` cuando $\psi = \varphi$ (asociada).
- **Kernel Numba** `_compute_drucker_prager_3d`: predictor elástico desviador-volumétrico en Voigt 6D tensorial (sustracción de $\boldsymbol{\varepsilon}_n^p$ antes del desviador — *necesario* en DP por la dilatancia plástica), detección automática regular/apex, tangente cerrada para cada rama.
- **Calibración**: reusa la función `_calibrate_drucker_prager` de `drucker_prager_2d.py` por composición (mismas fórmulas para outer/inner; `plane_strain_matched` queda filtrada por el validador de variant antes de llegar a la función compartida).
- **State schema**: `eps_p` ndarray(6,) `[xx, yy, zz, xy, yz, xz]` con cortantes tensoriales, `alpha` escalar. **No es incompresible**: `tr(eps_p) = 3·η_g·α` por construcción del flujo.
- `admissibility_scale`: `k(α) = k_0 + H_k·α` con `H_k` = ξ^2·H (radio corriente del cono).

- **Tests**:

- tests/test_materials_unit.py · `TestDruckerPrager3D` (17 tests unitarios) + `TestDruckerPrager3DvsP` (1 cruzado contra DP2D outer_cone plane_strain).

- tests/test_solid_3d_plasticity.py · 4 tests de integración: `TestHex8DruckerPrager3DElastic` (1 — respuesta elástica), `TestHex8DruckerPrager3DRegularPlastic` (1 — cortante puro + tangente asimétrica activa LU + invariante $\mathrm{tr}(\boldsymbol{\varepsilon}^p) = 3\eta_g\alpha$), `TestHex8DruckerPrager3DApex` (1 — expansión hidrostática a 8 GP en rama ápice), `TestTet4DruckerPrager3DBasic` (1 — smoke test Tet4).

- **Notas de traducción**:

- Convención mixta engineering/tensorial idéntica a VM3D: entrada `strain` engineering (γ_ij = 2·ε_ij), salida `sigma` con cortantes tensoriales, estado `eps_p` con cortantes tensoriales. Conversión engineering→tensorial una sola vez al construir el desviador (`strain[3..5]/2`).

- El predictor construye el desviador como $\mathbf{s}^{\mathrm{trial}} = 2G\ \mathrm{dev}(\boldsymbol{\varepsilon}^{\mathrm{tens}} - \boldsymbol{\varepsilon}^p_n)$, **no** como $2G\,(\mathrm{dev}(\boldsymbol{\varepsilon}^{\mathrm{tens}}) - \boldsymbol{\varepsilon}^p_n)$. A diferencia de J2, $\boldsymbol{\varepsilon}^p$ en DP tiene parte volumétrica no nula (dilatancia plástica si $\eta_g > 0$), así que la sustracción debe hacerse *antes* del operador desviador para que la parte volumétrica del flujo se contabilice correctamente.

- Tangente regular usa $\mathbf{I}_{\mathrm{dev}}$ 6×6 con 1/2 en los cortantes (mapea engineering input a tensorial output), idéntica estructura al I_dev de VM3D.

- Tangente apex: rigidez puramente volumétrica reducida con piso $K \cdot 10^{-6}$ cuando $H_k/(9K\eta_f\eta_g + H_k) < 10^{-6}$, es decir, en todo retorno al ápice con $H_k = 0$ (mismo patrón que DP2D para evitar matriz globalmente singular; §14).

- Validación de parámetros en constructor: `E > 0`, `c_0 > 0`, $0 \le \varphi <$ `90°`, $0 \le \psi \le \varphi$, `H` ≥ 0, $\nu \in$ `(-1, 0.5)`, `variant` $\in$ `{outer_cone, inner_cone}`, `density` ≥ 0. Mensajes en español; rechazo explícito de `plane_strain_matched` con explicación de por qué es 2D-only.

11.1.5 Diálogo

- **2026-05-21** · Spec pre-redactada por la IA al avanzar la sub-etapa **A.bis** (materiales 3D no lineales) tras cerrar `VonMises3D` con 824 tests verdes. Base: `DruckerPrager2D.md` (modelo físico, calibraciones, two-branch return mapping) + `VonMises3D.md` (estructura Voigt 6D, patrón cross-consistency).

- **2026-05-21** · **Decisión de plumbing (decide la IA, Reglas §3)**: kernels Numba separados `_compute_drucker_prager_3d` (este material) vs `_compute_drucker_prager_plane_strain` (existente en DP2D). **No** se extrae un `_DPCore` compartido en esta entrega, por las mismas razones que motivaron mantener separados los kernels de VM2D plane strain y VM3D: Numba *nopython* desfavorece arrays de tamaño variable, los kernels son cortos (~80 líneas cada uno), y el sub-patrón compartido (descomposición desviadora, branch detection, tangente cerrada) se documenta cruzadamente entre las specs.

- **2026-05-21 · Decisión de alcance de variantes**: `DruckerPrager3D` ofrece `outer_cone` (default) e `inner_cone`, omitiendo `plane_strain_matched`. Razón: la variante 2D `plane_strain_matched` es un artefacto matemático que reproduce MC exactamente solo bajo $\varepsilon_{zz} = 0$ cinemático impuesto; no tiene análogo 3D porque MC en 3D depende del Lode angle activo. Default `outer_cone` se elige por ser el modo dominante en problemas geotécnicos (cimentaciones, muros). Si la aplicación es extension-dominated (excavaciones), el usuario puede elegir `inner_cone` explícitamente.

- **2026-05-21 · Decisión de tipo de tests cruzados**: `cross_consistency` compara DP3D (`variant='outer_cone'`) contra DP2D (`variant='outer_cone'`, plane_strain), no contra `plane_strain_matched`. Razón: solo las dos calibraciones cono (outer/inner) existen en ambos modelos. La equivalencia matemática es exacta porque las fórmulas de calibración no dependen de la dimensión — solo de (c_0, ϕ, ψ) — y el cono 3D con $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ impuesto se reduce a la formulación 2D plane strain en las componentes activas.

- **2026-05-21 · Decisión de plumbing**: el predictor elástico construye el desviador como $\mathbf{s}^{\text{trial}} = 2G\ \text{dev}(\boldsymbol{\varepsilon}^{\text{tens}} - \boldsymbol{\varepsilon}_n^p)$, **no** como $2G\,(\text{dev}(\boldsymbol{\varepsilon}^{\text{tens}}) - \boldsymbol{\varepsilon}_n^p)$. Diferencia material: a diferencia de J2, $\boldsymbol{\varepsilon}^p$ en DP tiene parte volumétrica no nula ($\text{tr}(\boldsymbol{\varepsilon}^p) = 3\eta_g\alpha$), así que la sustracción debe hacerse *antes* del desviador para que la parte volumétrica del flujo plástico se contabilice correctamente. Esta es la lección documentada en el comentario del kernel de DP2D (línea 39–42) y se aplica desde el inicio en DP3D.

- **2026-05-21 · Implementación completa, status** `validated`. Kernel + clase + 17 tests unitarios + 1 test cruzado + 4 tests de integración (Hex8 elástico, Hex8 rama regular con tangente asimétrica + invariante $\text{tr}(\boldsymbol{\varepsilon}^p) = 3\eta_g\alpha$, Hex8 rama ápice bajo expansión hidrostática, Tet4 smoke test). **24 tests añadidos a la suite (824 → 848 verdes; suite global sin regresiones).** Todos los tests pasaron a la primera, incluyendo el cruzado DP3D ↔ DP2D outer_cone plane_strain a 10 decimales — señal fuerte de que la convención Voigt 6D, la calibración compartida y la sustracción de $\boldsymbol{\varepsilon}_n^p$ antes del desviador son correctas. La integración Hex8 con cortante puro y tangente asimétrica ($\psi \neq \phi$) confirma que el despachador algebraico (ADR 0003) selecciona LU correctamente vía el flag declarativo `IS_SYMMETRIC = False`. Validación contra solución analítica cerrada (cilindro con falla DP, esfera hueca bajo presión interna, etc.) **diferida a la campaña 3D consolidada** que se hará al final de la sub-etapa A.bis (tras completar `IsotropicDamage3D`).

- **2026-09-22** · Auditoría global: misma corrección del ápice que `DruckerPrager2D`: con η_g $= 0$ ($\psi = 0$) la rama del ápice lanza `ValueError` y el constructor avisa; antes se clampaba `A_apex` a 1e-30.

- **2026-09-22** · Ensamblaje por lotes (ADR 0014): declara `STATE_SCHEMA` y un adaptador `@njit` con firma `MAT_SIG` que llama al mismo return mapping; la tolerancia de admisibilidad (ADR 0006) se evalúa dentro del kernel con las mismas operaciones que `admissibility_tol`. Formulación sin cambios; equivalencia con `compute_state` exacta en esfuerzo, tangente y estado (`tests/test_batch_kernels.py`).

- **2026-09-29** · Revisión documental (sin cambio de código ni de resultados), la misma que en `DruckerPrager2D`: (1) α es el multiplicador plástico acumulado γ, no una deformación plástica

equivalente, y H endurece el radio del cono k respecto de γ, no la cohesión; conversión desde la parametrización $c(\bar{\varepsilon}^p)$ de de Souza Neto et al. (2008), $H_k = \xi^2 H_c$ con $\xi = k_0/c_0$ (§5). (2) El piso $K \cdot 10^{-6}$ de la tangente en el ápice actúa en todo retorno al ápice con $H_k = 0$, no sólo con $\psi = 0$, caso que desde el 2026-09-22 lanza `ValueError` (§14 y `numerical_caveats`).

- **2026-09-29** · Cambio del significado del parámetro `H`, aprobado por el usuario (hallazgo clas-6 de la auditoría): pasa a ser $H_c = dc/d\bar{\varepsilon}^p$, la parametrización de de Souza Neto et al. (2008, §8.3), y el material calcula $H_k = \xi^2 H_c$ con $\xi = k_0/c_0$. Antes `H` era directamente H_k: un H_c tomado de un ensayo o de la literatura daba un endurecimiento distinto por el factor ξ^2, sin aviso. Con endurecimiento lineal la física es la misma; cambia sólo el número que se escribe. El return mapping, la tangente y los resultados con `H = 0` no cambian. Blindado en `tests/test_drucker_prager_hardening.py` ($k(\gamma) = \xi\, c(\xi\gamma)$ en las tres calibraciones y estado final sobre la superficie endurecida con H_k); los tests con `H > 0` pasan a usar H_k en sus expresiones analíticas.

11.2 Elastic3D

Material elástico isótropo en 3D, espejo natural de `Elastic2D` *con la convención Voigt 6D fijada en ADR 0012.*

11.2.1 Especificación física

0. Descripción general

Modelo elástico lineal isótropo tridimensional, parametrizado por (E, ν). A diferencia de `Elastic2D`, **no tiene variantes de hipótesis cinemática**: en 3D no aplican `plane_stress` ni `plane_strain` — toda la deformación es activa. Independiente de la velocidad, sin variables internas. Es el material elástico canónico para todos los sólidos 3D del catálogo.

1. Descomposición de la deformación

No aplica — toda la deformación es elástica:

$$\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e$$

2. Ley elástica

$$\boldsymbol{\sigma} = \mathbf{C}_e\, \boldsymbol{\varepsilon}$$

con $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ en **notación Voigt 6D del proyecto** (ADR 0012, `Reglas.md §5`):

$$\boldsymbol{\varepsilon} = [\varepsilon_{xx},\ \varepsilon_{yy},\ \varepsilon_{zz},\ \gamma_{xy},\ \gamma_{yz},\ \gamma_{xz}]^\top, \qquad \gamma_{ij} = 2\,\varepsilon_{ij}$$

$$\boldsymbol{\sigma} = [\sigma_{xx},\ \sigma_{yy},\ \sigma_{zz},\ \sigma_{xy},\ \sigma_{yz},\ \sigma_{xz}]^\top$$

Matriz constitutiva elástica 6×6:

$$\mathbf{C}_e = \frac{E}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1-\nu & \nu & \nu & 0 & 0 & 0 \\ \nu & 1-\nu & \nu & 0 & 0 & 0 \\ \nu & \nu & 1-\nu & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix}$$

Tangente constante $\mathbf{C}_e$ en todo régimen. Simétrica y positiva definida en el rango admisible de ν.

El factor $\frac{1-2\nu}{2}$ en los cortantes (no $1-2\nu$) es consecuencia del uso de γ_{ij} *engineering*: $\sigma_{ij} = G\,\gamma_{ij}$ con $G = \frac{E}{2(1+\nu)} = \frac{E}{(1+\nu)(1-2\nu)} \cdot \frac{1-2\nu}{2}$.

3-6. Superficie de fluencia, flujo, endurecimiento, Kuhn-Tucker

No aplican — el material es indefinidamente elástico.

7. Variables internas

Ninguna — `state_vars = None` se acepta y se devuelve intacto. Sin `PRIMARY_STATE_VAR`.

8. Notación Voigt

Orden y factores fijados por **ADR 0012**:

- Componentes diagonales primero: `[xx, yy, zz]`.
- Bloque cortante después: `[xy, yz, xz]` (extensión natural del orden 2D del proyecto `[xx, yy, xy]`).
- Deformaciones angulares *engineering*: $\gamma_{ij} = 2\,\varepsilon_{ij}$.
- Esfuerzos tensoriales sin factor.

Compatibilidad con códigos externos: ABAQUS usa `[11, 22, 33, 12, 13, 23]`. La diferencia con la convención del proyecto es la permutación del bloque cortante (`yz` ↔ `xz`). Si se importan datos desde ABAQUS, aplicar permutación 5↔6 una vez en el preprocesador.

11.2.2 Formulación numérica

9. Esquema temporal

No aplica — sin historia.

10. Return mapping / actualización

No aplica. Evaluación reducida a producto matriz-vector:

$$\boldsymbol{\sigma}_{n+1} = \mathbf{C}_e\,\boldsymbol{\varepsilon}_{n+1}, \qquad \mathbf{C}^{\text{alg}} = \mathbf{C}_e$$

La matriz $\mathbf{C}_e$ se precalcula y guarda en el constructor — coste de `compute_state` es un producto $6{\times}6 \cdot 6$ sin alocación adicional.

11. Tangente algorítmica consistente

$$\mathbf{C}^{\text{alg}} = \mathbf{C}_e$$

Simétrica (`IS_SYMMETRIC = True`). Positiva definida para $\nu \in (-1, 0.5)$ estricto; diverge en $\nu \to 0.5$ (incompresible — requiere formulación mixta u-p, no soportada).

12. Caveats numéricos

- **Incompresibilidad** $\nu \to 0.5$: la matriz constitutiva diverge ($1-2\nu \to 0$). El constructor rechaza $\nu \geq 0.5$ con `ValueError`. Recomendado $\nu \leq 0.49$ para metales y polímeros casi-incompresibles.
- **Locking volumétrico** en `Hex8`/`Tet4` con $\nu \to 0.5$: declarado limitación arquitectural en la spec de cada elemento, sin mitigación implementada (B-bar/F-bar diferidas a caso de uso real, política idéntica al 2D).

- Para $\nu < 0$ el material es admisible termodinámicamente (auxético) pero infrecuente en la práctica; el constructor lo permite hasta $\nu > -1$.

11.2.3 Contrato de implementación

```
name: Elastic3D
kind: material
status: validated

interface:
  strain_dim: 6
  primary_state_var: null
  is_symmetric: true

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 2 (elastostática 3D en notación de Voigt)"
  general_codes:
    - abaqus: "*ELASTIC en C3D*"
    - opensees: "nDMaterial ElasticIsotropic"

parameters:
  - { name: E,       type: float, required: true,  desc: "Módulo de Young (>0)" }
  - { name: nu,      type: float, required: true,  desc: "Coeficiente de Poisson ∈ (-1,
      0.5)" }
  - { name: density, type: float, required: false, default: null,
      desc: "Densidad (kg/m³). Opcional al construir; obligatoria solo si se ensambla
          peso propio o masa (ADR 0008)" }

signature:
  compute_state: "ε(: ndarray(6,), state_vars=None) -> σ(: ndarray(6,), C_e: ndarray
      (6,6), state_vars)"
  strain_kind: "3D Voigt -ε [_xx, ε_yy, ε_zz, γ_xy, γ_yz, γ_xz] con γ_ij = 2·ε_ij"

state_variables: []

conventions:
  sign: "σ > 0 ⇔ tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] (ADR 0012); γ_ij = 2·ε_ij en deformación; σ_ij
      tensorial sin factor"
  hypothesis_dispatch: "no aplica en 3D"

validity:
  - "E > 0"
  - "ν ∈ (-1, 0.5) estricto; ν = 0.5 rechazado (incompresible)"
  - "pequeñas deformaciones ε(|| ≤ 1e-2)"

out_of_scope:
  - "anisotropía (ortótropo, monoclínico, totalmente anisótropo)"
  - "plasticidad ⇒ VonMises3D, DruckerPrager3D (sub-etapa posterior)"
  - "daño ⇒ IsotropicDamage3D (sub-etapa posterior)"
  - "incompresibilidad estricta ν(=0.5) ⇒ requiere formulación mixta u-p"
  - "grandes deformaciones"

acceptance:
```

```
  verification:
    - name: simetria_C_e
      setup: "construir Elastic3D y comparar C_e con su traspuesta"
      expect: "C_e == C_e.T exacto componente a componente"
      tol_abs: 1.0e-14
    - name: positividad_definida
      setup: "autovalores de C_e con ν∈ {0, 0.3, 0.49}"
      expect: "todos los autovalores estrictamente positivos"
      tol_abs: 0.0
    - name: traccion_uniaxial
      setup: "ε = ε(ε_0, ν·ε_0, ν·ε_0, 0, 0, 0)"
      expect: "σ = (E·ε_0, 0, 0, 0, 0, 0) exacto"
      tol_rel: 1.0e-12
    - name: cortante_puro_xy
      setup: "ε = (0, 0, 0, γ_0, 0, 0) con γ_0 = 1e-3"
      expect: "σ = (0, 0, 0, G·γ_0, 0, 0) con G = Eν/[2(1+)]"
      tol_rel: 1.0e-12
    - name: cortante_puro_yz
      setup: "ε = (0, 0, 0, 0, γ_0, 0)"
      expect: "σ = (0, 0, 0, 0, G·γ_0, 0)"
      tol_rel: 1.0e-12
    - name: cortante_puro_xz
      setup: "ε = (0, 0, 0, 0, 0, γ_0)"
      expect: "σ = (0, 0, 0, 0, 0, G·γ_0)"
      tol_rel: 1.0e-12
    - name: compresion_hidrostatica
      setup: "ε = ε(ε_0, ε_0, ε_0, 0, 0, 0) con ε_0 = -1e-3 (compresión isótropa)"
      expect: "σ = (K·ε3_0, K·ε3_0, K·ε3_0, 0, 0, 0) con K = Eν/[3(1-2)] (módulo de
          compresibilidad)"
      tol_rel: 1.0e-12
    - name: rechazo_inputs_invalidos
      setup: "construir Elastic3D con ≤E0, ν fuera de rango (-1, 0.5), density<0"
      expect: "ValueError con mensaje claro en cada caso"

references:
  - "Timoshenko S., Goodier J.N. (1970). Theory of Elasticity. McGraw-Hill. §6 (3D
      elasticity)."
  - "Bathe K.J. (2014). Finite Element Procedures. Prentice Hall. §6.6."
  - "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and Applications
      of FEA. Wiley. §6.8."
  - "ADR 0012 - Sólidos 3D: convención Voigt 6D y cierre del contrato internal_forces."
```

11.2.4 Implementación

- **Archivo**: solidum/materials/elastic_3d.py.
- **Clase**: `Elastic3D`, registrada vía `@MaterialRegistry.register`.
- **Despacho**: matriz `C` precalculada en el constructor. `compute_state` reduce a `self.C @ strain` sin ramificaciones — coste mínimo.
- **Validación temprana** en constructor: `E > 0`, $\nu \in$ `(-1, 0.5)`, `density` $\geq$ `0` si se pasa. Mensajes en español.

- **Tests**: tests/test_solid_3d.py — clase `TestElastic3D` (11 tests: simetría C, positividad definida, tracción uniaxial, los tres cortantes puros, compresión hidrostática con K = E/[3(1-2ν)], rechazos de inputs inválidos, density opcional).
- **Validación de integración**: `tests/validation/test_cube_lame_3d.py` consume Elastic3D end-to-end vía Hex8 y Tet4 (tracción uniaxial + compresión hidrostática con solución analítica exacta).

11.2.5 Diálogo

- **2026-05-19** · Spec creada como pieza inicial de la Etapa 7 (sólidos 3D, alcance acotado Hex8 + Tet4 + Elastic3D). Convención Voigt 6D fijada en ADR 0012. Sin variantes de hipótesis (no aplica plane_stress/plane_strain en 3D).
- **2026-09-22** · Ensamblaje por lotes (ADR 0014): `STATE_SCHEMA = {}` y kernel lineal compartido (`materials/_batch.linear_material_kernel`, σ = `C`·ε); `batch_matrix()` devuelve la `C` precalculada.

11.3 IsotropicDamage3D

*Orden de trabajo. El usuario revisa **especificación física**, **formulación numérica** y **contrato**; la IA propone, ejecuta y rellena **implementación + diálogo**.*

***Pre-redactada por la IA** sobre la base de `IsotropicDamage2D.md` (mismo modelo físico) y `Elastic3D.md` / `VonMises3D.md` (patrón Voigt 6D). Tercera y última entrega de la sub-etapa **A.bis** (materiales 3D no lineales). Desde el 2026-09-29 la **deformación equivalente es configurable** entre tres medidas de la literatura (ver §3 y Diálogo).*

11.3.1 Especificación física

0. Descripción general

Modelo de **daño isótropo escalar** en pequeñas deformaciones para sólidos 3D, formulado íntegramente en Voigt 6D del proyecto (ADR 0012). Captura degradación progresiva e isótropa de la rigidez en respuesta a la deformación máxima histórica, con ablandamiento exponencial. Pensado para hormigón, cerámicos y materiales cuasi frágiles donde la microfisuración distribuida puede modelarse mediante un escalar. Con las medidas de Mazars o de von Mises modificado (§3) el inicio del daño distingue tracción de compresión.

Diferencias respecto a `IsotropicDamage2D`:

- **Sin variantes de hipótesis** (igual que VM3D, DP3D, Elastic3D): en 3D todas las componentes son activas.
- **Algoritmo idéntico al 2D extendido a 6D**: misma ley de evolución exponencial, mismo régimen carga/descarga por κ, mismas tres medidas de deformación equivalente, misma estructura de tangente algorítmica consistente. **Sin Newton local** (el algoritmo es explícito).
- `IsotropicDamage2D` es la restricción exacta de este modelo en las dos hipótesis y con las tres medidas: en deformación plana bajo $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$; en tensión plana con $\varepsilon_{zz} = -\nu/(1-\nu)\,(\varepsilon_{xx} + \varepsilon_{yy})$ y $\gamma_{yz} = \gamma_{xz} = 0$ (y $\sigma_{zz} = 0$ resulta).

1. Cinemática

Pequeñas deformaciones, descomposición elástica pura: no hay deformación plástica (sin plasticidad). El daño se manifiesta como degradación de la rigidez efectiva. Sin variables internas tensoriales — solo el historial escalar κ y el daño d.

2. Esfuerzo nominal vs esfuerzo efectivo

$$\boldsymbol{\sigma} = (1-d)\,\boldsymbol{\sigma}^{\text{eff}}, \qquad \boldsymbol{\sigma}^{\text{eff}} = \mathbf{C}_e\,\boldsymbol{\varepsilon}$$

con $d \in [0, d_{\max}]$ la variable de daño isótropa y $\mathbf{C}_e$ el tensor constitutivo elástico isótropo 3D 6×6 de `Elastic3D` (sin variantes de hipótesis en 3D).

El esfuerzo efectivo $\boldsymbol{\sigma}^{\text{eff}}$ es el que el material intacto soportaría con la deformación corriente. El esfuerzo nominal $\boldsymbol{\sigma}$ es lo que efectivamente transmite considerando la degradación.

3. Deformación equivalente

Cantidad escalar que cuantifica la "intensidad" de la deformación y gobierna el daño. Se elige con el parámetro `equivalent_strain` entre tres medidas estándar, en notación Voigt 6D $[\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]$ con $\gamma_{ij} = 2\varepsilon_{ij}$. Las tres están **calibradas en tracción uniaxial de esfuerzo**: $\varepsilon_{\text{eq}} = \varepsilon$, de modo que el daño empieza en $\sigma = E\,\kappa_0$ y $\kappa_0 = f_t/E$. Se diferencian en compresión y en estados multiaxiales.

1. **Energética** (`energy`, por defecto; Simó y Ju 1987):

$$\varepsilon_{\text{eq}} = \sqrt{\frac{\boldsymbol{\varepsilon}^\top \mathbf{C}_e\, \boldsymbol{\varepsilon}}{E}} = \sqrt{\frac{\boldsymbol{\sigma}^{\text{eff}} : \boldsymbol{\varepsilon}}{E}}$$

Simétrica en tracción y compresión (en compresión uniaxial de esfuerzo, $\varepsilon_{\text{eq}} = |\varepsilon|$).

1. **Mazars** (`mazars`; Mazars 1986):

$$\varepsilon_{\text{eq}} = \sqrt{\sum_{i=1}^{3} \langle \varepsilon_i \rangle_+^2}$$

con ε_i las deformaciones principales. Sólo las extensiones dañan: en compresión uniaxial de esfuerzo daña la extensión lateral, $\varepsilon_{\text{eq}} = \sqrt{2}\,\nu\,|\varepsilon|$; en compresión hidrostática no hay daño.

1. **von Mises modificado** (`modified_von_mises`; de Vree, Brekelmans y van Gils 1995), con $k = f_c/f_t$:

$$\varepsilon_{\text{eq}} = \frac{k-1}{2k(1-2\nu)}\, I_1 + \frac{1}{2k}\sqrt{\left(\frac{k-1}{1-2\nu}\right)^2 I_1^2 + \frac{12k}{(1+\nu)^2}\, J_2}$$

con $I_1 = \varepsilon_{xx} + \varepsilon_{yy} + \varepsilon_{zz}$ y $J_2 = \frac{1}{6}\left[(\varepsilon_{xx} - \varepsilon_{yy})^2 + (\varepsilon_{yy} - \varepsilon_{zz})^2 + (\varepsilon_{zz} - \varepsilon_{xx})^2\right] + \frac{1}{4}(\gamma_{xy}^2 + \gamma_{yz}^2 + \gamma_{xz}^2)$. En compresión uniaxial de esfuerzo, $\varepsilon_{\text{eq}} = |\varepsilon|/k$; con $k > 1$ no hay daño en compresión hidrostática.

Hasta el 2026-09-29 la medida era la norma del tensor, $\varepsilon_{\text{eq}} = \sqrt{\boldsymbol{\varepsilon} : \boldsymbol{\varepsilon}} = \sqrt{\boldsymbol{\varepsilon}^\top \mathbf{M} \boldsymbol{\varepsilon}}$ con $\mathbf{M} = \text{diag}(1, 1, 1, \frac{1}{2}, \frac{1}{2}, \frac{1}{2})$: convención propia, no estándar y mal calibrada (en tracción uniaxial de esfuerzo daba $\varepsilon_{\text{eq}} = \varepsilon\sqrt{1 + 2\nu^2}$, así que $\kappa_0 \neq f_t/E$). Se retiró (ver *Diálogo*).

4. Variable histórica y condición de carga (Kuhn-Tucker)

$$\kappa_{n+1} = \max(\kappa_n, \varepsilon_{\text{eq}}(\boldsymbol{\varepsilon}_{n+1})), \qquad \kappa_0 \text{ inicial}$$

Esto codifica:

- **Carga activa** ($\dot{\kappa} > 0$): κ crece con ε_{eq}; el daño puede aumentar.
- **Descarga / recarga** ($\dot{\kappa} = 0$): κ queda congelado; el daño no cambia.
- **Irreversibilidad**: κ es monótono no decreciente.

5. Ley de evolución del daño

$$d(\kappa) = \begin{cases} 0 & \kappa \le \kappa_0 \\ 1 - \dfrac{\kappa_0}{\kappa}\, e^{-\alpha(\kappa-\kappa_0)} & \kappa > \kappa_0 \end{cases}$$

con saturación numérica $d \le d_{\max} =$ `DAMAGE_MAX` (evita rigidez nula).

Propiedades idénticas al 2D: $d(\kappa_0) = 0$ (continuidad); $d'(\kappa) > 0$ para $\kappa > \kappa_0$ (irreversibilidad); $\lim_{\kappa\to\infty} d = 1$ (asintótico); α controla la velocidad de degradación. **Misma función de ablandamiento** que en 1D y 2D — centralizada en `solidum.materials._softening.evaluate_exponential_damage`.

11.3.2 Formulación numérica

6. Algoritmo en un paso

Dado $\{\kappa_n\}$ y $\boldsymbol{\varepsilon}_{n+1}$:

1. $\boldsymbol{\sigma}^{\text{eff}} = \mathbf{C}_e\, \boldsymbol{\varepsilon}_{n+1}$ y ε_{eq} con la medida elegida (§3), junto con su derivada. Las deformaciones principales de Mazars salen de `numpy.linalg.eigh` compilado con Numba.
2. Si $\varepsilon_{\text{eq}} > \kappa_n$: **carga activa**, $\kappa_{n+1} = \varepsilon_{\text{eq}}$. Si no: **descarga**, $\kappa_{n+1} = \kappa_n$.
3. Aplicar ley de daño con κ_{n+1}; saturar a $d_{\max}$.
4. $\boldsymbol{\sigma}_{n+1} = (1-d)\, \mathbf{C}_e\, \boldsymbol{\varepsilon}_{n+1}$.
5. Tangente: **secante** en descarga / sin daño / saturación; **algorítmica consistente** en carga activa con daño no saturado (ver §7).

El algoritmo es **explícito**: no requiere Newton local. La actualización de κ y d es directa, idéntica al 2D.

7. Tangente algorítmica consistente

$$\mathbf{C}^{\text{alg}} = \frac{\partial \boldsymbol{\sigma}_{n+1}}{\partial \boldsymbol{\varepsilon}_{n+1}} = (1-d)\, \mathbf{C}_e - \boldsymbol{\sigma}^{\text{eff}} \otimes \frac{\partial d}{\partial \boldsymbol{\varepsilon}}, \qquad \frac{\partial d}{\partial \boldsymbol{\varepsilon}} = \frac{\partial d}{\partial \kappa}\, \frac{\partial \kappa}{\partial \boldsymbol{\varepsilon}}$$

con $\boldsymbol{\sigma}^{\text{eff}} = \mathbf{C}_e\, \boldsymbol{\varepsilon}$ (vector 6×1), $\partial d/\partial\kappa = (1-d)(1/\kappa + \alpha)$ (idéntica al 2D) y $\partial\kappa/\partial\boldsymbol{\varepsilon} = \partial\varepsilon_{\text{eq}}/\partial\boldsymbol{\varepsilon}$ en carga activa, $\mathbf{0}$ en descarga. Derivadas respecto de la deformación en Voigt *engineering*:

- **Energética**: $\dfrac{\partial \varepsilon_{\text{eq}}}{\partial \boldsymbol{\varepsilon}} = \dfrac{\mathbf{C}_e\, \boldsymbol{\varepsilon}}{E\, \varepsilon_{\text{eq}}}$.
- **Mazars**: $\dfrac{\partial \varepsilon_{\text{eq}}}{\partial \boldsymbol{\varepsilon}} = \dfrac{1}{\varepsilon_{\text{eq}}} \sum_i \langle \varepsilon_i \rangle_+ \dfrac{\partial \varepsilon_i}{\partial \boldsymbol{\varepsilon}}$, con $\partial\varepsilon_i/\partial\boldsymbol{\varepsilon}$ el proyector $\mathbf{n}_i \otimes \mathbf{n}_i$ en Voigt *engineering* (componente de γ_{ab}: $n_a n_b$). La suma es la parte positiva del tensor, $\boldsymbol{\varepsilon}_+ = \sum_i \langle \varepsilon_i \rangle_+\, \mathbf{n}_i \otimes \mathbf{n}_i$, escrita en Voigt con sus componentes tensoriales: $[(\varepsilon_+)_{xx}, (\varepsilon_+)_{yy}, (\varepsilon_+)_{zz}, (\varepsilon_+)_{xy}, (\varepsilon_+)_{yz}, (\varepsilon_+)_{xz}]/\varepsilon_{\text{eq}}$. Bien definida con deformaciones principales repetidas: $\sum_i \langle \lambda_i \rangle_+^2$ es una función simétrica C^1 de los valores principales y los proyectores de un autoespacio repetido llevan el mismo peso.

- **von Mises modificado**: con $S = \sqrt{b^2 I_1^2 + c\, J_2}$, $b = (k-1)/(1-2\nu)$, $c = 12k/(1+\nu)^2$:

$$\frac{\partial \varepsilon_{\text{eq}}}{\partial \boldsymbol{\varepsilon}} = \frac{b + b^2 I_1 / S}{2k} [1,1,1,0,0,0] + \frac{c}{4kS} [e_{xx}, e_{yy}, e_{zz}, \tfrac{1}{2}\gamma_{xy}, \tfrac{1}{2}\gamma_{yz}, \tfrac{1}{2}\gamma_{xz}]$$

con e la parte desviadora de la deformación.

Tangente final:

$$\mathbf{C}^{\text{alg}} = \begin{cases} \mathbf{C}_e & \kappa \le \kappa_0 \text{ (sin daño)} \\ (1-d)\,\mathbf{C}_e & \text{descarga} \\ (1-d)\,\mathbf{C}_e \;-\; (1-d)\left(\dfrac{1}{\kappa} + \alpha\right)(\mathbf{C}_e\boldsymbol{\varepsilon}) \otimes \dfrac{\partial \varepsilon_{\text{eq}}}{\partial \boldsymbol{\varepsilon}} & \text{carga activa, } d < d_{\max} \\ (1-d_{\max})\,\mathbf{C}_e & d = d_{\max} \text{ (saturación)} \end{cases}$$

8. Simetría de la tangente

- **Energética**: el término de carga activa es $\propto (\mathbf{C}_e\boldsymbol{\varepsilon}) \otimes (\mathbf{C}_e\boldsymbol{\varepsilon})$: tangente consistente **simétrica** (en la implementación, bit a bit).
- **Mazars y von Mises modificado**: $\mathbf{C}_e\boldsymbol{\varepsilon}$ y $\partial\varepsilon_{\text{eq}}/\partial\boldsymbol{\varepsilon}$ no son proporcionales en general: tangente **no simétrica**.

Consecuencia (ADR 0003): `IS_SYMMETRIC` es `False` en la clase (cota conservadora) y la **instancia** lo pone a `True` con la medida energética; `domain_is_symmetric` lo lee de la instancia. Con Mazars o von Mises modificado el despachador elige LU (o Pardiso). Con la energética puede elegir Cholesky si está disponible; como la tangente de ablandamiento puede no ser definida positiva, el corrector degrada a LU en cuanto Cholesky la rechaza.

9. Decisión de régimen durante el Newton

Idéntica al 2D: la transición loading↔unloading se decide con κ_n committed (no con κ trial dentro del Newton). Estándar y evita oscilaciones del Newton entre ramas en pasos con ambigüedad. Si el paso converge, κ se compromete y la decisión es consistente con la trayectoria.

11.3.3 Contrato de implementación

```yaml
name: IsotropicDamage3D
kind: material
status: validated        # draft → implemented → validated

interface:
  strain_dim: 6
  primary_state_var: damage
  is_symmetric: false      # cota de clase; la instancia es true con equivalent_strain
      ='energy' (ver §8)

admission:                 # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_borst_crisfield: "§6.2 (daño isótropo escalar con ablandamiento exponencial y
        medidas de deformación equivalente)"
```

```
    - de_souza_neto: "cap. 12 (mecánica del daño)"
  general_codes:
    - code_aster: "MAZARS, ENDO_ISOT_BETON (daño escalar isótropo; MAZARS con la
        deformación equivalente de Mazars)"

parameters:
  - { name: E,       type: float, required: true,  desc: "Módulo de Young intacto (>0)"
      }
  - { name: nu,      type: float, required: true,  desc: "Coeficiente de Poisson ∈ (-1,
      0.5)" }
  - { name: kappa_0, type: float, required: true,  desc: "Umbral de deformación
      equivalente; daño nulo mientras κ≤κ_0 (>0). Con las tres medidas κ_0 = f_t/E" }
  - { name: alpha,   type: float, required: true,  desc: "Velocidad de degradación
      exponencial (>0). α mayor → ablandamiento más abrupto" }
  - { name: equivalent_strain, type: str, required: false, default: "energy", desc: "
      energy (Simó-Ju 1987) | mazars (Mazars 1986) | modified_von_mises (de Vree et al.
      1995)" }
  - { name: k,       type: float, required: false, default: null, desc: "f_c/f_t ≥( 1);
      obligatorio con modified_von_mises y no admitido con las otras medidas" }
  - { name: density, type: float, required: false, default: null, desc: "Densidad (ADR
      0008)" }

signature:
  compute_state: "ε(: ndarray(6,), state_vars=None) -> σ(: ndarray(6,), C_tan: ndarray
      (6,6), state_vars')"
  strain_kind: "3D Voigt -ε [_xx, ε_yy, ε_zz, γ_xy, γ_yz, γ_xz] con γ_ij = 2·ε_ij"

state_schema:
  kappa:  "float ≥κ_0 - máxima deformación equivalente histórica"
  damage: "float ∈ [0, DAMAGE_MAX] - variable de daño escalar isótropa"

conventions:
  sign:  "σ > 0 ⇔ tracción; energy no distingue tracción de compresión; mazars sólo daña
      con extensiones principales; modified_von_mises daña en compresión uniaxial con
      ε_eq = ε||/k"
  voigt: "γ_ij = 2·ε_ij en deformación; en ∂ε_eq∂ε/ la componente de γ_ab es la
      derivada respecto de la deformación engineering"
  equivalent_strain: "energy: √ε^T(·C_e·ε/E); mazars: √Σ⟨ε()+_i^2); modified_von_mises: (
      -k1)/(2k-ν(12))·I1 + 1/(2k)·√(((-k1)-ν/(12))^2·I1^2 + 12kν/(1+)^2·J2). Las tres dan
      ε_eq = ε en tracción uniaxial de esfuerzo"
  damage_irreversibility: "κ monótono no decreciente vía maxκ(_old, ε_eq); dκ()
      creciente"

validity:
  - "E > 0, κ_0 > 0, α > 0"
  - "ν ∈ (-1, 0.5) estricto"
  - "equivalent_strain ∈ {'energy', 'mazars', 'modified_von_mises'}"
  - "k ≥ 1 finito, sólo con equivalent_strain = 'modified_von_mises' (obligatorio ahí);
      con k < 1 no dañaría en tracción triaxial y sí en compresión hidrostática (
      decisión del usuario, 2026-09-30)"

out_of_scope:
  - "modelo de Mazars completo con dos daños (d_t, d_c) ponderados por α_t, α_c; aquí s
      ólo su medida de deformación equivalente con un único daño escalar"
  - "anisotropía del daño (tensor de daño D vs escalar)"
```

```
  - "regularización para evitar mesh-dependency en regime de ablandamiento (longitud
      característica, no-local, gradient)"
  - "acoplamiento con plasticidad"
  - "fatiga / acumulación cíclica del daño"
  - "restitución de rigidez al cerrar fisuras (efecto unilateral)"

numerical_caveats:
  - "IS_SYMMETRIC por instancia: true con energy (tangente consistente simétrica),
      false con mazars y modified_von_mises (LU/Pardiso, ADR 0003). Con energy y
      Cholesky disponible, la tangente de ablandamiento puede no ser definida positiva:
       el corrector degrada a LU al primer rechazo."
  - "Transición ↔loadingunloading discontinua en la tangente. Newton converge antes de
      oscilar entre ramas en casos típicos. Para problemas patológicos cerca del umbral
      , considerar arc-length."
  - "Mazars: ε_eq es C¹ pero no C² donde una deformación principal cruza cero (entra o
      sale una extensión). ∂ε_eq∂ε/ y la tangente consistente son continuas en ese
      cruce; lo que salta es su derivada (la curvatura de ⟨ε⟩₊_i²), y la tangente sigue
       siendo Lipschitz, así que el Newton conserva la convergencia cuadrática. Con
      deformaciones principales repetidas la derivada está bien definida. Las
      principales salen de numpy.linalg.eigh compilado (LAPACK) por punto de Gauss."
  - "Saturación d=DAMAGE_MAX corta la corrección consistente; el algoritmo devuelve
      tangente secante en ese caso para evitar términos espurios."
  - "Sin regularización, la solución es mesh-dependent en régimen de ablandamiento (
      localización en una banda de elementos). Es limitación inherente del modelo
      continuo de daño isótropo escalar; out-of-scope esta entrega."

acceptance:
  unit:
    - name: paso_elastico_bajo_umbral
      setup: "ε pequeño 6D con todas las componentes activas, ε_eq < κ_0"
      expect: "σ = C_e·ε exacto; d=0; C_tan = C_e (sin corrección rank-1)"
      tol_rel: 1.0e-12

    - name: damage_evolution_exponential
      setup: "ε grande tal que ε_eq >> κ_0"
      expect: "d cumple la ley exponencial; κ = ε_eq de la medida (energy, ε = ε[_xx,
          0, …]: ε_eq = ε_xx·√-ν((1)ν/[(1+)-ν(12)]))"
      tol_rel: 1.0e-10

    - name: eps_eq_matches_independent_reference
      setup: "25 estados aleatorios por medida; referencia sobre el tensor 3D σε(: con
          Lamé, autovalores de numpy, J2 = ½·e:e)"
      expect: "κ tras un paso desde κ = 0 coincide con ε_eq de referencia"
      tol_rel: 1.0e-12

    - name: uniaxial_stress_calibration
      setup: "tracción y compresión uniaxial de esfuerzo ε( = ε[, -νε, -νε, 0, 0, 0])"
      expect: "tracción: ε_eq = ε con las tres medidas κ(_0 = f_t/E); compresión:
          energy ε||, mazars √2·ν·ε||, modified_von_mises ε||/k"
      tol_rel: 1.0e-13

    - name: hydrostatic_compression
      expect: "mazars y modified_von_mises (k > 1): ε_eq = 0; energy: ε_eq = e·
          √-ν(3/(12))"

    - name: rotational_invariance
```

```
    expect: "ε_eq(R·ε·ᵀR) = ε_eqε() con las tres medidas"
    tol_rel: 1.0e-12

  - name: tangent_equals_secant_on_unloading
    setup: "carga hasta κ > κ_0; descarga con ε de menor ε_eq"
    expect: "C_tan = (1-d)·C_e exacto (segundo término se anula con ∂κ∂ε/ = 0)"
    tol_rel: 1.0e-12

  - name: tangent_equals_secant_below_threshold
    setup: "ε pequeño, κ_old = κ_0 (estado inicial intacto)"
    expect: "C_tan = C_e (sin daño activo)"
    tol_rel: 1.0e-12

  - name: tangent_equals_secant_at_saturation
    setup: "carga hasta κ enorme tal que d alcanza DAMAGE_MAX"
    expect: "C_tan = (1-DAMAGE_MAX)·C_e (el término consistente se corta)"
    tol_rel: 1.0e-12

  - name: tangent_symmetry_in_loading
    setup: "carga activa con estado de deformación no degenerado ε(_xx ≠ε _yy ≠ε _zz,
        cortantes activos)"
    expect: "energy: C_tan = ᵀC_tan bit a bit; mazars y modified_von_mises: ‖C_tan
        -ᵀ‖ C_tan_F‖/‖C_tan_F ≥ 0.01"

  - name: tangent_finite_difference_consistency
    setup: "carga activa desde κ = κ_0 con cada medida; estados general, todo extensi
        ón y dos principales repetidas; diferencia centrada con h = 1e-10 sobre ε|| ~
        1e-4"
    expect: "|C_FD - C_alg| ≤ 1e-6·max|C_FD| (truncamiento ~1e-12 y redondeo ~1e-10
        relativos)"
    tol_rel: 1.0e-6

  - name: kappa_monotonic
    setup: "trayectoria multi-paso con carga, descarga, recarga"
    expect: "κ es monótono no decreciente en todos los pasos"

  - name: descarga_recarga_irreversibilidad
    setup: "cargar a κ_load, descargar elásticamente, recargar"
    expect: "d permanece constante durante toda la descarga; al recargar, d sigue
        creciendo desde el valor de descarga"

  - name: unit_invariance_MPa_vs_Pa
    setup: "mismo path adimensional con (MPa,mm,N) y (Pa,m,N)"
    expect: "d, κ y σ/E idénticos κ(_0 es una deformación, ε es adimensional →
        invariancia exacta)"
    tol_rel: 1.0e-12

  - name: rechazo_inputs_invalidos
    setup: "construir con ≤E0, κ≤_00, α≤0, ν∉(-1, 0.5), density<0, equivalent_strain
        desconocida, modified_von_mises sin k, k con otra medida, k ≤ 0 o no finito"
    expect: "ValueError con mensaje claro en cada caso"

  - name: is_symmetric_attribute
    expect: "IsotropicDamage3D.IS_SYMMETRIC == False en la clase; instancia True con
        energy, False con mazars y modified_von_mises"
```

```
    - name: batch_kernel_matches_compute_state
      expect: "el kernel por lotes reproduce bit a bit σ, C_tan y la fila de estado de
          compute_state con las tres medidas"

  cross_consistency:
    - name: equivalencia_plane_strain
      setup: "Damage3D con ε = ε(_xx, ε_yy, 0, γ_xy, 0, 0); Damage2D plane_strain con
          ε = ε(_xx, ε_yy, γ_xy);
              path multi-paso con carga, descarga, recarga"
      expect: "σ_xx, σ_yy, σ_xy, κ, d coinciden exactamente entre 3D y 2D PS;σ
              _zz en 3D no nulo (consistente con C_e plane_strain interno);
              componentes 3D ausentes en 2D PS σ(_yz, σ_xz) permanecen nulas"
      tol_rel: 1.0e-12

    - name: restriccion_2d_con_las_tres_medidas
      setup: "tests/test_damage_2d_3d_consistency.py: cuatro direcciones, tensión plana
           ε(_zz = -ν-ν/(1)·ε(_xxε+_yy)) y deformación plana, energy/mazars/
          modified_von_mises"
      expect: "κ, d y σ en el plano iguales entre 2D y 3D; σ_zz 3D = 0 en tensión plana
          "
      tol_rel: 1.0e-12

  integration:
    - name: hex8_elastico
      setup: "Hex8 unitario con carga uniforme muy por debajo del umbral"
      expect: "d = 0 en todos los Gauss points; σ = C_e·ε exacto"
      tol_rel: 1.0e-8

    - name: hex8_damage_newton_converges_in_few_iter
      setup: "Hex8 unitario, carga uniaxial hasta régimen plenamente dañado κ( >> κ_0),
           8 pasos"
      expect: "Newton converge en ≤6 iteraciones por paso (evidencia de convergencia
          cuadrática con tangente consistente)"

    - name: tet4_basic_pipeline
      setup: "Tet4 con confinamiento + tracción uniaxial, carga que activa daño"
      expect: "convergencia, d > 0 en el único Gauss point, κ≥κ  _0"

    - name: uniaxial_confinado_curva_analitica
      setup: "tests/validation/test_uniaxial_softening_damage_3d.py: Hex8 confinado,
          u_x prescrito, las tres medidas"
      expect: "σ_xx y d coinciden con la curva analítica con el ε_eq de cada medida
          bajo confinamiento"
      tol_rel: 1.0e-10

    - name: yaml_modified_von_mises
      setup: "Hex8 en tracción uniaxial de esfuerzo leído desde YAML con
          equivalent_strain: modified_von_mises y k"
      expect: "reacción -(1d)·E·δ analítica ε(_eq = δ)"
      tol_rel: 1.0e-8

references:
  - "Lemaitre J., Chaboche J.-L. (1990). Mechanics of Solid Materials. Cambridge UP.
      Cap. 7 (continuum damage mechanics)."
  - "Simó J.C., Ju J.W. (1987). Strain- and stress-based continuum damage models - I.
      Formulation. Int. J. Solids Struct. 23, 821-840 (deformación equivalente energé
```

```
    tica; tangente consistente para daño isótropo)."
- "Mazars J. (1986). A description of micro- and macroscale damage of concrete
    structures. Eng. Fract. Mech. 25, 729-737 (deformación equivalente de Mazars)."
- "de Vree J.H.P., Brekelmans W.A.M., van Gils M.A.J. (1995). Comparison of nonlocal
    approaches in continuum damage mechanics. Comput. Struct. 55, 581-588 (deformació
    n equivalente de von Mises modificado)."
- "de Borst R., Crisfield M.A., Remmers J.J.C., Verhoosel C.V. (2012). Nonlinear
    Finite Element Analysis of Solids and Structures, 2.ª ed. Wiley. Cap. 6 (daño)."
- "Oliver J. (1989). A consistent characteristic length for smeared cracking models.
    IJNME 28, 461-474."
- "de Souza Neto E.A., ćPeri D., Owen D.R.J. (2008). Computational Methods for
    Plasticity. Wiley. §12 (damage models)."
- "ADR 0012 - Sólidos 3D: convención Voigt 6D del proyecto."
- "ADR 0003 - Despachador algebraico: simetría declarada por instancia; tangente asim
    étrica fuerza LU global."
- "ADR 0006 - Tolerancias adimensionales: patrón atol + rtol·escala con escala física
     E·κ_0."
```

11.3.4 Implementación

- **Archivo**: solidum/materials/damage_3d.py; medidas de deformación equivalente en solidum/materials/_equivalent_strain.py, compartidas con `IsotropicDamage2D`.
- **Clase**: `IsotropicDamage3D`, registrada vía `@MaterialRegistry.register`. `IS_SYMMETRIC = False` como ClassVar; el constructor pone `self.IS_SYMMETRIC = True` con `equivalent_strain='energy'`.
- **Composición**:
- `Elastic3D` como base elástica (acceso a $\mathbf{C}_e$ 6×6 vía `self.elastic_base.C`).
- `evaluate_exponential_damage` de `solidum.materials._softening` para la ley de softening (centralizada, compartida con `IsotropicDamage1D` e `IsotropicDamage2D`).
- `equivalent_strain_3d` de `solidum.materials._equivalent_strain`: devuelve $(\varepsilon_{\text{eq}}, \mathbf{v}, s)$ con $\partial\varepsilon_{\text{eq}}/\partial\boldsymbol{\varepsilon} = s\,\mathbf{v}$. Con la energética $\mathbf{v} = \boldsymbol{\sigma}^{\text{eff}}$ y $s = 1/(E\varepsilon_{\text{eq}})$, de modo que la tangente sale simétrica bit a bit. Mazars con `numpy.linalg.eigh` (atajo exacto cuando todas las principales son ≥ 0 o ≤ 0).
- **Estado**: dict `{'kappa': float, 'damage': float}`. `PRIMARY_STATE_VAR = 'damage'`.
- **Algoritmo** `compute_state`: la física vive en el núcleo `@njit _compute_damage_3d`, que consumen `compute_state` y el adaptador por lotes `_damage_3d_batch` (ADR 0014). Estructura idéntica a `IsotropicDamage2D` en Voigt 6D.
- **Camino por lotes**: `batch_params = [E, κ_0, α, tol_abs, tol_rel, DAMAGE_MAX, ν, código de la medida, k]` (códigos 0 energy, 1 mazars, 2 modified_von_mises; `k = 0` sin uso fuera de von Mises modificado). Cacheable en disco (verificado: kernel de familia y núcleos del material cargan del caché de Numba sin recompilar).
- `admissibility_scale`: $E{\cdot}\kappa_0$ constante respecto al estado (idéntico al patrón 1D/2D — el criterio se mide contra κ_0, no contra κ corriente).

- **Tests**:
- tests/test_materials_unit.py · `TestIsotropicDamage3D` + `TestIsotropicDamage3DvsPlaneStrain` (cruzado contra Damage2D plane_strain).
- tests/test_damage_equivalent_strain.py: las tres medidas (valores frente a referencia independiente, calibración, compresión hidrostática, invariancia, tangente por diferencias finitas, lotes, simetría, constructor, YAML).
- tests/test_damage_2d_3d_consistency.py: el 2D es la restricción exacta del 3D con las tres medidas.
- tests/test_solid_3d_plasticity.py · 3 tests de integración: `TestHex8IsotropicDamage3DElastic` (1 — sin daño), `TestHex8IsotropicDamage3DActiveLoad` (1 — daño activo), `TestTet4IsotropicDama` (1 — smoke Tet4).
- tests/validation/test_uniaxial_softening_damage_3d.py: curva σ-ε analítica bajo confinamiento con las tres medidas.
- **Notas de traducción**:
- Convención engineering input (γ_ij): en J_2 los cortantes entran como $\gamma^2/4 = \varepsilon_{ij}^2$ y en el tensor de Mazars como $\gamma/2$; la derivada respecto de γ_{ab} es la componente tensorial correspondiente del gradiente.
- Validación de parámetros en constructor: `E > 0`, κ`_0 > 0`, α `> 0`, $\nu \in$ `(-1, 0.5)`, `density` $\geq$ `0` si se pasa, `equivalent_strain` y `k`. Mensajes en español con el nombre del material y del parámetro.

11.3.5 Diálogo

- **2026-05-21** · Spec pre-redactada por la IA al cerrar el segundo material de A.bis (`DruckerPrager3D`). Base: `IsotropicDamage2D.md` (modelo físico, tangente algorítmica consistente, semánticas carga/descarga, cap DAMAGE_MAX) + `VonMises3D.md`/`DruckerPrager3D.md` (estructura Voigt 6D, patrón cross-consistency).
- **2026-05-21** · **Decisión de plumbing (decide la IA, Reglas §3)**: reuso de la función centralizada `evaluate_exponential_damage` de `solidum.materials._softening`. Esta función ya está compartida entre `IsotropicDamage1D` e `IsotropicDamage2D` (auditoría H-3.4 cerrada el 2026-05-19); añadir `IsotropicDamage3D` como tercer consumidor satisface la regla de centralización del proyecto. Sin duplicación de la ley de softening.
- **2026-05-21** · **Decisión de plumbing**: el material usa `Elastic3D` internamente como base elástica (paralelo a cómo `IsotropicDamage2D` usa `Elastic2D`). Acceso a la matriz constitutiva 6×6 vía `self.elastic_base.C` — idéntico patrón al 2D, sin duplicación de la construcción de $\mathbf{C}_e$.
- **2026-05-21** · **Decisión de plumbing**: la matriz $\mathbf{M}$ del cálculo de ε_{eq} y de la derivada $\partial\kappa/\partial\boldsymbol{\varepsilon}$ se materializa implícitamente en código (multiplicaciones por 1/2 sobre los slots cortantes) en

lugar de allocar una 6×6 — ahorro mínimo de memoria pero código más legible y consistente con el patrón seguido en `damage_2d.py`. *(Obsoleta desde el 2026-09-29: la norma del tensor se retiró.)*

- **2026-05-21** · **Cross-consistency** contra `IsotropicDamage2D` plane_strain, no plane_stress. La equivalencia matemática es exacta bajo $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ impuesto (plane strain): la matriz $\mathbf{C}_e$ 3D restringida a ε_zz=0 input se reduce a la matriz $\mathbf{C}_e^{\text{plane strain}}$ 2D, y $\varepsilon_{\text{eq}}^{\text{3D}}$ se reduce a $\varepsilon_{\text{eq}}^{\text{2D}}$ bajo la misma restricción. Plane stress 2D usa un $\mathbf{C}_e$ proyectado distinto y por tanto no es comparable directamente con 3D bajo restricción cinemática. *(Desde el 2026-09-29 también la tensión plana es la restricción exacta del 3D, con el ε_{zz} que anula σ_{zz}; ver `IsotropicDamage2D.md`.)*

- **2026-05-21** · **Sin variantes de hipótesis** en 3D (igual que VM3D, DP3D, Elastic3D). Constructor más simple que el 2D.

- **2026-05-21** · **No se incluye consistent tangent para** `IsotropicDamage1D` (deuda gemela del 2D): fuera del alcance de esta entrega A.bis. La fórmula 1D es trivial reducción de la 3D si se prioriza después.

- **2026-05-21** · **Implementación completa, status** `validated`. Clase + 12 tests unitarios + 1 test cruzado + 3 tests de integración (Hex8 elástico, Hex8 con daño activo + tangente asimétrica, Tet4 smoke). **16 tests añadidos a la suite (848 → 866 verdes; suite global sin regresiones).** Todos los tests pasaron a la primera, incluyendo:

- Cruzado Damage3D ↔ Damage2D plane_strain a **14 decimales en `d` y** κ y 10 decimales en σ — la equivalencia bajo $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ es exacta a precisión máquina (misma ley centralizada, mismo $\mathbf{C}_e$ plane_strain bajo restricción cinemática).

- Diferencia finita de la tangente algorítmica vs cerrada con error rel. < 1e-5 en las 36 componentes 6×6 en carga activa.

- Asimetría significativa de la tangente en estados no degenerados (norma rel. ≥ 0.01).

- Invariancia bajo cambio de unidades (E en MPa vs Pa) — `d`, κ idénticos a 12 decimales; σ escala linealmente con E.

Con esta entrega cierra la sub-etapa A.bis (materiales 3D no lineales): VonMises3D + DruckerPrager3D + IsotropicDamage3D. Validación contra benchmark publicado (e.g. ensayo tensión-softening con valor cuantitativo de carga máxima en barra cargada axialmente con softening exponencial calibrado por G_f efectivo) **diferida a la campaña 3D consolidada** que se hará al cierre de A.bis junto con Hill 3D para VM3D y esfera hueca para DP3D.

- **2026-09-22** · Ensamblaje por lotes (ADR 0014): la física de `compute_state` se movió a un núcleo `@njit` (`_compute_damage_3d`) que consumen el camino por elemento y el adaptador por lotes; `evaluate_exponential_damage` se compila y recibe el cap como argumento. Sin cambio de formulación; `STATE_SCHEMA = {kappa, damage}` con estado inicial κ = κ_0.

- **2026-09-29 · Cambio de formulación aprobado por el usuario** (segunda mitad del hallazgo clas-3 de la auditoría de pureza; la primera, la ε_{zz} del 2D en tensión plana, en `IsotropicDamage2D.md`): se retira la norma del tensor $\sqrt{\boldsymbol{\varepsilon} : \boldsymbol{\varepsilon}}$, convención propia no estándar y mal calibrada (en tracción uniaxial de esfuerzo $\varepsilon_{\text{eq}} = \varepsilon\sqrt{1+2\nu^2}$, así que $\kappa_0 \neq f_t/E$). La deformación equivalente pasa a ser configurable con `equivalent_strain`: **energética** por defecto (Simó y Ju 1987; simétrica en tracción y compresión, como la anterior), y **Mazars** y **von Mises modificado** (este con `k = f_c/f_t` obligatorio) como opciones que distinguen tracción de compresión. Las tres dan $\varepsilon_{\text{eq}} = \varepsilon$ en tracción uniaxial ($\kappa_0 = f_t/E$). Ley de daño, variable histórica, `DAMAGE_MAX`, esfuerzo y ramas de la tangente sin cambios. Con la energética la tangente consistente es simétrica y la instancia declara `IS_SYMMETRIC = True` (§8). Las tres medidas viven en `solidum/materials/_equivalent_strain.py`, compartido con el 2D; el camino por lotes recibe la medida como código numérico en `batch_params`. El benchmark de validación uniaxial confinado se recalculó con el ε_{eq} de cada medida bajo confinamiento (energética: $\varepsilon_{xx}\sqrt{(1-\nu)/[(1+\nu)(1-2\nu)]}$).

11.4 VonMises3D

*Orden de trabajo. El usuario revisa **especificación física**, **formulación numérica** y **contrato**; la IA propone, ejecuta y rellena **implementación** + **diálogo** durante el trabajo.*

***Pre-redactada por la IA** sobre la base de `VonMises2D.md` y `Elastic3D.md`. Apertura de la sub-etapa **A.bis** (materiales 3D no lineales) tras cierre de la Etapa 7 (sólidos 3D acotados con ADR 0012).*

11.4.1 Especificación física

0. Descripción general

Modelo elastoplástico independiente de la velocidad ("rate-independent") basado en el criterio **J2 / Von Mises** con regla de flujo **asociada** y **endurecimiento isótropo lineal**, formulado **íntegramente en 3D** sobre la convención Voigt 6D del proyecto (ADR 0012). Captura plasticidad de metales dúctiles en régimen de pequeñas deformaciones, sin efectos cinemáticos (Bauschinger), térmicos ni viscosos.

A diferencia de `VonMises2D`, **no tiene variantes de hipótesis cinemática**: en 3D todas las componentes de $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ son activas; no hay proyección a un subespacio. La consecuencia es algorítmica: el return mapping es **único y radial cerrado**, sin Newton local proyectado.

`VonMises2D` en hipótesis **plane strain** es matemáticamente la restricción de este modelo bajo $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ (deformación total impuesta); las componentes plásticas $\varepsilon^p_{zz}, \varepsilon^p_{yz}, \varepsilon^p_{xz}$ son libres y evolucionan por incompresibilidad/isotropía. Esta relación se verifica como test cruzado en §13.

1. Descomposición aditiva

En pequeñas deformaciones, el tensor de deformación se descompone aditivamente:

$$\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e + \boldsymbol{\varepsilon}^p$$

La parte elástica $\boldsymbol{\varepsilon}^e$ gobierna el esfuerzo vía la ley elástica isótropa; la parte plástica $\boldsymbol{\varepsilon}^p$ es **histórica** y evoluciona según la regla de flujo. La parte plástica es **incompresible**: $\operatorname{tr}(\boldsymbol{\varepsilon}^p) = 0$.

2. Ley elástica

$$\boldsymbol{\sigma} = \mathbf{C}_e : \boldsymbol{\varepsilon}^e = \mathbf{C}_e : (\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p)$$

con $\mathbf{C}_e$ la matriz constitutiva isótropa 3D de `Elastic3D` (Voigt 6×6), parametrizada por (E, ν) o equivalentemente (K, G):

$$K = \frac{E}{3(1-2\nu)}, \qquad G = \frac{E}{2(1+\nu)}$$

Equivalente desviador + presión:

$$\boldsymbol{\sigma} = \mathbf{s} + p\,\mathbf{I}, \qquad \mathbf{s} = 2G\,\mathbf{e}^e, \qquad p = K\ \operatorname{tr}(\boldsymbol{\varepsilon})$$

con $\mathbf{e}^e = \operatorname{dev}(\boldsymbol{\varepsilon}^e)$ y, por incompresibilidad plástica, $\operatorname{tr}(\boldsymbol{\varepsilon}^e) = \operatorname{tr}(\boldsymbol{\varepsilon})$.

3. Superficie de fluencia (J2)

$$f(\boldsymbol{\sigma}, \alpha) = \|\mathbf{s}\| - \sqrt{\tfrac{2}{3}}\,(\sigma_y + H\,\alpha) \;\le\; 0$$

donde $\mathbf{s} = \boldsymbol{\sigma} - \frac{1}{3}\,\mathrm{tr}(\boldsymbol{\sigma})\mathbf{I}$ es el tensor desviador, $\|\mathbf{s}\| = \sqrt{\mathbf{s} : \mathbf{s}}$ la norma de Frobenius tensorial, $\sigma_y > 0$ la fluencia inicial y $H \ge 0$ el módulo de endurecimiento isótropo lineal. El factor $\sqrt{2/3}$ alinea σ_y con la fluencia uniaxial estándar.

Cómputo de $\|\mathbf{s}\|$ en Voigt 6D del proyecto (componentes cortantes tensoriales sin factor 2):

$$\|\mathbf{s}\|^2 = s_{xx}^2 + s_{yy}^2 + s_{zz}^2 + 2\,(s_{xy}^2 + s_{yz}^2 + s_{xz}^2)$$

El factor 2 contabiliza las dos posiciones simétricas $s_{ij} = s_{ji}$ del tensor de 2º orden.

4. Regla de flujo (asociada)

$$\dot{\boldsymbol{\varepsilon}}^p = \dot{\gamma}\,\frac{\partial f}{\partial \boldsymbol{\sigma}} = \dot{\gamma}\,\mathbf{N}, \qquad \mathbf{N} = \frac{\mathbf{s}}{\|\mathbf{s}\|}$$

El flujo es puramente desviador ($\mathrm{tr}(\mathbf{N}) = 0$) — **plasticidad incompresible**, propiedad central de J2. $\mathbf{N}$ es un tensor de 2º orden de norma unidad almacenado en Voigt 6D con cortantes tensoriales.

5. Endurecimiento isótropo lineal

$$\dot{\alpha} = \sqrt{\tfrac{2}{3}}\,\dot{\gamma}$$

α es la **deformación plástica acumulada equivalente** (escalar, monótona no decreciente). La superficie de fluencia se expande uniformemente con α manteniendo su centro en el origen del espacio de tensiones desviadoras. La fórmula es **idéntica a plane strain** porque la dimensión de $\mathbf{N}$ es la misma (norma unidad tensorial); contrasta con plane stress, donde $\Delta\gamma$ tiene unidades de [1/esfuerzo] por la proyección.

6. Condiciones de Kuhn-Tucker

$$\dot{\gamma} \ge 0, \qquad f \le 0, \qquad \dot{\gamma}\,f = 0$$

Garantizan que $\dot{\gamma} > 0$ solo si el estado intenta salir de la superficie, y que el estado real siempre cumple $f \le 0$.

7. Variables internas

Símbolo	Tipo	Significado
$\boldsymbol{\varepsilon}^p$	tensor (6 componentes Voigt, ver §8)	deformación plástica
α	escalar	deformación plástica acumulada equivalente

`alpha` es la variable principal exportable (`PRIMARY_STATE_VAR = 'alpha'`); `eps_p` es interna al algoritmo de return mapping.

8. Notación Voigt y manejo de ε^p

Convención del proyecto para 3D (ADR 0012):

$$\boldsymbol{\varepsilon} = [\varepsilon_{xx},\, \varepsilon_{yy},\, \varepsilon_{zz},\, \gamma_{xy},\, \gamma_{yz},\, \gamma_{xz}]^\top, \qquad \gamma_{ij} = 2\,\varepsilon_{ij}$$

$$\boldsymbol{\sigma} = [\sigma_{xx},\, \sigma_{yy},\, \sigma_{zz},\, \sigma_{xy},\, \sigma_{yz},\, \sigma_{xz}]^\top$$

Almacenamiento interno de ε^p: 6 componentes con cortantes **tensoriales sin factor 2**:

$$\boldsymbol{\varepsilon}^p = [\varepsilon^p_{xx},\, \varepsilon^p_{yy},\, \varepsilon^p_{zz},\, \varepsilon^p_{xy},\, \varepsilon^p_{yz},\, \varepsilon^p_{xz}]$$

Decisión consistente con `VonMises2D` (que almacena ε^p_{xy} tensorial en su slot 4). Razón: el flujo plástico $\mathbf{N} = \mathbf{s}/\|\mathbf{s}\|$ es un tensor de 2º orden con cortantes tensoriales; almacenar $\boldsymbol{\varepsilon}^p$ en la misma convención evita conversiones repetidas en el return mapping y mantiene la fórmula $\|\boldsymbol{\varepsilon}^p\|^2 = \sum_{\text{diag}} + 2\sum_{\text{off}}$ idéntica a la de $\mathbf{s}$.

Traducción al entrar al kernel: la deformación total llega con γ *engineering*; las componentes cortantes se dividen por 2 una sola vez al construir el tensor para deviator decomposition. Al salir, $\boldsymbol{\sigma}$ se devuelve en Voigt 6D *engineering-compatible* (cortantes tensoriales sin factor — esto es lo que el ensamblador espera).

9. Relación con `VonMises2D` plane strain

`VonMises2D` plane strain es la restricción de este modelo bajo:

$$\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0 \quad \text{(deformación total impuesta por la hipótesis 2D)}$$

con $\varepsilon^p_{zz} \neq 0$ libre (incompresibilidad plástica). El kernel 2D-plane-strain mantiene 4 componentes de $\boldsymbol{\varepsilon}^p$ porque $\varepsilon^p_{yz} = \varepsilon^p_{xz} = 0$ por isotropía + ausencia de cortantes activos en esos planos.

Implicación de testeo: si se ejecuta `VonMises3D` con $\varepsilon = [\varepsilon_{xx}, \varepsilon_{yy}, 0, \gamma_{xy}, 0, 0]$ (path plane strain), debe reproducir $\sigma_{xx}, \sigma_{yy}, \sigma_{xy}, \alpha$ de `VonMises2D` plane strain exactamente, además de devolver el σ_{zz} que `VonMises2D` calcula internamente pero no expone en el API público 2D. Ver §13.

11.4.2 Formulación numérica

10. Esquema temporal — Backward Euler implícito

Dado el estado convergido $\{\boldsymbol{\varepsilon}^p_n, \alpha_n\}$ al paso n y la deformación total prescrita $\boldsymbol{\varepsilon}_{n+1}$, resolver para $\{\boldsymbol{\sigma}_{n+1}, \boldsymbol{\varepsilon}^p_{n+1}, \alpha_{n+1}\}$ con discretización implícita:

$$\boldsymbol{\varepsilon}^p_{n+1} = \boldsymbol{\varepsilon}^p_n + \Delta\gamma\, \mathbf{N}_{n+1}, \qquad \alpha_{n+1} = \alpha_n + \sqrt{\tfrac{2}{3}}\,\Delta\gamma$$

más $f(\boldsymbol{\sigma}_{n+1}, \alpha_{n+1}) \leq 0$ con la condición de complementariedad.

11. Return mapping 3D — algoritmo radial cerrado

Descomposición volumétrica-desviadora del estado de prueba. Conversión de $\boldsymbol{\varepsilon}$ a tensorial dividiendo por 2 los cortantes engineering:

$$\boldsymbol{\varepsilon}^{\text{tens}} = [\varepsilon_{xx},\, \varepsilon_{yy},\, \varepsilon_{zz},\, \tfrac{\gamma_{xy}}{2},\, \tfrac{\gamma_{yz}}{2},\, \tfrac{\gamma_{xz}}{2}]$$

(esta es la conversión que ya hace `Elastic3D` implícitamente vía la matriz $\mathbf{C}_e$ con sus factores $(1-2\nu)/2$ en los cortantes).

Deformación de prueba elástica y su descomposición:

$$\boldsymbol{\varepsilon}^{e,\text{trial}}_{n+1} = \boldsymbol{\varepsilon}^{\text{tens}}_{n+1} - \boldsymbol{\varepsilon}^{p}_{n}$$

$$p^{\text{trial}} = K\ \text{tr}(\boldsymbol{\varepsilon}_{n+1}) = K\,(\varepsilon_{xx} + \varepsilon_{yy} + \varepsilon_{zz})$$

(la traza no depende de la sustracción de $\boldsymbol{\varepsilon}^p_n$ porque $\text{tr}(\boldsymbol{\varepsilon}^p) = 0$.)

$$\mathbf{e}^{\text{trial}} = \text{dev}(\boldsymbol{\varepsilon}^{e,\text{trial}}_{n+1}), \qquad \mathbf{s}^{\text{trial}} = 2G\,\mathbf{e}^{\text{trial}}$$

con $\mathbf{e}^{\text{trial}}$ y $\mathbf{s}^{\text{trial}}$ ambos en Voigt 6D tensorial.

Función de fluencia trial:

$$f^{\text{trial}} = \|\mathbf{s}^{\text{trial}}\| - \sqrt{\tfrac{2}{3}}\,(\sigma_y + H\,\alpha_n)$$

con $\|\mathbf{s}^{\text{trial}}\|^2 = \sum_{\text{diag}} s_i^2 + 2\sum_{\text{off}} s_{ij}^2$ (§3).

Si $f^{\text{trial}} \le \text{tol}$ → paso elástico. $\boldsymbol{\sigma}_{n+1} = \mathbf{C}_e\,\boldsymbol{\varepsilon}^e_n + \mathbf{C}_e\,\Delta\boldsymbol{\varepsilon}$ equivalente, evaluado como $\mathbf{s}^{\text{trial}} + p^{\text{trial}}\,\mathbf{I}$. Estado interno intacto.

Si $f^{\text{trial}} > 0$ → corrector plástico radial. Por isotropía + asociado en J2, la dirección de flujo no cambia entre trial y final: $\mathbf{N}_{n+1} = \mathbf{N}^{\text{trial}} = \mathbf{s}^{\text{trial}}/\|\mathbf{s}^{\text{trial}}\|$. Forma cerrada:

$$\boxed{\Delta\gamma = \frac{f^{\text{trial}}}{2G + \frac{2}{3}H}}$$

Actualización (todo en Voigt 6D tensorial):

$$\mathbf{s}_{n+1} = \mathbf{s}^{\text{trial}} - 2G\,\Delta\gamma\,\mathbf{N}, \qquad \boldsymbol{\sigma}_{n+1} = \mathbf{s}_{n+1} + p^{\text{trial}}\,\mathbf{I}$$

$$\boldsymbol{\varepsilon}^p_{n+1} = \boldsymbol{\varepsilon}^p_n + \Delta\gamma\,\mathbf{N}, \qquad \alpha_{n+1} = \alpha_n + \sqrt{\tfrac{2}{3}}\,\Delta\gamma$$

Coste: O(1) por punto de Gauss. Sin Newton local. **Algoritmo idéntico al de plane strain extendido a 6D** — solo cambian los índices de las componentes activas.

12. Tangente algorítmica consistente

Forma cerrada estándar (Simó-Hughes 1998, §3.3):

$$\mathbf{C}^{\text{alg}} = K\,\mathbf{1}\otimes\mathbf{1} + 2G(1-\beta)\,\mathbf{I}_{\text{dev}} - 2G\,\bar{\gamma}\,\mathbf{N}\otimes\mathbf{N}$$

con

$$\beta = \frac{2G\,\Delta\gamma}{\|\mathbf{s}^{\text{trial}}\|}, \qquad \bar{\gamma} = \frac{1}{1 + H/(3G)} - \beta$$

En Voigt 6D del proyecto:

- $\mathbf{1}\otimes\mathbf{1}$: matriz 6×6 con `1` en las entradas (i,j) con $i,j \in \{1,2,3\}$ y `0` resto.
- $\mathbf{I}_{\text{dev}}$: proyector desviador 6×6. En la base tensorial, $\mathbf{I}_{\text{dev}} = \mathbf{I} - \frac{1}{3}\mathbf{1}\otimes\mathbf{1}$ en el bloque diagonal (3×3) y un bloque identidad escalado en los cortantes con el factor adecuado para que $\mathbf{I}_{\text{dev}}\,\boldsymbol{\varepsilon} = \text{dev}(\boldsymbol{\varepsilon})$ cuando $\boldsymbol{\varepsilon}$ está en convención engineering. En la convención del proyecto (entrada engineering, salida engineering), el bloque cortante de $\mathbf{I}_{\text{dev}}$ es la identidad 3×3 simple.
- $\mathbf{N}\otimes\mathbf{N}$: producto exterior 6×6 de $\mathbf{N}$ (Voigt 6D tensorial) consigo mismo. **Cuidado con la convención**: el producto $\mathbf{N}\otimes\mathbf{N}$ del kernel debe ser consistente con $\mathbf{C}^{\text{alg}}\,\boldsymbol{\varepsilon}$ engineering. La forma simple: si $\mathbf{N}$ está almacenado tensorial (sin factor 2 en cortantes), entonces $\mathbf{C}^{\text{alg}}$ en convención engineering tiene $\mathbf{N}\otimes\mathbf{N}$ con un factor de 2 en las posiciones cortantes — equivalente a multiplicar las componentes cortantes de $\mathbf{N}$ por $\sqrt{2}$ antes del producto exterior (Mandel-like rescaling local al kernel, idéntico al patrón de VM2D plane strain).

Simetría: $\mathbf{C}^{\text{alg}}$ es simétrica (J2 asociado), `IS_SYMMETRIC = True`.
Coste: precalcular los cuatro escalares $K, G, \beta, \bar{\gamma}$ y construir la 6×6 sin loops anidados — O(1) constante.

13. Caveats numéricos

- **Locking volumétrico** en `Hex8`/`Tet4` cuando la plasticidad domina: la incompresibilidad plástica $\text{tr}(\boldsymbol{\varepsilon}^p) = 0$ combinada con ν no muy lejano de 0.5 produce el mismo locking documentado para `Elastic3D` (limitación declarada, blindada por test en la Etapa 7). B-bar/F-bar diferidas a caso de uso real.
- $\nu \to 0.5$: $K \to \infty$ y el predictor elástico pierde precisión. Constructor rechaza $\nu \geq 0.5$. Recomendado $\nu \leq 0.49$ en metales.
- $H = 0$ **(plasticidad perfecta)**: $\Delta\gamma = f^{\text{trial}}/(2G)$, sin singularidades. Carga aplicada que supere la capacidad plástica produce divergencia del Newton global del solver (`LoadExceedsCapacityError`, ADR 0011); responsabilidad del solver, no del material.
- **Tangente algorítmica vs continua**: con $\Delta\gamma > 0$, $\mathbf{C}^{\text{alg}} \neq \mathbf{C}^{\text{ep}}_{\text{continuo}}$ por el término β. Usar la algorítmica garantiza convergencia cuadrática del Newton global.
- **Tolerancia de fluencia** vía ADR 0006: `admissibility_scale =` $\sqrt{(2/3)}$`(`σ`_y + H·`α`)`. Idéntica a plane strain — la escala física es la misma norma desviadora en la frontera de fluencia.

11.4.3 Contrato de implementación

```
name: VonMises3D
kind: material
status: validated        # draft → implemented → validated

interface:
  strain_dim: 6
  primary_state_var: alpha      # deformación plástica acumulada equivalente
  is_symmetric: true            # tangente algorítmica simétrica (J2 asociado)

admission:                 # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_souza_neto: "cap. 7 (von Mises con endurecimiento isótropo: retorno radial y
        tangente consistente)"
    - de_borst_crisfield: "§7.2-7.4 (J2: retorno y tangente consistente)"
  general_codes:
    - abaqus: "*PLASTIC (Mises)"
    - ansys: "TB,BISO"
    - opensees: "nDMaterial J2Plasticity"

parameters:
  - { name: E,       type: float, required: true,  desc: "Módulo de Young (>0)" }
  - { name: nu,      type: float, required: true,  desc: "Coeficiente de Poisson ∈ (-1,
      0.5)" }
  - { name: sigma_y, type: float, required: true,  desc: "Esfuerzo de fluencia inicial
      (>0)" }
  - { name: H,       type: float, required: false, default: 0.0,
      desc: "Módulo de endurecimiento isótropo lineal ≥(0). H=0 ⇒ plasticidad perfecta"
        }
  - { name: density, type: float, required: false, default: null,
      desc: "Densidad (kg/m^3). Opcional al construir; obligatoria solo si se ensambla
          peso propio o masa (ADR 0008)" }

signature:
  compute_state: "ε(: ndarray(6,), state_vars=None) -> σ(: ndarray(6,), C_alg: ndarray
      (6,6), state_vars')"
  strain_kind: "3D Voigt -ε [_xx, ε_yy, ε_zz, γ_xy, γ_yz, γ_xz] con γ_ij = 2·ε_ij"

state_schema:
  eps_p: "ndarray(6,) -ε [^p_xx, ε^p_yy, ε^p_zz, ε^p_xy, ε^p_yz, ε^p_xz] (cortantes
      tensoriales, sin factor 2)"
  alpha: "float ≥ 0 - deformación plástica acumulada equivalente"

conventions:
  sign:  "σ > 0 ⇔ tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] (ADR 0012); γ_ij = 2·ε_ij en deformación total
      engineering;ε
          ^p_ij almacenada en componente tensorial (sin factor 2);σ
          _ij tensorial sin factor"

validity:
  - "E > 0, σ_y > 0, H ≥ 0"
  - "ν ∈ (-1, 0.5) estricto; ν = 0.5 rechazado (incompresible)"
  - "ε|| ≤ 1e-2 (régimen de pequeñas deformaciones)"
```

```
out_of_scope:
  - "endurecimiento cinemático (back-stress); usar variante futura VonMises3DKinematic"
  - "endurecimiento isótropo no lineal (Voce, exponencial, potencial)"
  - "ablandamiento (H<0); requiere regularización por longitud característica o no-
      local"
  - "regla de flujo no asociada"
  - "acoplamiento con daño; usar modelo plástico-dañado dedicado"
  - "viscoplasticidad y dependencia de la velocidad de deformación"
  - "grandes deformaciones (descomposición multiplicativa F = F^e·F^p)"
  - "anisotropía (Hill 1948, Barlat); el modelo es estrictamente J2 isótropo"
  - "incompresibilidad estricta ν(=0.5)"

numerical_caveats:
  - "Locking volumétrico en Hex8/Tet4 cuando la plasticidad domina (incompresibilidad
      plástica + ν moderado-alto). Mitigaciones B-bar/F-bar diferidas. Test específico
      documenta la limitación."
  - "ν cercano a 0.5: K diverge; recomendado ν≤  0.49 (típico ν=0.3 en metales)."
  - "H=0 (plasticidad perfecta): material correcto, divergencia bajo sobrecarga es
      responsabilidad del solver (ADR 0011)."

acceptance:
  # Cubierto por tests/test_materials_unit.py · TestVonMises3D (a crear).
  unit:
    - name: paso_elastico_bajo_fluencia
      setup: "ε = (1e-5, 0, 0, 0, 0, 0): tracción uniaxial por debajo del yield"
      expect: "σ = C_e·ε exacto; α = 0; eps_p = 0; tangente = C_e"
      tol_rel: 1.0e-12

    - name: yield_uniaxial_at_sigma_y
      setup: "tracción uniaxial monotónica creciente con ε_yy = ε_zz = ν-·ε_xx, ε_xy =
          ε_yz = ε_xz = 0"
      expect: "primer yield exactamente en σ_xx = σ_y; eps_yield = σ_y/E"
      tol_rel: 1.0e-10

    - name: traccion_biaxial_isotropa
      setup: "ε = (eps, eps, ν-2·epsν/(1-), 0, 0, 0) - tracción biaxial isótropa en
          plano x-y con ε_zz libre por σ_zz=0"
      expect: "σ_xx = σ_yy convergen a yield uniforme; α evoluciona; σ_zz ≈ 0"
      tol_rel: 1.0e-4

    - name: cortante_puro_xy_yield
      setup: "γ_xy creciente con resto de componentes 0"
      expect: "primer yield en τ_xy = σ_y√/3 (criterio J2 en cortante puro)"
      tol_rel: 1.0e-10

    - name: incompresibilidad_plastica
      setup: "cualquier paso plástico monotónico"
      expect: "tr(eps_p) = ε^p_xx + ε^p_yy + ε^p_zz = 0 exacto"
      tol_abs: 1.0e-12

    - name: alpha_monotonic
      setup: "carga monotónica creciente con incrementos plásticos no triviales"
      expect: "α no decrece nunca; α_final > 0 si hubo plasticidad"

    - name: descarga_recupera_C_e
      setup: "tras estado plástico, paso con menor ε (descarga elástica)"
```

```
      expect: "σ devuelto coherente con eps_p_n acumulado; tangente devuelta = C_e; α
          intacto"
      tol_rel: 1.0e-10

    - name: tangente_simetrica
      setup: "evaluar C_alg en estado plástico arbitrario"
      expect: "máx |C_alg - C_alg.T| ≤ tol; IS_SYMMETRIC = True"
      tol_abs: 1.0e-8

    - name: unit_invariance_MPa_vs_Pa
      setup: "mismo path adimensional con (E=210, σ_y=0.25) y (E=210e9, σ_y=0.25e9)"
      expect: "α, eps_p (adimensionales) idénticos; σ/E idéntico"
      tol_rel: 1.0e-12

    - name: degeneracion_a_elasticidad_sin_plasticidad
      setup: "trayectoria que nunca alcanza la frontera de fluencia"
      expect: "todos los pasos devuelven C_alg = C_e; eps_p y α permanecen en cero"
      tol_rel: 1.0e-12

    - name: rechazo_inputs_invalidos
      setup: "construir VonMises3D con ≤E0, ν fuera de rango, σ≤_y0, H<0"
      expect: "ValueError con mensaje claro en cada caso"

  # Test cruzado VM3D ↔ VM2D plane strain.
  cross_consistency:
    - name: equivalencia_plane_strain
      setup: "ejecutar VM3D con ε = ε(_xx, ε_yy, 0, γ_xy, 0, 0) y VM2D plane strain con
           ε = ε(_xx, ε_yy, γ_xy);
             mismo path multi-paso con plasticidad activa"
      expect: "σ_xx, σ_yy, σ_xy y α coinciden entre 3D y 2D PS;
              VM3D además devuelve σ_zz coherente con la incompresibilidad plástica"
      tol_rel: 1.0e-10

  # Cubierto por tests/test_solid_3d_plasticity.py (a crear, fuera de esta spec -
      pertenece a la integración Hex8/Tet4 + VM3D).
  integration:
    - name: pipeline_hex8_traccion_uniaxial_elastoplastica
      setup: "Hex8 unitario, condiciones de contorno equivalentes a tracción uniaxial
          pura
              (4 caras laterales restringidas en su dirección normal, una cara tirada en
                   x).
              Carga monotónica que cruza el yield."
      expect: "σ_xx en el centro coincide con material standalone bajo el path
          equivalente;α
                coincide; convergencia Newton global en pocos pasos"
      tol_rel: 1.0e-6

    - name: pipeline_hill_cylinder_3d
      setup: "cilindro hueco bajo presión interna creciente discretizado con Hex8 (
          octante con simetrías);
              cargar hasta plasticidad parcial"
      expect: "frontera elasto-plástica radial coincide con solución analítica cerrada
          de Hill (1950)
               al avanzar la presión"
      tol_rel: 1.0e-2
```

```
references:
  - "Simó J.C., Hughes T.J.R. (1998). Computational Inelasticity. Springer.
     §3.3 (3D J2 return mapping radial cerrado, tangente algorítmica consistente)."
  - "Crisfield M.A. (1991). Non-linear Finite Element Analysis of Solids and Structures
      , Vol. 1. Wiley.
     Cap. 6 (J2 plasticity)."
  - "de Souza Neto E.A., ćPeri D., Owen D.R.J. (2008). Computational Methods for
      Plasticity. Wiley.
     §7 (3D J2 plasticity)."
  - "Hill R. (1950). The Mathematical Theory of Plasticity. Oxford University Press.
     Cap. V (cilindro hueco bajo presión interna, solución analítica)."
  - "Bathe K.J. (2014). Finite Element Procedures. Prentice Hall. §6.6, §6.8.4."
  - "ADR 0012 - Sólidos 3D: convención Voigt 6D del proyecto."
  - "ADR 0006 - Tolerancias adimensionales: patrón atol + rtol·escala con escala física
      ."
```

11.4.4 Implementación

- **Archivo**: solidum/materials/von_mises_3d.py.
- **Clase**: `VonMises3D`, registrada vía `@MaterialRegistry.register`. Sin despacho por hipótesis (no aplica en 3D).
- **Kernel Numba** `_compute_j2_3d`: descomposición volumétrica-desviadora en Voigt 6D tensorial, return mapping radial cerrado, tangente algorítmica consistente cerrada. Predictor elástico construido como $\mathbf{s}_{\text{trial}} + p\,\mathbf{I}$ con $\mathbf{s}_{\text{trial}} = 2G(\mathbf{e}^{\text{dev}}_{\text{trial}} - \boldsymbol{\varepsilon}^p_n)$ — respeta $\boldsymbol{\varepsilon}^p_n$ en descargas y reevaluaciones (aplicada desde el inicio la lección del bug detectado en VM2D plane strain el 2026-05-14).
- **State schema**: `eps_p` ndarray(6,) `[xx, yy, zz, xy, yz, xz]` con cortantes tensoriales, `alpha` escalar.
- `admissibility_scale`: $\sqrt{(2/3)}\cdot(\sigma_y + H\cdot\alpha)$ — idéntica a VM2D plane strain (misma escala física en la frontera J2).
- **Tests**:
- tests/test_materials_unit.py · `TestVonMises3D` (12 tests unitarios) + `TestVonMises3DvsPlaneStrain` (1 test cruzado VM3D ↔ VM2D plane strain con path multi-paso).
- tests/test_solid_3d_plasticity.py · 5 tests de integración: `TestHex8VonMises3DUniaxialFree` (2 — cubo libre con Poisson, yield uniaxial), `TestHex8VonMises3DConfinedVsStandalone` (1 — cross-check contra material standalone), `TestHex8VonMises3DConvergence` (1 — Newton converge en pocas iteraciones), `TestTet4VonMises3DBasic` (1 — smoke test Tet4 + VM3D).
- **Notas de traducción**:
- Convención mixta engineering/tensorial idéntica a VM2D: entrada `strain` engineering ($\gamma_ij = 2\cdot\varepsilon_ij$), salida `sigma` con cortantes tensoriales (lo que el ensamblador espera), estado `eps_p` con cortantes tensoriales. La conversión engineering→tensorial se hace una sola vez al construir el desviador (`strain[3..5]/2`).

- La tangente algorítmica $\mathbf{C}^{\text{alg}}$ usa el patrón $K\,\mathbf{1}\otimes\mathbf{1} + 2G(1-\beta)\,\mathbf{I}_{\text{dev}} - 2G\,\bar{\gamma}\,\mathbf{N}\otimes\mathbf{N}$ con $\mathbf{I}_{\text{dev}}$ teniendo 1/2 en los cortantes (mapea engineering input a tensorial output) y $\mathbf{N}\otimes\mathbf{N}$ con $\mathbf{N}$ tensorial. Esta combinación produce la convención correcta `engineering→tensorial` en la salida sin factores adicionales — verificado por el test cruzado VM3D ↔ VM2D plane strain (mismos $\sigma_{xx}, \sigma_{yy}, \sigma_{xy}, \alpha$ a 10 decimales) y por el test de integración Hex8 confinado vs material standalone.
- Validación de parámetros en constructor: `E > 0`, σ`_y > 0`, `H` $\geq$ `0`, ν $\in$ `(-1, 0.5)`, `density` $\geq$ `0` si se pasa. Mensajes en español.

11.4.5 Diálogo

- **2026-05-21** · Spec pre-redactada por la IA al abrir la sub-etapa **A.bis** (materiales 3D no lineales) tras cierre de la Etapa 7 (ADR 0012). Base: `VonMises2D.md` (modelo físico) + `Elastic3D.md` (estructura Voigt 6D + sin variantes de hipótesis).
- **2026-05-21** · **Decisión de plumbing (decide la IA, Reglas §3)**: kernels Numba separados `_compute_j2_3d` (este material) vs `_compute_j2_plane_strain` (existente en VonMises2D). **No** se extrae un `_J2Core` compartido en esta entrega. Razones:

1. La diferencia operativa entre los dos kernels es el número de componentes activas (4 vs 6), que en Numba *nopython* requiere arrays de tamaño fijo — un kernel "genérico" obligaría a zero-padding sistemático con coste y oscurecería la lectura.
2. Cada kernel es corto (~40 líneas): la duplicación cabe en el mismo PR sin coste de mantenimiento desproporcionado.
3. La regla de centralización post-dos-casos del proyecto se satisface ya con la documentación cruzada (esta spec + VonMises2D referencian el mismo algoritmo de Simó-Hughes §3.3). Si aparece un tercer caso (e.g. `VonMises3DCorot` o variante con anisotropía leve) será el momento natural de abrir `_J2Core`.
4. Plane stress 2D queda explícitamente fuera de un eventual `_J2Core` por tener un algoritmo genuinamente distinto (Newton local proyectado).

Si al implementar la duplicación resulta más alta de lo previsto, se reconsidera y se eleva a zona gris.

- **2026-05-21** · **Decisión de plumbing**: el predictor elástico debe construirse como $\boldsymbol{\sigma}^{\text{trial}} = \mathbf{C}_e\,(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p_n)$ (o equivalente desviador + presión), **no** como $\mathbf{C}_e\,\boldsymbol{\varepsilon}$. Lección de VM2D plane strain (bug detectado el 2026-05-14): en descargas elásticas desde estado plástico o reevaluaciones vía `compute_gauss_state(U_final)`, ignorar $\boldsymbol{\varepsilon}^p_n$ devuelve σ inconsistente. Se aplica desde el inicio en VM3D y se cubre con `test_descarga_recupera_C_e` y con el test de integración `test_pipeline_hex8_traccion_uniaxial_elastoplastica`.
- **2026-05-21** · **Tests cruzados VM3D ↔ VM2D plane strain** (`cross_consistency`): garantía adicional de que VM3D no introduce regresión silenciosa sobre el comportamiento ya validado de VM2D. Es la red de seguridad principal para confirmar que la formulación 6D no contiene errores de signo/factor en la convención Voigt — los tests unitarios analíticos por sí solos no cubren toda la combinatoria de componentes activas.

- **2026-05-21** · **Validación contra Hill 3D**: el caso del cilindro hueco bajo presión interna con plasticidad perfecta tiene **solución analítica cerrada** (Hill 1950, §V) — la presión a la que la frontera elasto-plástica radial avanza a un radio dado es expresable como función explícita de σ_y, r_i, r_o. Es el benchmark canónico para validar J2 3D y ya está disponible en 2D plane strain (`tests/validation/test_hill_cylinder_j2.py`). El test 3D será su análogo discretizado por octante con simetrías. Se redactará y validará tras tener `Hex8 + VonMises3D` funcionando end-to-end.

- **2026-05-21** · **Implementación completa, status** `validated`. Kernel + clase + 12 tests unitarios + 1 test cruzado + 5 tests de integración (Hex8 uniaxial libre con Poisson, Hex8 confinado vs material standalone, Hex8 convergencia, Tet4 smoke test). **18 tests añadidos a la suite (804 → 824 verdes; suite global sin regresiones).** Todos los tests pasaron a la primera, incluyendo el test cruzado VM3D ↔ VM2D plane strain a 10 decimales — señal fuerte de que la convención Voigt 6D en el kernel y la tangente algorítmica son correctas. Validación Hill 3D (cilindro hueco bajo presión interna con plasticidad perfecta) **diferida a entrega separada** porque requiere setup de malla por octante con simetrías y carga incremental hasta plasticidad parcial; será el primer test de `tests/validation/` que ejercite VM3D contra solución analítica cerrada.

- **2026-09-22** · Ensamblaje por lotes (ADR 0014): declara `STATE_SCHEMA` y un adaptador `@njit` con firma `MAT_SIG` que llama al mismo return mapping; la tolerancia de admisibilidad (ADR 0006) se evalúa dentro del kernel con las mismas operaciones que `admissibility_tol`. Formulación sin cambios; equivalencia con `compute_state` exacta en esfuerzo, tangente y estado (`tests/test_batch_kernels.py`). El return mapping se reescribió sin asignaciones de memoria (`_j2_*_core`) con el mismo orden de operaciones: verificado bit a bit sobre 20 000 estados aleatorios en ramas elástica y plástica.

Capítulo 12

Modelos Constitutivos — Térmicos

12.1 ThermalConduction

Spec pre-redactada por la IA (2026-08-25) sobre las seis decisiones de alcance cerradas con el usuario para la Etapa 8. ***El usuario revisa la física; el plumbing está resuelto.*** *Los puntos abiertos de §Diálogo son físicos o de alcance, nunca arquitecturales.*

Primer material de la familia térmica. Etapa 8 — C1 núcleo mínimo: conducción pura, sin convección, radiación ni acoplamiento termomecánico.

12.1.1 Especificación física

0. Descripción general

Modelo constitutivo de **conducción de calor** según la ley de Fourier. Relaciona el vector de flujo de calor $\mathbf{q}$ con el gradiente de temperatura ∇T. Es el análogo térmico de `Elastic2D`/`Elastic3D`: lineal, sin variables internas, con "tangente" constante.

No pertenece a la familia `Material` mecánica. Los materiales mecánicos relacionan $\boldsymbol{\sigma}$ con $\boldsymbol{\varepsilon}$ en notación Voigt; éste relaciona dos vectores en el espacio físico, sin notación Voigt y sin tensor simétrico de por medio. Vive en una **familia paralela** `ThermalMaterial` con su propio registro y su sección YAML (`thermal_materials`), la pieza genérica de familias de material del registro (ADR 0020, P1) que resuelve este tipo de incompatibilidad semántica.

1. Ley constitutiva — Fourier

$$\mathbf{q} = -\,\mathbf{k}\,\nabla T$$

con:

- $\mathbf{q}$ — vector de flujo de calor [W/m^2]. Apunta en la dirección del transporte de energía.
- ∇T — gradiente de temperatura [K/m].
- $\mathbf{k}$ — tensor de conductividad térmica [W/(m·K)]. Matriz 2×2 en 2D, 3×3 en 3D.

El signo negativo es la física, no una convención elegible: el calor fluye de mayor a menor temperatura (segunda ley de la termodinámica). De ahí se sigue que $\mathbf{k}$ debe ser **definida positiva** — si no lo fuera, existiría una dirección en la que el calor fluiría espontáneamente hacia la región más caliente.

2. Conductividad tensorial e isótropa

El contrato guarda siempre $\mathbf{k}$ como tensor (decisión 4 del alcance). El caso isótropo es el particular

$$\mathbf{k} = k\,\mathbf{I}$$

y el constructor acepta un escalar k, expandiéndolo internamente. Esto evita romper el contrato cuando entren materiales ortótropos (madera, composites, roca estratificada, hormigón fibroreforzado), habituales en el dominio del usuario.

Requisitos sobre $\mathbf{k}$:

- **Simétrica**: $\mathbf{k} = \mathbf{k}^\top$. Lo exigen las relaciones recíprocas de Onsager para el transporte en medios sin campo magnético.
- **Definida positiva**: todos los autovalores > 0. Equivale a exigir $k > 0$ en el caso isótropo.

3. Ecuación de balance que este material alimenta

El material aporta $\mathbf{k}$ (y, para el transitorio, ρc) a la ecuación de conducción:

$$\rho c\,\frac{\partial T}{\partial t} = -\nabla\cdot\mathbf{q} + Q = \nabla\cdot(\mathbf{k}\nabla T) + Q$$

En régimen **estacionario** el término temporal se anula:

$$\nabla\cdot(\mathbf{k}\nabla T) + Q = 0$$

con Q la fuente volumétrica de calor [W/m^3].

4. Capacidad calorífica volumétrica

El producto ρc [J/(m^3·K)] es lo que gobierna el término transitorio:

- ρ — densidad [kg/m^3].
- c — calor específico [J/(kg·K)].

Se declaran por separado —no como producto— para que ρ tenga el mismo significado y las mismas unidades que en los materiales mecánicos, y para que el usuario pueda tomar c directamente de tabla.

Ambas son opcionales al construir, siguiendo el precedente de `density` en el ADR 0008: un análisis **estacionario** no necesita ninguna de las dos. Si un solver transitorio encuentra `None`, falla con `ValueError` identificando el material — nunca con capacidad cero silenciosa.

La **difusividad térmica** $\alpha = k/(\rho c)$ [m^2/s] es magnitud derivada, no parámetro: gobierna la velocidad de propagación del frente térmico y sirve para estimar el paso de tiempo característico. Se expone como propiedad calculada, no se almacena.

5. Variables internas

Ninguna. El modelo es lineal y sin memoria: $\mathbf{q}$ depende sólo del ∇T instantáneo. `PRIMARY_STATE_VAR = None`, igual que en `Elastic1D`/`Elastic2D`/`Elastic3D`.

La dependencia $\mathbf{k}(T)$ —conductividad función de la temperatura, que sí introduciría no linealidad— queda fuera de alcance (§out_of_scope).

6. Unidades y consistencia

El sistema **no convierte unidades** (`Reglas.md §5`); la consistencia es responsabilidad del usuario. En SI: $[k] = \mathrm{W/(m{\cdot}K)}$, $[c] = \mathrm{J/(kg{\cdot}K)}$, $[\rho] = \mathrm{kg/m^3}$, $[Q] = \mathrm{W/m^3}$, $[T] = \mathrm{K}$.

Sobre la escala de temperatura: la conducción pura sólo involucra **diferencias** de temperatura, así que Kelvin y Celsius dan resultados idénticos. La distinción sí importaría con radiación (donde T^4 exige escala absoluta), que está fuera de esta etapa. Se documenta para que la extensión futura no herede una ambigüedad.

12.1.2 Formulación numérica

7. Contribución a la matriz de conductividad

El elemento integra la forma débil de la ecuación de balance. El material aporta $\mathbf{k}$ al integrando:

$$\mathbf{K}_e = \int_\Omega \mathbf{B}^\top \mathbf{k}\, \mathbf{B}\, d\Omega, \qquad \mathbf{B} = \nabla \mathbf{N}$$

donde $\mathbf{B}$ es $2 \times n_{nodos}$ en 2D y $3 \times n_{nodos}$ en 3D — **el gradiente de las funciones de forma, sin reordenamiento de Voigt**. Es el mismo $\partial N/\partial x$ que ya calculan los kernels mecánicos; cambia sólo cómo se ensambla.

$\mathbf{K}_e$ es **simétrica** por serlo $\mathbf{k}$, luego el despachador algebraico (ADR 0003) puede elegir Cholesky. `IS_SYMMETRIC = True`.

8. Contribución a la matriz de capacidad

$$\mathbf{C}_e = \int_\Omega \rho c\, \mathbf{N}^\top \mathbf{N}\, d\Omega$$

Estructuralmente idéntica a la matriz de masa consistente, con ρc en lugar de ρ. Por la decisión 5 del alcance se ofrecen ambas formas, **con `lumped` por defecto** — al revés que en dinámica estructural. El motivo es físico y se detalla en la spec del elemento: la capacidad consistente produce oscilaciones espurias ante un frente térmico abrupto y puede violar el principio del máximo de la ecuación de difusión.

9. Interfaz de cómputo

$$\texttt{compute_flux}(\nabla T) \longrightarrow (\mathbf{q},\ \mathbf{k})$$

Devuelve el flujo y el tensor de conductividad —análogo a `(σ, C_tangent)` del contrato mecánico—. Sin `state_vars`: el modelo no tiene memoria, y añadir el parámetro por simetría

con `Material` sería ruido. Cuando entre un material térmico no lineal, la firma se ampliará entonces, con el caso real delante.

10. Caveats numéricos

- **Ninguna singularidad**: el modelo es lineal y **k** definida positiva por validación al construir. No hay régimen degenerado análogo al ápice de Drucker-Prager o a $d \to 1$ en daño.
- **Condicionamiento con contraste alto de conductividad**: en un dominio multimaterial con varios órdenes de magnitud entre las k (p. ej. metal contra aislante), **K** queda mal condicionada. Es propiedad del problema físico, no del modelo; se documenta como caveat operativo.
- **Anisotropía severa**: con $k_{\text{máx}}/k_{\text{mín}}$ muy grande el problema se vuelve fuertemente direccional y la malla debe alinearse razonablemente con los ejes de anisotropía, o la solución degrada. Igual que arriba: propiedad del problema.

12.1.3 Contrato de implementación

```
name: ThermalConduction
kind: thermal_material
status: validated        # draft → implemented → validated

interface:
  field: temperature     # campo primario que gobierna (1 DOF escalar por nodo)
  flux_dim: null         # 2 ó 3 - no es ClassVar: se fija por instancia al
                         # construir (escalar + dim, o tamaño del tensor k)
  primary_state_var: null
  is_symmetric: true

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 2 (conducción de calor con conductividad tensorial)"
    - zienkiewicz_taylor: "vol. 1: problemas de campo cuasi-armónicos (conducción anisó
      tropa)"
  general_codes:
    - abaqus: "*CONDUCTIVITY, TYPE=ANISO"

parameters:
  - { name: k,       type: float | array-like, required: true,
      desc: "Conductividad térmica [W/(m·K)]. Escalar ⇒ isótropo, se expande a k·I.
             Matriz 2×2 o 3×3 ⇒ ortótropo/anisótropo; debe ser simétrica y definida
                 positiva" }
  - { name: c,       type: float, required: false, default: null,
      desc: "Calor específico [J/(kg·K)]. Opcional al construir; obligatorio solo
             en análisis transitorio (mismo criterio que density en ADR 0008)" }
  - { name: density, type: float, required: false, default: null,
      desc: "Densidad [kg/m³]. Opcional al construir; obligatoria solo en transitorio"
        }

signature:
  compute_flux: "(grad_T: ndarray(d,)) -> (q: ndarray(d,), k: ndarray(d,d))  con d ∈
     {2, 3}"
  gradient_kind: "gradiente físico ∇T = ∂[T∂/x, ∂T∂/y(, ∂T∂/z)] - sin notación Voigt"
```

```
state_variables: []

conventions:
  sign: "q = -k·∇T; el flujo va de mayor a menor temperatura (segunda ley)"
  units: "k [W/(m·K)] · c [J/(kg·K)] · density [kg/m³] · Q [W/m³] · T [K o °C]"
  temperature_scale: "sólo intervienen diferencias de T ⇒ K y °C equivalentes en
                      conducción pura. La distinción importará al entrar radiación (⁴T)
                          "
  derived: "difusividad térmica α = kρ/(·c) [m²/s] expuesta como propiedad calculada,
            no almacenada"

validity:
  - "k simétrica y definida positiva (autovalores > 0); escalar k > 0"
  - "c > 0 y density > 0 cuando se declaran"
  - "conductividad independiente de la temperatura (modelo lineal)"
  - "medio continuo homogéneo dentro de cada elemento"

out_of_scope:
  - "conductividad dependiente de la temperatura k(T) ⇒ introduce no linealidad; etapa
     futura"
  - "convección (Robin) y radiación ⇒ condiciones de frontera, no del material; fuera
     de C1"
  - "cambio de fase / calor latente (Stefan) ⇒ etapa futura"
  - "acoplamiento termomecánico y deformación térmica α·ΔT ⇒ decisión 2 del alcance"
  - "medios porosos, convección forzada interna, transporte de masa"

acceptance:
  verification:
    - name: ley_de_fourier_exacta
      setup: "compute_flux con ∇T arbitrario; k isótropo y k ortótropo"
      expect: "q = -k·∇T exacto componente a componente; segundo retorno = k"
      tol_rel: 1.0e-14
    - name: expansion_escalar_a_tensor
      setup: "construir con k escalar en 2D y en 3D"
      expect: "k interno == k·I (2×2 y 3×3 respectivamente)"
      tol_abs: 1.0e-14
    - name: simetria_y_positividad
      setup: "construir con k escalar y con k tensorial válido"
      expect: "k == k.T exacto; todos los autovalores estrictamente positivos"
      tol_abs: 1.0e-14
    - name: flujo_opuesto_al_gradiente
      setup: "k isótropo; ∇T unitario en varias direcciones"
      expect: "q antiparalelo a ∇T; q·∇T < 0 estricto (disipación termodinámica)"
      tol_rel: 1.0e-14
    - name: anisotropia_desvia_el_flujo
      setup: "k = diag(k1, k2) con k1 ≠ k2; ∇T a 45° de los ejes"
      expect: "q NO es antiparalelo a ∇T; se desvía hacia el eje de mayor conductividad
          .
               Verificación cuantitativa contra q = -k·∇T calculado a mano"
      tol_rel: 1.0e-12
    - name: difusividad_derivada
      setup: "k, c, density conocidos"
      expect: "alpha == k/(density·c) para el caso isótropo"
      tol_rel: 1.0e-14
    - name: rechazo_inputs_invalidos
```

```
      setup: "k ≤ 0; k tensorial no simétrico; k tensorial no definido positivo;
              c ≤ 0; density ≤ 0; k de dimensión inconsistente con el elemento"
      expect: "ValueError con mensaje claro identificando el parámetro en cada caso"
    - name: capacidad_ausente_falla_en_transitorio
      setup: "construir sin c ni density; solicitar matriz de capacidad"
      expect: "ValueError identificando el material y el parámetro faltante,
               nunca capacidad cero silenciosa (precedente ADR 0008)"
    - name: mensaje_de_error_accionable
      setup: "construir sin c y/o sin density; solicitar matriz de capacidad"
      expect: "el mensaje nombra el material, nombra el/los parámetro(s) que faltan
               e indica la salida ('declara ambas' | 'usa un solver estacionario').
               No basta con reportar el atributo ausente"
    - name: estacionario_no_exige_capacidad
      setup: "construir sólo con k; ensamblar la matriz de conductividad"
      expect: "K_e se ensambla sin error; c y density no se consultan en ningún punto
               del camino estacionario"
      tol_rel: 1.0e-14

references:
  - "Incropera F.P., DeWitt D.P. (2007). Fundamentals of Heat and Mass Transfer. Wiley.
     §2 (ley de Fourier, conductividad tensorial)."
  - "Carslaw H.S., Jaeger J.C. (1959). Conduction of Heat in Solids. Oxford. §1 (
     formulación general, requisitos sobre k)."
  - "Lewis R.W., Nithiarasu P., Seetharamu K.N. (2004). Fundamentals of the Finite
     Element Method for Heat and Fluid Flow. Wiley. §3."
  - "Zienkiewicz O.C., Taylor R.L. (2000). The Finite Element Method, Vol. 1.
     Butterworth-Heinemann. §7 (problemas de campo escalar)."
```

12.1.4 Implementación

- Archivo: `solidum/materials/thermal_conduction.py`
- Clase: `ThermalConduction`, registrada en `ThermalMaterialRegistry`
- Clase base: `solidum/core/thermal_material.py` · `ThermalMaterial`
- Tests: `tests/test_thermal_material.py` — 31 verdes

Notas de implementación:

- `FLUX_DIM` **es propiedad de instancia, no** `ClassVar`**.** A diferencia de `STRAIN_DIM` y `JUMP_DIM`, la dimensión del flujo no la fija la clase sino el tensor con que se construye: el mismo `ThermalConduction` sirve en 2D y en 3D. Se deduce de `conductivity.shape[0]`, de modo que `k` matricial manda sobre el argumento `dim` (que sólo se usa para expandir un escalar). El contrato de la spec declara `flux_dim: null` por esta razón.
- **Validación de simetría con tolerancia escalada.** El criterio es `atol = 1e-12 · max|k|`, adimensional respecto a las unidades del usuario (ADR 0006). Un `allclose` con tolerancia absoluta fija habría rechazado tensores legítimos expresados en W/(mm·K) o aceptado asimetrías reales en W/(m·K).

- **Positividad vía `eigvalsh`, no vía el determinante.** El determinante positivo no implica definición positiva en dimensión ≥ 2 (dos autovalores negativos lo dejan positivo). Se comprueban todos los autovalores, que además es barato para matrices 2×2 y 3×3.

- `volumetric_capacity()` **vive en la clase base**, no en la subclase: el criterio del ADR 0008 y el formato del mensaje accionable son de familia, no de este material concreto. Cualquier material térmico futuro lo hereda.

- **Concordancia gramatical del mensaje de error.** El mensaje flexiona singular/plural según cuántos parámetros falten ("el parámetro 'density', que no se declaró… Decláralo" vs "los parámetros 'density' y 'c', que no se declararon… Decláralos"). Un mensaje mal concordado resta credibilidad justo cuando el usuario está diagnosticando.

- `thermal_diffusivity` **rechaza el caso anisótropo** en vez de devolver un valor plausible: con `k` tensorial la difusividad es un tensor, y un escalar sería una simplificación silenciosa. El mensaje remite a los autovalores de `conductivity` para estimar los extremos.

12.1.5 Diálogo

Puntos abiertos para el usuario. Sólo física y alcance.
2026-08-25 — IA → usuario. Tres puntos:

1. ~~**Nombre del material.**~~ ✓ **Resuelto 2026-08-25** — `ThermalConduction` (decisión del usuario). El nombre inicial `ThermalIsotropic` describía mal el contrato, que admite tensor completo; `ThermalConduction` nombra la física y no compromete la simetría del tensor. Un futuro material térmico no lineal —conductividad $\mathbf{k}(T)$, cambio de fase— entrará como nombre propio distinto, no como variante de éste.

1. ~~`c` **y** `density` **opcionales.**~~ ✓ **Resuelto 2026-08-25 — se traslada el criterio del ADR 0008** (decisión del usuario). Ambas son opcionales al construir: un análisis **estacionario** sólo usa $\mathbf{k}$, y exigir propiedades que no entran en ninguna ecuación obligaría a inventar valores ficticios. El fallo silencioso —el riesgo real de un parámetro opcional— está cerrado por diseño: cuando un consumidor transitorio las encuentra `None`, lanza `ValueError` identificando el material.

Requisito añadido en la decisión: el mensaje debe ser **accionable**, no sólo descriptivo — decir qué hacer, no sólo qué falta. Forma canónica:

> `ThermalConduction (id=1): el análisis transitorio requiere 'c' y 'density'. Declara ambas en el material, o usa un solver estacionario (LinearSolver).`

Es el espíritu de "validación temprana con mensajes claros" de `Reglas.md §1` aplicado al caso.

1. ~~**Fuente volumétrica** Q**.**~~ ✓ **Resuelto 2026-08-25 —** Q **es carga aplicada en el elemento, no propiedad del material** (decisión del usuario).

El material define la **ley constitutiva**, no las acciones sobre el medio; es el mismo reparto que en la familia mecánica, donde `Elastic2D` no sabe nada del peso propio ni de las fuerzas de cuerpo.

La razón que decide no es la simetría arquitectural sino la **variabilidad espacial y temporal**: la generación por hidratación del hormigón depende de la madurez, y el efecto Joule de la corriente.

Ninguna es constante del medio. Modelada como carga, Q puede variar por zona y por paso sin duplicar materiales; como propiedad del material, cada valor distinto exigiría un material propio.

Consecuencia para la spec del elemento: `compute_body_source(Q)` integra $\int_\Omega Q\,\mathbf{N}^\top d\Omega$, paralelo a `compute_body_load(b)` del mecánico. Se especifica allí, no aquí.

No cierra la puerta a un atajo futuro —un material que declare generación propia— si aparece el caso de uso que lo justifique.

- **2026-09-22** · Ensamblaje por lotes (ADR 0014): `STATE_SCHEMA = {}` y kernel lineal compartido (`materials/_batch.linear_material_kernel`, $k{\cdot}\nabla T$); `batch_matrix()` devuelve el tensor de conductividad.

Capítulo 13

Esquemas de Solución — Estáticos

13.1 ArcLengthSolver

*Spec **retroactiva**: el solver existe desde la fase de daño/softening y está validado por tests de snap-through, snap-back y daño post-pico. Esta spec lo documenta sin cambiar comportamiento — H-5.3 de la auditoría 2026-05-18.*

13.1.1 Especificación física

0. Descripción general

Análisis estático **no lineal** que traza curvas de equilibrio $\mathbf{U}(\lambda)$ atravesando puntos límite (snap-through, snap-back) donde $d\lambda/dU$ cambia de signo. A diferencia de `NonlinearSolver` (control de carga, λ creciente impuesto), el arc-length **trata λ como incógnita adicional** y restringe el avance combinado en (U, λ) a un arco fijo dl en el espacio aumentado. Esto permite seguir la respuesta postcrítica de estructuras con softening, pandeo geométrico, y daño con rama descendente.

Algoritmo: **Crisfield cilíndrico** (1981) — restricción cuadrática $\|\Delta\mathbf{U}\|^2 = dl^2$, λ libre.

1. Ecuación de equilibrio resuelta

Forma fuerte:

$$\mathbf{F}_{\text{int}}(\mathbf{U}) \;=\; \lambda\, \mathbf{F}_{\text{ext}}^{\text{ref}}$$

con λ incógnita, sujeta a la restricción cilíndrica de Crisfield en cada paso:

$$\|\Delta\mathbf{U}^{\text{paso}}\|^2 \;=\; dl^2$$

(versión **cilíndrica**: no incluye $\Delta\lambda^2$, equivalente a tomar $\psi = 0$ en el parámetro de escala carga-desplazamiento; la versión esférica $\|\Delta\mathbf{U}\|^2 + \psi^2\Delta\lambda^2\|\mathbf{F}_{\text{ext}}^{\text{ref}}\|^2 = dl^2$ no está implementada).

La norma $\|\Delta\mathbf{U}\|$ se evalúa sobre el vector completo de desplazamientos: DOF libres, esclavos de las MPC y prescritos (estos con $\Delta\lambda\,\mathbf{g}$). Los puntos trazados dependen de esa métrica. Con MPC no coinciden paso a paso con los de un modelo equivalente sin ellas, aunque todos son de equilibrio; con materiales dependientes de la trayectoria la diferencia queda dentro del error de discretización de la

trayectoria de carga (auditoría de las MPC, 2026-09-30). Un desplazamiento prescrito grande frente a la deformación acerca el paso a un control por carga.

2. Condiciones de contorno

Idénticas a `NonlinearSolver`: Dirichlet (homogéneo / no homogéneo) y MPC vía ADR 0004. Los desplazamientos prescritos no nulos (Dirichlet no homogéneo y término g de las MPC) se aplican proporcionalmente a λ y lo siguen también cuando la carga baja.

3. Salidas físicas

- **U** final en $\lambda \to \lambda_{\text{máx}}$ (o cuando se alcance `max_steps`).
- Historia disponible vía `step_callback(step, U, lambda)`.
- Trazas $\lambda(U_i)$ para visualizar snap-through/snap-back.

13.1.2 Formulación numérica

4. Esquema operativo

```
λ
 = 0
dl = (sin fijar)
mientras λ < max_lambda y step < max_steps:
    # Predictor tangente
    K_t, F_int = assemble_non_linear_system(U_iter)δ
    u_t = K_t^{-1} · F_ext^ref          # desplazamiento tangente unitario
    si dl sin fijar:                     # primer paso (§5)
        dl = Δıl = initial_dl  ó  initial_dlambda ·‖δ‖ u_t
    sign = signδ(U_prev ·δ u_t)          # evitar revertir direcciónΔλ
    _pred = sign · dl / ‖δ‖u_tΔ
    U_pred = Δλ_pred ·δ u_t

    # Corrector iterativo
    para iter en range(max_iter):
        K_t, F_int = assemble_non_linear_system(U_iter)
        R = λ( + Δλ)·F_ext^ref - F_intδ
        u_R = K_t^{-1} · R               # corrección por residuoδ
        u_t = K_t^{-1} · F_ext^ref       # tangente (refresh)

        # Restricción cuadrática de Crisfield
        # ‖ΔU_new + λdd·δ‖u_t² = dl²
        # ⇒ resolver ecuación cuadrática en λdd, elegir raíz por menor ángulολ
        dd = raiz_de_menor_angulo(...)Δ
        U += δu_R + λdd ·δ u_tΔλ
         += λdd
        U_iter = U_current + ΔU
        si converge (ADR 0007):
            si λ + Δλ > max_lambda:              # cruzó el objetivo (§10)
                no consolidar; llegada exacta:
                Newton puro con λ = max_lambda desde (U_current, λ),
                arrancando de la interpolación lineal del tramo recorrido
```

```
            commit_all_states()λ
             += Δλ
            ajustar_dl_adaptativamente(iter)
            break
    si no converge (o falla la llegada):
        dl /= 2
```

5. Predictor / corrector

Predictor tangente: $\delta\mathbf{u}_t = \mathbf{K}_t^{-1}\,\mathbf{F}_{\text{ext}}^{\text{ref}}$ (vector que mide la pendiente local del camino de equilibrio). $\Delta\lambda_{\text{pred}} = \pm dl/\|\delta\mathbf{u}_t\|$ con signo determinado por el producto escalar contra el incremento previo $\Delta\mathbf{U}^{\text{paso prev}}$ — así no se retrocede en el camino tras pasar un punto límite.

Primer paso (2026-09-23). *dl* es una longitud en unidades de desplazamiento, y quien plantea el modelo no conoce de antemano sus desplazamientos. El primer paso se declara, por omisión, como **fracción de la carga de referencia** $\Delta\lambda_1$ (`initial_dlambda`, 0.1 si no se declara) y la longitud de arco se deriva del predictor elástico del primer paso (Crisfield 1991, cap. 9):

$$\Delta l_1 = \Delta\lambda_1\,\|\mathbf{K}_0^{-1}\,\mathbf{F}_{\text{ext}}^{\text{ref}}\|.$$

En régimen elástico el primer paso lleva exactamente $\Delta\lambda_1$, en cualquier sistema de unidades. `initial_dl` fija Δl_1 directamente (longitud explícita) y es excluyente con `initial_dlambda`. Δl_1 queda en `dl_reference` y es la referencia de `dl_max_factor` y del umbral de aborto. Una carga de referencia que no produce desplazamiento ($\|\delta\mathbf{u}_t\| = 0$) no da escala al primer paso y es un error explícito. La norma mezcla todos los grados de libertad; en marcos, giros y desplazamientos, cuyo peso relativo depende de las unidades (deuda #22): el primer paso es invariante, los siguientes heredan esa mezcla.

Corrector (Newton modificado para arc-length, Crisfield): cada iteración resuelve dos sistemas (residuo y tangente), combina con $dd\lambda$ que satisface la restricción cuadrática:

$$\|\Delta\mathbf{U} + dd\lambda\,\delta\mathbf{u}_t\|^2 = dl^2$$

Ecuación cuadrática en $dd\lambda$ con dos raíces. **Selección de raíz**: la que produce **menor ángulo** con el incremento acumulado $\Delta\mathbf{U}$ (evita pivotear el sentido del arco).

6. Criterio de convergencia (ADR 0007)

Idéntico a `NonlinearSolver` — patrón dual fuerza + desplazamiento con escala autoderivada. Compartido vía `ConvergenceCriterion`. La calibración del solver usa $\|\mathbf{F}_{\text{ext}}^{\text{ref}}\|$ como referencia de fuerza (no $\lambda\,\mathbf{F}_{\text{ext}}^{\text{ref}}$, porque λ varía).

7. Imposición de Dirichlet y MPC

Idéntico al `NonlinearSolver` — `Assembler.reduce(...)` con `U_current` y `load_factor=lambda_iter`.

8. Backend algebraico

Régimen **postcrítico**: $\mathbf{K}_t$ puede ser indefinida (autovalores negativos tras bifurcación). El solver fuerza `is_positive_definite=False` al construir el `linalg`, así el despachador ADR 0003 elige LU automáticamente. Si el usuario fuerza `linear_algebra="cholesky"` desde YAML y $\mathbf{K}_t$ se vuelve indefinida en algún paso, hay fallback automático a LU con warning.

Tangente inicial (ADR 0019, 2026-09-30; decisión física del usuario): la primera factorización del análisis —la de la tangente en $\mathbf{U} = \mathbf{0}$, que el predictor del primer paso factoriza de todos modos para calcular $\mathbf{K}_0^{-1}\mathbf{F}_{\text{ref}}$— se inspecciona antes de usarla (`model_checks.check_initial_tangent`; no se factoriza dos veces). Si tiene un mecanismo interno (el criterio del estático lineal: un pivote pequeño frente a la diagonal dispara el cálculo del modo de menor energía, y es un mecanismo si ese modo no deforma ninguno de los elementos que mueve; ADR 0019, ampliación del 2026-10-20) y ningún componente del modelo puede ganar rigidez al deformarse —ningún elemento con `GEOMETRICALLY_NONLINEAR = True` ni material con `IS_UNILATERAL = True`—, el análisis se rechaza con `IllPosedSystemError`: en pequeños desplazamientos la matriz $\mathbf{B}$ es constante y la no linealidad del material sólo reduce la rigidez, así que un mecanismo en $\mathbf{U} = \mathbf{0}$ lo es durante todo el análisis, lo active la carga o no. Si hay elementos corrotacionales o materiales unilaterales (una red de cables sin pretensado se rigidiza al tensarse), sólo se avisa en el log. Con el backend iterativo, que no tiene pivotes, se avisa de que la comprobación no se hace. Las tangentes posteriores no se inspeccionan: cerca de un punto límite una tangente casi singular es legítima. La heredan las variantes que sólo cambian la restricción (`IndirectDisplacementSolver`); una variante que redefina `solve` debe llamar a `check_initial_tangent` justo después de `ensure_statically_restrained`, como hace `DissipationArcLengthSolver`.

9. Adaptatividad — ajuste de dl

Política de auto-ajuste basada en iteraciones del paso:

- Converge en $<$ `dl_grow_iter_threshold` iteraciones $\Rightarrow$ $dl \to dl \cdot$ `dl_grow_factor` (cota: $\Delta l_1 \cdot$ `dl_max_factor`).
- Converge en $>$ `dl_shrink_iter_threshold` iteraciones $\Rightarrow$ $dl \to dl \cdot$ `dl_shrink_factor`.
- No converge en `max_iter` $\Rightarrow$ $dl/ = 2$ y reintentar el paso.
- Cota inferior: $dl <$ `ARCLENGTH_MIN_DL_FACTOR` $\cdot \Delta l_1 \Rightarrow$ aborto.

10. Caveats numéricos

- **Cancelación o bifurcación verdadera**: en bifurcaciones simétricas múltiples, la selección de raíz por menor ángulo puede saltar a un camino paralelo. Para análisis de bifurcación auténtica usar perturbación + post-procesado modal.
- **Raíces imaginarias**: si el discriminante de la cuadrática es negativo (paso demasiado grande, mal condicionamiento severo), el solver aborta el paso y bisecta dl.
- **Llegada a $\lambda_{\text{máx}}$ (2026-09-23)**: todos los pasos llevan la restricción cilíndrica. Si uno converge con $\lambda > \lambda_{\text{máx}}$, su estado no se consolida y el paso se repite desde el último estado convergido con $\lambda = \lambda_{\text{máx}}$ fijo (Newton puro), arrancando de la interpolación lineal del tramo recién

recorrido; como ese tramo cruza $\lambda_{\text{máx}}$, el equilibrio existe y está cerca. Si la llegada no converge, se biseca dl. Sustituye al "último paso" anterior, que se decidía extrapolando la tangente ($\lambda + \Delta\lambda_{\text{pred}} \geq \lambda_{\text{máx}}$) e imponía $\lambda_{\text{máx}}$ en control de carga sin recorrer la curva: podía saltar un punto límite, y un paso de arco que convergía por encima de $\lambda_{\text{máx}}$ terminaba el trazado ahí (medido: arco de von Mises, $\lambda_{\text{final}} = 0.398$ con $\lambda_{\text{máx}} = 0.39$).

- **Softening con penalización cohesiva rígida** (elemento con discontinuidad embebida del módulo de usuario `discontinuities`, ADR 0020): se registró que no se atravesaba la transición elástico→softening por la casi-singularidad de $\mathbf{K}_t$ cerca de κ_0. El diagnóstico es anterior a la corrección de la tangente cohesiva del 2026-09-23 (`597f3d9`), que presentaba al solver una tangente de signo contrario en casi toda la rama, y no se ha vuelto a comprobar con este solver; con control por desplazamiento, `NonlinearSolver` sí recorre hoy esa rama (`tests/user/discontinuities/test_disc_embedded_uniaxial_softening.py`).

- **Diagnóstico de bifurcación por inercia** (Sturm sequence): no implementado. El solver no sigue la inercia de $\mathbf{K}_t$; el predictor se orienta sólo por el producto escalar con el incremento previo (§5). El método `_negative_pivots()`, un marcador que devolvía `None` y que ningún `solve` llamaba, se retiró el 2026-09-29. No habrá diagnóstico por inercia hasta que exista un backend LDL$^\top$ verdadero (Bunch-Kaufman) con conteo de pivotes negativos (deuda técnica #7 de STATUS).

13.1.3 Contrato de implementación

```yaml
name: ArcLengthSolver
kind: solver
status: validated

interface:
  yaml_type: arclength
  output: SolveResult
  pipeline_kind: static

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_borst_crisfield: "§4.2 (longitud de arco; 1.ª ed.: Crisfield 1991, cap. 9)"
    - bathe: "§8.4 (métodos con restricción carga-desplazamiento)"
  general_codes:
    - ansys: "ARCLEN"
    - opensees: "integrator ArcLength α( = 0: cilíndrico)"
    - abaqus: "*STATIC, RIKS"

parameters:
  - { name: convergence,             type: ConvergenceCriterion, required: false,
      default: "ConvergenceCriterion()",
      desc: "Política dual fuerza+desplazamiento (ADR 0007), compartida con
          NonlinearSolver" }
  - { name: max_iter,                type: int,   required: false, default: 20,
      desc: "Iteraciones Newton (corrector) máximas por paso" }
  - { name: max_lambda,              type: float, required: false, default: 1.0,
      desc: "Factor de carga objetivo" }
  - { name: initial_dlambda,         type: float, required: false, default: 0.1,
```

```
      desc: "Fracción de la carga de referencia del primer paso (adimensional, keyword-
          only); Δ₁l = Δλ₁·‖₀⁻K¹‖F_ref. Excluyente con initial_dl" }
  - { name: initial_dl,               type: float, required: false, default: null,
      desc: "Longitud de arco del primer paso en unidades de desplazamiento; sólo si se
           conoce la escala del problema. Excluyente con initial_dlambda" }
  - { name: max_steps,                type: int,   required: false, default: 100,
      desc: "Cota dura sobre el número de pasos" }
  - { name: dl_grow_factor,           type: float, required: false, default: 1.5 }
  - { name: dl_max_factor,            type: float, required: false, default: 5.0,
      desc: "dl máximo = Δ₁l · dl_max_factor" }
  - { name: dl_shrink_factor,         type: float, required: false, default: 0.6 }
  - { name: dl_grow_iter_threshold,   type: int,   required: false, default: 4 }
  - { name: dl_shrink_iter_threshold, type: int,   required: false, default: 8 }
  - { name: linear_algebra,           type: str,   required: false, default: "auto",
      desc: "'auto' (Pardiso si está instalado, si no LU: el solver declara la tangente
           potencialmente indefinida y no usa Cholesky), 'cholesky', 'pardiso', 'lu', '
          iterative[:amg|jacobi|none]' (CG que pasa a MINRES al detectar curvatura
          negativa)" }

requirements:
  - "Modelo con potencial snap-through/snap-back (geometría inestable, softening del
      material)"
  - "Apoyos suficientes; sin modos rígidos"
  - "Carga de referencia que produzca desplazamiento (escala del primer paso)"

conventions:
  units: "heredadas del modelo (ADR 0008)"
  stability: "incondicional respecto a puntos límite; bifurcaciones complejas pueden
      saltar de rama"

out_of_scope:
  - "Variante esférica del arc-length ψ( ≠ 0) - sólo cilíndrica implementada"
  - "Análisis dinámico ⇒ usar NewmarkSolver y derivados"
  - "Continuación de bifurcaciones múltiples - requiere perturbación explícita por el
      usuario"
  - "Buckling lineal (eigenproblema) ⇒ futuro BucklingSolver"
  - "Sign-of-pivot tracking (Sturm sequence) - no implementado hasta que exista un ᵀLDL
       verdadero (deuda #7)"

acceptance:
  verification:
    - name: barra_traccion_elastica_reproduce_caso_lineal
      setup: "Truss2D elástica, carga axial; arc-length avanza hasta λ=1"
      expect: "Uλ(=1) = U_lineal exacto; equilibrio en cada paso"
      tol_rel: 1.0e-10

    - name: snap_through_lee_frame
      setup: "frame Lee con dos barras corotacionales en V, carga vertical en el vé
          rtice"
      expect: "se atraviesa el punto límite; respuesta Uλ- coincide con Crisfield 1991
          Vol.1 Fig. 9.5"
      tol_rel: 1.0e-3

    - name: snap_back_truss_inestable
      setup: "armadura con elemento corotacional cuya rama descendente requiere d λ/dU
          < 0"
```

```
      expect: "se atraviesa el snap-back; signo de Δλ cambia coherentemente"
      tol_rel: 1.0e-3

    - name: damage_softening_post_pico
      setup: "Quad4 con IsotropicDamage2D, tracción monotónica que entra a rama
          descendente"
      expect: "rama post-pico trazada; convergencia paso a paso"
      tol_rel: 1.0e-3

    - name: ajuste_dl_adaptativo
      setup: "trayectoria con tramos elásticos rápidos y zona post-pico lenta"
      expect: "dl crece en tramos fáciles, se reduce cerca del pico; converge
          globalmente"

    - name: ultimo_paso_cierra_en_target
      setup: "max_lambda = 0.7 sobre estructura elástica"
      expect: "λ_final = 0.7 exacto (no por overshoot)"
      tol_abs: 1.0e-12

    - name: primer_paso_fraccion_de_carga
      setup: "placa elástica; sin declarar el primer paso, y con initial_dlambda = 0.25
          "
      expect: "λ del primer paso = 0.1 y 0.25 respectivamente"
      tol_abs: 1.0e-12

    - name: trazado_invariante_ante_unidades
      setup: "misma placa J2 en m y en mm (longitudes ×1000, módulos ⁶/10), primer paso
           por omisión"
      expect: "misma secuencia de λ, U_mm = 1000·U_m, dl_reference ×1000; con una
          longitud explícita igual en ambos, primeros λ distintos"
      tol_rel: 1.0e-8

    - name: llegada_exacta_dentro_del_tramo
      setup: "arco de von Mises h/L = 0.1, max_lambda = 0.39 (sobre el pico, 0.385),
          initial_dl = 0.2 y 1.0"
      expect: "λ_final = 0.39 exacto y equilibrio en ese estado (antes: 0.398 y 0.440)"
      tol_abs: 1.0e-12

  specific:
    - name: seleccion_raiz_no_revierte
      setup: "paso tras un snap-back con dos raíces de signos opuestos"
      expect: "se elige la raíz que continúa el camino (menor ángulo con incremento
          previo)"

    - name: fallback_cholesky_lu_indefinitud
      setup: "linear_algebra='cholesky' forzado, modelo con punto límite"
      expect: "warning + LU al detectar no-PD; análisis continúa"

    - name: mecanismo_interno_en_la_tangente_inicial
      setup: "armadura Pratt del ejemplo 11 sin la diagonal del segundo panel, carga
          horizontal (ADR 0019, 2026-09-30)"
      expect: "Truss2D ⇒ IllPosedSystemError en el predictor del primer paso, sin
          bisecar; Truss2DCorot ⇒ un aviso; con la diagonal, nada"

references:
```

```
- "Crisfield M.A. (1981). A fast incremental/iterative solution procedure that
    handles snap-through. Computers and Structures 13, 55-62."
- "Crisfield M.A. (1991). Non-linear Finite Element Analysis of Solids and Structures
    , Vol. 1. Wiley. §9 (arc-length cilíndrico)."
- "Riks E. (1979). An incremental approach to the solution of snapping and buckling
    problems. Int. J. Solids and Structures 15, 529-551."
- "de Borst R., Sluys L.J. (1999). Computational Methods in Non-linear Solid
    Mechanics. TU Delft. §3.5."
- "ADR 0003, ADR 0007."
```

13.1.4 Implementación

- **Archivo**: solidum/math/solvers/arclength.py.
- **Clase**: `ArcLengthSolver`, registrada vía `@SolverRegistry.register` con `PIPELINE_KIND = "static"`.
- **Restricción cuadrática y selección de raíz**: implementada en `solve`. Comparación de ángulos `theta1 = ΔU_iter · (ΔU_new + ddl1·δu_t)`, idem `theta2`, seleccionar el mayor → menor ángulo con la dirección previa.
- `_make_linalg`: fuerza `is_positive_definite=False` independientemente de la simetría del dominio (régimen postcrítico).
- **Problema de Newton por paso**: `ArcProblem`, exportado con nombre público en `solidum.math.solvers` (en el código de `arclength.py` sigue llamándose `_ArcProblem`). Lleva el iterado `(U, λ, ΔU)`, las dos resoluciones por iteración y la restricción que fija $dd\lambda$ como método `constraint`. Una variante de arco que sólo cambia la restricción del paso lo hereda y redefine ese método: así lo hacen `IndirectDisplacementSolver` en el programa principal y `DissipationArcLengthSolver` en el módulo de usuario `dissipation_control` (ADR 0020).
- **Entrypoint público**: `solidum.run(domain, solver=ArcLengthSolver(assembler, ...), F_applied=...)` o `solidum.run_yaml(path)`.
- **Tests**:
- tests/test_solver_robustness.py (`TestArcLengthRobustness`: punto límite, snap-back, recuperación lineal), tests/test_snap_through_corot.py (armadura de von Mises contra la solución cerrada) y tests/test_integration.py (`test_arclength_solver_elastoplastic`).
- tests/test_solver_robustness.py · `test_arc_length_traverses_damage_softening`.
- tests/test_arclength_paso_inicial.py — primer paso como fracción de carga, invariancia ante las unidades, declaración inválida, carga de referencia nula, clave YAML y llegada exacta a `max_lambda`.
- tests/test_safety_net_initial_tangent.py — tangente inicial y aviso del backend iterativo (ADR 0019, 2026-09-30).
- **Red de seguridad (ADR 0019)**: `ensure_statically_restrained` y `check_initial_tangent` al empezar `solve`. `_tangent_predictor` deja pasar `IllPosedSystemError` (el veredicto sobre la tangente inicial) en vez de tratarlo como tangente singular que bisecar.

13.1.5 Diálogo

- **2026-05-19** · Spec creada retroactivamente para cerrar el hueco H-5.3. Solver anterior a la convención de specs. La spec recoge la implementación tal como está al cierre de la sesión de saneamiento post-auditoría.

- **2026-09-22** · Auditoría global: corrector con un ensamblaje por iteración y convergencia evaluada antes de resolver: el estado committed corresponde exactamente a `U_current` (antes se evaluaba R en U_k, se comiteaba el trial de U_k y se guardaba U_k+1: un iterado de desfase, del orden de la tolerancia). La parada por `max_steps` sin alcanzar `max_lambda` emite WARNING y se expone en `reached_max_lambda`, `lambda_final` y `steps_done`; `solidum.run` escala `F_applied` por λ_final (reacciones coherentes) y reporta `converged`/`num_steps` reales (antes siempre `True`/1).

- **2026-09-22** · ADR 0015: el bucle de Newton pasa al corrector compartido `NewtonCorrector`; este solver aporta su problema por paso (`_ArcProblem`: iterado `(U, λ, ΔU)`, dos resoluciones por iteración y restricción cilíndrica como método `constraint`; raíces imaginarias → `CorrectionAborted`; predictor tangente en `_tangent_predictor`; retirados `_solve` y `_make_linalg`) y conserva su control de paso. Sin cambio de formulación ni de resultados (suite completa sin tocar ningún valor esperado).

- **2026-09-23** · ADR 0017-0019: `linear_algebra` admite `pardiso` e `iterative` (CG → MINRES automático ante curvatura negativa, adecuado a la tangente indefinida del régimen postcrítico); al empezar `solve` se rechaza un mecanismo rígido (`MechanismError`). Dentro del trazado no se rechaza ningún sistema casi singular: cerca de un punto límite es parte del algoritmo.

- **2026-09-23** · Revisión documental: `tests/test_arclength.py` ya no existe; la cobertura está repartida en `test_solver_robustness.py`, `test_snap_through_corot.py` y `test_integration.py`.

- **2026-09-23** · **Cambio de formulación validado por el usuario** (deuda #26, destapada en el ejemplo 4 del manual de ejemplos). (1) Primer paso adimensional: `initial_dlambda` (0.1 por omisión) y $\Delta l_1 = \Delta\lambda_1 \|\mathbf{K}_0^{-1}\mathbf{F}_{\text{ref}}\|$; `initial_dl` pasa a ser una longitud explícita opcional, excluyente, sin valor por omisión. Antes el valor por omisión era `initial_dl = 0.1`, una longitud: en el cilindro J2 del ejemplo 4 pedía $\Delta\lambda = 86$ en el primer paso, el predictor rebasaba `max_lambda` y el solver imponía $\lambda = 1.5$ en control de carga, por encima del colapso, sobre un equilibrio espurio del modelo discreto (0,98 m de desplazamiento). (2) Llegada exacta a `max_lambda` dentro del tramo recorrido, en lugar del paso final decidido por el predictor (§10). Los 84 tests existentes de los dos solvers de arco pasan sin cambiar ningún valor esperado; los nuevos fallan con el código anterior (9 de 11).

- **2026-09-29** · Frontera del programa principal (ADR 0020): el caveat del softening con penalización cohesiva rígida (§10) pasa a nombrar el módulo de usuario `discontinuities` y a declarar que su diagnóstico es anterior a la corrección de la tangente cohesiva (`597f3d9`) y no se ha vuelto a comprobar con este solver. Sin cambio de formulación ni de resultados.

- **2026-09-30** · Red de seguridad del ADR 0019 sobre la tangente inicial (decisión física del usuario; ver la entrada del mismo día en la spec de `NonlinearSolver`): el predictor del primer

paso inspecciona sus pivotes nulos; error si el modelo es geométricamente lineal, aviso si tiene elementos geométricamente no lineales o materiales unilaterales; con el backend iterativo, aviso. Sin cambio de resultados en modelos bien planteados.

- **2026-09-29** · Retirado `_negative_pivots()`, el marcador del diagnóstico por inercia que devolvía `None` y que nadie llamaba; §10 y `out_of_scope` lo recogen: no hay diagnóstico por inercia hasta que exista $\mathrm{LDL}^\top$ (deuda #7). El problema por paso (`_ArcProblem` en el código) se exporta con nombre público, `solidum.math.solvers.ArcProblem`, porque lo extienden `IndirectDisplacementSolver` y `DissipationArcLengthSolver`, este último trasladado hoy al módulo de usuario `dissipation_control`. Sin cambio de formulación ni de resultados.

- **2026-09-30** · Misma corrección que en `NonlinearSolver` (deuda #31, entrada del mismo día en su spec): el problema por paso mide $\|\mathtt{T}^\top\mathtt{R}\|$ en vez de $\|\mathtt{R[libres]}\|$, que con restricciones multipunto no se anulaba en la solución y hacía fallar todos los pasos. La heredan `IndirectDisplacementSolver` y, por `ArcProblem`, el solver del módulo `dissipation_control`. Sin cambio con sólo Dirichlet. La auditoría de las MPC del mismo día documentó en §1 y §2 que la métrica del arco incluye los DOF esclavos y prescritos y que el término `g` de las MPC sigue a λ; no se cambia la métrica, porque alteraría la secuencia de pasos de los modelos existentes sin corregir ningún error.

13.2 IndirectDisplacementSolver

Spec corta tipo extensión *(Reglas.md §4). Variante de* `ArcLengthSolver` *que* ***sólo cambia la restricción del paso****: en lugar de fijar la norma del incremento de todos los grados de libertad ($\|\Delta\mathbf{U}\| = \Delta l$), fija el incremento de una* ***combinación elegida*** *de ellos ($\mathbf{c}^\top\Delta\mathbf{U} = \Delta l$). Reusa el predictor tangente, el corrector de Newton compartido (ADR 0015), el primer paso adimensional (deuda #26), la llegada exacta a* `max_lambda`*, Dirichlet/MPC y el backend algebraico. Sin ADR nuevo.*

Motivación (medida el 2026-09-23 al preparar el ejemplo 5 del manual de ejemplos): en una barra con daño que se localiza en un elemento, el retroceso (snap-back) empieza en el propio pico. Ahí el incremento de la parte que se descarga apunta casi al revés que el del paso anterior, y ni el `ArcLengthSolver` *cilíndrico (su selección de raíz por menor ángulo elige la rama equivocada o no converge) ni el* `DissipationArcLengthSolver` *(ver deuda #27; desde el 2026-09-29 está en el módulo de usuario* `dissipation_control`*, ADR 0020, y no en el programa principal) siguen la curva. Controlar el alargamiento del elemento que se daña, como un ensayo de fractura controlado por la apertura de la boca de fisura (CMOD), la sigue exacta. Es también el control que el ADR 0010 dejó previsto para la fase 4 (Van Vliet).*

13.2.1 Especificación física

0. Descripción general

Solver estático no lineal que traza la curva de equilibrio $\mathbf{F}_{\text{int}}(\mathbf{U}) = \lambda\,\mathbf{F}_{\text{ref}}$ avanzando en cada paso un incremento fijo Δl de una **magnitud cinemática elegida por el usuario**: una combinación lineal de grados de libertad, típicamente el alargamiento de la zona que se ablanda o la apertura de una grieta. La carga λ es incógnita y puede bajar, así que la curva se sigue a través de puntos límite y de retrocesos, **siempre que la magnitud controlada crezca monótonamente** a lo largo del camino físico.

1. Restricción del paso

$$g(\Delta\mathbf{U}) = \mathbf{c}^\top\Delta\mathbf{U} - \Delta l = 0, \qquad \mathbf{c}^\top\mathbf{U} = \sum_k c_k\, u_{(n_k,\,d_k)}$$

con $\mathbf{c}$ el vector de coeficientes sobre los grados de libertad controlados. Para el alargamiento entre los nodos a y b en x: $c_{b,x} = +1$, $c_{a,x} = -1$.

Grados de libertad admisibles. $\mathbf{c}$ actúa sobre el incremento completo, que con las restricciones del modelo (ADR 0004) es $\Delta\mathbf{U} = \mathbf{T}\,\Delta\mathbf{u}_{\text{libre}} + \Delta\lambda\,\mathbf{g}$ e incluye los esclavos reconstruidos por $\mathbf{T}$. Por tanto

$$\mathbf{c}^\top\Delta\mathbf{U} = (\mathbf{T}^\top\mathbf{c})^\top\Delta\mathbf{u}_{\text{libre}} + \Delta\lambda\,\mathbf{c}^\top\mathbf{g}.$$

- Un grado de libertad **libre**, o el **esclavo de una restricción multipunto con maestros** (fila de $\mathbf{T}$ no vacía: $u_s = g_s + \sum_i \alpha_i\, u_{m_i}$), se puede controlar: su incremento lo fijan los grados de libertad libres.

- Un grado de libertad **prescrito** (fila de $\mathbf{T}$ vacía: apoyo declarado, o esclavo cuyos maestros están todos prescritos, que hereda el apoyo por el cierre transitivo) se rechaza: su incremento $\Delta\lambda\, g_s$ no depende del equilibrio (con $g_s = 0$ la restricción no determina $\Delta\lambda$; con $g_s \neq 0$ degeneraría en control de carga).
- Una combinación con $\mathbf{T}^\top\mathbf{c} = \mathbf{0}$ (p. ej. $u_s - u_m$ con $u_s = u_m + g$) se rechaza por la misma razón.

2. Predictor

Con $\mathbf{du}_t = \mathbf{K}_t^{-1}\mathbf{F}_{\text{ref}}$:

$$\Delta\lambda = \frac{\Delta l}{\mathbf{c}^\top \mathbf{du}_t}, \qquad \Delta\mathbf{U} = \Delta\lambda\, \mathbf{du}_t.$$

Sin regla de signo: el signo de $\Delta\lambda$ sale de la propia restricción (negativo en un retroceso, donde la tangente hace bajar la carga mientras la magnitud controlada sigue creciendo). El arco cilíndrico, en cambio, necesita decidir el sentido con el producto escalar con el incremento anterior.

3. Corrector

En cada iteración, con $\mathbf{du}_R = \mathbf{K}^{-1}\mathbf{R}$ y $\mathbf{du}_t = \mathbf{K}^{-1}\mathbf{F}_{\text{ref}}$:

$$\delta\lambda = \frac{\Delta l - \mathbf{c}^\top(\Delta\mathbf{U} + \mathbf{du}_R)}{\mathbf{c}^\top \mathbf{du}_t}, \qquad \delta\mathbf{U} = \mathbf{du}_R + \delta\lambda\, \mathbf{du}_t.$$

Es **lineal** en $\delta\lambda$: no hay raíces que elegir ni discriminantes negativos. Tras converger, $\mathbf{c}^\top\Delta\mathbf{U} = \Delta l$ se cumple a precisión de máquina.

4. Primer paso y tamaño de paso

- **Primer paso adimensional** (deuda #26): `initial_dlambda` (0.1 por omisión) es la fracción de la carga de referencia; $\Delta l_1 = \Delta\lambda_1\, \mathbf{c}^\top\mathbf{K}_0^{-1}\mathbf{F}_{\text{ref}}$. `initial_dl` fija Δl_1 explícito, en las unidades de la magnitud controlada.
- **Sin crecimiento por omisión** (`dl_max_factor = 1.0`, frente a 5.0 del padre): la magnitud avanza a ritmo constante, como en un ensayo a velocidad de CMOD constante. La razón es física, ver §5.
- Se conserva la reducción por iteraciones y la bisección del padre.

5. No unicidad del problema incremental con ablandamiento

Con ablandamiento local, el problema de un paso (equilibrio con la historia κ del paso anterior) admite varios equilibrios: además del físico, en que sólo se ablanda la zona debilitada, otros en que también se ablandan zonas que en el camino continuo se habrían descargado. El Newton converge a uno u otro según la estimación inicial. **Un paso que sobrepasa el pico más que la imperfección que localiza el daño puede converger a otro equilibrio.** Medido en la barra de 10 elementos con uno debilitado un 5 %: con pasos del 2 % o el 5 % de la carga el trazado es exacto; con el 10 %, o con el crecimiento ×5 del padre, se dañan los diez elementos. El control por desplazamiento del

extremo (`NonlinearSolver`) hace lo mismo con pasos grandes: no es una propiedad del solver sino del problema discreto. De ahí el valor por omisión sin crecimiento y la recomendación de pasos menores que el margen de la imperfección cerca del pico.

13.2.2 Contrato de implementación

```
name: IndirectDisplacementSolver
kind: solver
status: validated

parent_spec: ArcLengthSolver

interface:
  yaml_type: IndirectDisplacementSolver
  output: SolveResult
  pipeline_kind: static

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_borst_crisfield: "§4.2 (seguimiento de trayectoria; 1.ª ed.: Crisfield 1991,
        cap. 9)"
  general_codes:
    - diana: "indirect displacement control (p. ej. CMOD)"

parameters:
  # Específico de esta variante:
  - { name: control,                type: list,  required: true,
      desc: "Combinación controlada Σ coef·u(nodo, dof): lista de (nodo, dof, coef) o
          de {node, dof, coef} (coef = 1 si se omite); nodo = Node o id. Grados de
          libertad no prescritos (libres o esclavos de una restricción multipunto con
          maestros libres); la combinación debe crecer con la carga de referencia" }
  # Heredados del padre, con la semántica de la magnitud controlada:
  - { name: convergence,            type: ConvergenceCriterion, required: false,
      default: "ConvergenceCriterion()" }
  - { name: max_iter,               type: int,   required: false, default: 20 }
  - { name: max_lambda,             type: float, required: false, default: 1.0 }
  - { name: max_steps,              type: int,   required: false, default: 100 }
  - { name: initial_dlambda,        type: float, required: false, default: 0.1,
      desc: "Fracción de la carga de referencia del primer paso; Δ1l = Δλ1·ᵀ0⁻cK¹F_ref.
          Excluyente con initial_dl" }
  - { name: initial_dl,             type: float, required: false, default: null,
      desc: "Incremento explícito de la magnitud controlada en el primer paso, en sus
          unidades. Excluyente con initial_dlambda" }
  - { name: dl_grow_factor,         type: float, required: false, default: 1.5 }
  - { name: dl_max_factor,          type: float, required: false, default: 1.0,
      desc: "Sin crecimiento por omisión (ver §5)" }
  - { name: dl_shrink_factor,       type: float, required: false, default: 0.6 }
  - { name: dl_grow_iter_threshold, type: int,   required: false, default: 4 }
  - { name: dl_shrink_iter_threshold, type: int, required: false, default: 8 }
  - { name: linear_algebra,         type: str,   required: false, default: "auto" }

requirements:
  - "La magnitud controlada crece monótonamente a lo largo del camino físico (apertura
      de una grieta, alargamiento de la zona que se ablanda). Si decrece con la carga
      de referencia al empezar, el solver lo rechaza con un error que lo explica"
```

```
  - "Los grados de libertad controlados no están prescritos: son libres o esclavos de
      una restricción multipunto con maestros libres (§1). Un apoyo, o un esclavo cuyos
       maestros están todos prescritos, se rechaza. Para controlar un nodo respecto de
      un apoyo fijo basta con el nodo solo"
  - "Heredados del padre: F_ref no nulo; modelo sin mecanismos rígidos (ADR 0019) ni
      mecanismos internos en la tangente inicial si es geométricamente lineal (ADR
      0019, 2026-09-30)"

conventions:
  units: "Δl en las unidades de la magnitud controlada (longitud para desplazamientos
      con coeficientes adimensionales)"
  sign: "la magnitud controlada crece Δl > 0 en cada paso; λ puede crecer o decrecer"

out_of_scope:
  - "Elección automática de la magnitud controlada (control local adaptativo, p. ej. el
       mayor alargamiento de la malla en cada paso)"
  - "Magnitudes que cambian de signo de crecimiento a lo largo del camino (descarga
      global): el control exige monotonía"
  - "Análisis dinámico (régimen cuasiestático)"

acceptance:
  verification:
    - name: retroceso_barra_con_dano_localizado
      setup: "Barra de 10 Truss2D con IsotropicDamage1D α(·κ₀w = 0.475), elemento
          central con κ₀ un 5 % menor; control del alargamiento del elemento débil,
          initial_dlambda = 0.02"
      expect: "Sólo se daña el elemento débil; los puntos tras el pico están sobre la
          curva exacta (distancia normalizada < 1e-3); el desplazamiento del extremo
          retrocede hasta el mínimo exacto 0.616·u_pico"
      tol_rel: 2.0e-3

    - name: rama_descendente_sin_retroceso
      setup: "La misma barra con 2 elementos ((n - 1)·α·κ₀w < 1)"
      expect: "Rama elástica exacta y rama descendente sobre la curva exacta"
      tol_rel: 2.0e-3

    - name: restriccion_satisfecha
      setup: "Barra de 10 elementos del primer caso"
      expect: "El alargamiento controlado crece exactamente Δl por paso"
      tol_rel: 1.0e-8

  specific:
    - name: primer_paso_como_fraccion_de_carga
      setup: "Barra elástica, control del desplazamiento del extremo, initial_dlambda =
           0.25"
      expect: "λ₁ = 0.25 exacto; Δλ = 0.25 en cada paso; solución lineal exacta en λ =
          1"
    - name: errores_claros
      setup: "control vacío o mal formado, nodo o dof inexistentes, dof prescrito (
          apoyo, o esclavo de una restricción multipunto con el maestro apoyado), té
          rminos que se anulan (también tras aplicar las restricciones multipunto, ᵀTc
          = 0), magnitud que decrece con la carga"
      expect: "ValueError con un mensaje que explica el problema"
    - name: control_de_un_esclavo_de_mpc
      setup: "Barra de 10 elementos del primer caso con el nodo derecho del elemento dé
          bil duplicado y atado por una restricción multipunto ux_s = ux_m; el control
```

```
          usa el esclavo (y, aparte, el maestro)"
      expect: "Trazado idéntico al de la barra fundida (oráculo del nodo fundido), con
          el esclavo o con el maestro en el control"
      tol_rel: 1.0e-8
    - name: desde_yaml
      setup: "solver: {type: IndirectDisplacementSolver, control: [{node, dof, coef},
          …]}"
      expect: "Se construye y resuelve"
    - name: mecanismo_interno_en_la_tangente_inicial
      setup: "armadura Pratt del ejemplo 11 sin diagonal, carga horizontal, control
          sobre ux del rodillo (heredado del padre, ADR 0019, 2026-09-30)"
      expect: "Truss2D ⇒ IllPosedSystemError; Truss2DCorot ⇒ un aviso; con la diagonal,
           nada"

references:
  - "de Borst R. (1987). Computation of post-bifurcation and post-failure behavior of
      strain-softening solids. *Computers & Structures* 25(2), 211-224. (Control
      indirecto de desplazamiento.)"
  - "Crisfield M.A. (1991). *Non-linear Finite Element Analysis of Solids and
      Structures*, Vol. 1, Wiley, cap. 9. (Métodos de continuación y restricciones
      alternativas.)"
  - "de Borst R., Sluys L.J. (1999). *Computational Methods in Non-linear Solid
      Mechanics*, TU Delft, §3."
  - "Spec padre: ArcLengthSolver.md. ADR 0015 (corrector de Newton compartido)."
```

13.2.3 Implementación

- Archivo: `solidum/math/solvers/indirect_displacement.py` — `_IndirectProblem` (restricción lineal sobre el `_ArcProblem` del padre, el mismo problema por paso que `solidum.math.solvers` exporta con nombre público como `ArcProblem`) e `IndirectDisplacementSolver` (registrado en `SolverRegistry`, `PIPELINE_KIND = "static"`).
- El padre expone cuatro puntos de extensión que usa esta variante: `_control_measure` (magnitud que fija la restricción: norma en el padre, $\mathbf{c}^\top\mathbf{du}$ aquí), `_predictor_dlambda`, `_step_problem` y `_on_solve_start` (construye $\mathbf{c}$ sobre la numeración de ecuaciones). Sin cambio de comportamiento del padre.
- Tests: `tests/test_indirect_displacement.py` — todos los `acceptance` salvo el de la tangente inicial, que está en `tests/test_safety_net_initial_tangent.py` junto con los de los otros dos solvers estáticos no lineales.

13.2.4 Diálogo

- **2026-09-23** · Formulación propuesta por la IA y validada por el usuario al descubrir, preparando el ejemplo 5 del manual de ejemplos, que ninguno de los dos solvers de arco seguía el retroceso de una barra con daño localizado. Diagnóstico registrado como deuda #27 (arco por disipación) y en §5 (no unicidad del problema incremental).
- **2026-09-30** · Hereda del padre la comprobación de la tangente inicial de la red de seguridad (ADR 0019): el predictor del primer paso inspecciona sus pivotes nulos; error si el modelo es

geométricamente lineal, aviso si no. Sin cambio de formulación ni de resultados en modelos bien planteados.

- **2026-09-29** · Frontera del programa principal (ADR 0020 y criterio de admisión de Reglas.md §1): `DissipationArcLengthSolver`, uno de los «dos solvers de arco» de la entrada anterior, se trasladó al módulo de usuario `dissipation_control` (spec en `docs/user/dissipation_control/specs/DissipationArcLengthSolver.md`; se carga con `user_modules: [dissipation_control]`). En el programa principal quedan `ArcLengthSolver` y esta variante. El problema por paso del padre, que esta variante extiende, se exporta con nombre público como `solidum.math.solvers.ArcProblem`. Sin cambio de formulación ni de resultados.

- **2026-09-30** · Auditoría de las restricciones multipunto (EST-10): el solver rechazaba controlar el esclavo de una restricción multipunto aunque su incremento está determinado, porque exigía grados de libertad libres. Ahora el criterio es la fila de $\mathbf{T}$ (§1): se admite el esclavo con maestros y se rechaza el grado de libertad prescrito (apoyo, o esclavo que hereda el apoyo de sus maestros por el cierre transitivo); la comprobación de términos que se anulan pasa de $\mathbf{c} = \mathbf{0}$ a $\mathbf{T}^\top \mathbf{c} = \mathbf{0}$, que sin restricciones multipunto es lo mismo. El mensaje de rechazo dice «prescrito» en lugar de «restringido» y ya no da la pista del apoyo fijo en el caso de la restricción multipunto. La restricción del paso, el predictor y el corrector no cambian: $\mathbf{du}_t$ y $\mathbf{du}_R$ ya incluían los esclavos reconstruidos por $\mathbf{T}$ ($\mathbf{du}_t = \mathbf{T}\,(\mathbf{T}^\top \mathbf{K}\mathbf{T})^{-1}\mathbf{T}^\top(\mathbf{F}_{\text{ref}} - \mathbf{K}\mathbf{g}) + \mathbf{g}$). Los modelos sin restricciones multipunto dan los mismos resultados. Test nuevo `control_de_un_esclavo_de_mpc` contra el oráculo del nodo fundido.

13.3 LinearSolver

*Spec **retroactiva**: el solver existe desde la primera fase del proyecto y está validado por la inmensa mayoría de tests del pipeline estático. Esta spec lo documenta sin cambiar comportamiento — H-5.3 de la auditoría 2026-05-18.*

13.3.1 Especificación física

0. Descripción general

Análisis estático **lineal** de una estructura discretizada por FEM: resolución directa del sistema algebraico `K·U = F`. Hipótesis: linealidad geométrica y material (tangente constante = rigidez secante), apoyos Dirichlet (homogéneos o no homogéneos), restricciones lineales multipunto opcionales (ADR 0004). Sin iteración. Sin historia de carga.

1. Ecuación de equilibrio resuelta

Forma fuerte semidiscreta:

$$\mathbf{K}\,\mathbf{U} \;=\; \mathbf{F}_{\text{ext}}$$

donde $\mathbf{K}$ es la matriz de rigidez global ensamblada, $\mathbf{U}$ los desplazamientos nodales (incluidos los Dirichlet) y $\mathbf{F}_{\text{ext}}$ las cargas externas (incluidas peso propio si está activado).

2. Condiciones de contorno

Apoyos Dirichlet (homogéneos $u_s = 0$ o no homogéneos $u_s = g$) eliminados directamente por ADR 0004 fase 1. Restricciones lineales multipunto admitidas vía transformación $\mathbf{T}$ + offset $\mathbf{g}$ (ADR 0004 fase 2). El sistema reducido se resuelve en DOFs libres:

$$\mathbf{K}_{\text{red}} = \mathbf{T}^{\top}\mathbf{K}\,\mathbf{T}, \qquad \mathbf{F}_{\text{red}} = \mathbf{T}^{\top}\left(\mathbf{F} - \mathbf{K}\,\mathbf{g}\right)$$

3. Salidas físicas

- Vector de desplazamientos global $\mathbf{U} \in \mathbb{R}^{n_{\text{dof}}}$ (libres + prescritos consistentes).
- Reacciones se recuperan a posteriori vía `domain.reactions(U)`.
- Esfuerzos en puntos de Gauss vía `domain.recompute_gauss_state(U)`.
- El resultado se envuelve en `SolveResult` por la capa pública (`solidum.run`).

13.3.2 Formulación numérica

4. Esquema operativo

Un solo paso:

1. Ensamblar $\mathbf{K}$ (sparse COO → CSR) con caché topológica del Assembler.

2. Aplicar la reducción ADR 0004: $\mathbf{K}_{\text{red}}$, $\mathbf{F}_{\text{red}}$, operadores $\mathbf{T}$ y $\mathbf{g}$.
3. Calcular $\mathbf{F}_{\text{dir}} = \mathbf{T}^{\top}(\mathbf{K}\,\mathbf{g})$ (contribución de Dirichlet no homogéneo).
4. Factorizar $\mathbf{K}_{\text{red}}$ con el backend algebraico seleccionado.
5. Resolver $\mathbf{U}_{\text{red}} = \mathbf{K}_{\text{red}}^{-1}(\mathbf{F}_{\text{red}} - \mathbf{F}_{\text{dir}})$.
6. Expandir $\mathbf{U} = \mathbf{T}\,\mathbf{U}_{\text{red}} + \mathbf{g}$.

5. Predictor / corrector

No aplica — solver no iterativo.

6. Criterio de convergencia

No aplica — la solución directa es exacta (a precisión de máquina y condicionamiento de $\mathbf{K}$).

7. Imposición de Dirichlet y MPC

ADR 0004 fase 1 (eliminación directa de apoyos) y fase 2 (transformación $\mathbf{T}$ + offset $\mathbf{g}$ para MPC). Manejados uniformemente por `Assembler.reduce(...)`.

8. Backend algebraico

Despacho automático (ADR 0003, ampliado por ADR 0017):

- Si el dominio es **simétrico** (todos los materiales con tangente simétrica) ⇒ se asume PD y se factoriza con **Cholesky** (CHOLMOD) si `scikit-sparse` está instalado.
- Si no, o si el dominio declara material **no simétrico** ⇒ **Pardiso** (Intel MKL, multihilo) si `pypardiso` está instalado; si no, **LU** (SuperLU).
- Si Cholesky reporta no-positividad (casi-singular o mal condicionado) ⇒ **fallback automático** al siguiente backend.

Override manual: parámetro `linear_algebra` ∈ {`"auto"`, `"cholesky"`, `"pardiso"`, `"lu"`, `"iterative[:amg|jacobi|none]"`}. El iterativo (ADR 0018) nunca se elige de forma automática; sobre una matriz no simétrica cae al directo con aviso.

Red de seguridad (ADR 0019). Antes de factorizar se comprueba que el modelo no es un **mecanismo rígido** (`MechanismError`, con el movimiento libre descrito). Tras factorizar se buscan **mecanismos internos** y tras resolver se exige que la solución **resuelva el sistema**: el error hacia atrás normado de la solución, escalado por la diagonal, η = `‖r̃‖/(‖K‖·‖ũ‖ + ‖F‖)` ≤ 10^{-10} (`K = D^-½·K·D^-½`, `D = diag(K)`) (los dos con `IllPosedSystemError`). Hasta el 2026-10-20 se exigía `‖F - K·u‖ ≤ 1e-8·‖F‖`, que un solver directo estable no garantiza con rigideces muy desproporcionadas (su residuo es del orden de $\varepsilon\cdot\|K\|\cdot\|u\|$): rechazaba zonas rígidas ×1e6 en mm y $\nu = 0.4999999$, y la propia suite llegaba a 5.7e-9. Antes de esta red, un modelo mal apoyado devolvía en silencio desplazamientos de miles de kilómetros.

Un mecanismo interno se reconoce por su energía (ampliación del 2026-10-20, decisión del usuario): los tres backends directos exponen sus pivotes pequeños frente a la diagonal, `|U_ii|/`

$\sqrt{}$`(|K_rr|·|K_cc|) < 1e-8` (SuperLU con la diagonal de `U`, Cholesky con los pivotes de $\mathtt{LDL}^\top$ y su permutación, Pardiso con `pardiso_getdiag` sobre su matriz escalada); con alguno, se calcula con la misma factorización el modo de menor energía de `K·v = λ·diag(K)·v` (iteración inversa en bloque), y es un mecanismo si no deforma ninguno de los elementos que mueve (energía relativa `< 1e-12`). El error nombra esos elementos y, si se mueven como un sólido rígido, el movimiento ("giro alrededor del eje z que pasa por (1, 1)"). Un contraste de rigidez legítimo (zonas rígidas, ν cerca de 1/2) da pivotes pequeños, pero su modo deforma algún elemento y pasa. Si la factorización no se completa (pivote exactamente nulo), el modo se calcula con la de `K + σ·diag(K)` (σ `= 1e-10`, mismos modos), así que también ese caso sale localizado. Una factorización directa que no llega a completarse (pivote exactamente nulo o fila vacía: un nudo o grado de libertad sin rigidez) da también `IllPosedSystemError`, y no el `RuntimeError` del backend. El backend iterativo no tiene pivotes que inspeccionar: el solver avisa en el log, una vez por análisis, de que un mecanismo interno que la carga no active no se comprueba, y de cómo comprobarlo (análisis modal con densidad, o el solver directo).

Cache de factorización (ADR 0003 fase 2): la primera llamada a `solve` ensambla **K**, reduce y factoriza; la factorización se guarda en el solver. Llamadas posteriores con un **F** distinto reutilizan el factor sin reensamblaje — el coste se reduce a una resolución triangular barata. Si el usuario modifica el modelo entre llamadas (nuevos elementos, BCs, materiales) debe invocar `invalidate_cache()` explícitamente.

9. Adaptatividad / control de paso

No aplica.

10. Caveats numéricos

- **Modelos mal planteados**: un mecanismo rígido se rechaza antes de resolver y uno interno tras resolver (ADR 0019). Queda sin detectar el mal condicionamiento sin singularidad (rigideces muy desproporcionadas): un directo es estable hacia atrás y el residuo sale pequeño aunque el error de la solución pueda no serlo. Con `linear_algebra: iterative`, un mecanismo interno que la carga no activa tampoco se detecta; se avisa.
- **Suposición de PD por simetría**: el despachador asume que un dominio con materiales simétricos tiene **K** positiva definida. Es cierto para elasticidad estándar con suficientes apoyos; falso para casos degenerados (apoyos insuficientes, modos rígidos no restringidos, casi-singularidad geométrica). El fallback Cholesky→LU cubre estos casos.
- **Cache desactualizado**: si el modelo se modifica entre `solve` calls sin llamar a `invalidate_cache`, los resultados serán inconsistentes. La política deliberada es **cache silenciosa** (no se detectan modificaciones del modelo automáticamente) por velocidad — la responsabilidad de invalidar es del usuario.
- **No es no lineal**: si el problema es no lineal (material plástico, geometría corotacional con cargas significativas) usar `NonlinearSolver`. Este solver evaluará la tangente en estado virgen ($\mathbf{U} = 0$) y producirá una respuesta lineal incorrecta sin avisar.

13.3.3 Contrato de implementación

```yaml
name: LinearSolver
kind: solver
status: validated

interface:
  yaml_type: linear
  output: SolveResult
  pipeline_kind: static

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - bathe: "cap. 8 (solución de K·U = R en análisis estático)"
  general_codes:
    - abaqus: "*STATIC (lineal)"
    - opensees: "integrator LoadControl + algorithm Linear"

parameters:
  - { name: linear_algebra, type: str, required: false, default: "auto",
      desc: "Selección del backend algebraico: 'auto' (Cholesky → Pardiso → LU según
          simetría y dependencias instaladas), 'cholesky', 'pardiso', 'lu', 'iterative
          [:amg|jacobi|none]' (ADR 0003, 0017, 0018)" }

requirements:
  - "Modelo lineal: materiales con tangente constante, geometría no corotacional o
      cargas suficientemente pequeñas"
  - "Apoyos suficientes para descartar modos rígidos (estructura cinemáticamente
      estable)"
  - "Compatibilidad ensamblaje-elemento-material verificada por Domain"

conventions:
  units: "heredadas del modelo (ADR 0008)"
  stability: "trivial - un solo paso"

out_of_scope:
  - "Análisis no lineal ⇒ usar NonlinearSolver"
  - "Snap-through / snap-back ⇒ usar ArcLengthSolver"
  - "Análisis dinámico ⇒ usar NewmarkSolver, HHTSolver, etc."
  - "Mal condicionamiento sin singularidad: no hay estimador de condición (ADR 0019)"
  - "Mecanismo interno que la carga no activa con el backend iterativo: sin pivotes que
      inspeccionar; sólo aviso (ADR 0019)"

acceptance:
  verification:
    - name: viga_en_voladizo_carga_puntual
      setup: "viga de Frame2DEuler en voladizo con carga puntual P en el extremo"
      expect: "δ = P·L^3/(3·E·I) ± O(elementos)"
      tol_rel: 1.0e-8
    - name: barra_traccion_uniaxial
      setup: "Truss2D, longitud L, área A, carga axial F"
      expect: "δ = F·L/(E·A) exacto (sin discretización)"
      tol_rel: 1.0e-12
    - name: convergencia_h
      setup: "viga discretizada con N elementos, N ∈ {2, 4, 8, 16}"
      expect: "error decrece con orden teórico del elemento (4 para Hermite cúbicos)"
```

```
      tol_rel: 0.1
    - name: reactions_equilibrio
      setup: "estructura cargada en una dirección"
      expect: "Σ(reacciones) + Σ(cargas externas) ≈ 0 (Newton 3ª ley)"
      tol_abs: 1.0e-9
    - name: dirichlet_no_homogeneo
      setup: "barra con apoyo prescrito u_s = g ≠ 0 en un extremo"
      expect: "solución exacta esperada (translación rígida + alargamiento si carga)"
      tol_rel: 1.0e-12
    - name: cache_factorizacion
      setup: "dos llamadas consecutivas a solve con F distintos sin invalidar cache"
      expect: "segunda llamada reutiliza el factor (mensaje de log diferenciado);
          resultado igual al cómputo desde cero"
      tol_rel: 1.0e-12
    - name: fallback_cholesky_lu
      setup: "matriz simétrica pero casi-singular o no PD efectiva (mecanismo cinemá
          tico añadido y luego soltado)"
      expect: "Cholesky aborta, LU resuelve sin fallar"
    - name: mpc_simple
      setup: "Quad4 con MPC u_2 = u_1 (master/slave)"
      expect: "u_2 = u_1 en la solución, equilibrio respetado"
      tol_abs: 1.0e-12
    - name: mecanismo_interno_no_activado
      setup: "armadura Pratt del ejemplo 11 sin la diagonal del segundo panel, carga
          horizontal que no activa el mecanismo; backends lu, pardiso y cholesky"
      expect: "IllPosedSystemError por pivote nulo con los tres backends directos (ADR
          0019, capa 3; Cholesky desde el 2026-09-30)"
    - name: aviso_backend_iterativo
      setup: "misma armadura con linear_algebra: iterative"
      expect: "solución con un único aviso por análisis de que el mecanismo interno no
          se comprueba"

references:
  - "Bathe K.J. (2014). Finite Element Procedures. Prentice Hall. §2."
  - "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and Applications
      of FEA. Wiley. §2.4 (eliminación de DOFs Dirichlet)."
  - "ADR 0003 - selección automática de backend algebraico."
  - "ADR 0004 - manejo de Dirichlet y constraints lineales."
```

13.3.4 Implementación

- **Archivo**: solidum/math/solvers/linear.py.
- **Clase**: `LinearSolver`, registrada vía `@SolverRegistry.register` con `PIPELINE_KIND = "static"`.
- **Cache**: atributos `_factor`, `_T`, `_g_full`, `_F_dir`, `_n_free` se rellenan lazy en la primera llamada a `solve`. Método público `invalidate_cache()` los pone a `None` para forzar reensamblaje.
- **Fallback Cholesky→LU**: capturado en `_build_cache` vía `CholeskyNotPositiveDefiniteError` (ADR 0003 §5); registra un warning y reintenta con LU.
- **Red de seguridad (ADR 0019)**: `ensure_statically_restrained` en `_build_cache` antes de factorizar; una factorización que falla se traduce a `IllPosedSystemError(factorization_`

`error=...)`; `warn_if_pivots_unavailable` sobre el factor (aviso del backend iterativo, una vez por factorización); `ensure_no_internal_mechanisms` tras factorizar (mecanismos internos, una vez por factorización); `check_linear_solution` tras cada `solve` (error hacia atrás, con la escala de `backward_error_scaling` calculada una vez por factorización). Todas en `solidum/math/solvers/model_checks.py`. Pruebas: `test_model_safety_net.py`, `test_safety_net_initial_tangent.py` (aviso del iterativo, factorización imposible), `test_linalg_cholesky.py` (Cholesky) y `test_internal_mechanisms.py` (mecanismos internos por la energía del modo).

- **Entrypoint público**: `solidum.run(domain, solver=LinearSolver(assembler), F_applied=...)` o `solidum.run_yaml(path)`, que despacha por `PIPELINE_KIND` (regla C de la auditoría).

- **Tests**: cobertura masiva implícita — prácticamente todos los tests del pipeline estático lineal lo usan. Tests específicos del cache: `tests/test_solver_robustness.py::test_linear_solver_cache_reuse`.

13.3.5 Diálogo

- **2026-10-20** · Capa 2 del ADR 0019 (decisión del usuario): el residuo relativo $\|F - K{\cdot}u\|/\|F\| \leq$ `1e-8` se sustituye por el error hacia atrás normado escalado por la diagonal $\leq$ `1e-10`. Sin cambio de resultados; deja de rechazar contrastes de rigidez legítimos. Pruebas: `test_backward_error.py`.

- **2026-10-20** · Ampliación del ADR 0019 (decisión del usuario): un mecanismo interno ya no se declara por el tamaño de un pivote (`|p| < 1e-13·max|K|`, que dependía de las unidades y dejaba pasar los mecanismos de modelos de unos miles de grados de libertad, cuyo pivote es ruido de redondeo que crece con el tamaño) sino por la energía del modo de menor energía de $K{\cdot}v = \lambda{\cdot}\mathrm{diag}(K){\cdot}v$, calculado con la factorización existente cuando algún pivote es pequeño frente a la diagonal (`n_small_pivots`, filtro `SUSPECT_PIVOT_RTOL = 1e-8`): es un mecanismo si no deforma ninguno de los elementos que mueve (energía relativa `< MECHANISM_ENERGY_RTOL = 1e-12`). El mensaje nombra los elementos y describe el movimiento. Sin cambio de resultados en los modelos que se resolvían. Pruebas: `test_internal_mechanisms.py`.

- **2026-05-19** · Spec creada retroactivamente para cerrar el hueco H-5.3. Solver anterior a la convención de specs validadas. La cache de factorización se añadió en la sesión de saneamiento post-auditoría (commit `b517718`, H-4.2); esta spec recoge ese comportamiento como parte del contrato actual.

- **2026-09-22** · Auditoría global: `invalidate_cache()` invalida también las cachés del `Assembler`; además el `ConstraintSet` se reconstruye solo por huella de las restricciones, así que un apoyo añadido tras el primer `solve` surte efecto (antes se ignoraba en silencio). `solidum.run` comitea el estado final.

- **2026-09-30** · Cierres del ADR 0019: el backend Cholesky cuenta sus pivotes nulos (deuda #28) y la capa 3 actúa con los tres directos; una factorización que no se completa da `IllPosedSystemError` en vez del error del backend; con el backend iterativo, aviso explícito

una vez por análisis. Sin cambio de resultados en modelos bien planteados. Corregido en `out_of_scope` un punto obsoleto desde el 2026-09-23 (la detección de modos rígidos existe).

- **2026-09-23** · ADR 0017, 0018 y 0019: el despacho automático incluye Pardiso entre Cholesky y LU; `linear_algebra` admite `pardiso` e `iterative[:amg|jacobi|none]`; red de seguridad (mecanismo rígido antes de factorizar, equilibrio y pivotes nulos tras resolver). Corregidas dos inexactitudes previas de esta spec: Cholesky nunca se hizo con `splu` en modo simétrico (es CHOLMOD, opcional) y el entrypoint `solidum.run_static` no existe (es `solidum.run`).

13.4 NonlinearSolver

*Spec **retroactiva**: el solver existe desde la primera fase del proyecto y está validado por tests de plasticidad, daño y corotacional. Esta spec lo documenta sin cambiar comportamiento — H-5.3 de la auditoría 2026-05-18.*

13.4.1 Especificación física

0. Descripción general

Análisis estático **no lineal** con **control de carga** (load-controlled) en una estructura discretizada por FEM. La no-linealidad puede provenir de la geometría (formulación corotacional, grandes desplazamientos pequeñas deformaciones) o del material (plasticidad, daño). Esquema **incremental-iterativo**: la carga objetivo se aplica por incrementos $\Delta\lambda$, y en cada incremento se resuelve el equilibrio por **Newton-Raphson** con tangente reensamblada y paso bisectado al fallar.

1. Ecuación de equilibrio resuelta

Forma fuerte semidiscreta, residuo:

$$\mathbf{R}(\mathbf{U}, \lambda) \;=\; \lambda\, \mathbf{F}_{\text{ext}}^{\text{ref}} - \mathbf{F}_{\text{int}}(\mathbf{U}) \;=\; \mathbf{0}$$

con $\lambda \in [0, 1]$ factor de carga aplicado a la carga de referencia $\mathbf{F}_{\text{ext}}^{\text{ref}}$, y $\mathbf{F}_{\text{int}}(\mathbf{U})$ fuerzas internas no lineales (dependientes del estado actual de los puntos de Gauss).

2. Condiciones de contorno

Idénticas a `LinearSolver`: Dirichlet (homogéneo / no homogéneo) y MPC vía ADR 0004. Los términos independientes no nulos —valores de Dirichlet no homogéneos y término g de las MPC— se aplican proporcionalmente al factor de carga (incremento $\Delta\mathbf{g} = \mathbf{g} \cdot \Delta\lambda$).

3. Salidas físicas

- $\mathbf{U}$ final converged en $\lambda = 1$.
- Historia disponible vía `step_callback(step, U, load_factor)` que el usuario puede pasar al solver.
- Diagnóstico de divergencia con excepciones tipadas (ADR 0011): `OscillatingNewtonError`, `SingularTangentError`, `LoadExceedsCapacityError`, `UnknownDivergenceError`, todas subclases de `SolverDivergedError`.

13.4.2 Formulación numérica

4. Esquema operativo

```
λ
 = 0Δλ
 = 1 / num_steps
mientras λ < 1:
    intentar avanzar λ→λ   + Δλ:
        U_iter = U_current
        prepare_all_steps(U_current)              # gancho de inicio de paso (ADR 0020,
             P5)
        para iter en range(max_iter):
            K_t, F_int = assemble_non_linear_system(U_iter)
            R = λΔλ(+)·F_ext - F_intδ
            U = K_t^{-1} · R                      # reducido ADR 0004
            U_iter, |||R, F_int = line_search(...) # ADR 0011, opt-in
            si converge (ADR 0007 dual):
                commit_all_states()λ
                 += Δλ
                si iter < threshold y Δλ < base:Δλ←
                      minΔλ( · growth, base)   # adaptativo: acelerar
                break
        si no converge:Δλ
             /= 2                                 # bisección
            si Δλ < min_delta_lambda:
                clasificar divergencia y raise    # ADR 0011
```

5. Predictor / corrector

Predictor: $\Delta \mathbf{U}^{(0)} = \mathbf{0}$ — el incremento empieza desde el estado convergido del paso anterior.

Corrector (Newton iter k): $\mathbf{K}_t^{(k)}\, \delta\mathbf{U}^{(k)} = \mathbf{R}^{(k)}$, $\mathbf{U}^{(k+1)} = \mathbf{U}^{(k)} + \alpha\, \delta\mathbf{U}^{(k)}$, con α del line search (default $\alpha = 1$).

6. Criterio de convergencia (ADR 0007)

Patrón dual fuerza + desplazamiento, con escalas autoderivadas:

$$\|\mathbf{T}^\top \mathbf{R}\| \le \mathrm{atol}_F + \mathrm{rtol}_F \cdot \max(\|\mathbf{F}_{\mathrm{ext}}\|, \|\mathbf{F}_{\mathrm{int}}\|)$$

$$\|\Delta \mathbf{U}\| \le \mathrm{atol}_d + \mathrm{rtol}_d \cdot \|\mathbf{U}\|$$

con $\mathbf{T}$ el operador de restricciones $\mathbf{u} = \mathbf{T}\,\mathbf{u}_{\mathrm{libre}} + \mathbf{g}$ (ADR 0004): $\mathbf{T}^\top \mathbf{R} = \mathbf{0}$ es el equilibrio reducido que resuelve el corrector. Con sólo condiciones de Dirichlet, $\mathbf{T}^\top \mathbf{R} = \mathbf{R}[\text{libres}]$; con restricciones multipunto la fuerza de cada esclavo pasa a sus maestros. Semántica **AND** (ambos simultáneamente). Calibración: en el primer ensamblaje del solve, `force_scale = max(‖F_ext‖, ‖F_int‖, 1)` y `disp_scale = force_scale / max(|diag(K)|)`.

7. Imposición de Dirichlet y MPC

Idéntica a `LinearSolver` — ADR 0004. El vector $\mathbf{g}$ (Dirichlet no homogéneo y término independiente de las MPC) se aplica proporcionalmente al factor de carga λ a través del operador del Assembler (`U_current=U_iter, load_factor=next_load_factor` en la llamada a `reduce`).

8. Backend algebraico

Despacho automático idéntico a `LinearSolver` (Cholesky → Pardiso → LU; iterativo sólo a petición, ADR 0017-0018). Fallback Cholesky→LU activado.

Red de seguridad (ADR 0019): antes del primer paso se rechaza un **mecanismo rígido** (`MechanismError`, con el movimiento libre descrito). Dentro del Newton no se rechaza ningún sistema —cerca de un punto límite resolver uno casi singular es legítimo—, pero el corrector registra el residuo relativo de cada resolución lineal y, si el paso fracasa con un sistema mal resuelto, la divergencia se clasifica como `SingularTangentError` con la causa.

Tangente inicial (ADR 0019, 2026-09-30; decisión física del usuario): la primera factorización del análisis —la de la tangente en $\mathbf{U} = \mathbf{0}$, que el Newton factoriza de todos modos en la primera iteración del primer paso— se inspecciona antes de usarla (`model_checks.check_initial_tangent`; no se factoriza dos veces). Si tiene un mecanismo interno (el criterio del estático lineal: un pivote pequeño frente a la diagonal dispara el cálculo del modo de menor energía, y es un mecanismo si ese modo no deforma ninguno de los elementos que mueve; ADR 0019, ampliación del 2026-10-20) y ningún componente del modelo puede ganar rigidez al deformarse —ningún elemento con `GEOMETRICALLY_NONLINEAR = True` ni material con `IS_UNILATERAL = True`—, el análisis se rechaza con `IllPosedSystemError`: en pequeños desplazamientos la matriz $\mathbf{B}$ es constante y la no linealidad del material sólo reduce la rigidez, así que un mecanismo en $\mathbf{U} = \mathbf{0}$ lo es durante todo el análisis, lo active la carga o no. Si hay elementos corrotacionales o materiales unilaterales (una red de cables sin pretensado se rigidiza al tensarse), sólo se avisa en el log. Con el backend iterativo, que no tiene pivotes, se avisa de que la comprobación no se hace. Las tangentes posteriores no se inspeccionan: cerca de un punto límite una tangente casi singular es legítima.

Newton modificado (ADR 0003 fase 2): parámetro opcional `freeze_tangent_after_iter`. Si es `int N`, el solver factoriza fresca durante las primeras N iteraciones del paso y reutiliza la factorización en las siguientes iteraciones del mismo paso (no entre pasos — la tangente cambia con $\mathbf{U}$). Default `None` = Newton estándar (factoriza cada iter).

9. Adaptatividad / control de paso

Acelerado: si un paso converge en $<$ `NEWTON_ADAPTIVE_GROWTH_ITER_THRESHOLD` iteraciones y $\Delta\lambda < 1/$ `num_steps`, $\Delta\lambda$ se multiplica por `NEWTON_ADAPTIVE_GROWTH_FACTOR` (cota: $1/$`num_steps`).

Bisección: si un paso no converge en `max_iter`, $\Delta\lambda \to \Delta\lambda/2$ y se reintenta. Aborto si $\Delta\lambda <$ `min_delta_lambda`.

Line search ADR 0011 (opt-in, default `False`): variante GLL (Grippo-Lampariello-Lucidi) — backtracking simple por descenso no monótono. Justificación: line search Wolfe puro rechaza pasos Newton correctos en regímenes plásticos/dañados con residuo transitorio creciente. Documentado en ADR 0011.

10. Caveats numéricos

- **Snap-through / snap-back**: NO soportado — `NonlinearSolver` con control de carga falla en puntos límite con $dU/d\lambda \to \infty$ o $d\lambda/dU < 0$. Usar `ArcLengthSolver`. Excepción: el solver atraviesa puntos límite *suaves* (sin cambio de signo en $d\lambda$) con bisección + cinemática corotacional, hallazgo empírico de la auditoría 2026-05-18.

- **Plasticidad perfecta con line search off**: puede oscilar (residuo periódico, no diverge ni converge). Mitigación: activar `line_search=True`.
- **Tangente singular**: lanza `SingularTangentError` con el último estado. Un mecanismo interno de un modelo geométricamente lineal no llega hasta aquí: se rechaza en la tangente inicial (§8).
- **Newton modificado**: con `freeze_tangent_after_iter` no nulo, la convergencia cuadrática se degrada a lineal en las iteraciones congeladas. Útil cuando el coste de factorizar domina (`K_t` grande, materiales con tangente cara de evaluar).

13.4.3 Contrato de implementación

```
name: NonlinearSolver
kind: solver
status: validated

interface:
  yaml_type: nonlinear
  output: SolveResult
  pipeline_kind: static

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - de_borst_crisfield: "§2.4 (Newton-Raphson incremental-iterativo) y §4.1 (búsqueda
        lineal)"
    - bathe: "§8.4 (Newton-Raphson)"
  general_codes:
    - abaqus: "Abaqus/Standard (Newton con incremento automático)"
    - ansys: "NROPT,FULL con bisección"
    - opensees: "algorithm Newton / NewtonLineSearch"

parameters:
  - { name: convergence,                type: ConvergenceCriterion, required: false,
      default: "ConvergenceCriterion()",
      desc: "Política dual fuerza+desplazamiento (ADR 0007)" }
  - { name: max_iter,                   type: int,   required: false, default: 20,
      desc: "Iteraciones Newton máximas por paso antes de bisectar (keyword-only)" }
  - { name: num_steps,                  type: int,   required: false, default: 10,
      desc: "Número base de incrementos para alcanzar λ=1 (keyword-only)" }
  - { name: adaptive,                   type: bool,  required: false, default: true,
      desc: "Habilita acelerado + bisección de Δλ (keyword-only)" }
  - { name: min_delta_lambda,           type: float, required: false, default:
      NEWTON_DEFAULT_MIN_DELTA_LAMBDA,
      desc: "Cota inferior para Δλ; por debajo se aborta con SolverDivergedError (
          keyword-only)" }
  - { name: linear_algebra,             type: str,   required: false, default: "auto",
      desc: "'auto' (Cholesky → Pardiso → LU según simetría y dependencias instaladas),
           'cholesky', 'pardiso', 'lu', 'iterative[:amg|jacobi|none]' (ADR 0003, 0017,
          0018) (keyword-only)" }
  - { name: freeze_tangent_after_iter,  type: int|None, required: false, default: null
      ,
      desc: "Newton modificado: factoriza fresca primeras N iter, congela después (
          keyword-only)" }
```

```
  - { name: line_search,                type: bool,  required: false, default: false,
      desc: "Line search GLL ADR 0011 (keyword-only)" }

requirements:
  - "Modelo con no-linealidad bien definida (material con tangente válida en todo el
      rango de carga)"
  - "Desplazamientos prescritos (Dirichlet no homogéneo y término g de las MPC)
      constantes en magnitud (se aplican proporcionalmente a λ)"
  - "Apoyos suficientes para descartar modos rígidos"

conventions:
  units: "heredadas del modelo (ADR 0008)"
  stability: "no garantizada - los puntos límite con d λ/dU<0 abortan vía bisección"

out_of_scope:
  - "Snap-through / snap-back con d λ/dU<0 ⇒ usar ArcLengthSolver"
  - "Análisis dinámico ⇒ usar NewtonNewmarkSolver, NewtonHHTSolver, etc."
  - "Buckling lineal (eigenproblema generalizado) ⇒ futuro BucklingSolver"

acceptance:
  verification:
    - name: recuperacion_caso_lineal
      setup: "modelo enteramente lineal (Elastic2D + Quad4 + sin corotacional)"
      expect: "resultado idéntico a LinearSolver; Newton converge en 1 iteración por
          paso"
      tol_rel: 1.0e-12

    - name: plasticidad_uniaxial_truss
      setup: "Truss2D con Elastoplastic1D, carga axial creciente hasta plástico"
      expect: "respuesta bilineal exacta (E hasta yield, E_t = E·H/(E+H) después)"
      tol_rel: 1.0e-6

    - name: corotacional_voladizo_grande
      setup: "viga Frame2DEulerCorot, carga lateral grande (rotación >30°)"
      expect: "δ obtenido coincide con benchmark Bathe-Bolourchi 1979"
      tol_rel: 1.0e-3

    - name: damage_bulk_softening
      setup: "Quad4 con IsotropicDamage2D, tracción monotónica hasta post-pico"
      expect: "rama softening trazada; convergencia paso a paso con bisección"
      tol_rel: 1.0e-3

    - name: divergencia_clasificada
      setup: "carga que excede la capacidad última (plasticidad perfecta con mecanismo)
          "
      expect: "raise OscillatingNewtonError o LoadExceedsCapacityError con métricas
          estructuradas (ADR 0011)"

    - name: paso_adaptativo_acelera_y_bisecta
      setup: "trayectoria con tramos elásticos rápidos y zona plástica lenta"
      expect: "Δλ crece en tramos fáciles y bisecta en zonas duras; alcanza λ=1"

    - name: mecanismo_interno_en_la_tangente_inicial
      setup: "armadura Pratt del ejemplo 11 sin la diagonal del segundo panel, carga
          horizontal que no activa el mecanismo (ADR 0019, 2026-09-30)"
```

```
      expect: "Truss2D ⇒ IllPosedSystemError antes de iterar; Truss2DCorot ⇒ un aviso y
           convergencia; con la diagonal, o con material no lineal bien apoyado, ni
          aviso ni error; mismo número de factorizaciones con la comprobación que sin
          ella"

  specific:
    - name: newton_modificado_factoriza_menos
      setup: "freeze_tangent_after_iter=2, 5 iter típicas por paso"
      expect: "factorizaciones por paso ≤ 3 en lugar de 5; resultado dentro de
          tolerancia"
      tol_rel: 1.0e-8

    - name: line_search_corrige_oscilacion_DP
      setup: "Drucker-Prager 2D perfectamente plástico con carga que provoca oscilación
          "
      expect: "line_search=True converge; line_search=False oscila o requiere bisección
           extra"

references:
  - "Crisfield M.A. (1991). Non-linear Finite Element Analysis of Solids and Structures
      , Vol. 1. Wiley. §1-§9."
  - "Bonet J., Wood R.D. (2008). Nonlinear Continuum Mechanics for Finite Element
      Analysis. Cambridge. §8-§9."
  - "de Borst R., Sluys L.J. (1999). Computational Methods in Non-linear Solid
      Mechanics. TU Delft. §3 (Newton-Raphson y variantes)."
  - "ADR 0003, ADR 0004, ADR 0006, ADR 0007, ADR 0011."
```

13.4.4 Implementación

- **Archivo**: solidum/math/solvers/nonlinear.py.
- **Clase**: `NonlinearSolver`, registrada vía `@SolverRegistry.register` con `PIPELINE_KIND = "static"`.
- **Argumentos keyword-only**: todos menos `assembler` y `convergence` son keyword-only (auditoría H-1.5, sesión 2026-05-19, commit `4e4ed54`).
- **Bisección y clasificación**: en `solve`, al fallar $\Delta\lambda$ < `min_delta_lambda` se llama a `classify_divergence(residual_history, delta_history, singular_tangent_detected)` que devuelve la subclase apropiada de `SolverDivergedError` con métricas tipadas.
- **Entrypoint público**: `solidum.run(domain, solver=NonlinearSolver(assembler, ...), F_applied=...)` o `solidum.run_yaml(path)`.
- **Red de seguridad (ADR 0019)**: `ensure_statically_restrained` y `check_initial_tangent(assembler, corrector, …)` al empezar `solve`; la segunda pide al corrector su próxima factorización fresca (`NewtonCorrector.inspect_next_factorization`), que es la de la tangente en $\mathbf{U} = \mathbf{0}$.
- **Tests**: cobertura amplísima — todos los tests de plasticidad/daño/corotacional del pipeline estático lo ejercitan. Tests específicos del comportamiento adaptativo y de divergencia tipada en `tests/test_solver_robustness.py` y `tests/test_solver_diagnostics.py`; de la tangente inicial y del aviso del iterativo en `tests/test_safety_net_initial_tangent.py`.

13.4.5 Diálogo

- **2026-10-20** · Ampliación del ADR 0019 (decisión del usuario): un mecanismo interno ya no se declara por el tamaño de un pivote (`|p| < 1e-13·max|K|`, que dependía de las unidades y dejaba pasar los mecanismos de modelos de unos miles de grados de libertad, cuyo pivote es ruido de redondeo que crece con el tamaño) sino por la energía del modo de menor energía de `K·v = λ·diag(K)·v`, calculado con la factorización existente cuando algún pivote es pequeño frente a la diagonal (`n_small_pivots`, filtro `SUSPECT_PIVOT_RTOL = 1e-8`): es un mecanismo si no deforma ninguno de los elementos que mueve (energía relativa `< MECHANISM_ENERGY_RTOL = 1e-12`). El mensaje nombra los elementos y describe el movimiento. Sin cambio de resultados en los modelos que se resolvían. Pruebas: `test_internal_mechanisms.py`.

- **2026-05-19** · Spec creada retroactivamente para cerrar el hueco H-5.3. Solver anterior a la convención de specs. Comportamiento documenta el estado tras la sesión de saneamiento post-auditoría (commits `4e4ed54` para keyword-only, ADR 0011 ya integrado).

- **2026-09-22** · Auditoría global: un ensamblaje por iteración: la convergencia se evalúa al inicio de la iteración con (‖R(U_k)‖, ‖δU_k−1‖) y el estado trial que se comitea es el del propio ensamblaje en U_k (antes cada iteración ensamblaba dos veces para la misma secuencia de decisiones). `max_iter` es el número máximo de resoluciones por paso. La iteración 0 no evalúa convergencia porque el incremento de Dirichlet λ·g entra en la primera resolución (con control en desplazamiento el residuo en DOFs libres era nulo y declaraba convergencia sin mover el apoyo): todo paso hace al menos una resolución. Expone `steps_done` y `lambda_final`, que `solidum.run` vuelca en `SolveResult.num_steps` (antes siempre 1).

- **2026-09-22** · ADR 0015: el bucle de Newton pasa al corrector compartido `NewtonCorrector`; este solver aporta su problema por paso (`_IncrementalProblem`: residuo `λ·F_ext - F_int`, reducción con el incremento de Dirichlet del paso, sin evaluación en la iteración 0; retirados `_armijo_step` y `_solve_reduced`, ahora `corrector.line_search_step` y `corrector.solve`) y conserva su control de paso. Sin cambio de formulación ni de resultados (suite completa sin tocar ningún valor esperado).

- **2026-09-23** · ADR 0017-0019: despacho con Pardiso e iterativo a petición; comprobación de mecanismo rígido al empezar `solve`; diagnóstico de tangente singular a partir del residuo lineal (sólo diagnóstico, sin cambio del algoritmo).

- **2026-09-30** · **Decisión física del usuario** (red de seguridad, ADR 0019): un mecanismo interno que la carga no activa pasaba sin aviso (Pardiso perturba el pivote nulo y el Newton converge). Ahora se cuentan los pivotes nulos de la tangente inicial: error si el modelo es geométricamente lineal, aviso si tiene elementos geométricamente no lineales (contrato nuevo `Element.GEOMETRICALLY_NONLINEAR`) o materiales unilaterales —esto último, refinamiento de la IA sobre la decisión, confirmado por el usuario—. Reutiliza la primera factorización del Newton. Con el backend iterativo, aviso una vez por análisis. Sin cambio de resultados en modelos bien planteados (suite completa); el único test existente afectado modelaba de verdad un mecanismo interno (`test_model_safety_net.py`, bloques unidos por un nudo).

- **2026-09-29** · Frontera del programa principal (ADR 0020): la discontinuidad embebida es un módulo de usuario (`discontinuities`) y deja de figurar en la cobertura de esta spec.

Retirado el caveat «softening severo con penalización cohesiva rígida»: lo desmiente `tests/user/discontinuities/test_disc_embedded_uniaxial_softening.py`, donde este solver, con control por desplazamiento y $K_e = 10^{15}$, recorre el ablandamiento hasta la separación completa frente a la solución exacta. El fallo antiguo venía de la tangente cohesiva de signo erróneo, corregida el 2026-09-23 (`597f3d9`). Sin cambio de formulación ni de resultados.

- **2026-09-30** · Corrección sin cambio de formulación (deuda #31): con restricciones multipunto el criterio medía $\|\mathtt{R[libres]}\|$, que en la solución no se anula en los maestros —la fuerza del esclavo pasa a ellos—; el Newton, que ya resolvía el sistema reducido del ADR 0004, llegaba a la solución sin declarar convergencia y el solver acababa en `OscillatingNewtonError`. Ahora mide $\|\mathtt{T}^\top\mathtt{R}\|$ (§6), como `NewtonNewmarkSolver`. Con sólo Dirichlet las dos normas coinciden exactamente: ningún resultado previo cambia. Test `tests/test_mpc_nonlinear_static.py` (apoyo oblicuo y placas atadas: reproduce `LinearSolver` y cumple la restricción).

Capítulo 14

Esquemas de Solución — Modal y dinámicos

14.1 CentralDifferenceSolver

*Spec de **solver nuevo** (no variante). Cubre el integrador explícito de segundo orden para análisis dinámico transitorio lineal y no lineal. Aprovecha la masa lumped diagonal habilitada por la fase 2 del ADR 0009 para evitar resolver sistema lineal en cada paso.*

14.1.1 Especificación física

0. Descripción general

Análisis dinámico transitorio **explícito** de segundo orden. Resuelve `M·ü + C·u̇ + K·u = F(t)` (lineal) o `M·ü + C·u̇ + F_int(u) = F(t)` (no lineal) avanzando la aceleración en cada paso por inversión trivial de la masa diagonal lumped — sin sistema lineal, sin Newton interno. Apropiado para wave propagation, impacto, dinámica de respuesta rápida.

Hipótesis cinemáticas globales: las mismas que admite el bloque de elementos del modelo (corotacional, pequeñas deformaciones, etc.). Material lineal o no lineal con historia — en modo `nonlinear=True` el solver llama `Assembler.assemble_non_linear_system(u_n)` cada paso para reevaluar `F_int`.

1. Ecuación de equilibrio resuelta

$$\mathbf{M}\,\ddot{\mathbf{u}} + \mathbf{C}\,\dot{\mathbf{u}} + \mathbf{F}_{\text{int}}(\mathbf{u}) = \mathbf{F}_{\text{ext}}(t)$$

con $\mathbf{F}_{\text{int}}(\mathbf{u}) = \mathbf{K}\,\mathbf{u}$ en el caso lineal.

2. Condiciones iniciales / de contorno

- Apoyos Dirichlet constantes en el tiempo y restricciones multipunto (eliminación directa, ADR 0004), con la condición de diagonalidad de §7.

- $\mathbf{u}(0) = \mathbf{u}_0$, $\dot{\mathbf{u}}(0) = \dot{\mathbf{u}}_0$ (cero por defecto). Sólo se conservan sus componentes libres: los DOFs esclavos se reconstruyen como $\mathbf{T}\,\mathbf{u}_0[\text{libres}] + \mathbf{g}$ y $\mathbf{T}\,\dot{\mathbf{u}}_0[\text{libres}]$, igual que en `NewmarkSolver`; un valor dado en el esclavo de una MPC con maestros que la incumpla se descarta con un aviso de log.
- Cargas externas $\mathbf{F}_{\text{ext}}(t)$ vía `F_func(t)`.
- Amortiguamiento Rayleigh $\mathbf{C} = \alpha\,\mathbf{M} + \beta\,\mathbf{K}$ (mismo contrato que `NewmarkSolver`).

Restricciones multipunto admitidas mientras $\mathbf{T}^\top\mathbf{M}\,\mathbf{T}$ siga siendo diagonal: un solo maestro por esclavo tras el cierre transitivo (atado de nodos, periodicidad, apoyo oblicuo $u_y = \tan\theta\, u_x$). Una MPC con varios maestros cuyo esclavo tiene masa se rechaza (§7).

3. Salidas físicas

`TransientResult` con historiales:

- `u_history[ndof, n_steps+1]` desplazamientos.
- `udot_history[ndof, n_steps+1]` velocidades.
- `uddot_history[ndof, n_steps+1]` aceleraciones.
- `t_history[n_steps+1]` instantes de tiempo.

14.1.2 Formulación numérica

4. Esquema operativo — leapfrog Belytschko-Liu-Moran

Inicialización:

$$\mathbf{a}_0 = \mathbf{M}^{-1}\big(\mathbf{F}(0) - \mathbf{F}_{\text{int}}(\mathbf{u}_0) - \mathbf{C}\,\mathbf{v}_0 - \mathbf{F}_{\text{dir}}\big)$$

$$\mathbf{v}_{1/2} = \mathbf{v}_0 + \tfrac{\Delta t}{2}\,\mathbf{a}_0$$

Paso $n \to n+1$:

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t\,\mathbf{v}_{n+1/2}$$

$$\mathbf{F}_{\text{int}}^{n+1} = \mathbf{K}\,\mathbf{u}_{n+1} \quad \text{(lineal)} \quad \text{o} \quad \text{ensamble}(\mathbf{u}_{n+1}) \quad \text{(no lineal)}$$

$$\mathbf{a}_{n+1} = \mathbf{M}^{-1}\big(\mathbf{F}(t_{n+1}) - \mathbf{F}_{\text{int}}^{n+1} - \mathbf{C}\,\mathbf{v}_{n+1/2} - \mathbf{F}_{\text{dir}}\big)$$

$$\mathbf{v}_{n+1} = \mathbf{v}_{n+1/2} + \tfrac{\Delta t}{2}\,\mathbf{a}_{n+1} \quad \text{(salida de paso)}$$

$$\mathbf{v}_{n+3/2} = \mathbf{v}_{n+1} + \tfrac{\Delta t}{2}\,\mathbf{a}_{n+1} \quad \text{(para el paso siguiente)}$$

Sin Newton interno — la ecuación de aceleración es explícita en $\mathbf{F}_{\text{int}}^{n+1}$ que ya se evaluó en el estado actual.

5. Predictor / corrector

El esquema leapfrog **no tiene predictor/corrector** en el sentido implícito. La actualización $\mathbf{u}_{n+1}$ se calcula sólo con datos en n y $n+1/2$; la velocidad $\mathbf{v}_{n+1}$ se obtiene del semipaso.

6. Criterio de convergencia

No aplica — esquema no iterativo. La convergencia "del problema" se materializa en la condición CFL y en la detección de divergencia.

7. Imposición de Dirichlet y MPC

Eliminación directa idéntica a `NewmarkSolver` (operador $\mathbf{T}$ + offset $\mathbf{g}$), para los apoyos y para las restricciones multipunto: $\mathbf{M}_{\text{red}} = \mathbf{T}^\top \mathbf{M}\,\mathbf{T}$, $\mathbf{K}_{\text{red}} = \mathbf{T}^\top \mathbf{K}\,\mathbf{T}$, $\mathbf{F}_{\text{red}} = \mathbf{T}^\top \mathbf{F}$ y los historiales se expanden con $\mathbf{T}$. Verificado con el oráculo del nodo fundido (`tests/test_mpc_dynamic.py`).

Condición de diagonalidad. El esquema invierte $\mathbf{M}_{\text{red}}$ término a término, así que la exige estrictamente diagonal:

$$\max_{i \neq j} |M_{\text{red},ij}| \leq \texttt{LUMPED_MASS_OFF_DIAGONAL_RTOL} \cdot \max_i |M_{\text{red},ii}|, \qquad \texttt{RTOL} = 10^{-12}.$$

El umbral es puramente relativo (independiente de las unidades del modelo). Con masa concentrada $\mathbf{M} = \operatorname{diag}(m_s)$,

$$(\mathbf{T}^\top \mathbf{M}\,\mathbf{T})_{jk} = \sum_s m_s\, T_{sj}\, T_{sk},$$

de modo que sólo una fila de $\mathbf{T}$ con dos o más coeficientes (una MPC con varios maestros, tras el cierre transitivo) cuyo esclavo tiene masa $m_s > 0$ produce términos fuera de la diagonal: el nodo colgante $u_h = \frac{1}{2}u_a + \frac{1}{2}u_b$ de un refinamiento no conforme, o una unión rígida con giro. El solver la rechaza con `ValueError` nombrando los DOFs esclavos y recomienda `NewmarkSolver`/`HHTSolver` con masa concentrada. Un esclavo multimaestro sin masa no acopla y se admite. La otra causa de rechazo es Frame3D con eje oblicuo (masa global bloque-diagonal por nodo).

8. Backend algebraico

Ninguno — la única operación lineal es $\mathbf{M}^{-1}\cdot\mathbf{r}$ con $\mathbf{M}$ diagonal, implementada como $1/\operatorname{diag}(\mathbf{M})$ y multiplicación elemento a elemento. El solver verifica al construir $\mathbf{M}_{\text{red}}$ que la diagonal es estrictamente positiva y que los off-diagonals son numéricamente nulos (tolerancia $10^{-12} \cdot \max|\text{diag}|$); si no, lanza `ValueError`.

9. Adaptatividad / control de paso

No adaptativo en esta fase — paso temporal constante Δt. La adaptatividad explícita (e.g. estimación de $\omega_{\max}$ por power iteration con actualización por paso) queda diferida hasta que aparezca un caso de uso con $\omega_{\max}$ variable significativamente en el tiempo.

10. Caveats numéricos

- **Estabilidad condicional**. La condición CFL exige $\Delta t < 2/\omega_{\text{máx}}$ donde $\omega_{\text{máx}}$ es la frecuencia natural máxima del sistema discreto. Para una barra axial elástica con elemento de longitud L_e y velocidad de propagación $c_p = \sqrt{E/\rho}$: $\Delta t_{\text{crit}} \approx L_e/c_p$. Para sólidos 2D la fórmula análoga es $\Delta t_{\text{crit}} \approx h_e/c_d$ donde c_d es la velocidad dilatacional. Si se excede, la solución diverge exponencialmente — el solver lo detecta a posteriori (umbral configurable `divergence_threshold`, default $10^8 \cdot |\mathbf{u}_0|$) y aborta con `RuntimeError` informativo.

- **Frame3D oblicuo**. La fase 2 del ADR 0009 documenta que $\mathbf{M}_{\text{lumped}}$ es bloque-diagonal (no estrictamente diagonal) en Frame3D cuando el eje del elemento no coincide con un eje global. Para central differences esto significaría invertir bloques 6×6 por nodo cada paso, **anulando** la ventaja del esquema explícito. El solver rechaza ese caso con `ValueError`.

- **Lumping obligatorio**. `lumping="consistent"` se rechaza al construir el solver — invertir una $\mathbf{M}$ consistente cada paso es caro y elimina la razón de ser del explícito. El usuario debe pasar `lumping="lumped"` (default).

- **Modos espurios de altas frecuencias**. El esquema no introduce amortiguamiento numérico (a diferencia de HHT-α); las altas frecuencias de la malla pueden contaminar la respuesta de bajas frecuencias en problemas de wave propagation. Mitigaciones: refinamiento de malla, lumping puro (mejor filtrado que consistent), o cambio a HHT-α implícito.

14.1.3 Contrato de implementación

```yaml
name: CentralDifferenceSolver
kind: solver
status: validated

interface:
  yaml_type: CentralDifferenceSolver
  output: TransientResult

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - belytschko_liu_moran: "§6.2 (diferencias centradas explícitas)"
    - hughes: "cap. 9 (diferencias centradas: Newmark con β = 0)"
    - bathe: "§9.2 (diferencias centradas)"
  general_codes:
    - abaqus: "Abaqus/Explicit"
    - opensees: "integrator CentralDifference"

parameters:
  - { name: t_end, type: float, required: true, desc: "tiempo final (s)" }
  - { name: dt, type: float, required: true, desc: "paso temporal constante (s); debe
      satisfacer Δt < Δt_crit" }
  - { name: nonlinear, type: bool, required: false, desc: "default False; True
      recalcula F_int cada paso vía ensamble" }
  - { name: rayleigh, type: dict | null, required: false, desc: "C = α·M + β·K; mismo
      contrato que NewmarkSolver" }
  - { name: u0, type: ndarray | null, required: false, desc: "desplazamiento inicial
      global" }
```

```
  - { name: u0_dot, type: ndarray | null, required: false, desc: "velocidad inicial
      global" }
  - { name: F_func, type: callable | null, required: false, desc: "F(t) → ndarray
      global" }
  - { name: lumping, type: str, required: false, desc: "default 'lumped'; 'consistent'
      rechazado" }
  - { name: divergence_threshold, type: float, required: false, desc: "default 1e8;
      multiplicador sobre escala inicial para abortar" }

requirements:
  - "Materiales con `density` declarada (ADR 0008)."
  - "Todos los elementos del modelo deben soportar `compute_mass_matrix(lumping='lumped
      ')` (ADR 0009 fase 2)."
  - "Frame3D con eje oblicuo a globales NO soportado (M_red no es diagonal pura)."
  - "MPC admitidas sólo con un maestro por esclavo (tras el cierre transitivo) o con
      esclavo sin masa: T^T·M·T debe ser diagonal (umbral relativo
      LUMPED_MASS_OFF_DIAGONAL_RTOL·max|diag|)."

conventions:
  units:    "heredadas del modelo (ADR 0008)."
  stability: "Condicional -Δ t < ω2/_max (CFL). Para barras: Δt < L_el/c_p. Sin power
      iteration en esta fase - responsabilidad del usuario calcular o estimar Δt_crit."

out_of_scope:
  - "Estimación automática de Δt_crit (power iteration sobre ¯M¹K) - diferida."
  - "Paso adaptativo Δ(t variable) - diferido."
  - "MPC con varios maestros cuyo esclavo tiene masa (rechazada: T^T·M·T no diagonal;
      usar NewmarkSolver/HHTSolver)."
  - "Frame3D con eje oblicuo (rechazado por arquitectura del lumping)."

acceptance:
  verification:
    - name: free_vibration_1dof_undamped
      setup: "Truss2D 1 elemento con E=25, ρ=2, A=1, L=1 (lumped). ₀u=1, ₀v=0. Δt = T
          /200."
      expect: "u(t) = cosω(t) con ω=5 rad/s, error < 2%"
      tol_rel: 0.02
    - name: step_response_1dof
      setup: "Mismo modelo con ₀F=50·H(t)"
      expect: "u(t) = (₀F/K)(1 - cosω(t)), error < 2% del régimen permanente"
      tol_rel: 0.02
    - name: matches_newmark_at_small_dt
      setup: "Mismo oscilador, Newmark β=1/4 γ=1/2 con masa lumped vs CentralDiff con
          Δt = T/500"
      expect: "u_Newmark - u_CD máximo < 1%"
      tol_rel: 0.01
  specific:
    - name: cfl_violation_diverges
      setup: "Δt = 3·Δt_crit"
      expect: "RuntimeError con mensaje CFL informativo"
      tol_rel: 0
    - name: cfl_threshold_matches_2_over_omega_max_1dof
      setup: "Oscilador 1-GDL ω(=5, Δt_critω=2/=0.4): integrar con Δt = 0.95·Δt_crit vs
          1.05·Δt_crit"
      expect: "0.95·Δt_crit produce trayectoria acotada (30 ciclos); 1.05·Δt_crit lanza
          RuntimeError CFL"
```

```
      tol_rel: 0
    - name: cfl_threshold_matches_2_over_omega_max_2dof
      setup: |
        Cadena 2-DOF (3 trusses E=A=L=1, ρ=1, lumped) con K=[[2,-1],[-1,2]], M=I →
        autovalores cerrados ω²∈ {1, 3}, ω_max√=3, Δt_crit√=2/3. Estado inicial
        excita modo antisimétrico (1, -1) - puro ω_max.
      expect: |
        (1) Δt = 0.95·Δt_crit estable.
        (2) Δt = 1.05·Δt_crit lanza RuntimeError CFL.
        (3) Δt = Δt_crit/10 reproduce u(t) = (cos√(3·t), -cos√(3·t)) con error < 2%.
      tol_rel: 0.02
      ref: "tests/test_central_difference.py::TestCentralDifferenceCFLAnalytic"
    - name: linear_mode_matches_nonlinear_mode_for_linear_material
      setup: "Mismo problema con material lineal puro, solver con nonlinear=False vs
          nonlinear=True"
      expect: "Coincidencia exacta a precisión de máquina"
      tol_rel: 1.0e-9
    - name: consistent_lumping_rejected
      setup: "Construir solver con lumping='consistent'"
      expect: "ValueError"
      tol_rel: 0
    - name: frame3d_oblique_rejected
      setup: "Frame3D con eje en dirección (1,1,1)√/3 e Iy ≠ Iz"
      expect: "ValueError 'no es estrictamente diagonal'"
      tol_rel: 0
    - name: mpc_tie_matches_merged_node
      setup: "Ménsula Frame2DEuler inclinada 30°, partida en el nodo central y atada
          con u_s = u_m en ux, uy, rz (esclavo a cada lado), con y sin Rayleigh"
      expect: "u, u̇, ü y la historia de N, V, M coinciden con el modelo de nodo fundido
          "
      tol_rel: 1.0e-7
      ref: "tests/test_mpc_dynamic.py::test_transitorio_reproduce_el_nodo_fundido"
    - name: mpc_multimaster_with_mass_rejected
      setup: "Nodo colgante u_h = ½·u_a + ½·u_b entre Quad4, a escala de 1 m y de 10 µm
           (masas nodales ~1e-13 kg)"
      expect: "ValueError 'varios maestros' que nombra los DOFs esclavos, a ambas
          escalas"
      tol_rel: 0
      ref: "tests/test_mpc_dynamic.py::
          test_central_difference_rechaza_mpc_multimaestro_con_masa"

references:
  - "Belytschko, T., Liu, W. K., Moran, B. (2014). *Nonlinear Finite Elements for
      Continua and Structures*, §6.2."
  - "Hughes, T. J. R. (2000). *The Finite Element Method*, §9.1."
  - "Crisfield, M. A. (1997). *Non-Linear Finite Element Analysis of Solids and
      Structures*, vol. 2, §24.6."
  - "Bathe, K.-J. (2014). *Finite Element Procedures*, §9.2 (estabilidad y CFL)."
```

14.1.4 Implementación

- Archivo: solidum/math/solvers/central_difference.py
- Clase: `CentralDifferenceSolver` (registrada en `SolverRegistry`)

- Atributo de despacho: `PIPELINE_KIND = "transient"` — `run_yaml` despacha a `run_transient` por el mecanismo declarativo de la regla C (aplicada 2026-05-18 al introducir este solver, primer caso fuera de la jerarquía de Newmark)
- Tests: tests/test_central_difference.py (10 tests verdes)

Notas de traducción: ninguna — la convención de signos del proyecto (Reglas §5) ya es la stress-resultant en el interior, sin reinterpretación específica de central differences.

14.1.5 Diálogo

- *2026-05-18*: regla C aplicada al introducir este solver. Antes de él, `run_yaml::isinstance(NewmarkSolv` capturaba HHT-α por subclasificación; CentralDifference no podía heredar de NewmarkSolver (flujo radicalmente distinto, sin sistema lineal en cada paso, estabilidad condicional). En lugar de añadir una tercera rama `isinstance`, se refactorizó el dispatch a `PIPELINE_KIND` declarativo — futuros solvers no clásicos (Harmonic, ResponseSpectrum, …) no requieren tocar `entry.py`.
- **2026-09-22** · Auditoría global: con `nonlinear=True` el estado de los elementos se comitea en cada paso (antes **nunca**: los materiales con historia se reevaluaban desde el estado virgen y el oscilador elastoplástico conservaba la energía). `M_red` ya no se densifica (`toarray()` costaba O(n^2) de memoria). Documentado sin corregir: con amortiguamiento Rayleigh la fuerza viscosa se evalúa en v_n+1/2 (Belytschko) y el orden observado cae a 1; sin amortiguamiento es 2.
- **2026-09-22** · `record_internal_forces=True` (deuda #15): registra `elem.internal_forces(u_k)` en cada paso con el estado interno de ese paso (`TransientResult.element_forces_history`; `internal_forces_history(domain)` lo devuelve). Opt-in por coste; sin él, reconstrucción lazy con el estado final y aviso. Verificado contra análisis truncados en `t_k` con plasticidad (`tests/test_transient_forces_history.py`). Además `assemble_system` ya no deja el estado trial evaluado en `u = 0` (deuda #18).
- **2026-09-30** · Auditoría de las MPC (ADR 0004): la spec declaraba las restricciones multipunto «no soportadas», pero el solver las reduce con el `T` completo y reproduce el oráculo del nodo fundido hasta el redondeo (desplazamientos, velocidades, aceleraciones e historia de N, V, M; `tests/test_mpc_dynamic.py`). Se documentan como admitidas con la condición de diagonalidad de §7. Dos correcciones en la comprobación de `M_red` diagonal: (1) el umbral pasa a ser puramente relativo, `off_max > RTOL·diag_max`; antes era `RTOL·max(diag_max, 1)`, absoluto por debajo de 1 kg, y en un modelo a escala de 10 µm (masas nodales ~1e-13 kg) el acoplamiento de un nodo colgante quedaba por debajo, se descartaba en silencio y la masa del esclavo contaba ¼ en cada maestro en lugar de ½ ($\Sigma m\cdot\ddot{u}/\Sigma F = 1.098$ en vez de 1); (2) cuando la causa es una MPC con varios maestros cuyo esclavo tiene masa, el mensaje la nombra con sus DOFs en lugar de culpar a un Frame3D oblicuo, que queda como diagnóstico del resto de casos. Un esclavo multimaestro sin masa no acopla y se sigue admitiendo. Además, si `u0` o `u0_dot` incumplen una MPC con maestros, el solver lo avisa en el log (el valor del esclavo se descarta y se reconstruye como `T·u0[libres] + g`); las filas Dirichlet no avisan. Sin cambio de física en los modelos que ya funcionaban.

14.2 HHTSolver

*Variante de `NewmarkSolver` según la regla de variantes (Reglas.md §4). Documenta **solo lo que cambia** respecto al padre. Implementa el integrador HHT-α (1977) para introducir **amortiguamiento numérico controlado** en altas frecuencias preservando segundo orden de precisión y estabilidad incondicional. Reusa todas las decisiones arquitecturales del ADR 0009 — sin ADR nuevo.*

14.2.1 Qué cambia respecto a `NewmarkSolver`

`NewmarkSolver` integra `M·ü + C·u̇ + K·u = F(t)` evaluando todas las fuerzas en el instante final `t_{n+1}` del paso. Con la elección β=1/4, γ=1/2 (regla trapezoidal, default) el integrador es **no disipativo**: los modos de alta frecuencia espurios introducidos por la discretización persisten indefinidamente en la respuesta y pueden enmascarar la señal de interés.

HHT-α modifica la ecuación de equilibrio temporal para evaluar las **fuerzas viscosas, elásticas y externas** en un instante intermedio `t_{n+1-α}`, mientras la **fuerza inercial** se mantiene en `t_{n+1}`. Esto introduce disipación numérica controlada por un único parámetro escalar $\alpha \in$ `[-1/3, 0]`, sin sacrificar la precisión de segundo orden global ni la estabilidad incondicional. Con $\alpha = 0$ se recupera Newmark trapezoidal.

Ecuación de movimiento HHT-α

$$\mathbf{M}\,\ddot{\mathbf{u}}_{n+1} + (1+\alpha)\,\mathbf{C}\,\dot{\mathbf{u}}_{n+1} - \alpha\,\mathbf{C}\,\dot{\mathbf{u}}_n + (1+\alpha)\,\mathbf{K}\,\mathbf{u}_{n+1} - \alpha\,\mathbf{K}\,\mathbf{u}_n \;=\; (1+\alpha)\,\mathbf{F}(t_{n+1}) - \alpha\,\mathbf{F}(t_n)$$

Para preservar el orden 2 y la estabilidad incondicional, los parámetros Newmark se derivan automáticamente del α:

$$\beta = \frac{(1-\alpha)^2}{4}, \qquad \gamma = \frac{1-2\alpha}{2}$$

El usuario pasa solo `alpha`; β y γ se autoderivan salvo override explícito.

Sistema efectivo

Sustituyendo los predictores y correctores Newmark estándar (idénticos a la fase 3 con los β, γ autoderivados), el sistema efectivo a resolver en cada paso es:

$$\mathbf{A}_{\text{eff}}\,\ddot{\mathbf{u}}_{n+1} \;=\; \mathbf{b}$$

con

$$\mathbf{A}_{\text{eff}} \;=\; \mathbf{M} + (1+\alpha)\,\gamma\Delta t\,\mathbf{C} + (1+\alpha)\,\beta\Delta t^2\,\mathbf{K}$$

$$\mathbf{b} \;=\; (1+\alpha)\,\mathbf{F}_{n+1} - \alpha\,\mathbf{F}_n - \mathbf{F}_{\text{dir}} - (1+\alpha)\,\mathbf{C}\,\tilde{\dot{\mathbf{u}}} + \alpha\,\mathbf{C}\,\dot{\mathbf{u}}_n - (1+\alpha)\,\mathbf{K}\,\tilde{\mathbf{u}} + \alpha\,\mathbf{K}\,\mathbf{u}_n$$

`A_eff` es constante (depende solo de M, C, K y de los coeficientes), por lo que se **factoriza una sola vez** al inicio del análisis (igual que Newmark). Cada paso temporal es una resolución triangular barata.

Disipación numérica controlada

El radio espectral del operador de amplificación en alta frecuencia ($\Delta t \to \infty$) es

$$\rho_\infty = \frac{1+\alpha}{1-\alpha}$$

Para α = 0: ρ_∞ = 1 (sin disipación, persistencia indefinida del modo). Para α = -0.05: $\rho_\infty \approx$ 0.905 (disipación ligera, modo decae 9.5 % por periodo de alta frecuencia). Para α = -1/3 (límite teórico): ρ_∞ = 0.5 (disipación máxima manteniendo orden 2).

La curva de disipación es **selectiva**: amortigua significativamente solo los modos con $\omega\Delta t > \pi$, dejando casi intactos los modos de interés con $\omega\Delta t \ll 1$. Es lo que distingue a HHT-α de simplemente subir el γ Newmark (que disipa en *todas* las frecuencias por igual y baja el orden a 1).

Recuperación del comportamiento Newmark

Con α = 0, los coeficientes auto-derivados quedan β = 1/4, γ = 1/2 y la ecuación de movimiento HHT colapsa exactamente a la de Newmark trapezoidal. `HHTSolver(alpha=0.0)` reproduce **bit a bit** los resultados de `NewmarkSolver` con `beta=0.25, gamma=0.5`. Esto se valida en los acceptance.

Variante no lineal

`NewtonHHTSolver` (subclase de `NewtonNewmarkSolver`) aplica el mismo cambio de la ecuación de movimiento al residuo dinámico. El Newton interno y su jacobiano se ven afectados análogamente:

$$\mathbf{R}(\ddot{\mathbf{u}}_{n+1}) \;=\; (1{+}\alpha)\,\mathbf{F}_{n+1} - \alpha\,\mathbf{F}_n - [(1{+}\alpha)\,\mathbf{F}_{\text{int}}(\mathbf{u}_{n+1}) - \alpha\,\mathbf{F}_{\text{int}}(\mathbf{u}_n)] - [(1{+}\alpha)\,\mathbf{C}\,\dot{\mathbf{u}}_{n+1} - \alpha\,\mathbf{C}\,\dot{\mathbf{u}}_n] - \mathbf{M}\,\ddot{\mathbf{u}}_{n+1}$$

$$\mathbf{J} \;=\; \mathbf{M} + (1+\alpha)\,\gamma\Delta t\,\mathbf{C} + (1+\alpha)\,\beta\Delta t^2\,\mathbf{K}_{\text{t}}$$

Todo lo demás (criterio de convergencia, line search opt-in del ADR 0011, telemetría tipada de divergencia, commit/rollback de estado, amortiguamiento Rayleigh constante en el tiempo) se hereda de `NewtonNewmarkSolver` sin cambios.

14.2.2 Contrato de implementación

```yaml
name: HHTSolver
kind: solver
status: validated
extends: NewmarkSolver       # variante según Reglas §4

interface:
  type_yaml: HHTSolver
  pipeline_kind: transient    # mismo que NewmarkSolver
```

```
admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 9 (método α de Hilber-Hughes-Taylor)"
  general_codes:
    - abaqus: "*DYNAMIC (HHTα-, Abaqus/Standard)"
    - opensees: "integrator HHT"

parameters_extra_to_NewmarkSolver:
  - { name: alpha,  type: float, required: false, default: -0.05,
      desc: "Parámetro HHT en [-1/3, 0]. α=0 recupera Newmark trapezoidal.α
             =-0.05 (default) es disipación ligera estándar; α=-1/3 disipación máxima."
           }

parameters_overridden_with_autoderivation:
  - { name: beta,   type: float, required: false, default: "(1-alpha)^2/4",
      desc: "Auto-derivado del alpha si no se pasa; override explícito posible
             (no recomendado salvo experimentación)." }
  - { name: gamma,  type: float, required: false, default: "(1-2*alpha)/2",
      desc: "Auto-derivado del alpha si no se pasa." }

parameters_inherited:
  # Idénticos a NewmarkSolver: t_end, dt, rayleigh, u0, u0_dot, F_func,
  # linear_algebra, lumping. Ver docs/specs/NewmarkSolver.md.

acceptance:
  recovery:
    - name: alpha_zero_matches_NewmarkSolver
      setup: "Oscilador 1 GDL con HHTSolver(alpha=0.0) y NewmarkSolver(beta=0.25, gamma
          =0.5)"
      expect: "u_history coincide bit-a-bit (atol 1e-10)"

  damping:
    - name: high_frequency_dissipation
      setup: "Dos osciladores acoplados con ω₁=1 rad/s y ω₂=100 rad/s; condición
          inicial excita ambos modos"
      expect: "HHTSolver(alpha=-0.1) atenúa el modo alto en >50% tras 5 periodos del
          modo bajo,
               manteniendo amplitud del modo bajo dentro de 5% del valor inicial"

    - name: spectral_radius_at_high_frequency_limit
      setup: "Δt grande (>10 periodos del modo); HHTSolver con varios α"
      expect: "ρω_ observado = α(1+)α/(1-) dentro de tolerancia 5%"

    - name: amplification_matrix_spectral_radius_analytic
      setup: "Oscilador 1-GDL con HHTSolverαϵ({-0.05, -0.10, -1/3}); Δt tal que Ωω=·Δt
          =2"
      expect: |
        Razón de contracción por paso medida en la simulación
        coincide con maxλ|(A)|, donde A es la matriz de amplificación 3×3 HHTα-
        (Hughes 1987 §9.3, Bathe 2014 §9.5.4): Dβα=1+(1+)Ω², Aαβω[0,0]=(1+²)/D, …
        Tolerancia 1%.
      ref: "tests/test_hht.py::TestHHTNumericalDampingAnalytic"

  stability:
    - name: unconditional_stability_large_dt
      setup: "Sistema lineal con Δt = 100·T (T periodo natural), 10 pasos"
```

```
      expect: "u_history acotada (no diverge) para todo α∈  [-1/3, 0]"

  nonlinear:
    - name: oscilador_plastico_con_disipacion_HHT
      setup: "Truss elastoplástico 1 GDL, vibración libre desde u_0 más allá del yield"
      expect: "NewtonHHTSolver(alpha=-0.1) reduce oscilación espuria post-yielding
               manteniendo decay físico de la disipación plástica"

references:
  - "Hilber, H. M., Hughes, T. J. R. & Taylor, R. L. (1977). Improved numerical
     dissipation for time integration algorithms in structural dynamics.
     Earthquake Eng. Struct. Dyn. 5(3), 283-292."
  - "Hughes, T.J.R. (2000). The Finite Element Method, §9.3 (HHTα- y generalizedα-)."
  - "Chopra, A. K. (2017). Dynamics of Structures, §15.4 (integradores con disipación
      numérica)."
  - "ADR 0009 §Hoja de ruta - fila 3 (Newmark / HHTα-)."
  - "Spec padre: [docs/specs/NewmarkSolver.md](NewmarkSolver.md)."
  - "Spec hermana: [docs/specs/NewtonNewmarkSolver.md](NewtonNewmarkSolver.md) - padre
     del NewtonHHTSolver."
```

14.2.3 Implementación

- **Archivo**: solidum/math/solvers/newmark.py — junto a `NewmarkSolver` y `NewtonNewmarkSolver` (lógicamente cercanos; comparten predictores Newmark, reducción Dirichlet y amortiguamiento Rayleigh).
- **Clases**:
- `HHTSolver(NewmarkSolver)` — variante lineal con disipación numérica.
- `NewtonHHTSolver(NewtonNewmarkSolver)` — variante no lineal con Newton interno + disipación numérica.

Ambas registradas con `@SolverRegistry.register`.

- **Auto-derivación de β, γ**: en `__init__`, si el usuario no pasa β o γ explícitos, se computan desde α antes de invocar `super().__init__`. Esto asegura que el padre vea la β, γ ya correctas y la mayor parte del flujo herede sin cambios.
- **Solve overrideado**: `solve()` se reescribe en ambas subclases para usar las ecuaciones HHT-α (ver §"Sistema efectivo" y §"Variante no lineal" arriba). El estado anterior `F_int_n`, `F_n`, `u̇_n` necesario para el término $-\alpha\cdot$`X_n` se mantiene en variables locales del bucle.
- **Despacho YAML**: `solver: type: HHTSolver` (o `NewtonHHTSolver`). Como `HHTSolver` hereda de `NewmarkSolver` y `NewtonHHTSolver` hereda de `NewtonNewmarkSolver`, el `isinstance` en `entry.run_yaml` los detecta como `transient` automáticamente. **No dispara la regla C arquitectural** (que esperaría una rama no-clásica que NO sea subclase de las existentes).
- **Tests**: `tests/test_hht.py` nuevo con los acceptance arriba.

14.2.4 Diálogo

- **2026-05-18** · Spec creada como **variante** de `NewmarkSolver` (y `NewtonHHTSolver` como variante de `NewtonNewmarkSolver`) aplicando la regla §4. Sin ADR nuevo: reusa la decisión del ADR 0009 fila 3 ("Transitorio lineal Newmark / HHT-α"). Sin entrada propia en el manual estructural — visible al usuario solo a través del `type` YAML y el parámetro `alpha`.

- **2026-05-18** · Decisión sobre el default de `alpha`: **`−0.05`** (disipación ligera estándar) en lugar de `0` (sin disipación). Razones físicas: (i) si el usuario pide `HHTSolver`, está pidiendo disipación — default α=0 lo convierte en un alias trivial de `NewmarkSolver`, ruido en el catálogo; (ii) `-0.05` es el valor canónico de Hilber 1977 (artículo original) y de los códigos de referencia (Abaqus, OpenSees) por ser balance entre disipación útil (>9 % por ciclo en altas frecuencias) y precisión preservada (<0.5 % en modos de interés); (iii) no introduce regresiones — `HHTSolver` es un solver nuevo, nadie lo usa hoy. Diferente del default `line_search=False` del ADR 0011, que era *no* poder romper código existente; aquí no hay código existente que romper.

- **2026-05-18** · Decisión sobre auto-derivación de β, γ: por defecto sí, desde `alpha`. Override explícito posible pero no recomendado (combinaciones α, β, γ arbitrarias pueden perder estabilidad incondicional u orden 2). El parámetro de la API es **el** α, no la terna; β, γ son consecuencia matemática de α salvo experimentación expresa. Esto simplifica el uso típico y centraliza la responsabilidad: si el usuario solo conoce α, el solver hace el resto.

- **2026-09-22** · Auditoría global: mismos diagnósticos que `NewmarkSolver` (masa singular, conteo de pasos).

- **2026-09-22** · `record_internal_forces=True` (deuda #15): registra `elem.internal_forces(u_k)` en cada paso con el estado interno de ese paso (`TransientResult.element_forces_history`; `internal_forces_history(domain)` lo devuelve). Opt-in por coste; sin él, reconstrucción lazy con el estado final y aviso. Verificado contra análisis truncados en `t_k` con plasticidad (`tests/test_transient_forces_history.py`). Además `assemble_system` ya no deja el estado trial evaluado en `u = 0` (deuda #18).

- **2026-09-30** · Auditoría de las MPC (ADR 0004): hereda de `NewmarkSolver` la reducción con `T` y la proyección de las condiciones iniciales (esclavos reconstruidos como `T·u0[libres] + g`, con aviso de log si `u0` o `u0_dot` incumplen una MPC con maestros). Primer test con MPC: oráculo del nodo fundido en u, u̇, ü y N, V, M (`tests/test_mpc_dynamic.py`). Sin cambio de física.

14.3 HarmonicSolver

*Spec de **solver nuevo**. Resuelve `(-ω²M + iωC + K)·û = F` para un barrido de frecuencias y devuelve la amplitud compleja `û(ω)`. Análisis lineal en frecuencia, sin transitorio.*

14.3.1 Especificación física

0. Descripción general

Análisis **lineal** de respuesta forzada armónica en estado estacionario. La excitación `F(t) = Re{F·e^{iωt}}` produce una respuesta `u(t) = Re{û(ω)·e^{iωt}}` también armónica con la misma frecuencia. El solver calcula `û(ω)` para un barrido de frecuencias. Útil para:

- Funciones de transferencia (FRFs) entrada→salida.
- Diagnóstico de resonancias previo a análisis transitorio caro.
- Respuesta forzada en estructuras con cargas periódicas (maquinaria rotante, sismo armónico, viento turbulento descompuesto en armónicos).

Hipótesis cinemáticas y material: lineales (`K` y `M` constantes). No se modelan grandes desplazamientos, plasticidad ni daño — quien quiera no linealidad debe usar `NewtonNewmarkSolver` con excitación armónica.

1. Ecuación de equilibrio resuelta

$$\left(-\omega^2\mathbf{M} + i\omega\mathbf{C} + \mathbf{K}\right)\hat{\mathbf{u}}(\omega) = \hat{\mathbf{F}}$$

para cada ω del barrido. `C = α·M + β·K` Rayleigh.

2. Condiciones iniciales / de contorno

- Apoyos Dirichlet constantes y restricciones multipunto (eliminación directa, ADR 0004).
- `F` $\in \mathbb{C}^{\wedge}$ndof. Real puro si la carga lleva fase cero; complejo si lleva fase.
- **No** hay condiciones iniciales — el estado estacionario es independiente de ellas (el transitorio se descarta por definición).
- Restricciones multipunto por el mismo operador `T`: `û = T·û_libre`. El término independiente `g` no interviene (el problema armónico es homogéneo), igual que el valor de un apoyo.

3. Salidas físicas

`HarmonicResult` (inmutable):

- `omega[n_omega]` frecuencias evaluadas (rad/s).
- `frequencies_hz` propiedad derivada.

- `u_complex[ndof, n_omega]` amplitud compleja del desplazamiento.
- `F_complex[ndof]` amplitud compleja de la carga (informativa).
- Métodos auxiliares: `.amplitude()` → `|û|`, `.phase()` → `arg(û)`.

14.3.2 Formulación numérica

4. Esquema operativo

```
Ensamblar K, M.
C = α·M + β·K (Rayleigh).
Reducir por Dirichlet: K_red, M_red, C_red, F_red.
Para cada ω en el barrido:
    Zω() = ω-2·M_red + ωi·C_red + K_red   [matriz compleja]
    û_redω() = Zω()-1 · F_red             [spsolve complejo]
    ûω() = T · û_redω() [expand a globales]
```

5. Predictor / corrector

No aplica — un solve directo por frecuencia.

6. Criterio de convergencia

No aplica — esquema no iterativo. La precisión depende del backend algebraico (`scipy.sparse.linalg.spsolve` LU complejo).

7. Imposición de Dirichlet y MPC

Eliminación directa idéntica a `NewmarkSolver`. En DOFs prescritos `û = 0` (el apoyo no oscila). Las restricciones multipunto entran por el mismo operador: `Z_red(ω) = T`$^{\top}$`·Z(ω)·T`, `F_red = T`$^{\top}$`·F` y `û = T·û_libre` (sin `g`). Verificado con el oráculo del nodo fundido (`tests/test_mpc_dynamic.py`).

8. Backend algebraico

Factorización LU compleja por frecuencia vía `scipy.sparse.linalg.spsolve(Z.tocsc(), F_red)`. **No hay caché**: `Z` cambia con ω. Coste dominante del análisis es 'O(n_omega · coste_factorizacion(Z))'.

Posible optimización futura (fuera de esta fase): superposición modal — ortogonalizar la respuesta sobre los primeros `n_modes` modos del problema generalizado, lo que reduce el barrido a `n_omega` operaciones escalares. Justifica el dispositivo cuando `n_modes` $\ll$ `n_dof` y el operador es simétrico.

9. Adaptatividad / control de paso

Espaciado del barrido controlado por el usuario:

- `scale="linear"`: `np.linspace(omega_min, omega_max, n_omega)`.
- `scale="log"`: `np.geomspace(omega_min, omega_max, n_omega)` —

apropiado para FRFs sobre rangos de varias décadas.

- Vector explícito vía `omega=...` — usuarios con criterios

específicos (densificar alrededor de resonancias conocidas, etc.).

No hay refinamiento adaptativo de la malla de frecuencia.

10. Caveats numéricos

- **Resonancia exacta sin amortiguamiento**: `Z(ω_n)` es singular —

`spsolve` puede fallar o devolver valores enormes. Mitigación: evitar ω coincidente con un modo, o añadir amortiguamiento Rayleigh.

- **Mal condicionamiento cerca de resonancia**: la factorización LU

pierde precisión a medida que `Z` se aproxima a la singularidad. Es intrínseco al método directo; superposición modal es la alternativa estable.

- **Cargas con fase**: `F` compleja requiere construcción desde código

Python; YAML no soporta tipos complejos nativos. Si el usuario YAML necesita fase, debe construir el dominio programáticamente.

- **No analiza estabilidad** del estado estacionario (no aplica al

problema lineal — el estado estacionario siempre existe y es único para `C` definida positiva y `K` regular).

14.3.3 Contrato de implementación

```yaml
name: HarmonicSolver
kind: solver
status: validated

interface:
  yaml_type: HarmonicSolver
  output: HarmonicResult

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - zienkiewicz_taylor: "vol. 1 (6.ª ed.), §16.7: respuesta periódica forzada"
  general_codes:
    - abaqus: "*STEADY STATE DYNAMICS, DIRECT"
    - ansys: "HROPT,FULL"

parameters:
  - { name: omega, type: array-like | null, required: false, desc: "vector explícito (
      rad/s); excluye omega_min/omega_max/n_omega/scale" }
  - { name: omega_min, type: float, required: false, desc: "límite inferior del barrido
      (rad/s)" }
  - { name: omega_max, type: float, required: false, desc: "límite superior del barrido
      (rad/s)" }
```

```
  - { name: n_omega, type: int, required: false, desc: "número de puntos del barrido,
      default 100" }
  - { name: scale, type: str, required: false, desc: "'linear' (default) o 'log'" }
  - { name: F_amplitude, type: ndarray | null, required: false, desc: "amplitud
      compleja shape (ndof,); default ceros; YAML inyecta desde point_loads como real"
      }
  - { name: rayleigh, type: dict | null, required: false, desc: "C = α·M + β·K; sin
      amortiguamiento ⇒ pico singular en resonancia" }
  - { name: lumping, type: str, required: false, desc: "'consistent' (default) o '
      lumped'" }

requirements:
  - "Materiales con `density` declarada (ADR 0008)."
  - "Problema lineal - `K` y `M` constantes en el barrido."

conventions:
  units:    "heredadas del modelo (ADR 0008). ω en rad/s - coherente con frecuencias de
       ModalSolver."
  stability: "Lineal en frecuencia - estable para C definida positiva. Mal condicionado
       en ω→ω  _n sin amortiguamiento."

out_of_scope:
  - "Superposición modal - diferida; el barrido directo es más simple y suficiente para
       los tamaños actuales del catálogo."
  - "No linealidad (geométrica o material) - usar NewtonNewmarkSolver con excitación
      armónica."
  - "Cargas con fase declaradas desde YAML - usar construcción desde código."

acceptance:
  verification:
    - name: transfer_function_no_damping
      setup: "1 GDL con K=25, M=1; barrido en ω lejos de resonancia"
      expect: "û_realω() =^F/(K -ω ^2M); û_imag ≈ 0"
      tol_rel: 1.0e-10
    - name: transfer_function_rayleigh_damping
      setup: "1 GDL con α-mass Rayleigh"
      expect: "û =^F/(K -ω ^2M + ωic) complejo"
      tol_rel: 1.0e-9
  specific:
    - name: resonance_peak_location
      setup: "1 GDL con ξ=2% Rayleigh; barrido fino alrededor de ω_n"
      expect: "pico en ω = ω_n·√(1 -ξ 2^2)"
      tol_rel: 0.01
    - name: matches_newmark_steady_state
      setup: "Newmark con F(t)=₀Fcosω(t), suficiente t_end para descartar transitorio
          τ(8)"
      expect: "amplitud Newmark estacionaria = |û_harmonic|"
      tol_rel: 0.02
    - name: log_sweep_geometric_spacing
      setup: "scale='log', omega_min=1, omega_max=1000, n_omega=4"
      expect: "omega = geomspace(1, 1000, 4) = [1, 10, 100, 1000]"
      tol_rel: 1.0e-12
    - name: mpc_tie_matches_merged_node
      setup: "ménsula de Frame2DEuler inclinada 30° partida en su nodo central y atada
          con u_s = u_m en ux, uy y rz, con el esclavo a cada lado del corte; masa
          consistente y concentrada, con y sin Rayleigh"
```

```
      expect: "û del modelo atado (también en el nodo duplicado) = û del modelo de nodo
          fundido"
      tol_rel: 1.0e-10
      ref: "tests/test_mpc_dynamic.py::test_armonico_reproduce_el_nodo_fundido"

references:
  - "Bathe, K.-J. (2014). *Finite Element Procedures*, §9.5."
  - "Clough, R. W., Penzien, J. (1993). *Dynamics of Structures*, §4.6."
  - "Chopra, A. K. (2017). *Dynamics of Structures*, cap. 4."
```

14.3.4 Implementación

- Archivo: solidum/math/solvers/harmonic.py
- Clase: `HarmonicSolver` (registrada en `SolverRegistry`)
- Atributo de despacho: `PIPELINE_KIND = "harmonic"` → `solidum.entry.run_harmonic` → `HarmonicResult`.
- Tests: tests/test_harmonic.py (11 tests verdes).
- Resultado: `HarmonicResult` — dataclass inmutable con `omega`, `u_complex`, `F_complex`, propiedades `frequencies_hz`, `n_omega`, métodos `amplitude()` y `phase()`.

14.3.5 Diálogo

- *2026-05-18*: nuevo `PIPELINE_KIND = "harmonic"` añadido al despacho declarativo de `run_yaml` (regla C, ya aplicada en D2). Tercer valor del literal junto a `"modal"`, `"transient"`, `"static"`. El parser YAML inyecta automáticamente las `point_loads` del bloque estándar como `F_amplitude` real cuando el usuario no la especifica explícitamente — convención coherente con el uso de `external_forces` en análisis estáticos.
- **2026-09-30** · Auditoría de las MPC (ADR 0004): la spec declaraba las restricciones multipunto «no soportadas» y «diferidas», pero el solver ya reduce con el `T` completo ($T^\top KT$, $T^\top MT$, $T^\top CT$, $T^\top F$, expansión con `T`) y reproduce el oráculo del nodo fundido hasta el redondeo, con masa consistente y concentrada y con y sin Rayleigh (`tests/test_mpc_dynamic.py`). Se documentan como admitidas; `g` no interviene, como el valor de un apoyo. Sin cambio de código ni de física.

14.4 ModalSolver

Orden de trabajo. La especificación física y la formulación numérica son patrimonio del usuario (revisa con detalle). La IA implementa y rellena las secciones marcadas.

14.4.1 Especificación física

0. Descripción general

Análisis modal lineal de una estructura discretizada por FEM: cálculo de frecuencias naturales y modos de vibración no amortiguados. Hipótesis: linealidad geométrica y material, amortiguamiento nulo (`C = 0`), masa constante en el tiempo (lagrangeano total).

1. Problema físico

Vibración libre no amortiguada del sistema discreto:

$$\mathbf{M}\,\ddot{\mathbf{u}}(t) + \mathbf{K}\,\mathbf{u}(t) = \mathbf{0}.$$

Solución armónica $\mathbf{u}(t) = \boldsymbol{\phi}\, e^{i\omega t}$. Sustituyendo:

$$\left(\mathbf{K} - \omega^2 \mathbf{M}\right) \boldsymbol{\phi} = \mathbf{0}.$$

2. Problema de autovalor generalizado

Solución no trivial requiere $\det(\mathbf{K} - \omega^2\mathbf{M}) = 0$. Equivalente:

$$\mathbf{K}\,\boldsymbol{\phi}_n = \omega_n^2\,\mathbf{M}\,\boldsymbol{\phi}_n, \qquad n = 1, \ldots, N_{\text{dof}}.$$

Autovalores $\lambda_n = \omega_n^2 \geq 0$; autovectores $\boldsymbol{\phi}_n$ son los modos de vibración.

3. Propiedades del par $(\mathbf{K}, \mathbf{M})$

- $\mathbf{K}$ simétrica, positiva (semi)definida tras Dirichlet.
- $\mathbf{M}$ simétrica, **positiva semidefinida**. Es definida cuando todo grado de libertad libre recibe masa (de algún elemento con $\rho > 0$ o de una masa de nudo). Si no, los grados de libertad sin masa se condensan (§6 bis).
- Fuentes de masa: la densidad del material (ADR 0008) y las **masas concentradas en nudos** (ADR 0024), que se suman a la diagonal de $\mathbf{M}$ igual con masa consistente que concentrada.
- Espectro real no negativo: $\omega_n \in \mathbb{R}_{\geq 0}$.

4. Salidas

- Frecuencias naturales ω_n (rad/s), $f_n = \omega_n/(2\pi)$ (Hz), períodos $T_n = 1/f_n$ (s).
- Modos $\boldsymbol{\phi}_n$ M-ortonormales: $\boldsymbol{\phi}_i^\top \mathbf{M}\,\boldsymbol{\phi}_j = \delta_{ij}$.
- Ordenación ascendente: $\omega_1 \leq \omega_2 \leq \ldots \leq \omega_{n_{\text{modes}}}$.

14.4.2 Formulación numérica

5. Reducción por Dirichlet

Por eliminación directa (ADR 0004) con operador $\mathbf{T} \in \mathbb{R}^{N_{\text{dof}} \times N_{\text{libre}}}$ que selecciona DOFs libres:

$$\mathbf{K}_{\text{red}} = \mathbf{T}^\top \mathbf{K}\, \mathbf{T}, \qquad \mathbf{M}_{\text{red}} = \mathbf{T}^\top \mathbf{M}\, \mathbf{T}.$$

El problema reducido $\mathbf{K}_{\text{red}}\, \boldsymbol{\phi}_{\text{red}} = \omega^2\, \mathbf{M}_{\text{red}}\, \boldsymbol{\phi}_{\text{red}}$ se resuelve sobre los DOFs libres. El término $\mathbf{g}_{\text{indep}}$ de restricciones lineales no homogéneas no aplica: los modos son solución del problema homogéneo (los modos de la estructura con apoyos a desplazamiento prescrito no nulo coinciden con los de apoyos nulos; el campo prescrito solo entra en la respuesta estática). Tras resolver, expansión $\boldsymbol{\phi} = \mathbf{T}\, \boldsymbol{\phi}_{\text{red}}$; en DOFs prescritos $\phi_i = 0$.

6. Algoritmo numérico: Lanczos con shift-invert

ARPACK vía `scipy.sparse.linalg.eigsh`:

$$\text{eigsh}(\mathbf{K}_{\text{red}},\ k = n_{\text{modes}},\ \mathbf{M} = \mathbf{M}_{\text{red}},\ \sigma,\ \text{which}=\text{'LM'},\ \text{tol}).$$

- **Shift-invert** transforma el problema en $\left(\mathbf{K}_{\text{red}} - \sigma\, \mathbf{M}_{\text{red}}\right)^{-1} \mathbf{M}_{\text{red}}\, \boldsymbol{\phi} = \mu\, \boldsymbol{\phi}$ con $\mu = 1/(\omega^2 - \sigma)$. Buscar `which="LM"` (mayor magnitud de μ) equivale a buscar autovalores cercanos al shift σ.
- $\sigma = 0$ por defecto: las frecuencias más bajas, que son las relevantes en ingeniería estructural.
- Internamente ARPACK factoriza $(\mathbf{K}_{\text{red}} - \sigma\, \mathbf{M}_{\text{red}})$ una sola vez y la reusa en cada iteración Lanczos.

6 bis. Grados de libertad sin masa: condensación estática exacta (ADR 0024)

Con masas sólo en algunos nudos o elementos de densidad cero, hay grados de libertad libres con diagonal de $\mathbf{M}_{\text{red}}$ nula (cero exacto; en una matriz semidefinida positiva eso implica fila y columna nulas). Con m los grados de libertad con masa y 0 los demás, la fila 0 no tiene inercia:

$$\boldsymbol{\phi}_0 = -\mathbf{K}_{00}^{-1}\, \mathbf{K}_{0m}\, \boldsymbol{\phi}_m, \qquad \hat{\mathbf{K}}\, \boldsymbol{\phi}_m = \omega^2\, \mathbf{M}_{mm}\, \boldsymbol{\phi}_m, \qquad \hat{\mathbf{K}} = \mathbf{K}_{mm} - \mathbf{K}_{m0}\, \mathbf{K}_{00}^{-1}\, \mathbf{K}_{0m}.$$

Es exacta, no una reducción de Guyan aproximada: los valores propios de $\hat{\mathbf{K}}$ son las frecuencias finitas del problema original, tantas como grados de libertad con masa; las demás son infinitas.

- Con 32 o menos grados de libertad con masa, o pidiendo casi todos los modos: $\hat{\mathbf{K}}$ densa y `eigh`.
- Con más: ARPACK sobre el problema condensado sin formar $\hat{\mathbf{K}}$. Con $\mathbf{A} = \mathbf{K} - \sigma\mathbf{M}$, el bloque mm de $\mathbf{A}^{-1}$ es $(\hat{\mathbf{K}} - \sigma\, \mathbf{M}_{mm})^{-1}$ (complemento de Schur), así que el operador de desplazamiento e inversión usa la factorización dispersa de $\mathbf{A}$ del modelo completo, con el producto interno de $\mathbf{M}_{mm}$, que es definida. Con $\sigma = 0$ y $\mathbf{K}$ singular se reintenta con desplazamiento negativo, como en el camino sin condensar.
- $\boldsymbol{\phi}_0$ se recupera con la factorización de $\mathbf{K}_{00}$; los modos expandidos son $\mathbf{M}$-ortonormales.
- Si no hay solución (más modos que grados de libertad con masa, ninguno con masa, grados de libertad sin masa sin rigidez que los fije, o masa singular más allá de ellos), se lanza `MasslessDofsError`, que los nombra como (nodo, dof).

7. Normalización

`eigsh` con **M** explícita devuelve autovectores M-ortonormales sin reescalado adicional. Signo arbitrario (autovector definido módulo signo); convención de signo se documenta solo si emerge como preferencia (no parte del contrato).

8. Post-procesado

- $\omega_n = \sqrt{\lambda_n}$ (clip a cero los autovalores ligeramente negativos por ruido numérico, $|\lambda| < 10^{-14}\,\lambda_{\max}$).
- $f_n = \omega_n/(2\pi)$, $T_n = 1/f_n$ (con $T_n = \infty$ si $\omega_n = 0$, modo de cuerpo rígido).
- Reordenación ascendente por ω_n.

14.4.3 Contrato de implementación

```yaml
name: ModalSolver
kind: solver
status: validated       # draft → implemented → validated

interface:
  yaml_type: modal      # solver: { type: modal }
  output: ModalResult   # campos: frequencies_hz, frequencies_rad, periods, modes,
      n_modes

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - bathe: "cap. 10-11 (autoproblemas generalizados)"
    - hughes: "cap. 10 (métodos para autoproblemas)"
  general_codes:
    - abaqus: "*FREQUENCY (Lanczos)"
    - ansys: "MODOPT,LANB"
    - opensees: "eigen"

parameters:
  - { name: n_modes,   type: int,   required: true,  desc: "Número de modos a calcular"
      }
  - { name: sigma,     type: float, required: false, default: 0.0,     desc: "Shift (
      rad²/s²) para shift-invert; 0 → frecuencias más bajas" }
  - { name: which,     type: str,   required: false, default: "LM",    desc: "
      Estrategia ARPACK: LM | SM | LA | SA | BE" }
  - { name: tolerance, type: float, required: false, default: 1.0e-9,  desc: "
      Tolerancia ARPACK sobre el residuo del autovalor" }
  - { name: lumping,   type: str,   required: false, default: "consistent", desc: "
      Discretización de masa: 'consistent' o 'lumped' (ADR 0009 fases 1 y 2, cerradas)"
      }
  - { name: linear_algebra, type: str, required: false, default: "auto", desc: "Backend
      para la factorización de (K - σM) interna a shift-invert (ADR 0003)" }

requirements:
  - "Todos los materiales del modelo declaran `density` (ADR 0008); `density = 0.0` es
      válida para elementos sin masa propia."
  - "Al menos un DOF libre con masa (de la densidad o de una masa de nudo, ADR 0024)."
```

```
  - "n_modes ≤ número de DOF libres con masa: sólo ellos tienen frecuencia finita. Los
      DOF sin masa se condensan estáticamente (§6 bis); si no se puede, `
      MasslessDofsError`."

conventions:
  units:        "heredadas del modelo (ADR 0008); ω en rad/s, f en Hz, T en s
      consistentes con kg/N/m o equivalentes."
  frequencies:  "ω y f expuestos simultáneamente; el usuario consulta cualquiera."
  modes:        "M-ortonormales Φᵀ( M Φ = I); signo arbitrario por modo."
  ordering:     "ascendente en ω: ω₁≤ω₂≤…≤ω      _n_modes."
  rigid_body:   "modos de cuerpo rígido aparecen con ω≈0; Tω= se reporta sin error."

out_of_scope:
  - "Modos complejos con amortiguamiento no proporcional."
  - "Análisis modal sobre estado deformado (modos de pequeña amplitud alrededor de
      configuración prestressed) - diferido."
  - "Masa lumped y factores de participación / masas efectivas - implementados (fase 2
      y fase 7 del ADR 0009, cerradas 2026-05-18); accesibles vía `lumping='lumped'` y
      `ResponseSpectrumSolver` respectivamente."

acceptance:
  verification:
    - name: barra_axial_empotrada_libre
      setup: |
        Barra 1D elástica empotrada en x=0, libre en x=L. Discretización con
        N=20 elementos Truss2D alineados con el eje x. E=210e9 Pa, ρ=7850 kg/m³,
        A=1e-4 m², L=1.0 m.
      expect: |
        Frecuencias propias analíticas (ondas axiales en barra empotrada-libre):ω
          _n = ((2n-1)π / (2L)) · sqrt(Eρ/),    n = 1, 2, 3, …
        Los primeros 3 modos calculados coinciden con la solución analítica.
      tol_rel: 0.01
    - name: viga_bernoulli_simplemente_apoyada
      setup: |
        Viga Bernoulli-Euler simplemente apoyada en sus dos extremos.
        Discretización con N=20 elementos Frame2DEuler alineados con el eje x.
        E=210e9 Pa, ρ=7850 kg/m³, A=1e-4 m², I=8.33e-10 ⁴m, L=1.0 m.
      expect: |
        Frecuencias propias analíticas (flexión transversal):ω
          _n = (πn/L)² · sqrt(E·I / ρ(·A)),    n = 1, 2, 3, …
        Los primeros 3 modos coinciden con la solución analítica.
      tol_rel: 0.02
    - name: oscilador_un_gdl_masa_de_nudo
      setup: |
        Columna en voladizo de Frame2DEuler sin masa propia (density = 0.0),
        con 1 y 4 elementos; masa m en el ux de la cabeza (ADR 0024).
        E = 2.0e11 Pa, I = 8.5e-5 ⁴m, L = 3.5 m, m = 2.5e4 kg.
      expect: |
        T = π2·√(m/k) con k = 3EI/L³, con masa consistente y concentrada.
        Los DOF sin masa del modo siguen la elástica de la carga en la punta:
        v(y)/v(L) = y²(3L - y)/(2L³), θ(L) = -3/(2L)·v(L). Con una inercia J
        en el rz de la cabeza, T = π2·√(J/(EI/L)).
      tol_rel: 1.0e-6
      ref: "tests/test_nodal_mass.py::TestOsciladorDeUnGradoDeLibertad"
    - name: edificio_cortante_tres_pisos
      setup: |
```

```
      Tres niveles, dos columnas biempotradas de Frame2DEuler sin masa por
      nivel (2 elementos cada una), vigas rígidas impuestas por cinemática
      (uy = rz = 0 en los nudos de piso y ux del nudo derecho igual al del
      izquierdo por restricción lineal) y la masa de cada piso repartida
      por mitades entre sus dos nudos.
    expect: |
      Periodos y modos iguales a los del problema 3×3 a mano, rigidez de
      entrepiso 2·12EI/h³: con pisos iguales, la forma cerradaω
      _j = 2·√(k/m)·sin((2-j1)π/14), φ_j(i) ∝ sin(i(2-j1)π/7); con pisos
      distintos, el 3×3 con sus masas y rigideces.
    tol_rel: 1.0e-6
    ref: "tests/test_nodal_mass.py::TestEdificioDeCortante"
specific:
  - name: ortonormalidad_de_masa
    setup: "Cualquier modelo bien formado tras resolver el problema modal."
    expect: "ΦᵀMΦ = I (n_modes × n_modes)."
    tol_abs: 1.0e-10
  - name: ortogonalidad_de_rigidez
    setup: "Mismo modelo."
    expect: "ΦᵀKΦ = diagω(²₁, …, ω²_n)."
    tol_rel: 1.0e-8
  - name: dirichlet_anula_modos_en_prescritos
    setup: "Barra empotrada-libre; nodo x=0 con ux=0."
    expect: "Componente del modo en el DOF prescrito = 0 exacto."
    tol_abs: 1.0e-14
  - name: mpc_tie_matches_merged_node
    setup: "ménsula de Frame2DEuler inclinada 30° partida en su nodo central y atada
        con u_s = u_m en ux, uy y rz, con el esclavo a cada lado del corte; masa
        consistente y concentrada"
    expect: "Frecuencias y modos expandidos Φ = T·Φ_red (también en el nodo duplicado
        ) = los del modelo de nodo fundido; ΦᵀΦM = I con la masa global del modelo
        atado"
    tol_rel: 1.0e-8
    ref: "tests/test_mpc_dynamic.py::test_modal_reproduce_el_nodo_fundido"
  - name: density_faltante_lanza_value_error
    setup: "Material sin density declarada en el modelo."
    expect: "ModalSolver.solve() lanza ValueError listando los materiales afectados (
        mismo patrón que assemble_self_weight, ADR 0008)."
  - name: masa_de_nudo_entrada_exacta
    setup: "Columna en voladizo de Frame2DEuler con densidad; masa m en el ux del
        nudo de la cabeza (ADR 0024); masa consistente y concentrada"
    expect: "M_uxux aumenta exactamente en m y ninguna otra entrada cambia; lo mismo
        con una inercia rotacional en rz"
    tol_abs: 0.0
    ref: "tests/test_nodal_mass.py::TestEnsamblajeDeLaMasaDeNudo"
  - name: dof_sin_masa_marco_de_un_nivel
    setup: "Marco de acero de un nivel (h = 3 m, L = 5 m, columnas 0.30 × 0.50 m de
        20 fibras y viga 0.40 × 0.70 m de 28 fibras, 4 elementos por miembro, 3
        puntos de Gauss) con columnas de densidad cero: 18 de 33 DOF libres sin masa"
    expect: "₁T = 0.03097 s y ₂T = 0.01295 s, iguales a la solución densa (valores
        propios de ⁻K¹M); residuo relativo de cada modo ‖φK −ω ²φ‖M‖/φ‖K ≤ 1e-8; con
        12 elementos por miembro (39 DOF con masa) el mismo criterio por el camino
        ARPACK, con masa consistente y concentrada"
    tol_rel: 1.0e-6
    ref: "tests/test_modal_massless_dofs.py::TestMarcoDeUnNivelSinMasaEnColumnas"
  - name: dof_sin_masa_error_tipado
```

```
      setup: "Voladizo sin masa propia con masa sólo en el ux de la cabeza y n_modes =
          2; modelo sin ninguna masa; dos barras alineadas cuyo nudo intermedio no
          tiene masa ni rigidez transversal"
      expect: "MasslessDofsError (ValueError) con causa too_many_modes, no_mass y
          massless_mechanism; el mensaje nombra los DOF sin masa como nodo <id> (<dof>)
          "
      ref: "tests/test_modal_massless_dofs.py::TestErrorTipado"
    - name: masa_en_todos_sin_cambios
      setup: "Modelos con masa en todos los DOF libres (toda la suite modal y dinámica
          previa)"
      expect: "Siguen el camino de siempre, sin condensación, y dan los mismos
          resultados que antes del ADR 0024"
      ref: "tests/test_modal_massless_dofs.py::TestMarcoDeUnNivelSinMasaEnColumnas::
          test_con_masa_en_todos_sigue_el_camino_de_siempre"

references:
  - "Bathe K.-J., Finite Element Procedures (2014), cap. 9 (matriz de masa consistente)
       y cap. 11 (algoritmos de autovalor)."
  - "Cook, Malkus & Plesha, Concepts and Applications of Finite Element Analysis, §
      -11.3§11.5."
  - "Hughes T. J. R., The Finite Element Method, cap. 7."
  - "Wilson E. L., Three-Dimensional Static and Dynamic Analysis of Structures, cap.
      10."
  - "Lehoucq, Sorensen, Yang - ARPACK Users' Guide (Lanczos con shift-invert)."
  - "ADR 0003 - Capa algebraica (factorización reutilizable de (K -σ M))."
  - "ADR 0008 - Material.density."
  - "ADR 0024 - Masas concentradas en nudos y análisis modal con grados de libertad sin
       masa."
  - "Bathe K.-J., Finite Element Procedures (2014), §10.3.1 (condensación estática en
      problemas de valores propios)."
  - "Chopra A. K., Dynamics of Structures, §9.3 (condensación estática de los grados de
       libertad sin masa)."
  - "ADR 0009 - Análisis modal y dinámico (este solver es la fase 1: el análisis modal
      estricto, problema generalizado K·φ = ω²·M·φ; el análisis dinámico transitorio
      entra en fases 3-7)."
```

14.4.4 Implementación

- **Archivo**: solidum/math/solvers/modal.py (clase `ModalSolver`, registrada vía `@SolverRegistry.register`).
- **Backend algebraico**: solidum/math/linalg/eigen.py (clase `EigenSolver` envolviendo `scipy.sparse.linalg.eigsh`).
- **Tipo de resultado**: `ModalResult` — dataclass frozen con `frequencies_rad`, `frequencies_hz`, `periods`, `modes`, `n_modes`, `converged`. Expone `free_vibration(M, u0, u0_dot, t)` para reconstruir analíticamente la respuesta temporal de vibración libre sin amortiguamiento por superposición modal — solución cerrada de `M·ü + K·u = 0` proyectada sobre los modos calculados (modos rígidos tratados aparte con evolución lineal `q_n(t) = a_n + b_n·t`).
- **Entrypoint público**: `solidum.run_modal` para uso programático; `solidum.run_yaml` despacha automáticamente a `run_modal` cuando el YAML declara `solver: type: ModalSolver`.

- **Pipeline**: `Assembler.assemble_system()` → `Assembler.assemble_mass_matrix()` → `Assembler.reduce_pair(K, M)` → `EigenSolver.solve(K_red, M_red, n_modes)` → expansión Φ = `T` · Φ_`red` → `ModalResult`.
- **Caché**: `Assembler` cachea `M_global` con el lumping usado; reusos posteriores del mismo análisis no recomputan.
- **Validación de density**: pre-chequeo agregado en `Assembler.assemble_mass_matrix` con el mismo patrón de ADR 0008 — `assemble_self_weight`.
- **Tests**:
- tests/test_modal.py — 13 tests cubriendo:
- `TestModalAxialBar`: barra empotrada-libre, 20 elementos Truss2D, frecuencias contra ω_`n` = `((2n-1)`π`/(2L))·`$\sqrt{}$`(E/`ρ`)` (tol 1 %).
- `TestModalSimplySupportedBeam`: viga biapoyada, 20 elementos Frame2DEuler, frecuencias contra ω_`n` = `(n`π`/L)`^{2}·$\sqrt{}$`(EI/(`ρ`A))` (tol 2 %).
- `TestModalAlgebraicProperties`: ortonormalidad $\Phi^\top$`MΦ = I` (atol 1e-10) y ortogonalidad $\Phi^\top$`KΦ = diag(`ω^2`)` (rtol 1e-8) en ambos modelos.
- `TestModalSolverContract`: errores agregados (`density` faltante, `run_modal` sin solver ni n_modes) y verificación de que `lumping="lumped"` corre tras ADR 0009 fase 2 (cerrada 2026-05-18).
- `TestModalYamlPipeline`: pipeline end-to-end desde YAML.
- **Notas de traducción**:
- El parámetro `linear_algebra` está en el constructor por coherencia con los demás solvers, pero en fase 1 no se usa: ARPACK gestiona internamente la factorización LU de `(K -` σ`M)` para shift-invert. Cuando se enchufe a la capa algebraica (ADR 0003), se podrá inyectar Cholesky o LDL$^\top$.
- El bloque `convergence:` del YAML (ADR 0007) se omite silenciosamente para `ModalSolver` igual que para `LinearSolver`: no hay iteración Newton ni norma de residuo configurable; la tolerancia es la propia de ARPACK (`tolerance:`).
- Matriz de masa consistente "translacional invariante rotacional" para `Truss*` y `Cable*` (`M = kron([[2,1],[1,2]]` · ρ`AL/6, I_dim)`). Para frames se ensambla la masa local axial + Hermitiana cúbica y se rota con `T^T` · `M_local` · `T`. Para Frame3D se incluye masa torsional con momento polar geométrico `Jp = Iy + Iz` (no la constante de Saint-Venant `J`, que rige la rigidez). Para sólidos 2D, integración por la cuadratura del propio elemento (Tri3 usa fórmula analítica exacta porque su cuadratura 1-punto sería insuficiente).
- Inercia rotacional propia de sección (ρ`I·L`) **incluida** en los DOFs rotacionales de Frame2D (Euler, Timoshenko, EulerCorot) y Frame3D — coherente con Timoshenko y con la rigidez `K_local` que tiene términos rotacionales puros (ADR 0009 §1).

14.4.5 Diálogo

- **2026-05-13** · Validado. Los 13 tests de `tests/test_modal.py` cubren los criterios de `acceptance` (verificación analítica + propiedades algebraicas + contrato de errores + pipeline YAML) y pasan con las tolerancias declaradas. Promovido `status: validated`.

- **2026-09-22** · Auditoría global: `K` singular con $\sigma = 0$ (estructura free-free) fallaba según la malla ("Factor is exactly singular" con 1, 2 o 4 elementos, éxito por redondeo con 3 o 10): reintento automático con un shift negativo relativo a tr(K)/tr(M). `ArpackNoConvergence` → `converged=False` con los modos obtenidos (`n_modes` reales); antes el flag estaba cableado a `True`. Modelos con n_free ≤ 32 o n_modes $\geq$ n_free $- 1$ usan `scipy.linalg.eigh` denso (M-ortonormal), así que un modelo de 1 GDL admite análisis modal.

- **2026-09-30** · Auditoría de las MPC (ADR 0004): las restricciones multipunto entran por `Assembler.reduce_pair` con el mismo `T` que los apoyos (`K_red = T`$^\top$`KT`, `M_red = T`$^\top$`MT`, Φ `= T·`Φ`_red`). Primer test con MPC: el oráculo del nodo fundido se cumple hasta el redondeo en frecuencias y modos, y los modos expandidos son M-ortonormales con la masa global (`tests/test_mpc_dynamic.py`). Nota sobre `ModalResult.free_vibration(M, u0, u0_dot, t)`: proyecta las condiciones iniciales en norma M (`a =` $\Phi^\top$`M·u0`), así que su resultado no depende de qué nodo se declaró esclavo; los solvers transitorios, en cambio, conservan las componentes libres y reconstruyen los esclavos como `T·u0[libres] + g`. Con condiciones iniciales coherentes con las MPC ambos caminos coinciden; con incoherentes difieren (los transitorios lo avisan en el log). Unificar las dos proyecciones cambiaría resultados con condiciones iniciales incoherentes: queda como decisión física pendiente del usuario. Sin cambio de código.

- **2026-09-30** · Revisión del paquete dinámico de la auditoría de las MPC: el parámetro `lumping` del contrato aún decía «solo 'consistent' en fase 1», pero `ModalSolver` admite `lumping='lumped'` desde el cierre de la fase 2 del ADR 0009 (lo ejercitan `tests/test_modal.py::test_lumped_runs_after_phase2` y el caso `mpc_tie_matches_merged_node` con masa concentrada). Corregida la descripción aquí y en el catálogo. Sin cambio de código.

- **2026-11-09** · ADR 0024. Masas concentradas en nudos (`Domain.add_nodal_mass`, bloque YAML `nodal_masses`), sumadas a la diagonal de **M** con los dos esquemas de masa. Con grados de libertad libres sin masa, ARPACK lanzaba `ArpackError -9999` sin causa y el camino denso exigía **M** definida; ahora se condensan estáticamente de forma exacta (§6 bis) y, si el problema no tiene solución, `MasslessDofsError` los nombra. Criterios nuevos: `oscilador_un_gdl_masa_de_nudo`, `edificio_cortante_tres_pisos`, `masa_de_nudo_entrada_exacta`, `dof_sin_masa_marco_de_un_nivel`, `dof_sin_masa_error_tipado`, `masa_en_todos_sin_cambios`. Medido: error relativo de los periodos de 1e-12 a 1e-14 contra la solución densa, residuos de 1e-11. `ResponseSpectrumSolver` lo hereda (masa total y masas efectivas con masas de piso en `tests/test_nodal_mass.py`).

14.5 NewmarkSolver

Orden de trabajo. La especificación física y la formulación numérica son patrimonio del usuario (revisa con detalle). La IA implementa y rellena las secciones marcadas.

14.5.1 Especificación física

0. Descripción general

Análisis dinámico transitorio **lineal** de una estructura discretizada por FEM: integración temporal directa de la ecuación semidiscreta de movimiento bajo cargas dependientes del tiempo. Hipótesis: linealidad geométrica y material (K constante), masa M constante (lagrangeano total), amortiguamiento Rayleigh proporcional `C = α·M + β·K`. Apoyos Dirichlet constantes en el tiempo. La no-linealidad (Newton dentro de cada paso) se difiere a la fase 4 del ADR 0009.

1. Ecuación de movimiento semidiscreta

Tras la discretización espacial por FEM, el problema continuo

$$\rho\,\ddot{\mathbf{u}} - \operatorname{div}\boldsymbol{\sigma} = \mathbf{b}(\mathbf{x}, t)$$

se reduce al sistema acoplado de ODEs:

$$\mathbf{M}\,\ddot{\mathbf{u}}(t) + \mathbf{C}\,\dot{\mathbf{u}}(t) + \mathbf{K}\,\mathbf{u}(t) = \mathbf{F}(t),$$

con condiciones iniciales $\mathbf{u}(0) = \mathbf{u}_0$, $\dot{\mathbf{u}}(0) = \dot{\mathbf{u}}_0$ y condiciones de contorno Dirichlet $\mathbf{u}_s(t) = \mathbf{g}$ constantes. Las restricciones multipunto $u_s = g_s + \sum_i \alpha_{si}\, u_{m_i}$ (ADR 0004) entran por el mismo operador $\mathbf{T}$ (§7).

De las condiciones iniciales sólo se conservan las componentes libres: los DOFs esclavos se reconstruyen como $\mathbf{u}_0 \leftarrow \mathbf{T}\,\mathbf{u}_0[\text{libres}] + \mathbf{g}$ y $\dot{\mathbf{u}}_0 \leftarrow \mathbf{T}\,\dot{\mathbf{u}}_0[\text{libres}]$. En un apoyo eso impone el valor prescrito; en el esclavo de una MPC, la combinación de sus maestros. Si el valor dado en un esclavo incumple su MPC, se descarta con un aviso de log y la respuesta depende de qué DOF se declaró esclavo; `ModalResult.free_vibration`, que proyecta en norma M, daría otra.

2. Amortiguamiento Rayleigh

$$\mathbf{C} = \alpha\,\mathbf{M} + \beta\,\mathbf{K},$$

con coeficientes calibrados a partir de dos pares modales (ξ_1, ω_1) y (ξ_2, ω_2):

$$\alpha = \frac{2\,\omega_1\omega_2\,(\xi_1\omega_2 - \xi_2\omega_1)}{\omega_2^2 - \omega_1^2}, \qquad \beta = \frac{2\,(\xi_2\omega_2 - \xi_1\omega_1)}{\omega_2^2 - \omega_1^2}.$$

La razón de amortiguamiento del modo n-ésimo resulta $\xi_n = \frac{1}{2}(\alpha/\omega_n + \beta\omega_n)$ — exacta en ω_1, ω_2 y suavemente sobreamortiguada fuera del rango. El usuario también puede pasar `(α, β)` directos.

3. Salidas

- $\mathbf{u}(t_k)$, $\dot{\mathbf{u}}(t_k)$, $\ddot{\mathbf{u}}(t_k)$ para cada paso temporal $k = 0, 1, \ldots, N_t$.
- Vector de instantes $t_k = k\,\Delta t$.
- Diagnósticos: α, β Rayleigh efectivos; número de pasos; tiempo de factorización vs tiempo de pasos.

14.5.2 Formulación numérica

4. Método de Newmark-β (a-form)

Familia paramétrica con (β, γ). Predictores:

$$\tilde{\mathbf{u}}_{n+1} = \mathbf{u}_n + \Delta t\,\dot{\mathbf{u}}_n + \tfrac{\Delta t^2}{2}(1 - 2\beta)\,\ddot{\mathbf{u}}_n,$$

$$\tilde{\dot{\mathbf{u}}}_{n+1} = \dot{\mathbf{u}}_n + \Delta t\,(1 - \gamma)\,\ddot{\mathbf{u}}_n.$$

Sistema efectivo en aceleraciones:

$$\left(\mathbf{M} + \gamma\Delta t\,\mathbf{C} + \beta\Delta t^2\,\mathbf{K}\right)\ddot{\mathbf{u}}_{n+1} \;=\; \mathbf{F}_{n+1} - \mathbf{C}\,\tilde{\dot{\mathbf{u}}}_{n+1} - \mathbf{K}\,\tilde{\mathbf{u}}_{n+1}.$$

Correctores:

$$\mathbf{u}_{n+1} = \tilde{\mathbf{u}}_{n+1} + \beta\Delta t^2\,\ddot{\mathbf{u}}_{n+1}, \qquad \dot{\mathbf{u}}_{n+1} = \tilde{\dot{\mathbf{u}}}_{n+1} + \gamma\Delta t\,\ddot{\mathbf{u}}_{n+1}.$$

5. Aceleración inicial

$\ddot{\mathbf{u}}_0$ se calcula consistentemente con la ecuación de movimiento en $t = 0$:

$$\mathbf{M}\,\ddot{\mathbf{u}}_0 = \mathbf{F}(0) - \mathbf{C}\,\dot{\mathbf{u}}_0 - \mathbf{K}\,\mathbf{u}_0.$$

6. Parámetros canónicos

Esquema	β	γ	**Propiedades**
Average acceleration (default)	1/4	1/2	Incondicionalmente estable, sin amortiguamiento numérico, exactitud $O(\Delta t^2)$
Linear acceleration	1/6	1/2	Condicionalmente estable ($\Delta t \le 2\sqrt{3}/\omega_{\text{máx}}$), $O(\Delta t^2)$
Fox-Goodwin	1/12	1/2	Condicionalmente estable, $O(\Delta t^4)$
Diferencias centradas	0	1/2	Explícito; condicionalmente estable ($\Delta t \le 2/\omega_{\text{máx}}$), $O(\Delta t^2)$

7. Imposición de Dirichlet con apoyos constantes

Mismo mecanismo que estática (ADR 0004 fase 1) con operador $\mathbf{T}$ que selecciona DOFs libres y $\mathbf{g}$ que recoge los valores prescritos: $\mathbf{u} = \mathbf{T}\,\mathbf{u}_{\text{free}} + \mathbf{g}$. Como $\mathbf{g}$ es constante, $\dot{\mathbf{u}} = \mathbf{T}\,\dot{\mathbf{u}}_{\text{free}}$ y $\ddot{\mathbf{u}} = \mathbf{T}\,\ddot{\mathbf{u}}_{\text{free}}$. El sistema efectivo se proyecta a DOFs libres:

$$\mathbf{A}_{\text{eff,red}}\,\ddot{\mathbf{u}}_{\text{free},n+1} = \mathbf{T}^{\top}\left(\mathbf{F}_{n+1} - \mathbf{K}\,\mathbf{g}\right) - \mathbf{C}_{\text{red}}\,\tilde{\dot{\mathbf{u}}}_{\text{free}} - \mathbf{K}_{\text{red}}\,\tilde{\mathbf{u}}_{\text{free}},$$

donde $\mathbf{A}_{\text{eff,red}} = \mathbf{M}_{\text{red}} + \gamma\Delta t\,\mathbf{C}_{\text{red}} + \beta\Delta t^2\,\mathbf{K}_{\text{red}}$. El término $\mathbf{T}^{\top}\,\mathbf{K}\,\mathbf{g}$ se precalcula una vez. La expansión final a globales es $\mathbf{u}_n = \mathbf{T}\,\mathbf{u}_{\text{free},n} + \mathbf{g}$.

Con restricciones multipunto las filas esclavas de $\mathbf{T}$ llevan los coeficientes α_{si} de sus maestros y el mismo esquema vale sin cambios ($\mathbf{M}_{\text{red}} = \mathbf{T}^{\top}\mathbf{M}\,\mathbf{T}$, $\mathbf{C}_{\text{red}} = \mathbf{T}^{\top}\mathbf{C}\,\mathbf{T}$); verificado con el oráculo del nodo fundido en `tests/test_mpc_dynamic.py`.

8. Factorización reutilizable

Con Δt constante y problema lineal, $\mathbf{A}_{\text{eff,red}}$ es constante. Se factoriza una vez al inicio (ADR 0003 — `FactorizedSolver`) y cada paso es una resolución triangular barata. Para N_t pasos sobre N_{libre} DOFs: coste $\sim O(\text{factor})_{\text{una vez}} + N_t \cdot O(N_{\text{libre}})_{\text{por paso}}$.

14.5.3 Contrato de implementación

```yaml
name: NewmarkSolver
kind: solver
status: validated       # draft → implemented → validated

interface:
  yaml_type: NewmarkSolver
  output: TransientResult   # campos: t_history, u_history, udot_history, uddot_history

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 9 (familia de Newmark)"
    - bathe: "§9.2 (método de Newmark)"
  general_codes:
    - opensees: "integrator Newmark"
    - ansys: "TRNOPT,FULL (Newmark)"

parameters:
  - { name: t_end,     type: float, required: true,                desc: "Tiempo final
      del análisis (s)" }
  - { name: dt,        type: float, required: true,                desc: "Paso temporal
       constante (s)" }
  - { name: beta,      type: float, required: false, default: 0.25, desc: "Parámetro β
      de Newmark (default: average acceleration)" }
  - { name: gamma,     type: float, required: false, default: 0.5,  desc: "Parámetro γ
      de Newmark" }
  - { name: rayleigh,  type: dict,  required: false, default: null, desc: "
      Amortiguamiento Rayleigh: {alpha, beta} directos o {xi1, omega1, xi2, omega2}
      para calibración modal" }
  - { name: u0,        type: ndarray, required: false, default: null, desc: "
      Desplazamiento inicial (ndof,); cero por defecto" }
```

```
  - { name: u0_dot,    type: ndarray, required: false, default: null, desc: "Velocidad
      inicial (ndof,); cero por defecto" }
  - { name: F_func,    type: callable, required: false, default: null, desc: "Callback
      Python F_func(t) -> ndarray (ndof,); cero por defecto" }
  - { name: linear_algebra, type: str, required: false, default: "auto", desc: "Backend
       para factorizar A_eff (ADR 0003)" }
  - { name: lumping,   type: str, required: false, default: "consistent", desc: "
      Discretización de masa; solo 'consistent' en esta fase" }

requirements:
  - "Todos los materiales del modelo declaran `density > 0` (ADR 0008)."
  - "K lineal y constante (hipótesis de pequeñas deformaciones, material elástico)."
  - "Apoyos Dirichlet constantes en el tiempo."

conventions:
  units:   "heredadas del modelo (ADR 0008); t en s, ω en rad/s, F en N consistente con
       kg/N/m."
  damping: "Rayleigh proporcional: C = α·M + β·K. Si se pasan ξ₁ω₁(,)ξ₂ω₂,(,), αβ(,) se
       derivan automáticamente."
  stability: |
    Default β(=1/4, γ=1/2) es incondicionalmente estable. Para β=0 (diferencias
    centradas) o β=1/6 (lineal) el usuario es responsable de que Δt ≤ω 2/_max;
    el solver no comprueba esta cota automáticamente.

out_of_scope:
  - "Apoyos dependientes del tiempo (multi-support excitation sísmica) - diferido."
  - "Δt adaptativo - diferido."
  - "Generalizedα-, Bossakα- - diferidos (variantes adicionales de la familia con
      amortiguamiento numérico)."
  - "No-linealidad y disipación numérica controlada - implementados como variantes en
      clases hermanas (`NewtonNewmarkSolver`, `HHTSolver`, `NewtonHHTSolver`); fases 4
      y 3-bis del ADR 0009, cerradas."

acceptance:
  verification:
    - name: oscilador_1gdl_no_amortiguado
      setup: |
        Sistema 1 GDL: M=1, Kω=²=25 ω(=5 rad/s), C=0. Condiciones iniciales₀
        u=1, ̇₀u=0. F(t)=0. Δtω=·T/100 (100 pasos por período). Integrar
        durante 4 períodos.
      expect: |
        u(t) = cosω(·t) exacto. Error máximo en u durante el intervalo
        < 1% β(=1/4 tiene error de período OΔ(t²) bajo control con 100
        pasos por período).
      tol_rel: 0.01
    - name: oscilador_1gdl_amortiguado_rayleigh
      setup: |
        Sistema 1 GDL: M=1, K=25, C=2·ξ·ω·M con ξ=0.05 (sub-amortiguado).₀
        u=1, ̇₀u=0, F=0. Δt = T/100, t_end = 4·T. Construir C vía
        Rayleigh con un único par ξ(, ω).
      expect: |
        u(t) = eξω^(-t)·(cosω(_d·t) + ξωω(/_d)·sinω(_d·t)) conω
        _d = ω·√ξ(1-²). Error máximo < 2 %.
      tol_rel: 0.02
    - name: viga_voladizo_carga_subita_modal_vs_newmark
      setup: |
```

```
        Voladizo Frame2DEuler (20 elementos) con carga puntual aplicada en
        el extremo en t=0 como step function. Δt = ₁T/40 (₁T período del
        primer modo), t_end = 2·₁T. Sin amortiguamiento.
      expect: |
        La reconstrucción modal de la respuesta (superposición de primeros
        5 modos forzados por F(t)=H(t)·₀F con solución analítica conocida)
        coincide con Newmark a < 5 %.
      tol_rel: 0.05
    - name: orden_de_convergencia
      setup: |
        Oscilador 1 GDL no amortiguado integrado con β=1/4, γ=1/2 aΔ
        t, Δt/2, Δt/4. Calcular el error máximo en u contra solución
        analítica.
      expect: |
        El cociente de errores eΔ(t)/eΔ(t/2) tiende a 4 (orden 2 exacto).
      tol_rel: 0.5  # tolerancia sobre el cociente: 4 ± 2
  specific:
    - name: rayleigh_calibration
      setup: "Dados ξ₁(=0.02, ω₁=10), ξ₂(=0.05, ω₂=100), calibrar α(, β)."
      expect: |α
         = 2·ω₁·ω₂·ξ₁(·ω₂ -ξ₂ ·ω₁)ω₂/(² -ω₁ ²) ≈ 0.303,β
         = 2·ξ₂(·ω₂ -ξ₁ ·ω₁)ω₂/(² -ω₁ ²) ≈ 9.6e-4.
        Y verificar ξω(_n) = αω(/_n + β·ω_n)/2 reproduce ξ₁ en ω₁ y ξ₂ en ω₂.
      tol_rel: 1.0e-10

references:
  - "Newmark N. M. (1959), A method of computation for structural dynamics, J. Eng.
     Mech. Div. ASCE."
  - "Bathe K.-J., Finite Element Procedures (2014), cap. 9 (matriz de masa) y cap. 9.4
     (Newmark)."
  - "Hughes T. J. R., The Finite Element Method, cap. 9 (algoritmos de integración
     temporal)."
  - "Chopra A. K., Dynamics of Structures, cap. 5 (1 GDL) y cap. 15 (MDOF Newmark)."
  - "ADR 0003 - Capa algebraica (factorización reutilizable de A_eff)."
  - "ADR 0007 - Convergencia y tolerancias."
  - "ADR 0009 - Análisis modal y dinámico (este solver es la fase 3)."
```

14.5.4 Implementación

- **Archivo**: solidum/math/solvers/newmark.py (clase `NewmarkSolver`, registrada vía `@SolverRegistry.register`).
- **Amortiguamiento**: solidum/math/damping.py (`rayleigh_from_modes`, `rayleigh_xi`).
- **Tipo de resultado**: `TransientResult` — dataclass frozen con `t_history`, `u_history`, `udot_history`, `uddot_history`, `n_steps`, `alpha_rayleigh`, `beta_rayleigh`, `converged`.
- **Entrypoint público**: `solidum.run_transient` para uso programático; `solidum.run_yaml` despacha automáticamente cuando el YAML declara `solver: type: NewmarkSolver`.
- **Pipeline**: `Assembler.assemble_system()` → `Assembler.assemble_mass_matrix()` → `C = α·M + β·K` → reducción simultánea por Dirichlet (operador `T`) → cálculo de $\ddot{u}_0$ → factorización única de `A_eff_red = M_red + γΔt·C_red + βΔt²·K_red` → bucle de pasos con resolución triangular reutilizada.

- **Tests**:
- tests/test_newmark.py — 14 tests:
- `TestRayleighCalibration`: (α, β) reproduce ξ objetivo en ω_1, ω_2; errores agregados (ω iguales, no positivos).
- `TestNewmark1DofUndamped`: vibración libre `u(t) = cos(ωt)` a 1 %; estado inicial preservado.
- `TestNewmark1DofDamped`: respuesta sub-amortiguada `e^(-ξωt)·(cos(ω_d·t) + ...)` a 2 %; coeficientes Rayleigh propagados al `TransientResult`.
- `TestNewmark1DofStepResponse`: `u(t) = (F0/K)(1 - cos(ωt))` a 0.1 %.
- `TestNewmarkConvergenceOrder`: cociente de errores al refinar $\Delta t \in T/40, T/80, T/160$ cae en `[3.5, 4.5]` → orden 2 confirmado.
- `TestNewmarkContract`: validaciones de constructor y `run_transient`.
- `TestNewmarkYamlPipeline`: end-to-end desde YAML.
- **Notas de traducción**:
- **Factorización única**: con Δt constante y problema lineal, `A_eff_red` es invariante; se factoriza al inicio (`A_solver.factorize(A_eff)`) y cada paso es una sustitución triangular barata (ADR 0003 — `FactorizedSolver`).
- `reduce_pair` **vs** `reduce` **triple**: en el problema modal `reduce_pair(K, M)` basta porque `g` no entra en autovalores. En Newmark se manejan tres matrices (`K, M, C`) y un término constante `F_dir = T^⊤K·g` para apoyos no nulos; se hace explícitamente en `NewmarkSolver.solve()` sin un helper unificado (no aporta valor centralizarlo con un solo consumidor).
- **Aceleración inicial consistente**: $M\ddot{u}_0 = F(0) - C\dot{u}_0 - Ku_0$ resuelve un sistema lineal aparte; se usa el dispatcher de la capa algebraica sobre `M_red` (siempre SPD si la masa es consistente y la densidad estrictamente positiva).
- `F_func` **opcional**: si es `None`, se trata como vector cero (vibración libre). El callback se invoca una vez por paso con `t_{n+1}`; el usuario es responsable de la coherencia temporal (no se reusan valores entre pasos).
- **El bloque** `convergence:` **del YAML** (ADR 0007) se omite para `NewmarkSolver` igual que para `LinearSolver` y `ModalSolver`: la fase 3 es lineal y no hay iteración Newton interna; cuando entre la fase 4 (Newton dentro de cada paso), el bloque se reincorporará.

14.5.5 Diálogo

- **2026-05-13** · Validado. Los 14 tests de `tests/test_newmark.py` cubren `acceptance.verification` (osciladores 1 GDL no amortiguado / amortiguado / step / orden de convergencia) y `acceptance.specific` (calibración Rayleigh). Promovido `status: validated`.

- **2026-09-22** · Auditoría global: masa reducida singular (DOF libre sin masa, `density = 0.0`) → `ValueError` que nombra la causa, en vez de un historial de `NaN` con un simple warning de scipy. `number_of_steps` evita el paso espurio cuando `t_end = n·dt` acumula redondeo (t_final = t_end + dt en el 5 % de los pares).
- **2026-09-22** · `record_internal_forces=True` (deuda #15): registra `elem.internal_forces(u_k)` en cada paso con el estado interno de ese paso (`TransientResult.element_forces_history`; `internal_forces_history(domain)` lo devuelve). Opt-in por coste; sin él, reconstrucción lazy con el estado final y aviso. Verificado contra análisis truncados en `t_k` con plasticidad (`tests/test_transient_forces_history.py`). Además `assemble_system` ya no deja el estado trial evaluado en `u = 0` (deuda #18).
- **2026-09-30** · Auditoría de las MPC (ADR 0004): primer test propio con restricciones multipunto (antes Newmark sólo servía de referencia en el test de periodicidad de los solvers de Newton). El oráculo del nodo fundido —ménsula de Frame2DEuler inclinada 30° partida y atada con u_s = u_m, esclavo a cada lado, masa consistente y concentrada con Rayleigh— se cumple hasta el redondeo en u, u̇, ü y en la historia de N, V, M, también con la carga aplicada en el esclavo; con un nodo colgante de dos maestros y masa concentrada se conserva Σm·ü = ΣF a escala de 1 m y de 10 µm (`tests/test_mpc_dynamic.py`). Documentada la proyección de las condiciones iniciales (§1): los esclavos se reconstruyen como `T·u0[libres] + g` y, si el valor dado incumple una MPC con maestros, el solver lo avisa en el log (las filas Dirichlet no avisan; su proyección tiene test propio). Sin cambio de física.

14.6 NewtonHHTSolver

Variante de `NewtonNewmarkSolver` según la regla de variantes (Reglas.md §4). Aplica el esquema temporal HHT-α (Hilber-Hughes-Taylor 1977) al residuo dinámico no lineal: amortiguamiento numérico controlado en altas frecuencias dentro del bucle Newton de cada paso. Reusa todas las decisiones arquitecturales del ADR 0009 (esquema temporal, contrato `PIPELINE_KIND`, telemetría tipada, line search opt-in) y de `HHTSolver` (formulación temporal, autoderivación de β/γ).
***Sin ADR nuevo**. Documenta solo lo que cambia respecto a los dos padres.*

14.6.1 Qué cambia respecto a `NewtonNewmarkSolver`

`NewtonNewmarkSolver` integra `M·ü + C·u̇ + F_int(u) = F(t)` con Newmark-β clásico (β=1/4, γ=1/2 default, sin amortiguamiento numérico). Cada paso abre un bucle Newton porque `F_int(u)` es no lineal en `u` (plasticidad, daño, corotacional).

`NewtonHHTSolver` reemplaza el residuo Newmark por el residuo HHT-α: las **fuerzas internas, viscosas y externas** se evalúan en un instante intermedio `t_{n+1-α}` mientras la **fuerza inercial** se mantiene en `t_{n+1}`. El bucle Newton se ejecuta sobre este residuo modificado.

Residuo dinámico HHT-α no lineal

$$\mathbf{R}(\ddot{\mathbf{u}}_{n+1}) \;=\; (1{+}\alpha)\,\mathbf{F}_{n+1}{-}\alpha\,\mathbf{F}_{n}{-}\big[(1{+}\alpha)\,\mathbf{F}_{\text{int}}(\mathbf{u}_{n+1}){-}\alpha\,\mathbf{F}_{\text{int}}(\mathbf{u}_{n})\big]{-}\big[(1{+}\alpha)\,\mathbf{C}\,\dot{\mathbf{u}}_{n+1}{-}\alpha\,\mathbf{C}\,\dot{\mathbf{u}}_{n}\big]{-}\mathbf{M}\,\ddot{\mathbf{u}}_{n+1}$$

El término no lineal $\mathbf{F}_{\text{int}}(\mathbf{u}_{n+1})$ depende del estado actualizado por los correctores Newmark, así que el residuo es implícito en $\ddot{\mathbf{u}}_{n+1}$ y requiere Newton.

Jacobiano efectivo

$$\mathbf{J} \;=\; \mathbf{M} + (1+\alpha)\,\gamma\,\Delta t\,\mathbf{C} + (1+\alpha)\,\beta\,\Delta t^2\,\mathbf{K}_t$$

donde $\mathbf{K}_t = \partial\mathbf{F}_{\text{int}}/\partial\mathbf{u}$ es la tangente material/geométrica reensamblada en cada iteración Newton.

Parámetros heredados de `HHTSolver`

- Rango admisible $\alpha \in [-1/3, 0]$. Valores fuera lanzan `ValueError`.
- β y γ auto-derivados desde α:

$$\beta = \frac{(1-\alpha)^2}{4}, \qquad \gamma = \frac{1-2\alpha}{2}$$

- Override explícito de β/γ permitido pero genera `RuntimeWarning`

(combinaciones distintas pueden romper estabilidad incondicional u orden 2).

- Radio espectral en alta frecuencia: $\rho_\infty = (1+\alpha)/(1-\alpha)$.

Disipación selectiva en modos con $\omega\Delta t > \pi$.

Comportamiento que NO cambia

Todo lo heredado de `NewtonNewmarkSolver`:

- Convergencia dual fuerza + desplazamiento (ADR 0007) con autocalibración.
- Line search GLL opt-in (ADR 0011, default `False`).
- Diagnóstico tipado de divergencia (`SolverDivergedError` y subclases).
- Rayleigh con $\mathbf{K}_0$ constante (matriz de rigidez inicial).
- Commit/rollback del estado material por paso.
- Detección de oscilación por ventana móvil.
- Despacho algebraico ADR 0003 con fallback Cholesky→LU.
- Lumping de masa (consistent / lumped) ADR 0009 fase 2.

Recuperación de `NewtonNewmark` con $\alpha = 0$

Con $\alpha = 0$: $\beta = 1/4$, $\gamma = 1/2$, el residuo HHT colapsa al de Newmark estándar. Verificado en tests: `tests/test_hht.py::TestNewtonHHT::test_alpha_zero_recovers_NewtonNewmark`.

Recuperación de `HHTSolver` con materiales lineales

Con materiales lineales ($\mathbf{F}_{\text{int}}(\mathbf{u}) = \mathbf{K}\,\mathbf{u}$), el residuo de Newton se anula en una iteración y el solver reproduce exactamente `HHTSolver`. Tangente constante = rigidez secante. Verificado: `tests/test_hht.py::TestNewtonHHT::test_linear_material_recovers_HHT_lineal`.

14.6.2 Contrato de implementación

```yaml
name: NewtonHHTSolver
kind: solver
status: validated

extends: NewtonNewmarkSolver        # variante por Reglas §4
also_references: HHTSolver          # formulación temporal heredada

interface:
  yaml_type: newton_hht
  output: TransientResult
  pipeline_kind: transient

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 9 (método α de Hilber-Hughes-Taylor)"
    - de_borst_crisfield: "§5.4 (integración implícita en dinámica no lineal)"
  general_codes:
    - abaqus: "*DYNAMIC no lineal (HHTα- + Newton)"
    - opensees: "integrator HHT + algorithm Newton"
```

```
parameters:
  # Solo se listan los parámetros propios o con default distinto al padre.
  # Resto heredados de NewtonNewmarkSolver.
  - { name: alpha, type: float, required: false, default: -0.05,
      desc: "Parámetro HHTα- ∈ [-1/3, 0]. α=0 recupera NewtonNewmark; α=-1/3 disipación
           máxima preservando orden 2" }
  - { name: beta,  type: float, required: false, default: null,
      desc: "Override del β autoderivado (no recomendado - emite RuntimeWarning)" }
  - { name: gamma, type: float, required: false, default: null,
      desc: "Override del γ autoderivado (no recomendado - emite RuntimeWarning)" }
  # heredados de NewtonNewmarkSolver:
  #   t_end, dt, convergence, max_iter, freeze_tangent_after_iter,
  #   line_search, rayleigh, u0, u0_dot, F_func, linear_algebra, lumping

requirements:
  - "Modelo dinámico con no-linealidad de material o geometría"
  - "M con density > 0 en todos los materiales activos (ADR 0008)"
  - "Δt razonable (incondicionalmente estable para sistemas lineales, no garantizado
     para fuertes no-linealidades)"

conventions:
  units: "heredadas del modelo (ADR 0008)"
  stability: "incondicionalmente estable en problemas lineales para α∈  [-1/3, 0];
     condicionalmente para no lineales severos"
  disipacion: "ρω_ = α(1+)α/(1-); selectiva en alta frecuencia"

out_of_scope:
  - "α fuera de [-1/3, 0] (pierde estabilidad u orden 2)"
  - "Generalizedα- / Bossak (variantes con disipación distinta) - diferidos"
  - "Análisis estático ⇒ usar NonlinearSolver"

acceptance:
  verification:
    - name: alpha_cero_recupera_newton_newmark
      setup: "mismo problema con NewtonNewmarkSolver y NewtonHHTSolverα(=0)"
      expect: "respuesta idéntica paso a paso"
      tol_rel: 1.0e-12

    - name: material_lineal_recupera_hht
      setup: "mismo problema con HHTSolver y NewtonHHTSolver, ambos α-=0.1"
      expect: "respuesta idéntica; Newton converge en 1 iteración por paso"
      tol_rel: 1.0e-12

    - name: disipacion_modo_alta_frecuencia
      setup: "respuesta libre con modos altos espurios excitados; α-=0.1"
      expect: "amplitud de modos espurios decae ρ~ω_ por periodo"
      tol_rel: 0.05

    - name: respuesta_plastica_dinamica
      setup: "barra Truss2D con Elastoplastic1D, carga dinámica que cruza el yield"
      expect: "estado plástico converged paso a paso; histéresis dinámica coherente"
      tol_rel: 1.0e-3

    - name: rechazo_alpha_fuera_de_rango
      setup: "NewtonHHTSolver(alpha=-0.5) o (alpha=0.1)"
      expect: "ValueError con mensaje claro"
```

```
    - name: warning_override_beta_gamma
      setup: "NewtonHHTSolver(alpha=-0.1, beta=0.3)"
      expect: "RuntimeWarning emitido al construir"

  specific:
    - name: tangente_singular_dispara_excepcion_tipada
      setup: "modelo con material que produce K_t singular en algún paso"
      expect: "raise SingularTangentError con métricas estructuradas"

references:
  - "Hilber H.M., Hughes T.J.R., Taylor R.L. (1977). Improved numerical dissipation for
      time integration algorithms in structural dynamics. Earthquake Eng. Struct. Dyn.
      5, 283-292."
  - "Hughes T.J.R. (2000). The Finite Element Method. Dover. §9.3."
  - "Newmark N.M. (1959). A method of computation for structural dynamics. ASCE 85(EM3)
     , 67-94."
  - "ADR 0009 (fase 4: HHTα- - formulación temporal en residuo no lineal)."
  - "ADR 0011 (excepciones tipadas, line search opt-in)."
  - "Spec padre: HHTSolver.md (formulación temporal lineal)."
  - "Spec padre: NewtonNewmarkSolver.md (Newton dentro de Newmark no lineal)."
```

14.6.3 Implementación

- **Archivo**: solidum/math/solvers/newmark.py (clase `NewtonHHTSolver`).
- **Clase**: hereda de `NewtonNewmarkSolver`; añade el atributo `self.alpha` y reemplaza el cómputo del residuo y del jacobiano en `solve()`.
- **Auto-derivación** de β y γ vía `_hht_autoderive_beta_gamma(alpha)` (función helper compartida con `HHTSolver`).
- **Validación temprana** en `__init__`: `alpha` $\in$ `[-1/3, 0]`, warning si override de β/γ.
- **Entrypoint público**: `solidum.run_transient(model, solver="newton_hht", ...)` (despacho por `PIPELINE_KIND = "transient"`).
- **Tests**:
- tests/test_hht.py (clase `TestNewtonHHT`) — incluye recuperación de `NewtonNewmark` con α=0, recuperación de `HHTSolver` con materiales lineales, respuesta plástica dinámica.

14.6.4 Diálogo

- **2026-05-19** · Spec corta tipo extensión creada para cerrar el hueco H-5.3 (variante de `HHTSolver`/`NewtonNewmarkSolver` por Reglas §4). Sin ADR nuevo: reúsa ADR 0009 (formulación temporal HHT) y ADR 0011 (Newton + diagnóstico). El comportamiento no se modifica.
- **2026-09-22** · Auditoría global: misma reestructuración que `NewtonNewmarkSolver` (un ensamblaje por iteración, `g` antes de ensamblar, MPC vía `T`, `prepare_all_steps`); el line search opcional evalúa ahora el residuo HHT-α propio, no el de Newmark.

- **2026-09-22** · `record_internal_forces=True` (deuda #15): registra `elem.internal_forces(u_k)` en cada paso con el estado interno de ese paso (`TransientResult.element_forces_history`; `internal_forces_history(domain)` lo devuelve). Opt-in por coste; sin él, reconstrucción lazy con el estado final y aviso. Verificado contra análisis truncados en `t_k` con plasticidad (`tests/test_transient_forces_history.py`). Además `assemble_system` ya no deja el estado trial evaluado en `u = 0` (deuda #18).
- **2026-09-22** · ADR 0015: el bucle de Newton pasa al corrector compartido `NewtonCorrector`; este solver aporta su problema por paso (`_DynamicNewtonProblem` con residuo HHT-α y factor `1+`α en el jacobiano; `F_int_n` se toma del estado del ensamblaje convergido) y conserva su control de paso. Sin cambio de formulación ni de resultados (suite completa sin tocar ningún valor esperado).
- **2026-09-23** · Revisión documental: las referencias a `tests/test_dynamic_nonlinear_hht.py` y a sus dos tests apuntaban a nombres que ya no existen; la cobertura vive en `tests/test_hht.py::TestNewtonHHT` (`test_alpha_zero_recovers_NewtonNewmark`, `test_linear_material_recovers_HHT_lineal`, `test_plastic_oscillator_converges_with_alpha_negative`).
- **2026-09-30** · Auditoría de las MPC (ADR 0004): oráculo del nodo fundido añadido a la cobertura (`tests/test_mpc_dynamic.py`) y aviso de log, como en `NewtonNewmarkSolver`, cuando `u0` o `u0_dot` incumplen una MPC con maestros (el esclavo se reconstruye como `T·u0[libres] + g`). Sin cambio de física.

14.7 NewtonNewmarkSolver

*Variante de `NewmarkSolver` según la regla de variantes (Reglas.md §4). Documenta **solo lo que cambia** respecto al padre. Implementa la **fase 4 del ADR 0009**; reusa todas las decisiones arquitecturales del ADR padre — sin ADR nuevo.*

14.7.1 Qué cambia respecto a `NewmarkSolver`

`NewmarkSolver` integra `M·ü + C·u̇ + K·u = F(t)` asumiendo **K constante**: factoriza `A_eff = M + γΔt·C + βΔt²·K` una vez y cada paso temporal es una resolución triangular barata.

`NewtonNewmarkSolver` resuelve el mismo problema cuando **la rigidez no es constante**: hay materiales con historia (plasticidad, daño) o no linealidad geométrica. En cada paso temporal se introduce un bucle de **Newton-Raphson** sobre el residuo dinámico, hasta cumplir el criterio de convergencia.

Residuo dinámico en cada paso

En el instante t_{n+1} las incógnitas son $(\mathbf{u}_{n+1}, \dot{\mathbf{u}}_{n+1}, \ddot{\mathbf{u}}_{n+1})$ ligadas por los correctores Newmark:

$$\mathbf{u}_{n+1} = \tilde{\mathbf{u}}_{n+1} + \beta \Delta t^2 \, \ddot{\mathbf{u}}_{n+1}, \qquad \dot{\mathbf{u}}_{n+1} = \tilde{\dot{\mathbf{u}}}_{n+1} + \gamma \Delta t \, \ddot{\mathbf{u}}_{n+1}$$

(con predictores idénticos a la fase 3). El residuo dinámico es:

$$\mathbf{R}(\ddot{\mathbf{u}}_{n+1}) \;=\; \mathbf{F}_{\text{ext}}(t_{n+1}) \;-\; \mathbf{F}_{\text{int}}(\mathbf{u}_{n+1}) \;-\; \mathbf{C}\,\dot{\mathbf{u}}_{n+1} \;-\; \mathbf{M}\,\ddot{\mathbf{u}}_{n+1} \;-\; \mathbf{F}_{\text{dir}}$$

donde $\mathbf{F}_{\text{int}}(\mathbf{u})$ es el vector de fuerzas internas no lineales (plástico, dañado, etc.) que el `Assembler.assemble_non_linear_system` ya calcula para los solvers estáticos (ADR 0003 + ADR 0006 + ADR 0008).

Jacobiano dinámico tangente

Derivando $\mathbf{R}$ respecto a $\ddot{\mathbf{u}}_{n+1}$ usando los correctores:

$$\mathbf{J} \;=\; -\frac{\partial \mathbf{R}}{\partial \ddot{\mathbf{u}}_{n+1}} \;=\; \mathbf{M} \;+\; \gamma \Delta t \, \mathbf{C} \;+\; \beta \Delta t^2 \, \mathbf{K}_{\text{t}}$$

donde $\mathbf{K}_{\text{t}} = \partial \mathbf{F}_{\text{int}} / \partial \mathbf{u}$ es la rigidez tangente del estado corriente. La iteración es:

$$\mathbf{J}\,\delta\ddot{\mathbf{u}} = \mathbf{R}, \qquad \ddot{\mathbf{u}}_{n+1}^{(k+1)} = \ddot{\mathbf{u}}_{n+1}^{(k)} + \delta\ddot{\mathbf{u}}$$

Los desplazamientos y velocidades se actualizan con los mismos correctores Newmark tras cada update de $\ddot{\mathbf{u}}_{n+1}$.

Amortiguamiento Rayleigh — heredado, constante en el tiempo

$\mathbf{C} = \alpha\,\mathbf{M} + \beta_R\,\mathbf{K}_0$ con $\mathbf{K}_0$ la rigidez **elástica de referencia** (la primera ensamblada en $t = 0$, $\mathbf{u} = 0$). No se actualiza con la plasticidad: el Rayleigh es modelo aproximado de amortiguamiento estructural, no consecuencia de la cinemática elastoplástica. Mantenerlo constante evita un acoplamiento ad-hoc que ningún código de referencia (Abaqus, ANSYS, OpenSees) realiza por defecto.

Reducción y commit de estado

- **Reducción por Dirichlet** heredada de la fase 3 (ADR 0004): mismo operador $\mathbf{T}$ y $\mathbf{g}$ constantes; el residuo y el jacobiano se proyectan a DOFs libres antes de cada solve.
- **Commit de variables internas**: al converger el paso, se llama `assembler.commit_all_states()` — los estados $(\boldsymbol{\varepsilon}^p, \alpha, \kappa, d, \ldots)$ pasan de *trial* a *committed*. Mismo patrón que `NonlinearSolver` y `ArcLengthSolver`. Si el paso no converge, los estados trial se descartan (no se llama commit) y se reporta error.

Convergencia (ADR 0007)

Criterio dual fuerza + desplazamiento, calibrado al primer paso temporal con la escala del ensamblaje inicial:

- $\|\mathbf{R}\| / (\text{atol} + \text{rtol}\,\|\mathbf{F}\|) \le 1$
- $\|\delta\mathbf{u}\| / (\text{atol} + \text{rtol}\,\|\mathbf{u}\|) \le 1$

Idéntico al `NonlinearSolver` estático. La calibración se hace una vez al inicio del análisis; las tolerancias son adimensionales (invariantes bajo cambio de unidades) gracias a la calibración con `force_scale`/`disp_scale`.

Factorización por iteración

A diferencia de `NewmarkSolver` (que factoriza $\mathbf{J}$ una sola vez al inicio), aquí $\mathbf{K}_t$ cambia con cada iteración, así que $\mathbf{J}$ debe re-factorizarse. Se mantiene el patrón de **Newton modificado opcional** (ADR 0003 fase 2): si `freeze_tangent_after_iter` es `int`, se factoriza fresco las primeras N iteraciones del paso y se reusa la factorización en las siguientes. `None` ⇒ Newton estándar (re-factoriza cada iteración).

Recuperación del comportamiento lineal

Si todos los materiales del modelo son lineales, $\mathbf{K}_t \equiv \mathbf{K}_0$ constante y el residuo se anula en una iteración de Newton (Δx exacto en el primer solve). Por tanto **`NewtonNewmarkSolver` con materiales lineales reproduce exactamente `NewmarkSolver`** (sin discretización extra ni error adicional). Esto se valida en los acceptance.

14.7.2 Contrato de implementación

```yaml
name: NewtonNewmarkSolver
kind: solver
status: validated
extends: NewmarkSolver       # variante según Reglas §4

interface:
  type_yaml: NewtonNewmarkSolver
  pipeline_kind: transient    # mismo que NewmarkSolver

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 9 (familia de Newmark)"
    - de_borst_crisfield: "§5.4 (integración implícita en dinámica no lineal)"
  general_codes:
    - opensees: "integrator Newmark + algorithm Newton"
    - ansys: "transitorio no lineal (Newmark + Newton-Raphson)"

parameters_extra_to_NewmarkSolver:
  - { name: convergence,               type: ConvergenceCriterion, required: false,
      desc: "Política ADR 0007. Default: ConvergenceCriterion() con rtols de solidum.
      constants." }
  - { name: max_iter,                  type: int,                  required: false,
      default: 20, desc: "Máximo iteraciones Newton por paso temporal." }
  - { name: freeze_tangent_after_iter, type: int or None,          required: false,
      default: null, desc: "Si int, Newton modificado: factoriza fresco las primeras N
      iter y reusa después (ADR 0003 fase 2)." }

parameters_inherited:
  # Idénticos a NewmarkSolver: t_end, dt, beta, gamma, rayleigh, u0, u0_dot,
  # F_func, linear_algebra, lumping. Ver docs/specs/NewmarkSolver.md.

state_handling:
  commit: "assembler.commit_all_states() tras converger cada paso"
  rollback: "estado trial descartado si el paso no converge → RuntimeError"

acceptance:
  recovery_linear:
    - name: linear_material_matches_NewmarkSolver
      setup: "Mismo modelo y carga que un caso analítico de NewmarkSolver (oscilador 1
          GDL, viga), pero con NewtonNewmarkSolver"
      expect: "u_history idéntico (rtol 1e-10) al de NewmarkSolver - convergencia en
          1-2 iter por paso"

  nonlinear:
    - name: oscilador_1gdl_elastoplastico_libre
      setup: "1 nodo, Truss2D con Elastoplastic1D, masa concentrada, condición inicial
          u_0 más allá del yield, vibración libre (F=0)"
      expect: "u(t) refleja decaimiento por disipación plástica; α (deformación plá
          stica acumulada) > 0 al final"

    - name: viga_voladizo_elastoplastica_pulso
      setup: "Frame2DEuler con varios elementos, masa distribuida, pulso de carga
          concentrada en el extremo libre"
```

```
      expect: "respuesta no lineal con redistribución plástica; converge en pocas iter
          por paso (max ≤6 típicamente)"

  convergence:
    - name: newton_converges_in_few_iter
      setup: "paso plástico activo en un benchmark"
      expect: "número de iteraciones Newton por paso ≤ max_iter; raramente excede 8 con
           tangente algorítmica consistente"

references:
  - "ADR 0009 §Fase 4 - Transitorio no lineal Newmark + Newton"
  - "Hughes T.J.R. (2000). The Finite Element Method, cap. 7-9 (Newmark, Newton dentro
      de paso temporal)"
  - "Crisfield M.A. (1991). Non-Linear Finite Element Analysis of Solids and Structures
      , vol. 2 cap. 24 (dinámica no lineal)"
  - "Spec padre: [docs/specs/NewmarkSolver.md](NewmarkSolver.md)"
```

14.7.3 Implementación

- **Archivo**: solidum/math/solvers/newmark.py — junto al padre.
- **Clase**: `NewtonNewmarkSolver(NewmarkSolver)`, registrada con `@SolverRegistry.register`. Hereda `__init__` con kwargs adicionales `convergence`, `max_iter`, `freeze_tangent_after_iter`. Sobrescribe `solve()`.
- **Flujo de `solve()`**:

1. Ensamblar $\mathbf{M}$, ensamblar sistema no lineal inicial para obtener $\mathbf{K}_0$ y $\mathbf{F}_{\text{int},0}$.
2. Calibrar amortiguamiento Rayleigh con $\mathbf{K}_0$ (constante en el tiempo).
3. Reducir por Dirichlet (mismo $\mathbf{T}$, $\mathbf{g}$, $\mathbf{F}_{\text{dir}}$ que la fase 3).
4. Aceleración inicial consistente: $\mathbf{M}\,\ddot{\mathbf{u}}_0 = \mathbf{F}_{\text{ext}}(0) - \mathbf{F}_{\text{int}}(\mathbf{u}_0) - \mathbf{C}\,\dot{\mathbf{u}}_0 - \mathbf{F}_{\text{dir}}$.
5. Bucle temporal con Newton interno: predictor → iteración ($\mathbf{R}, \mathbf{J} = \mathbf{M} + \gamma\Delta t\,\mathbf{C} + \beta\Delta t^2\,\mathbf{K}_t$), factoriza/solve, actualiza correctores; converger; commit.
6. Volcar historial; reportar éxito.

- **Refactor del padre**: la fase 3 (`NewmarkSolver`) sigue intacta. La subclase no extrae métodos del padre por ahora (el flujo no lineal es lo suficientemente distinto como para que la herencia sea estructural, no compartición de pasos internos). Cuando emergió la variante HHT-α (2026-05-18, `NewtonHHTSolver` como subclase de `NewtonNewmarkSolver`) la herencia estructural funcionó tal cual — los métodos `solve()` se sobrescriben íntegramente en cada subclase sin compartir pasos internos. Si en el futuro entran generalized-α u otras variantes y se observa que comparten *este* flujo no lineal específico, entonces se refactorizará al patrón con `_solve_step` virtual.
- **Despacho YAML**: `solver: type: NewtonNewmarkSolver` activado vía `SolverRegistry` y `run_yaml`. Como en `NewmarkSolver`, `PIPELINE_KIND = "transient"` declarativo: si futura regla disparo C reemplaza el `isinstance` en `entry.run_yaml`, no se rompe.

- **Tests**:
- tests/test_newmark_nonlinear.py nuevo: acceptance.
- Cobertura de regresión de los tests de `NewmarkSolver` permanece intacta (la subclase no toca el padre).

14.7.4 Diálogo

- **2026-05-14** · Spec creada como **variante** de `NewmarkSolver` aplicando la nueva regla §4 (registrada en el mismo commit que esta spec). Sin ADR nuevo: reusa las decisiones de ADR 0009 fase 4 (residuo, jacobiano, conmutar Newton dentro del paso). Sin entrada propia en el manual estructural — la subclase es invisible al usuario API (excepto por el `type` en YAML y los parámetros extra).
- **2026-05-14** · Decisión: **subclase** sobre `NewmarkSolver` en lugar de bandera `nonlinear=True` en la misma clase. Razones: (i) más explícito en el código y en el YAML (`type` distinto para análisis distinto); (ii) evita condicionales que ensucian el flujo lineal de la fase 3; (iii) deja sitio para variantes adicionales (HHT-α no lineal, generalized-α) por composición. El ADR 0009 fila "Fase 4" decía "`NewmarkSolver` (mismo, con `nonlinear=True`)" — interpretación textualista vs. arquitectural: la regla §4 nueva legitima la elección de subclase, y el patrón conserva el espíritu (mismo análisis, mismas decisiones, mismo entrypoint) sin la implementación literal.
- **2026-05-14** · Amortiguamiento Rayleigh queda fijado con $\mathbf{K}_0$ (rigidez elástica de referencia) y constante en el tiempo. Alternativa rechazada: Rayleigh con $\mathbf{K}_t$ corriente — no es la convención estándar y crea coupling artificial entre la disipación viscosa y la plástica. Documentado en sección "Amortiguamiento Rayleigh — heredado".
- **2026-09-22** · Auditoría global: un ensamblaje por iteración: la convergencia se evalúa al inicio de la iteración con (‖R(U_k)‖, ‖δU_k−1‖) y el estado trial que se comitea es el del propio ensamblaje en U_k (antes cada iteración ensamblaba dos veces para la misma secuencia de decisiones). `max_iter` es el número máximo de resoluciones por paso. `g` se impone en u_0 **antes** de ensamblar F_int(u_0) (la aceleración inicial era inconsistente con apoyos prescritos no nulos: $\ddot{u}_0 = 0$ en vez de $K(g_0 - u_0)/M$). La incógnita reducida se reconstruye con `T` (`ü = T·ü_free`), así que las restricciones lineales con maestros funcionan (antes el esclavo quedaba congelado y el Newton divergía). `prepare_all_steps` por paso (hook ADR 0010 §5, antes ausente en dinámica). Masa reducida singular → `ValueError` accionable.
- **2026-09-22** · `record_internal_forces=True` (deuda #15): registra `elem.internal_forces(u_k)` en cada paso con el estado interno de ese paso (`TransientResult.element_forces_history`; `internal_forces_history(domain)` lo devuelve). Opt-in por coste; sin él, reconstrucción lazy con el estado final y aviso. Verificado contra análisis truncados en `t_k` con plasticidad (`tests/test_transient_forces_history.py`). Además `assemble_system` ya no deja el estado trial evaluado en `u = 0` (deuda #18).
- **2026-09-22** · ADR 0015: el bucle de Newton pasa al corrector compartido `NewtonCorrector`; este solver aporta su problema por paso (`_DynamicNewtonProblem`: iterado `(u, u̇, ü, ü_libre)`, residuo dinámico inyectado, jacobiano `M + γΔt·C + βΔt²·K_t`, norma ‖$T^\top$R‖; retirado

`_armijo_step_dynamic`) y conserva su control de paso. Sin cambio de formulación ni de resultados (suite completa sin tocar ningún valor esperado).

- **2026-09-30** · Auditoría de las MPC (ADR 0004): el oráculo del nodo fundido se añade a la cobertura (u, u̇, ü y N, V, M, esclavo a cada lado; `tests/test_mpc_dynamic.py`), junto con una MPC con término independiente `g` $\neq$ `0` que da la misma historia que `NewmarkSolver`. Si `u0` o `u0_dot` incumplen una MPC con maestros, el solver lo avisa en el log: el valor del esclavo se descarta y se reconstruye como `T·u0[libres] + g`, lo que ya hacía. Sin cambio de física.

14.8 ResponseSpectrumSolver

*Spec de **solver nuevo**. Combina las respuestas máximas modales bajo un espectro de respuesta dado (SRSS o CQC). Cierra el ADR 0009 completo y aplica la regla D de refactor arquitectural (free_vibration sale de `results.py` al nuevo `solidum/math/modal_response.py`).*

14.8.1 Especificación física

0. Descripción general

Análisis sísmico clásico: el suelo se caracteriza por un **espectro de respuesta** (en aceleración `S_a(T)` o desplazamiento `S_d(T)` vs período), no por un acelerograma temporal. Para cada modo del sistema se calcula la respuesta máxima individual `u_n^max = Γ_n·φ_n·S_d(ω_n)` y se combinan las respuestas modales en una envolvente máxima. La fase temporal entre modos se pierde por construcción — sólo se conservan amplitudes envolventes.

Es **lineal** y **estadístico**: aproxima los máximos absolutos sin reconstruir la historia temporal. Útil para diseño normativo donde el espectro reglamentario es la entrada y los máximos modales son la salida solicitada (e.g., ASCE 7, Eurocódigo 8, NCh 433, NTC-DS México).

Hipótesis: lineales (K, M constantes). Para no-linealidad usar transitorio (Newton-Newmark con acelerograma).

1. Ecuación de equilibrio resuelta

Para cada modo `n`:

$$u_n^{\text{máx}} = \Gamma_n \cdot \phi_n \cdot S_d(\omega_n)$$

con $\Gamma_n = \phi_n^T M\, r$ el factor de participación modal en la dirección de excitación `r`.
Combinación SRSS (Square Root of Sum of Squares):

$$u_k^{\text{máx}} = \sqrt{\sum_n (u_{k,n}^{\text{máx}})^2}$$

Combinación CQC (Complete Quadratic Combination, Der Kiureghian 1980):

$$u_k^{\text{máx}} = \sqrt{\sum_i \sum_j \rho_{ij} \cdot u_{k,i}^{\text{máx}} \cdot u_{k,j}^{\text{máx}}}$$

$$\rho_{ij} = \frac{8\xi^2(1+r)r^{3/2}}{(1-r^2)^2 + 4\xi^2 r(1+r)^2}, \quad r = \omega_i/\omega_j$$

Cuando $\xi \to 0$ o $r \to 0$: $\rho_{ij} \to \delta_{ij}$ y CQC degenera a SRSS.

2. Condiciones iniciales / de contorno

- Apoyos Dirichlet constantes y restricciones multipunto (eliminación directa, ADR 0004), heredados del `ModalSolver` interno.

- **Dirección de excitación rígida `r`** (vector unitario o por DOF name): vector `r` con valor 1 en los DOFs traslacionales en la dirección sísmica, 0 en el resto.
- **No** hay condiciones iniciales — la respuesta es envolvente máxima.

3. Salidas físicas

`ResponseSpectrumResult` (inmutable):

- `u_combined[ndof]` respuesta máxima envolvente combinada (positiva).
- `u_per_mode[ndof, n_modes]` contribución modal individual con signo.
- `frequencies_rad[n_modes]` frecuencias naturales empleadas.
- `participation_factors[n_modes]` (Γ_n).
- `effective_masses[n_modes]` (Γ_n^2). Suma converge a la masa total proyectada en la dirección al incluir suficientes modos.
- `combination` (`"SRSS"` o `"CQC"`).
- `damping` (ξ usado para CQC; 0.0 si SRSS).
- `direction[ndof]` vector de excitación.
- Método `.cumulative_effective_mass_ratio()` para verificar el porcentaje de masa modal capturado.

14.8.2 Formulación numérica

4. Esquema operativo

```
1. Ensamble interno vía ModalSolver(n_modes, sigma, lumping):→
    modes, frequencies_rad
2. Resolver direction (np.ndarray o {"dof_name": "ux"}).
3. Resolver spectrum (callable o dict tabulado/constante).
4. Calcular Γ_n = φ^T_n M r (participation_factors).
5. Calcular S_dω(_n) para cada modo.
6. u_per_mode[:, n] = Γ_n ·φ _n · S_dω(_n).
7. Combinar: SRSS (raíz cuadrada de suma de cuadrados elemento a elemento)
   o CQC (forma cuadrática con ρ_ijω(, ξ)).
8. Devolver ResponseSpectrumResult.
```

5. Predictor / corrector

No aplica — un solve por análisis.

6. Criterio de convergencia

No aplica como Newton — pero la calidad del análisis depende del número de modos incluidos. Se verifica con `.cumulative_effective_mass_ratio()` $\geq$ `0.9` (objetivo normativo típico).

7. Imposición de Dirichlet y MPC

Eliminación directa heredada del `ModalSolver` interno, también para las restricciones multipunto: los modos son φ`_n = T·`φ`_red,n` (sin `g`) y los factores de participación `Γ_n =` φ`_n`$^{\top}$`·M·r` se calculan con la masa global, de modo que la masa de un nodo duplicado y atado entra una sola vez. Verificado con el oráculo del nodo fundido (`tests/test_mpc_dynamic.py`). La suma de las masas efectivas de todos los modos es la parte de `r`$^{\top}$`·M·r` compatible con las restricciones (`r`$^{\top}$`M·T·M_red`$^{-1}$`·T`$^{\top}$`·M·r`): igual que la masa de los DOFs apoyados, la de la parte de `r` que incumple una MPC (apoyo oblicuo excitado en otra dirección) no participa. Con MPC de atado de nodos o de periodicidad, `r` las cumple y no resta nada.

8. Backend algebraico

Delega en `ModalSolver` para el cálculo de modos (`scipy.sparse.linalg.eigsh` con shift-invert) y luego operaciones de álgebra densa para la combinación modal (los modos son densos por columna).

9. Adaptatividad / control de paso

No aplica. El usuario controla `n_modes`. Recomendación: iterativamente subir `n_modes` hasta que la masa efectiva acumulada supere el 90 % en la dirección de excitación.

10. Caveats numéricos

- **Cuántos modos**: el método pierde precisión si los modos significativos en la dirección de excitación no están incluidos. Verifica con `cumulative_effective_mass_ratio`.
- **CQC vs SRSS**: usar CQC cuando los modos están cercanos en frecuencia (cociente < 1.3 según práctica habitual). Para modos bien separados, SRSS es la opción estándar y CQC se degenera a SRSS automáticamente.
- **Espectro fuera de rango**: `spectrum_tabulated` aplica clamp a los valores extremos (no extrapola). Si el barrido de frecuencias del modelo cae fuera del rango tabulado, el usuario debe extender la tabla.
- **Excitación multi-direccional**: este solver maneja una sola dirección por análisis. Para combinación direccional (CQC3, regla 100/30/30 de norma) se requieren dos o tres análisis independientes y una combinación adicional fuera del solver.
- **Cargas con fase**: no aplica al método espectral por construcción — la fase se pierde en el cómputo de máximos modales.

14.8.3 Contrato de implementación

```yaml
name: ResponseSpectrumSolver
kind: solver
status: validated

interface:
  yaml_type: ResponseSpectrumSolver
  output: ResponseSpectrumResult
```

```yaml
admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - bathe: "§9.3 (superposición modal, base del método)"
    - chopra: "cap. 13 (análisis por espectro de respuesta, combinación modal SRSS y
        CQC); texto añadido a la lista de Reglas §1 por el usuario el 2026-09-30"
  general_codes:
    - abaqus: "*RESPONSE SPECTRUM (SRSS, CQC)"
    - ansys: "SPRS con SRSS / CQC"
    - code_aster: "COMB_SISM_MODAL (SRSS, CQC)"

parameters:
  - { name: n_modes, type: int, required: true, desc: "Número de modos a calcular y
      combinar" }
  - { name: direction, type: "ndarray | dict", required: true, desc: "Vector r o {'
      dof_name': 'ux'}" }
  - { name: spectrum, type: "callable | dict", required: true, desc: "S_dω() o {'type':
       'tabulated'|'constant_acceleration', ...}" }
  - { name: combination, type: str, required: false, desc: "'SRSS' (default) o 'CQC'" }
  - { name: damping, type: float, required: false, desc: "ξ modal (default 0.05); sólo
      usado por CQC" }
  - { name: sigma, type: float, required: false, desc: "Shift para ARPACK (default 0)"
      }
  - { name: lumping, type: str, required: false, desc: "'consistent' (default) o '
      lumped'" }

requirements:
  - "Materiales con `density` declarada (ADR 0008)."
  - "Problema lineal - K y M constantes."

conventions:
  units:    "heredadas del modelo (ADR 0008)."
  stability: "No iterativo; estabilidad numérica gobernada por ModalSolver interno."

out_of_scope:
  - "Excitación multi-direccional simultánea (CQC3) - usuario combina afuera."
  - "Espectros con dependencia explícita de ξ por modo ξ(_n distinto) - `damping` es
      global."
  - "No linealidad."

acceptance:
  verification:
    - name: single_mode_recovers_gamma_phi_Sd
      setup: "Modelo 2-DOF, n_modes=1, S_d constante=1"
      expect: "u_combined = |u_per_mode[:, 0]| (SRSS trivial sobre un modo)"
      tol_rel: 1.0e-12
    - name: laplaciano_1d_3dof_analitico
      setup: |
        Cadena 4-truss n1-n2-n3-n4-n5 con n1, n5 fijos. E=A=L=1, ρ=1, lumped →
        K Toeplitz tridiagonal [[2,-1,0],[-1,2,-1],[0,-1,2]], M=I. Autovalores
        cerrados del Laplaciano 1D discreto: ω_n² = 2 − 2·cos(πn/4); modosφ
        _n[i] = √(2/4)·sin(n·i·π/4).
      expect: |
        (1) ω₁ = √−√(22), ω₂ = √2 a precisión doble.
        (2) Con d = (1, √1/2, 0): γ₁ = 1, γ₂ = √1/2, γ₃ = 0 φ₃ᵀ(d = 0 por construcción)
            .
```

```
        (3) Para Sd ≡ 1 y SRSS: u_combined = √(1/2, √1/2, √1/2) exacto.
        (4) Con d = e_{₃ux} (centro): sólo modo 1 contribuye, u_combined = (c√/(22), c
            /2, c√/(22)).
      tol_rel: 1.0e-10
      ref: "tests/test_response_spectrum.py::TestResponseSpectrumAnalytic2DOF"
    - name: srss_manual_combination
      setup: "Modelo 3-DOF, 2 modos, comparar contra raíz cuadrada de suma de cuadrados
          componente a componente"
      expect: "Coincidencia exacta a precisión de máquina"
      tol_rel: 1.0e-12
    - name: well_separated_modes_cqc_equals_srss
      setup: "Modelo 3-DOF con cociente ω₂ω₁/ > 2"
      expect: "|u_CQC - u_SRSS| / |u_SRSS| < 1%"
      tol_rel: 0.01
  specific:
    - name: close_modes_cqc_differs_from_srss
      setup: "Sistema sintético con ω₂ω₁/ = 1.05 y modos no-ortogonales-en-componentes"
      expect: "Diferencia relativa CQC vs SRSS > 5%"
      tol_rel: 0
    - name: dof_name_builds_unit_vector
      setup: "direction={'dof_name': 'ux'}"
      expect: "direction[node.dofs['ux']] = 1 en todos los nodos con ux"
      tol_rel: 0
    - name: cumulative_effective_mass_monotonic
      setup: "n_modes=2 sobre 3-DOF chain"
      expect: "cumulative crece monotonamente y termina en 1.0"
      tol_rel: 1.0e-10
    - name: free_vibration_wrapper_unchanged
      setup: "Regla D aplicada - ModalResult.free_vibration sigue funcionando"
      expect: "Resultado idéntico a llamada directa a modal_response.free_vibration"
      tol_rel: 1.0e-12
    - name: mpc_tie_matches_merged_node
      setup: "ménsula de Frame2DEuler inclinada 30° partida en su nodo central y atada
          con u_s = u_m en ux, uy y rz, con el esclavo a cada lado del corte; SRSS y
          CQC, dirección ux y uy"
      expect: "u_combined, u_per_mode, Γ||, masas efectivas y total_mass del modelo
          atado = los del modelo de nodo fundido"
      tol_rel: 1.0e-9
      ref: "tests/test_mpc_dynamic.py::test_espectral_reproduce_el_nodo_fundido"

references:
  - "Chopra, A. K. (2017). *Dynamics of Structures*, cap. 13."
  - "Wilson, E. L. (2002). *Three-Dimensional Static and Dynamic Analysis of Structures
      *, cap. 15."
  - "Der Kiureghian, A. (1980). 'Structural Response to Stationary Excitation'. *J. Eng
      . Mech. Div. ASCE*, 106(EM6), 1195-1213."
```

14.8.4 Implementación

- Archivo solver: solidum/math/solvers/response_spectrum.py
- Clase: `ResponseSpectrumSolver` (registrada en `SolverRegistry`)
- Atributo de despacho: `PIPELINE_KIND = "spectrum"` → `solidum.entry.run_response_spectrum` → `ResponseSpectrumResult`.

- Algoritmos centralizados: solidum/math/modal_response.py (`response_spectrum_srss`, `response_spectrum_cqc`, `participation_factors`, helpers `spectrum_from_sa`, `spectrum_tabulated`).
- Resultado: `ResponseSpectrumResult` — dataclass inmutable.
- Tests: tests/test_response_spectrum.py (19 tests verdes).

14.8.5 Diálogo

- *2026-05-18*: regla D aplicada al introducir este solver — segundo método de cómputo sobre `ModalResult` (el primero fue `free_vibration`). El algoritmo de free_vibration se movió a `solidum/math/modal_response.py`; `ModalResult.free_vibration` queda como wrapper delgado de 3 líneas, preservando la API histórica. Los nuevos `response_spectrum_srss` y `response_spectrum_cqc` viven en el mismo módulo. `ResponseSpectrumSolver` orquesta `ModalSolver` interno + delegación a `modal_response`.
- *2026-05-18*: el cuarto valor de `PIPELINE_KIND` (`"spectrum"`) extiende el despacho declarativo de `run_yaml` introducido en D2. Cierra el ADR 0009 completo (fases 1-7 + variante HHT-α).
- **2026-09-22** · Auditoría global: `cumulative_effective_mass_ratio` normaliza por la masa total en la dirección de excitación $\mathbf{r}^\top \cdot \mathbf{M} \cdot \mathbf{r}$ (nuevo campo `total_mass` del resultado); antes dividía por la suma de los modos calculados, que vale siempre 1.0 en el último modo y volvía trivial el criterio del 90 %.
- **2026-09-30** · Auditoría de las MPC (ADR 0004): la spec declaraba las restricciones multipunto fuera de alcance, pero el `ModalSolver` interno ya las reduce con `T` y la combinación reproduce el oráculo del nodo fundido hasta el redondeo (SRSS y CQC, dirección ux y uy: `u_combined`, `u_per_mode`, Γ, masas efectivas y `total_mass`; `tests/test_mpc_dynamic.py`). Se documentan como admitidas, con la nota de §7 sobre la masa efectiva cuando `r` no cumple las restricciones. Sin cambio de código ni de física.

14.9 ThetaMethodSolver

Spec pre-redactada por la IA (2026-08-25). ***El usuario revisa la física; el plumbing está resuelto.***

Cuarto y último componente de la Etapa 8 — C1 núcleo mínimo. Resuelve el transitorio de conducción sobre `Quad4Thermal` *y* `Hex8Thermal`*.*

14.9.1 Especificación física

0. Descripción general y por qué es un solver nuevo

Integrador temporal para sistemas semidiscretos de **primer orden**:

$$\mathbf{C}\,\dot{\mathbf{T}} + \mathbf{K}\,\mathbf{T} = \mathbf{F}(t)$$

No es una variante de `NewmarkSolver`. La familia Newmark integra la ecuación de **segundo orden** $\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{F}$ mediante hipótesis sobre la aceleración dentro del paso. En conducción no existe segunda derivada temporal: no hay aceleración, no hay inercia, y las hipótesis de Newmark no tienen sobre qué aplicarse. Forzar Newmark con $\mathbf{M} = \mathbf{0}$ degenera el esquema.

Es, por tanto, un solver de familia propia, no una subclase — y no le corresponde spec corta de extensión sino spec completa (`Reglas.md §4`).

El **régimen estacionario** no lo usa: $\mathbf{KT} = \mathbf{F}$ lo resuelve el `LinearSolver` existente sin modificación.

1. Esquema θ

Se aproxima la ecuación en un punto intermedio del paso, ponderando el estado con el parámetro $\theta \in [0, 1]$:

$$\mathbf{C}\,\frac{\mathbf{T}_{n+1} - \mathbf{T}_n}{\Delta t} + \mathbf{K}\left[\theta\,\mathbf{T}_{n+1} + (1-\theta)\,\mathbf{T}_n\right] = \theta\,\mathbf{F}_{n+1} + (1-\theta)\,\mathbf{F}_n$$

Reordenando, el sistema a resolver por paso:

$$\underbrace{\left(\mathbf{C} + \theta\,\Delta t\,\mathbf{K}\right)}_{\mathbf{A}}\,\mathbf{T}_{n+1} = \left[\mathbf{C} - (1-\theta)\,\Delta t\,\mathbf{K}\right]\mathbf{T}_n + \Delta t\left[\theta\,\mathbf{F}_{n+1} + (1-\theta)\,\mathbf{F}_n\right]$$

$\mathbf{A}$ es **simétrica y definida positiva** para todo $\theta \in [0, 1]$ (suma de $\mathbf{C}$ definida positiva y $\mathbf{K}$ semidefinida, **con o sin Dirichlet**: el modo de temperatura uniforme anula $\mathbf{K}$ pero no $\mathbf{C}$), luego el despachador algebraico (ADR 0003) elige Cholesky. Con Δt constante, **$\mathbf{A}$ se factoriza una sola vez** y se reutiliza en todos los pasos.

2. Casos particulares de θ

θ	Nombre	Orden	Estabilidad
0	Euler explícito (forward)	1	**Condicional**: $\Delta t \leq 2/\lambda_{\text{máx}}$
1/2	Crank-Nicolson	**2**	Incondicional (A-estable)
2/3	Galerkin	1	Incondicional
1	Euler implícito (backward)	1	Incondicional, **L-estable**

Con $\theta \geq 1/2$ el esquema es **incondicionalmente estable**: ningún Δt produce divergencia. Es la propiedad que distingue este solver del `CentralDifferenceSolver` mecánico, cuya estabilidad es siempre condicional (CFL).

3. Estabilidad no es lo mismo que ausencia de oscilaciones

Punto físico central de esta spec, y la razón de la elección de default.

Crank-Nicolson ($\theta = 1/2$) es de **segundo orden** y por tanto el más preciso de la tabla, pero **A-estable y no L-estable**: su factor de amplificación tiende a -1 para los modos de frecuencia alta, en lugar de a 0. Consecuencia práctica: ante un **cambio brusco** —un escalón de temperatura impuesto en la frontera, típico en el arranque de un análisis— produce **oscilaciones amortiguadas lentamente** en los primeros pasos, con temperaturas que pueden salirse del rango de los datos.

Euler implícito ($\theta = 1$) es sólo de primer orden, pero **L-estable**: amortigua los modos altos por completo en un paso. Nunca oscila.

Esto es el mismo fenómeno físico que motivó el default `lumped` en la matriz de capacidad, atacado desde el otro lado: **el principio del máximo** de la ecuación de difusión dice que la solución exacta nunca excede los extremos de los datos iniciales y de frontera. Un esquema que lo viola no es simplemente impreciso — produce un resultado cualitativamente imposible.

Por eso el default propuesto es $\theta = 1$ (Euler implícito), robusto ante el arranque típico. Crank-Nicolson queda disponible para quien busque precisión de segundo orden y sepa que su problema es suave. Se documenta el criterio para que la elección sea informada y no una preferencia oculta.

4. Condición inicial

El campo $\mathbf{T}_0$ debe declararse. Dos formas admitidas:

- **Uniforme**: un escalar, el caso más frecuente (cuerpo inicialmente a temperatura ambiente).
- **Por nodo**: vector completo, para arrancar de un estado no uniforme (p. ej. el resultado de un análisis estacionario previo).

A diferencia del mecánico, **no hay velocidad inicial**: la ecuación es de primer orden y $\mathbf{T}_0$ la determina por completo.

Consistencia con Dirichlet: si $\mathbf{T}_0$ contradice una temperatura impuesta en un nodo con Dirichlet, existe una discontinuidad en $t = 0$. Es físicamente legítimo (un choque térmico es exactamente eso) pero es la situación que más excita las oscilaciones descritas en §3. El solver **avisa** cuando lo detecta, sin abortar: es modelización válida, no error. La comparación se hace sólo en los

DOF con Dirichlet declarado y contra el valor prescrito en $t = 0$ (el de `dirichlet_func(0)` si se da); los esclavos de las restricciones multipunto (MPC) no entran, porque su término independiente no es una temperatura impuesta.

5. Selección del paso de tiempo

El paso no se elige por estabilidad —con $\theta \geq 1/2$ no hay límite— sino por **precisión**. La escala física la da la difusividad $\alpha = k/(\rho c)$ [m^2/s] del material y el tamaño característico de elemento h:

$$\Delta t_{\text{car}} \sim \frac{h^2}{\alpha}$$

Es el tiempo que tarda el frente térmico en atravesar un elemento. Un $\Delta t \gg \Delta t_{\text{car}}$ es estable pero no resuelve el transitorio: salta por encima de la física que se quiere ver.

El solver **no calcula ni impone** este valor —igual que el `CentralDifferenceSolver` no calcula el Δt crítico—, pero sí lo **reporta como diagnóstico** al arrancar, para que el usuario contraste su elección. Es información, no restricción.

6. Sin amortiguamiento de Rayleigh

El amortiguamiento de Rayleigh $\mathbf{C} = \alpha\mathbf{M} + \beta\mathbf{K}$ del subsistema dinámico **no aplica**: es un modelo de disipación de la ecuación de segundo orden. En conducción la "disipación" es la propia conducción, ya contenida en $\mathbf{K}$. El parámetro no existe en este solver.

14.9.2 Formulación numérica

7. Algoritmo

```
A = C + θ·Δt·K                      # una vez, si Δt constante
factor = factorize(A)               # Cholesky (A es SPD); reutilizada
B = C - (1-θ)·Δt·K                  # una vez

T = T_0
para cada paso n:
    rhs = B·T_n + Δt·θ·F_{n+1} + (1-θ)·F_n
    aplicar Dirichlet (eliminación, ADR 0004)
    T_{n+1} = factor.solve(rhs_reducido)
    almacenar según output_every
```

Coste por paso: una sustitución hacia adelante/atrás. La factorización es única mientras Δt y las matrices no cambien — el problema es lineal y sin historia, así que $\mathbf{K}$ y $\mathbf{C}$ se ensamblan una sola vez.

8. Dirichlet dependiente del tiempo

Una temperatura impuesta puede variar en el tiempo, $\bar{T}(t)$ — es un caso de uso central (una superficie sometida a un ciclo térmico). La eliminación del ADR 0004 se aplica en cada paso con el valor correspondiente; el término de acoplamiento $\mathbf{K}_{fp}\bar{\mathbf{T}}_p$ se recalcula, la factorización de $\mathbf{A}_{ff}$ **no**.

Con restricciones multipunto. El modelo afín del ADR 0004 es $\mathbf{T} = \mathbf{T}_{op}\,\mathbf{T}_{\text{libre}} + \mathbf{g}$, y $\mathbf{g}$ reúne dos cosas distintas: el valor prescrito $\bar{T}_d(t)$ en cada DOF con Dirichlet declarado y, en cada esclavo s de una MPC, el término independiente que resulta del cierre transitivo,

$$g_s(t) = g_{\text{mpc},s} + \sum_d \alpha_{sd}\,\bar{T}_d(t),$$

con $g_{\text{mpc},s}$ la parte propia de las MPC de su cadena y α_{sd} el peso con que el Dirichlet d le llega a través de ella (un esclavo atado a un maestro con Dirichlet hereda su valor). `dirichlet_func(t)` da sólo los $\bar{T}_d(t)$; el solver lee sus entradas en los DOF con Dirichlet declarado y reconstruye las filas esclavas con la MPC (`ConstraintSet.build_g`). El conjunto de DOF prescritos lo fija el modelo: un valor no nulo en un DOF libre se rechaza. La condición inicial almacenada en $t = 0$ usa $\mathbf{g}(0)$.

9. Resultado

Devuelve un `ThermalTransientResult` con `t_history`, `T_history`, `n_steps` y `converged`.

Decisión de zona gris que se señala: no se reutiliza `TransientResult`, cuyos campos `udot_history`, `uddot_history`, `alpha_rayleigh` y `beta_rayleigh` no tienen significado térmico. Rellenarlos con `None` o ceros produciría un objeto que miente sobre su contenido. Se sigue el criterio ya aplicado en el proyecto con `ModalResult`, `HarmonicResult` y `ResponseSpectrumResult`: un tipo de análisis con semántica propia tiene su dataclass propia.

10. Caveats numéricos

- **Sin Dirichlet (cuerpo aislado)**: en el estacionario $\mathbf{K}$ es singular por el modo de temperatura uniforme y el `LinearSolver` lo rechaza nombrando el campo `T` sin valor prescrito (red de seguridad, ADR 0019). En el transitorio el problema está bien planteado y se admite (desde el 2026-10-20): $\mathbf{A} = \mathbf{C} + \theta\Delta t\,\mathbf{K}$ es definida positiva por $\mathbf{C}$, el nivel de temperatura lo fijan $\mathbf{T}_0$ y el calor aportado, y la energía almacenada $\mathbf{1}^\top\mathbf{C}\,\mathbf{T}$ crece exactamente lo que aportan las fuentes. Con Δt muy grande el condicionamiento de $\mathbf{A}$ empeora como $\theta\Delta t\,\lambda_{\text{máx}}(\mathbf{K})/\lambda_{\text{mín}}(\mathbf{C})$, sin perder la definición positiva.
- **Oscilaciones con Crank-Nicolson ante escalón**: descrito en §3. No es un error del solver; se documenta y se cuantifica con test.
- $\theta = 0$ **(explícito)**: se admite por completitud pero requiere $\Delta t \le 2/\lambda_{\text{máx}}$, y con capacidad consistente $\lambda_{\text{máx}}$ es grande. Sin masa lumped es prácticamente inutilizable; se avisa.
- **Contraste alto de conductividad**: mal condicionamiento heredado del problema, no del esquema.

14.9.3 Contrato de implementación

```yaml
name: ThetaMethodSolver
kind: solver
status: validated        # draft → implemented → validated

interface:
  pipeline_kind: thermal_transient
```

```
  field: temperature
  equation_order: 1      # primer orden en el tiempo (Newmark integra segundo)

admission:                # criterio de admisión al programa principal (Reglas.md §1)
  textbooks:
    - hughes: "cap. 8 (familia trapezoidal generalizada)"
    - zienkiewicz_taylor: "vol. 1: aproximación discreta en el tiempo, esquemas de un
       paso de primer orden"
  general_codes:
    - code_aster: "THER_LINEAIRE con PARM_THETA"
    - ansys: "TINTP con THETA (transitorio térmico)"

parameters:
  - { name: theta, type: float, required: false, default: 1.0,
      desc: "Peso del esquema ∈ [0,1]. 1.0 Euler implícito (L-estable, default);
             0.5 Crank-Nicolson (2º orden, A-estable, puede oscilar ante escalón);
             2/3 Galerkin; 0.0 explícito (condicionalmente estable)" }
  - { name: dt, type: float, required: true,
      desc: "Paso de tiempo [s]. No limitado por estabilidad si θ≥ 0.5, sí por
             precisión: contrastar con el Δt característico h²α/ reportado" }
  - { name: n_steps, type: int, required: true, desc: "Número de pasos" }
  - { name: T_initial, type: float | array-like, required: true,
      desc: "Condición inicial. Escalar ⇒ campo uniforme; vector ⇒ valor por nodo" }
  - { name: lumping, type: str, required: false, default: "lumped",
      desc: "Forma de la matriz de capacidad. Default lumped por el mismo criterio
             físico que en los elementos térmicos (Quad4Thermal §5)" }
  - { name: output_every, type: int, required: false, default: 1,
      desc: "Almacenar el campo cada N pasos; limita la memoria en corridas largas" }
  - { name: linear_algebra, type: str, required: false, default: "auto",
      desc: "Backend algebraico (ADR 0003). A es SPD ⇒ Cholesky por defecto" }
  # Sin `rayleigh`: el amortiguamiento de Rayleigh es de la ecuación de 2º orden.

conventions:
  stability: "θ ≥ 0.5 incondicionalmente estable; θ = 1 además L-estable
              (amortigua modos altos por completo, nunca oscila)"
  dt_guidance: "Δt característico ~ h²α/ con α = kρ/(c). Reportado como
                diagnóstico al arrancar; no impuesto"
  order_reporting: "el solver reporta el orden efectivo del esquema según θ
                    (2 si θ=0.5, 1 en el resto), para que el coste en precisión
                    del default robusto sea visible y no silencioso"
  factorization: "A = C + θ·Δt·K se factoriza UNA vez si Δt es constante"

validity:
  - "sistema lineal de primer orden: C·Ṫ + K·T = F"
  - "θ ∈ [0, 1]; dt > 0; n_steps ≥ 1"
  - "no exige Dirichlet: sin temperatura impuesta el cuerpo está aislado y
     A = C + θΔt·K sigue siendo definida positiva (decisión del usuario,
     2026-10-20); el estacionario K·T = F sí lo exige"
  - "material con c y density declaradas (ThermalConduction §4)"

out_of_scope:
  - "no linealidad: k(T), cambio de fase, radiación ⇒ requerirían Newton por paso"
  - "paso de tiempo adaptativo ⇒ diferido hasta caso de uso real"
  - "amortiguamiento de Rayleigh (no aplica a 1er orden)"
  - "acoplamiento termomecánico"
```

```yaml
acceptance:
  verification:
    - name: soluciones manufacturadas (ADR 0022)                    # 2026-10-27
      setup: "T*(x, t) con fuente Q*(x, t), flujo¯q*(x, n, t) y Dirichlet T*(x, t)
          variables en el tiempo; k anisótropa; en el tiempo, T* lineal en x sobre
          malla fija distorsionada (4×4 Quad4Thermal, 2×2×2 Hex8Thermal) con capacidad
          consistente, θ = 1, 2/3, 1/2 y 0, Δt halvado; en el espacio, Crank-Nicolson
          con Δt = h/2"
      expect: "orden temporal observado entre los dos últimos niveles ≥ orden del
          esquema - 0.1 (2 con θ = 1/2, 1 con los demás), en L² y en energía; con T*
          lineal en x y en t, error de redondeo para todo θ (parcela en el tiempo); por
           encima de su límite el explícito diverge y se detecta"
      tol_abs: 0.1
    - name: decaimiento_exponencial_1dof
      setup: "sistema de 1 DOF c·Ṫ + k·T = 0 con T(0) = T_0; comparar contra
             la solución analítica T(t) = T_0·exp-(k·t/c)"
      expect: "error < tol para θ=1 y θ=0.5; el error de Crank-Nicolson decrece
              como OΔ(t²) y el de Euler implícito como OΔ(t) - verificar AMBAS
              tasas por refinamiento sucesivo de Δt"
      tol_rel: 1.0e-3
    - name: orden_de_convergencia_temporal
      setup: "mismo problema, Δt sucesivamente halvado"
      expect: "pendiente log-log ≈ 1 para θ=1 y ≈ 2 para θ=0.5.
               Es el test que distingue realmente los dos esquemas"
      tol_abs: 0.15
    - name: estabilidad_incondicional
      setup: "θ = 1 con Δt tres órdenes de magnitud mayor que h²α/"
      expect: "la solución NO diverge; converge monótonamente al estacionario.
               Con θ = 0 y el mismo Δt sí diverge (contraste explícito)"
    - name: convergencia_al_estacionario
      setup: "problema de pared plana con Dirichlet fijo en ambos extremos;
             integrar hasta t →ϖ "
      expect: "el campo converge al perfil lineal que da el LinearSolver sobre
              el MISMO modelo; comparación nodo a nodo"
      tol_rel: 1.0e-8
  specific:
    - name: L_estabilidad_ante_escalon
      setup: "cuerpo a T_0 uniforme; escalón de Dirichlet en una cara;Δ
             t grande; comparar θ=1 contra θ=0.5"
      expect: "θ=1 monótono, sin temperaturas fuera del rango [T_0, T_pared];θ
               =0.5 exhibe oscilación amortiguada en los primeros pasos.
               El test DOCUMENTA la diferencia, no la corrige"
    - name: principio_del_maximo
      setup: "θ=1, capacidad lumped, sin fuente; condiciones de frontera
             acotadas entre T_min y T_max"
      expect: "en TODO paso y TODO nodo, T ∈ [T_min, T_max]. Ningún valor
               fuera del rango de los datos"
      tol_abs: 1.0e-12
    - name: conduccion_transitoria_semi_infinita
      setup: "barra larga inicialmente a T_0, con la cara x=0 llevada
             súbitamente a T_s; comparar con la solución analítica de
             Carslaw-Jaeger T(x,t) = T_s + (-T_0T_s)·erf(x√α/(2(t)))"
      expect: "perfil coincidente en varios instantes mientras el frente no
               alcanza el extremo opuesto (hipótesis de medio semi-infinito)"
      tol_rel: 2.0e-2
    - name: balance_energetico_transitorio
```

```
      setup: "dominio aislado (todo Neumann homogéneo) con fuente Q constante;
             malla distorsionada y campo inicial no uniforme; 2D y 3D;θ∈
               {1, 2/3, 1/2, 0}, capacidad lumped y consistent"
      expect: "la energía acumulada ᵀ1·C·(T_n - T_0) iguala a Q·V·t_n en cada
              paso - el esquema conserva la energía introducida"
      tol_rel: 1.0e-10
    - name: cuerpo_aislado_calentamiento_uniforme
      setup: "dominio aislado con fuente Q uniforme y T_0 uniforme, sin ninguna
             temperatura impuesta; 2D (espesor ≠ 1) y 3D; θ∈ {1, 2/3, 1/2}
             con Δt ≫ h²α/ y θ = 0 dentro de su límite; lumped y consistent"
      expect: "T(t) = T_0 + Q·tρ/(c) en todos los nodos. Es lineal en el
               tiempo, así que todo θ la reproduce sin error de truncación.
               El estacionario sobre el mismo modelo se rechaza (MechanismError)"
      tol_rel: 1.0e-12
    - name: dirichlet_variable_en_el_tiempo
      setup: "temperatura impuesta con variación sinusoidal en el tiempo"
      expect: "el campo sigue la excitación; la factorización de A_ff se reutiliza
               (verificado por contador de llamadas) mientras el acoplamiento
               K_fp·T_p se recalcula por paso"
      tol_rel: 1.0e-10
    - name: condicion_inicial_escalar_y_vectorial
      setup: "arrancar con T_initial escalar y con vector equivalente"
      expect: "resultados idénticos; el escalar se expande correctamente"
      tol_abs: 1.0e-14
    - name: aviso_incompatibilidad_inicial_dirichlet
      setup: "T_initial contradice un Dirichlet impuesto en un nodo"
      expect: "el solver AVISA identificando los nodos, y continúa: un choque
               térmico es modelización legítima, no un error"
    - name: restricciones_multipunto
      setup: "oráculo del nodo fundido: modelo atado con T_s = T_m en una
             columna interior frente a la malla conforme, 2D Quad4Thermal y
             3D Hex8Thermal; θ∈ {1, 1/2, 0}, capacidad lumped y consistent;
             estacionario con LinearSolver; Dirichlet variable sobre un maestro
             de MPC y MPC con término independiente g"
      expect: "misma historia de temperaturas hasta el redondeo; el esclavo
               sigue al maestro con Dirichlet y conserva el salto g; el aviso de
               T_initial cuenta sólo el Dirichlet declarado; sin ningún
               Dirichlet, el modelo atado y el fundido coinciden"
      tol_abs: 1.0e-11
    - name: reporte_del_orden_efectivo
      setup: "construir el solver con θ = 1, 0.5 y 2/3"
      expect: "reporta orden 1, 2 y 1 respectivamente. Hace visible el coste en
               precisión del default robusto en vez de dejarlo implícito"
    - name: rechazo_inputs_invalidos
      setup: "θ fuera de [0,1]; dt ≤ 0; n_steps < 1; material sin c ni density"
      expect: "ValueError con mensaje accionable en cada caso"

references:
  - "Zienkiewicz O.C., Taylor R.L. (2000). The Finite Element Method, Vol. 1.
     Butterworth-Heinemann. §18 (esquemas de un paso para 1er orden; análisis de
     estabilidad)."
  - "Lewis R.W., Nithiarasu P., Seetharamu K.N. (2004). Fundamentals of the FEM for
     Heat and Fluid Flow. Wiley. §6 θ(-method, oscilaciones, criterios de Δt)."
  - "Hughes T.J.R. (2000). The Finite Element Method: Linear Static and Dynamic FEA.
     Dover. §8 (algoritmos para problemas parabólicos; A-estabilidad vs L-estabilidad)
     ."
```

```
  - "Carslaw H.S., Jaeger J.C. (1959). Conduction of Heat in Solids. Oxford. §2.4 (sólido semi-infinito, solución con función error)."
  - "Wood W.L. (1990). Practical Time-Stepping Schemes. Oxford."
```

14.9.4 Implementación

- **Archivo**: `solidum/math/solvers/theta_method.py`
- **Clase**: `ThetaMethodSolver` (`@SolverRegistry.register`, `PIPELINE_KIND = "thermal_transient"`)
- **Resultado**: `ThermalTransientResult` en `solidum/results.py`
- **Entrypoint**: `solidum.entry.run_thermal_transient`
- **Tests**: `tests/test_theta_method.py` (79 tests); restricciones multipunto en `tests/test_theta_method_mpc.py`

Resultados medidos

Criterio de `acceptance`	Predicción de la spec	Medido
`orden_de_convergencia_temporal`, θ=1	pendiente ≈ 1	**0.9944**
`orden_de_convergencia_temporal`, θ=1/2	pendiente ≈ 2	**2.0001**
Galerkin θ=2/3	1 (no gana orden)	**0.9933**
`convergencia_al_estacionario`	coincide con `LinearSolver`	**7.1e-14**
`conduccion_transitoria_semi_infinita`	error $< 2\,\%$	**3.0e-3**
`L_estabilidad_ante_escalon`, θ=1	`T` $\in$ `[0, 100]`, monótono	**exacto, monótono**
`L_estabilidad_ante_escalon`, θ=1/2	sobrepasa el rango	**T_max = 151.27**
`estabilidad_incondicional`, θ=1 vs θ=0	uno converge, el otro diverge	**confirmado**
`dirichlet_variable_en_el_tiempo`	1 sola factorización	**1**
`soluciones manufacturadas` (2026-10-27), θ = 1 / 2/3 / 1/2 / 0	orden 1 / 1 / 2 / 1	**0.979 / 1.03 / 2.001 / 1.001** (L^2, últimos niveles)
Parcela en el tiempo, todo θ	redondeo	**$4{\cdot}10^{-16}$**

Notas de implementación

Medir el orden contra el sistema semidiscreto, no contra el continuo. El error total mezcla discretización espacial (fija) y temporal (la que se refina); el error espacial actúa como suelo y enmascara la tasa temporal en cuanto el paso se hace pequeño. Los tests comparan contra $\mathbf{T}(t) = \exp(-\mathbf{C}^{-1}\mathbf{K}\,t)\,\mathbf{T}_0$ — la solución exacta en el tiempo del **mismo** sistema de ODEs que el solver integra —, de modo que sólo queda el error temporal. Es lo que hace que la pendiente salga 2.0001 y no un valor contaminado por el suelo espacial.

Término de capacidad en el Dirichlet variable en el tiempo. El acoplamiento con los DOFs prescritos tiene *dos* contribuciones, no una:

$$-\Delta t\,\mathbf{K}_{fp}\left[\theta\,\bar{\mathbf{g}}_{n+1}+(1-\theta)\,\bar{\mathbf{g}}_{n}\right]\;-\;\mathbf{C}_{fp}\left(\bar{\mathbf{g}}_{n+1}-\bar{\mathbf{g}}_{n}\right)$$

El segundo término se anula cuando $\bar{\mathbf{g}}$ es constante —el caso habitual— y por eso es fácil de omitir; pero con un ciclo térmico impuesto representa la energía que entra al calentar el propio nodo prescrito, y sin él el resultado queda sesgado. Blindado por `test_dirichlet_constante_via_funcion_iguala_al_declarado`, que verifica que el camino con `dirichlet_func` constante reproduce exactamente el camino sin ella.

El diagnóstico del paso característico funciona en los dos sentidos. Durante la validación, un primer sondeo del criterio `convergencia_al_estacionario` dio error 0.12 en vez de ~1e-13. No era un defecto del esquema: el sondeo integraba hasta $t_{\text{end}} = 2\times10^5$ s con un tiempo de difusión global $L^2/\alpha = 3.2\times10^5$ s — el transitorio simplemente **no había terminado**. Al integrar más allá de L^2/α el error cae a 7.1e-14. El test conserva ambos lados: la convergencia y el contraste `test_transitorio_inacabado_no_ha_convergido_todavia`, para que un cambio futuro que hiciera "converger" antes de tiempo —lo que sí sería un error— quede detectado.

Detección de divergencia. El umbral (`_DIVERGENCE_FACTOR = 1e6` sobre el rango de los datos) es deliberadamente generoso: distingue una explosión numérica —$\theta < 1/2$ fuera de su límite de estabilidad, donde el campo crece órdenes de magnitud por paso— de una oscilación espuria acotada, que es un resultado malo pero no un fallo del bucle. Esta segunda se diagnostica con `ThermalTransientResult.extremes()` contra el principio del máximo, no con `converged`.

Captura de avisos en los tests. `caplog` de pytest no sirve sobre el logger del proyecto: declara `propagate = False` (ADR 0005) para no contaminar a la aplicación que embeba Solidum, así que sus registros nunca llegan al logger raíz que `caplog` intercepta. Los tests enganchan un handler propio (`capturar_avisos`).

Notas de traducción: ninguna. La formulación se implementó directamente en la convención del proyecto; no hay conflicto de signos con las referencias — Zienkiewicz §18, Lewis §6 y Hughes §8 usan todos $\mathbf{C}\,\dot{\mathbf{T}}+\mathbf{K}\,\mathbf{T}=\mathbf{F}$ con la misma orientación de flujo.

Hallazgo colateral, resuelto

Mensaje de error del `Assembler` para materiales de familias no mecánicas. La validación de `Assembler.assemble_mass_matrix` aconsejaba *"usa `0.0` explícitamente si el material es sin masa por diseño (penalty, restricción)"* — consejo correcto en el dominio mecánico (ADR 0008), pero **físicamente incorrecto en térmico**: `density = 0` da $\rho c = 0$, es decir capacidad calorífica nula, que hace singular la matriz $\mathbf{C}$ y deja el transitorio irresoluble. Un usuario que siguiera el consejo llegaba a un fallo peor y más críptico.

El mensaje accionable correcto ya existía en `ThermalMaterial.volumetric_capacity` —orienta hacia el `LinearSolver` estacionario, que no necesita ρc— pero era inalcanzable porque la validación del ensamblador cortaba antes.

Resolución: el ensamblador delega el mensaje al material cuando el material expone `volumetric_capacity`, y conserva el mensaje mecánico genérico cuando no. El ensamblador no necesita conocer las familias de material; sólo pregunta si el material sabe explicarse a sí mismo (Reglas.md §1, coste del componente N+1). Ningún material mecánico cambia de comportamiento, blindado por `test_un_material_mecanico_sin_density_conserva_su_mensaje`.

Fuera de alcance confirmado

`out_of_scope` se cumplió sin excepciones: no hay `k(T)`, ni paso de tiempo adaptativo, ni amortiguamiento de Rayleigh (blindado por `test_no_acepta_rayleigh`), ni acoplamiento termomecánico.

14.9.5 Diálogo

Puntos abiertos para el usuario. Sólo física y alcance.
2026-08-25 — IA → usuario. Un punto:

1. ~~**Valor por defecto de θ.**~~ ✓ **Resuelto 2026-08-25** — $\theta = 1$ **(Euler implícito)**. El usuario delegó la decisión en la IA por ser materia de criterio técnico; queda razonada aquí para que sea auditable.

La disyuntiva. Crank-Nicolson ($\theta = 1/2$) es de orden 2 y más preciso para un mismo Δt; Euler implícito ($\theta = 1$) es de orden 1 pero **L-estable**. Ante el arranque típico —un escalón de temperatura en la frontera— Crank-Nicolson produce oscilaciones que pueden dar **temperaturas fuera del rango de los datos**, violando el principio del máximo.

El criterio que decide no es el orden de convergencia sino qué debe hacer el programa cuando el usuario no elige: un **default protege a quien no eligió**. Los dos modos de fallo no son simétricos — un resultado impreciso se detecta refinando el paso y se corrige; un resultado *imposible* desconcierta, y quien no conozca la teoría de A-estabilidad frente a L-estabilidad no tiene forma de diagnosticarlo. La precisión perdida es recuperable por el usuario; la confianza en un resultado cualitativamente absurdo, no.

Coherencia interna: es el mismo criterio que fijó `lumped` como default de la matriz de capacidad, aplicado ahora al eje temporal en vez del espacial. Ambos atacan el mismo fenómeno —oscilación espuria ante frente abrupto— y que apunten en la misma dirección hace el sistema predecible.

El default no encierra a nadie: Crank-Nicolson queda a un parámetro de distancia para quien sepa que su problema es suave y busque orden 2.

Requisito añadido con la decisión: el solver **reporta el orden efectivo del esquema** según el θ en uso, para que el coste en precisión del default sea visible y no una penalización silenciosa. Es la contrapartida honesta de elegir robustez.

- **2026-09-22** · Auditoría global: el aviso de estabilidad condicional ($\theta < 0.5$) cita el límite correcto $\Delta t \leq$ `2/((1 - 2θ)·λ_max)`, que sólo coincide con `2/λ_max` en $\theta = 0$.

- **2026-09-30** · Auditoría de las restricciones multipunto (MPC). El solver trataba como Dirichlet todo DOF eliminado del sistema reducido, esclavos de MPC incluidos. Cuatro consecuencias, corregidas: (i) `dirichlet_func` sustituía el **g** completo, así que un esclavo atado a un maestro con Dirichlet quedaba a 0 y una MPC con término independiente perdía su salto, sin aviso; ahora `dirichlet_func` sólo da los valores de los DOF con Dirichlet declarado y las filas esclavas se reconstruyen con la MPC, $g_s(t) = g_{\mathrm{mpc},s} + \sum_d \alpha_{sd} \bar{T}_d(t)$ (§8); (ii) el aviso de `T_initial` citaba los esclavos como Dirichlet contradichos; (iii) la guarda «sin Dirichlet» dejaba pasar un modelo con sólo MPC, que es tan indeterminado en el estacionario como el fundido; (iv) el instante inicial almacenado y el aviso de `T_initial` usaban el valor declarado y no `dirichlet_func(0)` (esto último también sin MPC). El aviso de choque inicial ya no llama «A-estable» a $\theta < 1/2$: dice que la estabilidad es condicional. Sin MPC y sin `dirichlet_func` los resultados

son idénticos; con `dirichlet_func` sin MPC también, salvo el instante inicial almacenado cuando `dirichlet_func(0)` difiere del valor declarado. Blindado por `tests/test_theta_method_mpc.py` (oráculo del nodo fundido en 2D y 3D, transitorio y estacionario). **Pregunta abierta para el usuario**: la guarda se mantiene en el transitorio, donde $\mathbf{A} = \mathbf{C} + \theta\Delta t\,\mathbf{K}$ es definida positiva sin Dirichlet (para $\theta \geq 0$ con $\mathbf{C}$ definida positiva) y un cuerpo aislado calentado por una fuente es un problema bien planteado (el del criterio `balance_energetico_transitorio`), que hoy se rechaza. Recomendación de la IA: levantarla en el transitorio y mantenerla en el estacionario, donde $\mathbf{K}$ sí es singular. ✓ **Resuelta el 2026-10-20** (entrada siguiente).

- **2026-10-20 · Guarda «sin Dirichlet» retirada del transitorio, con aprobación del usuario.** El solver ya no rechaza un modelo sin temperatura impuesta: es un cuerpo aislado, bien planteado porque $\mathbf{A} = \mathbf{C} + \theta\Delta t\,\mathbf{K}$ es definida positiva por $\mathbf{C}$; deja una línea informativa en el registro para quien olvidó fijar una temperatura. El estacionario la mantiene, a cargo de la red de seguridad del `LinearSolver` (ADR 0019). Sin cambio de resultados en los modelos con Dirichlet. Esto permite por fin el criterio `balance_energetico_transitorio`, que la guarda hacía imposible y no tenía test, y añade `cuerpo_aislado_calentamiento_uniforme`: ambos en `TestCuerpoAislado` de `tests/test_theta_method.py` (2D y 3D, cuatro θ, las dos capacidades); el oráculo del nodo fundido sin Dirichlet en `tests/test_theta_method_mpc.py`. Con el código anterior fallan 34 de las 35 pruebas (la que pasa es el rechazo del estacionario).

- **2026-10-27** · Verificación por soluciones manufacturadas (ADR 0022, fase 3), sin cambios en el solver. Con fuente, flujo y Dirichlet variables en el tiempo sobre mallas distorsionadas y conductividad anisótropa, el orden temporal medido es el del esquema (1 con $\theta = 1$, 2/3 y 0; 2 con Crank-Nicolson). Con `T*` lineal en `x` y en `t` el esquema es exacto para todo θ («parcela en el tiempo»): se comprobó que esa prueba detecta la falta del término `C_fp·(ḡ_{n+1} - ḡ_n)` (error $3.5{\cdot}10^{-2}$) y una carga evaluada sólo al final del paso ($5{\cdot}10^{-3}$). Con Δt = h/2, Crank-Nicolson conserva los órdenes del espacio (2 y 1) con capacidad concentrada y consistente.

Capítulo 15

Elementos — catálogo

Referencia rápida de los elementos implementados. Una entrada por elemento y, tras los marcos, la familia de ***secciones de marco*** *que estos consumen (`FiberSection`, ADR 0021). Para detalles físicos/numéricos → código fuente.*

Ensamblaje por lotes (ADR 0014)*: los sólidos isoparamétricos 2D/3D y los térmicos declaran `BATCH_KINEMATICS` (la misma cinemática compilada que usa `compute_element_state`;* ***no lanza****: devuelve `det J` $\leq$ `0` si el jacobiano degenera y el ensamblador lanza el `ValueError` con el id), los planos `BATCH_SCALE = "thickness"` y, para el post-proceso por familia, las funciones de forma (`BATCH_SHAPE_FUNCTIONS` en los lineales; las bases de orden superior y térmica las toman de `_SHAPE_FN` / `_shape_functions`); el `Assembler` los agrupa en familias y los evalúa en un único kernel, serie o paralelo. Los estructurales 1D siguen el camino por elemento, igual que todo elemento con estado propio (`element_is_batchable` lo excluye, ADR 0020) y el elemento del módulo de usuario `discontinuities` (al final del catálogo).*

Convenciones*: `STRAIN_DIM` = dimensión Voigt esperada del material asociado (1 = axial escalar, 3 = 2D `[ε_xx, ε_yy, γ_xy]`, 6 = 3D). DOFs por nodo = `DOF_NAMES`.*

15.1 Truss2D — barra axial 2D

Elemento sólido 1D de primer orden, inmerso en el plano. Dos nodos; transmite exclusivamente fuerza axial.

15.1.1 Cinemática

- Interpolación lineal del desplazamiento entre los dos nodos.
- Deformación axial uniforme en el elemento.

15.1.2 Formulación

Cosenos directores del eje: `c = (x2-x1)/L`, `s = (y2-y1)/L`.

```
ε
     = B · u_e,           B = (1/L) ·- [c, -s, c, s]
K_e   = (E·A / L) ·ᵀ dd,   d = -[c, -s, c, s]
F_int = σ·A · d
```

15.1.3 Convenciones

- Convención de signos: σ > 0 $\Leftrightarrow$ ε > 0 (elongación).
- `STRAIN_DIM = 1`: la deformación que recibe el material es un escalar (no Voigt).
- La orientación de los nodos no afecta el resultado (`K_e` y `F_int` son invariantes bajo su permutación).

15.1.4 Parámetros

- `A` — área de la sección transversal.

15.1.5 Cargas distribuidas

- `compute_body_load(b)` reparte `b` · `A` · L_0 `/ 2` por nodo en cada componente global (forma cerrada exacta para `b` uniforme). Útil para peso propio: `b = (0, -`ρ`·g)`. Acepta también un campo `b(x)` (ADR 0022), integrado con Gauss de 4 puntos, cada componente global con las funciones lineales.

15.1.6 Régimen de validez

- Pequeñas deformaciones (|ε| $\lesssim$ 10^{-2}) y pequeños desplazamientos.
- Cargas exclusivamente axiales. Si la carga transversal sobre la barra no es despreciable → `Frame2DEuler` / `Frame2DTimoshenko`.

15.1.7 Validación

- Tests: tests/test_truss.py · `TestTruss2D`; tests/validation/test_mms_frames.py (soluciones manufacturadas, ADR 0022) y tests/test_line_elements.py (campo interior y carga como campo): con `u*` polinómico el error nodal es de redondeo (exactitud nodal, Tong 1969) y con `u*` suave, orden 2 en L^2 y 1 en energía.
- Archivo fuente: solidum/elements/truss.py · clase `Truss2D`.
- Spec: docs/specs/Truss2D.md.
- Referencia: Bathe, *Finite Element Procedures*, §4.2.1.

15.2 Truss2DCorot — armadura 2D corotacional

Barra axial 2D en régimen de **grandes desplazamientos y rotaciones con pequeña deformación** (Updated Lagrangian). Hereda de `Truss2D`; comparte DOFs, parámetros y contrato con el material.

15.2.1 Cinemática

- Longitud y cosenos directores se recalculan en configuración **corriente** en cada evaluación.
- Deformación ingenieril corotacional: ε `= (l -` L_0`)/`L_0.

15.2.2 Formulación

```
d = -[θc_, -θs_, θc_, θs_]      (dirección corriente del eje)
n = -[θs_, θ c_, θs_, -θc_]     (perpendicular al eje)
K_M   = (E·A / ₀L) · d·ᵀd
K_G   = (N / l) · n·ᵀn           (rigidez geométrica)
K_T   = K_M + K_G
F_int = N · d
```

15.2.3 Cargas distribuidas

- Heredado de `Truss2D`: `compute_body_load(b)` reparte `b · A · L₀ / 2` por nodo, evaluado en geometría de referencia (aproximación estándar para cargas conservadoras en formulaciones corotacionales).

15.2.4 Régimen de validez

- $|\varepsilon| \lesssim 10^{-2}$ (pequeña deformación axial).
- Desplazamientos y rotaciones de cualquier magnitud.
- Cargas exclusivamente axiales.
- **Fuera de alcance**: pandeo por bifurcación (se capta ablandamiento precrítico pero no el punto crítico; para snap-through usar arc-length).

15.2.5 Validación

- Tests: tests/test_truss.py · `TestTruss2DCorot` (3 tests, uno por criterio de aceptación).
- Archivo fuente: solidum/elements/truss.py · clase `Truss2DCorot`.
- Spec: docs/specs/Truss2DCorot.md.
- Referencias: Crisfield §3.3, Belytschko §4.5.

15.3 Truss3D — barra axial 3D

Elemento sólido 1D de primer orden, inmerso en el espacio tridimensional. Dos nodos articulados; transmite exclusivamente fuerza axial. Régimen estrictamente de **linealidad geométrica**.

15.3.1 Cinemática

- Interpolación lineal del desplazamiento entre los dos nodos.
- Deformación axial uniforme en el elemento.
- Configuración inicial fija: L_0, `cx`, `c_y`, `c_z` no se actualizan con los desplazamientos.

15.3.2 Formulación

Cosenos directores del eje: $c_x = (x_2-x_1)/L$, $c_y = (y_2-y_1)/L$, $c_z = (z_2-z_1)/L$.

$$
\begin{aligned}
&\varepsilon \\
&\quad = B \cdot u_e, \qquad B = (1/L) \cdot [-c_x, -c_y, -c_z, c_x, c_y, c_z] \\
&K_e = (E \cdot A / L) \cdot d^{\top} d, \quad d = -[c_x, c_y, c_z, c_x, c_y, c_z] \quad (6\times 6, \text{rango } 1) \\
&F_{int} = \sigma \cdot A \cdot d
\end{aligned}
$$

15.3.3 Convenciones

- Convención de signos: $\sigma > 0 \Leftrightarrow \varepsilon > 0$ (elongación).
- `STRAIN_DIM = 1`: la deformación que recibe el material es un escalar (no Voigt).
- La orientación de los nodos no afecta el resultado (`K_e` y `F_int` son invariantes bajo su permutación).
- Acepta nodos con 2 ó 3 coordenadas (completa con `z = 0` si la tercera falta).

15.3.4 Parámetros

- `A` — área de la sección transversal.

15.3.5 Cargas distribuidas

- `compute_body_load(b)` reparte $b \cdot A \cdot L_0 / 2$ por nodo en cada componente global (forma cerrada exacta). Útil para peso propio: `b = (0, 0, -ρ·g)`. Acepta también un campo `b(x)` (ADR 0022), integrado con Gauss de 4 puntos, cada componente global con las funciones lineales.

15.3.6 Régimen de validez

- Pequeñas deformaciones ($|\varepsilon| \lesssim 10^{-2}$), pequeños desplazamientos y pequeñas rotaciones.
- Cargas exclusivamente axiales.
- Para grandes rotaciones en 3D → `Truss3DCorot`.

15.3.7 Validación

- Tests: tests/test_truss.py · `TestTruss3D`; tests/validation/test_mms_frames.py (soluciones manufacturadas, ADR 0022) y tests/test_line_elements.py (campo interior y carga como campo), con la línea orientada al azar.
- Archivo fuente: solidum/elements/truss.py · clase `Truss3D`.
- Spec: docs/specs/Truss3D.md.
- Referencias: Bathe §4.2.1; Cook §2.3.

15.4 Truss3DCorot — barra axial 3D corotacional

Barra axial 3D en régimen de **grandes desplazamientos y rotaciones con pequeña deformación** (Updated Lagrangian). Hereda de `Truss3D`; comparte DOFs, parámetros y contrato con el material.

15.4.1 Cinemática

- Longitud y cosenos directores se recalculan en configuración **corriente** en cada evaluación.
- Deformación ingenieril corotacional: ε = `(l - L0)/L0`.
- La dirección perpendicular al eje es un **plano** (dimensión 2), no un vector único.

15.4.2 Formulación

```
d = -[xc, -c_y, -c_z, xc, c_y, c_z]        (dirección corriente del eje)
ê = (xc, c_y, c_z),   P = 3I - ê·ê^T        (proyector 3×3 al plano perpendicular)

K_M   = (E·A / 0L) · d·^Td                  (6×6, rango 1)
K_G   = (N / l) · [[P, -P], -[P, P]]        (6×6, rango 2)
K_T   = K_M + K_G
F_int = N · d
```

15.4.3 Cargas distribuidas

- Heredado de `Truss3D`: `compute_body_load(b)` reparte `b · A · L0 / 2` por nodo, evaluado en geometría de referencia.

15.4.4 Régimen de validez

- $|\varepsilon| \lesssim 10^{-2}$ (pequeña deformación axial).
- Desplazamientos y rotaciones de cualquier magnitud en el espacio.
- Cargas exclusivamente axiales.

15.4.5 Validación

- Tests: tests/test_truss.py · `TestTruss3DCorot` (3 tests, uno por criterio de aceptación; el de rigidez geométrica verifica dos direcciones transversas independientes del plano perpendicular).
- Archivo fuente: solidum/elements/truss.py · clase `Truss3DCorot`.
- Spec: docs/specs/Truss3DCorot.md.
- Referencias: Crisfield §3.3; Belytschko §4.5.

15.5 Cable2DCorot — cable 2D corotacional

Elemento 1D inmerso en el plano que modela un cable: dos nodos, transmite sólo fuerza axial de tracción. Cinemática corotacional; la unilateralidad la aporta el material (típicamente `CableMaterial1D`).

15.5.1 Cinemática

- Idéntica a una barra corotacional: longitud y cosenos directores se recalculan en configuración corriente en cada evaluación.
- Deformación ingenieril corotacional: ε `= (l - L0)/L0`, con `L0` la longitud sin tensión.
- La respuesta física de cable aparece **solo** si el material devuelve σ `= 0` en compresión.

15.5.2 Pretensión

- Parámetro opcional `L0` (longitud sin tensión, > 0); por defecto, la longitud geométrica `L` entre nodos.
- `L0 < L` $\Rightarrow$ el cable arranca pretensado con `N0 = E·A·(L - L0)/L0` y rigidez geométrica `N0/L` desde el primer paso; para una pretensión `N0` dada, `L0 = L/(1 + N0/(E·A))`. `L0 > L` $\Rightarrow$ arranca flojo.
- Masa y carga de cuerpo usan ρ`·A·L0`: el material del cable descargado.

15.5.3 Formulación

```
d = -[θc_, -θs_, θc_, θs_]
n = -[θs_, θ c_, θs_, -θc_]

K_M   = (E_t · A / ₀L) · d·ᵀd        (0 si material destensado)
K_G   = (N / l) · n·ᵀn                (0 si N = 0)
K_T   = K_M + K_G
F_int = N · d                         (0 si N = 0)
```

15.5.4 Cargas distribuidas

- `compute_body_load(b)` reparte `b · A · L0 / 2` por nodo en cada componente global, evaluado en geometría de referencia. Útil para peso propio.

15.5.5 Régimen de validez

- `|`ε`|` $\lesssim$ `10`$^{-2}$ en régimen tensado.
- Grandes desplazamientos y rotaciones en el plano.
- Cables completamente destensados aportan `K_T = 0` al sistema global: otros elementos deben garantizar la estabilidad numérica.

15.5.6 Independencia del diseño

El elemento **no hereda** de `Truss2DCorot`. La cinemática corotacional se implementa dentro de la clase para que futuras modificaciones de las armaduras no afecten a los cables, y viceversa.

15.5.7 Validación

- Tests: tests/test_cable_elements.py · `TestCable2DCorot` (4 tests de aceptación: tensado, destensado, rotación rígida, cruce por cero) y `TestPretensionLongitudSinTension` (pretensión exacta, cuerda pretensada frente a la solución analítica no lineal, cable flojo, `L0` por defecto e inválida, masa y carga de cuerpo, lectura desde YAML).
- Archivo fuente: solidum/elements/cable.py · clase `Cable2DCorot`.
- Spec: docs/specs/Cable2DCorot.md.
- Referencias: Crisfield §3.3; Irvine §1.2.

15.6 Cable3DCorot — cable 3D corotacional

Elemento 1D inmerso en el espacio que modela un cable: dos nodos, transmite sólo fuerza axial de tracción, puede rotar libremente en el espacio. Cinemática corotacional 3D; unilateralidad aportada por el material (típicamente `CableMaterial1D`).

15.6.1 Cinemática

- Idéntica en filosofía a una barra corotacional 3D: longitud y cosenos directores se recalculan en configuración corriente.
- Deformación ingenieril corotacional: ε `= (l -` L_0`)/`L_0, con L_0 la longitud sin tensión.
- La dirección perpendicular al eje es un **plano** (dimensión 2).

15.6.2 Pretensión

- Parámetro opcional `L0`, con la misma semántica que en `Cable2DCorot`: con `L0 < L` la rigidez geométrica inicial N_0`/L` actúa en las dos direcciones perpendiculares al eje.

15.6.3 Formulación

```
d = -[ₓc, -c_y, -c_z, ₓc, c_y, c_z]
ê = (ₓc, c_y, c_z),  P = ₃I - ê·êᵀ       (proyector 3×3)

K_M   = (E_t · A / ₀L) · d·ᵀd             (0 si material destensado)
K_G   = (N / l) · [[P, -P], -[P, P]]      (0 si N = 0)
K_T   = K_M + K_G
F_int = N · d                             (0 si N = 0)
```

15.6.4 Cargas distribuidas

- `compute_body_load(b)` reparte `b · A · L₀ / 2` por nodo en cada componente global, evaluado en geometría de referencia.

15.6.5 Régimen de validez

- $|\varepsilon| \lesssim 10^{-2}$ en régimen tensado.
- Grandes desplazamientos y rotaciones en el espacio.
- Cables completamente destensados aportan `K_T = 0` al sistema global.

15.6.6 Independencia del diseño

No hereda de `Cable2DCorot` ni de `Truss3DCorot`. La maquinaria cinemática 3D se implementa íntegra dentro de la clase.

15.6.7 Validación

- Tests: tests/test_cable_elements.py · `TestCable3DCorot` (4 tests de aceptación) y la parte 3D de `TestPretensionLongitudSinTension` (pretensión exacta en un cable oblicuo, cuerda pretensada con carga según −z).
- Archivo fuente: solidum/elements/cable.py · clase `Cable3DCorot`.
- Spec: docs/specs/Cable3DCorot.md.
- Referencias: Crisfield §3.3; Belytschko §4.5; Irvine §1.2.

15.7 Frame2DEuler — marco/viga 2D Euler-Bernoulli

Viga esbelta 2D con hipótesis de Euler-Bernoulli: secciones planas perpendiculares al eje neutro deformado. Dos nodos rígidamente conectados; transmite axial + cortante + momento flector.

15.7.1 Cinemática

- Interpolación lineal del desplazamiento axial, Hermite cúbica del desplazamiento transverso.
- Pequeños desplazamientos y rotaciones (régimen geométricamente lineal).
- Configuración inicial fija: `L₀, c, s, T` se calculan una vez y no se actualizan.

15.7.2 Formulación

Matriz de transformación global→local $\mathbf{T}$ (6×6) con cosenos directores del eje. Matriz local analítica:

```
K_local = [[ EA/L,       0,       0, -EA/L,       0,       0 ],
           [    0, 12EI/L³,  6EI/L²,     0,-12EI/L³,  6EI/L²],
           [    0,  6EI/L²,   4EI/L,     0, -6EI/L²,   2EI/L],
           [-EA/L,       0,       0,  EA/L,       0,       0 ],
           [    0,-12EI/L³, -6EI/L²,     0, 12EI/L³, -6EI/L²],
           [    0,  6EI/L²,   2EI/L,     0, -6EI/L²,   4EI/L]]
K_global = ᵀT · K_local · T
F_int    = ᵀT · F_int_local    (componente axial con σ·A del material)
```

15.7.3 Constitutiva: dos caminos (ADR 0021)

- `material + A, I`: la forma cerrada de arriba. Sólo materiales elásticos lineales (`IS_LINEAR_ELASTIC = True`: `Elastic1D`); un material inelástico se rechaza al construir con un mensaje que remite a la sección por fibras.
- `section + n_ip` (3 por defecto, ≥ 2): elemento en desplazamientos que integra la sección (`FiberSection`) en `n_ip` puntos de Gauss-Legendre: `K_local = Σ w_g L B_g^⊤ k_s B_g`, `F_local = Σ w_g L B_g^⊤ s_g` (pesos sobre `(0, 1)`), con `e_g = [ε₀, κ_z(x_g)]` de la interpolación de Hermite. Plasticidad y daño distribuidos fibra a fibra, con cualquier material 1D. Con fibras elásticas y `n_ip` $\geq$ `2` reproduce la forma cerrada exactamente.

15.7.4 Cargas distribuidas

- `compute_body_load(b)` distribuye `q = A · b` (carga por unidad de longitud; con sección, `A = Σ A_i`) con la fórmula consistente Hermite cúbica: axial mitad-mitad por nodo, transversal `q·L/2` en fuerza y `±q·L²/12` en momento (signo positivo en nodo i, negativo en j). Transformación local↔global vía la misma `T` del elemento. El peso propio usa `ρA` (con sección, `Σ ρ_i A_i`). Acepta también un campo `b(x)` (ADR 0022), integrado con Gauss de 4 puntos, proyectado a ejes locales y repartido con las funciones de la rigidez (axial lineal, flexión de Hermite); lo comparte `Frame2DEulerCorot`, en la geometría de referencia.

15.7.5 Masa

Sin inercia rotatoria de la sección (Euler-Bernoulli la desprecia por hipótesis): sólo depende de ρ`A` (ρ·`A`; con sección, `Σ ρ_i A_i`), no de `I`.

```
Consistente:  axial ρ AL/6 · [[2,1],[1,2]]                          en (₁u, ₂u)
              flexión ρAL/420 · [[ 156,  22L,   54, -13L],
                                 [ 22L,  4L²,  13L, -3L²],
                                 [  54,  13L,  156, -22L],-
                                 [13L, -3L², -22L,  4L²]]          en (₁v, θ₁, ₂v, θ₂)
Concentrada (HRZ): ρAL/2 en ux y uy, ρAL³/78 en rz, por nodo (diagonal también en
    globales)
M_global = ᵀT · M_local · T
```

La consistente reproduce exactamente la energía cinética de la traslación y del giro rígidos. Hasta el 2026-09-29 sumaba la inercia rotatoria `ρIL/6·[[2,1],[1,2]]` en los giros y la concentrada usaba `ρIL/2` (cinemática de Timoshenko; cambio aprobado por el usuario, spec §13).

15.7.6 Régimen de validez

- Vigas esbeltas (`L/h` $\gtrsim$ `10`); peraltadas → `Frame2DTimoshenko`.
- Pequeños desplazamientos, pequeñas rotaciones.
- Con sección, la curvatura es lineal y ε_0 constante en el elemento: una rótula plástica se reparte sobre el elemento y la carga de colapso converge por arriba, en primer orden del tamaño del elemento (hacen falta varios elementos por miembro cerca de las rótulas; ver `FiberSection`).

15.7.7 Independencia del diseño

Vive en el paquete `solidum/elements/frame/` junto a `Frame2DTimoshenko` y `Frame2DEulerCorot`, sin herencia entre ellas. La construcción de longitud, cosenos directores y matriz de transformación 6×6 se delega a `build_geometry_2d` en `solidum/elements/frame/_shared.py`, compartida con `Frame2DTimoshenko` (la versión corotacional reconstruye T desde `alpha0` por su lógica propia).

15.7.8 Validación

- Tests: tests/test_frame.py · `TestFrame2DEulerAcceptance` (3 tests de aceptación + registro, incluye flecha analítica del voladizo $PL^3/(3EI)$). Camino con sección y guarda: tests/validation/test_fiber_section_validation.py (reproducción elástica, M–κ, interacción N–M, colapso del voladizo y de la viga biempotrada) y tests/test_fiber_section.py. Masa: tests/test_frame_mass.py (matrices frente a las fórmulas, energía cinética de movimientos rígidos, frecuencias como cotas superiores de Euler-Bernoulli; también `Frame2DEulerCorot` y `Frame2DTimoshenko`). tests/validation/test_mms_frames.py (soluciones manufacturadas, ADR 0022) y tests/test_line_elements.py (campo interior y carga como campo): exactitud nodal con campos polinómicos y, con campos suaves, órdenes 2 (`u`), 4 (`v`), 3 (θ) en L^2 y 1 (ε), 2 (κ) en energía.
- Archivo fuente: solidum/elements/frame/euler.py · clase `Frame2DEuler`; constitutiva común de los marcos en solidum/elements/frame/_section.py (`FrameSectionMixin`).
- Spec: docs/specs/Frame2DEuler.md.
- Referencias: Bathe cap. 5; Cook §2.7.

15.8 Frame2DTimoshenko — marco/viga 2D Timoshenko

Viga 2D con deformación por cortante transversal explícita. Dos nodos rígidamente conectados; transmite axial + cortante + momento flector. Apropiado para vigas peraltadas o cortas (`L/h` $\lesssim$ `10`).

15.8.1 Cinemática

- Secciones planas, **no** perpendiculares al eje neutro deformado (rotación independiente de la pendiente de la elástica).
- Distorsión angular $\gamma = dv/ds - \theta$ tratada como campo independiente.
- Configuración inicial fija (régimen geométricamente lineal).

15.8.2 Formulación

Factor de corrección contra shear locking:

```
Φ
 = 12 · E · I / (G · A_s · L²),      G = E / [2(1 + ν)]
```

Matriz local con coeficientes ajustados:

```
a = 12·EI / [L³Φ(1+)],   b = 6·EI / [L²Φ(1+)]
c = Φ(4+)·EI / [LΦ(1+)], d = Φ(2-)·EI / [LΦ(1+)]

K_local = [[ EA/L,  0,  0, -EA/L,  0,  0],
           [    0,  a,  b,     0, -a,  b],
           [    0,  b,  c,     0, -b,  d],
           [-EA/L,  0,  0,  EA/L,  0,  0],
           [    0, -a, -b,     0,  a, -b],
           [    0,  b,  d,     0, -b,  c]]
```

Para $\Phi \to 0$ (viga esbelta) se recupera la matriz Euler-Bernoulli.

15.8.3 Parámetros

- `A`, `I`, `As` (área efectiva de cortante).
- `nu` (opcional): se toma de `material.nu` si está disponible; en su defecto, del parámetro del elemento con default 0.3 y aviso por consola.

15.8.4 Cargas distribuidas

- `compute_body_load(b)` con `b` constante usa la misma fórmula que `Frame2DEuler` (`qL/2`, $\pm$`qL²/12`), que es también la de sus propias funciones de forma. Un campo `b(x)` (ADR 0022) se reparte con las funciones de interdependencia del elemento (Friedman y Kosmatka 1993), que dependen de Φ y dan exactamente su rigidez (**decisión del usuario**, 2026-10-27): conservan la exactitud nodal, que con Hermite se perdería (error nodal de 12 % con 2 elementos en la prueba).

15.8.5 Masa

Con inercia rotatoria de la sección (el giro es independiente de la pendiente): la consistente de `Frame2DEuler` más ρ`IL/6·[[2,1],[1,2]]` en `(rz_i, rz_j)`; la concentrada, ρ`AL/2` traslacional y ρ`IL/2` rotacional por nodo. Se conserva hasta la etapa 2 de las fibras (ADR 0021).

15.8.6 Régimen de validez

- Vigas gruesas/peraltadas (`L/h` $\lesssim$ `10`); para esbeltas → `Frame2DEuler` (más simple y sin shear locking).
- Pequeños desplazamientos, pequeñas rotaciones.
- **Sólo material elástico lineal** (`IS_LINEAR_ELASTIC = True`, ADR 0021): un material inelástico se rechaza al construir. La sección por fibras con cortante es la etapa 2 del ADR 0021, pendiente; para plasticidad distribuida, los marcos Euler-Bernoulli con `section`.

15.8.7 Independencia del diseño

Vive en el paquete `solidum/elements/frame/`, sin herencia con las otras vigas 2D. Comparte `build_geometry_2d` con `Frame2DEuler` en `solidum/elements/frame/_shared.py` — la construcción de la matriz de transformación 6×6 ya no se duplica.

15.8.8 Validación

- Tests: tests/test_frame.py · `TestFrame2DTimoshenkoAcceptance` (convergencia a Euler en viga esbelta, axial puro, simetría); tests/validation/test_mms_frames.py (soluciones manufacturadas, ADR 0022) y tests/test_line_elements.py (campo interior y carga como campo): exactitud nodal y órdenes `u` 2, θ 3, `v` 2 en L^2 y ε 1, κ 2, γ 1 en energía. La `v` da orden 2 y no 4 porque al refinar $\Phi \propto$ `1/h`^{2} crece y el elemento aproxima `v''` con θ`'`, que difieren en γ`'`; con $\Phi \ll$ `1` da 4.
- Archivo fuente: solidum/elements/frame/timoshenko.py · clase `Frame2DTimoshenko`.
- Spec: docs/specs/Frame2DTimoshenko.md.
- Referencias: Reddy cap. 5; Bathe §5.4.

15.9 Frame2DEulerCorot — viga 2D Euler-Bernoulli corotacional

Viga 2D esbelta Euler-Bernoulli en formulación **corotacional** (Updated Lagrangian). Grandes desplazamientos y grandes rotaciones rígidas del elemento; deformaciones axiales pequeñas y rotaciones nodales deformacionales moderadas.

15.9.1 Cinemática corotacional

- Rotación rígida α_e del eje separada de las rotaciones deformacionales de las secciones $\bar{\theta}_1, \bar{\theta}_2$.
- DOFs deformacionales locales: $[u_l = l - L_0, \bar{\theta}_1, \bar{\theta}_2]$.
- Todo en configuración corriente: c, s, l, α se recalculan por evaluación.

15.9.2 Formulación

```
Rigidez local (3×3) en DOFs deformacionales:
  K_ll = diag-block([EA/₀L], [[4EI/₀L, 2EI/₀L], [2EI/₀L, 4EI/₀L]])

Matriz B (3×6) en configuración corriente acopla DOFs locales y globales.

Rigidez tangente:
  K_T = B ᵀ · K_ll · B + σK_σ
  K_ = (N/l)·z·ᵀz − ((₁M+₂M)/l²)·(r·ᵀz + z·ᵀr)

con r = −[c−,s,0,c,s,0], z = −[s,c,0,s−,c,0].
```

15.9.3 Constitutiva: dos caminos (ADR 0021)

- `material + A, I`: `K_ll` y fuerzas básicas `[N, M1, M2]` en forma cerrada. Sólo materiales elásticos lineales; un material inelástico se rechaza al construir.
- `section + n_ip`: la sección (`FiberSection`) se integra en el sistema básico corrotado, con ε_0 = `Δl/L0` y κ`_z(x) = [(6x/L0 - 4)·θ̂1 + (6x/L0 - 2)·θ̂2]/L0`; `[N, M1, M2]` y `K_ll` integrados entran en la misma cinemática corrotacional, `K_`σ incluida. Verificado a 1e-10 contra la forma cerrada con giros de sólido rígido de hasta 150° y en el arco de Bathe-Bolourchi.

15.9.4 Cargas distribuidas

- `compute_body_load(b)` evalúa la integral consistente en la configuración de referencia (misma fórmula Hermite cúbica que `Frame2DEuler`). Para grandes rotaciones con cargas conservadoras (gravedad) la diferencia frente a la integración sobre la geometría corriente es de segundo orden y se asume aceptable.

15.9.5 Masa

La de `Frame2DEuler` en la configuración de referencia (L_0, α_0): sin inercia rotatoria, consistente axial + hermítica y concentrada HRZ ($\rho A L_0/2$, $\rho A {L_0}^3/78$).

15.9.6 Régimen de validez

- `|ε_axial|` $\lesssim 10^{-2}$.
- Rotaciones rígidas del elemento ilimitadas entre commits (hasta $|\alpha_e| < \pi$ por paso).
- Rotaciones deformacionales de las secciones moderadas (`|θ̂|` $\lesssim 30°$).
- Rotaciones continuas $> \pi$ requerirían tracking de vueltas (fuera de alcance).

15.9.7 Dominios que abre

Pandeo por flexión de columnas esbeltas, post-pandeo, snap-through de arcos, brazos flexibles, cables con rigidez a flexión. Problemas que la viga lineal `Frame2DEuler` no puede atacar.

15.9.8 Independencia del diseño

Vive en el paquete `solidum/elements/frame/` junto a las otras vigas 2D, **sin herencia**: la cinemática corotacional se implementa íntegra dentro de la clase (reconstruye `T` desde `alpha0` en lugar de usar `build_geometry_2d` compartido). Sí comparte con `Frame2DEuler` y `Frame2DTimoshenko` los helpers libres de `_shared.py` para carga de cuerpo (`_frame2d_consistent_body_load`), masa (`_frame2d_consistent_mass_local`) y traducción a `ElementForces` (`_frame2d_forces_from_local`).

15.9.9 Validación

- Tests: tests/test_frame.py · `TestFrame2DEulerCorotAcceptance` (4 criterios físicos + **chequeo por diferencias finitas de $\mathbf{K}_T$ contra $\mathbf{F}_{\text{int}}$** + registro). Camino con sección: caso 8 de tests/validation/test_fiber_section_validation.py y tangente con plasticidad y giro grande en tests/test_fiber_section.py.
- Archivo fuente: solidum/elements/frame/euler_corot.py · clase `Frame2DEulerCorot`.
- Spec: docs/specs/Frame2DEulerCorot.md.
- Referencias: Crisfield §7.3; Belytschko §4.11; Wriggers cap. 4.

15.10 Frame3D — marco/viga 3D Euler-Bernoulli

Viga 3D esbelta con 6 DOFs por nodo (`ux, uy, uz, rx, ry, rz`). Transmite axial + cortante en dos planos + flexión en dos planos + torsión. Régimen geométricamente lineal.

15.10.1 Cinemática

- Secciones planas perpendiculares al eje neutro deformado (Euler-Bernoulli).
- Torsión de Saint-Venant pura (sin alabeo).
- Flexiones en ejes principales desacopladas.
- Configuración inicial fija (no hay variante corotacional 3D por ahora).

15.10.2 Ejes locales y orientación de la sección

El eje $\hat{\mathbf{x}}$ local va del nodo 1 al nodo 2. Los ejes $\hat{\mathbf{y}}, \hat{\mathbf{z}}$ de la sección se definen proyectando un **vector de referencia** (`ref_vector`) sobre el plano perpendicular al eje. Default: `[0, 0, 1]` (eje z global); fallback a `[1, 0, 0]` con advertencia si la barra es casi vertical.

15.10.3 Formulación

Matriz de transformación $\mathbf{T}$ 12×12 en bloques de $\boldsymbol{\lambda}$ (3×3). Matriz local 12×12 desacoplada:

```
Axial      (2×2) : EA/L
Torsión    (2×2) : GJ/L       con G = Eν/[2(1+)]
Flexión xy (4×4) : 12EIz/L³, 6EIz/L², 4EIz/L, 2EIz/L
Flexión xz (4×4) : 12EIy/L³, 6EIy/L², 4EIy/L, 2EIy/L  (signos invertidos)

K_global = ᵀT K_local T
```

15.10.4 Constitutiva: dos caminos (ADR 0021)

- `material` + `A, Iy, Iz, J` (+ `nu`): la forma cerrada de arriba. Sólo materiales elásticos lineales; un material inelástico se rechaza al construir.
- `section` + `n_ip`: la sección (`FiberSection`) se integra en `n_ip` puntos de Gauss-Legendre con `e = [ε₀, κ_y, κ_z]`, `κ_y = -w''` (`w' = -θ_y` por la regla de la mano derecha); la torsión es elástica y desacoplada, `T = GJ·θₓ'` con el `GJ` de la sección, obligatorio. Flexión biaxial y acoplamiento `N`–`My`–`Mz` salen de las fibras.

15.10.5 Parámetros

- Camino con material: `A`, `Iy`, `Iz` (área y momentos de inercia respecto a ejes locales y, z), `J` (constante torsional de Saint-Venant) y `nu` (opcional, del material si lo expone, sino del elemento con default 0.3).
- Camino con sección: `section` (id del bloque `sections`, con `GJ`) y `n_ip` (default 3, ≥ 2).
- `ref_vector` (opcional, default `[0, 0, 1]`), en los dos caminos: fija también la orientación de los ejes `y`, `z` en que se declaran las fibras.

15.10.6 Cargas distribuidas

- `compute_body_load(b)` proyecta `q = A · b` (con sección, `A = Σ A_i`; el peso propio usa `Σ ρ_i A_i`) sobre los ejes locales y reparte con la fórmula Hermite cúbica en ambos planos de flexión: `q · L / 2` en fuerzas, `±q · L² / 12` en momentos (con signos `+` en nodo i y `−` en nodo j para `Mz`, y signos opuestos para `My` por la regla de la mano derecha). No genera torsión (carga uniforme sin excentricidad respecto al centro de cortadura). Útil para peso propio: `b = (0, 0, -ρ·g)`. Acepta también un campo `b(x)` (ADR 0022), integrado con Gauss de 4 puntos, proyectado a ejes locales y repartido con las funciones de la rigidez.

15.10.7 Régimen de validez

- Vigas esbeltas (`L/h` $\gtrsim$ `10`).
- Pequeños desplazamientos, pequeñas rotaciones.
- Sin alabeo (secciones macizas o cerradas).
- Para grandes rotaciones en 3D: pendiente `Frame3DCorot`.

15.10.8 Masa

Bloques desacoplados en ejes locales. La flexión **no lleva inercia rotatoria** (Euler-Bernoulli); la torsión sí, con el momento polar geométrico `J_p = I_y + I_z` (no la `J` de Saint-Venant). Con sección, `ρA = Σ ρ_i A_i` y `ρJ_p = ρI_y + ρI_z`.

```
Consistente:  axial ρ     AL/6  · [[2,1],[1,2]]          en (₁u, ₂u)
              torsión ρ   J_pL/6 · [[2,1],[1,2]]         en θ(₁x, θ₂x)
              flexión xy ρAL/420 · H                      en (₁v, θ₁z, ₂v, θ₂z)   (H
                  de Frame2DEuler)
```

```
              flexión xz ρAL/420 · S·H·S, S = diag--(1,1,1,1)  en (₁w, θ₁y, ₂w, θ₂y)
Concentrada: ρ AL/2 en ux, uy, uz; ρJ_pL/2 en rx; ρAL³/78 (HRZ) en ry y rz, por nodo
```

En el plano `xz`, por `w' = −θ_y`, los términos de acoplamiento traslación-giro tienen el signo opuesto al del plano `xy`, como en la rigidez. Hasta el 2026-09-29 la flexión sumaba la inercia rotatoria ρ`I_yL/6`, ρ`I_zL/6` (concentrada ρ`I_yL/2`, ρ`I_zL/2`); la torsión no cambió.

15.10.9 Masa lumped: limitación en orientación oblicua

La masa concentrada es estrictamente diagonal en ejes locales. Tras la rotación `T^T·M·T` a globales, el bloque traslacional 3×3 por nodo permanece diagonal porque `m_t·I`$_3$ es invariante bajo SO(3), pero el bloque rotacional 3×3 queda **lleno** porque `diag(`ρ`J_pL/2, `$\rho AL^3/78$`, `$\rho AL^3/78$`)` no es escalar (los dos valores sólo coincidirían con `J_p/A = L`^{2}`/39`, imposible en una barra esbelta). `M_global` resulta entonces **bloque-diagonal por nodo** (no estrictamente diagonal). Es la limitación estándar del lumping en frames 3D (Cook-Malkus-Plesha §11.4); aceptable para Newmark/HHT pero `CentralDifferenceSolver` rechaza con `ValueError` y orienta al usuario hacia Newmark o hacia un eje del elemento alineado con un eje global.

15.10.10 Independencia del diseño

Archivo propio. No hereda de `Frame2DEuler`, `Frame2DTimoshenko` ni de los trusses 3D. Comparte con los marcos planos la constitutiva genérica de `solidum/elements/frame/_section.py` (ADR 0021) y los bloques de masa de la flexión de `solidum/elements/frame/_shared.py` (`hermite_consistent_mass`, `hrz_rotational_mass`).

15.10.11 Validación

- Tests: tests/test_frame3d.py · `TestFrame3DAcceptance` (5 criterios físicos + validación de `ref_vector` + registro). Camino con sección: tests/validation/test_fiber_section_validation.py (reproducción elástica en elemento oblicuo, M–κ en los dos ejes, interacción N–M, sección circular a 45°) y tests/test_fiber_section.py (tangente con torsión frente a diferencias finitas, torsión desacoplada, `GJ` desde el YAML). Masa: tests/test_frame_mass.py · `TestFrame3D` y energía cinética del giro rígido oblicuo. tests/validation/test_mms_frames.py (soluciones manufacturadas, ADR 0022) y tests/test_line_elements.py (campo interior y carga como campo): exactitud nodal y órdenes `u` 2, `v` y `w` 4, θ`y` y θ`z` 3 en L^2 y ε 1, κ`y` y κ`z` 2 en energía; torsión exacta (lineal, sin torsor distribuido).
- Archivo fuente: solidum/elements/frame3d.py · clase `Frame3D`.
- Spec: docs/specs/Frame3D.md.
- Referencias: Przemieniecki Tabla 11.3; Cook §2.8; Bathe cap. 5.

Familia propia, **no** de elementos: una sección es la ley constitutiva de un marco a nivel de sección transversal. Recibe las deformaciones generalizadas del eje en un punto de Gauss, `e = [`ε_0`, `κ`_z]` (2D) o `[`ε_0`, `κ`_y, `κ`_z]` (3D), y devuelve las fuerzas internas `s = [N, M_z]` o `[N, M_y, M_z]` y su tangente `k_s = ∂s/∂e`. Es a un marco lo que un material es a un sólido, con otro contrato. Registro `SectionRegistry`, sección YAML `sections`, base abstracta `Section` (solidum/core/section.py); las

specs declaran `kind: section` y el manual de referencia las reúne en el capítulo «Secciones de marco». Se declara una vez y la comparten muchos elementos; **no guarda estado**: la historia de cada punto de la sección en cada punto de Gauss vive en el elemento, con la semántica trial/commit de siempre.

La consumen `Frame2DEuler`, `Frame2DEulerCorot` y `Frame3D` con su parámetro `section`. `Frame2DTimoshenko` no la admite todavía (etapa 2 del ADR 0021).

15.11 FiberSection — sección por fibras de los marcos Euler-Bernoulli

- **Propósito**: plasticidad y daño **distribuidos en la sección**: cada fibra es un punto material uniaxial con su propio material 1D del catálogo y su propia historia, y la fluencia o el daño avanzan fibra a fibra desde las fibras extremas hacia el eje neutro. Secciones mixtas (concreto y acero) en una misma sección.

- **Cinemática** (Reglas.md §5): $\varepsilon_i = \varepsilon_0 - y_i \cdot \kappa_z$ (2D), $\varepsilon_i = \varepsilon_0 - y_i \cdot \kappa_z + z_i \cdot \kappa_y$ (3D). `M_z > 0` tracciona `y < 0`; `M_y > 0` tracciona `z > 0`. Coordenadas `(y, z)` en los ejes locales del elemento, medidas **desde el eje del elemento** (la línea de nodos), no desde el centroide: una sección excéntrica acopla `N` y `M`.

- **Fuerzas internas y tangente**: $s = \Sigma\ \sigma_i\ A_i\ w_i$, $k_s = \Sigma\ E_t,i\ A_i\ w_i\ w_i^{\top}$ con `w_i = [1, -y_i]` o `[1, z_i, -y_i]` (regla del punto medio sobre las fibras). `k_s` es exacta respecto de esa aproximación y simétrica para cualquier `E_t,i`, también negativa.

- **Materiales de fibra**: cualquiera con `STRAIN_DIM = 1` (`Elastic1D`, `Elastoplastic1D`, `IsotropicDamage1D`, `CableMaterial1D`); un material 2D o 3D se rechaza al construir. Las fibras unilaterales (`CableMaterial1D`: tirantes, tendones) se admiten mezcladas con fibras bilaterales, pero una sección hecha **sólo** de fibras unilaterales se rechaza al construir, porque perdería toda la rigidez al invertirse la deformación (decisión del usuario, 2026-09-30).

- **Discretización** (primitivas con la API de OpenSees): `RectangularPatch` (rejilla `ny` × `nz`, fibra en el centro de cada celda; `n` capas integran $EI = E \cdot b \cdot h^3/12 \cdot (1 - 1/n^2)$), `CircularPatch` (corona maciza o hueca en `n_rad` × `n_circ` sectores, fibra en el centroide del sector con su área exacta; ángulos desde `+y` hacia `+z`, primer sector en 0), `BarLayer` (`n` barras equiespaciadas de `start` a `end` inclusive), `Fiber` (fibra suelta).

- **Barras dentro de un parche** (spec §3, decisión del usuario 2026-09-30): por defecto el parche cubre también el sitio de las barras y el área cuenta dos veces, $N_b = (\sigma_s + \sigma_c) \cdot A_b$, como en OpenSees. Con `displaces: <id del material del parche>` en una `layer` o en una fibra suelta, la sección añade en cada barra una **fibra de compensación** de área `-A_b` con el material desplazado, con su propia ley e historia: el concreto queda con su área neta `A_c - ΣA_b` en rigidez, fuerzas internas, masa y área bruta, también en régimen inelástico. `displaces` debe ser el material de alguna fibra declarada y distinto del de la barra.

- **Torsión** (3D): elástica y desacoplada, $T = GJ \cdot \theta_x'$ con `GJ` declarado en la sección; obligatorio sólo si la usa un `Frame3D`. No se degrada cuando las fibras fluyen.

- **Masa y peso propio**: ρ`A = Σ ρ_i A_i`, ρ`I_z = Σ ρ_i y_i`^{2} `A_i`, ρ`I_y = Σ ρ_i z_i`^{2} `A_i`, con las fórmulas de masa consistente y concentrada de los marcos elásticos. Sin inercia rotatoria de flexión (Euler-Bernoulli), la masa sólo usa ρ`A`; ρ`I_y + ρI_z` entra en la inercia torsional de `Frame3D`. Una fibra sin `density` da el error agregado de siempre.
- **Elemento**: en desplazamientos, Gauss-Legendre con `n_ip` puntos (3 por defecto, mínimo 2; con 2 se reproduce exactamente el marco elástico en forma cerrada).
- **YAML**:

```yaml
sections:
  - id: 1
    type: FiberSection
    GJ: 1.0e6                  # sólo si la usa un Frame3D
    patches:
      - {shape: rectangle, material: 1, y: [-0.25, 0.25], z: [-0.15, 0.15], ny: 50,
         nz: 1}
      - {shape: layer, material: 2, start: [-0.2, -0.1], end: [-0.2, 0.1], n: 3, area
         : 3.1e-4, displaces: 1}
    fibers:
      - {y: 0.1, z: 0.0, area: 1.0e-4, material: 2}   # sin displaces: se suma al
         concreto
elements:
  - {id: 1, type: Frame2DEuler, nodes: [1, 2], section: 1, n_ip: 3}
```

- **Limitaciones declaradas** (spec §7-§8):
- Curvatura lineal y ε_0 constante en el elemento: una rótula plástica se reparte sobre el elemento, y la carga de colapso converge **por arriba y en primer orden del tamaño del elemento** (medido: voladizo +7.1 % con 4 elementos y +0.9 % con 32; biempotrada +35 % con 4 y +3.5 % con 32). Hacen falta varios elementos por miembro cerca de las rótulas.
- Sin deformación por cortante (etapa 2, `Frame2DTimoshenko`) y torsión no degradada.
- **Ablandamiento sin regularizar**: con fibras en ablandamiento (`IsotropicDamage1D`) el daño se localiza en uno o dos puntos de Gauss y la respuesta posterior al pico depende de la malla y de `n_ip`; deuda #29 de STATUS.md (regularización genérica).
- Rendimiento: evaluación agrupada por material con un bucle escalar sobre las fibras; los marcos no van por el camino por lotes del ADR 0014.
- **Validación**: los ocho casos del contrato en tests/validation/test_fiber_section_validation.py — reproducción elástica a 1e-10 en los tres marcos; M–κ de la sección rectangular elastoplástica frente a Neal (error 1.7e-5 `M_p` con 200 capas, cota `(5/3)/n`^{2}); interacción `M` → `M_p[1 - (N/N_p)`^{2}`]`; colapso del voladizo (`P_c = M_p/L`) y de la biempotrada (`P_c = 8M_p/L`) con mallas de 4 a 32 elementos, decreciente y dentro de la cota de análisis de límite del modelo discreto; sección circular a 45° idéntica a la uniaxial; signos de Reglas.md §5; corrotacional elástico a 1e-10 con giros grandes. Pruebas unitarias en tests/test_fiber_section.py (tangentes frente a diferencias finitas con daño en ablandamiento y plasticidad, YAML, masa, VTK; `displaces` frente a la suma a mano fibra a fibra sin doble conteo en carga, descarga y recarga, y bit a bit sin él).

- **Spec**: docs/specs/FiberSection.md. **ADR**: 0021.
- **Archivo**: solidum/sections/fiber.py; constitutiva de los marcos en solidum/elements/frame/_section.py.
- **Referencias**: Spacone, Filippou y Taucer (1996); Bathe (2014) cap. 6; Neal (1977).

Continuo plano isoparamétrico, lineal y cuadrático, con convención Voigt 2D del proyecto (`Reglas.md §5`) y `STRAIN_DIM = 3`.

15.12 Quad4 — cuadrilátero bilineal 2D

- **Propósito**: continuo 2D isoparamétrico; problema mecánico plano.
- **DOFs por nodo**: `['ux', 'uy']` · 4 nodos (antihorario) · `STRAIN_DIM = 3`.
- **Cinemática**: matriz `B(ξ, η)` calculada por mapeo isoparamétrico estándar; deformación Voigt `[ε_xx, ε_yy, γ_xy]`.
- **Integración**: por defecto Gauss 2×2 (4 puntos). Configurable vía `quadrature` desde `QuadratureRegistry`. **Aviso**: integración reducida 1×1 → riesgo de hourglassing.
- **Parámetros**: `thickness` (espesor para estado plano), `quadrature` (opcional).
- **Cargas distribuidas**: `compute_body_load(b)` integra $\int \mathbf{N}^\top \mathbf{b}\, dV$ con la regla 2×2 de la masa (también con rigidez 1×1); `compute_edge_traction(edge, t̄)` integra sobre la geometría real del borde con Gauss de 3 puntos. Las dos aceptan una constante o un campo (`b(x)`, `t(x, n)` con `n` la normal exterior; una presión es `-p·n`; ADR 0022). Tracción en globales. Bordes 0..3 según la conectividad.
- **Verificación por soluciones manufacturadas** (ADR 0022): orden 2 en L^2 y 1 en energía en un cuadrado distorsionado y en una corona, con deformación plana, esfuerzo plano y ortótropo girado (`test_mms_solid_2d.py`).
- **Salida por Gauss**: `compute_gauss_state(U)` devuelve σ, ε y coordenadas (naturales y globales) en cada punto de Gauss. Habilita el suavizado nodal del exporter VTK (`Sigma_*_nodal`).
- **Implementación**: kernels `_compute_kinematics` (interno del Quad4) y `compute_integrands` (compartido con Tri3) con `@njit` (Numba) — JIT en primera llamada. Viven en `solidum/elements/solid_2d/_shared.py`; `compute_integrands` es API pública de cinemática para elementos de usuario (ADR 0020, P6) y se importa desde `solidum.elements.solid_2d`.
- **Limitaciones**: bloqueo volumétrico con materiales casi-incompresibles ($\nu \to 0.5$); en ese régimen usar formulación mixta (no implementada).
- **Archivo**: solidum/elements/solid_2d/quad4.py

15.13 Tri3 — triángulo lineal 2D (CST)

- **Propósito**: triángulo de deformación constante; útil para transiciones de malla.
- **DOFs por nodo**: `['ux', 'uy']` · 3 nodos · `STRAIN_DIM = 3`.
- **Cinemática**: matriz `B` constante (deformación uniforme dentro del elemento).
- **Integración**: 1 punto central, peso 0.5 (área triángulo en coordenadas naturales).
- **Parámetros**: `thickness`.
- **Cargas distribuidas**: `compute_body_load(b)` integra con la regla `tri_3` (1/3 a cada nodo con b uniforme, exacta también con b lineal); `compute_edge_traction(edge, t)` integra sobre la geometría real del borde con Gauss de 3 puntos. Las dos aceptan una constante o un campo (`b(x)`, `t(x, n)` con `n` la normal exterior; una presión es `-p·n`; ADR 0022). Tracción en globales. Bordes 0..2.
- **Verificación por soluciones manufacturadas** (ADR 0022): orden 2 en L^2 y 1 en energía en un cuadrado distorsionado y en una corona, con deformación plana, esfuerzo plano y ortótropo girado (`test_mms_solid_2d.py`).
- **Salida por Gauss**: `compute_gauss_state(U)` devuelve el único σ, ε y centroide del elemento (mismo formato que Quad4).
- **Limitaciones**: shear locking severo; convergencia lenta. **Preferir** `Quad4` salvo en transiciones donde Quad4 no encaja geométricamente.
- **Implementación**: kernels `compute_kinematics_tri3` y `compute_integrands` con `@njit` en `solidum/elements/solid_2d/_shared.py`. Son API pública de cinemática para elementos de usuario (ADR 0020, P6): se importan desde `solidum.elements.solid_2d`, no desde `_shared`, que es interno.
- **Archivo**: solidum/elements/solid_2d/tri3.py

15.14 Quad8 — cuadrilátero serendípito 2D de orden 2

- **Propósito**: continuo 2D cuadrático sin shear locking severo en flexión; reproduce campos cuadráticos exactamente.
- **DOFs por nodo**: `['ux', 'uy']` · 8 nodos (4 vértices antihorarios + 4 medios de borde) · `STRAIN_DIM = 3`.
- **Funciones de forma**: serendípitas (sin término $\xi^2\eta^2$).
- **Integración**: Gauss 3×3 (9 puntos) por defecto. Reducida 2×2 disponible vía `quadrature` (con riesgo de modos espurios).

- **Cargas**: body load con la regla 3×3 de la masa; `compute_edge_traction(edge, t)` integra sobre la geometría real del borde con Gauss de 3 puntos. Las dos aceptan una constante o un campo (`b(x)`, `t(x, n)` con `n` la normal exterior; una presión es `-p·n`; ADR 0022). Tracción en globales. Con tracción uniforme sobre un borde recto con el nodo medio centrado reparte 1/6, 4/6, 1/6; con el nodo descentrado o el borde curvo, lo que da la geometría (hasta el 2026-10-27 se aplicaba siempre el 1-4-1 sobre la cuerda, deuda #13).
- **Verificación por soluciones manufacturadas** (ADR 0022): orden 3 en L^2 y 2 en energía, también con la regla 2×2, en un cuadrado distorsionado y en una corona, con deformación plana, esfuerzo plano y ortótropo girado (`test_mms_solid_2d.py`).
- **Salida por Gauss**: `compute_gauss_state(U)` con 9 puntos por defecto.
- **Spec**: docs/specs/Quad8.md.
- **Archivo**: solidum/elements/solid_2d/quad8.py (subclase de la base interna `_HigherOrderSolid2D` definida en `_shared.py`, compartida con Quad9 y Tri6).

15.15 Quad9 — cuadrilátero Lagrangiano 2D de orden 2

- **Propósito**: continuo 2D cuadrático con espacio polinómico completo Q_2; añade un noveno nodo central interior al Quad8.
- **DOFs por nodo**: `['ux', 'uy']` · 9 nodos · `STRAIN_DIM = 3`.
- **Funciones de forma**: producto tensorial Lagrange 1D-1D.
- **Integración**: Gauss 3×3.
- **Cargas y salida por Gauss**: idénticas a Quad8 (el nodo 8 es interior y no participa en bordes).
- **Verificación por soluciones manufacturadas** (ADR 0022): orden 3 en L^2 y 2 en energía en un cuadrado distorsionado y en una corona, con deformación plana, esfuerzo plano y ortótropo girado (`test_mms_solid_2d.py`).
- **Spec**: docs/specs/Quad9.md.
- **Archivo**: solidum/elements/solid_2d/quad9.py (subclase de la base interna `_HigherOrderSolid2D`, compartida con Quad8 y Tri6).

15.16 Tri6 — triángulo 2D cuadrático completo P_2

- **Propósito**: triángulo isoparamétrico cuadrático que cura el shear locking del Tri3; reproduce campos cuadráticos exactamente.
- **DOFs por nodo**: `['ux', 'uy']` · 6 nodos (3 vértices + 3 medios de borde) · `STRAIN_DIM = 3`.
- **Integración**: 3 puntos en los puntos medios (cuadratura `tri_3`).

- **Cargas**: body load con la regla `tri_6` de la masa; `compute_edge_traction(edge, t)` integra sobre la geometría real del borde con Gauss de 3 puntos. Las dos aceptan una constante o un campo (`b(x)`, `t(x, n)` con `n` la normal exterior; una presión es `-p·n`; ADR 0022). Tracción en globales.
- **Verificación por soluciones manufacturadas** (ADR 0022): orden 3 en L^2 y 2 en energía en un cuadrado distorsionado y en una corona, con deformación plana, esfuerzo plano y ortótropo girado (`test_mms_solid_2d.py`).
- **Spec**: docs/specs/Tri6.md.
- **Archivo**: solidum/elements/solid_2d/tri6.py (subclase de la base interna `_HigherOrderSolid2D` en `_shared.py`, compartida con Quad8 y Quad9; declara `_MASS_QUADRATURE = "tri_6"` para integrar exactamente la masa consistente, que la cuadratura del elemento subintegraría).

Sólidos tridimensionales con convención Voigt 6D del proyecto (`Reglas.md §5`): '`[ε_xx, ε_yy, ε_zz, γ_xy, γ_yz, γ_xz] con γ_ij = 2·ε_ij`'. Esta familia **no expone** `internal_forces` — la primitiva natural de salida en sólidos es '`compute_gauss_state(U), que devuelve {stress, strain, points_*}`' por punto de integración (ADR 0012: cierre del contrato `internal_forces` por dominio explícito).

15.17 Hex8 — hexaedro trilineal 3D

- **Propósito**: sólido 3D isoparamétrico de primer orden, espejo natural de Quad4.
- **DOFs por nodo**: `['ux', 'uy', 'uz']` · 8 nodos (orden VTK_HEXAHEDRON) · `STRAIN_DIM = 6` · `N_INTEGRATION_POINTS = 8` (default Gauss 2×2×2).
- **Cinemática**: matriz `B(ξ, η, ζ)` 6×24 por mapeo isoparamétrico estándar; deformación Voigt 3D.
- **Integración**: por defecto Gauss 2×2×2 (8 puntos). Configurable vía `quadrature` (`hex_3x3x3` para no-lineales severos, `hex_1x1x1` reducido con riesgo de hourglass).
- **Caras (ADR 0012)**: 6 caras numeradas con normal saliente — 0 ($-\zeta$), 1 ($+\zeta$), 2 ($-\eta$), 3 ($+\xi$), 4 ($+\eta$), 5 ($-\xi$). Ver `Hex8.FACE_NODES`.
- **Cargas distribuidas**: `compute_body_load(b)` integra $\int$ `N`$^\top$`b dV` con la regla 2×2×2 de la masa (también con rigidez 1×1×1); `compute_face_traction(face, t)` integra sobre la cara bilineal real con Gauss 2×2. Las dos aceptan una constante o un campo (`b(x)`, `t(x, n)` con `n` la normal exterior de la cara; una presión es `-p·n`; ADR 0022); `t` en globales.
- **Verificación por soluciones manufacturadas** (ADR 0022): orden 2 en L^2 y 1 en energía en un cubo distorsionado y en un sector de cilindro hueco (`test_mms_solid_3d.py`).
- **Salida por Gauss**: `compute_gauss_state(U)` devuelve `{points_natural, points_global, strain (n_g, 6), stress (n_g, 6)}`. Sin `internal_forces` (ADR 0012).

- **Implementación**: kernels `_compute_kinematics_hex8`, `_det_jacobian_hex8`, `_shape_functions_hex8` con `@njit` en `solidum/elements/solid_3d/_shared.py`. Mass lumping vía `lump_hrz` con `n_translational_dirs=3` y `_expand_scalar_mass_3d`.
- **Limitaciones declaradas**:
- **Locking volumétrico** con $\nu \to 0.5$ — sin mitigación (política idéntica al Quad4 2D). Blindado por `tests/test_volumetric_locking_3d.py`.
- **Hourglass** con integración reducida `hex_1x1x1`: 12 modos espurios; sin estabilización Flanagan-Belytschko.
- **Shear locking** con malla coarse 1 capa en sección: documentado en `tests/validation/test_macneal_beam_3d.py`.
- **Spec**: docs/specs/Hex8.md.
- **Archivo**: solidum/elements/solid_3d/hex8.py.

15.18 Hex20 — hexaedro serendípito 3D de orden 2 (20 nodos)

- **Propósito**: sólido 3D isoparamétrico de orden cuadrático con interpolación serendípita; análogo 3D natural del `Quad8`. Reproduce campos cuadráticos completos en 3D y mitiga drásticamente el shear locking del `Hex8` en problemas de flexión y geometrías curvas.
- **DOFs por nodo**: `['ux', 'uy', 'uz']` · 20 nodos (orden VTK_QUADRATIC_HEXAHEDRON: 8 vértices + 12 medios de arista, sin nodos de cara ni centroide) · `STRAIN_DIM = 6` · `N_INTEGRATION_POINTS = 27` (default Gauss 3×3×3).
- **Cinemática**: matriz `B(ξ, η, ζ)` 6×60 sobre Voigt 6D del proyecto (ADR 0012). Funciones de forma serendípitas: vértices con multiplicador `(ξ_i ξ + η_i η + ζ_i ζ − 2)` que anula la función en todos los demás nodos; medios `(1 − α²)` con α el eje del cual son punto medio.
- **Integración**: por defecto Gauss 3×3×3 (27 puntos, exacta para K con Jacobiano constante y para masa consistente cuadrática). Configurable vía `quadrature` (`hex_2x2x2` reducida con 6 modos de hourglass espurios por elemento aislado — sin estabilización implementada).
- **Caras (ADR 0012)**: 6 caras numeradas con normal saliente, **8 nodos por cara** (4 vértices + 4 medios). Numeración paritaria con `Hex8` en los vértices; cada cara añade los 4 medios en el mismo orden cíclico (medio entre vértice k y k+1). Ver `Hex20.FACE_NODES`.
- **Cargas distribuidas**: `compute_body_load(b)` integra $\int$ `N`$^\top$`b dV` con la regla 3×3×3 de la masa (reparto **no uniforme** entre vértices y medios serendípitos, suma global `b·V_e` exacta). `compute_face_traction(face, t)` integra sobre la cara serendípita Quad8 real con Gauss 3×3: con tracción uniforme sobre cara plana, cada vértice recibe `−A·t/12` y cada medio `4·A·t/12` (fenómeno serendípito conocido, Cook-Malkus-Plesha-Witt §6.5). Las dos aceptan una constante o un campo (`b(x)`, `t(x, n)` con `n` la normal exterior de la cara; una presión es `−p·n`; ADR 0022); `t` en globales.

- **Verificación por soluciones manufacturadas** (ADR 0022): orden 3 en L^2 y 2 en energía en un cubo distorsionado y en un sector de cilindro hueco (`test_mms_solid_3d.py`), también con 2×2×2 (que, con lados curvos, no pasa la prueba de la parcela exactamente; sin efecto en el orden).
- **Salida por Gauss**: `compute_gauss_state(U)` devuelve `{points_natural, points_global, strain (n_g, 6), stress (n_g, 6)}`. Sin `internal_forces` (ADR 0012).
- **Masa**: consistente con cuadratura `hex_3x3x3` **fija** (independiente de la cuadratura elegida para K, para que la masa siempre quede exacta). Lumped por HRZ canónico — para Hex20, row-sum produciría masas negativas en vértices serendípitos (Bathe FEP §9.2.4); HRZ es la única opción razonable.
- **Implementación**: funciones de forma y derivadas en `_N_hex20`/`_dN_hex20` (numpy puro, no `@njit` por la complejidad de la rama por tipo de nodo) en `solidum/elements/solid_3d/_shared.py`. Kinematics genérico de orden superior 3D en `_kinematics_higher_order_3d` (también `_shared.py`), reutilizable por Hex27 y Tet10 en sub-fases siguientes. Cara serendípita integrada con `_N_quad8`/`_dN_quad8` importados del paquete 2D.
- **Limitaciones declaradas**:
- **Locking volumétrico** con $\nu \to 0.5$ **atenuado** respecto al `Hex8` pero aún presente — sin mitigación implementada (política idéntica a `Quad8`/`Hex8`). Blindado por `tests/test_volumetric_locking_3d.py::TestHex20VolumetricLocking` (ratio u(0.4999)/u(0.3) ≈ 0.80 vs < 0.6 en `Hex8`).
- **Hourglass** con integración reducida `hex_2x2x2`: 6 modos espurios por elemento aislado; en mallas ensambladas con BC razonable los modos no propagan, pero un elemento aislado o capa pobre los mostraría. Sin estabilización.
- **Shear locking drásticamente mitigado** respecto al `Hex8` en problemas de flexión: cuantificado en `tests/validation/test_macneal_beam_3d.py` (Hex20 6×1×1 alcanza 97 % de u_EB, vs < 55 % de Hex8 12×1×1).
- **Validación externa**: NAFEMS LE10 thick plate pressure (`tests/validation/test_nafems_le10.py`, 6×6×2 → σ_yy(D) ≈ −5.38 MPa canónico dentro de banda 10 %). **Lamé thick cylinder 3D** (`tests/validation/test_lame_cylinder_3d.py`, 4×4×1 → error L^2-rel < 4 % en σ y < 2 % en u_r contra solución cerrada de Timoshenko-Goodier §28; convergencia h monótona del error $L^2(\sigma_{\theta\theta})$ verificada). Es la primera demostración cuantitativa del proyecto de **capacidad isoparamétrica curva** — los mid-edges quedan automáticamente sobre la superficie cilíndrica por la no-linealidad cos/sin del mapeo angular.
- **Spec**: docs/specs/Hex20.md.
- **Archivo**: solidum/elements/solid_3d/hex20.py.

15.19 Hex27 — hexaedro Lagrangiano 3D triquadrático (27 nodos)

- **Propósito**: sólido 3D isoparamétrico Lagrangiano completo de orden cuadrático; análogo 3D del `Quad9`. Reproduce **todos** los polinomios triquadráticos ‘`ξ^a η^b ζ^c con a, b, c ∈ {0, 1, 2}, incluyendo los términos puramente triquadráticos (ξ²η²ζ² y combinaciones) que faltan en el` Hex20 `serendípito. Se prefiere sobre` Hex20 `cuando la geometría tiene curvatura severa y se busca convergencia óptima en problemas donde el contenido espectral cubre el espacio completo; en flexión simple con geometría rectilínea` Hex27 `y` Hex20‘ dan resultados prácticamente idénticos a un coste mayor para el primero (Bathe FEP §5.3.2).
- **DOFs por nodo**: `['ux', 'uy', 'uz']` · 27 nodos (orden VTK_TRIQUADRATIC_HEXAHEDRON: 0-7 vértices, 8-19 medios de arista (paritarios con Hex20), 20-25 centros de cara en orden ‘`(-x, +x, -y, +y, -z, +z), 26 centro de cuerpo)` · STRAIN_DIM = 6 · N_INTEGRATION_POINTS = 27‘ (default Gauss 3×3×3).
- **Cinemática**: matriz `B(ξ, η, ζ)` 6×81 sobre Voigt 6D del proyecto (ADR 0012). Funciones de forma como producto tensorial de Lagrange cuadrático 1D: ‘`N_n = L_{i_n}(ξ)·L_{j_n}(η)·L_{k_n}(ζ) con i_n, j_n, k_n ∈ {-1, 0, +1}`‘.
- **Integración**: por defecto Gauss 3×3×3 (27 puntos, exacta para K y masa consistente sobre Jacobiano constante). Reducida `hex_2x2x2` disponible pero **introduce 27 modos de hourglass espurios por elemento aislado** — la combinación más problemática del catálogo 3D. Recomendación operativa estricta: usar 3×3×3.
- **Caras (ADR 0012)**: 6 caras numeradas con normal saliente, **9 nodos por cara** (4 vértices + 4 medios + 1 centro de cara). Numeración paritaria con `Hex8`/`Hex20` en los vértices y medios; el centro de cara correspondiente se añade al final (índice 20-25 según la cara). Las funciones de forma 2D de cara son las del `Quad9` Lagrangiano (`_N_quad9`/`_dN_quad9` del paquete 2D).
- **Cargas distribuidas**: en la base `_SolidLoads3D`, común a los cinco sólidos 3D. `compute_body_load(b)` integra $\int$ `N`$^\top$`b dV` con la regla 3×3×3 de la masa; `compute_face_traction(face, t)` integra sobre la cara Quad9 real con Gauss 3×3. Las dos aceptan una constante o un campo (`b(x)`, `t(x, n)` con `n` la normal exterior de la cara; una presión es `-p·n`; ADR 0022); `t` en globales.
- **Verificación por soluciones manufacturadas** (ADR 0022): orden 3 en L^2 y 2 en energía en un cubo distorsionado y en un sector de cilindro hueco (`test_mms_solid_3d.py`).
- **Salida por Gauss**: `compute_gauss_state(U)` con 27 puntos. Sin `internal_forces` (ADR 0012).
- **Masa**: consistente con cuadratura `hex_3x3x3` fija (independiente de la cuadratura de K). Lumped HRZ canónico — row-sum produciría masas negativas en nodos interiores (Bathe FEP §9.2.4).
- **Implementación**: funciones de forma y derivadas en `_N_hex27`/`_dN_hex27` (numpy puro, vía productos tensoriales de los polinomios Lagrange 1D `_L_lagrange_quad`/`_dL_lagrange_quad`). Subclase de la base `_HigherOrderSolid3D` introducida con la entrada del `Hex27` (regla de los dos casos reales aplicada). El cuerpo de `hex27.py` queda como declaración de atributos.

- **Limitaciones declaradas**:
- **Coste computacional**: 81 DOFs/elemento vs 60 del `Hex20` (35 % más). Para flexión simple `Hex20` da resultados equivalentes a menor coste.
- **Locking volumétrico** con $\nu \to 0.5$ comparable al `Hex20` (ratio ≈ 0.80 vs < 0.6 del `Hex8`). Sin mitigación implementada.
- **Hourglass** con integración reducida `hex_2x2x2`: 27 modos espurios por elemento aislado (peor del catálogo 3D); en mallas estructuradas la mayoría no propaga tras ensamblar.
- **Validación externa**: NAFEMS LE10 thick plate pressure (`tests/validation/test_nafems_le10.py`, 5×5×2 → σ_yy(D) ≈ −5.38 MPa canónico dentro de banda 10 %). **Lamé thick cylinder 3D** (`tests/validation/test_lame_cylinder_3d.py`, 4×4×1 → error L^2-rel $< 4\,\%$ en σ y $< 2\,\%$ en u_r contra Timoshenko-Goodier §28; convergencia h monótona). Mid-edges automáticos sobre superficie cilíndrica por la no-linealidad del mapeo angular.
- **Spec**: docs/specs/Hex27.md.
- **Archivo**: solidum/elements/solid_3d/hex27.py.

15.20 Tet10 — tetraedro cuadrático 3D (10 nodos)

- **Propósito**: sólido 3D isoparamétrico cuadrático sobre simplex tetraédrico. Análogo 3D del `Tri6` 2D. Mitiga drásticamente el shear locking y el locking volumétrico del `Tet4` (CST 3D). Adecuado para mallas no estructuradas y geometrías complejas donde el `Hex20`/`Hex27` requeriría mapeo estructurado.
- **DOFs por nodo**: `['ux', 'uy', 'uz']` · 10 nodos (orden VTK_QUADRATIC_TETRA: 0-3 vértices idénticos al `Tet4`, 4-9 medios de arista en orden `(0-1, 1-2, 2-0, 0-3, 1-3, 2-3)`) · `STRAIN_DIM = 6` · `N_INTEGRATION_POINTS = 4` (default Stroud `tet_4`).
- **Cinemática**: matriz `B(ξ, η, ζ)` 6×30 sobre Voigt 6D del proyecto (ADR 0012). Funciones de forma en coordenadas baricéntricas `L_i`: vértices `N_i = L_i(2L_i - 1)`; medios `N_ij = 4·L_i·L_j`. Reproduce todos los polinomios cuadráticos completos en `(ξ, η, ζ)`.
- **Integración**: por defecto Stroud `tet_4` (4 puntos, orden 2 — exacta para K con Jacobiano constante, pues B es lineal en barycentric). Alternativa `tet_15` (Keast 15 puntos, orden 5) para geometrías distorsionadas. **La masa consistente usa `tet_15` fijo** (independiente de la cuadratura de K) para integrar exactamente el producto cuadrático×cuadrático y garantizar correctitud en análisis modal/transitorio.
- **Caras (ADR 0012)**: 4 caras triangulares con normal saliente, paritarias con `Tet4` en los vértices. Cada cara añade los 3 medios de arista correspondientes en orden Tri6 (vértices antihorarios, medios entre 0-1, 1-2, 2-0). Cara i opuesta al vértice i. Las funciones de forma 2D de cara son las del `Tri6` (`_N_tri6`/`_dN_tri6` del paquete 2D).
- **Cargas distribuidas**: `compute_body_load(b)` integra $\int$ `N`$^\top$`b dV` con la regla `tet_15` de la masa; reparto no uniforme entre vértices y medios, suma global `b·V_e` exacta. `compute_face_`

`traction(face, t)` integra sobre la cara Tri6 real con `tri_6`. Las dos aceptan una constante o un campo (`b(x)`, `t(x, n)` con `n` la normal exterior de la cara; una presión es `-p·n`; ADR 0022); `t` en globales.

- **Verificación por soluciones manufacturadas** (ADR 0022): orden 3 en L^2 y 2 en energía en un cubo distorsionado y en un sector de cilindro hueco (`test_mms_solid_3d.py`). Con `tet_4` (por omisión) y lados curvos la prueba de la parcela no es exacta (error de consistencia ~h^3.3, sin efecto en el orden); con `tet_15` sí.
- **Salida por Gauss**: `compute_gauss_state(U)` con 4 puntos por defecto.
- **Implementación**: funciones de forma y derivadas en `_N_tet10`/`_dN_tet10` (numpy puro, vía coordenadas baricéntricas) en `solidum/elements/solid_3d/_shared.py`. Subclase de la base `_HigherOrderSolid3D`. Las nuevas cuadraturas `tet_4` y `tet_15` viven en `solidum/math/integration.py`.
- **Limitaciones declaradas**:
- **Locking volumétrico** con $\nu \to 0.5$ atenuado respecto al `Tet4` pero presente; sin mitigación implementada.
- **Distorsión severa**: para Tet10 con mid-edges curvos, el Jacobiano varía espacialmente; el usuario debería seleccionar `tet_15` vía el parámetro `quadrature`.
- **Superficie curva con descomposición ingenua hex→5tets**: no converge. La cara del tet que toca una superficie curva tiene 3 mid-edges; con descomposición genérica de cada hex en 5 tets, sólo 1 de los 3 mid-edges está sobre una arista del grid paramétrico (curva), los otros 2 corresponden a aristas diagonales del hex que se vuelven aristas del tet con mid-edges rectos. La representación isoparamétrica se pierde y el error σ_rr **crece con refinamiento h** (ver `tests/validation/test_lame_cylinder_3d.py` — tests Tet10 con `@pytest.mark.skip` y motivo). Para validar Tet10 sobre superficie curva se requiere mesher tetraédrico nativo (gmsh API) o descomposición específica del dominio en tets cuyas aristas estén todas sobre el grid paramétrico. Deuda técnica #8 en STATUS. La capacidad del Tet10 sobre geometría plana está validada en `test_cube_lame_3d.py` (cubo Lamé exacto a precisión máquina), así que NO es fallo del elemento.
- **Validación externa**: cubo Lamé 3D tracción uniaxial sobre malla 5-Tet (exacto a precisión máquina, `test_cube_lame_3d.py`). Superficie curva diferida — ver limitación arriba.
- **Spec**: docs/specs/Tet10.md.
- **Archivo**: solidum/elements/solid_3d/tet10.py.

15.21 Tet4 — tetraedro lineal 3D (CST 3D)

- **Propósito**: sólido 3D isoparamétrico de primer orden, espejo natural de Tri3. Útil para mallas no estructuradas y transiciones.

- **DOFs por nodo**: `['ux', 'uy', 'uz']` · 4 nodos (orden VTK_TETRA, volumen positivo) · `STRAIN_DIM = 6` · `N_INTEGRATION_POINTS = 1`.
- **Cinemática**: `B` constante 6×12 (deformación uniforme dentro del elemento — CST 3D). Reproduce campos lineales exactamente.
- **Integración**: 1 punto baricéntrico con peso 1/6 (exacto por linealidad de B).
- **Caras (ADR 0012)**: 4 caras triangulares numeradas con normal saliente; cara `i` opuesta al nodo `i`. Ver `Tet4.FACE_NODES`.
- **Cargas distribuidas**: `compute_body_load(b)` integra con la regla `tet_4` (`b·V_e/4` a cada nodo con b uniforme, exacta también con b lineal); `compute_face_traction(face, t̄)` integra con `tri_3` (`A_cara·t̄/3` a cada nodo de la cara con $\bar{t}$ uniforme, cero al opuesto). Las dos aceptan una constante o un campo (`b(x)`, `t(x, n)` con `n` la normal exterior de la cara; una presión es `-p·n`; ADR 0022); `t̄` en globales.
- **Verificación por soluciones manufacturadas** (ADR 0022): orden 2 en L^2 y 1 en energía en un cubo distorsionado y en un sector de cilindro hueco (`test_mms_solid_3d.py`); converge al orden asintótico desde abajo y despacio, por eso sus niveles son n = 4, 8, 16.
- **Salida por Gauss**: `compute_gauss_state(U)` con 1 punto (centroide); σ y ε constantes sobre el elemento.
- **Implementación**: kernel `_compute_kinematics_tet4` con `@njit` en `_shared.py`. Masa consistente analítica `M_ij = ρ·V_e·(1+δ_ij)/20`; lumped HRZ canónico (= row-sum por simetría).
- **Limitaciones declaradas**:
- **Shear locking severo** en flexión — peor que Hex8 por la pobreza del espacio lineal. Recomendación: usar Hex8 en mallas hexaédricas; Tet4 para transiciones de malla no estructurada o cuando geometría compleja impide hexaedros.
- **Locking volumétrico** con $\nu \to 0.5$ — aún peor que en Hex8.
- **Spec**: docs/specs/Tet4.md.
- **Archivo**: solidum/elements/solid_3d/tet4.py.

Familia de **conducción de calor**: un DOF escalar `T` por nodo, matriz de conductividad `K_e` = $\int$ `B`$^\top$ `k B dΩ` en vez de rigidez, y capacidad calorífica `C_e` = $\int$ `ρc N`$^\top$ `N dΩ` en vez de masa. La ecuación semidiscreta `C·Ṫ + K·T = F` es de **primer orden** en el tiempo — por eso el transitorio no usa Newmark sino un θ-method propio; el estacionario `K·T = F` lo resuelve el `LinearSolver` existente sin modificación.

Comparten los kernels de forma y jacobiano de sus gemelos mecánicos: la `B` térmica es literalmente el gradiente ∂`N/`∂`x`, la misma materia prima que el mecánico reordena en la matriz de Voigt. Consumen materiales de `ThermalMaterialRegistry` (ver catálogo de materiales); un material mecánico pasado por error se rechaza al construir con `TypeError` explícito.

Base compartida `_ThermalSolid` (solidum/elements/thermal/_shared.py): centraliza validación del material, bucles de Gauss, capacidad en ambos modos, fuente volumétrica y post-proceso. Las subclases sólo declaran geometría.

Modo nulo: `K_e` es **semidefinida positiva** — el campo de temperatura uniforme no produce gradiente ni flujo. Es el análogo térmico de los modos de sólido rígido: el **estacionario** `K·T = F` sólo es resoluble con al menos un Dirichlet de temperatura, y sin ninguno el `LinearSolver` lo rechaza nombrando el modo, igual que un modelo mecánico sin apoyos. El **transitorio** sí admite el cuerpo aislado: `C + θΔt·K` es definida positiva por la capacidad (ver `ThetaMethodSolver` en el catálogo de solvers).

15.22 Quad4Thermal — cuadrilátero bilineal de conducción 2D

- **Propósito**: sólido térmico 2D de primer orden; hermano del `Quad4` mecánico en geometría, cuadratura y numeración de bordes.
- **DOFs por nodo**: `['T']` · 4 nodos antihorarios · `FLUX_DIM = 2` · `N_INTEGRATION_POINTS = 4` (default Gauss 2×2).
- **Cinemática**: `B(ξ,η)` de 2×4 — el gradiente `∇T = B·T_e`, **sin notación Voigt**. El gradiente resulta bilineal por elemento.
- **Parámetros**: `thickness` (espesor de la rebanada plana [m], default 1.0 que reproduce el "por unidad de profundidad" de la literatura térmica), `quadrature` (default `"2x2"`).
- **Bordes**: `EDGE_NODES = ((0,1),(1,2),(2,3),(3,0))`, paritaria con el `Quad4` mecánico.
- **Cargas**: `compute_body_source(Q)` integra $\int$ `Q N`$^\top$ `t dΩ` con la regla de la capacidad (suma exacta `Q·A_e·t`, invariante ante distorsión); `compute_edge_flux(edge, q̄)` integra $-\int$ `q̄ N`$^\top$ `t ds` con Gauss de 3 puntos (con `q̄` constante, `-q̄·L·t/2` a cada nodo del borde). Las dos aceptan una constante o un campo: `Q(x)` y `q̄(x, n)`, con `n` la normal exterior del borde, así que un flujo de Fourier conocido es `q̄ = q·n` (ADR 0022). **Convención: `q̄ > 0` es flujo SALIENTE** del dominio (enfriamiento), coherente con la normal exterior y con `q = -k·∇T`; el signo negativo del vector viene de la forma débil, donde el término de frontera aparece como $-\int$ `w q̄ dΓ`. Un borde sin condición declarada queda **adiabático** (`q̄ = 0`), el default natural de Neumann homogéneo.
- **Capacidad**: `compute_capacity_matrix(lumping)` con `"lumped"` **por DEFAULT** — al revés que en dinámica estructural. Razón física: la capacidad consistente produce oscilaciones espurias ante un frente térmico abrupto, con temperaturas que pueden salirse del rango de los datos, violando el **principio del máximo** de la ecuación de difusión. Ambas conservan la capacidad total $\rho c \cdot$`A_e·t`.
- **Salida por Gauss**: `compute_gauss_state(T)` devuelve `{points_natural, points_global, grad_T (n_g,2), flux (n_g,2)}`. Sin `internal_forces` (cierre por dominio del ADR 0012).
- **Sin estado interno**: la conducción de Fourier es lineal y sin memoria; no se crea `ElementState`.
- **Limitaciones declaradas**:
- **Hourglass** con cuadratura reducida `"1x1"`: rango 2 frente a los 3 debidos, un modo espurio de temperatura. Sin estabilización; usar el default 2×2.

- **Sin locking de ningún tipo**: el problema escalar no tiene análogo de la incompresibilidad ni del bloqueo por cortante, así que **no hereda ninguna** de las limitaciones del `Quad4` mecánico.
- **Validación**: patch test de campo lineal (exacto en todos los Gauss), modo nulo y rango, capacidad conservada en ambos modos, cargas invariantes ante distorsión, y **pared plana end-to-end** contra el perfil lineal analítico con error 1.1e-13. **Cilindro hueco** (perfil logarítmico, Carslaw-Jaeger §7.2): error nodal 4.4e-4 relativo a `|T_b - T_a|` con 4 elementos por dirección, orden 2. **Soluciones manufacturadas** (ADR 0022) con conductividad anisótropa de tensor lleno: orden 2 en L^2 y 1 en energía en el estacionario y en el transitorio (Crank-Nicolson con `Δt = h/2`, capacidad concentrada y consistente), y el orden del esquema θ en el tiempo.
- **Spec**: docs/specs/Quad4Thermal.md.
- **Archivo**: solidum/elements/thermal/quad4_thermal.py.

15.23 Hex8Thermal — hexaedro trilineal de conducción 3D

- **Propósito**: sólido térmico 3D de primer orden; hermano del `Hex8` mecánico en geometría, cuadratura y numeración de caras.
- **DOFs por nodo**: `['T']` · 8 nodos en orden VTK_HEXAHEDRON · `FLUX_DIM = 3` · `N_INTEGRATION_POINTS = 8` (default Gauss 2×2×2).
- **Cinemática**: `B(ξ,η,ζ)` de 3×8 — el gradiente ∇`T`, sin Voigt. Matrices elementales 8×8.
- **Parámetros**: `quadrature` (default `"hex_2x2x2"`). **Sin** `thickness`: el volumen sale de la geometría, igual que en los sólidos 3D mecánicos. Es la diferencia visible más inmediata respecto al hermano 2D.
- **Caras**: `FACE_NODES` **paritaria con el** `Hex8` **mecánico** (ADR 0012) — 0($-\zeta$), 1($+\zeta$), 2($-\eta$), 3($+\xi$), 4($+\eta$), 5($-\xi$).
- **Cargas**: `compute_body_source(Q)` (suma exacta `Q·V_e`); `compute_face_flux(face, q̄)` integra $-\int \bar{q}\, N^\top \, d\Gamma$ con Gauss 2×2 sobre la geometría real de la cara, alabeada o no (con `q̄` constante en una cara plana, `-q̄·A/4` a cada uno de los 4 nodos). Las dos aceptan un campo, `Q(x)` y `q̄(x, n)`, como en 2D (ADR 0022). Misma convención de signo que en 2D: `q̄ > 0` **es saliente**.
- **Modo nulo**: `rango(K_e) = 7` de 8 — un único autovalor nulo, la temperatura uniforme.
- **Limitaciones declaradas**:
- **Hourglass** con `"hex_1x1x1"`: rango 3 frente a los 7 debidos ⇒ **4 modos espurios** de temperatura, frente al único del `Quad4Thermal` reducido. Sin estabilización; usar el default 2×2×2.
- Sin locking de ningún tipo (problema escalar).

- **Validación**: patch test lineal 3D exacto (también distorsionado), modo nulo y rango, conteo de hourglass en la reducida, flujo verificado en las 6 caras, fuente invariante ante distorsión, y **cross-check 2D↔3D**: la misma pared plana resuelta con `Quad4Thermal` y con una capa de `Hex8Thermal` con caras `z` adiabáticas da campos idénticos nodo a nodo (error 9.9e-14) y ambos coinciden con el analítico. Es el análogo térmico del cross-check 3D↔2D `plane_strain` de A.bis. **Cilindro hueco 3D** con las caras planas adiabáticas: el mismo error nodal que el 2D. **Soluciones manufacturadas** (ADR 0022) con conductividad anisótropa 3×3: orden 2 en L^2 y 1 en energía en el cubo distorsionado y en el cilindro hueco, estacionario y transitorio, y el orden del esquema θ en el tiempo.
- **Spec**: docs/specs/Hex8Thermal.md.
- **Archivo**: solidum/elements/thermal/hex8_thermal.py.

15.24 Cómo añadir un elemento nuevo

1. **Spec primero** — el usuario crea `docs/specs/<Nombre>.md` a partir de `docs/specs/_template_element.md` (especificación física + formulación + contrato YAML). Sin spec, la IA no escribe código.
2. **Scaffolding** — `/solidum-new element <Nombre>` genera archivo en `solidum/elements/`, decorador `@ElementRegistry.register` y esqueleto de test.
3. **Implementación + validación** — la IA codifica contra la spec; los tests cubren los casos de `acceptance` declarados.
4. **Catálogo** — cuando la spec pasa a `status: validated`, se añade aquí una entrada breve siguiendo el formato de arriba (la spec sigue siendo la referencia detallada).

Un elemento **no estándar** (formulación de investigación que no es FEM clásico) no entra en el programa principal: va a un **módulo de usuario** (ADR 0020, Reglas.md §4), con código en `solidum/user/<módulo>/` (scaffolding con `/solidum-new element <Nombre> --user <módulo>`), spec en `docs/user/<módulo>/specs/`, tests en `tests/user/<módulo>/` y entrada en una sección «Módulos de usuario — <módulo>» al final de este catálogo. Si guarda historia fuera de `ElementState`, es un elemento con estado propio: sobrescribe `commit_state` y `compute_global_stiffness` y no va por el camino por lotes (modelo: `CST_Embedded2D`).

Nota 2026-09-22 (auditoría global). En todos los sólidos el argumento `quadrature` acepta la clave del registro (`"2x2"`, `"hex_3x3x3"`, …) o una tupla `(points, weights)`; la regla se valida contra la familia del elemento (`resolve_quadrature`). La masa consistente y la capacidad térmica se integran siempre con la regla completa aunque `K` use integración reducida. La tolerancia del jacobiano es relativa y adimensional (`JACOBIAN_RTOL`). Los elementos estructurales 1D devuelven en `internal_forces` las fuerzas internas de extremo `F_int - f_eq` cuando hay carga distribuida (`solidum.run` pasa la carga nodal equivalente).

Capítulo 16

Modelos constitutivos — catálogo

Referencia rápida de los modelos constitutivos implementados. Una entrada por material. Para detalles del algoritmo → código fuente.

***Convenciones**: `STRAIN_DIM` = dimensión Voigt de la deformación de entrada (1 = escalar, 3 = 2D `[ε_xx, ε_yy, γ_xy]`, 6 = 3D). `PRIMARY_STATE_VAR` = variable interna que el `VtkExporter` lleva al output.*

***Estado interno (ADR 0014)**: todo material declara `STATE_SCHEMA = {nombre: forma}` con sus variables internas (`{}` sin historia); de ahí se deriva el almacenamiento por arreglos del ensamblaje por lotes. Los materiales de sólidos y el térmico declaran además un kernel puntual `@njit` (`BATCH_KERNEL`) que llama a su propio return mapping, de modo que el camino por lotes y el camino por elemento son la misma física. Los materiales 2D con plane strain aportan además `batch_out_of_plane_stress(sigma, S)`, la versión por arreglos de `out_of_plane_stress` que usa el post-proceso por familia (el default de la base evalúa punto a punto).*

***Densidad (ADR 0008)**: todos los materiales aceptan un parámetro opcional `density` (kg/m^3 en las unidades del problema). No requerido al construir — análisis estáticos sin peso propio funcionan sin declararla. Cuando un consumidor que requiere masa la pide (`Assembler.assemble_self_weight(g)` y futuras `assemble_mass_matrix`, etc.) y la encuentra como `None`, **falla con** `ValueError` listando los materiales sin densidad. Valor `0.0` declarado explícitamente cubre el caso legítimo de material sin masa física (penalty, restricción) y emite warning informativo.*

16.1 Elastic1D — elástico lineal 1D

- **Ley**: σ `=` `E` $\cdot$ ε (Hooke 1D).
- **STRAIN_DIM**: 1.
- **Parámetros**: `E` (módulo de Young).
- **Variables internas**: ninguna (sin historia).
- **Tangente**: constante = `E`.
- **Compatible con**: `Truss2D`, `Truss3D` (y sus corotacionales), los cuatro marcos con `material` + propiedades geométricas (declara `IS_LINEAR_ELASTIC = True`, ADR 0021) y como material de fibra de `FiberSection`.

- **Spec**: docs/specs/Elastic1D.md
- **Archivo**: solidum/materials/elastic.py

16.2 Elastic2D — elástico lineal 2D

- **Ley**: σ `=` `C` $\cdot$ ε, con `C` el tensor constitutivo isótropo en notación Voigt `[xx, yy, xy]`.
- **STRAIN_DIM**: 3.
- **Parámetros**: `E`, `nu`, `hypothesis` $\in$ `{'plane_stress', 'plane_strain'}`.
- **Variables internas**: ninguna.
- **Tangente**: constante = `C`.
- **Compatible con**: `Quad4`, `Tri3`, `Tri6`, `Quad8`, `Quad9` (todos los sólidos 2D con `STRAIN_DIM = 3`). Compatibilidad **verificada en los cinco**: patch test con constitutiva llena y constantes aparentes de Jones medidas sobre modelo FEM.
- **Validación**: constantes aparentes contra Jones (1999) §2.8 sobre los cinco elementos; patch test de MacNeal-Harder a $\theta \in 0°, 30°, 45°, 90°$ con σ verificado en cada punto de Gauss; regresión del ejemplo publicado vía YAML. 78 tests + 370 subtests.
- **Spec**: docs/specs/Elastic2D.md
- **Archivo**: solidum/materials/elastic_2d.py

16.3 Orthotropic2D — elástico lineal ortótropo 2D (plane stress)

- **Ley**: σ `=` `C_glob` $\cdot$ ε con `C_glob` `=` $\mathtt{T}^{\top}\cdot\mathtt{C_mat}\cdot\mathtt{T}$. Primer material **anisótropo** del catálogo.
- **STRAIN_DIM**: 3.
- **Parámetros**: `E1`, `E2`, `G12`, `nu12`, `theta` (grados, default 0.0), `density` (opcional).
- **Constantes independientes**: cuatro. `nu21` se deriva por reciprocidad $\nu_{12}/\mathtt{E_1} = \nu_{21}/\mathtt{E_2}$. `G12` **no se deriva** de E y ν como en isótropo — debe medirse.
- **Convención de Poisson** (Jones/Tsai, fijada en ADR 0013 §5): `nu12` `=` $-\varepsilon_{22}/\varepsilon_{11}$ bajo carga uniaxial en **1**. Primer índice = dirección de carga. La convención opuesta existe en la literatura y **invierte E1 y E2 sin fallo ruidoso**.
- **Orientación**: `theta` vive **en el material** — atajo consciente y desechable del ADR 0013. Válido con fibra uniforme (probetas, tiras a ángulo constante); migrará al elemento cuando aparezca un caso con orientación variable en la malla.
- **Acoplamiento tracción–cortante**: nulo en ejes principales (`theta` $\in$ `{0°, 90°}`), máximo cerca de 45°. Una tracción pura genera distorsión angular — no ocurre en ningún isótropo.

- **Admisibilidad**: `E1, E2, G12 > 0` y `|nu12| < √(E1/E2)`. **NO es el criterio isótropo**: con `E1/E2 = 20` el límite es 4.47, así que `nu12 > 0.5` es legítimo.
- **Variables internas**: ninguna.
- **Tangente**: constante = `C_glob`, simétrica y definida positiva para todo `theta`.
- **Caveat**: `plane_strain` **fuera de alcance** — requiere `E3`, `nu13`, `nu23`; el problema plano no cierra sin datos 3D.
- **Compatible con**: `Quad4`, `Tri3`, `Tri6`, `Quad8`, `Quad9` (todos los sólidos 2D con `STRAIN_DIM = 3`). Compatibilidad **verificada en los cinco**: patch test con constitutiva llena y constantes aparentes de Jones medidas sobre modelo FEM.
- **Validación**: constantes aparentes contra Jones (1999) §2.8 sobre los cinco elementos; patch test de MacNeal-Harder a $\theta \in 0°, 30°, 45°, 90°$ con σ verificado en cada punto de Gauss; regresión del ejemplo publicado vía YAML. 78 tests + 370 subtests.
- **ADR**: docs/adr/0013-orientacion-material-y-ortotropia.md
- **Spec**: docs/specs/Orthotropic2D.md
- **Archivo**: solidum/materials/orthotropic_2d.py

16.4 Elastic3D — elástico lineal isótropo 3D

- **Ley**: σ `= C` $\cdot$ ε, con `C` el tensor constitutivo isótropo 6×6 en notación Voigt 3D del proyecto `[xx, yy, zz, xy, yz, xz]` (ADR 0012, `Reglas.md §5`).
- **STRAIN_DIM**: 6.
- **Parámetros**: `E` (>0), `nu` ∈ `(-1, 0.5)`, `density` (opcional, ADR 0008).
- **Variables internas**: ninguna.
- **Tangente**: constante = `C`.
- **Hipótesis**: no aplican `plane_stress`/`plane_strain` en 3D — toda la deformación es activa.
- **Compatible con**: `Hex8`, `Tet4` (todos los sólidos 3D con `STRAIN_DIM = 6`).
- **Caveat de compatibilidad**: ABAQUS usa orden Voigt `[11, 22, 33, 12, 13, 23]` (permutación `yz` ↔ `xz` respecto al proyecto). Importar datos de ABAQUS requiere intercambiar los componentes 5↔6 una vez en el preprocesador.
- **Spec**: docs/specs/Elastic3D.md
- **Archivo**: solidum/materials/elastic_3d.py

16.5 IsotropicDamage1D — daño isótropo 1D con softening exponencial

- **Modelo**: daño escalar `d ∈ [0, DAMAGE_MAX]`, esfuerzo σ `= (1 − d) · E ·` ε. Versión 1D de `IsotropicDamage2D`.
- **Evolución del daño** (Kuhn-Tucker):
- Deformación equivalente: ε`_eq = |`ε`|`.
- Variable histórica: κ `= max(`κ`_old,` ε`_eq)`.
- Si $\kappa \leq \kappa$`_0` $\rightarrow$ `d = 0`. Si no: `d = 1 − (`κ`_0/`κ`) · exp(−`α`·(`κ `− `κ`_0))`, saturado en `DAMAGE_MAX`.
- **Tangente**: **algorítmica consistente** en carga activa con daño no saturado; **secante** `(1-d)·E` en descarga, sin daño activo o al saturar.
- Fórmula consistente: `E_tan = -(1-d)·E·`$\alpha\cdot\kappa$ — **negativa** durante toda la fase de softening (refleja la pendiente descendente σ-ε post-pico).
- Derivación: de `(1-d)·E −` $E\cdot\varepsilon\cdot(\partial$`d/`$\partial\kappa)\cdot$`sign(`$\varepsilon$`)` con ∂`d/`$\partial\kappa$ `= (1-d)(1/`κ `+` α`)` y `sign(`ε`)·`ε `= |`ε`|` $= \kappa$ en carga.
- **STRAIN_DIM**: 1 · **PRIMARY_STATE_VAR**: `'damage'` · **IS_SYMMETRIC**: `True` (la tangente escalar produce contribución elemental simétrica aunque pueda ser indefinida; el despachador algebraico ADR 0003 detecta no-PD y degrada Cholesky→LU automáticamente).
- **Parámetros**: `E`, `kappa_0` (umbral elástico), `alpha` (velocidad de degradación), `density` (opcional, ADR 0008).
- **Variables internas**: `kappa`, `damage`.
- **Admisibilidad (ADR 0006)**: `admissibility_scale = E · `κ`_0` — esfuerzo umbral inicial. Garantiza invariancia bajo cambio de unidades del check `f ≤ atol + rtol · escala`.
- **Limitaciones**: no distingue tracción/compresión en la activación del daño; control de carga global puede ser inestable tras el pico (requiere `ArcLengthSolver` para seguir la rama de softening).
- **Compatible con**: `Truss2D`, `Truss3D`, y como material de fibra de `FiberSection` (marcos Euler-Bernoulli con `section`, ADR 0021; en fibras, el ablandamiento se localiza en los puntos de Gauss sin regularizar, deuda #29 de STATUS). Los marcos con `material` + propiedades geométricas lo rechazan al construir (`IS_LINEAR_ELASTIC = False`).
- **Referencia**: ver `docs/specs/IsotropicDamage1D.md` y la hermana 2D para derivación detallada. Lemaitre & Chaboche, *Mechanics of Solid Materials*. Simó & Ju (1987, IJSS 23) marco de tangente consistente.
- **Archivo**: solidum/materials/damage_1d.py

16.6 IsotropicDamage3D — daño isótropo 3D con softening exponencial

- **Modelo**: extensión 3D de `IsotropicDamage2D`. Esfuerzo nominal σ `= (1 - d) · C_e · ε` con `C_e` 6×6 isótropa 3D; daño escalar `d` $\in$ `[0, DAMAGE_MAX]`. **Sin variantes de hipótesis** — en 3D todas las componentes son activas.

- **Deformación equivalente** (parámetro `equivalent_strain`, desde el 2026-09-29; Voigt 6D del proyecto). Las tres medidas dan `ε_eq = ε` en tracción uniaxial de esfuerzo, así que `κ_0 = f_t/E`:

- `energy` (por defecto; Simó y Ju 1987): `ε_eq = √(ε^⊤·C_e·ε / E)`. No distingue tracción de compresión.

- `mazars` (Mazars 1986): `ε_eq = √(Σ⟨ε_i⟩₊²)` sobre las deformaciones principales. Sólo dañan las extensiones: compresión uniaxial de esfuerzo `ε_eq = √2·ν·|ε|`; compresión hidrostática, sin daño.

- `modified_von_mises` (de Vree, Brekelmans y van Gils 1995), con `k = f_c/f_t` $\geq$ `1` obligatorio (con k $<$ 1 no dañaría en tracción triaxial): `ε_eq = (k-1)/(2k(1-2ν))·I₁ + 1/(2k)·√(((k-1)/(1-2ν))²·I₁² + 12k/(1+ν)²·J₂)`. Compresión uniaxial de esfuerzo `ε_eq = |ε|/k`.

- Hasta el 2026-09-29 era la norma del tensor `√(ε:ε)`, retirada por no estándar y mal calibrada (`ε_eq = ε·√(1+2ν²)` en tracción uniaxial).

- **Evolución**: κ máxima histórica (Kuhn-Tucker `κ_{n+1} = max(κ_n, ε_eq)`); ley exponencial centralizada en `solidum.materials._softening.evaluate_exponential_damage` (compartida con `IsotropicDamage1D` e `IsotropicDamage2D`); saturación a `DAMAGE_MAX`.

- **Tangente**: **algorítmica consistente** en carga activa con daño no saturado; **secante** `(1-d)·C_e` en descarga, sin daño activo ($\kappa \leq$ `κ_0`) o al saturar.

- Fórmula consistente: `C_alg = (1-d)·C_e - (1-d)·(1/κ + α) · (C_e·ε) ⊗ ∂ε_eq/∂ε`.

- `∂ε_eq/∂ε`: energética `C_e·ε/(E·ε_eq)`; Mazars `ε₊/ε_eq` (parte positiva del tensor, `Σ⟨ε_i⟩₊·n_i⊗n_i`, en Voigt con componentes tensoriales; bien definida con principales repetidas); von Mises modificado por `∂I₁/∂ε` y `∂J₂/∂ε` (ver spec §7).

- **Simétrica con `energy`** (`∂ε_eq/∂ε` ‖ `C_e·ε`, bit a bit en la implementación); no simétrica en general con `mazars` y `modified_von_mises`.

- **Relación con `IsotropicDamage2D`**: con cualquiera de las tres medidas, Damage2D es la **restricción exacta** de Damage3D — en deformación plana bajo `ε_zz = γ_yz = γ_xz = 0` y en tensión plana con `ε_zz = -ν/(1-ν)·(ε_xx + ε_yy)`: mismo `d`, mismo κ, mismas componentes en el plano y `σ_zz = 0` en tensión plana, en toda una historia de carga, descarga y recarga (`test_damage_2d_3d_consistency.py`). Test cruzado `TestIsotropicDamage3DvsPlaneStrain` a **14 decimales en `d` y** κ, 10 decimales en σ.

- **STRAIN_DIM**: 6 · **PRIMARY_STATE_VAR**: `'damage'` · **IS_SYMMETRIC**: `False` en la clase (cota conservadora); la **instancia** lo pone a `True` con `energy` (el despachador algebraico ADR 0003 lo lee de la instancia; con `mazars` y `modified_von_mises` elige LU/Pardiso).
- **Parámetros**: `E` (>0), `nu` ∈ (-1, 0.5), `kappa_0` (>0), `alpha` (>0), `equivalent_strain` ∈ `energy` (default), `mazars`, `modified_von_mises`, `k` (>0, sólo y obligatorio con `modified_von_mises`), `density` (opcional, ADR 0008).
- **Variables internas**: `kappa`, `damage`. **Sin variables tensoriales** (a diferencia de los modelos plásticos): el daño es escalar, el historial es escalar.
- **Admisibilidad (ADR 0006)**: `admissibility_scale = E · κ_0` — idéntica al caso 1D/2D, constante respecto al estado.
- **Validación**: curva σ-ε analítica bajo confinamiento con las tres medidas (`tests/validation/test_uniaxial_softening_damage_3d.py`); valores de ε`_eq` frente a una referencia independiente, tangente por diferencias finitas y camino por lotes bit a bit (`test_damage_equivalent_strain.py`).
- **Limitaciones**: un único daño escalar (el modelo de Mazars completo, con daños de tracción y compresión ponderados, queda fuera; aquí sólo su medida); sin restitución de rigidez al cerrar fisuras; sin regularización por longitud característica → mesh-dependency en régimen de ablandamiento (localización en una banda de elementos). Locking volumétrico en `Hex8`/`Tet4` con ν cercano a 0.5 → mismo régimen ya documentado para los demás materiales 3D, blindado por test del elemento.
- **Compatible con**: `Hex8`, `Tet4`.
- **Referencia**: ver `docs/specs/IsotropicDamage3D.md`. Simó & Ju (1987, IJSS 23) medida energética y tangente consistente; Mazars (1986, Eng. Fract. Mech. 25); de Vree, Brekelmans & van Gils (1995, Comput. Struct. 55); Lemaitre & Chaboche (1990); de Souza Neto, Perić & Owen (2008) §12.
- **Archivo**: solidum/materials/damage_3d.py (medidas en solidum/materials/_equivalent_strain.py)

16.7 IsotropicDamage2D — daño isótropo 2D con softening exponencial

- **Modelo**: extensión 2D de `IsotropicDamage1D`. Esfuerzo nominal σ `= (1 - d) · C_e ·` ε; daño escalar `d` ∈ `[0, DAMAGE_MAX]`.
- **Deformación equivalente** (parámetro `equivalent_strain`, desde el 2026-09-29): las mismas tres medidas que `IsotropicDamage3D`, evaluadas sobre el estado 3D asociado `[`ε`_xx, `ε`_yy, `ε`_zz, `γ`_xy, 0, 0]` con ε`_zz = -`ν`/(1-`ν`)·(`ε`_xx + `ε`_yy)` en tensión plana y ε`_zz = 0` en deformación plana:
- `energy` (por defecto): ε`_eq = `$\sqrt{(\varepsilon^{\top}\cdot \texttt{C_e}\cdot \varepsilon \texttt{ / E})}$ con la `C_e` plana del material (en tensión plana es σ`:`ε del 3D con σ`_zz = 0`).

- `mazars`: $\sqrt{(\Sigma\langle\varepsilon_i\rangle_+^2)}$ sobre las dos principales del plano (forma cerrada) y ε_zz.
- `modified_von_mises` (con `k = f_c/f_t`): función de I_1 y J_2 del estado 3D asociado.
- Las tres dan $\varepsilon_eq = \varepsilon$ en tracción uniaxial de esfuerzo (κ_0 = `f_t/E`) y el 2D es la restricción exacta de `IsotropicDamage3D` en ambas hipótesis. Hasta el 2026-09-29 era la norma del tensor $\sqrt{(\varepsilon:\varepsilon)}$ (retirada).
- **Evolución**: κ máxima histórica (Kuhn-Tucker $\kappa_n+1 = \max(\kappa_n, \varepsilon_eq)$), ley exponencial $d(\kappa) = 1 - (\kappa_0/\kappa)\cdot\exp(-\alpha(\kappa-\kappa_0))$ para $\kappa > \kappa_0$, saturado a `DAMAGE_MAX`.
- **Tangente**: **algorítmica consistente** en carga activa con daño no saturado; **secante** $(1-d)\cdot C_e$ en descarga, sin daño activo ($\kappa \leq \kappa_0$) o al saturar.
- Fórmula consistente: $C_alg = (1-d)\cdot C_e - (1-d)\cdot(1/\kappa + \alpha) \cdot (C_e\cdot\varepsilon) \otimes \partial\varepsilon_eq/\partial\varepsilon$; en Mazars y von Mises modificado la regla de la cadena por ε_zz añade $c_zz\cdot\partial\varepsilon_eq/\partial\varepsilon_zz$ a las componentes `xx` e `yy` ($c_zz = -\nu/(1-\nu)$ o 0).
- **Simétrica con** `energy`; no simétrica en general con `mazars` y `modified_von_mises`. Recupera convergencia cuadrática del Newton global frente a la secante (que daba convergencia lineal).
- **STRAIN_DIM**: 3 · **PRIMARY_STATE_VAR**: `'damage'` · **IS_SYMMETRIC**: `False` en la clase; `True` en la instancia con `energy` (ADR 0003).
- **Parámetros**: `E`, `nu`, `kappa_0`, `alpha`, `hypothesis` ∈ {`'plane_stress'` (default), `'plane_strain'`}, `equivalent_strain` ∈ `energy` (default), `mazars`, `modified_von_mises`, `k` (>0, sólo y obligatorio con `modified_von_mises`), `density` (opcional, ADR 0008).
- **Variables internas**: `kappa`, `damage`.
- **Admisibilidad (ADR 0006)**: $admissibility_scale = E \cdot \kappa_0$ — idéntica al caso 1D, el criterio de daño tiene el mismo umbral característico independientemente de la dimensión.
- **Limitación**: un único daño escalar (sin los daños separados de tracción y compresión del modelo de Mazars completo) y sin restitución de rigidez al cerrar fisuras. Sin regularización por longitud característica → mesh-dependency en régimen de ablandamiento (banda localizada en una fila de elementos).
- **Compatible con**: `Quad4`, `Tri3`, `Tri6`, `Quad8`, `Quad9`.
- **Referencia**: ver `docs/specs/IsotropicDamage2D.md`. Simó & Ju (1987, IJSS 23) medida energética y tangente consistente; Mazars (1986); de Vree, Brekelmans & van Gils (1995); Lemaitre & Chaboche (1990).
- **Archivo**: solidum/materials/damage_2d.py (medidas en solidum/materials/_equivalent_strain.py)

16.8 Elastoplastic1D — plasticidad J2 1D con endurecimiento isótropo lineal

- **Modelo**: plasticidad asociativa con criterio `f = |σ_trial| - (σ_y + H·α) ≤ 0`.
- **Algoritmo**: return mapping clásico 1D.
- Predictor elástico: `σ_trial = E · (ε - ε_p_old)`.
- Corrector: `Δγ = f / (E + H)`, `σ = σ_trial - Δγ·E·sign(σ_trial)`.
- Tangente algorítmica consistente: `E_t = E·H / (E + H)`.
- **STRAIN_DIM**: 1 · **PRIMARY_STATE_VAR**: `'alpha'`.
- **Parámetros**: `E`, `sigma_y` (fluencia inicial), `H` (módulo de endurecimiento; 0 = perfectamente plástico).
- **Variables internas**: `eps_p` (deformación plástica), `alpha` (acumulada equivalente).
- **Admisibilidad (ADR 0006)**: `admissibility_scale = σ_y + H · α` — fluencia corriente (adaptativa al endurecimiento). El check `f ≤ atol + rtol · escala` se hace contra la superficie de fluencia *en el estado de entrada del paso*.
- **Compatible con**: `Truss2D`, `Truss3D` (y sus corotacionales), y como material de fibra de `FiberSection`: plasticidad distribuida en la sección de los marcos Euler-Bernoulli con `section` (ADR 0021). Los marcos con `material` + propiedades geométricas lo rechazan al construir (`IS_LINEAR_ELASTIC = False`); hasta el 2026-09-29 lo aceptaban y su `E_t` axial escalaba toda la rigidez local, con flectores incorrectos.
- **Referencia**: Simo & Hughes, *Computational Inelasticity*, cap. 1.
- **Spec**: docs/specs/Elastoplastic1D.md
- **Archivo**: solidum/materials/plastic_1d.py

16.9 CableMaterial1D — cable axial 1D con elasticidad unilateral

- **Ley**: `σ = E·ε` si `ε > 0`; `σ = 0` si `ε ≤ 0` (sin rigidez en compresión).
- **Clasificación (Reglas §1)**: estándar, por decisión del usuario del 2026-09-30. El criterio no decidía: los textos de la lista sólo cubren la barra elástica base, pero el material de sólo tracción está en ANSYS (`LINK180`), Abaqus (`*NO COMPRESSION`) y OpenSees (`Elastic` con `Eneg = 0`).
- **STRAIN_DIM**: 1.
- **Parámetros**: `E` (módulo de Young en tracción, estrictamente positivo).
- **Variables internas**: ninguna (respuesta memoryless: la rigidez depende solo del signo instantáneo de ε).

- **Tangente**: `E_t = E` en tracción (`ε > 0`), `E_t = 0` en compresión o con el cable flojo (`ε ≤ 0`).
- **Implicación numérica**: la rigidez tangente colapsa a cero cuando el cable se afloja, lo que requiere precondicionamiento o regularización en el solver para no degenerar la matriz global. El uso típico es a través del elemento `Cable2DCorot`/`Cable3DCorot`, que detecta la situación y la maneja correctamente.
- **Compatible con**: `Cable2DCorot`, `Cable3DCorot`.
- **Referencia**: ver `docs/specs/CableMaterial1D.md`.
- **Archivo**: solidum/materials/cable_1d.py

16.10 DruckerPrager3D — plasticidad friccional cohesivo-friccional 3D

- **Modelo**: cono circular suave de Mohr-Coulomb 3D con cohesión y fricción interna. Criterio `f = √J2 + η_f·I1 - k(α) ≤ 0` con `k(α) = k_0 + H·α` (endurecimiento isótropo lineal del radio del cono `k`, con α el multiplicador plástico acumulado γ; no de la cohesión `c`). **Sin variantes de hipótesis** — en 3D todas las componentes son activas.
- **Calibraciones disponibles** (3D):
- `outer_cone` (default): η_f = 2·sin(φ)/[$\sqrt{3}$·(3 - sin(φ))], k_0 = 6·c_0·cos(φ)/[$\sqrt{3}$·(3 - sin(φ))]. Circunscribe Mohr-Coulomb en el meridiano de compresión triaxial.
- `inner_cone`: η_f = 2·sin(φ)/[$\sqrt{3}$·(3 + sin(φ))], k_0 = 6·c_0·cos(φ)/[$\sqrt{3}$·(3 + sin(φ))]. Inscribe MC en el meridiano de extensión triaxial.
- **No incluye** `plane_strain_matched` (existente en DP2D): esa calibración es 2D-only por construcción — MC en 3D depende del ángulo de Lode y no admite coincidencia circular global. El constructor rechaza explícitamente esa variante con mensaje claro.
- **Plasticidad no asociada por defecto**: ángulo de dilatancia ψ parámetro independiente, default ψ `=` φ (asociada). Para $\psi \neq \varphi$ la tangente algorítmica es **asimétrica** → el despachador algebraico ADR 0003 elige LU.
- **Algoritmo — dos ramas cerradas con detección automática**:
- **Return regular** (cone surface): $\Delta\gamma$ `= f_trial / (G + 9K·η_f·η_g + H_k)`, dirección desviadora invariante bajo carga radial. Tangente algorítmica `C_alg = K·v⊗v + 2G·(1-β)·I_dev + 4G·β·n̂⊗n̂ - (1/A)·b_g⊗b_f` con `n̂` y `b_f, b_g` en Voigt 6D tensorial. Idéntica estructura algebraica al kernel 2D, solo extendida a 6 componentes activas.
- **Return al ápice** (vértice hidrostático): $\Delta\gamma$_apex `= (I_1_trial·η_f - k(α))/(9K·η_f·η_g + H_k)`, σ_n+1 = (k/(3η_f))·I puramente hidrostático en las tres componentes diagonales. Tangente reducida `(K·H_k/(9K·η_f·η_g + H_k))·v⊗v` (sin rigidez desviadora). Piso de la tangente `K·1e-6·v⊗v` cuando `H_k/(9K·η_f·η_g + H_k) < 1e-6`, es decir, en todo retorno al ápice con `H_k = 0` (perfectamente plástico): regularización de la tangente, no cambia σ ni el estado. Con ψ `= 0` el retorno al ápice no tiene solución y lanza `ValueError`.

- Detección regular ↔ apex: si tras $\Delta\gamma$`_regular` resulta $\sqrt{J_2}$`_new < 0` → cae en apex; cambia a la fórmula apex.
- **Relación con** `DruckerPrager2D` **plane_strain**: DP2D plane_strain con `variant='outer_cone'` (o `'inner_cone'`) es la restricción de DP3D bajo ε`_zz = `γ`_yz = `γ`_xz = 0`. Test cruzado `TestDruckerPrager3DvsPlaneStrain` blindado a 10 decimales sobre σ`_xx, `σ`_yy, `σ`_xy, `α y componentes plásticas planas; las componentes 3D ausentes en plane strain (ε`^p_yz, `ε`^p_xz`) permanecen nulas en DP3D.
- **STRAIN_DIM**: 6 · **PRIMARY_STATE_VAR**: `'alpha'` · **IS_SYMMETRIC**: `False` declarativo (override por instancia a `True` cuando ψ `= `φ, asociada).
- **Parámetros** (físicos):
- `E`, `nu`: elásticos (ν ∈ `(-1, 0.5)` estricto).
- `cohesion`: cohesión inicial `c_0` (esfuerzo, >0).
- `phi_deg`: ángulo de fricción interna (grados, `0` ≤ φ `< 90`).
- `psi_deg`: ángulo de dilatancia (grados, `0` ≤ ψ ≤ φ; default φ ⇒ asociada).
- `H` (≥0, default 0): módulo de endurecimiento `dk/d`γ del radio del cono respecto del multiplicador plástico acumulado. No es `dc/d`$\bar{\varepsilon}^p$: desde la parametrización `c(`$\bar{\varepsilon}^p$`)` de de Souza Neto et al. (2008, §8.3), `H = `ξ^2`·H_c` con ξ `= k_0/c_0` de la calibración.
- `variant`: `'outer_cone'` (default) o `'inner_cone'`.
- `density` (opcional, ADR 0008).
- **Variables internas**: `eps_p` (6 componentes Voigt 6D con cortantes tensoriales `[xx, yy, zz, xy, yz, xz]`), `alpha` (multiplicador plástico acumulado, adimensional). **Invariante cinemático**: `tr(`ε`^p) = 3·`η`_g·`α en cualquier estado plástico (distinto de J2 donde `tr(`ε`^p) = 0`).
- **Admisibilidad (ADR 0006)**: `admissibility_scale = k(`α`) = k_0 + H·`α.
- **Limitaciones declaradas** (`out_of_scope` en spec): MC con aristas (pirámide hexagonal — DP no captura efectos de Lode angle, diferencia hasta 15-20 % en frontera fuera de los meridianos puros), variantes con dependencia explícita del Lode angle (Matsuoka-Nakai, Lade-Duncan), endurecimiento por fricción/dilatancia, endurecimiento cinemático, cap-cone, ablandamiento (`H<0` requiere regularización), grandes deformaciones. Locking volumétrico en `Hex8`/`Tet4` cuando ψ `> 0` (flujo dilatante) + ν moderado-alto — mismo régimen ya documentado para `Elastic3D`/`VonMises3D`, sin mitigación implementada.
- **Compatible con**: `Hex8`, `Tet4`.
- **Referencia**: ver `docs/specs/DruckerPrager3D.md`. Drucker & Prager (1952). Simó & Hughes (1998) §6 (cone plasticity, apex return). de Souza Neto, Perić & Owen (2008) §8 (calibraciones outer/inner 3D, tangentes algorítmicas detalladas).
- **Archivo**: solidum/materials/drucker_prager_3d.py

16.11 DruckerPrager2D — plasticidad friccional cohesivo-friccional 2D plane strain

- **Modelo**: cono circular suave de Mohr-Coulomb con cohesión y fricción interna. Criterio `f =` $\sqrt{J_2}$ `+` η`_f·I`$_1$ `- k(`α`) ≤ 0` con `k(`α`) = k_0 + H·`α (endurecimiento isótropo lineal del radio del cono `k`, con α el multiplicador plástico acumulado γ; no de la cohesión `c`).
- **Hipótesis**: solo `plane_strain` en esta entrega. `plane_stress` declarado *out-of-scope* (proyección con σ_zz=0 acoplada a flujo dilatante es notoriamente delicada).
- **Plasticidad no asociada por defecto**: ángulo de dilatancia ψ parámetro independiente, default ψ `=` φ (asociada). Para $\psi \neq \varphi$ la tangente algorítmica es **asimétrica** → `IS_SYMMETRIC = False`.
- **Calibración con Mohr-Coulomb** (variant):
- `plane_strain_matched` (default): η_f = tan(φ)/$\sqrt{(9 + 12\tan^2(\varphi))}$, k_0 = 3c_0/$\sqrt{(9 + 12\tan^2(\varphi))}$. Coincide exactamente con MC en plane strain.
- `outer_cone`: η_f = 2·sin(φ)/[$\sqrt{3}$·(3 - sin(φ))]. Circunscribe MC.
- `inner_cone`: η_f = 2·sin(φ)/[$\sqrt{3}$·(3 + sin(φ))]. Inscribe MC.
- **Algoritmo — dos ramas cerradas con detección automática**:
- **Return regular** (cone surface): $\Delta\gamma$ `= f_trial / (G + 9K·`η`_f·`η`_g + H)`, dirección desviadora preservada. Tangente algorítmica `C_alg = K·v⊗v + 2G·(1-`β`)·I_dev + 4G·`β`·n̂⊗n̂ - (1/A)·b_g⊗b_f` con β `= G·`$\Delta\gamma$/$\sqrt{J_2}$`_trial`, `b_g = 2G·n̂ + 3K·`η`_g·v`, `b_f = 2G·n̂ + 3K·`η`_f·v`.
- **Return al ápice** (vértice del cono, zona puramente hidrostática): $\Delta\gamma$`_apex = (`I_1`_trial·`η`_f - k(`α`))/(9K·`η`_f·`η`_g + H)`, σ_n+1 = (k/3η_f)·I puramente hidrostático. Tangente reducida a `K·H/(9K·`η`_f·`η`_g + H) · v⊗v` (simétrica, sin rigidez desviadora). Piso de la tangente `K·1e-6·v⊗v` cuando `H/(9K·`η`_f·`η`_g + H) < 1e-6`, es decir, en todo retorno al ápice con `H = 0` (perfectamente plástico): regularización de la tangente, no cambia σ ni el estado. Con ψ `= 0` el retorno al ápice no tiene solución y lanza `ValueError`.
- Detección regular ↔ apex: si tras $\Delta\gamma$`_regular` resulta $\sqrt{J_2}$`_new < 0` → cae en apex; cambia a la fórmula apex.
- **STRAIN_DIM**: 3 · **PRIMARY_STATE_VAR**: `'alpha'` · **IS_SYMMETRIC**: `False` (declarativo, conservador; el despachador algebraico ADR 0003 elige LU).
- **Parámetros** (físicos):
- `E`, `nu`: elásticos.
- `cohesion`: cohesión inicial `c_0` (esfuerzo).
- `phi_deg`: ángulo de fricción interna (grados, `0 ≤` φ `< 90`).
- `psi_deg`: ángulo de dilatancia (grados, `0 ≤` ψ `≤` φ; default φ ⇒ asociada).

- `H` (≥ 0, default 0): módulo de endurecimiento `dk/d`γ del radio del cono respecto del multiplicador plástico acumulado. No es `dc/d`$\bar{\varepsilon}^p$: desde la parametrización `c(`$\bar{\varepsilon}^p$`)` de de Souza Neto et al. (2008, §8.3), `H =` ξ^2`·H_c` con ξ `= k_0/c_0` de la calibración.
- `variant`: calibración con MC (default `'plane_strain_matched'`).
- `density` (opcional, ADR 0008).
- **Variables internas**: `eps_p` (tensorial 4 componentes `[xx, yy, zz, xy_tensorial]`), `alpha` (multiplicador plástico acumulado, adimensional).
- **Admisibilidad (ADR 0006)**: `admissibility_scale = k(`α`) = k_0 + H·`α (radio corriente del cono, unidades de esfuerzo igual que `f`).
- **Limitaciones** (out-of-scope):
- Sin `plane_stress`, endurecimiento por fricción/dilatancia, endurecimiento cinemático, cap (modelo cap-cone), ablandamiento (`H<0` requiere regularización), grandes deformaciones.
- Validación inicial cubierta por tests unitarios (incluyendo FD numérica de la tangente) y un benchmark de pipeline Quad4 (rama elástica + rama apex + caso asociado). El régimen "rama regular pura" en geometría confinada es difícil de calibrar sin caer en apex; se cubre por los tests unitarios de cortante puro.
- **Compatible con**: `Quad4`, `Tri3`, `Tri6`, `Quad8`, `Quad9` (todos con material 2D plane strain).
- **Referencia**: ver `docs/specs/DruckerPrager2D.md`. Drucker & Prager (1952). de Souza Neto, Perić & Owen (2008) cap. 8 (Drucker-Prager, tangentes algorítmicas).
- **Archivo**: solidum/materials/drucker_prager_2d.py

16.12 VonMises3D — plasticidad J2 3D con endurecimiento isótropo lineal

- **Modelo**: J2 (Von Mises) con regla de flujo asociada y endurecimiento isótropo lineal, formulado íntegramente en Voigt 6D del proyecto (ADR 0012). **Sin variantes de hipótesis** — en 3D todas las componentes de ε y σ son activas.
- **Algoritmo** (Simó-Hughes §3.3): return mapping radial cerrado único sobre la parte desviadora 6D. Predictor elástico `s_trial = 2G·(e_dev - e_p)` con presión `p = K·tr(`ε`)`; criterio `f =` $\|$`s`$\|$ `-` $\sqrt{}$`(2/3)·(`σ`_y + H·`α`)` $\leq$ `0` con norma Frobenius tensorial $\|$`s`$\|^2$ `= Σ_diag s_ii`^{2} `+ 2·Σ_off s_ij`^{2}; corrector radial $\Delta\gamma$ `= f_trial / (2G +` $\frac{2}{3}$`·H)` con `N = s_trial/`$\|$`s_trial`$\|$; actualización α`_new =` α `+` $\sqrt{}$`(2/3)·`$\Delta\gamma$. Tangente algorítmica consistente cerrada `C_alg = K·v⊗v + 2G(1-`β`)·I_dev - 2G·`$\bar{\gamma}$`·N⊗N`. Idéntica estructura al kernel `plane_strain` de `VonMises2D` ampliada a 6 componentes activas; comparte el principio físico pero el código kernel se mantiene separado por simplicidad (zero-padding sistemático en Numba sería contraproducente).

- **Relación con `VonMises2D` plane strain**: VM2D plane strain es la restricción de este modelo bajo $\varepsilon_zz = \gamma_yz = \gamma_xz = 0$ (deformación total impuesta) con $\varepsilon\hat{}p_zz$, $\varepsilon\hat{}p_yz$, $\varepsilon\hat{}p_xz$ libres por incompresibilidad/isotropía. Test cruzado `TestVonMises3DvsPlaneStrain` blindado a 10 decimales sobre σ_xx, σ_yy, σ_xy, α y componentes plásticas planas.
- **STRAIN_DIM**: 6 · **PRIMARY_STATE_VAR**: `'alpha'` · **IS_SYMMETRIC**: `True` (J2 asociado).
- **Parámetros**: `E` (>0), `nu` $\in$ `(-1, 0.5)`, `sigma_y` (>0), `H` (≥0, default 0), `density` (opcional, ADR 0008).
- **Variables internas**: `eps_p` (6 componentes Voigt 6D con cortantes tensoriales `[xx, yy, zz, xy, yz, xz]`), `alpha` (acumulada equivalente, adimensional). La parte plástica es **incompresible exacta**: `tr(`$\varepsilon\hat{}$`p) = 0` por construcción del flujo desviador.
- **Admisibilidad (ADR 0006)**: `admissibility_scale =` $\sqrt{}$`(2/3)·(`σ`_y + H·`α`)` — idéntica a plane strain (misma norma desviadora en la frontera J2).
- **Limitaciones declaradas** (`out_of_scope` en spec): no endurecimiento cinemático/Voce, no ablandamiento (`H<0` requiere regularización), no regla de flujo no asociada, no daño acoplado, no viscoplasticidad, no grandes deformaciones, no anisotropía. Locking volumétrico en `Hex8`/`Tet4` cuando la plasticidad domina + ν moderado-alto — mismo régimen ya documentado para `Elastic3D`, blindado por tests, sin mitigación implementada (B-bar/F-bar diferidas).
- **Implementación**: kernel `_compute_j2_3d` con `@njit`. Sin despacho por hipótesis (no aplica en 3D).
- **Compatible con**: `Hex8`, `Tet4` (todos los sólidos 3D con `STRAIN_DIM = 6`).
- **Referencia**: ver `docs/specs/VonMises3D.md`. Simó & Hughes, *Computational Inelasticity* (1998), §3.3 (3D J2 return mapping radial cerrado). de Souza Neto, Perić & Owen, *Computational Methods for Plasticity* (2008), §7.
- **Archivo**: solidum/materials/von_mises_3d.py

16.13 VonMises2D — plasticidad J2 2D con endurecimiento isótropo lineal

- **Modelo**: J2 (Von Mises) con regla de flujo asociada y endurecimiento isótropo lineal. Soporta **dos hipótesis cinemáticas** mutuamente excluyentes (selección por kwarg `hypothesis` al construir): `plane_strain` ($\varepsilon_zz = 0$) y `plane_stress` ($\sigma_zz = 0$). Cada hipótesis usa un kernel Numba especializado.
- **Algoritmo plane strain** (Simó-Hughes §3.3): return mapping radial cerrado sobre la parte desviadora 3D extendida. Predictor elástico desviador `s_trial = 2G·(e_dev − e_p)` con presión `p = K·tr(`ε`)`; criterio `f =` $\|$`s_trial`$\|$ `−` $\sqrt{}$`(2/3)·(`σ`_y + H·`α`)` $\leq$ `0`; corrector radial $\Delta\gamma$ `= f_trial / (2G +` $\frac{2}{3}$`·H)` con `N = s_trial/`$\|$`s_trial`$\|$; actualización α`_new =` α `+` $\sqrt{}$`(2/3)·`$\Delta\gamma$. Tangente algorítmica consistente cerrada.

- **Algoritmo plane stress** (Simó-Hughes §3.4.1, "plane stress projected algorithm"): predictor σ_trial = C_e_ps · (ε − e_p) en subespacio plane stress; función de fluencia proyectada f = ½·σ·P·σ − R²/3 con operador P = (1/3)·[[2,−1,0],[−1,2,0],[0,0,6]]. Newton local escalar sobre $\Delta\gamma$ en base de autovectores ortonormales v_1=[1,1,0]/√2, v_2=[1,−1,0]/√2, v_3=[0,0,1] de C_e_ps·P con autovalores μ_1=E/(3(1−ν)), μ_2=μ_3=2G. Converge en 3-6 iteraciones (tangente cerrada). Actualización de α con la regla físicamente correcta α_new = α + $\Delta\gamma$·√(2σPσ/3) (la heurística √(2/3)·$\Delta\gamma$ válida en plane strain rompe la invariancia bajo cambio de unidades en plane stress, donde $\Delta\gamma$ tiene unidades [1/esfuerzo]). Incompresibilidad plástica cierra e_p_zz = −(e_p_xx + e_p_yy). Tangente algorítmica consistente cerrada con corrección por dα/d$\Delta\gamma$ ≠ √(2/3).
- **STRAIN_DIM**: 3 · **PRIMARY_STATE_VAR**: 'alpha'.
- **Parámetros**: E, nu, sigma_y, H (≥0, default 0), hypothesis ∈ {'plane_strain' (default), 'plane_stress'}, density (opcional, ADR 0008).
- **Variables internas**: eps_p (tensorial 4 componentes [xx, yy, zz, xy_tensorial]), alpha (acumulada equivalente, adimensional en ambas hipótesis).
- **Admisibilidad (ADR 0006)**:
- plane_strain: admissibility_scale = √(2/3)·(σ_y + H·α) — radio desviador de la superficie J2.
- plane_stress: admissibility_scale = (σ_y + H·α)²/3 — escala de f proyectada.
- En ambos, la tolerancia se precomputa fuera del kernel @njit y se pasa como float.
- **Limitaciones declaradas** (ver out_of_scope en spec): no endurecimiento cinemático/Voce, no ablandamiento (H<0 requiere regularización), no regla de flujo no asociada, no daño acoplado, no viscoplasticidad, no grandes deformaciones, no anisotropía. Locking volumétrico en plane strain con elementos de bajo orden cuando $\nu \to$ 0.5 o dominio plástico (mitigaciones B-bar/F-bar diferidas).
- **Implementación**: kernels _compute_j2_plane_strain y _compute_j2_plane_stress con @njit. Despacho por hypothesis en construcción (sin coste runtime).
- **Compatible con**: Quad4, Tri3, Tri6, Quad8, Quad9 (configurados en cualquiera de las dos hipótesis).
- **Referencia**: ver docs/specs/VonMises2D.md. Simó & Hughes, *Computational Inelasticity* (1998), §3.3 (plane strain), §3.4 (plane stress projected, Box 3.1). de Souza Neto, Perić & Owen, *Computational Methods for Plasticity* (2008), §9.4.
- **Archivo**: solidum/materials/von_mises_2d.py

Familia paralela a los constitutivos mecánicos: relacionan el **flujo de calor** q con el **gradiente de temperatura** ∇T, no σ con ε. Ni el gradiente ni el flujo son tensores simétricos, así que **no usan notación Voigt**. Viven en ThermalMaterialRegistry y sólo los consumen elementos térmicos; se declaran en el bloque thermal_materials del YAML (la sección de su familia, paralela a materials) con su type propio. El programa principal tiene estas dos familias de material, mecánica y térmica; la cohesiva es del módulo de usuario discontinuities (al final del catálogo).

16.14 ThermalConduction — conducción de calor de Fourier con conductividad tensorial

- **Ley**: `q = -k·∇T`. Lineal y **sin variables internas** — el análogo térmico de `Elastic2D`/`Elastic3D`. El signo negativo es física (segunda ley), no convención elegible.
- **FLUX_DIM**: 2 ó 3 · **PRIMARY_STATE_VAR**: `None` · **IS_SYMMETRIC**: `True`.
- A diferencia de `STRAIN_DIM`/`JUMP_DIM`, **no es** `ClassVar`: la dimensión la fija el tensor con que se construye, así que la misma clase sirve en 2D y en 3D.
- **Conductividad tensorial**: `k` se guarda siempre como matriz. El constructor acepta **escalar** (isótropo, se expande a `k·I` con la dimensión de `dim`) o **matriz** 2×2 / 3×3 (ortótropo/anisótropo), en cuyo caso el tamaño del tensor manda sobre `dim`.
- Validada **simétrica** (relaciones recíprocas de Onsager) con tolerancia escalada `1e-12·max|k|`, adimensional respecto a las unidades del usuario (ADR 0006).
- Validada **definida positiva** por autovalores (`eigvalsh`), no por determinante: en dimensión ≥ 2 un determinante positivo no implica definición positiva.
- **Parámetros**: `k` (conductividad, W/(m·K); escalar o matriz), `c` (calor específico, J/(kg·K), opcional), `density` (kg/m^3, opcional), `dim` (2 ó 3, sólo para expandir un `k` escalar).
- **`c` y `density` opcionales** siguiendo el criterio del ADR 0008: un análisis **estacionario** sólo usa `k`, y exigir propiedades que no entran en ninguna ecuación obligaría a inventar valores. Si un consumidor transitorio las encuentra ausentes, **`ValueError` accionable** — nombra el material, los parámetros que faltan y la salida (declararlos o usar un solver estacionario).
- **Magnitud derivada**: `thermal_diffusivity` $= \alpha$ `= k/(ρc)` [m^2/s], que gobierna la velocidad del frente térmico y sirve para estimar Δ`t ~ h`$^2/\alpha$. **Rechaza el caso anisótropo** en vez de devolver un escalar plausible: con `k` tensorial la difusividad es un tensor.
- **Escala de temperatura**: la conducción pura sólo involucra **diferencias** de `T`, luego K y °C son equivalentes. La distinción importará al entrar radiación (`T⁴` exige escala absoluta).
- **Caveat de interoperabilidad**: no aplica el caveat de Voigt de ABAQUS — `k` es un tensor de orden 2 en ejes físicos, sin reordenamiento.
- **Limitaciones declaradas** (`out_of_scope` en la spec): conductividad dependiente de la temperatura `k(T)` (introduciría no linealidad), convección y radiación (son condiciones de frontera, no del material), cambio de fase / calor latente, acoplamiento termomecánico.
- **Compatible con**: `Quad4Thermal`, `Hex8Thermal` (elementos térmicos con `field='temperature'`).
- **Referencia**: ver `docs/specs/ThermalConduction.md`. Incropera & DeWitt (2007) §2; Carslaw & Jaeger (1959) §1; Lewis, Nithiarasu & Seetharamu (2004) §3.
- **Archivo**: solidum/materials/thermal_conduction.py · clase base en solidum/core/thermal_material.py

16.15 Cómo añadir un material nuevo

`/solidum-new material <Name>` — genera archivo en `solidum/materials/`, decorador `@MaterialRegistry.register`, esqueleto de test. Declarar `STRAIN_DIM` y, si tiene historia, `PRIMARY_STATE_VAR` (la variable que aparecerá en el VTK). Tras implementar el modelo constitutivo, **añadir una entrada a este catálogo** siguiendo el formato de arriba.

Para un material **térmico** nuevo: la clase hereda de `ThermalMaterial` (en `solidum/core/thermal_material.py`) y se registra con `@ThermalMaterialRegistry.register`. El método a implementar es `compute_flux(∇T) -> (q, k)`, no `compute_state`. Declarar `FLUX_DIM` (2 ó 3 — como propiedad de instancia si la clase sirve en ambas dimensiones), `PRIMARY_STATE_VAR` e `IS_SYMMETRIC`. La validación de `c`/`density` con mensaje accionable ya la aporta `volumetric_capacity()` de la clase base; no reimplementarla. En la spec, el contrato lleva `kind: thermal_material` e `interface.field: temperature`. Añadir la entrada en la sección "Materiales térmicos" de este mismo catálogo.

Un material **no estándar** (una ley de investigación que no es un modelo constitutivo clásico de elementos finitos) no entra en el programa principal: va a un **módulo de usuario** (ADR 0020, Reglas.md §4), con `/solidum-new material <Name> --user <módulo>`. Si no relaciona σ con ε, el módulo declara su propia familia de material: una subclase de `solidum.registry.Registry` con `YAML_SECTION` en el `registry.py` del módulo, que el lector YAML recorre sin conocerla.

Para un material **cohesivo** nuevo (ley tracción-salto): una ley cohesiva tracción-salto genérica con elementos de interfaz es **estándar** según el criterio de admisión (Reglas.md §1: de Borst, Crisfield, Remmers y Verhoosel 2012; elementos cohesivos de Abaqus). Lo que hace no estándar al módulo es la cinemática de discontinuidad embebida KOS de Retama (2010). Aun así, la familia cohesiva vive hoy en el módulo de usuario `discontinuities` porque su único consumidor es el elemento embebido, y el programa principal sólo gana piezas justificadas por casos reales propios (Reglas.md §1). Si entra al programa principal un elemento de interfaz estándar, la familia (registro y clase base, sin física nueva) pasa al programa principal y el módulo la reutiliza. Mientras tanto, un cohesivo nuevo para el elemento embebido va al módulo: archivo en `solidum/user/discontinuities/` (`--user discontinuities`); la clase hereda de `CohesiveMaterial` (no `Material`) y se registra con `@CohesiveMaterialRegistry.register`, el registro del módulo. Declarar `JUMP_DIM`, `PRIMARY_STATE_VAR` y `IS_SYMMETRIC`. La spec parte de `docs/user/discontinuities/specs/_template_cohesive_material.md`. Añadir la entrada en la sección «Módulos de usuario — discontinuities» de este catálogo.

Capítulo 17

Solvers — catálogo

Referencia rápida de los métodos de solución implementados. Una entrada por solver. Para detalles del algoritmo → código fuente.

***Corrector compartido (ADR 0015)**: los cinco solvers iterativos del programa principal (`NonlinearSolver`, `ArcLengthSolver` y su variante `IndirectDisplacementSolver`, `NewtonNewmarkSolver`, `NewtonHHTSolver`) ejecutan el mismo bucle de Newton, `NewtonCorrector` (`solidum/math/solvers/corrector.py`): un ensamblaje por iteración, criterio dual (ADR 0007), backend con preferencia Cholesky → Pardiso → LU (ADR 0003, ADR 0017), solver iterativo CG/MINRES + AMG a petición con `linear_algebra: iterative` (ADR 0018) y Newton modificado, line search y excepciones tipadas (ADR 0011). Cada uno aporta un `NewtonProblem` por paso con su residuo, su sistema tangente y su actualización, y conserva su control de paso. `solver.corrector` es la superficie de instrumentación. Las variantes de arco que sólo cambian la restricción del paso heredan su problema por paso, exportado como `solidum.math.solvers.ArcProblem`, y redefinen su método `constraint`.*

***Gancho de inicio de paso (ADR 0020, P5)**: los cinco solvers del programa principal que avanzan por pasos (`NonlinearSolver`, `ArcLengthSolver`, `IndirectDisplacementSolver`, `NewtonNewmarkSolver`, `NewtonHHTSolver`) invocan `Element.prepare_step(U_committed)` en cada elemento, una vez por intento de paso, con el campo convergido del paso anterior y antes del primer ensamblaje. Tras una bisección lo repiten con el mismo campo los solvers estáticos que bisecan (`NonlinearSolver`, `ArcLengthSolver` e `IndirectDisplacementSolver` en el programa principal, y `DissipationArcLengthSolver` en su módulo), así que el gancho debe ser idempotente; los dinámicos (`NewtonNewmarkSolver`, `NewtonHHTSolver`) no bisecan: un paso que no converge aborta el análisis con la excepción tipada. Es no-op salvo en los elementos que toman decisiones discretas entre pasos (hoy, la activación de la grieta en el módulo de usuario `discontinuities`). `LinearSolver` no lo invoca. Lo prueba `test_prepare_step_hook.py`. Un solver de un módulo de usuario que avance por pasos debe invocarlo igual (lo hace el de `dissipation_control`, al final de este catálogo).*

***Red de seguridad de los solvers estáticos** (ADR 0019): los cuatro estáticos (`LinearSolver`, `NonlinearSolver`, `ArcLengthSolver`, `IndirectDisplacementSolver`, y `DissipationArcLengthSolver` en su módulo) rechazan un mecanismo rígido antes de resolver (`MechanismError`). `LinearSolver` busca además **mecanismos internos** tras factorizar y comprueba tras resolver que la solución resuelve el sistema (error hacia atrás normado, desde el 2026-10-20; antes el residuo relativo, que rechazaba contrastes de rigidez legítimos). Desde el 2026-09-30 los no lineales buscan los mecanismos internos de la **tangente inicial** (`U = 0`),*

reutilizando su primera factorización: con el modelo geométricamente lineal un mecanismo interno es `IllPosedSystemError`; con elementos geométricamente no lineales (`Element.GEOMETRICALLY_NONLINEAR`) o materiales unilaterales, sólo un aviso. Desde el 2026-10-20 (decisión del usuario) un mecanismo interno no se declara por el tamaño de un pivote: un pivote pequeño frente a la diagonal (SuperLU, Pardiso y Cholesky) sólo dispara la búsqueda del modo de menor energía con la factorización existente, y es un mecanismo si ese modo no deforma ninguno de los elementos que mueve; el mensaje dice qué elementos son y, si se mueven como un sólido rígido, cómo. Con `linear_algebra: iterative`, que no tiene pivotes, se avisa una vez por análisis de que la comprobación no se hace. Los solvers dinámicos y el modal no comprueban nada de esto: ahí un modelo libre es legítimo.

***Solvers de los módulos de usuario** (ADR 0020): van en la sección «Módulos de usuario — <módulo>» al final del catálogo. Hoy, `DissipationArcLengthSolver` (módulo `dissipation_control`).*

17.1 LinearSolver — solución lineal directa en un paso

- **Propósito**: resolver `K · U = F` con una sola factorización.
- **Esquema**: comprobación de mecanismo rígido (ADR 0019) → ensamblaje único → eliminación directa de DOFs prescritos (ADR 0004) → factorización por el despachador (Cholesky → Pardiso → LU; ADR 0003, 0017) → búsqueda de mecanismos internos por la energía del modo y comprobación de equilibrio (ADR 0019).
- **Parámetros**: `linear_algebra` (default `auto`; también `lu`, `pardiso`, `cholesky`, `iterative[:amg|jaco`
ADR 0018).
- **Errores de planteamiento**: `MechanismError` (apoyos insuficientes, con el movimiento libre descrito) e `IllPosedSystemError` (mecanismo interno, matriz singular o factorización imposible), en vez de devolver desplazamientos absurdos. Con `linear_algebra: iterative`, aviso de que un mecanismo interno que la carga no active no se comprueba.
- **Cuándo usarlo**: problemas estrictamente lineales (todos los materiales con tangente constante, sin grandes desplazamientos, sin contacto).
- **No converge / no aplica**: cualquier no-linealidad material (daño, plasticidad) o geométrica (corotacional).
- **Spec**: docs/specs/LinearSolver.md
- **Archivo**: solidum/math/solvers/linear.py

17.2 NonlinearSolver — Newton-Raphson incremental con paso adaptativo

- **Propósito**: resolver problemas no lineales con control de carga, dividiendo la carga total en pasos y aplicando Newton-Raphson dentro de cada paso.
- **Esquema**:

- Bucle externo: incrementar factor de carga λ desde 0 hasta 1 en `num_steps` pasos.
- Bucle interno: Newton-Raphson con `K_t` · Δ`U = R =` λ·`F_ext - F_int`.
- **Criterio de convergencia dual** (ADR 0007, los dos a la vez): $\|$`T`$^\top$`R`$\| \leq$ `atol_force + rtol_force·max(`$\|$`F_ext`$\|$`, `$\|$`F_int`$\|$`)` y $\|\Delta$`U`$\| \leq$ `atol_disp + rtol_disp·`$\|$`U`$\|$, con `T`$^\top$`R` el residuo reducido del ADR 0004 (con sólo Dirichlet, el residuo en los DOF libres) y las tolerancias absolutas autoderivadas de las escalas del primer ensamblaje.
- **Adaptatividad** (`adaptive=True`): si converge en < 5 iteraciones, agranda el siguiente paso ($\times 1.5$); si no converge, biseca ($\div 2$). Falla si $\Delta\lambda <$ `min_delta_lambda`.
- Tras converger un paso → `assembler.commit_all_states()` (trial → committed en todos los elementos).
- El bucle interno es el `NewtonCorrector` compartido (ADR 0015) con el problema `_IncrementalProblem` (residuo λ·`F_ext - F_int`, reducción con el incremento de Dirichlet del paso); la convergencia no se evalúa en la iteración 0 (todo paso hace al menos una resolución).
- **Parámetros**: `convergence`, `max_iter`, `num_steps`, `adaptive`, `min_delta_lambda`, `linear_algebra`, `freeze_tangent_after_iter`, `line_search` (default `False`, ADR 0011).
- **Cuándo usarlo**: la opción por defecto para no-linealidad material o geométrica suave (sin snap-back).
- **Cuándo activar** `line_search=True`: cuando se observe oscilación del residuo entre iteraciones (síntoma clásico: ratios alternados sin descenso monótono). Caso documentado: plasticidad perfecta con carga cerca/sobre la capacidad. **Default** `False` porque en problemas con tangente consistente cuasi-cuadrática (daño activo, plasticidad estándar) el line search rechaza pasos correctos del Newton donde el residuo sube transitoriamente — ver ADR 0011 §"Enmiendas".
- **Diagnóstico al diverger**: lanza una subclase tipada de `RuntimeError` (`OscillatingNewtonError`, `SingularTangentError`, `LoadExceedsCapacityError`, `UnknownDivergenceError`) con métricas (último $\|$R$\|$, último $\|\delta$U$\|$, factor de carga, bisecciones consumidas) y `hint` textual. Ver `solidum/math/solvers/diagnostics.py`.
- **Tangente inicial** (ADR 0019, 2026-09-30): un mecanismo interno de un modelo geométricamente lineal se rechaza antes de iterar (`IllPosedSystemError`), aunque la carga no lo active; con elementos corrotacionales o materiales unilaterales, aviso.
- **Limitación**: con bisección adaptativa y cinemática no lineal (corotacional) puede atravesar puntos límite suaves; no atraviesa snap-back con `du/d`$\lambda <$ `0`. Para eso → `ArcLengthSolver`. Hallazgo de la auditoría fase A: el solver es más capaz de lo asumido históricamente (ver `docs/auditorias/solvers_robustez_fase_A.md`).
- **Referencia**: Crisfield, *Non-linear Finite Element Analysis of Solids and Structures*, vol. 1, cap. 9. Line search: Grippo-Lampariello-Lucidi 1986 (variante de descenso no monótono).
- **Spec**: docs/specs/NonlinearSolver.md
- **Archivo**: solidum/math/solvers/nonlinear.py

17.3 ArcLengthSolver — método de longitud de arco cilíndrico (Crisfield)

- **Propósito**: trazar curvas de equilibrio con snap-through, snap-back o pérdida de unicidad de carga, controlando simultáneamente desplazamientos y factor de carga.
- **Esquema**:
- **Predictor tangente**: `du_t = K_t⁻¹ · F_ext_ref`; $d\lambda$ `= sign · dl /` $\|$`du_t`$\|$.
- **Primer paso** (2026-09-23): se declara como fracción de la carga de referencia, `initial_dlambda` (0.1 por omisión), y el solver deriva la longitud de arco $\Delta l_1 = \Delta\lambda_1 \cdot \|K_0^{-1} \cdot F_{ref}\|$ con el predictor elástico: el primer paso lleva esa fracción de la carga en cualquier sistema de unidades. `initial_dl` fija Δl_1 como longitud explícita (excluyente).

El `sign` se elige por proyección con el incremento del paso anterior (evitar regresar).

- **Corrector iterativo**: en cada iteración resuelve dos sistemas (`du_R = K⁻¹·R` y `du_t = K⁻¹·F_ext`) y aplica la restricción cuadrática cilíndrica de Crisfield: $\|dU\|^2 = dl^2$.
- De las dos raíces se elige la que mantiene el ángulo positivo con el incremento previo.
- **Llegada a `max_lambda`** (2026-09-23): todos los pasos llevan la restricción; el que converge por encima de `max_lambda` no se consolida y se repite desde el estado anterior con $\lambda =$ `max_lambda` fijo (Newton puro), dentro del tramo recién recorrido. Nunca impone `max_lambda` sin haber seguido la curva hasta él (antes el paso final se decidía extrapolando la tangente, en control de carga).
- **Auto-ajuste de `dl`**: < 4 iter $\rightarrow$ ampliar ($\times 1.5$, tope $5 \cdot \Delta l_1$); > 8 iter $\rightarrow$ reducir ($\times 0.6$); no converge $\rightarrow$ biseca ($\div 2$).
- Convergencia: mismo criterio dual que `NonlinearSolver`.
- **Parámetros**: `convergence`, `max_iter`, `max_lambda`, `initial_dlambda`, `initial_dl`, `max_steps`, `dl_grow_factor`, `dl_max_factor`, `dl_shrink_factor`, `dl_grow_iter_threshold`, `dl_shrink_iter_threshold`, `linear_algebra`.
- **Cuándo usarlo**: problemas con softening pronunciado (daño, post-pandeo, snap-through de cúpulas), o cuando `NonlinearSolver` diverge cerca de un punto límite.
- **Limitación**: más caro por paso (dos resoluciones del sistema por iteración). La norma de la restricción mezcla todos los grados de libertad (en marcos, giros y desplazamientos: deuda #22).
- **Tangente inicial** (ADR 0019, 2026-09-30): el predictor del primer paso busca mecanismos internos, como `NonlinearSolver`; lo heredan `IndirectDisplacementSolver` y `DissipationArcLengthSolver`.
- **Referencia**: Crisfield, "A fast incremental/iterative solution procedure that handles snap-through" (Computers & Structures, 1981); Crisfield vol. 1, cap. 9.

- **Spec**: docs/specs/ArcLengthSolver.md
- **Archivo**: solidum/math/solvers/arclength.py

17.4 IndirectDisplacementSolver — control indirecto de desplazamiento (de Borst 1987)

- **Propósito**: trazar ramas con **retroceso** (*snap-back*) por localización, controlando el incremento de una magnitud cinemática elegida —el alargamiento de la zona que se ablanda, la apertura de una grieta (CMOD)— en lugar de la longitud de arco de todos los grados de libertad. Es como se controla en laboratorio un ensayo de fractura.
- **Esquema**:
- Subclase de `ArcLengthSolver`; sólo cambia la restricción del paso: $\mathtt{c}^\top \cdot \Delta\mathtt{U} = \Delta\mathtt{l}$, con `c` los coeficientes de la combinación controlada.
- Predictor $\Delta\lambda = \Delta\mathtt{l}/(\mathtt{c}^\top \cdot \mathtt{du_t})$ **sin regla de signo** (sale negativo en un retroceso) y corrector lineal $\delta\lambda = [\Delta\mathtt{l} - \mathtt{c}^\top(\Delta\mathtt{U} + \mathtt{du_R})]/(\mathtt{c}^\top \cdot \mathtt{du_t})$: sin selección de raíz.
- Primer paso adimensional (`initial_dlambda`, $\Delta\mathrm{l}_1 = \Delta\lambda_1 \cdot \mathrm{c}^\top \mathrm{K_0}^{-1}\mathrm{F_ref}$) y **sin crecimiento del paso por omisión** (`dl_max_factor = 1.0`): con ablandamiento local el problema incremental admite varios equilibrios y un paso grande puede saltar a otro (spec §5).
- **Parámetros**: `control` (obligatorio; lista de `(nodo, dof, coef)` o `{node, dof, coef}`), más todos los de `ArcLengthSolver`.
- **Cuándo usarlo**: retrocesos por localización (daño o cohesivo), ensayos controlados por CMOD/CTOD. Exige que la magnitud controlada crezca monótonamente; si decrece con la carga de referencia al empezar, el solver lo rechaza con un error explicativo.
- **Validación**: barra con daño localizado en un elemento, frente a la solución exacta: sigue el retroceso hasta el mínimo exacto (0.616·u_pico) con sólo el elemento débil dañado; el cilíndrico no. El arco por disipación del módulo de usuario `dissipation_control` lo sigue desde el 2026-09-30, con un primer paso pequeño (antes no, deuda #27).
- **Referencias**: de Borst (1987), *Computers & Structures* 25, 211-224; Crisfield (1991) vol. 1 cap. 9.
- **Spec**: docs/specs/IndirectDisplacementSolver.md (status `validated`).
- **Archivo**: solidum/math/solvers/indirect_displacement.py

17.5 ModalSolver — análisis modal por autovalores generalizados (ADR 0009)

- **Propósito**: calcular frecuencias naturales $\omega_\mathtt{n}$ y modos $\varphi_\mathtt{n}$ de vibración no amortiguados resolviendo $\mathtt{K} \cdot \varphi = \omega^2 \cdot \mathtt{M} \cdot \varphi$.

- **Esquema**:
- Ensamblaje de `K` (rigidez lineal en `u = 0`) y `M` (masa consistente, ADR 0009 §1) reusando la topología COO cacheada del `Assembler`. `M` incluye las **masas concentradas en nudos** (`Domain.add_nodal_mass`, bloque YAML `nodal_masses`, ADR 0024), sumadas a su diagonal con los dos esquemas.
- Reducción simultánea por Dirichlet y restricciones multipunto (ADR 0004) `Assembler.reduce_pair(K, M)` → `K_red = T`$^\top$`KT, M_red = T`$^\top$`MT, T`.
- `EigenSolver` envuelve `scipy.sparse.linalg.eigsh` con **shift-invert** en `sigma` (`sigma=0` → frecuencias más bajas) y `which="LM"`. ARPACK factoriza `(K_red - σ·M_red)` una sola vez y la reusa en todas las iteraciones Lanczos.
- **Grados de libertad sin masa** (masas sólo en algunos nudos, elementos con `density = 0`): se condensan estáticamente de forma exacta (φ_0 `= -K`$_{00}^{-1}$`K`$_{0m}\varphi_m$, ADR 0024) y sus componentes de los modos se recuperan. Hay tantas frecuencias finitas como grados de libertad con masa; si el problema no tiene solución, `MasslessDofsError` (`solidum.math.solvers.diagnostics`) los nombra como (nodo, dof).
- Expansión de modos al espacio completo: φ `= T · φ_red`. En DOFs prescritos por Dirichlet la componente es 0; en el esclavo de una MPC, la combinación de sus maestros (oráculo del nodo fundido en `tests/test_mpc_dynamic.py`). `ModalResult.free_vibration` proyecta las condiciones iniciales en norma M (`a = Φ`$^\top$`M·u0`), a diferencia de los transitorios, que reconstruyen los esclavos como `T·u0[libres] + g`: con condiciones iniciales incoherentes con las MPC ambos caminos difieren.
- Salida en `ModalResult`: `frequencies_rad`, `frequencies_hz`, `periods`, `modes` (M-ortonormales), `n_modes`, `converged`.
- **Parámetros**: `n_modes` (obligatorio), `sigma` (default 0.0), `which` (default `"LM"`), `tolerance` (default 1.0e-9), `lumping` (default `"consistent"`; también `"lumped"`, ADR 0009 fase 2), `linear_algebra` (reservado; ADR 0003).
- **Firma del solver**: `solve()` sin argumentos (no consume `F_applied`). El entrypoint público es `solidum.run_modal(domain, n_modes=N)` o YAML con `solver: type: ModalSolver`. `solidum.run_yaml` despacha automáticamente al detectar el tipo.
- **Cuándo usarlo**: caracterización dinámica de la estructura (frecuencias naturales, formas modales) antes de pasar a análisis transitorio o sísmico. Validación de la matriz de masa contra fórmulas analíticas (barra empotrada-libre, viga Bernoulli-Euler).
- **Limitaciones**: lineal (`K` evaluada en `u = 0`); sin amortiguamiento; modos complejos no soportados. Cómputos derivados (factores de participación, masas efectivas, combinación espectral SRSS/CQC) viven en `solidum/math/modal_response.py` y los consume `ResponseSpectrumSolver` (ADR 0009 fase 7 cerrada).
- **Referencia**: Bathe, *Finite Element Procedures*, cap. 9 (masa), §10.3.1 (condensación estática) y cap. 11 (algoritmos de autovalor); Chopra, *Dynamics of Structures*, §9.3; Hughes, *The Finite Element Method*, cap. 7; ARPACK Users' Guide (Lehoucq–Sorensen–Yang).

- **Spec**: docs/specs/ModalSolver.md
- **Archivos**: solidum/math/solvers/modal.py (clase `ModalSolver`), solidum/math/linalg/eigen.py (clase `EigenSolver`), solidum/results.py (`ModalResult`).

17.6 NewmarkSolver — análisis dinámico transitorio lineal (ADR 0009 fase 3)

- **Propósito**: integrar en el tiempo la ecuación de movimiento semidiscreta `M·ü + C·u̇ + K·u = F(t)` con amortiguamiento Rayleigh `C = α·M + β·K`, apoyos Dirichlet constantes y restricciones multipunto (ADR 0004).
- **Esquema**:
- Familia Newmark-β con `(β, γ)` parametrizables; default `(1/4, 1/2)` — average acceleration, incondicionalmente estable, sin amortiguamiento numérico, error `O(Δt²)`.
- **Predictores**: `ũ = u_n + Δt·u̇_n + (Δt²/2)(1−2β)·ü_n`, `ũ = u̇_n + Δt·(1−γ)·ü_n`.
- **Sistema efectivo**: `A_eff·ü_{n+1} = F_{n+1} − C·ũ − K·ũ` con `A_eff = M + γΔt·C + βΔt²·K`.
- **Correctores**: `u_{n+1} = ũ + βΔt²·ü_{n+1}`, `u̇_{n+1} = ũ + γΔt·ü_{n+1}`.
- Reducción por Dirichlet y MPC (operador `T`, `u = T·u_libre + g`) aplicada a `(K, M, C)` simultáneamente; factorización única de `A_eff_red` reusada en todos los pasos.
- Condiciones iniciales: se conservan las componentes libres de `u0`, `u0_dot` y los esclavos se reconstruyen como `T·u0[libres] + g` y `T·u0_dot[libres]`; si el valor dado en el esclavo de una MPC con maestros la incumple, se descarta con un aviso de log (común a los cinco transitorios).
- Aceleración inicial consistente con la ecuación de movimiento en `t=0`.
- **Parámetros**: `t_end`, `dt` (obligatorios), `beta` (default 0.25), `gamma` (default 0.5), `rayleigh` (None | `{alpha, beta}` directos | `{xi1, omega1, xi2, omega2}` para calibración modal), `u0`, `u0_dot` (default ceros), `F_func` (callback Python `t → F(t)`; default vibración libre), `linear_algebra`, `lumping`.
- **Firma del solver**: `solve()` sin argumentos, retorna `TransientResult` con historiales `t_history`, `u_history`, `udot_history`, `uddot_history` (shape `(n_dof, n_steps + 1)`).
- **Cuándo usarlo**: respuesta a cargas dependientes del tiempo (sísmica time-history, impacto, vibración forzada), evolución transitoria desde condiciones iniciales arbitrarias, validación de respuesta libre/forzada de estructuras lineales.
- **Limitaciones**:
- Solo lineal (`K`, `M` constantes). La extensión a no-linealidad (Newton dentro de cada paso) es fase 4 del ADR 0009.

- Apoyos Dirichlet constantes en el tiempo. Multi-support excitation sísmica diferida.
- Paso Δt fijo. Adaptativo diferido.
- HHT-α y generalized-α (con amortiguamiento numérico controlado) no incluidos.
- **Referencia**: Newmark (1959), J. Eng. Mech. Div. ASCE; Bathe, *FEP* cap. 9.4; Hughes, *FEM* cap. 9; Chopra, *Dynamics of Structures* cap. 5 y 15.
- **Spec**: docs/specs/NewmarkSolver.md
- **Archivos**: solidum/math/solvers/newmark.py (clase `NewmarkSolver`), solidum/math/damping.py (Rayleigh), solidum/results.py (`TransientResult`).

17.7 NewtonNewmarkSolver — análisis dinámico transitorio no lineal (ADR 0009 fase 4)

- **Propósito**: integrar en el tiempo `M·ü + C·u̇ + F_int(u) = F_ext(t)` con materiales no lineales (plasticidad, daño) o no linealidad geométrica. **Variante** de `NewmarkSolver` (Reglas §4): hereda predictores/correctores y reducción de Dirichlet; añade Newton-Raphson dentro de cada paso temporal.
- **Esquema**:
- Predictores Newmark idénticos a la fase 3.
- **Bucle interno de Newton-Raphson** sobre el residuo dinámico

`R(ü_{n+1}) = F_ext(t_{n+1}) − F_int(u_{n+1}) − C·u̇_{n+1} − M·ü_{n+1}`, con jacobiano `J = M + γΔt·C + βΔt²·K_t` y `K_t` la rigidez tangente corriente.

- **Convergencia dual** (ADR 0007): `R/(atol + rtol·F_ref)` $\leq$ `1` y δ`u/(atol + rtol·‖u‖)` $\leq$ `1`.
- **Commit de estado**: tras converger cada paso, `assembler.commit_all_states()` (trial → committed). Si no converge en `max_iter`, lanza `RuntimeError` y se descarta el state trial.
- **Amortiguamiento Rayleigh constante en el tiempo**, calibrado con la **rigidez elástica de referencia** `K_0` (al inicio del análisis, `u = 0`). Convención estándar (Abaqus, ANSYS, OpenSees); evita acoplamiento ad-hoc entre disipación viscosa y plástica.
- **Newton modificado opcional** (`freeze_tangent_after_iter`, ADR 0003 fase 2): factoriza fresco las primeras N iter de cada paso y reusa la factorización en las siguientes.
- **Recuperación del caso lineal**: con materiales lineales (`K_t ≡ K_0`), el residuo se anula en 1 iter y el resultado coincide exactamente con `NewmarkSolver`. Validado en tests.
- **Parámetros**: heredados de `NewmarkSolver` (`t_end`, `dt`, `beta`, `gamma`, `rayleigh`, `u0`, `u0_dot`, `F_func`, `linear_algebra`, `lumping`) más `convergence` (`ConvergenceCriterion`, ADR 0007), `max_iter` (default 20), `freeze_tangent_after_iter` (default `None`), `line_search` (default `False`, ADR 0011 — mismo criterio que `NonlinearSolver`).

- **Diagnóstico al diverger**: lanza una subclase tipada de `RuntimeError` con métricas y `hint` (mismo patrón que `NonlinearSolver`, ADR 0011).
- **Firma del solver**: `solve()` sin argumentos, retorna `TransientResult` (mismo formato que `NewmarkSolver`).
- **Cuándo usarlo**: respuesta dinámica de estructuras con plasticidad transitoria, fatiga de bajo ciclo, impacto en hormigón friccional, vibraciones de marcos con disipación plástica.
- **Limitaciones**: igual que `NewmarkSolver` en lo lineal (apoyos constantes, paso fijo). Además: no resuelve snap-back en problemas con softening severo — combinar con `ArcLengthSolver` si emerge.
- **Despacho YAML**: `solver.type: NewtonNewmarkSolver`. Hereda `PIPELINE_KIND = "transient"` de `NewmarkSolver`; `entry.run_yaml` despacha por ese atributo declarativo (regla C del ADR 0009, aplicada 2026-05-18) — sin `isinstance` ni cadenas hardcoded.
- **Referencia**: ADR 0009 §Fase 4; Hughes, *FEM* cap. 7-9 (Newton dentro de paso temporal); Crisfield vol. 2 cap. 24 (dinámica no lineal).
- **Spec**: docs/specs/NewtonNewmarkSolver.md
- **Archivo**: solidum/math/solvers/newmark.py (clase `NewtonNewmarkSolver`, junto al padre).

17.8 HHTSolver — análisis dinámico transitorio lineal con disipación numérica (Hilber-Hughes-Taylor α)

- **Propósito**: integrar `M·ü + C·u̇ + K·u = F(t)` con **amortiguamiento numérico controlado** que disipa selectivamente los modos de alta frecuencia espurios sin afectar los modos resueltos. **Variante** de `NewmarkSolver` (Reglas §4): hereda predictores/correctores, reducción Dirichlet y factorización única de `A_eff`.
- **Esquema**:
- Ecuación de equilibrio temporal: `M·ü_{n+1} + (1+α)·C·u̇_{n+1} − α·C·u̇_n + (1+α)·K·u_{n+1} − α·K·u_n = (1+α)·F_{n+1} − α·F_n`.
- **Auto-derivación de β, γ desde α** (Hilber 1977): β `= (1−α)²/4`, γ `= (1−2α)/2`. Preserva orden 2 y estabilidad incondicional.
- **Sistema efectivo**: `A_eff = M + (1+α)·γΔt·C + (1+α)·βΔt²·K`. Constante en el tiempo → factorización única reutilizable (como Newmark).
- **Radio espectral en alta frecuencia**: ρ_∞ `= (1+α)/(1−α)`. Con α`=0` → ρ_∞`=1` (sin disipación, recupera Newmark trapezoidal). Con α`=−0.05` → ρ_∞`≈0.905`. Con α`=−1/3` → ρ_∞`=0.5` (disipación máxima manteniendo orden 2).
- **Disipación selectiva**: modos con $\omega\Delta t$ `>` π se atenúan significativamente; modos con $\omega\Delta t$ `«` `1` casi intactos. Es lo que distingue HHT-α de subir γ Newmark (que disipa en todas las frecuencias y baja a orden 1).

- **Parámetros**: `t_end`, `dt` obligatorios. `alpha` (default `-0.05`, rango `[-1/3, 0]`). `beta`, `gamma` autoderivados desde `alpha` salvo override explícito (no recomendado fuera de experimentación). Resto heredado de `NewmarkSolver`: `rayleigh`, `u0`, `u0_dot`, `F_func`, `linear_algebra`, `lumping`.
- **Cuándo usarlo**: cuando la respuesta transitoria contenga modos de alta frecuencia espurios (mallas con modos altos no de interés, impacto, contacto, propagación con $\omega\Delta t$ cerca o por encima de π) que enmascaren la señal. Si quieres exactamente Newmark trapezoidal sin disipación, usa `NewmarkSolver`.
- **Default de `alpha`**: `-0.05` es el valor canónico (Hilber 1977; Abaqus, OpenSees, ANSYS): disipación >9 % por ciclo en altas frecuencias, <0.5 % en modos de interés. Si pides `HHTSolver` con `alpha=0.0`, es funcionalmente idéntico a `NewmarkSolver(beta=0.25, gamma=0.5)`.
- **Despacho YAML**: `solver.type: HHTSolver`. Hereda `PIPELINE_KIND = "transient"` de `NewmarkSolver`; `entry.run_yaml` despacha por el atributo declarativo (regla C ADR 0009).
- **Referencia**: Hilber, Hughes & Taylor (1977), *Earthquake Eng. Struct. Dyn.* 5(3) 283-292; Hughes *FEM* §9.3; Chopra *Dynamics of Structures* §15.4.
- **Spec**: docs/specs/HHTSolver.md
- **Archivo**: solidum/math/solvers/newmark.py (clase `HHTSolver`).

17.9 NewtonHHTSolver — análisis dinámico transitorio no lineal HHT-α

- **Propósito**: combinar HHT-α (disipación numérica) con Newton-Raphson interno para problemas con materiales no lineales (plasticidad, daño) o no linealidad geométrica. **Variante** de `NewtonNewmarkSolver` (Reglas §4) que aplica el esquema temporal HHT-α al residuo dinámico no lineal.
- **Esquema**:
- Residuo dinámico: `R(ü_{n+1}) = (1+α)·F_{n+1} − α·F_n − [(1+α)·F_int(u_{n+1}) − α·F_int(u_n)] − [(1+α)·C·u̇_{n+1} − α·C·u̇_n] − M·ü_{n+1}`.
- Jacobiano: `J = M + (1+α)·γΔt·C + (1+α)·βΔt²·K_t`.
- Todo lo demás idéntico a `NewtonNewmarkSolver`: convergencia dual (ADR 0007), commit/rollback, Rayleigh con K_0, Newton modificado opcional, `line_search` opt-in (ADR 0011), telemetría tipada al diverger.
- **Recuperación**: con materiales lineales, el residuo de Newton se anula en una iteración y el resultado coincide con `HHTSolver`. Validado en tests.
- **Parámetros**: `alpha` (default `-0.05`) + heredados de `NewtonNewmarkSolver` (`convergence`, `max_iter`, `freeze_tangent_after_iter`, `line_search`, …).
- **Cuándo usarlo**: respuesta dinámica de estructuras con plasticidad transitoria + presencia de modos altos espurios que el Newmark trapezoidal no atenuaría.

- **Despacho YAML**: `solver.type: NewtonHHTSolver`. Vía atributo `PIPELINE_KIND="transient"` heredado de `NewmarkSolver` → `run_transient`.
- **Spec**: docs/specs/NewtonHHTSolver.md (variante corta sobre la spec padre `HHTSolver.md`).
- **Archivo**: solidum/math/solvers/newmark.py (clase `NewtonHHTSolver`).

17.10 CentralDifferenceSolver — integración explícita por diferencias centradas (ADR 0009 fase 5)

- **Propósito**: integrar `M·ü + C·u̇ + F_int(u) = F(t)` con esquema **explícito** de segundo orden — la aceleración se actualiza en cada paso por `M⁻¹·rhs` con `M` diagonal lumped, sin sistema lineal ni Newton interno. Cubre análisis lineal y no lineal con un solo solver (parámetro `nonlinear: bool`).
- **Esquema (leapfrog Belytschko-Liu-Moran)**:
- Inicialización: `a_0 = M⁻¹·(F(0) − F_int(u_0) − C·v_0 − F_dir)`, `v_{1/2} = v_0 + (Δt/2)·a_0`.
- Paso `n` → `n+1`: `u_{n+1} = u_n + Δt·v_{n+1/2}`; reevaluar `F_int(u_{n+1})`; `a_{n+1} = M⁻¹·(F_{n+1} − F_int − C·v_{n+1/2} − F_dir)`; `v_{n+1} = v_{n+1/2} + (Δt/2)·a_{n+1}`.
- Modo lineal: `F_int = K·u` (K constante, ensamblada una vez). Modo no lineal: `Assembler.assemble_non_linear_system(u)` cada paso (descarta la tangente).
- **Estabilidad condicional CFL**: `Δt < 2/ω_max`. Para barras: `Δt < L_el/c_p = L_el·√(ρ/E)`. Si se excede, el solver detecta la explosión exponencial a posteriori y aborta con `RuntimeError` informativo. **El solver no calcula `Δt_crit` automáticamente** — responsabilidad del usuario en esta versión inicial (power iteration sobre `M⁻¹K` queda diferida).
- **Parámetros**: `t_end`, `dt`, `nonlinear` (default `False`), `rayleigh`, `u0`, `u0_dot`, `F_func`, `lumping` (default `"lumped"`; `"consistent"` rechazado), `divergence_threshold` (default `1e8`).
- **Cuándo usarlo**: wave propagation, impacto, dinámica de respuesta rápida — situaciones donde el `Δt` natural del problema cae por debajo del paso de estabilidad incondicional de Newmark. Para problemas con plasticidad transitoria de respuesta moderadamente rápida, sigue siendo competitivo frente a Newton-Newmark por evitar la factorización de la tangente cada iteración.
- **Restricciones multipunto**: admitidas mientras `M_red = TᵀMT` siga siendo diagonal, es decir, con un solo maestro por esclavo tras el cierre transitivo (atado de nodos, periodicidad, apoyo oblicuo `u_y = tanθ·u_x`) o con esclavo sin masa. Oráculo del nodo fundido en `tests/test_mpc_dynamic.py`.
- **Rechazos defensivos**: `lumping="consistent"` → `ValueError` (la inversión de M consistente cada paso anula la ventaja); `M_red` no estrictamente diagonal (umbral relativo `LUMPED_MASS_OFF_DIAGONAL_RTOL·max|diag|`, independiente de las unidades desde el 2026-09-30) →

`ValueError` con dos diagnósticos: una MPC con varios maestros cuyo esclavo tiene masa (nodo colgante; el mensaje nombra los DOFs y recomienda `NewmarkSolver`/`HHTSolver` con masa concentrada) o Frame3D con eje oblicuo a globales (ADR 0009 fase 2 documenta la bloque-diagonalidad inherente).

- **Despacho YAML**: `solver.type: CentralDifferenceSolver`. Vía atributo `PIPELINE_KIND="transient"` → `run_transient`. **Primer solver fuera de la jerarquía de Newmark**, lo que disparó la regla C de refactor (atributo declarativo en lugar de cadena de isinstance).
- **Spec**: docs/specs/CentralDifferenceSolver.md.
- **Archivo**: solidum/math/solvers/central_difference.py.

17.11 HarmonicSolver — respuesta forzada armónica en el dominio de la frecuencia (ADR 0009 fase 6)

- **Propósito**: resolver $(-\omega^2 M + i\omega C + K)\cdot\hat{u} = F$ para un barrido de frecuencias y devolver la amplitud compleja $\hat{u}(\omega)$ en cada frecuencia. Lineal, en estado estacionario — sin transitorio. Apropiado para FRFs, diagnóstico de resonancias y respuesta forzada periódica.
- **Esquema**: ensamble único de `K`, `M`, `C = α·M + β·K`; reducción de Dirichlet y MPC (`û = T·û_libre`; `g` no interviene, el problema es homogéneo); barrido `for ω in omega: spsolve(Z(ω), F_red)` con `Z(ω) = -ω²M + iωC + K` compleja. Factorización LU compleja por frecuencia (no se cachea — `Z` cambia con ω).
- **Barrido configurable**: lineal (`scale="linear"`), logarítmico (`scale="log"`), o vector explícito (`omega=array`). Default `n_omega=100`.
- **Parámetros**: `omega` o (`omega_min`, `omega_max`, `n_omega`, `scale`); `F_amplitude` (default ceros; YAML inyecta automáticamente las `point_loads`); `rayleigh`; `lumping`.
- **Cuándo usarlo**: diagnóstico de resonancias, FRFs, respuesta forzada armónica de máquinas rotantes, viento turbulento descompuesto en armónicos. Si el problema requiere no linealidad (geométrica o material), usar `NewtonNewmarkSolver` con excitación armónica.
- **Caveats**: resonancia exacta sin amortiguamiento ⇒ `Z(ω_n)` singular (`spsolve` puede fallar). Mitigación: añadir Rayleigh. Cargas con fase compleja requieren construcción desde código — YAML no soporta tipos complejos nativos.
- **Despacho YAML**: `solver.type: HarmonicSolver`. Vía atributo `PIPELINE_KIND="harmonic"` → `run_harmonic`. Tercer valor del literal (después de `"static"`, `"modal"`, `"transient"`).
- **Spec**: docs/specs/HarmonicSolver.md.
- **Archivo**: solidum/math/solvers/harmonic.py.

17.12 ResponseSpectrumSolver — análisis sísmico por combinación modal (ADR 0009 fase 7)

- **Propósito**: combinar las respuestas máximas de los primeros `n_modes` modos del sistema bajo un espectro de respuesta dado (`S_d(ω)` o `S_a(ω)`) en una envolvente máxima. Análisis estadístico, no temporal — apropiado para diseño sísmico normativo (ASCE 7, Eurocódigo 8, NCh 433, NTC-DS).
- **Clasificación (Reglas §1)**: estándar. El 2026-09-30 el criterio no decidía, porque la combinación modal SRSS/CQC no está en los textos de MEF de la lista; el usuario lo clasificó estándar y añadió Chopra (*Dynamics of Structures*, cap. 13) a la lista de textos. Está en Abaqus, ANSYS, OpenSees, Code_Aster y DIANA.
- **Esquema**:
- Análisis modal interno vía `ModalSolver` para obtener `(modes, frequencies)`.
- Factores de participación `Γ_n = φ_n^⊤·M·r` con `r` el vector de excitación rígida en la dirección sísmica.
- Respuesta máxima por modo: `u_n^max = Γ_n · φ_n · S_d(ω_n)`.
- Combinación SRSS (default) o CQC con coeficientes de correlación de Der Kiureghian.
- **Parámetros**: `n_modes`, `direction` (ndarray o `{"dof_name": "ux"}`), `spectrum` (callable o dict tabulado/constante), `combination` (`"SRSS"`/`"CQC"`), `damping` (sólo CQC, default 0.05), `sigma`, `lumping`.
- **Cuándo usarlo**: diseño sísmico contra espectro normativo; diagnóstico de modos dominantes en una dirección de excitación. Para no-linealidad usar transitorio (Newton-Newmark con acelerograma).
- **Masas de nudo y grados de libertad sin masa** (ADR 0024): heredados del `ModalSolver` interno. Con masas de piso y columnas sin masa propia, `total_mass` es la suma de las masas de piso y la suma de las masas efectivas de todos los modos finitos la alcanza (`tests/test_nodal_mass.py`). `n_modes` no puede pasar del número de grados de libertad con masa.
- **Restricciones multipunto**: heredadas del `ModalSolver` interno (`φ = T·φ_red`); oráculo del nodo fundido en `tests/test_mpc_dynamic.py`. Si `r` no cumple una MPC (apoyo oblicuo excitado en otra dirección), esa parte de `r^⊤Mr` no participa, igual que la masa de los DOFs apoyados: la suma de masas efectivas no alcanza `total_mass`.
- **Caveats**: precisión depende del número de modos incluidos — verifica `cumulative_effective_mass_ratio` ≥ 0.9. CQC requerido con modos cercanos en frecuencia (cociente < 1.3). Excitación multi-direccional simultánea (CQC3, 100/30/30) requiere combinación adicional fuera del solver.
- **Despacho YAML**: `solver.type: ResponseSpectrumSolver`. Vía atributo `PIPELINE_KIND="spectrum"` → `run_response_spectrum`. Cuarto valor del literal (después de `"static"`, `"modal"`, `"transient"`, `"harmonic"`).

- **Algoritmos centralizados**: `solidum/math/modal_response.py` — `response_spectrum_srss`, `response_spectrum_cqc`, `participation_factors`, `spectrum_from_sa`, `spectrum_tabulated`. Compartido con `ModalResult.free_vibration` (wrapper delgado tras regla D 2026-05-18).
- **Spec**: docs/specs/ResponseSpectrumSolver.md.
- **Archivo**: solidum/math/solvers/response_spectrum.py.

17.13 ThetaMethodSolver — conducción de calor transitoria por integración θ (Etapa 8)

- **Propósito**: integrar en el tiempo el sistema semidiscreto de **primer orden** `C·Ṫ + K·T = F(t)` de la conducción de calor. Es el único solver del proyecto que no integra la ecuación mecánica de segundo orden.
- **Por qué no es una variante de `NewmarkSolver`**: la familia Newmark integra `M·ü + C·u̇ + K·u = F` mediante hipótesis sobre la **aceleración** dentro del paso. En conducción no existe segunda derivada temporal — no hay aceleración ni inercia, y esas hipótesis no tienen sobre qué aplicarse. Forzar Newmark con `M = 0` degenera el esquema. Familia propia, no subclase.
- **El estacionario NO pasa por aquí**: `K·T = F` lo resuelve el `LinearSolver` existente sin modificación alguna.
- **Esquema**: `(C + θΔt·K)·T_{n+1} = [C − (1−θ)Δt·K]·T_n + Δt·[θ·F_{n+1} + (1−θ)·F_n]`. La matriz `A = C + θΔt·K` es SPD ⇒ Cholesky (ADR 0003), y con Δt constante **se factoriza una sola vez**: el problema es lineal y sin historia, así que `K` y `C` también se ensamblan una única vez. Coste por paso = una sustitución triangular.

θ	**Nombre**	**Orden**	**Estabilidad**
`0`	Euler explícito	1	Condicional ($\Delta t \leq 2/\lambda_{max}$)
`1/2`	Crank-Nicolson	**2**	Incondicional (A-estable)
`2/3`	Galerkin	1	Incondicional
`1`	Euler implícito	1	Incondicional, **L-estable** — *default*

- **Estabilidad no es ausencia de oscilaciones** (la razón del default): Crank-Nicolson es el más preciso, pero A-estable y **no** L-estable — su factor de amplificación tiende a `-1` para los modos altos en vez de a `0`. Ante un escalón de temperatura en la frontera produce oscilación amortiguada con temperaturas **fuera del rango de los datos**, violando el principio del máximo de la ecuación de difusión: un resultado cualitativamente imposible, no sólo impreciso. Medido: `T_max = 151.27` con `T_pared = 100`. Euler implícito nunca oscila. El default protege a quien no eligió; Crank-Nicolson queda a un parámetro de distancia. Mismo criterio que fijó `lumped` como default de la capacidad, aplicado al eje temporal en vez del espacial.
- **Reporte del orden efectivo**: contrapartida honesta del default robusto. `solver.order` (antes de correr) y `result.order` (después) devuelven `2` sólo con θ `= 1/2`, `1` en el resto — el coste

en precisión es consultable, no una penalización silenciosa. También `is_unconditionally_stable` e `is_l_stable`.

- **Parámetros**: `dt`, `n_steps`, `T_initial` (escalar ⇒ campo uniforme; vector ⇒ por nodo — **no hay velocidad inicial**, la ecuación es de primer orden y `T_0` la determina por completo); `theta` (default `1.0`), `lumping` (default `"lumped"`), `F_func`, `dirichlet_func`, `output_every`, `linear_algebra`. **Sin `rayleigh`**: es un modelo de disipación de la ecuación de segundo orden; en conducción la disipación es la conducción misma, ya contenida en `K`.

- **Dirichlet variable en el tiempo**: `dirichlet_func(t)` da los valores prescritos en el instante `t` y permite un ciclo térmico sobre una superficie. Sólo se leen los DOF con Dirichlet declarado; las filas de los esclavos de las restricciones multipunto se reconstruyen con la propia restricción (`ConstraintSet.build_g`: un esclavo atado a un maestro apoyado sigue su valor y el salto `g` de la MPC se conserva), y un valor no nulo en un DOF libre se rechaza. El acoplamiento `K_fp·ḡ` y el término de capacidad `C_fp·(ḡ_{n+1} − ḡ_n)` se recalculan cada paso; la factorización de `A_ff` **no** (verificado por contador).

- **Diagnóstico de** Δ`t`: reporta el paso característico h^2/α con α `= k/(ρc)` — el tiempo que tarda el frente térmico en cruzar un elemento. Es **información, no restricción**: con $\theta \geq$ `1/2` ningún Δ`t` diverge, pero uno $\gg h^2/\alpha$ se salta el transitorio que se quiere observar. Avisa si la razón supera 10. Análogamente, el transitorio no ha *terminado* hasta `t` $\gg L^2/\alpha$.

- **Cuándo usarlo**: arranque y enfriamiento de piezas, choque térmico, ciclos térmicos impuestos, calor de hidratación. Para el régimen permanente, `LinearSolver`.

- **Cuerpo aislado** (desde el 2026-10-20): **no exige Dirichlet**. Sin temperatura impuesta, `A = C + θΔt·K` sigue siendo definida positiva por `C`; el nivel de temperatura lo fijan `T_initial` y el calor aportado, y la energía almacenada $1^\top \cdot C \cdot T$ crece exactamente `Q·V·t` (validado contra `T = T_0 + Q·t/(ρc)` a precisión de máquina, 2D y 3D, cuatro θ). El **estacionario** sí lo exige: `K` es singular por el modo de temperatura uniforme (análogo térmico del sólido rígido) y el `LinearSolver` lo rechaza nombrándolo.

- **Caveats**: `T_initial` incompatible con un Dirichlet impuesto **avisa sin abortar**: un choque térmico es modelización legítima. $\theta <$ `1/2` con capacidad consistente es prácticamente inutilizable (λ`_max` grande) — se avisa. Requiere `density` y `c` declaradas en el material.

- **Despacho YAML**: `solver.type: ThetaMethodSolver`. Vía atributo `PIPELINE_KIND="thermal_transient"` → `run_thermal_transient`. Quinto valor del literal (tras `"static"`, `"modal"`, `"transient"`, `"harmonic"`, `"spectrum"`).

- **Resultado**: `ThermalTransientResult` — `t_history`, `T_history`, `theta`, `dt`, `order`, más `T_final`, `temperature_at(dof)` y `extremes()` (diagnóstico del principio del máximo). Dataclass propia y no `TransientResult`: sus campos `udot_history`, `uddot_history` y los coeficientes de Rayleigh no tienen significado térmico, y rellenarlos con ceros produciría un objeto que miente sobre su contenido.

- **Validación**: orden temporal medido **0.9944** (θ=1) y **2.0001** (θ=1/2); convergencia al estacionario del `LinearSolver` con error **7.1e-14**; sólido semi-infinito de Carslaw-Jaeger con error **3.0e-3**; cross-check 2D↔3D paso a paso. **Soluciones manufacturadas** (ADR 0022,

2026-10-27), con fuente, flujo y Dirichlet variables en el tiempo sobre mallas distorsionadas y conductividad anisótropa: orden en el tiempo 0.98 (θ=1), 1.03 (θ=2/3), 1.00 (θ=0, dentro de su límite) y 2.00 (θ=1/2); con `T*` lineal en `x` y en `t`, exacto a redondeo para todo θ, prueba que detecta la falta del término `C_fp·(ḡ_{n+1} - ḡ_n)`.

- **Spec**: docs/specs/ThetaMethodSolver.md.
- **Archivo**: solidum/math/solvers/theta_method.py.

17.14 Cómo añadir un solver nuevo

`/solidum-new solver <Name>` — genera archivo en `solidum/math/solvers/<snake>.py`, decorador `@SolverRegistry.register`, esqueleto de test.

Convenciones de interfaz:

- **Solvers estáticos** (lineales, no lineales, arc-length): `PIPELINE_KIND = "static"`. Constructor recibe `assembler` + parámetros; método `solve(F_ext_global, step_callback=None)` → `U_final`. Comprometen los estados internos vía `assembler.commit_all_states()` al converger cada paso. Retornan el campo de desplazamientos completo.
- **Solvers iterativos (Newton)**: no escribir el bucle. Construir un `NewtonCorrector(convergence, max_iter=…, is_symmetric=domain_is_symmetric(domain), is_positive_definite=…, linear_algebra=…, freeze_tangent_after_iter=…, line_search=…)` en el constructor y, por paso, un objeto con el protocolo `NewtonProblem` (`assemble`, `residual`, `residual_norm`, `calibration_scales`, `reference_force`, `x_norm`, `correction`, `apply`, `on_converged`); `corrector.run(problem, x0)` devuelve un `CorrectorResult` con `converged`, `x`, `n_solves` y `divergence_error(...)` para lanzar la excepción tipada. Ver `nonlinear.py` (el caso más simple) y ADR 0015.
- **Solvers modales / autovalor** (modal — ADR 0009 — y futuros pandeo lineal): `PIPELINE_KIND = "modal"`. Constructor recibe `assembler` + parámetros; método `solve()` → `ModalResult` (u otro tipo específico). No consumen vector de cargas. `run_yaml` despacha a `run_modal`.
- **Solvers transitorios** (Newmark, HHT, central differences): `PIPELINE_KIND = "transient"`. Constructor recibe `assembler`, `t_end`, `dt` + parámetros; método `solve()` → `TransientResult`. `run_yaml` despacha a `run_transient`.
- **Solvers térmicos transitorios** (θ-method): `PIPELINE_KIND = "thermal_transient"`. Constructor recibe `assembler`, `dt`, `n_steps`, `T_initial` + parámetros; método `solve()` → `ThermalTransientResult`. `run_yaml` despacha a `run_thermal_transient`. Integran la ecuación de **primer orden** `C·Ṫ + K·T = F`, no la de segundo orden mecánica: no admiten `rayleigh` ni condición inicial de velocidad.
- **Solvers en frecuencia / espectrales** (harmonic con `PIPELINE_KIND = "harmonic"`, response spectrum con `"spectrum"`). Constructor recibe `assembler` + parámetros del análisis; método `solve()` devuelve el resultado específico (`HarmonicResult`, `ResponseSpectrumResult`). `run_yaml` despacha a `run_harmonic` o `run_response_spectrum` según el atributo.

El **dispatch en** `run_yaml` se hace por el atributo de clase `PIPELINE_KIND` (regla C, 2026-05-18). Solvers nuevos no clásicos (que no hereden de los existentes) sólo declaran su `PIPELINE_KIND` y quedan automáticamente cableados — no requieren tocar `entry.py`.

Tras implementar, **añadir una entrada a este catálogo** siguiendo el formato de arriba y promover la spec a `status: validated`.

Un solver **no estándar** (una formulación que no cumple el criterio de admisión de Reglas.md §1: no aparece en los textos de referencia del área o ningún código general establecido la implementa) no entra en el programa principal: va a un **módulo de usuario** (ADR 0020, Reglas.md §4), con código en `solidum/user/<módulo>/` (scaffolding con `/solidum-new solver <Nombre> --user <módulo>`), spec en `docs/user/<módulo>/specs/`, tests en `tests/user/<módulo>/` y entrada en una sección «Módulos de usuario — <módulo>» al final de este catálogo. Se conecta por la parte pública del programa principal: `@SolverRegistry.register`, `PIPELINE_KIND` y, si es una variante de arco que sólo cambia la restricción del paso, `ArcLengthSolver` y `solidum.math.solvers.ArcProblem` (modelo: `DissipationArcLengthSolver`). Si avanza por pasos, invoca el gancho `assembler.prepare_all_steps(U_committed)` al empezar cada intento de paso.

Capítulo 18

Anexos técnicos

Esta sección es referencia técnica del subsistema interno que resuelve el sistema lineal `K·x = b` *que aparece en el corazón de cada solver no lineal. La elección del backend es* ***automática****; el usuario no decide. Esta sección documenta cómo se decide y cuáles son los puntos de override para diagnóstico.*

Diseño completo: ver ADR 0003 en `docs/adr/0003-despachador-de-solver-algebraico.md`.

18.1 Dos capas que se llaman ambas «solver»

Conviene distinguirlas:

- **Solver no lineal** — `LinearSolver`, `NonlinearSolver`, `ArcLengthSolver`. Orquesta la estrategia de paso, las iteraciones de Newton, el control de longitud de arco, los criterios de convergencia. Es lo que el usuario elige en el YAML con `solver.type`.
- **Capa algebraica** — `solidum.math.linalg`. Resuelve el sistema lineal `K·δU = R` que aparece dentro de cada iteración del solver no lineal. Tiene varios *backends* numéricos y un despachador interno que elige el adecuado según las propiedades de `K`. Es plumbing automático: el usuario no la ve salvo que pida diagnóstico.

18.2 Backends disponibles y criterio de selección

El despachador (`solidum.math.linalg.dispatcher.select_solver`) elige el backend en función de tres flags declarativos sobre `K`: simetría, positividad y talla.

Régimen de `K`	Backend elegido	Notas
Simétrica + positiva definida	**Cholesky** (CHOLMOD)	~2× más rápido y ~2× menos memoria que LU. Requiere la dependencia opcional `scikit-sparse`. Si falta, se pasa al siguiente disponible.
Simétrica indefinida	**Pardiso** si está; si no, LU (placeholder $\mathrm{LDL}^\top$)	Ideal para $\mathrm{LDL}^\top$ con conteo de pivotes negativos (Sturm sequence) en pandeo y snap-through. La interfaz está reservada; mientras no haya backend nativo se resuelve sin pérdida de corrección, solo sin diagnóstico de bifurcación.
No simétrica	**Pardiso** si está; si no, **LU general** (SuperLU)	Backend universal. Cubre plasticidad no asociada, follower loads, contacto con fricción.

El orden de preferencia es **Cholesky → Pardiso → LU**: Cholesky sólo aplica al caso SPD, pero ahí explota la simetría y es la mejor opción; Pardiso cubre el mismo terreno que LU —cualquier matriz real— y lo sustituye cuando está instalado (ADR 0017), porque es multihilo y reordena por disección anidada. LU (SuperLU, en SciPy) es el suelo que siempre está disponible.

Origen de los flags, todos derivables del modelo:

- `is_symmetric`: AND lógico de `material.IS_SYMMETRIC` y `element.PRESERVES_SYMMETRY` sobre todos los componentes del dominio. Defaults `True`.
- `is_positive_definite`: arranca en `True` para `LinearSolver` y `NonlinearSolver`; en `False` para `ArcLengthSolver`. Se degrada automáticamente si Cholesky reporta no-positividad.
- `size`: número total de DOFs.

Ningún flag se pide al usuario.

`EigenSolver` **(ADR 0009, problema generalizado)**. La capa algebraica incluye además `solidum.math.linalg.eigen.EigenSolver`, que resuelve el problema generalizado simétrico $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$ envolviendo `scipy.sparse.linalg.eigsh` (ARPACK Lanczos con shift-invert centrado en σ). No comparte el `Protocol` `LinearAlgebraSolver` de los backends de `K·x = b` porque su firma natural es `solve(K, M, n_modes)` → `(λ, φ)`. El `ModalSolver` lo invoca para el análisis modal; los cálculos internos de shift-invert generan una factorización de `(K − σ · M)` reutilizada en cada iteración Lanczos, así que la palanca de optimización es la misma — Cholesky enchufado en lugar de SuperLU bajaría 2× el coste del modal en problemas SPD (pendiente, ver memoria de cierre).

18.3 Red de seguridad del análisis estático (ADR 0019)

Un solver directo ante una matriz singular no avisa: SuperLU devuelve desplazamientos absurdos y Pardiso perturba los pivotes nulos y continúa. Como el usuario no elige el solver ni tiene por qué conocer ese comportamiento, Solidum comprueba tres cosas en todo análisis estático:

1. **Antes de resolver**: que los apoyos impidan todo movimiento de sólido rígido. Si no, se detiene y dice qué movimiento queda libre (*"traslación en la dirección x"*, *"giro alrededor del eje z que pasa por (0, 0.5)"*).

2. **Después de factorizar** (estático lineal): que el modelo no tiene un **mecanismo interno**, una parte que se mueve sin deformar ningún elemento aunque los apoyos impidan el movimiento rígido del conjunto. Desde el 2026-10-20 se decide por la energía, no por el tamaño de un pivote: si algún pivote es pequeño frente a la diagonal de su fila y su columna ($< 10^{-8}$; lo exponen SuperLU, Pardiso y Cholesky), se calcula con la misma factorización el modo de menor energía de $K \cdot v = \lambda \cdot \mathrm{diag}(K) \cdot v$, y es un mecanismo si no deforma ninguno de los elementos que mueve (energía relativa $< 10^{-12}$ en cada uno). El mensaje nombra esos elementos y, si se mueven como un sólido rígido, el movimiento. El tamaño del pivote no basta: el de un mecanismo real es ruido de redondeo que crece con el tamaño del modelo, y un contraste de rigidez legítimo (zonas rígidas, ν cerca de 1/2) también da pivotes pequeños. Una factorización que no llega a completarse (pivote exactamente nulo) también se explica.

3. **Después de resolver** (estático lineal): que el solver resolvió el sistema que se le dio, con el error hacia atrás normado de la solución, escalado por la diagonal, $\eta = \|\tilde{r}\| / (\|K\| \cdot \|\tilde{u}\| + \|F\|) \leq 10^{-10}$ ($K = D^{-½} \cdot K \cdot D^{-½}$, $D = \mathrm{diag}(K)$). Es la menor perturbación relativa de K y F de la que u es la solución exacta: un solver estable la deja en el orden del redondeo por desproporcionadas que sean las rigideces, y el escalado la hace independiente de las unidades. Hasta el 2026-10-20 se exigía $\|F - K \cdot u\| \leq 10^{-8} \cdot \|F\|$, que rechazaba modelos legítimos con zonas rígidas o materiales casi incompresibles.

En un análisis no lineal las dos últimas no se aplican a lo largo del trazado —cerca de un punto límite resolver un sistema casi singular es legítimo—; el Newton ya exige equilibrio real, y si el paso fracasa con un sistema tangente mal resuelto la causa se reporta como tangente singular. La excepción es la **tangente inicial** ($U = 0$, desde el 2026-09-30): su factorización, la primera que hace el solver, se inspecciona antes de usarla. Con un mecanismo interno y un modelo geométricamente lineal, el análisis se rechaza (un mecanismo en $U = 0$ lo es durante todo el análisis); con elementos geométricamente no lineales o materiales unilaterales, que pueden ganar rigidez al deformarse, sólo se avisa. Detalle y mensajes en el capítulo de diagnóstico del manual de usuario.

Con el backend iterativo la segunda comprobación no es posible: un método de Krylov no factoriza y no tiene pivotes, y distinguir un autovalor nulo de uno muy pequeño exigiría una precisión que no da de forma fiable. El análisis estático lo avisa una vez, con la manera de comprobarlo (análisis modal con densidad, o el solver directo).

18.4 Fallback automático SPD → LU

Si en algún paso K deja de ser positiva definida (paso a régimen postcrítico, daño con reblandecimiento), Cholesky lanza `CholeskyNotPositiveDefiniteError`. Los tres solvers no lineales lo capturan y reinstancian el backend con LU general para el resto del análisis. La transición es silenciosa — solo se imprime un mensaje informativo en stdout.

Esto significa que **forzar Cholesky desde YAML como diagnóstico nunca rompe el análisis**: si la matriz se vuelve incompatible, el sistema se autocorrige.

18.5 Factorización reusable y Newton modificado

La interfaz expone `solver.factorize(K)` que retorna un objeto `FactorizedSolver` con `solve(b)` reutilizable. Habilita:

- **Newton modificado**: el `NonlinearSolver` admite el parámetro `freeze_tangent_after_iter: int` que congela la factorización tras N iteraciones del paso. Útil cuando la tangente cambia poco y el coste dominante es la factorización.
- **Dinámica implícita lineal** (ADR 0009 fase 3): el `NewmarkSolver` factoriza una sola vez la matriz efectiva $\texttt{A_eff} = \texttt{M} + \gamma\Delta\texttt{t} \cdot \texttt{C} + \beta\Delta\texttt{t}^2 \cdot \texttt{K}$ al inicio del análisis y reutiliza la factorización en cada paso temporal. Con $\Delta\texttt{t}$ constante y problema lineal, los cientos o miles de pasos del análisis se reducen a sustituciones triangulares baratas. Es el mismo `FactorizedSolver` del despachador.
- **Análisis futuros** (pandeo lineal, dinámica implícita no lineal con factorización congelada entre pasos cuando la tangente cambia poco) reutilizan la misma interfaz sin extensiones adicionales.

Ejemplo YAML:

```yaml
solver:
  type: NonlinearSolver
  num_steps: 10
  tol: 1.0e-8
  freeze_tangent_after_iter: 2
```

18.6 Override desde YAML — herramienta de diagnóstico

Solo en caso de necesidad de *diagnóstico* o *benchmarking*, el bloque `solver` admite el campo opcional `linear_algebra`:

```yaml
solver:
  type: LinearSolver
  linear_algebra: auto          # default; el despachador decide
  # linear_algebra: cholesky    # forzar Cholesky (requiere scikit-sparse)
  # linear_algebra: pardiso     # forzar Pardiso (requiere pypardiso)
  # linear_algebra: iterative   # solver iterativo; ver sección propia
  # linear_algebra: lu          # forzar LU (siempre disponible)
  # linear_algebra: ldlt        # placeholder; degrada a LU con warning
```

Cuándo usar el override:

- Comparar tiempos entre Cholesky y LU en un mismo problema.
- Forzar LU si se sospecha un bug en la auto-detección de simetría tras añadir un material nuevo.
- Reproducir un caso de regresión bit-a-bit con un backend específico.

Cuándo NO usarlo: como parte del setup habitual de un caso de análisis. No es decisión de modelado; el default `auto` cubre todos los regímenes correctamente.

18.7 Backends opcionales: por qué importan

En un análisis grande, **la factorización del sistema es el cuello de botella**, no el ensamblaje. Medido sobre un análisis no lineal completo de `Hex8` 15^3 (12 288 grados de libertad): el 97 % del

tiempo se va en factorizar y el 1 % en ensamblar (ADR 0017). La razón de fondo es el **relleno** (*fill-in*): al factorizar, `L+U` tiene muchos más no-ceros que `K`, y la proporción empeora con la talla — de ×10 a 4 000 grados de libertad a ×42 a 53 000 —, llevándose por delante tiempo y memoria.

Los backends opcionales atacan exactamente eso. Sin ellos Solidum funciona igual, con SuperLU; con ellos los modelos grandes dejan de ser inviables.

18.7.1 Pardiso — recomendado (`pip install solidum-fem[fast]`)

```
pip install solidum-fem[fast]
```

Intel MKL Pardiso: factorización **multihilo** con reordenamiento por **disección anidada**. Trae binarios para Windows, Linux y macOS, así que no necesita compilador. Medido (16 hilos), resolviendo el sistema de `Hex8` n^3:

Malla	Grados de libertad	SuperLU	Pardiso	Aceleración
Hex8 10^3	3 993	0,47 s	0,10 s	×4,7
Hex8 15^3	12 288	3,24 s	0,27 s	×12
Hex8 20^3	27 783	18,32 s	0,78 s	×23,5
Hex8 25^3	52 728	110,10 s	1,97 s	×56

La aceleración **crece con el tamaño**, que es donde hace falta. Sobre el análisis no lineal completo de `Hex8` 20^3 (5 pasos): 212 s → 9,8 s. La solución es la misma a precisión de máquina (`1e-16` relativo en el campo de desplazamientos).

18.7.2 Cholesky / CHOLMOD (`scikit-sparse`)

```
conda install -c conda-forge scikit-sparse
```

Sólo aplica a matrices simétricas y positivas definidas, pero ahí explota la simetría y es la mejor opción. En Windows requiere Visual Studio para compilar, de ahí que la vía práctica sea `conda` y no `pip`. En Linux, `pip install solidum-fem[cholmod]` lo compila contra la SuiteSparse del sistema (`apt-get install libsuitesparse-dev`; la versión 0.5 de scikit-sparse exige SuiteSparse 7.4 o posterior). El backend admite las dos generaciones de su API, la 0.4 y la 0.5, que cambió la interfaz de la factorización.

Instalado cualquiera de los dos, el despachador empieza a usarlo automáticamente — sin tocar YAML, sin recompilar, sin reescribir specs.

18.8 Solver iterativo — a petición (ADR 0018)

Los tres backends anteriores son **directos**: factorizan la matriz y resuelven por sustitución. El cuarto es **iterativo**: no factoriza, sino que aproxima la solución por iteraciones de Krylov hasta una tolerancia. Es la misma división que ofrecen ANSYS (`SPARSE` frente a `PCG`) y Abaqus (*direct* frente a *iterative*), y con el mismo criterio de uso: el directo por defecto, el iterativo como opción que se pide expresamente.

```
solver:
  type: NonlinearSolver
  linear_algebra: iterative          # precondicionador automático (AMG si hay pyamg)
  # linear_algebra: iterative:amg    # multimalla algebraico (requiere pyamg)
  # linear_algebra: iterative:jacobi # diagonal (diagnóstico)
  # linear_algebra: iterative:none   # sin precondicionador
```

Para qué sirve. No para ir más rápido en modelos medianos: por debajo de ~10^5 grados de libertad Pardiso gana. Sirve para que **quepan los modelos grandes**. Un directo guarda los factores de la matriz, que ocupan un múltiplo creciente de ella (×14 a 26 000 grados de libertad, ×25 a 200 000); el iterativo ocupa un múltiplo constante (×4,2). Medido sobre `Hex8` n^3:

Grados de libertad	Pardiso	Iterativo (CG + AMG)	Memoria pico Pardiso	Memoria pico iterativo
26 460	**0,70 s**	1,43 s	313 MB	~92 MB
86 490	4,29 s	4,55 s	1,46 GB	~312 MB
201 720	17,6 s	**10,2 s**	4,43 GB	**~740 MB**

Cómo elige el método. Automáticamente, según la matriz:

- **CG** (gradiente conjugado) si es simétrica y definida positiva — el caso habitual.
- **MINRES** si es simétrica pero indefinida (arc-length cerca de un punto límite, ablandamiento). El cambio es automático: CG detecta la curvatura negativa y cede el paso.
- **Solver directo** si no es simétrica (Drucker-Prager no asociado, cargas seguidoras), con un aviso. CG y MINRES exigen simetría, igual que en los programas comerciales.

El precondicionador AMG (multimalla algebraico) es lo que hace útil al iterativo: el número de iteraciones queda en 10-15 **sin importar el tamaño del modelo**. Para funcionar en elasticidad necesita los **modos de cuerpo rígido** del modelo (traslaciones y rotaciones), que Solidum calcula a partir de las coordenadas y los grados de libertad, sin que el usuario haga nada. Sin ellos AMG es peor que no precondicionar.

Precisión. El iterativo se detiene cuando el residuo verdadero $\|\mathtt{b - K{\cdot}x}\|$ baja de $\mathtt{10^{-10}}\cdot\|\mathtt{b}\|$, cinco órdenes por debajo de la tolerancia del Newton. En un análisis no lineal el resultado coincide con el del directo; en uno lineal, la diferencia medida es del orden de 10^{-11}. Si no alcanza la tolerancia, **el análisis se detiene con un mensaje que dice por qué** (iteraciones, residuo alcanzado, qué hacer): nunca devuelve una solución a medias.

Cuándo no usarlo. Igual que advierten los manuales de ANSYS y Abaqus: en modelos mal condicionados —láminas, vigas muy esbeltas, elementos muy distorsionados, penalizaciones rígidas (discontinuidades embebidas como las del módulo de usuario `discontinuities`, cohesivos)— el iterativo converge mal o no converge. Ahí el directo es la herramienta.

18.8.1 Instalar AMG (`pyamg`)

```
pip install solidum-fem[iterative]
```

Trae binarios para Python 3.10–3.12. En **Python 3.14 sobre Windows** no hay binarios y pip lo compila, lo que requiere Visual Studio con el toolset de C++. Si aun teniéndolo pip responde *"Unable to find a compatible Visual Studio installation"*, la causa suele ser que la carpeta del instalador de Visual Studio no está en el `PATH`; añadirla durante la instalación lo resuelve:

```
PATH="/c/Program Files (x86)/Microsoft Visual Studio/Installer:$PATH" pip install pyamg
```

Sin `pyamg`, el iterativo funciona sin precondicionador (del orden de cientos de iteraciones por resolución en elasticidad) y avisa una vez.

Apéndice A

Módulos de usuario — discontinuities

Este apéndice recoge los **módulos de usuario**: formulaciones no estándar que viven fuera del programa principal, en `solidum/user/<módulo>/`, a la manera de los elementos y materiales de usuario de FEAP (ADR 0020). El programa principal no las carga por sí solo: un modelo las pide con `user_modules: [<módulo>]` en el YAML o con `solidum.load_user_module("<módulo>")` en Python. Cada capítulo reúne las entradas de catálogo de un módulo y sus specs.

Módulo `discontinuities`: discontinuidades interiores embebidas (ADR 0010, ADR 0020).

- Código: `solidum/user/discontinuities/`.
- Specs: `docs/user/discontinuities/specs/`.
- Carga en YAML: `user_modules: [discontinuities]`.
- Carga en Python: `solidum.load_user_module("discontinuities")`.

A.1 Elementos — catálogo

Módulo de usuario *(ADR 0020): formulación no estándar, fuera del programa principal, en `solidum/user/discontinuities/`, como un elemento de usuario de FEAP. Formulación: ADR 0010, Retama (2010).* `import solidum` *no la carga.*

Carga*: en YAML,* `user_modules: [discontinuities]` *en el primer nivel (sin esa clave, y si nada lo ha cargado antes en la sesión,* `CST_Embedded2D` *y la sección* `cohesive_materials` *son un error que sugiere declararla); en Python,* `solidum.load_user_module("discontinuities")` *o* `from solidum.user.discontinuities import CST_Embedded2D`. `from solidum import CST_Embedded2D` *ya no funciona.*

Specs en `docs/user/discontinuities/specs/`, tests en `tests/user/discontinuities/`. La ley cohesiva está en el catálogo de materiales, sección «Módulos de usuario — discontinuities».

Subfamilia de elementos con **DOFs enriquecidos elementales** y **condensación estática local**: el ensamblador no ve los grados de libertad del salto. Cuando se cumple un criterio de activación (Rankine en fase 1), el elemento materializa una discontinuidad interna `Γ_d` y enriquece su cinemática con un salto `[[u]]` gobernado por un material cohesivo (`CohesiveMaterial`, familia propia del módulo). La activación se evalúa en el gancho de inicio de paso `Element.prepare_step(U_committed)`, que los solvers que avanzan por pasos invocan **una vez por paso**, antes del

Newton (anti-chattering, ADR 0010 §5); es un contrato genérico del programa principal (ADR 0020, P5).

A.1.1 CST_Embedded2D — triángulo CST con discontinuidad interior embebida (KOS)

- **Propósito**: introducir fractura computacional en aproximación discreta sobre el CST padre. Fiel a Retama (2010), Caps. 2, 5, 6 y 7: cinemática KOS, condensación estática local, longitud efectiva `l_d = (A_e/h)·cos(θ−α)`.
- **DOFs por nodo**: `['ux', 'uy']` · 3 nodos · `STRAIN_DIM = 3` · `N_INTEGRATION_POINTS = 1`.
- **DOFs enriquecidos (elementales, no globales)**: `[[u]]` $\in \mathbb{R}^2$ en frame local `(n, s)` de `Γ_d`. Se condensan dentro de `compute_element_state` y nunca llegan al ensamblador.
- **Estado intacto**: bit-exact con Tri3 (mismo `B`, mismo material, misma cuadratura); la cinemática del CST es la pública del programa principal (`compute_kinematics_tri3` y `compute_integrands`, importadas de `solidum.elements.solid_2d`; ADR 0020, P6).
- **Estado agrietado**: Newton local sobre `[[u]]` hasta `R^{[[u]]} = 0`; después la condensación `K_cond = K_dd − K_du · K_{[[u]][[u]]}^{-1} · K_du^T` se devuelve al ensamblador. El bulk descarga elásticamente conforme `[[u]]` crece (discrete approach); la disipación va al cohesivo en `Γ_d`. Tolerancias del Newton local: `LOCAL_JUMP_RTOL` y `LOCAL_JUMP_MAX_ITER` en el `constants.py` del módulo.
- **Activación**: criterio Rankine (σ`_I > `σ`_t0` del cohesivo) en el centroide del CST, con el estado convergido del paso anterior. **Irreversible**: una vez activada, la discontinuidad persiste aunque el siguiente paso descargue.
- **Materiales aceptados**: bulk **solo** `Elastic2D` en fase 1 (la discrete approach presupone bulk elástico, ADR 0010); cohesivo cualquier `CohesiveMaterial` con `JUMP_DIM = 2`. Las dos referencias las declara `REFERENCE_KWARGS = {"material": MaterialRegistry, "cohesive_material": CohesiveMaterialRegistry}`: el lector YAML busca `material` en `materials` y `cohesive_material` en `cohesive_materials`.
- **Estado de la discontinuidad**: `DiscontinuityState` (en solidum/user/discontinuities/discontinuity_state.py) — paralela a `ElementState`, semántica trial/commit. Es un **elemento con estado propio**: sobrescribe `commit_state` y `compute_global_stiffness`, y no declara `BATCH_KINEMATICS`, así que va siempre por el camino por elemento.
- **YAML**:

```yaml
user_modules: [discontinuities]
cohesive_materials:
  - {id: 1, type: CohesiveDamageIsotropic, sigma_t0: 2.5e6, G_f: 100.0,
     K_e: 1.0e13, softening: linear}
elements:
  - {id: 1, type: CST_Embedded2D, nodes: [1, 2, 3],
     material: 1, cohesive_material: 1}
```

- **Post-procesamiento**: `compute_gauss_state(U)` añade clave `'discontinuity'` con `{normal, tangent, centroid, solitary_node, l_d, jump, traction, damage}` cuando el elemento está agrietado.
- **Limitaciones declaradas** (`out_of_scope` en la spec): modo mixto I-II (fase G del ADR 0010), reorientación de `n` tras activación (tracking no trivial, fase F), múltiples discontinuidades por elemento, contacto unilateral en compresión, bulks no elásticos, orden superior (`Tri6_Embedded`), 3D (`Tet4_Embedded`).
- **Solvers** (2026-09-29): con desplazamiento impuesto, `NonlinearSolver` atraviesa el ablandamiento hasta la separación con penalización cohesiva rígida (`test_disc_embedded_uniaxial_softening.py`, y el ensayo de Van Vliet de `test_disc_van_vliet.py`). El fallo registrado de `ArcLengthSolver` con esa penalización es anterior a la corrección de la tangente cohesiva (`597f3d9`) y no se ha vuelto a comprobar.
- **Spec**: docs/user/discontinuities/specs/CST_Embedded2D.md.
- **Archivo**: solidum/user/discontinuities/embedded_cst.py.
- **Tests**: `test_disc_cst_embedded.py`, `test_disc_cst_embedded_integration.py`, `test_disc_ld_chapter_6_validation.py`, `test_disc_embedded_uniaxial_softening.py` y el barrido de contrato del módulo, `test_disc_contratos.py`.

A.2 Modelos constitutivos — catálogo

***Módulo de usuario** (ADR 0020): formulación no estándar, fuera del programa principal, en `solidum/user/discontinuities/`. Formulación: ADR 0010, Retama (2010). `import solidum` no la carga.*

***Carga**: en YAML, `user_modules: [discontinuities]` en el primer nivel (sin esa clave, y si nada lo ha cargado antes en la sesión, la sección `cohesive_materials` es un error que sugiere declararla); en Python, `solidum.load_user_module("discontinuities")` o `from solidum.user.discontinuities import CohesiveDamageIsotropic`.*

Specs en `docs/user/discontinuities/specs/`, tests en `tests/user/discontinuities/`. El elemento que consume estos materiales, `CST_Embedded2D`, está en el catálogo de elementos, sección «Módulos de usuario — discontinuities».

Materiales cohesivos (familia propia del módulo). Los materiales cohesivos son una **jerarquía paralela e independiente** de los materiales continuos: operan sobre el salto de desplazamientos `[[u]]` sobre una superficie de discontinuidad `Γ_d` y devuelven tracciones `t`, no esfuerzos sobre ε. Viven en `solidum/user/discontinuities/`, heredan de `CohesiveMaterial` (`cohesive_material.py`), se registran vía `CohesiveMaterialRegistry` (el registro del módulo, en su `registry.py`: una familia de material con `YAML_SECTION = "cohesive_materials"`, ADR 0020) y se declaran en YAML bajo la sección `cohesive_materials` (separada de `materials`). El bulk del elemento sigue siendo un `Material` continuo del programa principal; el cohesivo entra al sistema sólo cuando el elemento activa una discontinuidad embebida (ver ADR 0010).

***Convenciones de la familia**: `JUMP_DIM` = dimensión del vector de salto (2 en 2D, 3 en 3D); `PRIMARY_STATE_VAR` = variable interna que se exporta al post-proceso; `IS_SYMMETRIC` = simetría de*

la contribución a la tangente del elemento. Parámetros físicos típicos: `sigma_t0` (Pa), `G_f` (N/m), `K_e` (Pa/m). No tienen densidad — la inercia es del bulk.

A.2.1 CohesiveDamageIsotropic — daño cohesivo isótropo Modo-I con softening lineal/exponencial

- **Modelo**: daño escalar $\omega \in$ `[0, 1]` sobre el salto. Tracción `t_n = (1 − ω)·K_e·[[u_n]]` en dirección normal, `t_s = 0` en tangencial (Modo-I puro). Activación tipo Rankine (`f = ⟨[[u_n]]⟩ − κ`); historial monótono `κ_{n+1} = max(κ_n, ⟨[[u_n]]⟩)` con `κ_0 = σ_t0/K_e`. Energía de fractura `G_f` cierra la curva analíticamente:
- **Lineal**: `T_soft(κ) = σ_t0·(w_c − κ)/(w_c − κ_0)` con apertura crítica `w_c = 2·G_f/σ_t0`. `ω(κ) = 1 − σ_t0·(w_c − κ)/[K_e·κ·(w_c − κ_0)]`. `ω = 1` en `κ ≥ w_c`.
- **Exponencial**: `T_soft(κ) = σ_t0·exp[−σ_t0·(κ − κ_0)/H]` con `H = G_f − σ_t0·κ_0/2`. `ω(κ) = 1 − T_soft(κ)/(K_e·κ)`. Asintótica a 1.
- **Tangente**: simétrica por construcción (rank-1 sobre `n⊗n`). Algorítmica consistente en carga activa: `T_tan = K_e·[(1−ω) − [[u_n]]·dω/dκ]·(n⊗n)`. Se evalúa como la pendiente de la envolvente, `dT_soft/dκ`, negativa en todo el ablandamiento y nula con la grieta totalmente abierta; sin tope `DAMAGE_MAX` (hasta el 2026-09-23 el tope le cambiaba el signo en casi toda la rama). Secante reducida `(1−ω)·K_e·(n⊗n)` en descarga y sin daño. En compresión (`[[u_n]] < 0`) se recupera la rigidez inicial: `t_n = K_e·[[u_n]]`, sin daño. ω y la tracción usan el valor físico (puede llegar a 1.0 exacto).
- **JUMP_DIM**: 2 · **PRIMARY_STATE_VAR**: `'damage'` · **IS_SYMMETRIC**: `True`.
- **Parámetros**: `sigma_t0` (resistencia a tracción, Pa), `G_f` (energía de fractura, N/m), `K_e` (rigidez del salto / penalty, Pa/m; sin default automático, ver §12 de la spec para guía `K_e` ≈ `10·E_bulk/ℓ_c`), `softening` ∈ `{'linear', 'exponential'}`.
- **Variables internas**: `kappa` (historial del salto equivalente, [m]), `damage` (ω).
- **Validación energética**: por construcción `∫_0^{w_c} t·d[[u_n]] = G_f` (lineal) o `∫_0^∞` ≈ `G_f` (exponencial); cubierto por tests con cuadratura trapezoidal.
- **Limitaciones declaradas** (`out_of_scope` en la spec): sólo Modo-I (mixto I–II diferido a fase G del ADR 0010), contacto en compresión sólo por la penalización normal `K_e` (sin fricción), sin anisotropía del daño, sin acoplamiento viscoso/cíclico, sin regularización para mesh-objectivity (se aborda a nivel del elemento `CST_Embedded2D`, no del material).
- **Origen**: modelo de daño isótropo con penalización de Alfaiate, Wells y Sluys (2002, ecs. 8-17), que Retama (2010) adopta en las ecs. 3.2-3.16.
- **Compatible con**: elemento `CST_Embedded2D` (fase 2 del ADR 0010), del mismo módulo. Validado dentro del elemento hasta la separación completa en `test_disc_embedded_uniaxial_softening.py`.
- **Tests**: `test_disc_cohesive_damage_isotropic.py`; tangente frente a diferencias finitas, monotonía de κ y ω y validación de la rama lineal en `test_disc_contratos.py`.

- **Referencia**: ver `docs/user/discontinuities/specs/CohesiveDamageIsotropic.md`. Retama (2010) Cap. 3; Hillerborg, Modéer & Petersson (1976); Simó & Ju (1987).
- **Archivo**: solidum/user/discontinuities/damage_isotropic.py

A.3 CST_Embedded2D

Orden de trabajo. Pre-redactada por la IA como borrador para validación del usuario (autor de la formulación, Retama 2010). La cinemática KOS y la fórmula `l_d = (A/h)·cos(θ−α)` son traducción directa de los Caps. 2, 5, 6 y 7 de la tesis; la mecánica de implementación (Voigt, registry, signatura, contrato `Element`, condensación) sigue las convenciones del proyecto. El usuario revisa, corrige donde proceda, y aprueba antes de que la IA toque código.

A.3.1 Especificación física

0. Descripción general

Elemento finito sólido 2D triangular de tres nodos (CST padre) **enriquecido** con una **discontinuidad interior embebida** `Γ_d` que cruza el dominio del elemento cuando se cumple un criterio de activación. Es la primera implementación de la familia de elementos con DOFs enriquecidos elementales introducida por **ADR 0010**, y la **fase 2** de la hoja de ruta de discontinuidades embebidas.

Estructura del elemento:

- **Estado intacto** (`discontinuity_state is None`): se comporta exactamente como un `Tri3` estándar — CST con un único punto de Gauss, B constante, integración exacta del bulk con peso `A_e·t`.
- **Estado agrietado** (`discontinuity_state is not None`): el campo de desplazamientos del bulk se enriquece con un salto `[[u]]` $\in \mathbb{R}^2$ localizado en `Γ_d`. El bulk vecino descarga elásticamente; la disipación se concentra en `Γ_d` y la gobierna un material cohesivo (`CohesiveMaterial`, ADR 0010 §1).

Aproximación adoptada: **discrete approach** (Retama 2010), **KOS** (Kinematically Optimal Symmetric) según la clasificación de Jirásek (2000). Tracking trivial: la normal `n` y la posición de `Γ_d` se fijan al activar el elemento (perpendicular a σ`_I` por criterio Rankine; `Γ_d` pasa por el centroide) y no se reorientan después. Modo-I puro (consistente con `CohesiveDamageIsotropic` fase 1); modo mixto y KSON quedan diferidos (fases G y H del ADR 0010).

Régimen: pequeños desplazamientos y deformaciones, cuasi-estático. Pensado para hormigón, roca, cerámicos cuasi-frágiles. Bulk **elástico** por construcción de la formulación (la disipación está localizada en `Γ_d`); materiales no lineales del bulk quedan `out_of_scope` en fase 1.

1. Cinemática de desplazamientos (formulación KOS)

Campo total de desplazamientos en el elemento:

$$\mathbf{u}(\mathbf{x}) = \underbrace{\sum_{i=1}^{3} N_i(\mathbf{x})\,\mathbf{d}_i}_{\mathbf{u}^{\mathrm{std}}(\mathbf{x})} + \underbrace{\big(H_{\Gamma_d}(\mathbf{x}) - \varphi(\mathbf{x})\big)\,\mathbf{G}\,\mathbf{u}}_{\mathbf{u}^{\mathrm{enr}}(\mathbf{x})}$$

donde:

- `N_i(x)` son las funciones de forma estándar del CST (coordenadas baricéntricas; ver `Tri3` §7).

- `d_i` $\in \mathbb{R}^2$ son los desplazamientos nodales estándar (DOFs globales del modelo, 2 por nodo).
- `[[u]]` $\in \mathbb{R}^2$ es el salto de desplazamientos a través de `Γ_d`, expresado en el **frame local** `(n, s)` de la discontinuidad. Es un **DOF enriquecido elemental** — vive en el elemento, no se ensambla al sistema global (ADR 0010 §2).
- `H_{Γ_d}(x)` es la función escalón de Heaviside relativa a `Γ_d` (0 en Ω^-, 1 en Ω^+). Ω^+ es el subdominio del lado al que apunta `n`.
- φ`(x)` es una función de soporte continua que vale `1` en los nodos de Ω^+ y `0` en los de Ω^-. Para CST con `Γ_d` cortando dos lados, φ`(x)` es la combinación lineal Σ `N_i^+(x)` sobre los nodos del lado positivo (típicamente un único nodo solitario, ver §6).
- `G` es la matriz que transforma el salto del frame local al sistema cartesiano global:

$$\mathbf{G} = \begin{bmatrix} n_x & s_x \\ n_y & s_y \end{bmatrix}$$

La construcción `(H_{Γ_d} -` φ`)` garantiza que `u^enr` sea **continuo en el contorno del elemento** (porque `H_{Γ_d} =` φ en los nodos) y que el salto correcto `[[u]] = G^T·[[u]]_global` ocurra únicamente sobre `Γ_d` interior. Es la elección **kinematically optimal**: el campo discontinuo está bien localizado y el salto no se transmite a elementos vecinos (a diferencia de XFEM, donde el salto sí cruza fronteras inter-elemento).

2. Cinemática de deformaciones

Deformación en el bulk (parte regular del campo):

$$\boldsymbol{\varepsilon}^{\text{bulk}}(\mathbf{x}) = \mathbf{B}_{\text{std}}\,\mathbf{d} \;-\; \mathbf{B}^{\varphi}(\mathbf{x})\,\mathbf{G}\,\mathbf{u}$$

donde `B_std` es la matriz `B` del CST estándar (Voigt 2D, 3×6, constante, ver `Tri3` §8) y `B^`φ`(x)` es la matriz Voigt construida con $\nabla\varphi$:

$$\mathbf{B}^{\varphi} = \begin{bmatrix} \partial\varphi/\partial x & 0 \\ 0 & \partial\varphi/\partial y \\ \partial\varphi/\partial y & \partial\varphi/\partial x \end{bmatrix}$$

Como φ `=` Σ `N_i^+`, las derivadas $\partial\varphi/\partial$`x`, $\partial\varphi/\partial$`y` son **constantes sobre el elemento** (CST lineal). Por tanto `B^`φ es constante.

Salto cinemático en `Γ_d` (en frame local):

$$\mathbf{u}(\mathbf{x}_{\Gamma_d}) = \mathbf{G}^{\top}\left[\,\mathbf{u}(\mathbf{x}^{+}) - \mathbf{u}(\mathbf{x}^{-})\,\right] \;=\; \mathbf{u} \qquad \text{(i.e. los propios DOFs enriquecidos)}$$

por construcción de la cinemática KOS — el salto en `Γ_d` coincide exactamente con la variable enriquecida, sin contribución de `d` (continuidad del campo estándar atravesando `Γ_d`).

3. Ecuación constitutiva

Bulk (en todo el dominio del elemento Ω_e):

$$\boldsymbol{\sigma}(\mathbf{x}) = \mathbf{C}_e\, \boldsymbol{\varepsilon}^{\text{bulk}}(\mathbf{x})$$

con C_e la matriz constitutiva elástica isótropa (parámetros E, ν, hipótesis plane stress / plane strain del material Elastic2D).

Discontinuidad (sobre Γ_d):

$$\mathbf{t}(\mathbf{x}_{\Gamma_d}) = \mathbf{T}_{\text{coh}}\big(\mathbf{u}, \text{estado cohesivo}\big)$$

con T_coh el operador del material cohesivo (en frame local; ver spec CohesiveDamageIsotropic).

4. Equilibrio — forma fuerte

Balance estático en cada subdominio (Ω^-, Ω^+) más continuidad de tracciones a través de Γ_d:

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = \mathbf{0} \quad \text{en } \Omega \setminus \Gamma_d, \qquad \boldsymbol{\sigma} \cdot \mathbf{n} = \mathbf{t} \quad \text{en } \Gamma_d.$$

5. Equilibrio — forma débil (residuales acoplados)

Aplicando el principio de trabajos virtuales con variaciones independientes δd y δ[[u]], se obtienen **dos residuales acoplados**:

$$\mathbf{R}^d = \int_{\Omega_e} \mathbf{B}_{\text{std}}^{\top}\, \boldsymbol{\sigma}\, d\Omega \;-\; \mathbf{f}_d^{\text{ext}} = \mathbf{0}$$

$$\mathbf{R}^u = -\int_{\Omega_e} \mathbf{G}^{\top}\, (\mathbf{B}^{\varphi})^{\top}\, \boldsymbol{\sigma}\, d\Omega \;+\; l_d\, \mathbf{t} = \mathbf{0}$$

con l_d la longitud efectiva de Γ_d dentro del elemento (ver §11). El primer residual se ensambla al sistema global; el segundo se resuelve **localmente** dentro del elemento (ver §10, condensación estática).

A.3.2 Formulación numérica (FEM)

6. Discretización del CST padre

Idéntica a Tri3: tres nodos en orden antihorario, mapeo isoparamétrico desde el triángulo de referencia con vértices (0,0), (1,0), (0,1). Jacobiano constante; aborta con error si det J $\leq$ tol.

Identificación del nodo solitario. Cuando Γ_d corta dos lados del triángulo, exactamente un nodo queda aislado en Ω^+ (lado al que apunta n); los otros dos quedan en Ω^-. El nodo solitario es el **único** que tiene φ = 1; los otros dos tienen φ = 0. La identificación se hace al activar el elemento, comparando el producto (x_i − x_c)·n para cada nodo (x_c = centroide), donde el signo del producto determina el lado.

7. Funciones de forma + función φ

`N_i(x)` idénticas a `Tri3` §7. Para φ:

$$\varphi(\mathbf{x}) = N_{i^*}(\mathbf{x})$$

donde `i*` es el índice del nodo solitario. Es lineal y `C^0` continua, vale 1 en `i*` y 0 en los otros dos. Sus derivadas son `(∂N_{i*}/∂x, ∂N_{i*}/∂y)`, **constantes** sobre el elemento.

8. Matrices B estándar + B^φ

`B_std` es la matriz B clásica del CST (constante, 3×6); ver `Tri3` §8.
`B^`φ es la matriz Voigt 3×2 construida con `(∂`φ`/∂x, ∂`φ`/∂y)`:

$$\mathbf{B}^{\varphi} = \begin{bmatrix} \partial\varphi/\partial x & 0 \\ 0 & \partial\varphi/\partial y \\ \partial\varphi/\partial y & \partial\varphi/\partial x \end{bmatrix}$$

Por construcción, `B^`φ coincide con la **submatriz B_std correspondiente al nodo solitario** (las columnas `2·i*` y `2·i*+1` de `B_std`). Eso permite implementación trivial sin recomputar derivadas.

9. Sistema acoplado — matrices elementales

Linealización del residual acoplado conduce al sistema 8×8 local (6 DOFs estándar + 2 enriquecidos):

$$\begin{bmatrix} \mathbf{K}_{dd} & \mathbf{K}_{du} \\ \mathbf{K}_{ud} & \mathbf{K}_{uu} \end{bmatrix} \begin{bmatrix} \Delta\mathbf{d} \\ \Delta\mathbf{u} \end{bmatrix} = \begin{bmatrix} -\mathbf{R}^d \\ -\mathbf{R}^u \end{bmatrix}$$

con (todas las cantidades evaluadas en el único punto de Gauss central, peso `A_e·t`):

- `K_dd = B_std^T · C_e · B_std · A_e · t` (6×6) — rigidez estándar del CST.
- `K_{d[[u]]} = -B_std^T · C_e · B^`φ `· G · A_e · t` (6×2) — acoplamiento.
- `K_{[[u]] d} = -G^T · (B^`φ`)^T · C_e · B_std · A_e · t` (2×6) — traspuesta de `K_{d[[u]]}` en KOS (simétrica por construcción).
- `K_{[[u]][[u]]} = G^T · (B^`φ`)^T · C_e · B^`φ `· G · A_e · t + l_d · T_coh` (2×2) — rigidez del salto, suma de contribución del bulk (descarga elástica del modo enriquecido) y de la rigidez tangente cohesiva `T_coh ≡ ∂t/∂[[u]]` proyectada sobre `Γ_d`.

Convención de signos: el `-` en `K_{d[[u]]}` y `K_{[[u]]d}` viene del signo del término enriquecido en `B^enr = -B^`φ`·G` (recordar `(H - φ)` con `∇H = δ_{Γ_d}·n`).

10. Condensación estática local

El sistema 8×8 se condensa eliminando los DOFs enriquecidos `Δ[[u]]` antes de devolver `K_e` al ensamblador (ADR 0010 §3):

$$\Delta\mathbf{u} = -\mathbf{K}_{uu}^{-1}\left(\mathbf{R}^{u} + \mathbf{K}_{ud}\,\Delta\mathbf{d}\right)$$

Sustituyendo en la primera ecuación:

$$\boxed{\mathbf{K}_{\text{cond}} = \mathbf{K}_{dd} - \mathbf{K}_{du}\,\mathbf{K}_{uu}^{-1}\,\mathbf{K}_{ud}}$$

$$\boxed{\mathbf{F}_{\text{int}}^{\text{cond}} = \mathbf{F}_{\text{int}}^{d} - \mathbf{K}_{du}\,\mathbf{K}_{uu}^{-1}\,\mathbf{R}^{u}}$$

donde `F_int^d = B_std^T · σ · A_e · t` es la fuerza interna estándar antes de condensar.

Inversión de `K_{[[u]][[u]]}` (2×2): cerrada analíticamente. El despachador algebraico (ADR 0003) no participa al ser una matriz elemental densa de 2×2.

Sin recuperación posterior (corregido 2026-09-29): el salto se resuelve con un Newton local dentro de `compute_element_state()` para cada `d` (Retama 2010, §7.2), y la condensación conserva el residuo local. `commit_state()` sólo copia `trial` → `committed`.

11. Activación: criterio Rankine + dirección `n` + `l_d` del Cap. 6

Criterio de activación. Al inicio de cada paso de carga (no dentro del Newton — ver §14), el elemento intacto evalúa su esfuerzo principal mayor `σ_I` en el centroide:

$$\sigma_I = \frac{\sigma_{xx} + \sigma_{yy}}{2} + \sqrt{\left(\frac{\sigma_{xx} - \sigma_{yy}}{2}\right)^2 + \sigma_{xy}^2}$$

Si `σ_I > σ_t0` (resistencia a tracción del material cohesivo), se activa:

- `n` = autovector unitario de `σ_I` (perpendicular a la dirección de tracción máxima, criterio Rankine).
- `s` = ortogonal a `n` orientada por regla de la mano derecha con `+z` saliendo del plano (consistente con Reglas.md §5).
- `Γ_d` pasa por el centroide del CST (decisión cerrada: la posición no es DOF; se fija en el activación, igual que `n`).
- Identificación del nodo solitario `i*` (§6).

Longitud efectiva `l_d` — Cap. 6 de Retama (2010). Fórmula cerrada:

$$\boxed{l_d = \frac{A_e}{h}\,\cos(\theta - \alpha)}$$

donde:

- `A_e` es el área del triángulo.
- `h` es la altura desde el nodo solitario `i*` al lado opuesto.

- θ es el ángulo de `Γ_d` respecto a `x` global (i.e. θ `= atan2(s_y, s_x)` con `s = (s_x, s_y)` la tangente a `Γ_d`).
- α es el ángulo del lado opuesto al nodo solitario respecto a `x` global.

Esta fórmula es **decisión cerrada en el ADR 0010** (§6); no se relitiga aquí. La alternativa ingenua `l_d = A_e/h` (independiente de θ) produce stress locking cuando la grieta no es paralela al lado opuesto; ver fase 3 del ADR 0010 para test numérico.

12. Estado de la discontinuidad — `DiscontinuityState`

Dataclass paralela a `ElementState` (ADR 0010 §4), específica de la discontinuidad. Vive en `solidum/user/discontinuities/discontinuity_state.py`:

```
@dataclass
class DiscontinuityState:
    normal: np.ndarray          # n ∈ℝ ², unitario, fijado al activar
    tangent: np.ndarray         # s ∈ℝ ², s = -(n_y, n_x)
    centroid: np.ndarray        # punto por el que pasa Γ_d
    solitary_node: int          # índice (0, 1 o 2) del nodo en Ω⁺
    l_d: float                  # (A_e/h)·cosθ( -α )
    jump_committed: np.ndarray   # [[u]] al cierre del último paso convergido
    jump_trial: np.ndarray       # [[u]] dentro de la iteración Newton
    cohesive_state_committed: dict  # estado interno del CohesiveMaterial (committed)
    cohesive_state_trial: dict      # estado interno del CohesiveMaterial (trial)
```

Semántica **trial / commit** consistente con `ElementState`: `jump_trial` y `cohesive_state_trial` evolucionan en cada iteración del Newton global; al converger el paso, `commit_state()` copia `trial` → `committed` (el salto ya lo resolvió el Newton local, §10).

13. Cuadratura

Un único punto de Gauss central, peso `A_e·t` — idéntico a `Tri3`. Las matrices `B_std`, `B^φ` y la deformación del bulk son constantes; la integración es exacta. El término cohesivo `l_d·t` se evalúa directamente sobre `Γ_d` (no requiere cuadratura porque `t` es constante a lo largo de `Γ_d` por la cinemática KOS — el salto es constante).

14. Activación al principio del paso — interacción con el solver

ADR 0010 §5 fija que la activación se evalúa **al inicio de cada paso de carga, no dentro del Newton**. Razón: dentro del Newton, `σ_I` oscila por el predictor lineal y puede cruzar/descruzar el umbral varias veces antes de converger — *chattering*.

Implementación (en uso; desde el ADR 0020 es el gancho genérico P5 del programa principal): el método opcional de `Element` `prepare_step(U_committed)` que el `NonlinearSolver`/`ArcLengthSolver` invoca **una vez por paso** antes del primer Newton. La base lo hace no-op (la mayoría de elementos no necesitan preparación); `CST_Embedded2D` lo sobreescribe para evaluar la activación con el estado convergido del paso anterior. Una vez activado, el elemento queda agrietado **irreversiblemente** (no revierte si el siguiente paso lo descarga).

A.3.3 Caveats numéricos

- **Invertibilidad de `K_{[[u]][[u]]}` en ablandamiento**. La tangente cohesiva es `diag(dT_soft/dκ, 0)`: negativa en el ablandamiento, nula con la grieta totalmente abierta y nula siempre en la dirección tangencial (Modo-I). La matriz 2×2 sigue siendo invertible por el término del bulk `G^T·(B^φ)^T·C_e·B^φ·G·vol`, definido positivo, mientras el elemento no sea tan grande que el ablandamiento lo supere (aproximadamente `l_d·|dT_soft/dκ| < E/h`). Por eso el cohesivo no necesita ninguna rigidez residual; la que tuvo hasta el 2026-09-23 (`(1 - DAMAGE_MAX)·K_e`) daba a la tangente condensada errores del 26 al 45 % en el ablandamiento (`TestCondensedTangentInSoftening`).

- **Mecanismo de deslizamiento**. Con rigidez de modo II nula (Retama 2010, p. 67), una grieta que atraviesa todo el sólido deja deslizar sin resistencia una parte respecto de la otra en la dirección de la grieta. Las condiciones de apoyo tienen que impedir ese movimiento; si no, el Newton global deriva sin converger en desplazamientos aunque el residuo sea nulo.

- **Solvers que atraviesan el ablandamiento con penalización cohesiva rígida** (nota del 2026-09-29, recogida de la limitación que se retiró ese día de la spec de `NonlinearSolver`). `NonlinearSolver` con control por desplazamiento atraviesa el ablandamiento hasta la separación completa con $K_e = 10^{15}$, frente a la solución exacta (`test_disc_embedded_uniaxial_softening.py`); la limitación antigua («no atraviesa la transición elástico→ablandamiento con $\mathbf{K}_t$ casi singular en κ_0») venía de la tangente cohesiva de signo erróneo, corregida el 2026-09-23 (commit `597f3d9`). El fallo registrado de `ArcLengthSolver` con esa misma penalización es anterior a esa corrección y **no se ha vuelto a comprobar**: puede que ya no exista, pero ningún test lo confirma.

- **Activación al principio del paso**. Si el incremento es muy grande, `σ_I` puede saltar muy por encima de `σ_t0` antes de la activación, generando un Δ[[u]] grande en el primer Newton. Mitigación: pasos suficientemente pequeños o `ArcLengthSolver` cerca del pico.

- **Bulk descarga elástica**. Por la cinemática KOS, el bulk evalúa `ε^bulk = B_std·d - B^φ·G·[[u]]` — descarga conforme `[[u]]` crece. Esto es físicamente correcto (la disipación va a `Γ_d`) y consistente con la discrete approach de Retama 2010. El bulk **no acumula daño** propio (out_of_scope fase 1).

- **`Γ_d` paralelo a un lado**. Caso degenerado: `cos(θ−α) = 1`, `l_d = A_e/h`. La fórmula del Cap. 6 sigue siendo correcta, pero numéricamente el nodo solitario debe identificarse con tolerancia para evitar oscilación entre dos nodos casi en el mismo plano.

- **`Γ_d` casi vertical en CST con jacobiano malacondicionado**. Sin patología nueva: hereda los caveats del CST padre. El test `det J > tol` de `Tri3` cubre el caso.

- **Stress locking**. La fase 3 del ADR 0010 valida numéricamente que `l_d = (A_e/h)·cos(θ−α)` evita el locking que aparece con `l_d = A_e/h` ingenuo. Test cubierto en `tests/user/discontinuities/test_disc_ld_chapter_6_validation.py`: con `Γ_d` oblicua respecto al lado opuesto al nodo solitario (`cos(θ−α) = √3/2` en el setup), la versión correcta produce una apertura `[[u_n]]` ≈ 5.45 % mayor que la versión ingenua para una misma carga, en rama de softening exponencial activo. La versión ingenua "infla" la rigidez cohesiva (`l_d · T_coh`)

por un factor `1/cos(θ−α) > 1`, restringe la apertura y el elemento se comporta como si fuera más rígido — eso es el *stress locking*. Cuando `Γ_d` es paralela al lado opuesto (`cos(θ−α) = ±1`), las dos fórmulas coinciden exactamente y no hay locking.

A.3.4 Contrato de implementación

```yaml
name: CST_Embedded2D
kind: element
status: validated

interface:
  dof_names: [ux, uy]            # 2 DOFs globales por nodo (los enriquecidos NO son
      globales)
  n_nodes: 3
  strain_dim: 3                  # Voigt 2D ε[_xx, ε_yy, γ_xy]
  n_integration_points: 1        # CST con cuadratura central

parameters:
  - { name: thickness,          type: float, required: false, default: 1.0,
      desc: "Espesor para estado plano (idéntico a Tri3)." }
  - { name: cohesive_material, type: str,   required: true,
      desc: "Nombre (en CohesiveMaterialRegistry) del material cohesivo que gobierna
         Γ_d cuando se activa." }

material_contract:
  bulk:
    signature: "compute_stateε(, state) -> σ(, C_tangent, state')"
    strain_kind: "Voigt 2D ε[_xx, ε_yy, γ_xy]"
    accepted: ["Elastic2D"]       # fase 1: bulk elástico; otros → out_of_scope
  cohesive:
    signature: "compute_traction([[u]], state) -> (t, T_tan, state')"
    jump_kind: "frame local - [[u]] = ([[u_n]], [[u_s]]) en ejes (n, s) de Γ_d"
    accepted: ["CohesiveDamageIsotropic"]  # cualquier CohesiveMaterial con JUMP_DIM=2

conventions:
  sign: "tracción interna positiva; convención stress-resultant del proyecto (Reglas.md
      §5)."
  voigt: "[xx, yy, xy] con γ_xy = 2·ε_xy."
  node_orientation: "antihorario (det J > 0)."
  frame_local_discontinuity: "n perpendicular a σ_I al activarse; s = -(n_y, n_x) (RHR
      con +z saliendo del plano)."
  activation: "irreversible; evaluada al principio de cada paso con el estado
      convergido previo."

validity:
  - "Pequeños desplazamientos y deformaciones (no large strain)."
  - "Bulk elástico (Elastic2D) y cohesivo Modo-I (JUMP_DIM = 2)."
  - "Γ_d cortando exactamente dos lados del triángulo (configuración geométrica está
      ndar)."

out_of_scope:
  - "Bulk no lineal (J2, daño continuo, Drucker-Prager): la disipación debe localizarse
       en Γ_d, no co-existir con disipación de bulk."
  - "Modo mixto I-II en el cohesivo (fase G del ADR 0010)."
  - "Reorientación de n tras la activación (tracking no trivial, fase F)."
```

```
  - "Múltiples discontinuidades por elemento."
  - "Contacto con fricción en la grieta cerrada (el cohesivo sólo recupera la
      penalización normal K_e en compresión)."
  - "Elementos de orden superior con discontinuidad embebida (Tri6_Embedded,
      Quad8_Embedded; fase J)."
  - "3D (fase I)."

acceptance:
  verification:
    - name: estado_intacto_reproduce_Tri3
      setup: "elemento sin activar (discontinuity_state=None), bulk Elastic2D; aplicar
          campo lineal de desplazamientos."
      expect: "K_e y F_int_e idénticos (bit-exact en doble) a los de un Tri3 con el
          mismo bulk."
      tol_rel: 1.0e-14

    - name: patch_test_intacto
      setup: "patch MacNeal-Harder con 4 CST_Embedded2D distorsionados, ninguno
          activado, σ < σ_t0 en todo el dominio."
      expect: "campo lineal exacto en nodos interiores; ε constante por elemento."
      tol_rel: 1.0e-10

    - name: activacion_rankine_correcta
      setup: "tracción uniaxial pura en x sobre un único CST con σ_t0 conocido;
          incrementar carga hasta cruzar σ_t0."
      expect: "activación en el paso donde σ_xx > σ_t0; n = (1, 0) y s = (0, 1) con
          error angular < 1e-10."
      tol_rel: 1.0e-10

    - name: bulk_descarga_elasticamente_post_activacion
      setup: "activar elemento, después fijar [[u]] e imponer Δd; verificar que la
          deformación del bulk responde como elástica con C_e."
      expect: "σ_bulk = C_e · (B_std·d − Bφ̃·G·[[u]]); coincide con C_e aplicado a la
          ε neta."
      tol_rel: 1.0e-12

    - name: condensacion_simetrica_KOS
      setup: "elemento activado con cohesivo en carga activa simétrica (Modo-I puro)."
      expect: "K_cond simétrica: ‖K_cond − K_cond̃ᵀ‖_F / ‖‖K_cond‖_F < 1e-12."
      tol_rel: 1.0e-12

    - name: ld_formula_cerrada
      setup: "varias orientaciones de Γ_d θ( ∈ {0, π/6, π/4, π/3, π/2}) sobre un triá
          ngulo conocido."
      expect: "l_d coincide con (A_e/h)·cos(θ−α()) calculado analíticamente."
      tol_rel: 1.0e-14

    - name: recuperacion_local_del_salto
      setup: "carga monótona hasta varios pasos post-activación; tras cada commit, [[u
          ]]_committed debe satisfacer R̃{[[u]]} = 0."
      expect: "‖R̃{[[u‖]]} < 1e-8 tras commit_state."
      tol_rel: 1.0e-8

  specific:
    - name: bulk_no_lineal_rechazado
      setup: "instanciar CST_Embedded2D con bulk_material=IsotropicDamage2D."
```

```
      expect: "ValueError clara en construcción, mensaje listando los bulks aceptados (
          fase 1: Elastic2D)."

    - name: jump_dim_validado
      setup: "instanciar con cohesive_material cuyo JUMP_DIM=3 (futuro 3D)."
      expect: "ValueError con mensaje sobre incompatibilidad dimensional (elemento 2D
          requiere JUMP_DIM=2)."

    - name: activacion_no_chattering_dentro_de_newton
      setup: "elemento cerca del umbral; correr iteraciones Newton sin invocar
          prepare_step."
      expect: "estado discontinuity_state no cambia durante las iteraciones; sólo
          prepare_step modifica activación."

    - name: identificacion_nodo_solitario
      setup: "construir CST con coordenadas tales que solo un nodo cae en Ω⁺ para una n
          dada."
      expect: "DiscontinuityState.solitary_node identifica correctamente el índice; los
          otros dos tienen φ=0."

    - name: ld_invariante_bajo_reflejo_de_n
      setup: "activar con n y comparar con activar con -n (mismo problema físico)."
      expect: "l_d, K_cond, F_int^cond idénticos (la convención de n positivo está bien
          definida - apunta hacia el nodo solitario)."
      tol_rel: 1.0e-14

  arch:
    - name: registro_en_ElementRegistry
      expect: "CST_Embedded2D registrado con kind='element'."

    - name: STRAIN_DIM_y_DOF_NAMES_correctos
      expect: "STRAIN_DIM=3, DOF_NAMES=['ux','uy'], N_NODES=3, N_INTEGRATION_POINTS=1."

    - name: prepare_step_es_no_op_en_Element_base
      expect: "Element.prepare_step existe y no hace nada por defecto; CST_Embedded2D
          lo sobreescribe."

references:
  - "Retama Velasco, J. (2010). Formulation and Approximation to Problems in Solids by
      Embedded Discontinuity Models. Tesis Doctoral, UNAM. **Caps. 2 (cinemática KOS),
      5 (formulación variacional discreta), 6 (l_d correcto), 7 (condensación estática)
      **."
  - "Jirásek, M. (2000). Comparative study on finite elements with embedded
      discontinuities. CMAME 188, 307-330. - Clasificación SOS/KOS/SKON."
  - "Oliver, J., Huespe, A.E., Sánchez, P.J. (2006). A comparative study on finite
      elements for capturing strong discontinuities. CMAME 195, 4732-4752."
  - "Linder, C., Armero, F. (2007). Finite elements with embedded strong
      discontinuities. IJNME 72, 1391-1433."
  - "ADR 0010 - Discontinuidades interiores embebidas (hoja de ruta; fase 2 = este
      elemento)."
  - "Spec [CohesiveDamageIsotropic](CohesiveDamageIsotropic.md) - material cohesivo
      consumido en Γ_d."
  - "Spec [Tri3](../../../specs/Tri3.md) - CST padre del elemento; mismo bulk en estado
       intacto."
```

A.3.5 Implementación

- Archivo: `solidum/user/discontinuities/embedded_cst.py`
- Clase: `CST_Embedded2D` (registrada en `ElementRegistry`)
- Estado de la discontinuidad: `solidum/user/discontinuities/discontinuity_state.py` (`DiscontinuityState`)
- Hook de paso en clase base: `Element.prepare_step(U_committed)` (no-op por defecto)
- Integración en solvers:
- `NonlinearSolver`: invoca `self.assembler.prepare_all_steps(U_current)` al inicio de cada paso, antes del Newton.
- `ArcLengthSolver`: idem, antes del predictor.
- Parser YAML: nueva sección `cohesive_materials:` + campo `cohesive_material:` por elemento.
- Tests: `tests/user/discontinuities/test_disc_cst_embedded.py` — 24 tests cubriendo `acceptance` completo (verification, specific, arch) + integración end-to-end del YAML. El aislamiento del trial del salto frente a `compute_global_stiffness` (deuda #18) se prueba en `tests/user/discontinuities/test_disc_contratos.py::TestTrialIsolation`.
- Notas de traducción:
- Estado intacto (`discontinuity_state is None`) → bit-exact con `Tri3` (mismo `B`, mismo `compute_integrands`). La cinemática del CST se importa de la API pública del programa principal, `solidum.elements.solid_2d` (`compute_kinematics_tri3`, `compute_integrands`; ADR 0020, P6), no del módulo interno `_shared`.
- Estado agrietado: Newton local sobre `[[u]]` hasta `R^{[[u]]} = 0`, después condensación estática local cerrada (`K_jj` es 2×2). Tolerancia interna: `_LOCAL_JUMP_RTOL = 1e-10`, `_LOCAL_JUMP_MAX_ITER = 30`.
- `B^φ` se obtiene como la submatriz `B_std[:, 2·i*:2·i*+2]` correspondiente al nodo solitario (consistente con `φ = N_{i*}`).
- Activación: `n` apunta hacia el nodo solitario (convención fija); si el autovector de `σ_I` deja 2 nodos en lado positivo, se invierte el signo de `n`.
- `_compute_ld` usa `|cos(θ − α)|` (valor absoluto): la longitud es positiva y simétrica bajo `n` → `−n` (test `ld_invariante_bajo_reflejo_de_n`).
- `compute_gauss_state` extendido: en estado agrietado añade clave `'discontinuity': {normal, tangent, centroid, solitary_node, l_d, jump, traction, damage}` para post-proceso VTK.
- Bulks aceptados en fase 1: `_ACCEPTED_BULKS = ('Elastic2D',)`. Restricción declarativa, ampliación futura es aditiva.

A.3.6 Diálogo

- **2026-05-18** · Spec pre-redactada por la IA como **orden de trabajo** sobre los Caps. 2, 5, 6 y 7 de Retama 2010. La cinemática KOS, la condensación estática y el `l_d = (A/h)·cos(θ−α)` son traducción directa de la tesis; las decisiones arquitecturales del ADR 0010 (familia paralela, condensación local, irreversibilidad, activación al principio del paso) están fijadas. Pendiente de validación del usuario.

- **2026-05-18** · Usuario delega las decisiones a la IA. Cierre con justificación arquitectural:

- **A — Bulk restringido a `Elastic2D` en fase 1.** Adoptado. Razón: la *discrete approach* del Cap. 3-7 de la tesis presupone bulk elástico (la disipación se localiza en `Γ_d`); mezclar plasticidad continua del bulk con cohesivo de `Γ_d` es una formulación distinta no cubierta por el ADR 0010. Validación cruzada en `__init__` con mensaje listando bulks aceptados. Apertura futura a otros materiales es aditiva.

- **B — Hook `prepare_step(U_committed)` en `Element` base, no-op por defecto.** Adoptado. Lo prescribe el ADR 0010 §5 explícitamente; aditivo (no rompe llamadores) y testeable directamente. Alternativa de meter activación en `compute_element_state` con guarda `_step_id` descartada por ser frágil y mezclar conceptos (preparación vs cálculo).

- **C — `INTERNAL_DOF_NAMES` no se añade al contrato declarativo todavía.** Resistir abstracción especulativa (Reglas.md §1: justificar con dos casos reales). Cuando entre el segundo elemento enriquecido (Quad4_Embedded o Tet4_Embedded), se centraliza la convención.

- **D — Post-proceso vía `compute_gauss_state(U)` extendido.** Adoptado. En estado intacto se comporta como `Tri3`; en estado agrietado añade la clave `'discontinuity': {'normal', 'jump', 'traction', 'damage', 'l_d'}`. Consistente con el patrón actual del `Tri3` y aditivo para `VtkExporter`. Un método separado `crack_state()` duplicaría el patrón sin beneficio.

- **E — Constructor `__init__(element_id, nodes, material, cohesive_material, thickness=1.0)`.** Mantengo el nombre `material` para el bulk (consistente con `Element.__init__`, que ya lo recibe así); `cohesive_material` es el parámetro nuevo. El YAML mapea `bulk_material: <name>` → `material=...` y `cohesive_material: <name>` → `cohesive_material=...`.

- **F — Tolerancia `1e-14` (bit-exact)** en `estado_intacto_reproduce_Tri3`. Si en la implementación emerge un desliz numérico documentado (por canceladas algebraicas tipo `G·G^T = I`), se relaja con razón explícita en el Diálogo. Empezar estricta es mejor que pre-laxar.

- **2026-05-18** · Status → en draft hasta que el código + tests cierren; se promoverá a `validated` cuando los `acceptance` pasen. La IA arranca implementación.

- **2026-09-22** · Auditoría global: tolerancia del jacobiano ahora **relativa** (`det J ≤ JACOBIAN_RTOL · Π‖fila_i(J)‖`, cota de Hadamard, adimensional): la absoluta `1e-10` sobre `det J` rechazaba mallas finas en metros sin distorsión.

- **2026-09-22** · Deuda #18 (estado): `compute_global_stiffness` restaura al salir el trial del material y el del salto (`jump_trial`, `cohesive_state_trial`), que el Newton local escribía al evaluar en `u = 0` (`tests/test_trial_isolation.py`).

- **2026-09-29** · Correcciones documentales: §10 y §12 describían una recuperación del salto en `commit_state()` que el código no hace (Newton local de la §7.2); §14 ya no es punto abierto; se retira el parámetro `activation_criterion` del contrato, que el constructor no acepta (la activación es siempre Rankine).

- **2026-09-29** · **Validación: ensayo de Van Vliet con grieta prescrita** (Retama 2010, §8.1), `tests/user/discontinuities/test_disc_van_vliet.py`, figura en `docs/user/discontinuities/validacion/van_vliet_fase1.png`. Probeta en hueso de perro con carga excéntrica; banda de `CST_Embedded2D` a media altura; `NonlinearSolver` con desplazamiento impuesto en pasos de 0.5 µm (convergidos: 0.25 y 0.125 µm cambian el pico < 0.1 kN). Frente a la fig. 8.3 de la tesis (valores leídos): exponencial, pico 26.6 kN (tesis 27.0) a 7.1 µm, P(20/50/100 µm) = 17.8 / 8.3 / 3.3 kN (19.1 / 8.3 / 3.1); lineal, pico 27.4 kN (27.4), 22.0 / 8.5 / 3.0 kN (22.4 / 8.5 / 2.4). Criterio de aceptación validado por el usuario: pico ±0.8 kN, δ en el pico entre 5 y 9 µm, P(20) ±1.5, P(50) ±1.0, P(100) ±0.7 kN. `K_e` = 1e13 o 1e14 da lo mismo. Diferencias abiertas: (a) más allá de ~100 µm la carga queda alta (exponencial 1.5 kN a 200 µm frente a 0.8): las mitades giran sobre el borde de la banda opuesto a la carga, ese elemento queda comprimido ($\sigma_yy \approx -31$ MPa) y nunca se activa, y la cola depende de la malla (1.19 kN con banda más fina); (b) con pasos de 1 µm el pico sube ~0.9 kN por el sobrepaso de la activación con el estado del paso anterior (fase 2 del plan de 2026-09-24). Supuestos no fijados por la tesis: acero E = 210 GPa, rótulas sin desplazamiento lateral (reacción horizontal nula).

- **2026-09-29** · Correcciones documentales tras endurecer la frontera del programa principal (ADR 0020): (1) la entrada del 2026-09-22 sobre la deuda #18 cita `tests/test_trial_isolation.py`; el aislamiento del trial del salto se prueba ahora en `tests/user/discontinuities/test_disc_contratos.py::TestTrialIsolation` (sección Implementación). (2) La cinemática que reutiliza el estado intacto es la API pública P6 del programa principal, `solidum.elements.solid_2d` (`compute_kinematics_tri3`, `compute_integrands`, sin guion bajo). (3) Nuevo caveat con lo que sigue siendo cierto de la limitación retirada de la spec de `NonlinearSolver`: este solver, con control por desplazamiento, atraviesa el ablandamiento hasta la separación; el fallo de `ArcLengthSolver` con penalización rígida es anterior a `597f3d9` y no se ha vuelto a comprobar. Sin cambio de formulación.

A.4 CohesiveDamageIsotropic

Orden de trabajo. Pre-redactada por la IA como borrador para validación del usuario (autor de la formulación, Retama 2010). La física sigue el Cap. 3 de la tesis, que adopta el modelo de daño isótropo con penalización de Alfaiate, Wells y Sluys (2002, ecs. 8-17): de esa referencia, y no de la tesis, vienen `K_e`, κ`_0` y las leyes $\omega(\kappa)$ (revisión del 2026-09-23); la mecánica de implementación (Voigt, registry, signatura, tests) sigue las convenciones del proyecto. El usuario revisa, corrige donde proceda, y aprueba antes de que la IA toque código.

A.4.1 Especificación física

0. Descripción general

Material cohesivo **traction-jump** para la superficie de discontinuidad `Γ_d` interior a un elemento finito sólido 2D. Modelo de **daño escalar isótropo** con activación tipo Rankine (Modo-I), ablandamiento gobernado por la energía de fractura `G_F`, y bulk vecino que descarga elásticamente sin disipación adicional.

Distinto en dominio de los materiales continuos `Elastic2D`, `VonMises2D`, `IsotropicDamage2D`:

- **No opera sobre puntos de Gauss del bulk.** Opera sobre el salto `[[u]]` $\in \mathbb{R}^2$ definido sobre `Γ_d` y devuelve tracciones `t` $\in \mathbb{R}^2$ definidas sobre la misma superficie.
- **Unidades**: parámetros físicos son σ`_t0` (Pa, resistencia a tracción) y `G_F` (N/m, energía de fractura) — no `E` y ν.
- **Es la primera implementación** de la familia `CohesiveMaterial` introducida por ADR 0010.

Pensado para hormigón, roca, cerámicos en régimen cuasi-frágil bajo tracción dominante. Diseñado específicamente para alimentar el elemento `CST_Embedded2D` (fase 2 del ADR 0010), pero la abstracción es reutilizable por futuros elementos de interfaz cohesiva clásica (CZM) si entran al proyecto.

1. Descomposición del salto

Sin descomposición elástica-plástica (modelo de daño puro, no plasticidad cohesiva). El salto total `[[u]]` se proyecta sobre el frame local `(n, s)` de `Γ_d`:

$$\mathbf{u} = u_n\,\mathbf{n} + u_s\,\mathbf{s}$$

donde `n` es la normal a `Γ_d` (fijada al activarse el elemento, perpendicular a σ`_I` por criterio Rankine) y `s` es la tangente.

Modo-I (fase 1): sólo $\langle$`[[u_n]]`$\rangle$ (parte positiva del salto normal) gobierna la disipación. La componente tangencial `[[u_s]]` desliza sin restricción y sin disipación. Modo mixto I-II queda diferido a la fase G del ADR 0010.

2. Ley elástica del salto (sin daño)

$$\mathbf{t} = (1-\omega)\,\mathbf{T}^{el}\,\mathbf{u}$$

con `T^el` la rigidez elástica del salto:

$$\mathbf{T}^{el} = K_e\,(\mathbf{n}\otimes\mathbf{n})$$

es decir: rigidez `K_e` en dirección normal, cero en dirección tangencial (consecuencia explícita de Modo-I).

`K_e` es una rigidez de penalty — debe escogerse suficientemente grande para que la fase elástica de la grieta sea geométricamente despreciable (`κ_0 = σ_t0/K_e` $\ll$ escala del problema), pero no tan grande que perjudique el condicionamiento del sistema. Guía típica: `K_e` $\approx$ `E_bulk / ℓ_c` con `ℓ_c` una longitud característica del elemento. Ver §12.

3. Variable equivalente y función de fluencia

$$u_{\text{eq}} = \langle u_n \rangle$$

donde $\langle\cdot\rangle$ son los brackets de McAuley (parte positiva). La función de fluencia (Eq. 3.13 de Retama 2010):

$$f(\mathbf{u},\kappa) = u_{\text{eq}} - \kappa \le 0$$

donde κ es el historial del salto equivalente máximo.

Interpretación: la grieta sólo "siente" la tracción (y por tanto sólo daña) cuando se abre (`[[u_n]] > 0`). En compresión (`[[u_n]] < 0`) o deslizamiento puro, `f = -κ < 0`, no hay evolución de daño.

4. Variable histórica y evolución (Kuhn-Tucker)

$$\kappa_{n+1} = \max\big(\kappa_n,\ \langle u_{nn+1}\rangle\big), \qquad \kappa_0 = \frac{\sigma_{t0}}{K_e}$$

Codifica:

- **Carga activa** ($\langle$`[[u_n]]`$\rangle$ `> κ_n`): κ crece; el daño puede aumentar.
- **Descarga / recarga elástica** ($\langle$`[[u_n]]`$\rangle \le$ `κ_n`): κ queda congelado; daño no cambia (secante respecto al origen con rigidez reducida `(1-ω)·K_e`).
- **Compresión** (`[[u_n]] < 0`): $\langle$`[[u_n]]`$\rangle$ `= 0` $\le$ `κ_n`, κ congelado. El daño no crece. Se recupera la relación elástica inicial `t_n = K_e·[[u_n]]`, sin daño: la penalización impide la interpenetración de las caras (Alfaiate et al. 2002, p. 667, *"if crack closure occurs, the initial elastic constitutive relation is recovered"*).
- **Irreversibilidad**: κ monótono no decreciente; `ω(κ)` monótono no decreciente.

Condiciones de Kuhn-Tucker estándar (Eq. 3.15):

$$\dot\omega \ge 0, \qquad f \le 0, \qquad \dot\omega\, f = 0$$

5. Ley de evolución del daño — softening lineal y exponencial

El daño $\omega(\kappa)$ se define implícitamente para que la tracción en carga monótona siga una curva prescrita `T_\{}text{soft}(κ)`:

$$t_n(\kappa) = (1-\omega(\kappa))\,K_e\,\kappa \;\overset{!}{=}\; T_{\text{soft}}(\kappa) \;\Rightarrow\; \omega(\kappa) = 1 - \frac{T_{\text{soft}}(\kappa)}{K_e\,\kappa}$$

con $\kappa \geq$ `κ_0`. Para $\kappa \leq$ `κ_0`, ω `= 0` por definición.

Softening lineal:

$$T_{\text{soft}}(\kappa) = \sigma_{t0}\left(1 - \frac{\kappa-\kappa_0}{w_c-\kappa_0}\right) = \frac{\sigma_{t0}\,(w_c-\kappa)}{w_c-\kappa_0}, \qquad w_c = \frac{2\,G_F}{\sigma_{t0}}$$

válida en $\kappa \in$ `[κ_0, w_c]`. Para $\kappa \geq$ `w_c`: `T_\{}text{soft} = 0`, ω `= 1` (grieta totalmente abierta). Forma cerrada de $\omega(\kappa)$:

$$\omega(\kappa) = 1 - \frac{\sigma_{t0}\,(w_c-\kappa)}{K_e\,\kappa\,(w_c-\kappa_0)} \qquad (\kappa \in [\kappa_0,\, w_c])$$

Verificación en los extremos: ω`(κ_0) = 1 - σ_{t0}/(K_e·κ_0) = 0` (por `κ_0 = σ_{t0}/K_e`), y ω`(w_c) = 1`.

Softening exponencial:

$$T_{\text{soft}}(\kappa) = \sigma_{t0}\,\exp\left(-\frac{\sigma_{t0}\,(\kappa-\kappa_0)}{H}\right), \qquad H = G_F - \frac{\sigma_{t0}\,\kappa_0}{2}$$

asintótica a cero; ω`(κ) → 1` cuando $\kappa \to \infty$. `H` se deriva imponiendo que la integral total de tracción a lo largo de la curva sea `G_F` (consistencia energética).

La validez de la rama exponencial exige `G_F > σ_{t0}·κ_0/2`, es decir `K_e > σ_{t0}²/(2·G_F)` — condición geométrica trivial cuando `K_e` se elige como penalty.

6. Variables internas

- κ `: float` — historial del salto equivalente máximo. **PRIMARY_STATE_VAR alternativa** (más fundamental, controla todo el resto).
- ω `: float ∈ [0, 1]` — daño escalar. **PRIMARY_STATE_VAR adoptada** (más interpretable físicamente; se exporta al post-proceso).
- ω es función determinista de κ (ec. §5); se cachea en el estado para evitar recálculo y para diagnóstico.

7. Frame local y notación

Convención del proyecto:

- `n`: normal a `Γ_d`, unitaria. Se fija al activar el elemento (perpendicular a `σ_I`).
- `s`: tangente, ortogonal a `n`, orientada por regla de la mano derecha con `z` saliendo del plano (consistente con Reglas.md §5).

- `[[u]] = [[u_n]]·n + [[u_s]]·s` con `[[u_n]] = [[u]]·n`, `[[u_s]] = [[u]]·s`.
- `t = t_n·n + t_s·s` con `t_n = K_e·(1−ω)·[[u_n]]` en Modo-I, `t_s = 0`.

El material recibe `[[u]]` y devuelve `t` en **ejes locales `(n, s)` de `Γ_d`**. La transformación a ejes globales es responsabilidad del elemento que lo consume, **no** del material. Mismo patrón que en `VonMises2D` (recibe ε en Voigt, no se ocupa de transformaciones de elemento).

A.4.2 Formulación numérica

8. Algoritmo en un paso (explícito, sin Newton local)

Dado κ`_n` y `[[u]]_{n+1}` (en ejes locales):

1. Calcular `[[u]]_eq` = $\langle$`[[u_n]]`$\rangle$ = `max(0, [[u]]·n_local)`. *(En el frame local, `n_local = (1, 0)` y `[[u_n]] = [[u]][0]`. Se mantiene la notación abstracta por claridad física.)*
2. Si `[[u]]_eq >` κ`_n` y `[[u]]_eq >` κ`_0`: **carga activa**, κ`_{n+1} = [[u]]_eq`. Si no: **descarga / no daño**, κ`_{n+1} =` κ`_n`.
3. Aplicar ω`(`κ`_{n+1})` por la fórmula §5.
4. Tracción: `t_n = (1 −` ω`)·K_e·[[u_n]]` si `[[u_n]]` $\geq$ `0` (en carga activa, `t_n = T_soft(`κ`)` evaluada directamente, sin pasar por `1 −` ω, que con `K_e` de penalización está a `~`κ`_0/`κ de cero); `t_n = K_e·[[u_n]]` si `[[u_n]] < 0`; `t_s = 0`.
5. Tangente: ver §9.

El algoritmo es **explícito**: no requiere Newton local. La actualización de κ y ω es directa. Mismo patrón que `IsotropicDamage2D`.

9. Tangente algorítmica consistente

$$\mathbf{T}^{\text{alg}} = \frac{\partial\, \mathbf{t}_{n+1}}{\partial\, \mathbf{u}_{n+1}}$$

Derivando `t = (1−`ω`)·K_e·[[u_n]]·n` en el frame local con `n = (1,0)`:

- **Sin daño activo** ($\kappa \leq$ κ`_0`):

$$\mathbf{T}^{\text{alg}} = K_e\,(\mathbf{n}\otimes\mathbf{n}) = K_e \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

- **Descarga** (`[[u]]_eq` $\leq$ κ`_n`):

$$\mathbf{T}^{\text{alg}} = (1-\omega)\, K_e\,(\mathbf{n}\otimes\mathbf{n})$$

(secante reducida; $\partial\kappa/\partial$`[[u]] = 0` anula el segundo término).

- **Carga activa** (`[[u]]_eq > κ_n`, `κ > κ_0`), en toda la rama de ablandamiento:

$$\mathbf{T}^{\text{alg}} = (1-\omega)\,K_e\,(\mathbf{n}\otimes\mathbf{n}) - \frac{d\omega}{d\kappa}\,K_e\,u_n\,(\mathbf{n}\otimes\mathbf{n})$$

$$= \left[(1-\omega) - \frac{d\omega}{d\kappa}\,u_n\right] K_e\,(\mathbf{n}\otimes\mathbf{n})$$

Es la pendiente de la envolvente, `dT_soft/dκ`: `-σ_{t0}/(w_c - κ_0)` en lineal y `-(σ_{t0}/H)·T_soft(κ)` en exponencial. Negativa en toda la rama y nula con la grieta totalmente abierta (lineal, $\kappa \geq$ `w_c`). Se evalúa así, directamente, y no por la expresión de arriba, que con `K_e` de penalización resta dos números casi iguales.

- **Cierre** (`[[u_n]] < 0`): `T^alg = K_e·(n⊗n)`, la rigidez inicial sin daño.

Sin rigidez residual. No se aplica ningún tope a la tangente. El sistema local del salto no se vuelve singular aunque `T^alg_nn` sea nula o negativa, porque la `K_jj` del elemento (Retama 2010, ec. 6.12) incluye el término del volumen `G^⊤·B_c^⊤·C·B_c·G·vol`, definido positivo mientras el elemento no sea tan grande que el ablandamiento lo supere (condición de tamaño, aproximadamente `l_d·|dT_soft/dκ| < E/h`). Hasta el 2026-09-23 un tope $\omega \geq$ `DAMAGE_MAX` sustituía la tangente de carga por `+1e-3·K_e`: como `1 - ω = T_soft/(K_e·κ)`, con `K_e` de penalización el tope actuaba desde $\kappa \approx$ `0.026·w_c` (tracción al 97 % de `σ_{t0}`), es decir, en casi todo el ablandamiento, con el signo cambiado y el elemento presentándose al solver como si endureciera.

Las derivadas `dω/dκ` cerradas:

- **Lineal**, en $\kappa \in$ `[κ_0, w_c]`. Reescribiendo `ω(κ) = 1 - C·(w_c - κ)/κ = 1 - C·w_c/κ + C` con `C = σ_{t0} / [K_e·(w_c - κ_0)]` constante:

$$\frac{d\omega}{d\kappa} = \frac{C\,w_c}{\kappa^2} = \frac{\sigma_{t0}\,w_c}{K_e\,\kappa^2\,(w_c-\kappa_0)}$$

Positiva y finita en todo el rango `[κ_0, w_c]`. Verificación en extremos: en `κ = κ_0` (con `κ_0 = σ_{t0}/K_e` ⇒ `K_e·κ_0² = σ_{t0}·κ_0`) toma el valor `w_c / [κ_0·(w_c - κ_0)]`; en `κ = w_c` toma `σ_{t0} / [K_e·w_c·(w_c - κ_0)]`. Dimensionalmente, `[dω/dκ] = m⁻¹` (ω adimensional sobre κ con unidad de longitud). ✓

- **Exponencial**, en `κ > κ_0`:

$$\frac{d\omega}{d\kappa} = \frac{T_{\text{soft}}(\kappa)}{K_e\,\kappa}\left(\frac{1}{\kappa} + \frac{\sigma_{t0}}{H}\right) = (1-\omega(\kappa))\left(\frac{1}{\kappa} + \frac{\sigma_{t0}}{H}\right)$$

(Análoga a la forma de `IsotropicDamage2D` con $\alpha \leftrightarrow$ `σ_{t0}/H`.)

10. Simetría de la tangente

A diferencia de `IsotropicDamage2D` con Mazars o von Mises modificado (donde el producto externo `(C_e·ε) ⊗ ∂ε_eq/∂ε` rompe la simetría; con la medida energética, la de por defecto desde el 2026-09-29, es simétrico), aquí ambos vectores que entran al rank-1 colapsan a `n`:

$$\mathbf{T}^{\text{alg}} = \alpha(\kappa, \omega, u_n)\, K_e\, (\mathbf{n} \otimes \mathbf{n})$$

con α escalar. `n ⊗ n` es simétrica, por tanto `T^alg` es **simétrica** en Modo-I.

Consecuencia: `IS_SYMMETRIC = True` para este material. El despachador algebraico (ADR 0003) puede usar Cholesky/LDL$^\top$ si el resto del dominio también es simétrico. **Modo mixto (fase G) reintroducirá asimetría** — se evaluará entonces.

11. Decisión de régimen durante el Newton global

Dentro de una iteración del Newton global, el material recibe `[[u]]_trial` y decide carga/descarga comparando con κ`_n` (estado committed del paso anterior). Mismo patrón que `IsotropicDamage2D` §9. La transición loading↔unloading es discontinua en la tangente; en problemas patológicos cerca del umbral, considerar arc-length (`ArcLengthSolver` ya disponible).

12. Caveats numéricos

- **Elección de `K_e` (penalty)**. Si `K_e` muy pequeño: κ`_0 = σ_{t0}/K_e` no es despreciable y la respuesta elástica deforma significativamente la curva (la grieta "se abre" antes del pico). Si `K_e` muy grande: condicionamiento del sistema empeora (`K_e ≫ E_bulk·L_e` introduce diferencia de magnitudes en `K_{ũũ}`). Regla práctica: `K_e ≈ 10·E_bulk/ℓ_c` con ℓ`_c` la longitud característica del elemento (su lado más corto). El usuario lo declara explícitamente en YAML; no hay default automático.

- **Cierre en compresión**. Con `[[u_n]] < 0` se recupera `t_n = K_e·[[u_n]]` sin daño (§4): la penalización impide la interpenetración aunque la grieta esté totalmente abierta. No hay fricción: la componente tangencial sigue sin rigidez.

- **Sin tope sobre ω ni sobre la tangente**. ω es físico (1.0 exacto en lineal con $\kappa \geq$ `w_c`) y la tangente es la consistente en toda la rama (§9). Truncar ω contaminaría la energía disipada en ~10-30 % (`K_e` de penalización y `u_n` macroscópico), y truncar la tangente le daba el signo contrario en casi todo el ablandamiento. Diferencia con `IsotropicDamage2D`, donde `K_e ≡ E` es moderado y el tope `DAMAGE_MAX` sí es inocuo.

- **Sensibilidad de malla** en régimen de softening. Sin regularización, el patrón de fallo localiza en función del tamaño y orientación de elementos. La integración con el elemento `CST_Embedded2D` (fase 2 del ADR 0010) y con el `l_d = (A/h)·cos(θ−α)` de Cap. 6 da objetividad parcial respecto a la longitud de la grieta — pero la dirección de propagación sigue dependiendo de las direcciones principales de esfuerzo en el elemento. Mitigación a nivel del *elemento*, no del material.

- **Validación energética**. Por construcción $\int$`_0^{w_c} t(w)·dw = G_F`. Esto se verifica en los tests (§acceptance). Si la curva implementada no integra a `G_F`, hay error en `w_c` o en `H`.

A.4.3 Contrato de implementación

```
name: CohesiveDamageIsotropic
kind: cohesive_material          # nueva familia, ADR 0010
status: validated

interface:
  jump_dim: 2                    # [[u]] ∈ℝ ² en 2D (componentes n, s en frame local)
  primary_state_var: damage      # ω, escalar exportable al post-proceso
  is_symmetric: true             # tangente simétrica en Modo-I (ver §10)

parameters:
  - { name: sigma_t0,            type: float, required: true,  desc: "Resistencia a
      tracción [Pa]; iniciación del daño" }
  - { name: G_f,                 type: float, required: true,  desc: "Energía de
      fractura [N/m]; área bajo la curva -t[[u_n]]" }
  - { name: K_e,                 type: float, required: true,  desc: "Rigidez elástica
      del salto en dirección normal [Pa/m] (penalty); ver §12" }
  - { name: softening,           type: str,   required: true,  desc: "Forma de la curva
      de softening: 'linear' | 'exponential'" }

signature:
  compute_traction: "([[u]]: ndarray(2,), state_vars=None) -> (t: ndarray(2,), T_tan:
      ndarray(2,2), state_vars')"
  jump_kind: "frame local - [[u]] = ([[u_n]], [[u_s]]) en ejes (n, s) de Γ_d"

state_schema:
  kappa: "float ≥κ _0 - historial del salto equivalente máximo"
  damage: "float ∈ [0, 1] - variable de daño escalar (1 - Sκ()/K_e, S la secante)"

conventions:
  sign: "[[u_n]] > 0 ⇔ apertura (tracción); el modelo daña sólo en apertura (Modo-I)"
  frame: "ejes locales (n, s) de Γ_d; transformación a globales por el elemento
      consumidor"
  damage_irreversibility: "κ monótono no decreciente vía maxκ(_old, ⟨[[u_n⟩]]); ωκ()
      monótono creciente"
  units: "σ[_t0] = Pa, [G_F] = N/m, [K_e] = Pa/m (asumiendo SI; el sistema no convierte
       unidades)"

validity:
  - "σ_t0 > 0, G_F > 0, K_e > 0"
  - "K_e > σ_t0² / (2·G_F) -  consistencia energética para softening exponencial (§5)"
  - "softening ∈ {'linear', 'exponential'}"

out_of_scope:
  - "Modo mixto I-II (cizalla acoplada al daño); diferido a fase G del ADR 0010"
  - "Fricción en la grieta cerrada; en compresión sólo se recupera la penalización
      normal K_e"
  - "Anisotropía del daño (tensor de daño D vs escalar ω)"
  - "Acoplamiento viscoso (rate-dependent); rate-independent puro en fase 1"
  - "Fatiga / acumulación cíclica del daño"
  - "Regularización para mesh-objectivity en propagación; se aborda a nivel de elemento
       (l_d correcto, Cap. 6) y no de material"

numerical_caveats:
```

```
  - "Penalty K_e: balance entre κ_0 despreciable (K_e grande) y condicionamiento (K_e
      moderado). Guía: K_e ≈ 10·E_bulk ℓ/_c. Declarado por el usuario en YAML, sin
      default automático."
  - "Cierre en compresión: t_n = K_e·[[u_n]] sin daño (Alfaiate et al. 2002, p. 667);
      sin fricción."
  - "Sin tope DAMAGE_MAX: ω físico y tangente consistente en toda la rama; la K_jj del
      elemento no se vuelve singular por el término del volumen (ver §9)."
  - "Régimen ↔elasticsoftening discontinuo en la tangente al cruzar κ_0; régimen
      ↔loadingunloading discontinuo al cruzar κ_n committed. Mismo patrón que
      IsotropicDamage2D."
  - "Con K_e de penalización, 1 -ω  está a κ~_0κ/ de cero: t_n y la tangente de carga
      se evalúan desde T_softκ(), no desde (1 -ω )·K_e, para no perder cifras por
      cancelación."

acceptance:
  verification:
    - name: respuesta_elastica_bajo_umbral
      setup: "carga monótona en [[u_n]] ∈ [0, κ_0); [[u_s]] = 0"
      expect: "ω = 0; t_n = K_e·[[u_n]] exacto; t_s = 0; T_tan = K_e·(⊗nn)"
      tol_rel: 1.0e-12

    - name: inicio_dano_en_umbral
      setup: "[[u_n]] cruza κ_0 = σ_t0/K_e por encima por primera vez"
      expect: "ω salta de 0 a un valor positivo continuo en κ_0 ωκ((_0) = 0+); t_nκ(_0)
          = σ_t0 exacto"
      tol_rel: 1.0e-10

    - name: energia_fractura_softening_lineal
      setup: "carga monótona [[u_n]] de 0 a w_c = 2·G_Fσ/_t0 con softening='linear',
          K_e tal que κ_0 ≪ w_c"
      expect: "∫_0^{w_c} t_n([[u_n]])·d[[u_n]] = G_F  (cuadratura trapezoidal con ≥1000
          puntos)"
      tol_rel: 1.0e-3
      note: "La tolerancia 1e-3 absorbe el error de cuadratura; la consistencia analí
          tica es exacta por construcción de w_c."

    - name: energia_fractura_softening_exponencial
      setup: "carga monótona [[u_n]] de 0 a 20·G_Fσ/_t0 con softening='exponential',
          K_e tal que κ_0 ≪ G_Fσ/_t0"
      expect: "∫ t_n·d[[u_n]] ≈ G_F·(1 -ε _truncamiento), con ε_truncamiento < 1e-6 por
          la cola exponencial cortada"
      tol_rel: 1.0e-3

    - name: descarga_secante
      setup: "carga hasta κ∈κ  (_0, w_c/2), después descarga [[u_n]] → 0"
      expect: "rama de descarga lineal con pendiente -ωκ(1())·K_e; al volver a [[u_n
          ]]=0, t_n = 0 y ω no ha cambiado"
      tol_rel: 1.0e-10

    - name: recarga_hasta_mismo_kappa_sin_dano_extra
      setup: "carga → descarga → recarga hasta [[u_n]] = κ (mismo κ histórico)"
      expect: "ω no cambia κ(_{n+1} = maxκ(_n, [[u_n]]) = κ_n)"
      tol_rel: 1.0e-12

    - name: recarga_mas_alla_continua_danando
      setup: "carga → descarga → recarga hasta [[u_n]] > κ histórico"
```

```
      expect: "ω evoluciona desde el valor previo hacia ω([[u_n]]_nuevo) sin saltos"
      tol_rel: 1.0e-10

  specific:
    - name: tangente_simetrica_en_carga_y_descarga
      setup: "estados de carga activa, descarga, sin daño, saturado"
      expect: "T_tan = T_tan^T (simetría exacta en Modo-I); ‖T_tan - T_tan^‖T_F /
          ‖‖T_tan_F < 1e-14"
      tol_rel: 1.0e-14

    - name: tangente_diferencia_finita_consistente
      setup: "carga activa con [[u_n]] ∈κ (_0, w_c/2); perturbar [[u]] con ε = 1e-7"
      expect: "‖T_tan_FD -‖ T_tan_analitica / ‖‖T_tan < 1e-5"
      tol_rel: 1.0e-5

    - name: tangente_consistente_en_todo_el_ablandamiento
      setup: "K_e = 1e15 (penalización real, κ_0/w_c ~ 3e-5), carga activa en κ/w_ref
          ∈ {0.01, 0.05, 0.3, 0.7, 0.95}, lineal y exponencial; y descarga a la mitad
          de κ"
      expect: "carga: T_tan < 0 e igual a la diferencia finita centrada; descarga:
          T_tan = secante T_softκ()κ/"
      tol_rel: 1.0e-6

    - name: tangencial_no_dana_ni_transmite
      setup: "[[u_s]] arbitrario, [[u_n]] = 0"
      expect: "t_s = 0; t_n = 0; ω no cambia"
      tol_rel: 1.0e-14

    - name: compresion_no_dana
      setup: "[[u_n]] < 0 puro (penetración)"
      expect: "⟨[[u_n]⟩] = 0 ⇒ f = -κ≤  0 ⇒ω  y κ no cambian. t_n = K_e·[[u_n]] y T_tan
          = K_e·(⊗nn) aunque la grieta esté parcial o totalmente abierta; t_n continua
          a través de [[u_n]] = 0"
      tol_rel: 1.0e-12

    - name: saturacion_en_w_c_softening_lineal
      setup: "[[u_n]] = w_c·1.01 con softening='linear'"
      expect: "ω = 1.0 exacto, t_n = 0 exacto y T_tan = 0 (pendiente de la envolvente;
          grieta totalmente abierta)."
      tol_rel: 1.0e-10

    - name: degeneracion_a_elasticidad_intacta
      setup: "σ_t0 →ϖ  (10^20 Pa) o cualquier [[u_n]] tan pequeño que jamás active"
      expect: "ω = 0, t_n = K_e·[[u_n]], T_tan = K_e·(⊗nn) (material intacto puro)"
      tol_rel: 1.0e-12

  arch:
    - name: registry_kind_es_cohesive
      expect: "Registro vía CohesiveMaterialRegistry, no MaterialRegistry; kind='
          cohesive_material' en YAML"

    - name: is_symmetric_attribute
      expect: "CohesiveDamageIsotropic.IS_SYMMETRIC == True"

references:
```

```
- "Retama Velasco, J. (2010). Formulation and Approximation to Problems in Solids by
    Embedded Discontinuity Models. Tesis Doctoral, UNAM. **Cap. 3 - Discrete damage
    models**. Fórmulas §3.1 (energía libre, ec. -3.13.4); §3.1.1 (tensor tangente, ec
    . -3.73.12); §3.1.2 (función de fluencia, ec. -3.133.14); §3.1.3 (Kuhn-Tucker, ec.
     -3.153.16)."
- "Alfaiate, J., Wells, G.N., Sluys, L.J. (2002). On the use of embedded
    discontinuity elements with crack path continuity for mode-I and mixed-mode
    fracture. *Engineering Fracture Mechanics* 69, 661-686. **Modelo de daño isótropo
     con penalización que sigue la tesis** (su cita [4] en la ec. 3.2): ecs. 8-17
    ωK(() exponencial, K₀≈  0, D_el = f_t0K₀/·I, condiciones de arranque) y p. 667 (
    recuperación de la rigidez inicial al cerrarse la grieta)."
- "Hillerborg, A., Modéer, M., Petersson, P.-E. (1976). Analysis of crack formation
    and crack growth in concrete by means of fracture mechanics and finite elements.
    *Cement and Concrete Research* 6, 773-782. **Origen del modelo cohesivo con energ
    ía de fractura G_F**."
- "Barenblatt, G.I. (1962). The mathematical theory of equilibrium cracks in brittle
    fracture. *Advances in Applied Mechanics* 7, 55-129."
- "Simó, J.C., Ju, J.W. (1987). Strain- and stress-based continuum damage models - I.
     Formulation. *Int. J. Solids Struct.* 23, 821-840."
- "ADR 0010 - Discontinuidades interiores embebidas (familia CohesiveMaterial, hoja
    de ruta)."
- "Spec [IsotropicDamage2D](../../../specs/IsotropicDamage2D.md) - patrón análogo de
    daño isótropo escalar para el caso continuo."
```

A.4.4 Implementación

- Archivo: `solidum/user/discontinuities/damage_isotropic.py`
- Clase: `CohesiveDamageIsotropic` (registrada en `CohesiveMaterialRegistry`)
- Base abstracta: `solidum/user/discontinuities/cohesive_material.py` (`CohesiveMaterial`)
- Registry: `solidum/user/discontinuities/registry.py` (`CohesiveMaterialRegistry`, familia de material del módulo de usuario, paralela a `MaterialRegistry`)
- Carga: explícita, nunca automática (ADR 0020). `user_modules: [discontinuities]` en YAML o `solidum.load_user_module("discontinuities")` en Python importan recursivamente los submódulos del módulo, y sus decoradores registran el material en `CohesiveMaterialRegistry`; la familia declara la sección YAML `cohesive_materials` (`YAML_SECTION`) y la etiqueta de sus mensajes (`YAML_LABEL = "material cohesivo"`). `solidum.autodiscover.initialize()` no la descubre.
- Tests:
- `tests/user/discontinuities/test_disc_cohesive_damage_isotropic.py` — todos los casos de `acceptance` (verification + specific + arch).
- `tests/user/discontinuities/test_disc_cst_embedded.py::TestCondensedTangentInSoftenin` y `tests/user/discontinuities/test_disc_embedded_uniaxial_softening.py` — el material dentro del elemento: tangente condensada frente a diferencias finitas y tracción uniaxial hasta la separación completa frente a la solución exacta.

- Notas de traducción:
- El algoritmo §8 es explícito (sin Newton local); no se usa `is_admissible` en este material.
- `JUMP_DIM = 2`, `PRIMARY_STATE_VAR = 'damage'`, `IS_SYMMETRIC = True`.
- Frame local interno: `n = (1, 0)`, `s = (0, 1)`; el material no transforma a globales (responsabilidad del elemento consumidor).
- `t_s` y `T_tan[1,1]` son 0 por construcción (Modo-I). El elemento que consuma el material debe estar preparado para rigidez tangencial nula.
- `K_e` obligatorio en el constructor (sin default). `kappa_0`, `w_c` (lineal) y `H` (exponencial) se derivan en `__init__`.

A.4.5 Diálogo

- **2026-05-13** · Spec pre-redactada por la IA como **orden de trabajo** sobre el Cap. 3 de Retama 2010 (tesis del usuario). La física es traducción directa de la tesis; la mecánica del proyecto (Voigt, registry, signatura, tests) sigue convenciones de `IsotropicDamage2D`. Pendiente de validación del usuario.
- **2026-05-18** · Usuario delega las tres decisiones a la IA. Cierre con justificación arquitectural:
- **PRIMARY_STATE_VAR = `damage` (ω)** — adoptado. Razones: (a) coherencia con `IsotropicDamage2D` (mismo criterio en toda la familia de daños), (b) $\omega \in [0, \text{DAMAGE_MAX}]$ es adimensional e interpretable físicamente y es lo que se colorea en el post-proceso VTK, (c) κ tiene unidades [m] y depende de `K_e` (no es portable entre materiales). κ se almacena como variable interna secundaria, controla todo determinísticamente vía $\omega(\kappa)$ cerrada.
- `dω/dκ` **lineal cerrada en §9** — hecho. Forma simplificada: `dω/dκ = σ_t0·w_c / [K_e·κ²·(w_c - κ_0)]`. Documentada en la spec para que el docstring del código pueda referenciarla en vez de derivarla.
- `K_e` **obligatorio en YAML, sin default automático** — adoptado. ADR 0010 §1 separa explícitamente `bulk_material` y `cohesive_material`; derivar `K_e` de `E_bulk·ℓ_c` los acoplaría a nivel de material y ensuciaría el contrato (el material recibiría `E_bulk` y `ℓ_c` como inputs ocultos). Si en una iteración futura el YAML pide derivado automático, la fábrica vivirá en el elemento (`CST_Embedded2D` construye el `CohesiveDamageIsotropic` con `K_e` derivado si se omite), no en el material.
- **2026-05-18** · Status → `validated`. La IA arranca implementación.
- **2026-05-18** · *Nota tras implementación*: durante los tests el corte por `DAMAGE_MAX` aplicado a ω introducía ~10 % de exceso en $\int$`t·d[[u_n]]` respecto a `G_F` (vs ~0.001 esperado). Razón física: en cohesivos `K_e` es penalty (≫ E_bulk) y `u_n` recorre rango macroscópico, así que `(1−DAMAGE_MAX)·K_e·u_n` no es despreciable. Fix arquitectural: el cap por `DAMAGE_MAX` se aplica **sólo a la rigidez tangente** (para mantener el Newton no singular), no al ω reportado ni a `t_n`. Actualizadas §9, §12 y `acceptance.saturacion_en_w_c_softening_lineal` consecuentemente. Distinto del patrón en `IsotropicDamage2D` por la diferencia en escalas de `K_e` y de la variable de entrada.

- **2026-09-22** · Auditoría global: el softening lineal exige `w_c >` κ_0 (`K_e >` σ`_t0`^{2}`/(2·G_F)`), la misma condición que el exponencial; antes sólo validaba el exponencial y con `w_c` $\leq \kappa_0\ \omega$ saltaba a 1 en cuanto $\kappa > \kappa_0$, disipando ½·σ_t0·κ_0 > G_F sin aviso.

- **2026-09-23** · *Revisión contra la tesis* (pedida por el usuario antes de usar la formulación en el manual de ejemplos). La tesis no escribe `K_e`, κ`_0` ni $\omega(\kappa)$: en la ec. 3.2 cita a Alfaiate, Wells y Sluys (2002) y reproduce casi literalmente su modelo de daño isótropo con penalización (ecs. 8-17), que es el implementado. El usuario confirmó mantener la penalización. Dos correcciones, ambas siguiendo esa referencia y validadas por el usuario: (1) se elimina el tope `DAMAGE_MAX` de la tangente, que en carga activa devolvía `+1e-3·K_e` en casi todo el ablandamiento (desde $\kappa \approx$ `0.026·w_c` con `K_e = 1e15`) en lugar de la pendiente negativa, de modo que la tangente condensada del elemento erraba del 26 al 45 %; (2) en compresión se recupera la rigidez inicial `K_e` sin daño, en vez de `(1 -` ω`)·K_e`, que dejaba atravesarse las caras de una grieta abierta. Tests nuevos: barrido de la tangente en toda la rama, compresión con grieta abierta, tangente condensada frente a diferencias finitas (los tres fallan con el código anterior) y tracción uniaxial hasta la separación completa frente a la solución exacta.

- **2026-09-29** · Corrección documental (sección Implementación): el material no lo descubre `solidum.autodiscover.initialize()`, sino la carga explícita del módulo de usuario (`user_modules: [discontinuities]` o `solidum.load_user_module("discontinuities")`, ADR 0020). La familia cohesiva sigue en este módulo porque su único consumidor es `CST_Embedded2D`; una ley tracción-salto genérica con elementos de interfaz sería estándar según el criterio de admisión (Reglas.md §1) y pasaría al programa principal con el primer elemento de interfaz que la use. Sin cambio de formulación.

Apéndice B

Módulos de usuario — dissipation_control

Este apéndice recoge los **módulos de usuario**: formulaciones no estándar que viven fuera del programa principal, en `solidum/user/<módulo>/`, a la manera de los elementos y materiales de usuario de FEAP (ADR 0020). El programa principal no las carga por sí solo: un modelo las pide con `user_modules: [<módulo>]` en el YAML o con `solidum.load_user_module("<módulo>")` en Python. Cada capítulo reúne las entradas de catálogo de un módulo y sus specs.

Módulo `dissipation_control`: control del paso por disipación de energía (ADR 0020).

- Código: `solidum/user/dissipation_control/`.
- Specs: `docs/user/dissipation_control/specs/`.
- Carga en YAML: `user_modules: [dissipation_control]`.
- Carga en Python: `solidum.load_user_module("dissipation_control")`.

B.1 Solvers — catálogo

Módulo de usuario *(ADR 0020): formulación no estándar, fuera del programa principal, en* `solidum/user/dissipation_control/`*, como un macro de usuario de FEAP.* `import solidum` *no la carga. Estuvo en el programa principal del 2026-05-19 al 2026-09-29.*

Clasificación *(criterio de admisión, Reglas.md §1, 2026-09-29):* ***no estándar****, porque no cumple la condición (ii). Ningún código general establecido implementa de forma nativa el control por liberación de energía de Gutiérrez (2004) y Verhoosel, Remmers y Gutiérrez (2009): DIANA ofrece como restricciones del arco el Spherical Path y el Updated Normal Plane, y su método «energy based» es una regla de tamaño de paso, no esta restricción; el* `PILOTAGE` *de Code_Aster no la incluye entre sus tipos; en Abaqus sólo existen implementaciones de investigación mediante subrutinas de usuario.*

Carga*: en YAML,* `user_modules: [dissipation_control]` *en el primer nivel (sin esa clave, y si nada lo ha cargado antes en la sesión,* `solver: {type: DissipationArcLengthSolver}` *es un error que sugiere declararla y lista los módulos disponibles); en Python,* `solidum.load_user_module("dissipation_control")` *o* `from solidum.user.dissipation_control import`

`DissipationArcLengthSolver. from solidum.math.solvers import DissipationArcLengthSolver` ya no funciona.

Spec en `docs/user/dissipation_control/specs/`, tests en `tests/user/dissipation_control/`. Los benchmarks cohesivos de su spec usan además el módulo de usuario `discontinuities` (`user_modules: [discontinuities, dissipation_control]`).

B.1.1 DissipationArcLengthSolver — variante de arc-length por disipación (Gutiérrez 2004)

- **Propósito**: trazar la rama post-pico de problemas con **softening severo** controlando la disipación incremental de energía por paso en lugar de la longitud de arco euclídea. Pensado para curvas U-λ con tramos casi verticales en el plano (U, λ), incluidos los retrocesos (*snap-back*).

- **Alcance (2026-09-29)**: **daño con descarga secante** (continuo o cohesivo), la hipótesis de la que sale la expresión de Δ`E_d`, y **pequeñas deformaciones** (el criterio de arranque por la tangente de carga, desde el 2026-09-30, lo exige). **Plasticidad, fuera del alcance**: con deformación plástica la descarga no es secante; la variante para plasticidad de Verhoosel et al. (2009) queda como ampliación futura.

- **Esquema**:

- Subclase de `ArcLengthSolver` (programa principal); reusa predictor tangente, corrector Newton, ajuste de paso, Dirichlet/MPC y backend algebraico. Su problema por paso hereda de `solidum.math.solvers.ArcProblem` y sólo redefine la restricción.

- **Restricción de paso**: `g(`Δ`U, `$\Delta\lambda$`) = ½·(`λ`_n·F·`Δ`U - `$\Delta\lambda$`·F·U_n) = `τ. **Lineal** en (`$\Delta$`U, `$\Delta\lambda$`) $\Rightarrow$ corrector con una sola raíz en `dd`λ, sin selección por menor ángulo ni patología de raíces imaginarias.

- **Tangente de carga** (desde 2026-09-30, decisión del usuario): el predictor del modo de disipación y el arranque de ese modo usan `K_L`, la tangente del último iterado convergido del paso anterior (la que tenía el Newton al converger), en lugar de la del estado consolidado, que con daño y descarga secante es la secante y anula el denominador de la restricción (α = `-½·F·K_s`$^{-1}$`·r_n` $\approx$ `0`, sólo el residuo). Con `du_L = K_L`$^{-1}$`·F`, α`_L = ½·(`λ`_n·F·du_L - F·U_n) = ½·`λ`_n·F·(K_L`$^{-1}$` - K_s`$^{-1}$`)·F` (más el ruido del residuo) mide el crecimiento del daño. Una resolución lineal más por paso.

- **Switching automático cilíndrico↔disipación**: arranque del modo de disipación en el predictor cuando `|`α`_L| > 1e-3·½·|`λ`_n·F·du_L|` (la tangente de carga se separa de la secante), con τ = `initial_tau`; conmuta en el paso siguiente al inicio del daño. Sustituye desde 2026-09-30 al umbral de Δ`E_d` tras un paso cilíndrico, que en un pico anguloso no se alcanza. Vuelta a cilíndrico sin cambios: α $\approx$ `0` en el predictor o Δ`E_d < dissipation_threshold·‖F_ref‖·‖U‖` tras el commit (umbral **relativo**, desde 2026-09-29 sin el piso `max(·, 1)`, que estaba en unidades de energía). Arranque obligado en cilíndrico (α es 0 cuando λ`_n = U_n = 0`).

- **Predictor**: en modo cilíndrico, el del padre, orientado por el signo del producto escalar con el incremento del paso anterior. En modo de disipación, Δ`U = `$\Delta\lambda$`·du_L` con $\Delta\lambda$ = τ/α`_L` (desde 2026-09-29 τ/α; con la tangente de carga desde 2026-09-30): la restricción lineal da `g =`

$\alpha \cdot \Delta\lambda$ y exigir `g = τ > 0` fija valor y signo; con α `< 0` el paso reduce la carga, como exige un retroceso. Antes del 2026-09-29, `sign·|τ/α|` con el signo del padre, que arrancaba en `g = −τ` cuando discrepaba de `sgn(α)`. **Sin seguimiento de inercia** de `K_t`: el diagnóstico por signo del determinante (`_negative_pivots`), que ningún solver llamaba, se retiró el 2026-09-29; un seguimiento exacto requiere LDL$^\top$ Bunch-Kaufman (deuda técnica #7 STATUS.md).

- **Llegada a `max_lambda`**: la del padre (paso que cruza, no consolidado, repetido con λ fijo dentro del tramo). Sustituye desde 2026-09-23 a la salvaguarda contra el `final_step` prematuro.
- **Detección de $\alpha \approx 0$**: threshold relativo `|α| < 1e-6·escala`; revierte a cilíndrico. $\alpha \equiv 0$ en problemas lineales monótonos y cuando la tangente de carga vuelve a ser la secante (descarga, daño saturado). Con el arranque en `1e-3` forma una histéresis que evita oscilar entre pasos contiguos. La guarda contra la división por α nulo es también relativa desde 2026-09-30 (antes, `|α| < ZERO_TOL` en unidades de energía rompía el trazado con energías de ~1e-12): el trazado no depende del sistema de unidades.
- **Parámetros heredados**: todos los de `ArcLengthSolver` (`max_iter`, `max_lambda`, `initial_dlambda`, `initial_dl`, `max_steps`, factores `dl_*`, `linear_algebra`). **Nuevos**: `initial_tau` (obligatorio), `tau_grow_factor`, `tau_max_factor`, `tau_shrink_factor`, `tau_grow_iter_threshold`, `tau_shrink_iter_threshold`, `dissipation_threshold`.
- **YAML**:

```yaml
user_modules: [dissipation_control]
solver:
  type: DissipationArcLengthSolver
  initial_dlambda: 0.05
  initial_tau: 1.0e-4
  max_steps: 300
```

- **Cuándo usarlo**: daño con descarga secante (bulk 1D/2D o cohesivo) donde `ArcLengthSolver` cilíndrico bisecta excesivamente cerca del pico, y retrocesos por localización cuando no se quiere elegir una magnitud cinemática que controlar (la alternativa del programa principal es `IndirectDisplacementSolver`). Con un primer paso pequeño (ver el límite de abajo).
- **Cuándo NO usarlo (limitación documentada)**: cohesivo+embedded discontinuity (módulo de usuario `discontinuities`) con `K_e` stiff (`~1e13`). La activación discreta del Rankine introduce un salto en `F_int/K_t` que el `sign` por producto escalar no maneja graciosamente; el predictor del paso siguiente reorienta hacia descarga. Resolverlo requiere LDL$^\top$ Bunch-Kaufman real (item #7 deuda STATUS.md) o control por CMOD/CTOD del salto (spec separada). En problemas lineales puramente elásticos coincide con `ArcLengthSolver` cilíndrico a paridad de bits (regresión validada).
- **Retroceso (validado 2026-09-30; deuda #27)**: sigue el retroceso exacto de una barra de 10 elementos con daño localizado en uno (caso del test de `IndirectDisplacementSolver`, con sus tolerancias): 172 puntos post-pico sobre la curva exacta con distancia normalizada máxima $3.2 \cdot 10^{-4}$, `u_min/u_pico = 0.616376` (el exacto) y sólo el elemento débil dañado, con `initial_dlambda = 0.02`. La rama de ablandamiento se resuelve entera en modo de

disipación; al saturar el daño el solver vuelve a cilíndrico. Hasta el 2026-09-29 no lo seguía: el predictor, con la tangente secante del estado consolidado, devolvía cada paso a cilíndrico (en los tests del módulo ningún problema de Newton se construía en modo de disipación) y el arco cilíndrico se atascaba en el pico anguloso sin conmutar. Test: `test_dctl_retroceso.py`. En 2D (franja de Quad4 con `IsotropicDamage2D`, medida energética, columna débil) arranca en disipación tras el inicio del daño, sigue el retroceso y la disipación de sus pasos coincide con la del material en los puntos de Gauss (`test_dctl_dissipation_arclength.py`, test 6).

- **Límites conocidos (2026-09-30)**: (1) **primer paso grande**: el primer paso con daño es siempre cilíndrico, y con `initial_dlambda = 0.1` el que cruza el pico converge a otro equilibrio, con los diez elementos ablandándose, que el solver sigue después sin retroceso. Es la no unicidad del problema incremental con ablandamiento, no un defecto del método; queda fijado como caso conocido en un test. (2) **Alcance del arranque por** α`_L`: pequeñas deformaciones y descarga secante; con no linealidad geométrica o plasticidad α`_L` $\neq$ `0` sin disipación. Supone además `rtol_force` $\ll$ `1e-3`. (3) El paso de disipación que cruza la saturación del daño (`DAMAGE_MAX`) salta por la rama de rigidez residual (en la barra, de ε`_w` $\approx$ `10.4`$\cdot\kappa_0$`w` a $\approx$ `46.5`$\cdot\kappa_0$`w`), sobre la curva exacta pero sin trazar el tramo intermedio.
- **Referencias**: Gutiérrez (2004), *Communications in Numerical Methods in Engineering* 20, 19-29; Verhoosel, Remmers, Gutiérrez (2009), *IJNME* 77, 1290-1321; Crisfield (1991) vol. 1 §9.4.
- **Spec**: docs/user/dissipation_control/specs/DissipationArcLengthSolver.md (status `implemented`, validación parcial).
- **Archivo**: solidum/user/dissipation_control/dissipation_arclength.py.
- **Tests**: `test_dctl_dissipation_arclength.py` (con el modo de cada problema de Newton y, en la barra 1D, la disipación de los pasos de disipación frente a la analítica del material), el retroceso de la barra con daño localizado y el límite con un primer paso grande, `test_dctl_retroceso.py`, el barrido de contrato del módulo (solver, gancho de inicio de paso, carga YAML), `test_dctl_contratos.py`, y la validación de la spec, `test_dctl_specs.py`.

B.2 DissipationArcLengthSolver

Módulo de usuario `dissipation_control` *(ADR 0020): código en* `solidum/user/dissipation_control/`*, tests en* `tests/user/dissipation_control/`*.* `import solidum` *no lo carga: en YAML,* `user_modules: [dissipation_control]`*; en Python,* `solidum.load_user_module("dissipation_control")`*. Estuvo en el programa principal (*`docs/specs/`*,* `solidum/math/solvers/`*) del 2026-05-19 al 2026-09-29.*

Clasificación *(criterio de admisión, Reglas.md §1, 2026-09-29):* ***no estándar****. La condición (ii) no se cumple: ningún código general establecido implementa la restricción de Gutiérrez de forma nativa. DIANA ofrece como restricciones del arco el Spherical Path y el Updated Normal Plane, y su método «energy based» es una regla de tamaño de paso (fija el incremento de carga con el producto de los incrementos del primer predictor), no esta restricción; los* `TYPE` *del* `PILOTAGE` *de Code_Aster (*`DDL_IMPO`*,* `LONG_ARC`*,* `ANA_LIM`*,* `DEFORMATION`*,* `PRED_ELAS`*,* `SAUT_IMPO`*,* `SAUT_LONG_ARC`*) no la incluyen; en Abaqus sólo existen implementaciones de investigación mediante subrutinas de usuario.*

Spec corta tipo extensión *(Reglas.md §4). Variante de* `ArcLengthSolver` *(programa principal) que* ***solo cambia la restricción del paso****: pasa de cilíndrica ($\|\Delta\mathbf{U}\|^2 = dl^2$) a basada en* ***disipación de energía*** *($g(\Delta\mathbf{U}, \Delta\lambda) = \tau$). Reusa todo el plumbing del padre (predictor tangente, corrector Newton, adaptatividad de paso, Dirichlet/MPC, backend algebraico) a través de su parte pública: la clase* `ArcLengthSolver` *y su problema de Newton por paso* `solidum.math.solvers.ArcProblem`*. Sin ADR nuevo: reusa el contrato arquitectural ya aceptado del arc-length.*

Motivación: el `ArcLengthSolver` *cilíndrico no atraviesa la transición elástico→softening con penalty stiff (*`K_e` *de la discontinuidad embebida, ADR 0010 fase 4). La restricción cuadrática queda casi degenerada cerca del pico — el discriminante de la cuadrática tiende a cero, el solver bisecta indefinidamente y aborta. (Nota 2026-09-29: ese diagnóstico es anterior a la corrección de la tangente cohesiva del 2026-09-23, commit* `597f3d9`*, que presentaba a los solvers una tangente de signo contrario en casi toda la rama de ablandamiento, y no se ha vuelto a comprobar; con control por desplazamiento,* `NonlinearSolver` *sí atraviesa hoy esa rama. Ver el caveat de* `CST_Embedded2D`*.) La restricción de disipación es* ***lineal en*** *$(\Delta\mathbf{U}, \Delta\lambda)$ y se activa* ***proporcionalmente a la energía consumida*** *por la grieta cohesiva, navegando la rama post-pico de forma estable. Esta variante apuntaba a desbloquear el benchmark de fractura faithful Van Vliet (*`CST_Embedded2D` *§"Out of scope"), el balance energético end-to-end de G_F (hueco abierto en [[project_validacion_fase_b]]), y validación cuantitativa de cohesivos en pipeline real. Esos benchmarks usan componentes de* ***otro módulo de usuario****,* `discontinuities` *(elemento embebido y ley cohesiva): un modelo que los combine con este solver declara los dos,* `user_modules: [discontinuities, dissipation_control]`*.*

B.2.1 Especificación física

0. Descripción general

Solver estático no lineal con **traza por disipación**: la magnitud que se mantiene constante de paso a paso ya no es la longitud euclídea $\|\Delta\mathbf{U}\|$ sino la **disipación incremental de energía** ΔE_d asociada al paso. Pensado para:

- Fractura cohesiva (Mode-I y, en el futuro, mixto) con penalty `K_e` stiff.
- Daño con softening severo donde la rama post-pico tiene pendiente casi vertical en el plano (U, λ).

Alcance (2026-09-29): daño con descarga secante —continuo o cohesivo—, es decir, materiales cuya fuerza interna es $\mathbf{F}_{\text{int}} = \mathbf{K}_s(\text{daño})\,\mathbf{U}$ y que descargan hacia el origen. Es la hipótesis de la que sale la ΔE_d de §1, y el código no tiene otra. **Plasticidad, fuera del alcance**: con deformación plástica la descarga no es secante y esa expresión no mide la disipación. La variante para plasticidad de Verhoosel et al. (2009), que descuenta la energía de la descarga elástica, queda como ampliación futura.

Estado de uso medido (2026-09-30): desde el cambio de formulación del 2026-09-30 (§5: tangente de carga en el predictor y arranque del modo de disipación por α_L) la rama de ablandamiento se resuelve de verdad con la restricción de disipación, y el solver sigue el retroceso exacto de la barra con daño localizado (acceptance `retroceso_barra_dano_localizado`: 172 puntos post-pico, distancia normalizada máxima $3.2 \cdot 10^{-4}$, $u_{\text{mín}}/u_{\text{pico}} = 0.616376$). Hasta el 2026-09-29 el predictor del modo de disipación resultaba degenerado ($\alpha \approx 0$ con la tangente secante del estado consolidado) y el solver se comportaba en la práctica como el arco cilíndrico del padre (deuda #27 de STATUS.md).

Régimen elástico (sin disipación): la restricción degenera a $g = 0$. El solver detecta este caso y **conmuta automáticamente** a arc-length cilíndrico durante el régimen elástico, y pasa a control por disipación cuando la **tangente de carga** del último paso convergido se separa de la secante (§5). La detección es global, sobre matrices y vectores ensamblados: el solver no consulta las variables internas del material (daño, apertura de la grieta).

1. Ecuación de equilibrio resuelta

Idéntica al padre:

$$\mathbf{F}_{\text{int}}(\mathbf{U}) = \lambda\,\mathbf{F}_{\text{ext}}^{\text{ref}}$$

con λ incógnita. La **restricción del paso** cambia respecto al padre.

Restricción de disipación (Gutiérrez 2004, Verhoosel et al. 2009) Hipótesis: daño con descarga secante, $\mathbf{F}_{\text{int}} = \mathbf{K}_s\,\mathbf{U}$ con $\mathbf{K}_s$ función sólo del daño. La energía elástica almacenada es $\frac{1}{2}\mathbf{U}^T\mathbf{K}_s\mathbf{U}$ y la tasa de disipación es la liberación de energía por degradación de $\mathbf{K}_s$:

$$\dot{E}_d = -\tfrac{1}{2}\,\mathbf{U}^T\dot{\mathbf{K}}_s\,\mathbf{U} = \tfrac{1}{2}\left(\lambda\,\mathbf{F}_{\text{ext}}^{\text{ref}T}\,\dot{\mathbf{U}} - \dot{\lambda}\,\mathbf{F}_{\text{ext}}^{\text{ref}T}\,\mathbf{U}\right),$$

donde la segunda igualdad sale de derivar el equilibrio $\mathbf{K}_s\mathbf{U} = \lambda\,\mathbf{F}_{\text{ext}}^{\text{ref}}$. Integrada en el paso con el estado inicial (Gutiérrez 2004), la energía disipada por paso es:

$$\Delta E_d = \frac{1}{2}\left(\lambda_n\,\mathbf{F}_{\text{ext}}^{\text{ref}T}\,\Delta\mathbf{U} \;-\; \Delta\lambda\,\mathbf{F}_{\text{ext}}^{\text{ref}T}\,\mathbf{U}_n\right)$$

donde $(\mathbf{U}_n, \lambda_n)$ son los valores committed al inicio del paso y $(\Delta\mathbf{U}, \Delta\lambda)$ los incrementos del paso. Con deformación plástica la descarga no es secante y esta expresión no mide la disipación (§0). La restricción es:

$$g(\Delta\mathbf{U}, \Delta\lambda) \;\equiv\; \frac{1}{2}\left(\lambda_n\,\mathbf{F}_{\text{ext}}^{\text{ref}T}\,\Delta\mathbf{U} \;-\; \mathbf{U}_n^T\,\mathbf{F}_{\text{ext}}^{\text{ref}}\,\Delta\lambda\right) \;=\; \tau$$

con τ el **incremento de disipación objetivo** (parámetro del solver, papel análogo al `initial_dl` del padre).

Linealidad: g es lineal en $(\Delta\mathbf{U}, \Delta\lambda)$. Por eso el corrector tiene **una sola raíz**, no dos como en la cuadrática cilíndrica — eliminando la selección por menor ángulo y la patología de raíces imaginarias en pasos grandes.

2. Condiciones de contorno

Idénticas al padre (`ArcLengthSolver` §2). Dirichlet homogéneo/no homogéneo y MPC vía ADR 0004.

3. Salidas físicas

Idénticas al padre: `SolveResult` con $\mathbf{U}$ final y `step_callback(step, U, λ)` para la historia, con la misma firma que en el padre. El solver **no expone** la disipación del paso ni la acumulada $\sum_n \Delta E_d^{(n)}$: la calcula tras cada paso convergido para decidir la vuelta a cilíndrico (§5) y sólo la escribe en el registro (`ΔE_d=…` en el log del solver). Quien la necesite —por ejemplo, para contrastar G_F **de extremo a extremo** con $\int_{\Gamma_d} t \cdot d[\![u]\!] \approx \sum_n \Delta E_d^{(n)}$— la reconstruye desde la historia $(\mathbf{U}_n, \lambda_n)$ del callback con la fórmula de ΔE_d de §1, como hace el test 5 de `test_dctl_dissipation_arclength.py`.

B.2.2 Formulación numérica — solo lo que cambia respecto al padre

4. Esquema operativo

Estructura idéntica al padre con cuatro diferencias: el predictor y la restricción del corrector dependen del modo, el arranque del modo de disipación se decide en el predictor con la tangente de carga, y tras cada paso convergido se decide la vuelta a cilíndrico (§5). El esquema describe el `solve` del código (`dissipation_arclength.py`):

```
λ
 = 0;  U = 0; Δ U_prev = 0τ
 = initial_tau          # análogo a initial_dl, pero en unidades de energía
modo = "cylindrical"      # siempre arranca así: con λ_n = 0, U_n = 0 es α = 0
dl = (sin fijar)
K_L = (ninguna)           # tangente de carga: la del último iterado convergido
mientras λ < max_lambda y step < max_steps:
    step += 1
    prepare_all_steps(U)                  # gancho de inicio de paso (ADR 0020, P5)

    # Predictor tangente (el del padre), ensamblado en el estado consolidado U_nδ
    u_t = K_t(U_n)^{-1} · F_ext^ref      # con daño y descarga secante, K_t(U_n) = K_s
    si K_t singular:                      # el predictor no puede resolver
        si dl fijado: dl /= 2τ
         /= 2                            # se dividen los dos, sin comprobar cotas
        continuar                         # reintento del paso
    si dl sin fijar: dl = dl_ref = Δ₁l     # primer paso del padre (initial_dlambda /
        initial_dl)
    sign = signΔ(U_prev ·δ u_t)           # regla del padre (sólo modo cilíndrico)

    # Tangente de carga (desde 2026-09-30)δ
    u_L = K_L^{-1} · F_ext^ref           # ninguna si aún no hay paso convergido o K_L
        es singular
```

```
    # (b) Arranque del modo de disipación por α_L (desde 2026-09-30)
    si modo == "cylindrical" y δu_L existe:α
        _L = ½·λ(_n·F_ext^ref·δu_L - F_ext^ref·U_n)
        si α|_L| > 1e-3 · |½·λ_n·F_ext^ref·δu_L|:   # la tangente de carga se separa de
             la secante
            modo = "dissipation"; τ  = initial_tau

    modo_paso = modoδ
    u = δu_t
    si modo_paso == "dissipation":δ
        u = δu_L si existe                # (a) predictor sobre la tangente de cargaα
         = ½·λ(_n·F_ext^ref·δu - F_ext^ref·U_n)
        escala = |½·λ_n·F_ext^ref·δu|
        si (escala > 0 y α|| < 1e-6·escala) o α_nuloα():
            modo_paso = modo = "cylindrical"   # α no significativo (§5)
        # α_nuloα(): α|| ≤ ZERO_TOL·½·λ(|_n·F·δu| + |F·U_n|), relativa (2026-09-30)
    si modo_paso == "cylindrical":Δλ
        _pred = sign · dl / ‖δ(‖u_t + ZERO_TOL)δ
        u = δu_t
    sino:Δλ
        _pred = τα/                      # la restricción fija valor y signo (desde
            2026-09-29)Δ
    U_pred = Δλ_pred ·δ u

    # Corrector iterativo (NewtonCorrector compartido, ADR 0015)
    para iter en range(max_iter):
        K_t, F_int = assemble(...)
        R = λ( + Δλ)·F_ext^ref - F_intδ
        u_R = K_t^{-1} · Rδ
        u_t = K_t^{-1} · F_ext^ref
        si modo_paso == "cylindrical":
            # Restricción cuadrática, idéntica al padre (ArcLengthSolver §5)λ
            dd = raiz_de_menor_angulo(...)
        sino:
            # Restricción lineal de Gutiérrez: gΔ(U + δu_R + λdd·δu_t, Δλ + λdd) = τ
            num = τ-  ½·λ(_n·F_ext^ref·Δ(Uδ+u_R) -Δλ ·F_ext^ref·U_n)
            den = ½·λ(_n·F_ext^ref·δu_t - F_ext^ref·U_n)     # α con el δu_t de la
                iteración
            si α_nulo(den): abortar el paso                   # CorrectionAborted →
                bisecciónλ
            dd = num / denΔ
        U += δu_R + λdd·δu_tΔλ
         += λdd
        si converge: break

    si converge y λ + Δλ > max_lambda:    # llegada exacta del padre (ArcLengthSolver.
        _land)
        repetir desde (U_n, λ_n) con λ = max_lambda fijo
        si converge: consolidar y continuar     # sin decidir modo ni adaptar dlτ/
        sino: tratar como paso fallido
    si converge:
        K_L = K_t del ensamblaje en el iterado convergido   # prueba de carga respecto
            del estado anteriorΔ
        E_d = ½·λ(_n·F_ext^ref·ΔU -Δλ ·F_ext^ref·U_n)       # paso completo, tras
            corregir
        umbral = dissipation_threshold ·‖ F_ext^‖ref·‖U_n‖+1    # relativo, sin piso (
```

```
            desde 2026-09-29)
        si modo == "dissipation" y ΔE_d < umbral:
            modo = "cylindrical"          # el arranque ya no se decide aquí (desde
                2026-09-30)
        adaptar dl (modo cilíndrico) o τ (modo disipación) según el modo del SIGUIENTE
            paso (§9)
        consolidar: U += ΔU; λ  += Δλ; Δ U_prev = ΔU
    sino:                                 # bisección según el modo del paso fallido
        si modo_paso == "cylindrical":
            dl *= 0.5;  si dl < ARCLENGTH_MIN_DL_FACTOR·dl_ref: abortar (RuntimeError)
        sino:τ
             *= 0.5;   si τ < ARCLENGTH_MIN_DL_FACTOR·initial_tau: abortar (
                 RuntimeError)
```

El paso fallido reduce τ a la mitad (τ `*= 0.5`); `tau_shrink_factor` sólo actúa sobre pasos que convergen lentamente (§9). La tangente de carga se guarda sólo tras un paso con restricción que converge: un paso fallido conserva la del último paso convergido, y la llegada a `max_lambda` termina el trazado.

Coste: una resolución lineal más por paso (la de $\delta\mathbf{u}_L$, con el mismo backend y la misma reducción de Dirichlet/MPC que el predictor del padre) y una matriz tangente guardada entre pasos. El predictor del padre, en el estado consolidado, se mantiene: calibra el criterio de convergencia en el primer paso, detecta la tangente singular y da la dirección del modo cilíndrico.

5. Predictor / corrector — punto clave del cambio

Switching automático cilíndrico ↔ disipación El solver mantiene un atributo `modo` con dos estados:

- `"cylindrical"`: arc-length cilíndrico estándar; restricción $\|\Delta\mathbf{U}\|^2 = dl^2$. **Régimen elástico**, sin disipación activa.
- `"dissipation"`: arc-length por energía; restricción $g = \tau$. **Régimen disipativo**.

Tangente de carga (formulación adoptada, decisión del usuario del 2026-09-30) En el predictor de un paso intervienen dos tangentes:

- $\mathbf{K}_s$: la que ensambla el predictor del padre en el estado consolidado $\mathbf{U}_n$. Ahí los materiales de daño con descarga secante no están en carga activa ($\varepsilon_{eq} = \kappa$, sin incremento) y devuelven la rigidez secante. Como $\mathbf{F}_{\text{int}}(\mathbf{U}_n) = \mathbf{K}_s\,\mathbf{U}_n = \lambda_n\mathbf{F} + \mathbf{r}_n$, con $\delta\mathbf{u}_t = \mathbf{K}_s^{-1}\mathbf{F}$ el denominador de la restricción es sólo el residuo de equilibrio, $\alpha = -\frac{1}{2}\,\mathbf{F}^T\mathbf{K}_s^{-1}\mathbf{r}_n$ (§ «Detección de $\alpha \approx 0$»).
- $\mathbf{K}_L$, **tangente de carga**: la del **último iterado convergido del paso anterior**, la que tenía el Newton al converger. Está ensamblada en $\mathbf{U}_n$ con la prueba de carga de Kuhn-Tucker respecto del estado consolidado anterior (κ_{n-1}), así que los elementos que dañaban en ese paso aportan su tangente consistente. El solver la toma del corrector compartido: es la matriz del `(K, F_int)` del ensamblaje en el iterado convergido (`CorrectorResult.state`), sin ensamblaje adicional.

Con $\delta\mathbf{u}_L = \mathbf{K}_L^{-1}\mathbf{F}$ y $\mathbf{U}_n = \mathbf{K}_s^{-1}(\lambda_n\mathbf{F} + \mathbf{r}_n)$:

$$\alpha_L = \tfrac{1}{2}\left(\lambda_n \mathbf{F}\cdot\delta\mathbf{u}_L - \mathbf{F}\cdot\mathbf{U}_n\right) = \tfrac{1}{2}\,\lambda_n\,\mathbf{F}^T\left(\mathbf{K}_L^{-1} - \mathbf{K}_s^{-1}\right)\mathbf{F} \;-\; \tfrac{1}{2}\,\mathbf{F}^T\mathbf{K}_s^{-1}\mathbf{r}_n.$$

El primer término mide el crecimiento del daño (se anula si $\mathbf{K}_L = \mathbf{K}_s$); el segundo es el ruido del equilibrio. En la barra con retroceso, en el paso siguiente al inicio del daño, $\alpha_L \approx -0.42$ J frente a una energía elástica $\frac{1}{2}\mathbf{F}\cdot\mathbf{U}_n \approx 1.35$ J; es negativo, como corresponde a un retroceso.

(a) Predictor del modo de disipación: $\Delta\mathbf{U} = \Delta\lambda\,\delta\mathbf{u}_L$ y $\Delta\lambda_{\text{pred}} = \tau/\alpha_L$ (siguiente apartado). Si $\mathbf{K}_L$ no existe aún o es singular, el paso usa $\delta\mathbf{u}_t$ como antes del 2026-09-30, con lo que $\alpha \approx 0$ lo devuelve a cilíndrico.

(b) Arranque del modo de disipación: en modo `"cylindrical"`, antes de cada paso con $\mathbf{K}_L$ disponible,

$$|\alpha_L| > 10^{-3}\cdot\tfrac{1}{2}\,|\lambda_n\,\mathbf{F}\cdot\delta\mathbf{u}_L| \;\Rightarrow\; \text{modo} \leftarrow \texttt{'dissipation'},\quad \tau \leftarrow \texttt{initial_tau},$$

es decir, cuando la tangente de carga se separa de la secante (constante `ALPHA_SWITCH_RTOL` del módulo). Umbral: en régimen sin disipación $\mathbf{K}_L = \mathbf{K}_s$ y $|\alpha_L|/\text{escala} = |\mathbf{F}^T\mathbf{K}_s^{-1}\mathbf{r}_n|/|\lambda_n\,\mathbf{F}^T\mathbf{K}_s^{-1}\mathbf{F}| \le (\|\mathbf{r}_n\|/\|\lambda_n\mathbf{F}\|)/\cos\theta$, con θ el ángulo entre $\mathbf{F}$ y $\mathbf{K}_s^{-1}\mathbf{F}$: del orden de la tolerancia relativa de fuerza del criterio de convergencia (10^{-5} por omisión). Medido en la barra con retroceso: $\approx 10^{-15}$ en la rama elástica y $\approx 10^{-6}$ con la secante tras el inicio del daño. 10^{-3} deja dos órdenes de margen, y con daño en crecimiento la razón es $O(0.1\text{–}1)$ (0.45 en el paso que conmuta en la barra). Si el usuario afloja el criterio de convergencia hasta rtol $\gtrsim 10^{-4}$, el margen desaparece. Como $\mathbf{K}_L$ es la del paso anterior, el cambio de modo llega en el paso **siguiente** al inicio del daño: si el daño empieza en el paso convergido N, los pasos $1..N$ son cilíndricos y los siguientes, de disipación (acceptance `switching_cilindrico_a_disipacion_en_pico`).

Sustituye al criterio anterior, $\Delta E_d > \varepsilon_{\text{diss}}$ tras un paso cilíndrico, que en un pico anguloso no se alcanza: en la barra con daño localizado el arco cilíndrico se atascaba en la esquina con pasos de $\sim 10^{-10}$ m cuya disipación ($\le 1.1\cdot10^{-12}$ J) quedaba bajo el umbral ($\approx 5.3\cdot10^{-12}$ J). Con α_L, un paso con daño en crecimiento, por pequeño que sea, basta: la tangente del elemento débil salta de E a $-\alpha_d\,\kappa_{0w}\,E = -0.475\,E$ (α_d, parámetro de ablandamiento del material). Al entrar, τ vuelve a `initial_tau` (antes, máx(ΔE_d, `initial_tau`) con la ΔE_d del paso que conmutaba).

Alcance del criterio: $\alpha_L \neq 0$ también sin disipación cuando la tangente no es la secante por otra causa: con no linealidad geométrica ($\mathbf{K}_L = \mathbf{K}_t(\mathbf{U}_n)$ y $\mathbf{U}_n \neq \lambda_n\mathbf{K}_t^{-1}\mathbf{F}$) y con plasticidad (tangente elastoplástica frente a descarga elástica). El criterio sólo vale dentro del alcance de §0: **pequeñas deformaciones y descarga secante**. No se ha podido confirmar en el texto de Verhoosel et al. (2009) qué tangente usa su predictor; queda como comprobación bibliográfica pendiente.

Vuelta a cilíndrico (sin cambios el 2026-09-30)

- **En el predictor** de un paso en `"dissipation"`: $|\alpha| < 10^{-6}\cdot$escala (constante `ALPHA_LINEAR_RTOL`) o α nulo a precisión de máquina (guarda relativa, abajo) ⇒ el paso se hace en cilíndrico y el modo pasa a `"cylindrical"` (apartado «Detección de $\alpha \approx 0$»). Con la tangente de carga ocurre cuando $\mathbf{K}_L$ vuelve a ser la secante: el último paso no hizo crecer el daño (descarga) o el daño saturó en `DAMAGE_MAX`, donde el material devuelve la secante. Con el umbral de arranque forma una **histéresis** (10^{-3} para entrar, 10^{-6} para salir) que evita que el modo oscile entre pasos contiguos.
- **Tras cada paso convergido** en `"dissipation"` que no es la llegada a `max_lambda`: $\Delta E_d < \varepsilon_{\text{diss}}$ (disipación nula o negativa) ⇒ el siguiente paso vuelve a `"cylindrical"`, con

$$\varepsilon_{\text{diss}} = \texttt{dissipation_threshold} \cdot \|\mathbf{F}_{\text{ext}}^{\text{ref}}\| \, \|\mathbf{U}_{n+1}\|.$$

Umbral **relativo** a la escala energética del estado, sin piso absoluto (desde el 2026-09-29). Hasta entonces era `dissipation_threshold` $\cdot$ máx($\|\mathbf{F}\| \, \|\mathbf{U}\|$, 1): ese 1 está en unidades de energía del modelo y hacía que el umbral dependiera del sistema de unidades cuando $\|\mathbf{F}\| \, \|\mathbf{U}\| < 1$ (ADR 0006). Un paso de disipación converge con $\Delta E_d = \tau$, así que este criterio sólo actúa si τ cae por debajo de $\varepsilon_{\text{diss}}$; en la práctica la vuelta la decide el predictor.

Predictor en modo de disipación (desde el 2026-09-29) Con el predictor tangente $\Delta\mathbf{U} = \Delta\lambda \, \delta\mathbf{u}$ —desde el 2026-09-30, $\delta\mathbf{u} = \delta\mathbf{u}_L = \mathbf{K}_L^{-1}\mathbf{F}_{\text{ext}}^{\text{ref}}$, la tangente de carga— la restricción lineal da $g = \alpha \, \Delta\lambda$ con

$$\alpha = \tfrac{1}{2}\left(\lambda_n \, \mathbf{F}_{\text{ext}}^{\text{ref}} \cdot \delta\mathbf{u} - \mathbf{F}_{\text{ext}}^{\text{ref}} \cdot \mathbf{U}_n\right),$$

así que exigir $g = \tau > 0$ (disipación positiva) fija el valor **y el signo** del incremento de carga, sin regla de signo (Gutiérrez 2004; Verhoosel et al. 2009):

$$\Delta\lambda_{\text{pred}} = \tau/\alpha.$$

Con $\alpha < 0$ el paso reduce la carga, que es lo que exige un retroceso (*snap-back*). Hasta el 2026-09-29 el código usaba sign $\cdot \, |\tau/\alpha|$ con el signo del producto escalar del padre: cuando sign $\neq$ sgn(α) el predictor arrancaba en $g = -\tau$ (disipación negativa) y el corrector tenía que darle la vuelta. En el prototipo de diagnóstico del 2026-09-29 (tangente de carga en el predictor, arriba) el signo del padre discrepaba del de α en 57 de 141 predictores del retroceso, y con la regla antigua el trazado se perdía.

Detección de $\alpha \approx 0$ en el predictor **En el predictor** de un paso en `"dissipation"`, si $|\alpha| < 10^{-6} \cdot \frac{1}{2}|\lambda_n \, \mathbf{F}_{\text{ext}}^{\text{ref}} \cdot \delta\mathbf{u}|$, o α es nulo a precisión de máquina, el paso se hace en cilíndrico y el modo pasa a `"cylindrical"`. Tras ese paso vuelve a aplicarse el criterio de arranque por α_L, que puede devolver el modo a `"dissipation"`. Dentro del corrector, α nulo con el $\delta\mathbf{u}_t$ de la iteración (la tangente del iterado, en carga) aborta el paso (`CorrectionAborted`), que se biseca.

Guarda de α nulo, relativa (revisión adversaria del 2026-09-30): $|\alpha| \le$ `ZERO_TOL` $\cdot \, \frac{1}{2}\big(|\lambda_n \, \mathbf{F} \cdot \delta\mathbf{u}| + |\mathbf{F} \cdot \mathbf{U}_n|\big)$, es decir, relativa a los dos términos de los que α es diferencia; salta cuando ambos son nulos ($\lambda_n = 0$, $\mathbf{U}_n = \mathbf{0}$). Hasta entonces se comparaba $|\alpha|$ con `ZERO_TOL` $= 10^{-12}$ en unidades de energía: con energías del orden de 10^{-12} (una probeta de micras en SI; medido con la barra de validación en unidades de longitud y fuerza 10^6 veces mayores) el α de un paso de ablandamiento genuino ($\approx 4 \cdot 10^{-13}$) caía por debajo, el paso volvía a cilíndrico en la esquina del pico y el solver abortaba en el paso 24. Con la guarda relativa el trazado coincide con el de SI (1.5e-10); lo fija `test_el_trazado_no_depende_de_las_unidades`. Queda una operación no covariante en el camino, el `+ ZERO_TOL` del $\Delta\lambda$ del predictor cilíndrico, heredado de la fórmula de `ArcLengthSolver`: sólo mueve el punto de partida de Newton (un 1 % con $\|\mathbf{K}^{-1}\mathbf{F}\| \approx 10^{-10}$).

α es nulo en estas situaciones, y la detección no las distingue:

- **Problema lineal monótono**: $\mathbf{U}_n = \lambda_n \mathbf{K}^{-1}\mathbf{F} \Rightarrow \alpha \equiv 0$. Es la matemática del régimen sin disipación.

- **Tangente de carga igual a la secante**: el último paso no hizo crecer el daño (régimen elástico, descarga) o el daño saturó en `DAMAGE_MAX`. Entonces $\alpha_L = -\frac{1}{2}\,\mathbf{F}^T\mathbf{K}_s^{-1}\mathbf{r}_n$, sólo el residuo. Es físicamente correcto: moverse sobre la secante (descarga y recarga elásticas) no disipa, y la restricción no puede fijar $\Delta\lambda$ en esa dirección. En la barra con retroceso, al saturar el daño del elemento débil el solver vuelve así a cilíndrico.

- **Tangente secante en el predictor** (medido 2026-09-29, **resuelto el 2026-09-30** con la tangente de carga). Con $\delta\mathbf{u}_t = \mathbf{K}_s^{-1}\mathbf{F}$ del estado consolidado resultaba

$$\alpha = \tfrac{1}{2}\left(\lambda_n\,\mathbf{F}^T\mathbf{K}_s^{-1}\mathbf{F} - \mathbf{F}^T\mathbf{U}_n\right) = -\tfrac{1}{2}\,\mathbf{F}^T\mathbf{K}_s^{-1}\mathbf{r}_n$$

incluso con el daño creciendo, y la detección lo tomaba por régimen lineal. Medido entonces en los tests del módulo: con `IsotropicDamage1D` (barra de un elemento) y con `IsotropicDamage2D` (Quad4), ningún problema de Newton se construía en modo de disipación (500 de 500 y 300 de 300 en cilíndrico), con $|\alpha|/\text{escala} \le 7\cdot10^{-14}$; el modo `"dissipation"` sólo se veía en el callback, entre la conmutación tras el commit y el predictor siguiente, que lo revertía. En la barra con retroceso, $|\alpha|/\text{escala} \approx 10^{-6}$, el nivel de la tolerancia de equilibrio. Hoy sólo ocurre si $\mathbf{K}_L$ es singular y el paso recurre a $\delta\mathbf{u}_t$.

No hay criterio por número de iteraciones: la spec describía hasta el 2026-09-29 una vuelta a cilíndrico con un `tau_relax_iter_threshold` que el código no tiene.

Puntos físicos resueltos (2026-09-29 y 2026-09-30) Los tres registrados al trasladar el solver al módulo quedaron resueltos el 2026-09-29 con visto bueno del usuario: predictor $\Delta\lambda = \tau/\alpha$ (arriba), umbral relativo sin piso absoluto (arriba) y alcance restringido a daño con descarga secante (§0). Los dos que dejó abiertos el test de retroceso (`test_dctl_retroceso.py`) quedaron resueltos el **2026-09-30 por decisión del usuario**, con la formulación de «Tangente de carga» (arriba):

1. **Tangente del predictor en modo de disipación** → (a), la tangente de carga $\mathbf{K}_L$ del último iterado convergido. Se descartó la alternativa del predictor secante por extrapolación del paso anterior ($\Delta\mathbf{U}^0 = s\,\Delta\mathbf{U}_{n-1}$, $\Delta\lambda^0 = s\,\Delta\lambda_{n-1}$, con $s = \tau/g_n(\Delta\mathbf{U}_{n-1}, \Delta\lambda_{n-1})$): no sirve en el primer paso tras el pico, porque el paso anterior está todavía sobre la rama elástica y da otra vez $\alpha \approx 10^{-6}\cdot$escala.

2. **Arranque del modo de disipación en un pico anguloso** → (b), conmutación por $|\alpha_L| > 10^{-3}\cdot$escala. Ningún umbral de ΔE_d por paso detecta la esquina con fiabilidad, porque los pasos del cilíndrico atascado apenas disipan ($\Delta E_d/\Delta W_{\text{ext}} \approx 4\cdot10^{-7}$ en la barra).

Resultado, igual al del prototipo de diagnóstico: la barra con daño localizado sigue el retroceso con las tolerancias del test de `IndirectDisplacementSolver` (172 puntos post-pico, distancia máxima normalizada $3.2\cdot10^{-4}$, $u_{\text{mín}}/u_{\text{pico}} = 0.616376$, el exacto; sólo el elemento débil dañado), con el primer paso del caso de validación ($\Delta\lambda_1 = 0.02$). Límites que quedan (§10): con un primer paso grande el paso cilíndrico que cruza el pico salta a otra rama; el criterio (b) sólo vale dentro del alcance de §0.

Orientación del predictor (sin seguimiento de inercia) En modo cilíndrico el sentido del predictor sale de la regla del padre: el signo del producto escalar del predictor tangente con el incremento del paso anterior, $\text{sign}(\Delta\mathbf{U}_{\text{prev}}\cdot\delta\mathbf{u}_t)$ (Crisfield 1991, cap. 9). En modo de disipación lo fija la restricción ($\Delta\lambda = \tau/\alpha$, desde el 2026-09-29; con $\alpha = \alpha_L$ de la tangente de carga desde el

2026-09-30). El solver **no sigue la inercia de $\mathbf{K}_t$** (número de pivotes negativos), y ninguna decisión del paso depende de ella.

Un seguimiento de inercia exigiría una factorización $\mathrm{LDL}^\top$ de Bunch-Kaufman que exponga el número de pivotes negativos (ley de inercia de Sylvester): permitiría detectar el paso por un punto límite y orientar el predictor por el cambio de signo de un pivote, no sólo por el producto escalar. Es la deuda técnica #7 de STATUS.md. El caso patológico de bifurcación simétrica, en el que el "menor ángulo" elige una rama paralela (caveat documentado en `ArcLengthSolver` §10), queda por tanto sin resolver aquí.

Historia: hasta el 2026-09-29 el solver tenía un método `_negative_pivots()` que aproximaba la paridad del número de pivotes negativos con el signo del determinante de $\mathbf{K}_t$ (densificando la matriz), y la spec afirmaba que orientaba el predictor. Ningún `solve` lo llamaba; se retiró junto con el placeholder homónimo de `ArcLengthSolver`, sin cambio de resultados.

6. Criterio de convergencia (ADR 0007)

Idéntico al padre — patrón dual fuerza + desplazamiento, escala autoderivada.

7. Imposición de Dirichlet y MPC

Idéntico al padre. La restricción de disipación se evalúa con los vectores completos (todos los DOF): $\mathbf{F}_{\text{ref}}^\top \Delta\mathbf{U}$ y $\mathbf{F}_{\text{ref}}^\top \mathbf{U}_n$, con $\Delta\mathbf{U} = \mathbf{T}\,\Delta\mathbf{u}_{\text{libre}} + \Delta\lambda\,\mathbf{g}$. Con restricciones homogéneas equivale a la forma reducida, porque $\mathbf{F}_{\text{ref}}^\top \mathbf{T}\,\Delta\mathbf{u}_{\text{libre}} = (\mathbf{T}^\top \mathbf{F}_{\text{ref}})^\top \Delta\mathbf{u}_{\text{libre}}$.

8. Backend algebraico

Idéntico al padre (`is_positive_definite=False`, despachador LU). Sin adiciones: el solver no pide al backend ningún dato de inercia.

Red de seguridad (ADR 0019): como redefine `solve`, llama él mismo a las dos piezas genéricas y públicas del programa principal (`solidum.math.solvers.model_checks`), igual que el padre: `ensure_statically_restrained` (mecanismo rígido) y, desde el 2026-09-30, `check_initial_tangent`, que busca mecanismos internos en la primera factorización del corrector (por la energía del modo de menor energía desde el 2026-10-20), la de la tangente en $\mathbf{U} = \mathbf{0}$ del predictor del primer paso. Mecanismo interno con el modelo geométricamente lineal ⇒ `IllPosedSystemError`; con elementos geométricamente no lineales o materiales unilaterales ⇒ aviso; con el backend iterativo, aviso de que no se comprueba. Ver §8 de la spec de `ArcLengthSolver`. Un seguimiento de inercia (§5) requeriría $\mathrm{LDL}^\top$ de Bunch-Kaufman con conteo de pivotes negativos; hoy las factorizaciones exponen `n_negative_pivots` sólo en Cholesky (0 por construcción) y lo dejan en `None` en LU, Pardiso y el iterativo.

9. Adaptatividad — ajuste de τ

Análoga a la adaptatividad de *dl* del padre, con τ en lugar de *dl*:

- Converge en $<$ `tau_grow_iter_threshold` iteraciones ⇒ $\tau \to \tau\cdot$ `tau_grow_factor` (cota: `initial_tau` · `tau_max_factor`).
- Converge en $>$ `tau_shrink_iter_threshold` iteraciones ⇒ $\tau \to \tau\cdot$ `tau_shrink_factor`.

- No converge en `max_iter` (o el corrector aborta, o falla la llegada a `max_lambda`) ⇒ $\tau \to \tau/2$ (τ `*= 0.5`, no `tau_shrink_factor`) y reintentar.
- Cota inferior: $\tau <$ `ARCLENGTH_MIN_DL_FACTOR` · `initial_tau` ⇒ aborto (el mismo factor que acota dl en el padre, `solidum/constants.py`).
- Tangente singular en el predictor ⇒ se dividen a la vez dl (si ya está fijado) y τ entre 2, sea cual sea el modo, y se reintenta el paso; aquí no se comprueba la cota inferior.
- Al pasar de cilíndrico a disipación (en el predictor, por α_L), $\tau \leftarrow$ `initial_tau` (§5). Hasta el 2026-09-29, cuando la conmutación se decidía tras el commit, era máx(ΔE_d, `initial_tau`).

Las iteraciones que se comparan con los umbrales son las resoluciones del corrector en el paso (`n_solves`). La adaptatividad se aplica según el modo del **siguiente** paso, ya decidido: en `"cylindrical"` (régimen elástico inicial o vuelta a cilíndrico) opera sobre dl, idéntica al padre; en `"dissipation"`, sobre τ.

10. Caveats numéricos

Heredados del padre (`ArcLengthSolver` §10) más los específicos:

- **Régimen disipativo no estrictamente monótono**: si el material vuelve transitoriamente a régimen elástico (descarga local), ΔE_d puede ser negativo. El solver tolera este caso pero conmuta a `"cylindrical"` si la disipación neta del paso cae por debajo del umbral, o en el predictor si la tangente de carga vuelve a ser la secante ($\alpha \approx 0$).
- **Primer paso grande: salto de rama** (medido 2026-09-30). El primer paso con daño es siempre cilíndrico: el arranque del modo de disipación necesita un paso convergido con daño en crecimiento (§5). Si ese paso cilíndrico es grande, puede converger a otro equilibrio. En la barra con daño localizado, con $\Delta\lambda_1 = 0.1$ los pasos crecen ($\times 1.5$ hasta $5\,\Delta l_1$) y el que cruza el pico lleva el predictor elástico, de deformación uniforme, por encima del umbral de los diez elementos a la vez (el margen entre κ_{0w} y κ_0 es del 5 %): Newton converge a un equilibrio con los diez elementos ablandándose en serie ($u/u_{\text{pico}} = 1.313$, $F/F_{\text{pico}} = 0.923$), y desde ahí el solver sigue esa rama, ya en modo de disipación, sin retroceso. No es un defecto del método sino la **no unicidad del problema incremental con ablandamiento**: a una misma fuerza cada elemento puede cargar o descargar, y el arco no tiene cómo elegir la rama localizada. Remedio: un primer paso lo bastante pequeño para que el paso que cruza el pico no rebase el umbral de los elementos que deben descargarse (con $\Delta\lambda_1 = 0.02$ la barra sigue el retroceso). El test `test_limite_conocido_con_un_primer_paso_grande` lo deja fijado.
- **Fin de la rama de ablandamiento (saturación del daño)** (medido 2026-09-30). g es la regla del trapecio del trabajo externo menos el cambio de energía elástica, exacta sólo si el camino del paso es lineal en $(\mathbf{U}, \lambda)$. El paso de disipación que cruza la saturación (`DAMAGE_MAX`) tiene por delante menos disipación que τ y cumple $g = \tau$ acreditando trabajo a lo largo de la rama de rigidez residual $(1 -$ `DAMAGE_MAX`$)\,E$: en la barra salta de $\varepsilon_w \approx 10.4\,\kappa_{0w}$ a $\approx 46.5\,\kappa_{0w}$, sobre la curva exacta. Después la tangente de carga es la secante y el solver vuelve a cilíndrico. El punto es un equilibrio, pero el tramo intermedio no se traza. En la franja 2D de 10×2 Quad4 con retroceso (revisión adversaria del 2026-09-30, fuera de la suite) ese paso acredita $7.8 \cdot 10^{-3}$ J más de lo que disipa el material (0.57 % de la disipación total).

- $\sum \Delta E_d$ **como medida de la energía disipada** (medido 2026-09-30). En los pasos de disipación $\Delta E_d = \tau$ coincide con la disipación del material hasta el error del trapecio (1.4e-6 relativo en una franja 2D de 6×2 Quad4 sin retroceso). Los pasos cilíndricos, en cambio, no controlan la disipación: el que cruza el pico, crecido hasta $5\,\Delta l_1$, recorta con su cuerda la esquina del pico, y en esa franja $\sum \Delta E_d$ queda $5.2 \cdot 10^{-3}$ J por debajo (3.7 % de lo disipado en ese paso, 0.18 % al final del trazado). Sumado al paso de saturación (arriba), $\sum \Delta E_d$ de todo el trazado es una estimación, no un balance exacto; la disipación exacta es la del material, $\sum_{gp} V_{gp}\, D(\kappa_{gp})$.

- **Inicialización de** τ: debe ser del orden de la disipación esperada por paso, es decir, una fracción de la disipación total esperada en la rama post-pico (en fractura cohesiva, típicamente una fracción pequeña de $G_F \cdot l_d$ por elemento agrietado). Demasiado pequeño ⇒ pasos numerosos; demasiado grande ⇒ saltos sobre la rama post-pico.

- **Criterio de convergencia laxo**: el umbral de arranque 10^{-3} de §5 supone que el ruido de equilibrio en α_L (del orden de `rtol_force`) queda dos órdenes por debajo. Con `rtol_force` $\gtrsim 10^{-4}$ el margen desaparece y el modo de disipación podría arrancar sin daño en crecimiento.

- **Disipación de origen no físico**: si el modelo numérico introduce disipación espuria (algorithmic damping en transitorio, regularización no consistente), ΔE_d contiene esa componente. **Este solver es estático**: si el usuario invoca `DissipationArcLengthSolver` sobre un modelo con material no disipativo (elástico puro, pequeñas deformaciones), el modo permanece en `"cylindrical"` indefinidamente — comportamiento equivalente al padre. No es un caveat operacional, es la definición del switching. Con no linealidad geométrica, en cambio, $\alpha_L \neq 0$ sin disipación y el modo de disipación arrancaría (fuera del alcance, §0 y §5).

- **Desplazamiento prescrito no nulo o MPC con término independiente g**: la restricción de disipación, evaluada con vectores completos, incluye el trabajo de $\mathbf{F}_{\text{ref}}$ sobre la parte prescrita $\lambda\,\mathbf{g}$ de los DOF esclavos cargados, pero no el de las reacciones ni el de las fuerzas de las MPC. Sin test: validado sólo con restricciones homogéneas (`tests/user/dissipation_control/test_dctl_mpc.py`, barra atada frente a fundida).

B.2.3 Contrato de implementación

```yaml
name: DissipationArcLengthSolver
kind: solver
status: implemented      # draft → implemented → validated. Validación parcial - ver §"
    Implementación".

parent_spec: ArcLengthSolver     # programa principal: docs/specs/ArcLengthSolver.md

interface:
  yaml_type: dissipation_arclength
  output: SolveResult
  pipeline_kind: static

parameters:
  # Heredados del padre (todos con misma semántica):
  - { name: convergence,              type: ConvergenceCriterion, required: false,
      default: "ConvergenceCriterion()" }
```

```
  - { name: max_iter,                 type: int,   required: false, default: 20 }
  - { name: max_lambda,               type: float, required: false, default: 1.0 }
  - { name: max_steps,                type: int,   required: false, default: 100 }
  - { name: initial_dlambda,          type: float, required: false, default: 0.1,
      desc: "Heredado: fracción de la carga de referencia del primer paso (modo cili
          ndrico); Δ₁l = Δλ₁·‖₀⁻K¹‖F_ref. Excluyente con initial_dl" }
  - { name: initial_dl,               type: float, required: false, default: null,
      desc: "Heredado: longitud de arco explícita del primer paso, en unidades de
          desplazamiento. Excluyente con initial_dlambda" }
  - { name: dl_grow_factor,           type: float, required: false, default: 1.5 }
  - { name: dl_max_factor,            type: float, required: false, default: 5.0 }
  - { name: dl_shrink_factor,         type: float, required: false, default: 0.6 }
  - { name: dl_grow_iter_threshold,   type: int,   required: false, default: 4 }
  - { name: dl_shrink_iter_threshold, type: int,   required: false, default: 8 }
  - { name: linear_algebra,           type: str,   required: false, default: "auto",
      desc: "Heredado de ArcLengthSolver: 'auto' (Cholesky → Pardiso → LU según simetrí
          a y dependencias instaladas), 'cholesky', 'pardiso', 'lu', 'iterative[:amg|
          jacobi|none]' (ADR 0003, 0017, 0018)" }

  # Específicos de esta variante:
  - { name: initial_tau,              type: float, required: true,
      desc: "Incremento de disipación objetivo (unidades de energía: F·U). Calibrar
          como fracción de la disipación total esperada en la rama post-pico, del orden
           de E_d,total/n_pasos_post_pico (en fractura cohesiva, E_d,total ≈ G_F·l_d
          por elemento agrietado)" }
  - { name: tau_grow_factor,          type: float, required: false, default: 1.5 }
  - { name: tau_max_factor,           type: float, required: false, default: 5.0 }
  - { name: tau_shrink_factor,        type: float, required: false, default: 0.6 }
  - { name: tau_grow_iter_threshold,  type: int,   required: false, default: 4 }
  - { name: tau_shrink_iter_threshold,type: int,   required: false, default: 8 }
  - { name: dissipation_threshold,    type: float, required: false, default: 1.0e-12,
      desc: "Umbral relativo: tras un paso en modo de disipación, ΔE_d <
          dissipation_threshold·|||F_ref·|||U se considera 0 y devuelve el modo cilíndrico
          . Sin piso absoluto desde 2026-09-29. Desde 2026-09-30 no decide el arranque
          del modo de disipación (criterio por α_L con la tangente de carga, §5)" }

requirements:
  - "Modelo con material de daño con descarga secante (daño continuo o cohesivo) - en
      modelo puramente elástico el solver opera equivalente a ArcLengthSolver cili
      ndrico"
  - "F_ext^ref no nulo (la restricción de disipación se evalúa sobre F_ext^ref·U)"
  - "Heredados del padre: apoyos suficientes; sin modos rígidos; dl_inicial razonable"
  - "Módulo de usuario cargado: user_modules: [dissipation_control] en el YAML o
      solidum.load_user_module('dissipation_control')"

conventions:
  units: "heredadas del modelo (ADR 0008); τ en unidades de energía consistentes"
  stability: "incondicional respecto a puntos límite y rama post-pico de softening
      severo. Ante bifurcaciones, la misma que el padre: el sentido del predictor sale
      del producto escalar con el incremento previo (sin seguimiento de inercia, §5)"

out_of_scope:
  - "Plasticidad - la ΔE_d de §1 supone descarga secante; la variante para plasticidad
      de Verhoosel et al. (2009) queda como ampliación futura"
  - "Bifurcaciones - el solver no sigue la inercia de K_t (sin sign-of-pivot tracking,
      deuda #7); una bifurcación, simple o múltiple, requiere perturbación explícita"
```

```
  - "Análisis dinámico - la formulación de disipación es válida solo en régimen cuasi-
      estático conservativo"
  - "Modo II y mixto I-II en cohesivo - heredado del scope ADR 0010 fases F+"
  - "3D - coherente con el scope global del proyecto en 2026-05"

acceptance:
  verification:
    - name: recuperacion_del_arclength_cilindrico_en_elastico
      setup: "Modelo elástico puro (sin disipación); ejecutar
          DissipationArcLengthSolver y ArcLengthSolver con los mismos parámetros y
          misma malla"
      expect: "El modo permanece 'cylindrical'; trayectoria Uλ() coincide a paridad de
          bits con el padre"
      tol_rel: 1.0e-12

    - name: snap_through_lee_frame
      setup: "Mismo benchmark que ArcLengthSolver §acceptance - frame de Lee, dos
          barras corotacionales"
      expect: "Atraviesa el punto límite; Uλ- coincide con Crisfield 1991 Fig. 9.5"
      tol_rel: 1.0e-3

    - name: damage_softening_post_pico
      setup: "Quad4 con IsotropicDamage2D, tracción que entra en rama descendente"
      expect: "Modo conmuta a 'dissipation' al detectar Δω>0; rama post-pico trazada;
          coincide con ArcLengthSolver del padre dentro de tolerancia (ambos resuelven
          el mismo problema)"
      tol_rel: 1.0e-3

    - name: retroceso_barra_dano_localizado              # validado 2026-09-30 (tangente
        de carga, §5); xfail(strict) del 2026-09-29 al 2026-09-30
      setup: "Barra de 10 Truss2D con IsotropicDamage1D, elemento central con K₀ un 5 %
           menor (caso de IndirectDisplacementSolver); initial_dlambda 0.02,
          initial_tau 1e-3"
      expect: "Puntos post-pico sobre la curva exacta (distancia normalizada < 1e-3, má
          s de 100 puntos) y desplazamiento del extremo que retrocede hasta el mínimo
          exacto 0.616·u_pico (±2e-3); sólo el elemento débil dañado; la rama de
          ablandamiento, resuelta en modo de disipación. Medido: 172 puntos, distancia
          máxima 3.2e-4, u_min/u_pico = 0.616376. Con initial_dlambda 0.1 el paso cilí
          ndrico que cruza el pico salta a la rama con los diez elementos ablandándose
          (límite conocido, §10)"
      tol_rel: 1.0e-3

    # Los tres casos siguientes usan componentes de OTRO módulo de usuario,
    # `discontinuities` (CST_Embedded2D + CohesiveDamageIsotropic): el modelo
    # declara user_modules: [discontinuities, dissipation_control] y su test
    # irá en tests/user/dissipation_control/, cargando los dos módulos.
    - name: cohesive_embedded_softening_postpeak       # benchmark CRÍTICO - desbloquea
        fase 4 ADR 0010
      setup: "Requiere el módulo de usuario discontinuities. Probeta rectangular con
          CST_Embedded2D + CohesiveDamageIsotropic; tracción uniaxial monotónica que
          activa Rankine y entra en rama post-pico con penalty K_e stiff (1e13)"
      expect: "El solver atraviesa la transición elá→sticosoftening (régimen donde el
          ArcLengthSolver cilíndrico aborta); cierra el ciclo de softening; daño
          cohesivo committed alcanza saturación"
      tol_rel: 1.0e-3
```

```
    - name: balance_energetico_GF
      setup: "Requiere el módulo de usuario discontinuities. Mismo modelo cohesivo+
          embedded del test anterior; integrar ΣΔE_d sobre toda la historia hasta
          saturación ω→(1) y comparar con G_F · longitud_grieta_abierta del material"
      expect: "Σ ΔE_d ≈ G_F · l_d a tolerancia de discretización temporal (1% típico)"
      tol_rel: 1.0e-2

    - name: convergencia_temporal_tau
      setup: "Requiere el módulo de usuario discontinuities. Reducir initial_tau en
          factor 2 sucesivamente sobre el benchmark cohesivo, observar Uλ(_final)"
      expect: "Convergencia primer orden en τ (esquema lineal en restricción): error →
          0 como Oτ()"
      tol_rel: 0.2

  specific:
    - name: switching_cilindrico_a_disipacion_en_pico     # cubierto 2026-09-30 con el
        modo de los problemas de Newton
      setup: "Modelo con daño que empieza en el paso convergido N"
      expect: "Problemas de Newton 'cylindrical' en pasos 1..N (el N, cilíndrico por
          construcción: el arranque necesita la tangente de carga de un paso con daño
          en crecimiento); 'dissipation' desde N+1 en adelante mientras el daño crece;
          sin oscilación de modo en pasos contiguos"

    - name: sign_of_pivot_tracking_bifurcacion_simple     # no implementado: requiere
        ᵀLDL (deuda #7), §5
      setup: "Modelo con un punto límite simple (snap-back); registrar el número de
          pivots negativos antes y después del paso del límite"
      expect: "Conteo de pivots negativos cambia por 1 al atravesar el límite;
          predictor mantiene la dirección física"

    - name: fallback_a_cilindrico_en_descarga_global
      setup: "Modelo con grieta saturada ω≈(1) y descarga: λ decrece, F_ext invertida"
      expect: "Modo conmuta de vuelta a 'cylindrical' al detectar ΔE_d < umbral"

references:
  - "Gutiérrez M.A. (2004). Energy release control for numerical simulations of failure
      in quasi-brittle solids. *Communications in Numerical Methods in Engineering*
      20(1), 19-29. (Restricción lineal de disipación; fórmula central de esta spec.)"
  - "Verhoosel C.V., Remmers J.J.C., Gutiérrez M.A. (2009). A dissipation-based arc-
      length method for robust simulation of brittle and ductile failure. *
      International Journal for Numerical Methods in Engineering* 77(9), 1290-1321. (
      Versión robusta con switching y aplicación a cohesivos.)"
  - "Crisfield M.A. (1991). *Non-linear Finite Element Analysis of Solids and
      Structures*, Vol. 1, Wiley. §9.4 (alternative constraints to cylindrical/
      spherical)."
  - "de Borst R., Sluys L.J. (1999). *Computational Methods in Non-linear Solid
      Mechanics*, TU Delft. §3.5 (controlled descent)."
  - "ADR 0010 §Hoja de ruta (fase 4 diferida por solver)."
  - "ADR 0020 (módulos de usuario) y criterio de admisión de Reglas.md §1."
  - "Spec padre: docs/specs/ArcLengthSolver.md (programa principal)."
```

B.2.4 Implementación

- **Archivo**: solidum/user/dissipation_control/dissipation_arclength.py (hasta el 2026-09-29, `solidum/math/solvers/dissipation_arclength.py`).

- **Clase**: `DissipationArcLengthSolver(ArcLengthSolver)` — subclase. Reusa predictor, corrector, ajuste de paso, manejo de Dirichlet/MPC y backend del padre. Override completo de `solve()` con bifurcación por `self._mode`. Su problema por paso, `_DissipationArcProblem`, hereda de `solidum.math.solvers.ArcProblem` (el problema de Newton del arco que el programa principal exporta como punto de extensión) y sólo redefine `constraint`.

- **Atributo** `self._mode`: `"cylindrical" | "dissipation"`. Arranque del modo de disipación en el predictor, por `|α_L| > ALPHA_SWITCH_RTOL·½·|λ_n·F·du_L|` con `ALPHA_SWITCH_RTOL = 1e-3` (§5, desde el 2026-09-30; antes, tras el commit por Δ`E_d > dissipation_threshold`·‖`F_ref`‖·‖`U`‖). Vuelta a cilíndrico en el predictor por $\alpha\approx 0$ o tras el commit por Δ`E_d` bajo el umbral relativo (§5; sin el piso `max(·, 1)` desde el 2026-09-29).

- **Tangente de carga** (desde el 2026-09-30): atributo `self._K_load`, la matriz del `(K, F_int)` que devuelve el corrector compartido en `CorrectorResult.state` tras cada paso con restricción que converge; `_load_tangent_direction` resuelve `du_L = K_L`$^{-1}$`·F_ref` con la misma reducción de Dirichlet/MPC (`assembler.reduce/expand`) y el mismo backend (`corrector.solve`) que el predictor del padre, y devuelve `None` si aún no hay `K_L` o es singular. Todo por la API pública del programa principal; el programa principal no gana piezas nuevas.

- **Predictor**: en modo cilíndrico, el del padre, orientado por `sign(`Δ`U_prev · du_t)`; en modo de disipación, Δ`U = `$\Delta\lambda$`·du_L` y $\Delta\lambda$ `= `τ/α`_L`, con valor y signo fijados por la restricción (§5, desde el 2026-09-29 τ/α, antes `sign·|`τ/α`|`; con la tangente de carga desde el 2026-09-30). Sin seguimiento de inercia (§5); el método `_negative_pivots()` (signo del `slogdet` de `K_t`, que ningún `solve` llamaba) se retiró el 2026-09-29.

- **Llegada a** `max_lambda` (desde 2026-09-23, la del padre): todos los pasos llevan su restricción (cilíndrica o de disipación); el que converge por encima de `max_lambda` no se consolida y se repite con $\lambda =$ `max_lambda` fijo desde el estado anterior (`ArcLengthSolver._land`). Si la llegada falla se biseca dl o τ según el modo del paso. Sustituye a la salvaguarda anterior contra el `final_step` prematuro, que bisecaba cuando el predictor rebasaba `max_lambda` más de 3 veces lo que faltaba y, por debajo de ese factor, imponía `max_lambda` en control de carga.

- **Detección de $\alpha\approx 0$ en el predictor**: `|`α`| < ALPHA_LINEAR_RTOL·½·|`λ`_n·F·du|` (`ALPHA_LINEAR_RTOL = 1e-6`, umbral relativo) o α nulo a precisión de máquina (`_alpha_is_null`: `|`α`|` $\leq$ `ZERO_TOL·½·(|`λ`_n·F·du| + |F·U_n|)`, relativa desde el 2026-09-30; antes `|`α`| < ZERO_TOL` en unidades de energía, §5) revierte a cilíndrico. En problemas lineales monótonos $\alpha\equiv 0$ exactamente, y con la tangente de carga también cuando ésta vuelve a ser la secante (descarga, daño saturado). Hasta el 2026-09-29 ocurría además con el daño creciendo, porque el predictor usaba la tangente del estado consolidado, que es la secante (§5), y la restricción de disipación no llegaba a usarse. En el corrector, α nulo con la misma guarda aborta el paso (`CorrectionAborted`) y se biseca τ.

- **Bisección**: según el modo del paso fallido, `dl *= 0.5` o τ `*= 0.5`, con aborto (`RuntimeError`) por debajo de `ARCLENGTH_MIN_DL_FACTOR` veces `dl_ref` o `initial_tau`. Si el predictor encuentra la tangente singular, divide a la vez `dl` y τ y reintenta.

- **Entrypoint público**: registrado vía `@SolverRegistry.register` al cargar el módulo; en YAML, `user_modules: [dissipation_control]` y `solver: {type: DissipationArcLengthSolver`

`...}`; en Python, `solidum.load_user_module("dissipation_control")` o `from solidum.user.dissipation_control import DissipationArcLengthSolver`. Sin el módulo cargado, el YAML falla con un error que sugiere declararlo y lista los módulos disponibles.

Tests (suite tests/user/dissipation_control/)

test_dctl_dissipation_arclength.py (hasta el 2026-09-29, `tests/test_dissipation_arclength.py`) cubre:

- ✓ Recuperación cilíndrica en régimen elástico puro (regresión a paridad de bits con `ArcLengthSolver`; todos los problemas de Newton cilíndricos).
- ✓ Daño 1D con softening pronunciado (rama post-pico atravesada, con pasos post-pico en modo de disipación).
- ✓ Switching automático cilíndrico→disipación (acceptance `switching_cilindrico_a_disipacion_en_pico`): problemas de Newton cilíndricos hasta el paso en que empieza el daño, de disipación desde el siguiente, sin oscilación.
- ✓ Daño 2D continuo (Quad4 + IsotropicDamage2D plane strain), con pasos post-pico en modo de disipación.
- ✓ Balance energético sobre una barra de daño continuo 1D, sin grieta ni cohesivo: $\Sigma\Delta E_d > 0$, cada paso de disipación con $\Delta E_d > 0$ y, desde el 2026-09-30, la $\Sigma\Delta E_d$ de los pasos en modo de disipación igual a la disipación analítica del material entre sus estados inicial y final, $E_d(\kappa) = W(\kappa) - \Psi(\kappa)$ (tolerancia derivada 1e-4; medido 1.3e-10).
- ✓ Daño 2D localizado con retroceso (revisión adversaria del 2026-09-30): franja de 4×2 Quad4 con `IsotropicDamage2D` en tensión plana ($\nu = 0.2$, medida energética por omisión) y la columna central un 5 % más débil. El daño se localiza en la columna, el extremo retrocede, los problemas de Newton son cilíndricos hasta el paso de inicio del daño y de disipación desde el siguiente (67 pasos), y la $\Sigma\Delta E_d$ de esos pasos coincide con la disipación del material en los puntos de Gauss, $\sum V_{gp}\,[D(\kappa_{\text{fin}}) - D(\kappa_{\text{ini}})]$ con $D(\kappa) = \int \frac{1}{2} E \kappa^2 d'(\kappa)\, d\kappa$, dentro de una cota sin márgenes ajustados: residuo de equilibrio en los extremos más el error del trapecio estimado por duplicación de paso (medido 5.9e-4 J sobre 3.23 J, frente a una estimación del trapecio de 5.3e-4 J). Con el solver anterior al 2026-09-30 falla. Fuera de la suite, la misma franja con 10 columnas sigue la curva de `IndirectDisplacementSolver` (distancia máxima 7.4e-6, mismo $u_{\text{mín}}/u_{\text{pico}} = 0.61494$).

Qué restricción resolvió cada paso lo comprueba un espía del corrector compartido (_dctl_espia.py), que anota el modo de cada problema de Newton. *Nota 2026-09-29*: hasta entonces los tests 2-5 sólo observaban el modo `"dissipation"` en el callback, sin que ninguno ejercitara la restricción de disipación: el predictor siguiente revertía el modo a cilíndrico por $\alpha \approx 0$ (§5), y todos los problemas de Newton de esos trazados eran cilíndricos. Desde el 2026-09-30 esos trazados resuelven la rama de ablandamiento en modo de disipación (495 de 500 pasos en la barra 1D; 291 de 300 en el Quad4) y los tests pasan sin cambiar sus valores esperados.

test_dctl_retroceso.py cubre `retroceso_barra_dano_localizado`: la barra de 10 elementos con daño localizado del test de `IndirectDisplacementSolver`, con sus tolerancias: puntos post-pico

sobre la curva exacta, retroceso hasta el mínimo exacto y sólo el elemento débil dañado. Del 2026-09-29 al 2026-09-30 los dos primeros iban con `xfail(strict=True)` (el arco cilíndrico se atascaba en el pico sin conmutar y abortaba, con $u_{\text{mín}}/u_{\text{pico}} = 1.00001$); desde el 2026-09-30 pasan sin marca (172 puntos, distancia máxima $3.2 \cdot 10^{-4}$, $u_{\text{mín}}/u_{\text{pico}} = 0.616376$). Dos tests más: la rama de ablandamiento del elemento débil sale entera de problemas de Newton en modo de disipación (cilíndricos antes y en el paso de inicio del daño, y de nuevo tras la saturación) y el límite conocido con `initial_dlambda = 0.1` (§10): el paso cilíndrico que cruza el pico salta a la rama con los diez elementos ablandándose, que el solver sigue después en modo de disipación sobre su curva analítica. De la revisión adversaria del 2026-09-30: todos los puntos con daño, también los de la rama de rigidez residual tras la saturación (hasta $\varepsilon_w \approx 230\,\kappa_{0w}$, que el filtro $\varepsilon_w \leq 60\,\kappa_{0w}$ del test de la curva deja fuera), sobre la solución exacta evaluada en su propia ε_w (medido 7.7e-6); y el trazado no depende de las unidades: con el modelo en mm (energías $\times$1e3) y en unidades 10^6 veces mayores (energías $\times$1e-12) la secuencia de intentos es idéntica y los puntos coinciden con los de SI salvo redondeo.

`test_dctl_red_seguridad.py` (2026-09-30) cubre la red de seguridad sobre la armadura del ejemplo 11 del manual de ejemplos sin diagonal y con carga horizontal: `IllPosedSystemError` con `Truss2D`, un aviso con `Truss2DCorot`, nada con la diagonal (trazado hasta `max_lambda`) y el aviso del backend iterativo.

Los dos tests del `_negative_pivots` se retiraron con el método. `test_dctl_contratos.py` cubre el contrato de solver del barrido del programa principal, el primer paso heredado, `initial_tau`, el gancho de inicio de paso (ADR 0020, P5) y la carga desde YAML con y sin `user_modules`; `test_dctl_specs.py`, esta spec frente a la clase registrada.

Validación parcial — qué falta para status `validated`

Cubiertos desde el 2026-09-30: `retroceso_barra_dano_localizado` y `switching_cilindrico_a_disipacion_en_pico`. No están cubiertos: `snap_through_lee_frame` (con no linealidad geométrica el criterio de arranque por α_L queda fuera de su alcance, §5), los tres casos con componentes del módulo de usuario `discontinuities` (`cohesive_embedded_softening_postpeak`, `balance_energetico_GF`, `convergencia_temporal_tau`), `sign_of_pivot_tracking_bifurcacion_simple` (capacidad no implementada, §5) y `fallback_a_cilindrico_en_descarga_global`.

El acceptance `cohesive_embedded_softening_postpeak` **no está cubierto** en esta primera implementación. Diagnóstico:

La activación discreta del cohesivo en `prepare_step` (Rankine onset al cruzar σ`_t0`) introduce un salto en `F_int` y `K_t` entre el paso pre-activación y el paso post-activación. El predictor por `sign = sign(dot(`δ`U_prev, du_t))` puede heredar una orientación inconsistente cuando el Newton del paso de activación reorienta `dU_iter` para acomodar el nuevo `K_t` con el cohesivo activo. El paso siguiente entonces predice hacia atrás (descarga) en lugar de continuar la rama post-pico.

Resolverlo limpiamente requiere **una de las dos vías siguientes**, ambas más allá del scope incremental de esta spec:

1. **Sign-of-pivot tracking con LDL$^\top$ Bunch-Kaufman**. Permitiría detectar la inercia exacta de `K_t` y orientar el predictor por cambio de signo de pivot en lugar del producto escalar δ`U·du_t`. Esto desacopla la orientación del predictor de la cinemática perturbada por la activación. Requiere extender el backend algebraico (ADR 0003 fase 2) con un solver LDL$^\top$ verdadero.

2. **Control por CMOD/CTOD del salto** mediante MPC sobre el DOF interno del cohesivo, en lugar de la restricción de disipación global. Cambia la naturaleza del solver — sería

una spec separada (`CMODControlSolver`) en lugar de variante de arc-length. *Nota 2026-09-29*: desde el 2026-09-23 el programa principal tiene el control indirecto de desplazamiento, `IndirectDisplacementSolver` (de Borst 1987), que controla una combinación de desplazamientos nodales (p. ej. una apertura medida entre nodos), no el DOF interno del salto.

Nota 2026-09-29: este diagnóstico (2026-05-19) es anterior a la corrección de la tangente cohesiva del 2026-09-23 (`597f3d9`) y no se ha repetido; con la tangente corregida el comportamiento de este solver en el caso cohesivo+embebido no está comprobado.

Recomendación: dejar esta validación como deuda técnica priorizada en STATUS.md §"Deuda técnica priorizada" hasta que el LDL$^\top$ Bunch-Kaufman entre en otro contexto (lo amerita por más razones, no solo esta).

B.2.5 Diálogo

- **2026-05-19 (tarde)** · Spec promovida de `draft` a `implemented`. 7/7 tests verde sobre los casos de daño continuo (1D y 2D); el caso crítico cohesivo+embedded reveló una limitación más profunda del `sign` por producto escalar tras activación discreta del Rankine, que requiere LDL$^\top$ Bunch-Kaufman real o control CMOD/CTOD — ambos fuera del scope incremental de esta variante. Documentado en §"Implementación" como deuda técnica. La spec mantiene su valor: provee la arquitectura del switching cilíndrico↔disipación y la fórmula lineal de Gutiérrez funcional para todos los problemas con softening continuo (daño bulk 1D y 2D), que es donde más casos prácticos de Solidum se concentran.

- **2026-05-19 (mañana)** · Spec creada como variante de `ArcLengthSolver` para desbloquear ADR 0010 fase 4 (embedded discontinuity con softening severo) y validación cuantitativa de G_F en pipeline real. Motivada por el cierre de la campaña de validación externa Tandas 12-14 ([[project_validacion_externa_tandas_12_14]]) que identificó esta pieza como **el primer componente nuevo priorizado** tras la campaña. Sin ADR nuevo: reusa decisiones arquitecturales del padre (estructura del bucle predictor/corrector, manejo de Dirichlet/MPC, ADR 0003 algebraico, ADR 0007 convergencia, ADR 0008 unidades). El cambio se circunscribe a la **restricción del paso** (cuadrática → lineal) más mejoras secundarias (switching automático cilíndrico↔disipación, sign-of-pivot tracking).

- **Puntos abiertos para el usuario** (físicos/de alcance, no de plumbing — ver `[[feedback_filtrar_puntos_abiertos]]`):

1. ¿La restricción de disipación de Gutiérrez 2004 es la fórmula que quieres, o prefieres una alternativa (p. ej. constraint sobre `K_t`-norma de Crisfield 1991 §9.4)? La de Gutiérrez es la más extendida en literatura cohesiva.

2. ¿El switching automático cilíndrico↔disipación es deseable, o prefieres dos solvers separados (`CylindricalArcLengthSolver` y `DissipationArcLengthSolver` puros) que el usuario elige según el caso? El switching es estándar en Verhoosel et al. 2009; dos solvers separados son más predecibles pero exigen que el usuario diagnostique cuándo cambiar.

3. ¿Sign-of-pivot tracking se aborda en esta spec o se difiere a una spec independiente (`PivotTracker`)? Está acoplado al backend algebraico (ADR 0003) — si el LDL$^\top$ Bunch-Kaufman se introduce como helper compartido, otros solvers se beneficiarían. Mi propuesta es

abordarlo aquí en versión "signo del determinante LU" (aproximación válida para indefinitud simple) y dejar el $LDL^\top$ verdadero como deuda técnica del proyecto. *(2026-09-29: la versión por signo del determinante se implementó pero nunca se conectó al predictor, y se retiró; ver §5.)*

- **2026-09-22** · Auditoría global: hereda la reestructuración del corrector y los metadatos de trazado de `ArcLengthSolver` (ensamblaje único por iteración, estado committed coherente, `reached_max_lambda`).
- **2026-09-22** · ADR 0015: el bucle de Newton pasa al corrector compartido `NewtonCorrector`; este solver aporta su problema por paso (`_DissipationArcProblem`: sobreescribe `constraint` con la restricción lineal de Gutiérrez para el modo de disipación y hereda los modos cilíndrico y de cierre exacto; $\alpha \approx 0 \rightarrow$ `CorrectionAborted`) y conserva su control de paso. Sin cambio de formulación ni de resultados (suite completa sin tocar ningún valor esperado).
- **2026-09-23** · ADR 0017-0019: hereda de `ArcLengthSolver` los valores nuevos de `linear_algebra` y la comprobación de mecanismo rígido al empezar `solve`.
- **2026-09-30** · Red de seguridad sobre la tangente inicial (ADR 0019; decisión física del usuario para los solvers estáticos no lineales): `solve` llama a la pieza genérica `check_initial_tangent` tras `ensure_statically_restrained`, como el padre. Test nuevo `test_dctl_red_seguridad.py`. Sin cambio de formulación ni de resultados en modelos bien planteados.
- **2026-09-23** · Hereda el cambio de formulación del padre validado por el usuario: primer paso como fracción de carga (`initial_dlambda`, 0.1 por omisión; `initial_dl` explícito y excluyente) y llegada exacta a `max_lambda` dentro del tramo recorrido, que sustituye a la salvaguarda del `final_step`. Ver el Diálogo de `ArcLengthSolver`. Los tests de esta variante pasan sin cambiar ningún valor esperado (todos declaran `initial_dl` explícito).
- **2026-09-29** · **Sale del programa principal al módulo de usuario** `dissipation_control` (ADR 0020), por el criterio de admisión de Reglas.md §1: ningún código general establecido implementa la restricción de Gutiérrez de forma nativa (ver la clasificación al principio). Spec, código y tests pasan a `docs/user/dissipation_control/specs/`, `solidum/user/dissipation_control/` y `tests/user/dissipation_control/`; el YAML necesita `user_modules: [dissipation_control]`. El programa principal gana una sola pieza, genérica: el nombre público `solidum.math.solvers.ArcProblem` para el problema de Newton por paso del arco, que ya extendía `IndirectDisplacementSolver`. En la misma revisión se retira `_negative_pivots()` —que ningún `solve` llamaba, en este solver ni en el padre— con sus dos tests, y la spec pasa a describir lo que hace el código: el predictor se orienta por el producto escalar con el incremento previo, sin seguimiento de inercia (§5, §8); el callback no expone la disipación acumulada (§3); el retorno a cilíndrico usa el umbral de ΔE_d y la detección de $\alpha \approx 0$, no un `tau_relax_iter_threshold` (§5). Los tres acceptance con `CST_Embedded2D` declaran que requieren el módulo `discontinuities`. Sin cambio de formulación ni de resultados.
- **2026-09-29** · Segunda revisión documental del día, contra `solidum/user/dissipation_control/dissipation_arclength.py` (sin cambiar el código ni la formulación). (a) §3 ya no promete que el callback exponga $\sum \Delta E_d$: dice que no lo hace y cómo se reconstruye. (b) §5 describe el criterio real de vuelta a cilíndrico ($\Delta E_d < \varepsilon_{\text{diss}}$ tras el commit y detección de $\alpha \approx 0$

en el predictor) y retira el de `tau_relax_iter_threshold`, que no existe; recoge también que al entrar en disipación $\tau \leftarrow$ máx(ΔE_d, `initial_tau`), no ΔE_d, y §0 que la detección es por energía, no por variables internas del material. (c) El pseudocódigo de §4 pasa a describir el `solve`: al fallar un paso τ se divide entre 2 (τ `*= 0.5`), no entre `shrink_factor`; con tangente singular en el predictor se dividen a la vez dl y τ; se añaden la detección de $\alpha \approx 0$, el aborto del corrector con $\alpha \approx 0$ y la llegada exacta a `max_lambda`, que no decide modo ni adapta el paso. §9 e Implementación, en consonancia. Quedan registrados en §5, **sin resolver y pendientes del usuario**, los tres puntos físicos abiertos: el signo del predictor en modo de disipación (sign $\cdot\, |\tau/\alpha|$ frente a τ/α), el piso absoluto 1 de la escala del umbral y el alcance en plasticidad. Notas nuevas en la Motivación y en la Validación parcial: los diagnósticos con penalización cohesiva rígida son anteriores a la corrección de la tangente cohesiva (`597f3d9`) y no se han repetido.

- **2026-09-29 · Cambio de formulación con visto bueno del usuario** (tercera revisión del día). (1) Predictor del modo de disipación $\Delta\lambda = \tau/\alpha$: la restricción lineal fija valor y signo; sustituye a sign $\cdot\, |\tau/\alpha|$, que con sign $\neq$ sgn(α) arrancaba en $g = -\tau$. (2) Umbral de conmutación relativo, `dissipation_threshold` $\cdot\, \|\mathbf{F}\|\, \|\mathbf{U}\|$, sin el piso máx($\cdot$, 1) en unidades de energía. (3) Alcance restringido a daño con descarga secante (§0, §1 con la derivación de ΔE_d, `requirements`, `out_of_scope`); la variante para plasticidad de Verhoosel et al. (2009) queda como ampliación futura. Los tres puntos abiertos de la revisión anterior quedan así resueltos. Los tests existentes del módulo pasan sin cambios (19 y 6 subtests).

La hipótesis de que (1) resolvería la deuda #27 **no se cumple**. Nuevo test `test_dctl_retroceso.py` (acceptance `retroceso_barra_dano_localizado`, barra de 10 elementos con daño localizado, tolerancias del test de `IndirectDisplacementSolver`) en `xfail(strict=True)`. Diagnóstico con registro por paso (§5): el predictor τ/α no llega a ejecutarse, porque con la tangente secante del estado consolidado $\alpha = -\frac{1}{2}\mathbf{F}^T\mathbf{K}_s^{-1}\mathbf{r}_n \approx 0$ y el paso vuelve a cilíndrico; en los tests existentes, con daño 1D y 2D, ningún problema de Newton se construye en modo de disipación. Además, en el pico anguloso de la barra el arco cilíndrico se atasca sin dar un paso con ΔE_d sobre el umbral: el solver no conmuta y aborta ($u_{\text{mín}}/u_{\text{pico}} = 1.00001$). El cambio (2) es neutro en este caso ($\|\mathbf{F}\|\, \|\mathbf{U}\| \approx 5.3 > 1$: el piso no actuaba). Un prototipo de diagnóstico, fuera del repositorio, con la tangente del último iterado convergido en el predictor y la conmutación por α_L, sigue el retroceso con las tolerancias del test; con esa tangente, la regla de signo antigua pierde el trazado ($u_{\text{mín}}/u_{\text{pico}} = -1.0$) y la nueva no: (1) es necesario pero no suficiente. Quedan dos decisiones de formulación para el usuario (§5, puntos abiertos 1 y 2).

- **2026-09-30 · Cambio de formulación por decisión del usuario**: resueltos los puntos abiertos 1 y 2 de §5. (a) **Tangente de carga en el predictor** del modo de disipación: $\delta\mathbf{u}_L = \mathbf{K}_L^{-1}\mathbf{F}$ con $\mathbf{K}_L$ la tangente del último iterado convergido del paso anterior (prueba de carga respecto de κ_{n-1}), en lugar de la del estado consolidado, que con daño y descarga secante es la secante y da $\alpha = -\frac{1}{2}\mathbf{F}^T\mathbf{K}_s^{-1}\mathbf{r}_n \approx 0$; entonces $\alpha_L = \frac{1}{2}\lambda_n\, \mathbf{F}^T(\mathbf{K}_L^{-1} - \mathbf{K}_s^{-1})\mathbf{F}$ (más el ruido del residuo) mide el crecimiento del daño y en el retroceso es negativo, así que $\Delta\lambda = \tau/\alpha_L < 0$. (b) **Arranque del modo de disipación por** α_L: conmuta cuando $|\alpha_L| > 10^{-3} \cdot \frac{1}{2}|\lambda_n\, \mathbf{F} \cdot \delta\mathbf{u}_L|$, en lugar de exigir $\Delta E_d > \varepsilon_{\text{diss}}$ tras un paso cilíndrico; al entrar, $\tau \leftarrow$ `initial_tau`. La vuelta a cilíndrico no cambia ($\alpha \approx 0$ en el predictor, $\Delta E_d < \varepsilon_{\text{diss}}$ tras el commit); `dissipation_threshold` sólo interviene en ella. No se añadió la condición $\Delta E_d \geq 0$ al arranque: el prototipo no la usaba, y $\alpha_L \neq 0$ exige que el paso anterior acabara con daño en crecimiento, que disipa.

Implementación sólo en el módulo, con API pública del programa principal (`CorrectorResult.state`, `corrector.solve`, `assembler.reduce/expand`); el programa principal no cambia. Resultados, iguales a los del prototipo: la barra con daño localizado sigue el retroceso exacto (172 puntos post-pico, distancia máxima $3.2 \cdot 10^{-4}$, $u_{\text{mín}}/u_{\text{pico}} = 0.616376$, sólo el elemento débil dañado) y los dos tests de `test_dctl_retroceso.py` pasan sin `xfail` con las tolerancias del test de `IndirectDisplacementSolver`; el acceptance `retroceso_barra_dano_localizado` queda validado (deuda #27). Tests nuevos: un espía del corrector (`_dctl_espia.py`) comprueba el modo de los problemas de Newton, no sólo el `_mode` del callback; en la barra, la rama de ablandamiento sale entera en modo de disipación y el solver vuelve a cilíndrico al saturar el daño; en los tests existentes, la rama post-pico se resuelve ahora en modo de disipación (495 de 500 pasos en la barra 1D, 291 de 300 en el Quad4) y pasan sin cambiar ningún valor esperado; el switching queda cubierto con el modo de los problemas de Newton (acceptance `switching_cilindrico_a_disipacion_en_pico`); balance energético cuantitativo en la barra 1D: la $\sum \Delta E_d$ de los pasos de disipación coincide con la disipación analítica del material (1.3e-10 relativo, tolerancia derivada 1e-4). Límites documentados en §10: con un primer paso grande ($\Delta\lambda_1 = 0.1$) el paso cilíndrico que cruza el pico salta a la rama con los diez elementos ablandándose —no unicidad del problema incremental, fijada como caso conocido en un test—; el paso que cruza la saturación del daño salta por la rama residual; el criterio (b) sólo vale con pequeñas deformaciones y descarga secante, y supone `rtol_force` $\ll 10^{-3}$. Pendiente: confirmar en el texto de Verhoosel et al. (2009) qué tangente usa su predictor. `status` sigue en `implemented` (quedan acceptance sin cubrir).

- **2026-09-30** · Revisión adversaria de (a) y (b), sin cambio de formulación. Comprobado: $\mathbf{K}_L$ es la tangente del iterado convergido del paso anterior (en la barra coincide con la tangente analítica de carga del elemento débil a 1e-15 en los 399 predictores) y no toca el estado de los elementos (no se ensambla); tras un paso fallido, también con fallos inyectados en pasos cilíndricos y de disipación, el intento siguiente usa la misma $\mathbf{K}_L$ y el trazado sigue sobre la curva exacta; el paso que rebasa `max_lambda` no la actualiza, porque no se consolida. En la rama elástica $|\alpha_L|/\text{escala} \leq 1.6 \cdot 10^{-15}$. Corregido: la guarda de α nulo del predictor y del corrector comparaba $|\alpha|$ con `ZERO_TOL` en unidades de energía y rompía el trazado con energías de ~1e-12; pasa a ser relativa (§5, «Guarda de α nulo»). Tests nuevos: invariancia frente a las unidades y puntos de la rama residual en `test_dctl_retroceso.py`; daño 2D localizado con retroceso y balance de disipación en `test_dctl_dissipation_arclength.py` (Tests, arriba). Los valores esperados de los tests existentes no cambian.

- **2026-09-30** · Restricciones multipunto (deuda #31 del programa principal): el solver hereda de `ArcProblem` la norma del residuo reducido $\|\mathtt{T}^\top\mathtt{R}\|$; antes medía $\|\mathtt{R[libres]}\|$ y no convergía con MPC. Test `test_dctl_mpc.py`: la barra con daño localizado con un nodo atado por MPC recorre paso a paso el mismo retroceso que la barra de malla conforme, incluido el tramo en modo de disipación. Corregidos §7 y la viñeta de §10 sobre el término g: la restricción de disipación usa los vectores completos, no los reducidos (auditoría de las MPC del mismo día). Sin cambio de formulación.